



Birch Stream Platform

RAS Technology Integration and Validation Guide

Rev. 1.5

August 2025

Intel Confidential



Notice: This document contains information on products in the design phase of development. The information here is subject to change without notice. Do not finalize a design with this information.

Intel technologies may require enabled hardware, software or service activation.

You may not use or facilitate the use of this document in connection with any infringement or other legal analysis concerning Intel products described herein. You agree to grant Intel a non-exclusive, royalty-free license to any patent claim thereafter drafted which includes subject matter disclosed herein.

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

Performance results are based on testing as of dates shown in configurations and may not reflect all publicly available updates. See backup for configuration details. No product or component can be absolutely secure.

Performance varies by use, configuration, and other factors. Learn more on the [Performance Index site](#).

Your costs and results may vary.

"Conflict-free" refers to products, suppliers, supply chains, smelters, and refiners that, based on our due diligence, do not contain or source tantalum, tin, tungsten or gold (referred to as "conflict minerals" by the U.S. Securities and Exchange Commission) that directly or indirectly finance or benefit armed groups in the Democratic Republic of the Congo or adjoining countries.

All product plans and roadmaps are subject to change without notice.

Code names are used by Intel to identify products, technologies, or services that are in development and not publicly available. These are not "commercial" names and not intended to function as trademarks.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

Copies of documents which have an order number and are referenced in this document may be obtained by calling 1-800-548-4725 or visiting the [Resource and Documentation Center](#).

Intel, the Intel logo, Xeon, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

Copyright © 2025, Intel Corporation. All rights reserved.

Contents

Revision History.....	10
1.0 Introduction.....	12
1.1 Target Audience.....	12
1.2 How to Use This Document.....	12
1.3 References.....	12
1.4 Terminology.....	13
2.0 Validation Tools Overview.....	18
2.1 In-Band/Out of Band.....	18
2.1.1 CScripts.....	18
2.2 Hardware Error Injector Cards.....	18
2.2.1 PEI Card.....	18
2.3 CrashDump	19
Cscripts Crashdump.....	19
Autonomous Crash-Dump (ACD).....	19
Crashdump Summary.....	19
2.4 Kernel* Tools and Scripts.....	20
Error Injection (EINJ).....	20
FTrace.....	20
RAS-Tools Scripts.....	20
iotools.....	20
3.0 Advanced RAS Features Overview.....	21
4.0 OS RAS Review.....	24
5.0 Standard System Configuration.....	25
5.1 BIOS Settings.....	25
5.2 Linux* Configuration.....	27
5.3 Enhanced MCA Logging (ELOG).....	28
CScripts.....	29
OS.....	30
Other OS Notes.....	30
5.4 PCI Express* (PCIe*) Device Configuration.....	31
Gathering PCIe Device Information.....	31
Setting Up PCIe Device.....	33
5.5 Compute Express Link* (CXL*) Device Configuration.....	34
5.5.1 Gathering CXL Device Information.....	34
6.0 IIO RAS.....	38
6.1 PCIe/CXL.io Correctable Advisory Non-Fatal Flow.....	38
6.1.1 PCIe/CXL.io Correctable Error (UR, ANF).....	38
6.2 PCIe/CXL.io Uncorrectable Non-Fatal Flow.....	43
6.2.1 PCIe/CXL.io Non-Fatal Error (UR, NF).....	43
6.2.2 Endpoint Cyclic Redundancy Check (ECRC).....	48
6.3 PCIe/CXL.io Uncorrectable Fatal Flow.....	51
6.3.1 PCIe/CXL.io Fatal Error (UR, F).....	51
6.4 Retraining and Recovery.....	56
6.4.1 Replay Number Rollover	56

6.4.2 Correctable Error Threshold.....	58
6.5 Link CRC Error Check and Retry.....	60
6.5.1 Overview.....	60
6.6 Surprise Hot-Plug.....	63
6.6.1 Surprise Hot-Add.....	63
6.6.2 Surprise Hot-Remove.....	65
6.7 Enhanced Downstream Port Containment (eDPC).....	68
6.7.1 CTO Triggers DPC - FFM.....	68
6.7.2 CTO Triggers DPC - OS Native AER.....	72
6.7.3 eDPC Software Trigger - FFM.....	75
6.8 Leaky Bucket.....	77
6.8.1 PCIe Leaky Bucket	78
6.9 Intel® Data Streaming Accelerator (Intel® DSA).....	85
6.9.1 Intel® DSA Error Injection (Advisory Non-Fatal).....	85
6.10 Runtime S3M Error Reporting.....	90
6.10.1 Uncorrectable AER.....	90
6.10.2 SERR.....	95
7.0 Intel® Ultra Path Interconnect (Intel® UPI) RAS.....	100
7.1 Dynamic Link Width Reduction.....	100
7.1.1 Disable Low Octet by RDC (Failover to High Lanes).....	100
7.1.2 Disable High Octet by RDC (Failover to Low Lanes).....	106
7.1.3 Disable Central Octet by RDC (Failover to Low Lanes).....	112
7.2 Link Level Retry.....	118
7.2.1 MBE and Successful LLR without Phy Reinit.....	118
7.2.2 MBE and Successful LLR with PHY Reinit.....	126
8.0 CPU RAS.....	133
8.1 Uncore Machine Check Bank CSMI Thresholding.....	133
8.1.1 IMC Channel MC Bank.....	133
8.1.2 IMC Channel MC Bank - CSMI Threshold at Zero.....	137
8.2 MCA Bank Error Control (Cloaking).....	140
8.2.1 Memory, UPI, and PCIe Correctable Errors.....	140
8.2.2 UCNA Errors.....	145
8.3 First and Last Corrected Error Mode of Reporting.....	147
8.3.1 Retry Read Error Logs and IMC Machine Check Bank.....	147
8.4 MCA Recovery.....	152
8.4.1 SRAR Data Poison Consumption.....	152
8.4.2 SRAR Instruction Poison Consumption.....	156
8.4.3 MEC SRAR Data Poison Consumption (Intel® Xeon® 6900/6700-Series Processors with E-Cores).....	159
8.4.4 FEC SRAR Instruction Poison Consumption (Intel® Xeon® 6900/6700-Series Processors with E-Cores).....	164
9.0 System RAS	169
9.1 Viral: IIO - Crash Dump.....	169
Test Objective.....	169
Configuration.....	169
Preconditions and Setup	169
Recipe.....	169
9.2 Viral: Intel UPI - Crash Dump.....	179
Test Objective.....	179

Configuration.....	179
Preconditions and Setup	179
Recipe.....	180
9.3 TOR Timeout and Three-Strike Timeout	193
Test Objective.....	193
Configuration.....	193
Preconditions and Setup	193
Recipe.....	193
9.4 Inject MCERR.....	196
Test Objective.....	196
Configuration.....	196
Preconditions and Setup	196
Recipe.....	197
9.5 Inject IERR.....	200
Test Objective.....	200
Configuration.....	200
Preconditions and Setup	200
Recipe.....	200
9.6 Error Spoofing (AMEI).....	203
9.6.1 AWR Flow: Spoofed AMEI CHA TOR TO.....	204
9.6.2 Manually Spoofed AMEI DCU.....	207
9.6.3 Spoofed AMEI UPI.....	209
9.6.4 Spoofed AMEI LLC.....	212
10.0 Memory RAS.....	216
10.1 Single Device Data Correction (SDDC).....	216
10.1.1 Single Device Multi-bit Correctable ECC Error.....	217
10.1.2 Permanent Fault Detection - 125 Bit ECC Mode.....	221
10.2 Adaptive Double Device Data Correction (ADDDC).....	225
10.2.1 One Rank on the Channel.....	231
10.2.2 Four Ranks on the Channel.....	259
10.3 Address Range/Partial Memory Mirroring.....	295
10.3.1 Uncorrectable Error in a Mirrored Region.....	295
10.3.2 Persistent Uncorrectable Error in a Mirrored Region.....	299
10.4 Full Memory Mirroring.....	304
10.4.1 Correctable Error on Primary Channel.....	304
10.4.2 UCE on Primary Channel.....	308
10.4.3 Persistent UCE on Primary Channel.....	311
10.4.4 UCE on Primary, CE on Secondary.....	315
10.5 DDR Command/Address Parity Check and Retry.....	320
10.5.1 DDR Transient C/A Parity Error.....	321
10.6 Memory Demand and Patrol Scrub.....	324
10.6.1 Patrol Scrub.....	325
10.6.2 Demand Scrub.....	329
10.7 Runtime Post Package Repair (Runtime PPR).....	332
10.7.1 Runtime Soft PPR.....	332
10.8 Boot Time Post Package Repair (Boot Time PPR).....	336
10.8.1 Boot Time Post Package Repair (PPR) with Correctable Errors.....	337
10.8.2 PPR with Uncorrectable Errors.....	341
10.9 PMIC	343
10.9.1 PMIC Failure Isolation.....	344

10.10 SMBus Hang Recovery.....	349
10.10.1 SMBus Hang Recovery.....	349
11.0 CXL RAS.....	352
11.1 CXL 101.....	352
Key Terminology.....	353
11.1.1 Finding Address.....	353
11.1.2 CXL Type 3 Address Translation.....	361
11.1.3 CXL Mailbox and Event Logs.....	362
11.1.4 CXL Firmware First Versus OS First.....	369
11.1.5 CXL FTrace.....	371
11.2 CXL Link/Protocol Errors.....	372
11.2.1 CXL Corrected Error "Flow" Injection.....	373
11.2.2 CXL Uncorrected Error "Flow" Injection.....	383
11.2.3 CXL Correctable Data Path/Protocol Error (Host to Device).....	391
11.3 Event Log Notifications.....	396
11.3.1 Poison Injection (CScripts).....	397
11.3.2 Poison Injection (OS).....	401
11.3.3 Temperature Alert	404
11.3.4 Temperature Alert (OS).....	410
11.4 Poison Consumption.....	414
Test Objective.....	414
Configuration	415
Precondition and Setup	415
Recipe	415
11.5 Intel Flat Memory Mode.....	419
11.5.1 CXL Flat Memory Mode 101.....	419
11.5.2 Setup	421
11.5.3 Prelude	426
11.5.4 Cache Line Reads Recipes.....	428
11.5.5 Cache Line Write Recipes.....	446
12.0 Cross-RAS.....	447
12.1 Runtime Soft PPR then VLS Action.....	447
Test Objective.....	447
Configuration.....	447
Preconditions and Setup.....	447
Recipe.....	448
12.2 Full Memory Mirroring and Runtime Soft PPR on the Primary Channel with	
Correctable Errors.....	453
Test Objective.....	453
Configuration.....	453
Preconditions and Setup.....	454
Recipe.....	454
12.3 [TDX][LMCE=ON][OS-First] Inject a UC Non-Fatal Error into a Trust Domain Guest....	458
Test Objective.....	458
Configuration.....	458
Preconditions and Setup.....	458
Recipe.....	460
12.4 [TDX][LMCE=ON][OS-First] Inject a Correctable Error into a TD Guest.....	465
Test Objective.....	465
Configuration.....	465

Preconditions and Setup.....	465
Recipe.....	466
13.0 RAS Offload.....	472
13.1 PCIe: Receiver Error.....	472
Test Objective.....	472
Configuration.....	472
Preconditions and Setup.....	472
Recipe.....	473
13.2 PCIe: CTO Triggers DPC	474
Test Objective.....	474
Configuration.....	474
Preconditions and Setup.....	475
Recipe.....	475
13.3 Memory: ADDDC.....	478
Test Objective.....	478
Configuration.....	478
Preconditions and Setup	478
Recipe.....	479
13.4 Memory: Correctable Error Reporting.....	483
Test Objective.....	483
Configuration.....	483
Preconditions and Setup	484
Recipe.....	484
13.5 Memory: Soft Post Package Repair (SPPR).....	486
Test Objective.....	486
Configuration.....	486
Preconditions and Setup	487
Recipe.....	487
13.6 Memory: Mirror Mode - Correctable Error on Primary Channel.....	489
Test Objective.....	489
Configuration.....	489
Preconditions and Setup	490
Recipe.....	490
13.7 Intel UPI CE (Polling).....	493
Test Objective.....	493
Configuration.....	493
Preconditions and Setup	494
Recipe.....	494

Figures

1	WHEA/EMCA Logging Flow Enhancement for Birch Stream.....	29
2	CSMI Thresholding Flow.....	133
3	MCA Decode for MC Bank 1 (DCU).....	155
4	MCA Decode for MC Bank 16 (MCCHAN3).....	156
5	MCA Decode for MC Bank 16 (MCCHAN3).....	159
6	MCA Decode for MC Bank 0 (IFU).....	159
7	Cacheline of Data from 10x4 and 5x8 DIMMs.....	216
8	DDR5 - Intel BKC BIOS VLS Buddy Rank Selection Order.....	226
9	One Rank on the Channel - One Bank VLS Cacheline Data Organization.....	227
10	One Rank on the Channel - Possible 2nd Strike Locations.....	228
11	Two Ranks on the Channel - One Bank VLS Cacheline Data Organization.....	229
12	Two Ranks on the Channel - Possible 2nd Strike Locations.....	229
13	Correctable ECC Thresholding.....	231
14	Command Table.....	297
15	Command Table.....	301
16	CXL Error Logging and Signaling.....	369
17	CXL.mem Versus DDR RAS.....	370
18	CXL Correctable flow.....	374
19	Uncorrected Error Flow.....	384
20	Poison Injection Flow	398
21	Poison Injection Flow	402
22	Poison Consumption.....	414
23	Read NM Miss NM UCE.....	429
24	Read NM Hit NM UCE.....	438

Tables

1	References.....	12
2	Acronyms.....	13
3	Advanced RAS Features.....	21
4	Target Device Information.....	39
5	Target Device Information.....	44
6	Target Device Information.....	52

Revision History

Revision	Description	Date
1.5	- Updated S3M Error Reporting - Uncorrectable_AER UR recipe in IIO RAS Chapter. - Added a note about Runtime PPR BIOS knob to Boot Time PPR with Correctable Errors recipe in Memory RAS Chapter. - Updated CTO Triggers DPC - OS Native AER recipe in IIO RAS Chapter. - Updated CXL Flat Memory Mode sections in CXL2.0 RAS Chapter. - Updated DDR Transient C/A Parity Error recipe in Memory RAS Chapter. - Updated PCIe/CXL.io ANF/NF/F recipes in IIO RAS Chapter. - Updated Crashdump section in Validation Tools Overview Chapter. - Updated Viral, 3-strike TO, inject IERR/MCERR, and error spoofing recipes in System RAS Chapter.	August 2025
1.4	- Added DAM BIOS Knob to Error Injection Enabling in BIOS Standard System Configuration section. - Updated PCIe/CXL.io ANF/NF/F recipes in the PCIe Chapter. - Updated PMIC recipe in Memory RAS Chapter. - Added PTTR BIOS Knob to Patrol Scrub recipe in Memory RAS Chapter. - Added Flat Memory Mode recipes to the CXL RAS Chapter.	December 2024
1.3	- Updated Granite Rapids codename usage to official branding: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores. - Updates to PCIe Device Configuration section in the Standard System Configuration Chapter. - Updates to PCIe/CXL.io ANF/NF/F Recipes in the PCIe Chapter. - Added Uncorrectable Error on Primary Channel, Correctable Error on Secondary Channel Memory Mirroring Recipe to Memory RAS Chapter. - Updates to various recipes in the CXL RAS Chapter. - Added Flat Memory Mode Section to CXL RAS Chapter.	October 2024
1.2	- Updated Sierra Forest codename usage to official branding: Intel® Xeon® 6900/6700-Series Processors with E-Cores. - Updates to PCIe Chapter to include PCIe/CXL.io. - Updates to System RAS Chapter. New Spoofing recipes, MCERR, and IERR recipes. - New recipes added for CXL RAS chapter. - New Recipes added for Cross-RAS Functionality: TDX x RAS.	July 2024
1.1	- Static Lockstep Mode, CPU RAS, System RAS, and Memory recipes have all been updated for this revision. - Static Lockstep Chapter has been added. - System RAS has added AWR Flow: Spoofed AMEI CHA TOR TO; TOR TO and Three Strike Timeout (Crash Dump) - CXL RAS recipes have been added.	May 2024
1.0	- IIO Reliability, Availability, and Serviceability (RAS), CPU RAS, System RAS, and Memory recipes have all been updated for this revision. - IIO RAS chapter has now added S3M content. Recipes have been added for Uncorrectable AER and SERR for S3M. In addition, recipes that included PEI 4.0 card have now been updated with the PEI 5.0. Recipes that only are supported with PEI 4.0 were remained the same: ECRC, Flow Control. Also, Unsupported request and DSA Error Injection has been added. - CPU RAS has made edits to the MCA Recovery chapter. UCNA Cloaking has been added. - System RAS has added PCIe/HIOP viral.	March 2024
continued...		

Revision	Description	Date
	- Memory RAS chapter added recipes for SMBus Hang Recovery, PMIC Failure Isolation, and FRB with PMIC failure. Also Patrol Scrub and new single rank ADDDC recipes have been added. - More recipes have added OS eLog content.	
0.95	- IIO Reliability, Availability, and Serviceability (RAS), CPU RAS, Cross-RAS recipes have all been updated for this revision. - IIO RAS has changed PCIe to IIO. Small changes to PCIe Leaky Bucket, Link CRC Error Check and Retry, and Retraining and Recovery and ECRC. - Added a PCIe Uncorrectable Fatal Flow Chapter. - CPU RAS has made edits to the MCA Recovery Chapter. - Cross RAS chapter was added for RAS features that can be enabled together.	December 2023
0.90	- PCI Express* (PCIe*) Reliability, Availability, and Serviceability (RAS), CPU RAS, CXL RAS, and RAS Offload recipes have all been updated for the Birch Stream Integration and Validation Guide (IVG) revision 0.90. - PCIe RAS has added surprise hot-plug recipes. - CPU RAS has added MCA Recovery recipes. - CXL RAS has added CXL101, Link Errors, and Event Logs Chapter. - RAS Offload has added sPPR and Mirror Failover recipes.	October 2023
0.75	- PCI Express* (PCIe*) Reliability, Availability, and Serviceability (RAS), Memory RAS, and System RAS recipes have all been updated for the Birch Stream Integration and Validation Guide (IVG), Revision 0.75. - An additional recipe has been added for PCIe RAS including DSA Injection recipe. DSA goes through RCIEP SIEH flow. - System RAS has added a UPI Viral recipe. - Memory RAS has added SDDC, ADDDC, Full Memory Mirroring, PFD, and Boot-Time PPR.	August 2023
0.5	- PCI Express* (PCIe*) Reliability, Availability, and Serviceability (RAS), Memory RAS, RAS Offload, UPI RAS, and System RAS recipes have all been updated for the Birch Stream Integration and Validation Guide (IVG), Revision 0.5. - Additional recipes have been added for PCIe RAS including Leaky Bucket, Link CRC Error Check and Retry, Retraining and Recovery, and ECRC. - UPI RAS has added Dynamic Link Width Reduction and Link Level Retry. - System RAS has added a Three Strike Timeout recipe with Crash Dump. - RAS Offload has added PCIe/IIO CE reporting recipes. - Memory RAS has added ADDDC, Full Memory Mirroring, and DDR Command/Address Parity Check and Retry.	July 2023
0.3	- Initial Release of document. - PCI Express* (PCIe*) Reliability, Availability, and Serviceability (RAS), Memory RAS, and RAS Offload recipes have all been updated for Birch Stream IVG revision 0.3. PCIe* RAS has added Enhanced Downstream Port Containment (eDPC) and Retraining and Recovery. Memory RAS has added Demand Scrub and Runtime Post Package Repair (PPR). RAS Offload has added Intel® Ultra Path Interconnect (Intel® UPI) CE Reporting. All recipes now include the configuration the recipes was tested with.	May 2023

1.0 Introduction

1.1 Target Audience

The target audience for this document is system architects, system developers, UEFI-FW/ SMM developers, operating system developers, and various validation engineering disciplines who are focusing on Reliability, Availability, and Serviceability (RAS) technology. It is assumed that the audience has a baseline understanding of the RAS technology.

1.2 How to Use This Document

It should be noted that some tools and documentation referenced in this document are targeted to specific consumers, and that these items may not be available or applicable to other consumers of this material. This document describes how to configure, use, and validate specific RAS features. It is not intended to be used as or to replace reference architecture or architecture specification documents. It provides guidelines to integrate RAS features, which require UEFI-FW, OS, and application support. This document also provides guidance on how to validate RAS features. Some RAS features are implemented within the processor and do not require intervention from the UEFI-FW, the operating system, or higher-level management software. Conversely, many of the RAS features require coordination or orchestration by these entities. These interactions are described in the feature chapters as appropriate.

Documenting specific tools, command syntax, comparing features and the functionality of the tools is out of the scope for this document. See the specific tool documentation for supported features and functionality, usage, and command syntax.

1.3 References

Table 1. References

Collateral Type	Title	Document Number
Guide	<i>Birch Stream Platform RAS Technology Integration and Validation Guide</i>	742874
Guide	<i>Birch Stream Server Crash Log Overview</i>	TBD
Guide	<i>Birch Stream-AP Platform Design Guide (PDG)</i>	635354
Guide	<i>Birch Stream-SP Platform Design Guide (PDG)</i>	633502
Guide	<i>Birch Stream Platform BIOS Writer's Guide (BWG)</i>	647898
Guide	<i>Memory Mapping over Enhanced Serial Peripheral Interface (eSPI) Communication Between Host and BMC</i>	640332
Specification	<i>Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores Architecture Specification</i>	639866
Specification	<i>Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores Registers Specification</i>	637776
continued...		

Collateral Type	Title	Document Number
Specification	<i>Intel® Xeon® 6900/6700-Series Processors with E-Cores Architecture Specification</i>	736624
Specification	<i>Intel® Xeon® 6900/6700-Series Processors with E-Cores Registers Specification</i>	737845
Specification	<i>Intel® Seamless Update - RAS Offload to BMC Architecture Specification</i>	633972
Specification	<i>Memory Mapped BMC Interface</i>	DSP0282 (DMTF Publication Identifier)
Specification	<i>MMBI over MCTP Transport Protocol Binding Specification</i>	DSP0284 (DMTF Publication Identifier)
Specification	<i>Intel's Seamless Firmware Updates for Birch Stream - RAS Offload to BMC - Firmware Design Specification</i>	734308
Specification	<i>System Management Specification</i>	648590
RAS Training	<i>Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS</i>	728644
Tool	<i>CScripts for Birch Stream Platforms</i>	686308
Tool	<i>Compute Express Link (CXL) Injector Card</i>	TBD
Tool	<i>PEI (PCIe Error Injector Card) Collaterals</i>	610667
Tool	<i>SelfTest 7 - Windows* x64</i>	524152
Tool	<i>Leaky Bucket Calculator - Memory and PCIe*</i>	655405

1.4 Terminology

The following table contains important terms used in this specification.

Table 2. Acronyms

Word/Acronym	Definition
Agent	A logical device connected to a shared interconnect that can either initiate accesses or be the target of accesses
Asserted	Asserted Signal is set to a level that represents logical true. For signals that end with "#" this means to drive a low voltage. For other signals, it is a high voltage.
ACK (TLP Acknowledgment)	PCI Express* (PCIe*) packet used in the Data Link Layer to indicate the successful receipt of a Transaction Layer Packet (TLP) and all TLPs transmitted before it. Consult Section 3.4 of the <i>PCI Express Base Specification v3.0</i> for more details.
ACPI	Advanced Configuration and Power Interface
AER	Advanced Error Reporting
AP	Application Processor. All the processor threads other than System Boot Strap Processor (SBSP).
API	Application Programming Interface. The defined set of methods, functions, variables, class members, and so on, that governs how a programmer interacts with a program, script, or library.
APEI	ACPI Platform Error Interface
APIC	Advanced Programmable Interrupt Controller residing in the processor agents to generate and accept interrupt messages from other processor and I/O agents. Sometimes it is referred as Local APIC.
continued...	

Word/Acronym	Definition
BMC	Intelligent Platform Management Interface (IPMI) compliant Board Management Controller
BWG	BIOS Writer's Guide
Bank	DRAM chips are divided into multiple banks internally. Each bank acts somewhat like a separate DRAM, opening and closing pages independently, allowing different pages to be open in each. Most commands to a DRAM target a specific bank, but some commands (that is, pre-charge All) are targeted at all banks. Multiple banks allow higher performance by interleaving the banks and reducing page miss cycles.
BKC	Best Known Configuration
Cache Line	The unit of memory that is copied to and individually tracked in a cache. Specifically, 64 bytes of data or instructions aligned on a 64-byte physical address boundary.
CDC	Corrupt Data Containment. Also known as Data Poisoning.
Coherent	Transactions that ensure that the processor's view of memory through the cache is consistent with that obtained through the main memory and I/O subsystem.
Core	The portion of the processor comprising the processing units such as Execution Engine (EE) and the dedicated caches
CHA	Cache and Home Agent
CMCI	Corrected Machine Check Interrupt is an interrupt generated by the processor following a processor socket corrected error.
CSR	Configuration Space Register, registers used for configuring various ingredients.
DIMM	Dual-in-Line Memory Module. A packaging arrangement of memory devices on a socketable substrate.
CRC	Cyclic Redundancy Check. An error checking methodology where data is hashed prior to storage or transmission and the hash (CRC code) is stored/sent with the data. The consumer of the data can run the data received through the same hash algorithm and compare the result with the CRC code received in order to assess the validity of the data. More information can be found here: https://en.wikipedia.org/wiki/Cyclic_redundancy_check .
CTO	Completion Time-Out.
CXL	Compute Express Link
DDDC	Double device data correction.
DFX	Design For "X", where "X" could be Testing/Debug/RAS.
DDR4	Double Data Rate version 4
DMI	Proprietary data path between CPU and PCH
DLLP	Data Link Layer Packet. The fundamental portion of data sent via the PCIe Data Link Layer. The Data Link Layer resides between the Transaction Layer and the Physical Layer in PCIe. It transmits information about link traffic progress, link reliability, sequence numbers, and error detection. See Chapter 3 of the <i>PCI Express Base Specification v3.0</i> for more details.
ECC	Error Correcting Code
EDAC	Error Detection And Correction Driver. It is an optional component of the Linux* OS Kernel* and is used to detect and report hardware errors. It is also used to extract additional platform specific information.
eDPC (DPC)	Enhanced Downstream Port Containment PCIe capability
EDR	Error Disconnect Recover - OSPM capability related to eDPC
ELOG	Enhanced Error Logs
FM	Far Memory
continued...	

Word/Acronym	Definition
FFM	Firmware First Mode
FRB	Fault Resilient Boot. Intel® Xeon® based servers implement several features to manage faults during boot process and allow successful booting to video screen, for example, core disable for FRB, socket disable for FRB, and memory disable/map-out for FRB.
HA	Home Agent
Host	This term is used synonymously with processor.
IEH	Integrated Error Handler. The Processor Implements a unified and hierarchical error handler.
IMC	Integrated Memory Controller in the processor die
I/O	Input/Output. When used as a qualifier to a transaction type, specifies that transaction targets Intel architecture specific I/O space (for example, I/O read).
IIO (Integrated I/O [block])	The block of I/O logic integrated into the CPU design. It contains the logic for translating transactions between the ring, PCIe interfaces, and DMI interfaces. It also contains other logic like the Crystal Beach DMA engines and Intel® Virtualization Technology (Intel® VT) for Directed I/O (Intel® VT-d) translation engine.
In-band	Communication that is multiplexed on the standard lines of an interface, rather than requiring a dedicated signal
Inbound (IB) /Outbound (OB), also known as Upstream/ Downstream, Northbound/ Southbound, Up-bound/ Down-bound	Inbound, Upstream, Northbound, or Up-bound is in the direction of the processor from I/O or memory. Outbound, Downstream, Southbound, or Down-bound ins in the direction of I/O or memory from the CPU.
Intel® ITP	Intel® In-Target Probe (Intel® ITP)
Intel® ME	Intel® Management Engine (Intel® ME)
Intel® Ultra Path Interconnect (Intel® UPI)	Next generation of cache-coherent, links-based electrical interconnect specification for Intel processor, Chipset, and I/O bridge components
IPMI	Intelligent Platform Management Interface
Layer	A level of abstraction commonly used in interface specifications as a tool to group elements related to a basic function of the interface within a layer and to identify key interactions between layers.
Legacy	Functional requirements handed down from previous chipset or PC compatibility requirements from the past.
Lock	A sequence of transactions that must be completed automatically.
LCRC (PCIe Link CRC)	CRC code used to protect the transmission of Data Link Layer Packets (DLLP)s in PCIe. As each DLLP is received, the receiver must compare its computed CRC value with the CRC value transmitted in the packet. If they do not match, it must communicate this error back to the transmitter via a Negative Acknowledgment (NACK).
LLC	Last Level Cache (L3 cache)
LPC	Low Pin Count connection used to connect to the super I/O device.
LER	Live Error Recovery
LTSSM	Link Training and Status State Machine. Finite State Machine (FSM) defined by the PCIe specification that governs how the logical physical layer initially trains the link, retrains the link, and recovers from errors.
Mirroring	Provides 100% duplication of data. Offers more reliability, but doubles storage cost.
MMIO	Memory Mapped I/O. Any memory access to PCIe.
MMCFG	Memory Mapped Configuration. A memory transaction that accesses configuration space.
continued...	

Word/Acronym	Definition
MSR	Model Specific Registers for the processor
MCTP	Management Component Transport Protocol, a media independent, message transport protocol for intra-system communication of management traffic.
Microcode	CPU's firmware coordinating the CPU's actions
NACK (TLP Negative Acknowledgment)	PCIe packet used in the Data Link Layer to indicate the unsuccessful receipt of a TLP. Typically, a NACK response results in the retransmission of the identified packet and all packets sent after that packet. Consult Section 3.4 of the <i>PCI Express Base Specification v3.0</i> for more details.
Non-Coherent	Transactions that may cause the processor's view of memory through the cache to be different with that obtained through the I/O subsystem.
NM	Near Memory
NMI	Non-maskable Interrupts
NTB	Non transparent Bridge
OS	Operating System
PCH	Intel Platform Controller Hub
Packet	The indivisible unit of data transfer and routing, consisting of a header, data, and CRC
Page Hit	Access to an open page, or DRAM row. The data can be supplied from the sense amps at low latency.
Page Miss (Empty Page)	An access to a page that is not buffered in sense amps and must be fetched from DRAM array. Address Bit Permuting Address bits are distributed among channel selects, DRAM selects, bank selects to so that a linear address stream accesses these resources in a certain sequence.
Page Replace also known as Page Miss, Row Hit/Page Miss	An access to a row that has another page open. The page must be transferred back from the sense amps to the array, and the bank must be pre-charged.
Posted	A transaction that is considered complete by the initiating agent or source before it actually completes at the target of the request or destination. All agents or devices handling the request on behalf of the original Initiator must then treat the transaction as being system visible from the initiating interface all the way to the final destination. Commonly refers to memory writes.
PATA	Parallel ATA
PCH	Platform Controller Hub
PCODE	Firmware Code executing by PCU
PCLS	Partial Cache Line Sparing
PCU	Power Control Unit, a controller inside the processor that performs power and other miscellaneous functionality.
PMC	Power management controller for PCH
Powergood Reset	A reset during which power is removed. For example, using a power-button. RTC power and Standby power is not affected.
Row	A group of DRAM chips that fill out the data bus width of the system and are accessed in parallel by each DRAM command.
Row Address	The row address is presented to the DRAMs during an activate command, and indicates which page to open within the specified bank. (The bank number is presented also.)
SDDC	Single Device Disable Code; also known as x4 or x8 chip-disable Hamming code to protect single DRAM device (x4 or x8 data width) failure.
SDRAM	Synchronous Dynamic Random Access Memory
continued...	

Word/Acronym	Definition
Serial Presence Detect (SPD)	A two-signal serial bus used to access the EEPROM located on the DIMM via the SMBus protocol. This EEPROM contains the key information about the DIMM, for example, serial number, DRAM device time, and so on.
SMBus	System Management Bus. Mastered by a system management controller to read and write configuration registers. Signaling and protocol are loosely based on I2C, limited to 100 kHz.
Snooping	A means of ensuring cache coherency by monitoring all coherent accesses on a common multi drop bus to determine if an access is to information resident within a cache. The Cache Agent ensures coherency by initiating snoops on the processor buses with the address of any line that might appear in a cache on that bus.
SCI	System Control Interrupt as defined in the <i>ACPI Specification</i> .
SDC	Silent Data Corruption
SDDC	Single device data correction (DIMM CRC)
SECCED	Single bit Error Correction and Double bit Error Detection
SIPI	Startup IPI
SMBIOS	System Management BIOS
SPI	Serial Peripheral Interconnect
SOS (Skip Ordered Set)	A special set of symbols defined by the PCIe specification used to compensate for slightly different frequencies in the bit rates of the two ends of the link. An SOS must be transmitted at a minimum frequency on the link in order to ensure reliable operation.
SS	Stop and Scream
TLP	PCIe Transaction Layer Packet
UCODE	Microcode. It is a low level firmware executed within the Execution Engine of each Processor Core.
UEFI	Unified Extensible Firmware Interface
Uncore	Refers to the functionality in processor socket other than processor cores. Uncore encompasses CHA, M2M, Intel® UPI, iMC, IVR, PCU, UBOX*, and IRPRING. Note that IIO module is considered as outside of the Uncore but it is still integrated as part of the same die.
VPP	Virtual Pin Port
VCU	Validation Control Unit
VT	Virtualization Technology
Warm Reset	A reset during which power is not removed. For example, software initiated reset.
WDB	Write Data Buffer

2.0 Validation Tools Overview

2.1 In-Band/Out of Band

2.1.1 CScripts

CScripts is a probe based tool that uses the Intel Design for (DFx) technology built in the Intel Silicon to provide observability to registers and multiple error injection capabilities. For a basic guide on CScripts, see the User Guide and *CScripts for Birch Stream Platforms*, document number 686308. These documents provide more instructions on error injection.

NOTE

CScripts versions are highlighted in the configuration sections of each recipe. Some recipes may be executed with additional patches.

2.2 Hardware Error Injector Cards

2.2.1 PEI Card

The PCI Express* (PCIe*) 4.0 interposer card (PEI 4.0) is used to inject PCIe errors on Intel processor based platforms for RAS configuration enabling. The PEI is an Field Programmable Gate Arrays (FPGA) based interposer that contains a PCIe compliant switch core and an Endpoint core. It supports operations of up to PCIe 4.0 speeds and has a x4 card-edge connector at its base and a x8 female connector on its top that only supports x4. For a basic guide on the PEI 4.0 card and necessary collateral, see the *PEI (PCIe Error Injector Card) Collaterals*, document number 610667.

Basic steps to make sure the PEI card is up and running and ready to test IVG recipes are listed next. See the *PEI (PCIe Error Injector Card) Collaterals*, document number 610667, for additional troubleshooting steps.

1. Plug in the PEI 4.0 interpose card on any slot and boot SUT into the OS.
2. Install the PEI 4.0 software and check the supported error types. See the *PEI 4.0 Guide* for setup/configuration steps.
3. Verify the link state and link speed of the Intel PEI 4.0 card.
4. Verify PEI 4.0 functionality.

```
C:\Program Files\PEI\PCIe>python
Python 3.6.8 (tags/v3.6.8:3c6b436a57, Dec 24 2018, 00:16:47) [MSC v.1916 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from pei_python import *
>>> pei.status()
```

```
PEI operating as Endpoint mode
RTL Version: 4.1
Upstream : L0 @Gen4 x4
INFO: No test is running.
>>>
```

2.3 CrashDump

Crashdump is a mechanism that allows the collection of critical CPU internal state such as MSRs and CSRs after a fatal error (such as a Machine Check Error or IERR) has occurred. This document explores two methods of crashdump collection, either using Autonomous Crash-Dump (ACD) or using Cscripts.

Cscripts Crashdump

Cscripts crashdump is the in-band method to collect crash-record. It is initiated by the user after fatal condition is encountered. Run the following command to collect Crashdump. Refer to the user guide mentioned in [Cscripts](#) for more information.

```
>>> error.crashdump() # Run this command to collect the crashdump data as a JSON
file.

Conditions are met to use accelerated crashdump...
Please wait...
Collecting socket0 io0 Die
Collecting socket0 io1 Die registers
.
.
.
-----Elapsed Time:1783.8862063884735 seconds -----
crashdump() log file created at: C:\Users\lab_gdclabdcdae\PythonSv
\crashdump_xxxx.json
```

Autonomous Crash-Dump (ACD)

Autonomous Crash-Dump (ACD) is the out-of-band method to collect the Crash-record. The BMC autonomously detects a failure; use the BMC to collect crash-record; and the BMC will generate a standardized format file.

NOTE

For additional information, see the *Autonomous Crash Dump*, document number 610610.

Crashdump Summary

The Crashdump data retrieved from either of the above methods would be in a JSON format, and must be extracted to a readable .txt format. This is done with the following CScripts command.

```
>>> error.crashdump_summary() #Run this command to process the JSON file
extracted from cscripts or ACD, and decode the crashdump in a readable format as
a summary_crashdump_xxxx.txt file.

Processing crashdump file: C:\Users\lab_gdclabdcdae\PythonSv\crashdump_xxxx.json
.
.
```

```
.  
=> Output summary file: summary_crashdump_xxxx.txt
```

2.4 Kernel* Tools and Scripts

Error Injection (EINJ)

Error Injection (EINJ) is used to inject errors through software, useful for debugging and testing ACPI Platform Error Interface (APEI) and general RAS features.

See <https://www.kernel.org/doc/Documentation/acpi/apei/einj.txt> for a basic guide on EINJ.

FTrace

Linux* FTrace is used to observe the Machine Check Architecture (MCA) errors when applicable.

FTrace is by default installed on most Linux distributions. See <https://www.kernel.org/doc/Documentation/trace/ftrace.txt> for a basic guide on FTrace.

RAS-Tools Scripts

RAS-tools is an excellent set of tools to inject and test RAS ability on X86 and ARM platform through the APEI EINJ interface.

When the Linux environment is ready, download the Linux tool through the following link: <https://git.kernel.org/pub/scm/linux/kernel/git/aegl/ras-tools.git>

iotools

iotools is a simple set of command line tools which allow access to hardware device registers.

When the Linux environment is ready, download iotools through the following link: <https://code.google.com/archive/p/iotools/>

3.0 Advanced RAS Features Overview

1. IIO: RAS features that are incorporated within the PCI Express* (PCIe*) interconnect subsystem (RP, CDMA, Intel® Virtualization Technology (Intel® VT) for Directed I/O [Intel® VT-d]) and may require runtime UEFI FW, but does not require any OS/SW support during runtime.
2. Intel® Ultra Path Interconnect (Intel® UPI): RAS features that are incorporated within the Intel UPI interconnect subsystem and may require runtime UEFI FW, but does not require any OS/SW support during runtime.
3. CPU and Core: RAS features that are incorporated within processor and does not require any UEFI-FW/OS/SW support during runtime.
4. System: RAS features that require OS/SW support in addition to the UEFI FW during runtime.
5. Memory: RAS features that are incorporated within the memory subsystem (M2M, iMC, DDR5, UEFI FW) and may require runtime UEFI FW, but does not require any OS/SW support during runtime.
6. CXL: RAS features that are incorporated within the CXL system and may require runtime UEFI FW, but does not require any OS/SW support during runtime.

Table 3. Advanced RAS Features

Category	Platform RAS Feature	Reference
IIO	PCIe Link Retraining and Recovery	Retraining and Recovery on page 56
	PCIe Link CRC Error Check and Retry	Link CRC Error Check and Retry on page 60
	PCIe Corrupt Data Containment (Data Poisoning)	Not planned
	PCIe ECRC	Endpoint Cyclic Redundancy Check (ECRC) on page 48
	PCIe Enhanced Downstream Port Containment (eDPC)	Enhanced Downstream Port Containment (eDPC) on page 68
	PCIe Card Hot-Plug (Add/Remove/Swap)	Not planned
	PCIe Card Surprise Hot-Plug	Surprise Hot-Plug on page 63
	Integrated Error Handler (IEH)	No section in this document explicitly tests this feature, but correct functionality is required for tests in this document to pass.
Intel® UPI	Intel® UPI Link Level Retry	Link Level Retry on page 118
	Intel® UPI Protocol Protection via CRC (16 bit)	Not planned
CPU and Core	Intel® UPI Dynamic Link Width Reduction	Dynamic Link Width Reduction on page 100
	Error Detection and Correction (coverage at socket level)	No section in this document explicitly tests this feature, but correct functionality is required for tests in this document to pass.

continued...

Category	Platform RAS Feature	Reference
	Corrupt Data Containment - Uncore	Not planned
	Corrupt Data Containment - Core	Planned
	Advanced Error Detection and Correction (AEDC)	Not planned - embedded functionality, not testable
	Time-out timer schemes	Not planned - refer to document number 639866
	Error reporting (MCA, AER) - Core, Uncore, and IIO	No section in this document explicitly tests this feature, but correct functionality is required for tests in this document to pass.
	Error reporting through MCA 2.0 (EMCA Gen2)	No section in this document explicitly tests this feature, but correct functionality is required for tests in this document to pass.
	Processor BIST	Not injectable
	MCA Bank Error Control (Cloaking)	MCA Bank Error Control (Cloaking) on page 140
	First Corrected Error Mode of Error Reporting	First and Last Corrected Error Mode of Reporting on page 147
	PCIe Corrected Error Reporting	No section in this document explicitly tests this feature, but correct functionality is required for tests in this document to pass.
	Thresholding for Corrected Errors (all uncore MCBank for CSMI)	Uncore Machine Check Bank CSMI Thresholding on page 133
	CSR Error Log Cloaking (using DEVHIDE)	Not planned
	DCU/IFU Poison Enhancements	Planned
	DCU Scrubbing	Not planned - embedded functionality, not testable.
	Core Disable for FRB	Not planned
	Enhanced SMM (ESMM)	No section in this document explicitly tests this feature, but correct functionality is required for tests in this document to pass.
	MCA Recovery	Single Device Data Correction (SDDC) on page 216
	Viral Mode	Viral: Intel UPI - Crash Dump on page 179
Memory	Memory Single Device Data Correction (SDDC)	MCA Recovery on page 152
	DDR Command/Address Parity Check and Retry	DDR Command/Address Parity Check and Retry on page 320
	Memory Data Scrambling with Command and Address	Not planned - validated at silicon level, system-level validation not required
	Memory Demand and Patrol Scrubbing	Memory Demand and Patrol Scrub on page 324
	Memory Mirroring	Full Memory Mirroring on page 304
	Post Package Repair (PPR)	Runtime Post Package Repair (Runtime PPR) on page 332
	Memory Disable/Map-Out for FRB	Not planned
	Memory SMBus Hang Recovery	SMBus Hang Recovery on page 349
	DDR5 Address Range/Partial Memory Mirroring	Address Range/Partial Memory Mirroring on page 295
	x4 DRAM: ADDDC Multi-Region	Adaptive Double Device Data Correction (ADDDC) on page 225
	continued...	

Category	Platform RAS Feature	Reference
CXL	CXL Link CRC Error Check and Retry	Planned. Use <code>ei.mem.injectCxlError(err="ce_crc_error")</code> .
	CXL Link Retraining and Recovery	Retraining and Recovery on page 56
	CXL Corrupt Data Containment (Data Poisoning)	Poison Injection (OS) on page 401
	CXL ECRC	Endpoint Cyclic Redundancy Check (ECRC) on page 48

4.0 OS RAS Review

Windows*

The platform is targeted to be Windows* Hardware Error Architecture (WHEA) Compliant. For a detailed description of how Windows uses RAS features, consult documents about WHEA, for example, [https://msdn.microsoft.com/en-us/library/windows/hardware/ff559509\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/hardware/ff559509(v=vs.85).aspx).

Linux

There are many independent implementations of Linux, each with their own schedule to adopt new features from the "upstream" kernel.org version of the Kernel. Here primarily the major enterprise class distributors are dealt with: Red Hat* Enterprise Linux* (RHEL*) 7.5 and higher; and SUSE* Linux Enterprise Server (SLES*) 11 SP2 and higher, SLES* 12, upstream Kernel 5.1.0 and higher.

5.0 Standard System Configuration

5.1 BIOS Settings

The following BIOS settings paths are from the Intel reference platform. Locations of these settings may be different on your BIOS implementation. These are the standard BIOS settings used for the majority of RAS recipes. Review the specific recipe to see any changes made. For more information on the BIOS knobs, see the BIOS RAS specification.

NOTE

Avoiding Page Fault when Doing Error Injection on Test Images: When PcdCpuSmmRestrictedMemoryAccess is enabled, access to non-SMRAM memory (in SMM) is restricted. When PcdCpuSmmRestrictedMemoryAccess is set to true, the system will cause a Page Fault when the user is doing EINJ. Review BHS WW32 MoW for more information.

Customer Scripts (CScripts) Enabling

The following is a quick reference of the key BIOS knobs for CScripts:

- EDKII > Socket Configuration > Processor Configuration > Lock Chipset = Disable
- EDKII > Socket Configuration > Processor Configuration > MSR Lock Control = Disable
- EDKII > Socket Configuration > IIO Configuration > DFX Global Configuration >
 - EV DFX Features = Enable
 - Disable BIOS Done = [X]

ELOG Enabling

- EDKII > Platform Configuration > System Event Log > eMCA Settings >
 - EMCA Error Support= Enable
 - EMCA Logging Support= Enable
 - Corrected Error eLog = Enable
 - Memory Error eLog = Enable
 - Processor Error eLog = Enable

Error Reporting Enabling

- EDKII > Platform Configuration > System Event Log > WHEA Settings >
 - WHEA Support = Enable
 - WHEA Log Memory Error = Enable

- WHEA Log Processor Error = Enable
- WHEA Log PCI Error = Enable

Error Injection Enabling

- EDKII > Platform Configuration > System Event Log >
 - System Errors = Enable
 - System Memory Poison = Enable
- EDKII > Platform Configuration > System Event Log > Error Injection Settings >
 - MCA Bank Error Injection Support = Enable
 - WHEA Error Injection Support = Enable
 - WHEA PCIe* Error Injection Support = Enable
 - WHEA PCIe* Error Action Table = Enable
- EDKII > Platform Configuration > System Event Log > IIO Error Enabling >
 - IIO/PCH Global Error Support = Enable
 - IIO MCA Support = Disable
 - IIO eDPC Support = Disable
- EDKII > Platform Configuration > System Event Log > PCIE Error Enabling > PCIe* Corrected Error Threshold Counter = Disable
- EDKII > Platform Configuration > System Event Log > PCIE Error Enabling > PCIe AER Corrected Errors = Enable
- EDKII > Platform Configuration > System Event Log > EMCA Settings > LMCE Support = Enable
- EDKII > Platform Configuration > System Event Log > Memory Error Enabling > Memory Corrected Error = Enable
- EDKII > Socket Configuration > Uncore Configuration > Uncore General Configuration > Directory Mode Enable = Disable
- EDKII > Platform Configuration > Miscellaneous Configuration > Enable DAM Pin = Enable
 - See the 2023 WW51 MOW on how to enable/configure DAM on your platform.

Firmware First Model

- EDKII > Platform Configuration > System Event Log > EMCA Settings >
 - EMCA Logging Support = Enable
 - EMCA CMCI-SMI Morphing = EMCA gen2 CSMI
 - EMCA CMCI-SMI Threshold = 0
 - EMCA MCA-SMI Enable = EMCA gen 2 - MSMI
- EDKII > Platform Configuration > System Event Log > IIO Error Enabling > OS Native AER Support = Disable

Crash Log

- EDKII > Platform Configuration > System Event Log > CPU CrashLog Feature = Disable

5.2 Linux* Configuration

Disable Control-Flow Enforcement Technology Indirect Branch Tracking

At the time this document's publication, Disable Control-Flow Enforcement Technology Indirect Branch Tracking [CET-IBT] in the OS must be disabled. Several recipes in this document may fail if CET-IBT is enabled. CScrips command may fail silently while the system is halted. In Linux, add the argument "ibt=off" in the kernel command line.

FTrace and DMESG for Machine Check Exception (MCE) Observability

The FTrace output will be used as example in this section. In Linux, you can also grep dmesg, but the output will depend on the applied settings, version, and more aspects. Either way, users can choose between the FTrace or the dmesg. It is recommended to do this command in a separate terminal. The MCEs and the WHEA events will show up in this stage if they are delivered to the OS.

```
# ftrace instance only needs to be created once per boot
cd /sys/kernel/debug/tracing

# Unless otherwise specified, enable the following events for all recipes.
cd /sys/kernel/debug/tracing/
echo 1 > events/mce/mce_record/enable
echo 1 > events/ras/aer_event/enable
echo 1 > events/ras/memory_failure_event/enable

# There are many ways to retrieve the ftrace output. The recipes in this document uses
# these 2 methods.
# This method will keep the "pipe" open, so you'll see live whenever an event shows up.
# Once read by by trace_pipe, the buffer is cleared.
cat trace_pipe
# This method reads whatever is in the trace event buffer at the time.
# Unless cleared, the buffer will contain a history of all events seen.
cat trace
```

If your test recipe will result in the system panicking, you can enable "ftrace_dump_on_oops". This flag can be enabled either as a Linux cmd line or via /proc/sys/kernel/ftrace_dump_on_oops".

NOTE

This feature will only work if the error was sent to FTrace before system decided to panic.

Here are the common events that are used in this document:

- mce/mce_record
 - For all machine check events
 - Bank 255 is a CPER that has been converted to a pseudo MCE event.
- ras/aer_event
 - For all PCIe events
- ras/memory_failure_event
 - Whenever Linux's memory failure codepath is executed, for example, page offlining.
- ras/mc_event
 - This is EDAC FTrace events.

- ras/extlog_mem_event
 - This is for all eLog events.

Example output:

```
# from ftrace
kworker/0:1-7 [000] .... 1911.284005: mce_record: CPU: 0, MCGc/s:f000c1c/0, MC255:
940000000000009f, IPID: 0000000000000000, ADDR/MISC/SYND:
000004470219000/0000000000000000/0000000000000000, RIP: 00:<0000000000000000>,TSC: 0,
PROCESSOR: 0:606a4, TIME: 1583523670, SOCKET: 0, APIC: 0

# from dmesg
[ 1915.288464] {1}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 0
[ 1915.288464] {1}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 0
[ 1915.288466] {1}[Hardware Error]: It has been corrected by h/w and requires no further
action
[ 1915.288467] {1}[Hardware Error]: event severity: corrected
[ 1915.288469] {1}[Hardware Error]: Error 0, type: corrected
[ 1915.288470] {1}[Hardware Error]: section_type: memory error
[ 1915.288471] {1}[Hardware Error]: error_status: 0x0000000000000000
[ 1915.288473] {1}[Hardware Error]: physical_address: 0x0000004470219000
[ 1915.288474] {1}[Hardware Error]: physical_address_mask: 0xffffffffffffffff
[ 1915.288475] {1}[Hardware Error]: node: 1 Card: 2 module: 1
[ 1915.288477] {1}[Hardware Error]: error_type: 14, scrub uncorrected error
[ 1915.288466] {1}[Hardware Error]: It has been corrected by h/w and requires no further
action
[ 1915.288467] {1}[Hardware Error]: event severity: corrected
[ 1915.288469] {1}[Hardware Error]: Error 0, type: corrected
[ 1915.288470] {1}[Hardware Error]: section_type: memory error
[ 1915.288471] {1}[Hardware Error]: error_status: 0x0000000000000000
[ 1915.288473] {1}[Hardware Error]: physical_address: 0x0000004470219000
[ 1915.288474] {1}[Hardware Error]: physical_address_mask: 0xffffffffffffffff
[ 1915.288475] {1}[Hardware Error]: node: 1 Card: 2 module: 1
[ 1915.288477] {1}[Hardware Error]: error_type: 14, scrub uncorrected error
```

Disabling Lockup Panics and Watchdog Timer Messages

Halting the system via CScripts may cause the system to lockup and potentially panic. To keep the system up and have observability of FTrace and registers, disable soft lockup and hard lockup panics.

```
echo 0 > /proc/sys/kernel/hardlockup_panic
echo 0 > /proc/sys/kernel/softlockup_panic
```

The system may also be displaying messages regarding the watchdog timers during this lockup. Disable these messages with the following commands:

```
echo 0 > /proc/sys/kernel/nmi_watchdog
echo 0 > /proc/sys/kernel/watchdog
echo 0 > /proc/sys/kernel/soft_watchdog
```

5.3 Enhanced MCA Logging (ELOG)

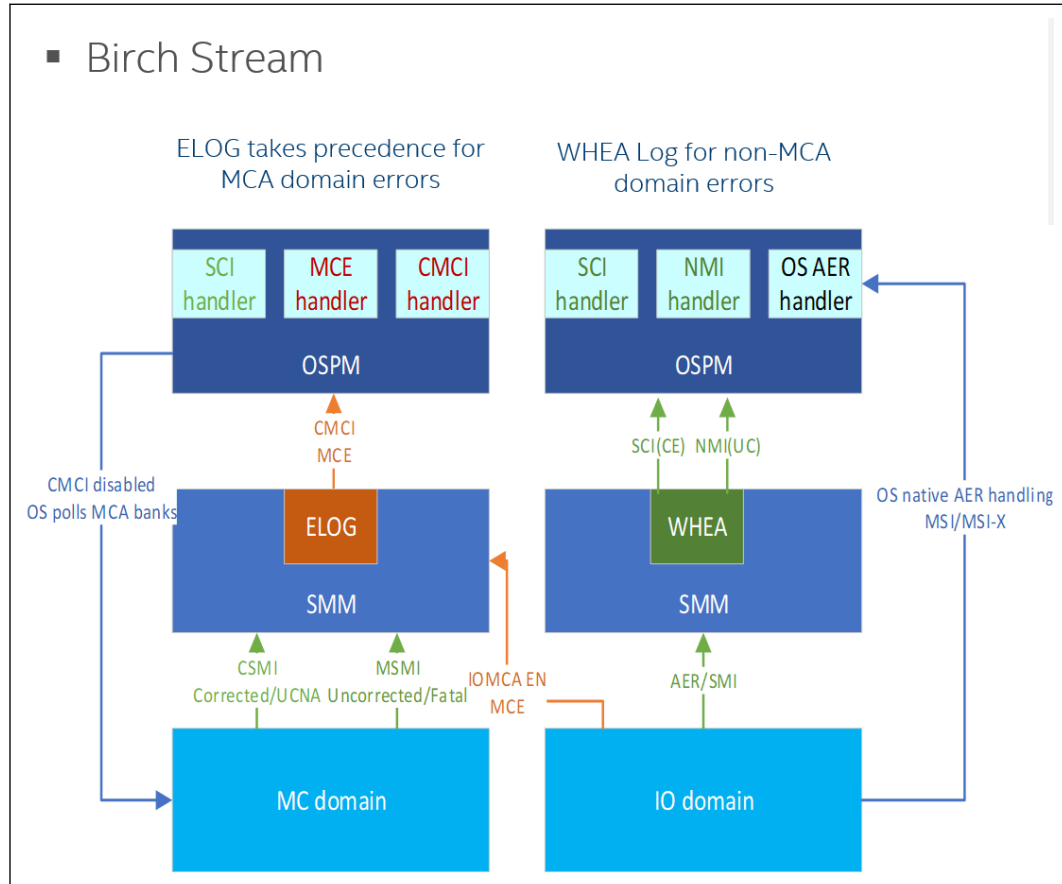
There are two types of error logging mechanisms: Enhanced MCA Logging (ELOG) and WHEA logging. The ELOG Directory is a contiguous data structure that contains a number of entries, each of which is in the form of an CPER log. ELOG complement machine check bank content synchronous with MCE/CMCI. For more information, see the *Birch Stream FW Deep Dive Event*, document number 752114.

NOTE

WHEA (old legacy naming) is also known as APEI GHES¹ or UEFI CPER².

1. ACPI Platform Error Interfaces Generic Hardware Error Status
2. Unified Extensible Firmware Interface Common Platform Error Record

Figure 1. WHEA/EMCA Logging Flow Enhancement for Birch Stream

**CScripts**

- When needed, CScripts can dump out ELOG information. To do that, you need the ELOG Directory base address. The base address is printed to the serial console during POST, look for "EmcaL1DirAddr". Then use "ras.sys.dump_EMCA2_logs" command to print out the current ELOG Directory structure.

```
>>> ras.sys.dump_EMCA2_logs(EmcaL1DirAddr)
```

OS

Linux's ELOG module is CONFIG_ACPI_EXTLOG. Make sure this is compiled in the Kernel that you are using. If lsmod does not return anything, that implies ELOG is not configured. Make sure to enable ELOG before you begin your test.

```
#modprobe acpi_extlog
#lsmod | grep acpi_extlog

acpi_extlog          20480  0
```

Data for ELOG will either be printed to dmesg or FTrace, but not both. As of the time of this document, the decision on whether ELOG data is printed to DMESG or ftrace depends on if a "rasdaemon" is active. To ensure that the ELOG data gets sent to FTrace, manually set that "rasdaemon" to active. You can also install the program called "rasdaemon", but that is outside the scope of this document.

Enable events/ras/extlog_mem_event for FTrace observation.

```
# less /sys/kernel/debug/ras/daemon_active &
# cd /sys/kernel/debug/tracing
# echo 1 > events/ras/extlog_mem_event/enable

... inject error ...

# cat trace_pipe

kworker/0:2-56477 [000] ..... 33600.243780: extlog_mem_event: {1721}
recoverable error: unknown physical addr: ffffffffffffffff (mask lsb: ff) FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnE, DIMM0
```

Other OS Notes

There could arise a situation, where it could be necessary to compile out or disable the RAS_CEC driver. RAS_CEC (Correctable Error Collector) is a Linux driver used for predictive failure analysis. If the RAS_CEC driver is enabled, it will soft-offline pages with high correctable error counts. A side effect of this is that correctable errors will be hidden outside of the Kernel. This means no DMESG or FTrace events.

```
#grep RAS_CEC /boot/config*

"CONFIG_RAS_CEC=y" indicates the driver is enabled, which implies the CEs will be
masked.

"CONFIG_RAS_CEC is not set", indicates the driver is disabled.
```

You can disable RAS_CEC with a Kernel command line update with '**ras=cec_disable**'.

```
BOOT_IMAGE=(hd0,gpt2)/vmlinuz-6.6.0 root=/dev/mapper/cs-root ro resume=/dev/
mapper/cs-swap rd.lvm.lv=cs/root rd.lvm.lv=cs/swap rhgb quiet selinux=0
console=tty0 console=ttyS0,115200 crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M
ras=cec_disable
```

This can be set during boot in the grub menu or most modern Linux distros use GRUB2*. Therefore, you can use grub2-mkconfig to specify "ras=cec_disable" by modifying /etc/default/grub's "GRUB_CMDLINE_LINUX" with "ras=cec_disable". Check your Linux Distro's documentation on complete instructions.

5.4 PCI Express* (PCIe*) Device Configuration

Gathering PCIe Device Information

Finding the PCIe Device

Run `pcie.topology()` and look for a PCIe device:

```
>>> pcie.topology()
*****
* Socket 0 (B0) PCIe Port Mappings Segment #0, Buses 0x00,0x01 *
* ,0x03,0x05,0x06,0x08,0x0a,0x0b,0x0d,0x0f,0x10,0x12,0x14,0x29 *
* ,0x3a,0x4b,0x5c,0x6d,0x7e,0x7f *
*****
=====
| PCIe port | config as | width | speed | slot/down | ASPM en | secbusno | subbusno | linkstate | Vendor | Vendor ID | Device ID | RevID | ASPM en | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0a | | x16 | | x4 | Gen3 | slot | no | 15h | 15h | Up.L0 | Intel Corporation | 8086h | 2701h | 00h | no |
| 1a | | x8 | | | | | | | | | | | | |
| 1e | | x2 | | x1 | Gen1 | slot | L1 | 2Ah | 2Ah | Up.L1.1 | Intel Corporation | 8086h | 1533h | 03h | L1 |
| 1f | | x2 | | x2 | Gen3 | slot | L1 | 2Bh | 2Bh | Up.L1.1 | Intel Corporation | 8086h | 09adh | 03h | L1 |
| 1g | | x2 | | x1 | Gen2 | slot | no | 2Ch | 2Dh | Up.L0 | ASPEED Technology, Inc. | 1a03h | 1150h | 06h | no |
| 1h | | x2 | | x1 | Gen2 | slot | L1 | 2Eh | 2Eh | Up.L1.1 | Renesas Technology Corp. | 1912h | 0014h | 03h | L1 |
| 2a | | x16 | | | | | | | | | | | | |
| 3a | | x16 | | x4 | Gen4 | slot | no | 4Ch | 4Ch | Up.L0 | Intel Corporation | 8086h | 0d4bh | 04h | no |
| 4a | | x4 | | | | | | | | | | | | |
| 4c | | x4 | | | | | | | | | | | | |
| 4e | | x4 | | | | | | | | | | | | |
| 4g | | x4 | | | | | | | | | | | | |
| 5a | | x4 | | | | | | | | | | | | |
| 5c | | x4 | | | | | | | | | | | | |
| 5e | | x4 | | | | | | | | | | | | |
| 5g | | x4 | | | | | | | | | | | | |
=====
```

Using `pcie.port_map()`, you can figure out which Flexbus/Root Port the PCIe device is under:

```
>>> pcie.port_map()
=====
| PCIe Port | GNR | GNR Nomenclature |
|-----|-----|-----|
| # | CScriptsII | CScriptsII |
| 0 | 0a | pxp0.rp0 |
| ... |
| 4 | 0e | pxp0.rp2 |
=====
```

Gathering Basic Device Topology

NOTE

PCIe EP

Notice that the PCIe device is on bus 0x15. So the PCIe device is BDF=0x1500(15:0.0) and on PCIe port "0a". (See "secbusno" column in `pcie.topology()`.)

NOTE

PCIe Satellite IEH Stack

Using the `port_map()` info, find the "internal_bus" number of that PCIe 5.0/Flexbus port.

```
>>> ei.iio.showIehMapping()

----- io0 Mapping -----
=====
| pxp/dsa# | ieh# | hiop# | bus# |
=====
| pxp0 | satieh3 (PE0) | hiop3 | 0x2 |
| pxp1 | satieh4 (PE1) | hiop4 | 0x3 |
| pxp2 | satieh2 (PE2) | hiop2 | 0x1 |
| pxp3 | satieh5 (PE3) | hiop5 | 0x4 |
| dsa0 | satieh1 (HC0) | unknown | 0x8 |
| dsa1 | satieh6 (HC1) | unknown | 0xc |
=====

----- io1 Mapping -----
=====
| pxp/dsa# | ieh# | hiop# | bus# |
=====
| pxp4 | satieh4 (PE4) | hiop4 | 0x1d |
| pxp5 | satieh3 (PE5) | hiop3 | 0x1a |
| dsa0 | satieh1 (HC0) | unknown | 0x10 |
| dsa1 | satieh6 (HC1) | unknown | 0x14 |
=====
```

Get the satellite IEH BIOS assigned bus:

```
>>> sv.socket0.io0.uncore.satieh.satieh3.iehcap.ieh_busnum
0x14
```

SATIEH3 has an assigned BDF of 0x1400 (14:0.4). See the Registers Specification for the IEH Device Function Number assignment.

The Root Port has the same BIOS assigned bus, thus its BDF is 14:2.0. See Registers Specification for the PCIe Root Port Device Function Number assignment.

NOTE

PCIe RCEC

The RCEC of the IEH stack is always at Dev 0, Func 4.

You can access these registers via `sv.socket0.io0.uncore.satieh.satieh3` or use `pci.config` and `pci.decode` to get a more familiar PCIe layout:

```
>>> sv.socket0.io0.uncore.satieh.satieh3.vid
0x8086
>>> sv.socket0.io0.uncore.satieh.satieh3.did
0xb23
>>> sv.socket0.io0.uncore.satieh.satieh3.pcicmd
0x504

>>> pci.decode(0x14,0,4)
```



```

+-----+
| Seg 0x00, Bus 0x14, Device 0x00, Function 4 |
+-----+
+-----+
| Standard PCI Registers |
+-----+
0x00: Vendor ID           = 0x8086
0x02: Device ID           = 0x0b23
0x04: Command             = 0x0504
      [10] Interrupt Disable = 1
      [9] Fast Back-to-back Enable = 0
      [8] SERR# Enable       = 1
      [6] Parity Error Response = 0
      [5] VGA Palette Snoop   = 0
      [4] Mem Write & Inval Enable = 0
      [3] Special Cycle Enable = 0
      [2] Bus Master Enable   = 1
      [1] Memory Access Enable = 0
      [0] IO Access Enable    = 0

```

Summary

Device	Bus/ Device/ Function (BDF)	Path	Other
PCIe EP	0x1500(15:0.0)	N/A	-
IEH/ PCIe RCEC	0x1404(14:0.4)	sv.socket0.io0.uncore.satieh.satieh3	-
PCIe Root Port	0x1410(14:2.0)	sv.socket0.io0.uncore.pi5.pxp0.rp0	PE0

In Linux, you can confirm that the devices in the table are visible by using `lspci -tv`.

Setting Up PCIe Device

Using the CScripts node nomenclature of the device from `pcie.port_map()` or `pci.config()/pci.decode()`.

Printing Downstream Port Registers

Using `.show()`, print the bit fields of the registers.

```

>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.devsts.show()
0x00000000 : tp (05:05) -- When set, this bit indicates that the...
0x00000000 : apd (04:04) -- Auxiliary Power is not supported.
0x00000000 : urd (03:03) -- This bit indicates that this function...
0x00000000 : fed (02:02) -- This bit indicates that this function has...
0x00000000 : nfd (01:01) -- This bit indicates that this function...
0x00000000 : ced (00:00) -- This bit indicates that this function has...

```

Printing and Preparing RCEC Registers

The BDF of the RCEC with `pci.decode()` can be used to print the values of its registers and their bit fields.

For Device Control, the least three bits can be set to enable error reporting for correctable, non-fatal, and fatal errors.

```

>>>pci.config(0x14,0,4,0x48,2,0x7)
>>>pci.decode(0x14,0,4)

0x48: Device Control      = 0x0007
      [14:12] Max Read Request Size      = 0x0 (128B)

```

```
[11] Enable No Snoop = 0x0
[10] Auxiliary (AUX) Power PM Enable = 0x0
[9] Phantom Functions Enable = 0x0
[8] Extended Tag Field Enable = 0x0
[7:5] Max Payload Size = 0x0 (128B)
[4] Enable Relaxed Ordering = 0x0
[3] Unsupported Request Reporting Enable = 0x0
[2] Fatal Error Reporting Enable = 0x1
[1] Non-Fatal Error Reporting Enable = 0x1
[0] Correctable Error Reporting Enable = 0x1
```

Clearing IEH and MC Errors

Use `error.clearAllErrors()` to clear IEH and MC registers.

```
>>> error.clearAllErrors()
```

5.5 Compute Express Link* (CXL*) Device Configuration

5.5.1 Gathering CXL Device Information

Finding the CXL Device

Run `pcie.topology()` or `cxl.topology()` and look for a CXL device.

```
>>> pcie.topology()
...
*
* Socket 0 (B0) PCIe Port Mappings Segment #0, Buses 0x00,0x01
* ,0x03,0x05,0x06,0x08,0x0a,0x0b,0x0d,0x0f,0x10,0x12,0x14,0x3d
* ,0x63,0x89,0xaf,0xd5,0xfe,0xff
*
*
=====
| PCIe port | config as | width | speed | slot/down | ASPM en | secbusno | subbusno | linkstate | Vendor | Vendor ID | Device ID | RevID | ASPM
en |
-----
| 0a | | x8 | | | | | | | | | | | |
|
|
| 0e | | x2 | x2 | Gen4 | slot | L1 | 15h | 15h | UpL1.1 | Samsung Electronics Co Ltd | 144dh | a80ah | 00h |
| L1 |
|
| 0f | | x2 | x1 | Gen1 | slot | L1 | 16h | 16h | UpL1.1 | Intel Corporation | 8086h | 1533h | 03h |
| L1 |
|
| 0g | | x2 | x1 | Gen2 | slot | no | 17h | 18h | Up.L0 | ASPEED Technology, Inc. | 1a03h | 1150h | 06h |
| no |
|
| 0h | | x2 | x1 | Gen2 | slot | L1 | 19h | 19h | UpL1.1 | Renesas Technology Corp. | 1912h | 0014h | 03h |
| L1 |
|
| 1a | | x16 | x16 | Gen5 (CXL) | slot | no | 3Eh | 3Eh | Up.L0 | 0x1dfa | 1dfah | 01e2h | 01h |
| no |
|
| 2a | | x16 | x16 | Gen5 (CXL) | slot | no | 64h | 64h | Up.L0 | 0x1dfa | 1dfah | 01e2h | 01h |
| no |
|
| 3a | | x16 | x16 | Gen5 (CXL) | slot | no | 8Ah | 8Ah | Up.L0 | 0x1dfa | 1dfah | 01e2h | 01h |
| no |
|
| 4a | | x4 | | | | | | | | | | | |
|
|
| 4c | | x4 | | | | | | | | | | | |
|
|
| 4e | | x4 | | | | | | | | | | | |
|
|
| 4g | | x4 | | | | | | | | | | | |
|
|
| 5a | | x4 | | | | | | | | | | | |
|
|
| 5c | | x4 | | | | | | | | | | | |
|
```



```
----- io1 Mapping -----
=====
| pxp/dsa# |      ieh#      | hiop# | bus# |
=====
|   pxp4   | satieh4 (PE4) | hiop4 | 0x1d |
|   pxp5   | satieh3 (PE5) | hiop3 | 0x1a |
|   dsa0   | satieh1 (HC0) | unknown | 0x10 |
|   dsa1   | satieh6 (HC1) | unknown | 0x14 |
=====
```

Get the IEH BIOS assigned BDF:

```
>>> sv.socket0.io0.uncore.satieh.satieh4.iehcap.ieh_busnum
0x3d
```

SATIEH3 has an assigned BDF of 0x3D00 (3D:0.4). See the Registers Specification for the IEH Device Function Number assignment

NOTE

CXL Root Port (RP).

The root port has the same BIOS assigned bus, thus its BDF is 3d:2.0. See the Registers Specification for the PCIe Root Port Device Function Number assignment.

This is an example of how to access the RP via CScripts.

```
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.devsts.show()
0x00000000 : tp (05:05) --
0x00000000 : apd (04:04) --
0x00000000 : urd (03:03) --
0x00000000 : fed (02:02) --
0x00000000 : nfed (01:01) --
0x00000000 : ced (00:00) --
```

To find the CXL RP BDF:

```
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.pcicmd.get_address_map_info
{'SharedAddrmap_6': {'address_offset': 4,
  'local_offset': 4,
  'absolute_offset': 4,
  'offset': 4,
  'iosfsb_address': 4,
  'pci_offset': 4,
  'cfg_inst': {'type': 'interface',
    'space': 'Cfg',
    'is_sideband': False,
    'logically_exposed': True,
    'pci_bus': 2, # Internal bus number. ie bus number used in EDS-R
    'pci_device': 2, # RP Device Number
    'pci_function': 0, # RP Function Number
    ...
  }
}

>>> pci.decode(0x3d,2,0)
...
```

NOTE

CXL RCEC

The RCEC of the IEH stack is always at Dev 0, Func 4.

You can access these registers via `sv.socket0.ieh3` or use `pci.config` and `pci.decode` to get a more familiar PCIe layout.

```
>>> sv.socket0.io0.uncore.satieh.satieh3.pcicmd
0x504

>>> pci.decode(0x3d, 0, 4)
...
```

Summary

Device	BDF	Path
CXL EP	0x3E00(3E:0.0)	Use <code>cxl.capabilities()</code>
CXL RP	0x3D20(3D:2.0)	<code>sv.socket0.io0.uncore.pi5.pxp1.rp0</code>
IEH / RCEC	0x3D04(3D:0.4)	<code>sv.socket0.io0.uncore.satieh.satieh4</code>

6.0 IIO RAS

6.1 PCIe/CXL.io Correctable Advisory Non-Fatal Flow

Advisory non-fatal errors are correctable errors that were determined by the detecting agent to be "advisory non-fatal" rather than "non-fatal". An ERR_COR message is sent instead of ERR_NONFATAL.

6.1.1 PCIe/CXL.io Correctable Error (UR, ANF)

Test Objective

This test injects an Advisory Non-Fatal Unsupported Request Error to a PCIe/CXL2.0 Device. This triggers the PCIe/CXL Device to signal a correctable PCIe/CXL.io error.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B step
BIOS	WW35'24 BKC
OS Version	CentOS* 9
Kernel	6.8.9
HW Configuration	1 Socket
CScripts Version	BHS.2422.6001
Mode of Operation	Firmware First Mode

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration. This recipe is valid in both Operating System First Mode and Firmware First Mode.

Ensure CXL and IEH hierarchy are cleared and endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

In this recipe, a UR to target device 15:0.0 is being injected.

Table 4. Target Device Information

Device	BDF	CScripts Node
End Point	15:0.0	-
Root Port	14:2.0	pxp0.rp0
IEH	14:0.4	sieh3 (PE0)

NOTE

See the *Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores Registers Specification*, document number 637776, for more details on selecting the correct IEH Device Configuration.

1. Halt the platform. Ensure that the relevant masks are cleared and the severity is as expected for the root port and endpoint device. Check that the platform is in a clean and healthy state. RPPIO is useful for observability.

```
>>> halt()

# Run "error.clearAllErrors()" to clear any errors before injection.
>>> error.clearAllErrors()

# Clear and check the RPPIO status
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.rppiosts=-1
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.rppiosts.show()
0x00000000 : memcto (18:18) -- Memory Request Completion Timeout
0x00000000 : memcac (17:17) -- Memory Request received CA Completion
0x00000000 : memurc (16:16) -- Memory Request received UR Completion
0x00000000 : iocac (09:09) -- I/O Request received CA Completion
0x00000000 : iourc (08:08) -- I/O Request received UR Completion
0x00000000 : cfgcto (02:02) -- Configuration Request Completion Timeout
0x00000000 : cfgcac (01:01) -- Configuration Request received CA Completion
0x00000000 : cfgurc (00:00) -- Configuration Request received UR Completion

>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.rppiomsk=0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.rppiomsk.show()
0x00000000 : memcto (18:18) -- Memory Request Completion Timeout Mask
0x00000000 : memcac (17:17) -- Memory Request received CA Completion Mask
0x00000000 : memurc (16:16) -- Memory Request received UR Completion Mask
0x00000000 : ioccto (10:10) -- I/O Request Completion Timeout Mask
0x00000000 : iocac (09:09) -- I/O Request received CA Completion Mask
0x00000000 : iourc (08:08) -- I/O Request received UR Completion Mask
0x00000000 : cfgcto (02:02) -- Configuration Request Completion Timeout Mask
0x00000000 : cfgcac (01:01) -- Configuration Request received CA Completion Mask
0x00000000 : cfgurc (00:00) -- Configuration Request received UR Completion Mask
```

2. Inject an advisory non-fatal UR to a PCIe/CXL device by causing a failed memory read. This memory read is non-posted, with a severity mode of non-fatal.

```
>>> ei.iio.injectURLError(0x15,0,0,0,"mem")
Mode: mem, selecting BAR0 with Base Address = 0x92000000 : 64-bits
Disabling memory space...
Calculated BAR: 0x92000000

Setting Severity as non-fatal

Checking for successful error injection to PCI BDF...
```

```
UR_reg value: 0x1
```

```
Error successfully injected to PCIe -> Bus:21,Device:0,Function:0!
Re-enabling memory space...
```

3. Evaluate the PCIe status registers. RPPIOSTS should show a memory request received UR completion. The endpoint should log the UR error.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.rppiosts.show()
0x00000000 : memcto (18:18) -- Memory Request Completion Timeout
0x00000000 : memcac (17:17) -- Memory Request received CA Completion
0x00000001 : memurc (16:16) -- Memory Request received UR Completion
0x00000000 : iocto (10:10) -- I/O Request Completion Timeout
0x00000000 : iocac (09:09) -- I/O Request received CA Completion
0x00000000 : iourc (08:08) -- I/O Request received UR Completion
0x00000000 : cfgcto (02:02) -- Configuration Request Completion Timeout
0x00000000 : cfgcac (01:01) -- Configuration Request received CA Completion
0x00000000 : cfgguc (00:00) -- Configuration Request received UR Completion

>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.rooterrsts.show()
0x00000000 : aemn (31:27) -- This register indicates which MSI/MSI-X vector is used for the
interrupt message genera...
0x00000000 : femr (06:06) -- Fatal Error Message Received
0x00000000 : nfemr (05:05) -- Non-Fatal Error message Received
0x00000000 : fuf (04:04) -- First Uncorrectable Fatal
0x00000000 : mefr (03:03) -- Multiple Error Fatal/Non-Fatal Received
0x00000000 : efr (02:02) -- Error Fatal/Non-Fatal Received
0x00000000 : mcer (01:01) -- Multiple Error Correctable Error Received
0x00000001 : cer (00:00) -- Correctable Error Received

>>>sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.errsrcid.show()
0x00000000 : efsid (31:16) -- Loaded with the Requester ID indicated in the received ERR_FATAL
or ERR_NONFATAL Message when the ERR_FATAL/NONFATAL Received bit is not already set.
0x00001500 : ecsid (15:00) -- Loaded with the Requester ID indicated in the received ERR_COR
Message when the ERR_COR Received bit is not already set.

>>> pci.decode(15,0,0)
0x104: Uncorrectable Error Status Register = 0x100000
[25] TLP Prefix Blocked Error Status (Optional) = 0x0
[24] AtomicOp Egress Blocked Status (Optional) = 0x0
[23] MC Blocked TLP Status (Optional) = 0x0
[22] Uncorrectable Internal Error Status (Optional) = 0x0
[21] ACS Violation Status (Optional) = 0x0
[20] Unsupported Request Error Status = 0x1
[19] ECRC Error Status (Optional) = 0x0
[18] Malformed TLP Status = 0x0
[17] Receiver Overflow Status (Optional) = 0x0
[16] Unexpected Completion Status = 0x0
[15] Completer Abort Status (Optional) = 0x0
[14] Completion Timeout Status = 0x0
[13] Flow Control Protocol Error Status (Optional) = 0x0
[12] Poisoned TLP Status = 0x0
[5] Surprise Down Error Status (Optional) = 0x0
[4] Data Link Protocol Error Status = 0x0
[0] Undefined = 0x0
0x110: Correctable Error Status Register = 0x2000
[15] Header Log Overflow Status = 0x0
[14] Corrected Internal Error Status = 0x0
[13] Advisory Non-Fatal Error Status = 0x1
[12] Replay Timer Timeout Status = 0x0
[8] REPLAY_NUM Rollover Status = 0x0
[7] Bad DLLP Status = 0x0
[6] Bad TLP Status = 0x0
[0] Receiver Error Status = 0x0
0x118: Advanced Error Capabilities and Control Register = 0x00b4
[11] TLP Prefix Log Present = 0x0
[10] Multiple Header Recording Enable = 0x0
[9] Multiple Header Recording Capable = 0x0
[8] ECRC Check Enable = 0x0
```


[7]	ECRC Check Capable	= 0x1
[6]	ECRC Generation Enable	= 0x0
[5]	ECRC Generation Capable	= 0x1
[4:0]	First Error Pointer	= 0x14
0x11c:	Header Log Register (1st DW)	= 0x0001
0x120:	Header Log Register (2nd DW)	= 0xfd01000f
0x124:	Header Log Register (3rd DW)	= 0x92000000
0x128:	Header Log Register (4th DW)	= 0x0000

4. Check IEH Errors. Ensure that the Global IEH and Satellite IEH correspond to the correct PCIe/CXL Device. This step is not applicable if this recipe is run in OSFM.
 - Check satieh3 detected a correctable error status, which points to dev2_co_status. The Registers Specification defines this bit to x16 Root Port 0 or BDF X:2.0 or ppxXXX. This should match the RP information gathered at the beginning of this recipe.
 - Check globalieh detected a corrected error status coming from DEV5_CO_STS. The Registers Specification defines this bit to PE0 or SATIEH3. This step is not applicable if this recipe is run in OSFM.

```
>>> error.check_ieh_errors()

...

=====
=====
IEH_3_IO0: REGISTERS AND FIELDS DESCRIPTIONS
=====
=====
Register: socket0.io0.uncore.satieh.satieh3.gerrcorsts
dev2_co_sts = 0x1
Dev2 Error Status (DEV2_CO_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.satieh.satieh3.gerrcortime
0x0000000293464200 : g_coferr_time (63:00) -- Global COFERR Time Stamp (G_COFERR_TIME)

=> socket0.io0.uncore.satieh.satieh3.gtime
0x0000000293465717 : gtime_value (63:00) -- Global Error Timer value (GTIME_Value)

-----
-----
Register: socket0.io0.uncore.satieh.satieh3.gsysevtsts
co_sysev_sts = 0x1
Correctable Error Status - A correctable error was detected when set (CO_SYSEV_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.satieh.satieh3.gsysevtctl
0x00000001 : fa_sysev_m (02:02) -- Fatal Error Mask(FA_SYSEV_M) Fatal Error System
Event Mask: 1 - Unmask sy...
0x00000001 : nf_sysev_m (01:01) -- Nonfatal Error Mask(NF_SYSEV_M) NonFatal Error
System Event Mask: 1 - Unm...
0x00000001 : co_sysev_m (00:00) -- Correctable Error Mask(CO_SYSEV_M) Correctable
Error System Event Mask: 1...

-----
-----
Register: socket0.io0.uncore.satieh.satieh3.gerrsrcid
gecsid = 0x421
ERR_COR Source Identification (ECSID) Requester ID of the source when a correctable error
is received.

-----
-----
...
```

```
=====
=====
      IEH_GLOBAL: REGISTERS AND FIELDS DESCRIPTIONS
=====
=====
Register: socket0.io0.uncore.globalieh.iehregs.gerrcorsts
  dev5_co_sts = 0x1
    Dev5_Error Status (DEV5_CO_STS)

ADDITIONAL INFO REGISTERS:
  => socket0.io0.uncore.globalieh.iehregs.gerrcortime
0x0000049997ee2838 : g_coferr_time (63:00) -- Global COFERR Time Stamp (G_COFERR_TIME)

  => socket0.io0.uncore.globalieh.iehregs.gtime
0x000004a81146ab2f : gtime_value (63:00) -- Global Error Timer value (GTIME_Value)

-----
-----
Register: socket0.io0.uncore.globalieh.iehregs.gsysevtsts
  co_sysev_sts = 0x1
    Correctable Error Status - A correctable error was detected when set (CO_SYSEV_STS)

ADDITIONAL INFO REGISTERS:
  => socket0.io0.uncore.globalieh.iehregs.gsysevtctl
    0x00000001 : fa_sysev_m (02:02) -- Fatal Error Mask(FA_SYSEV_M) Fatal Error System
Event Mask: 1 - Unmask sy...
    0x00000001 : nf_sysev_m (01:01) -- Nonfatal Error Mask(NF_SYSEV_M) NonFatal Error
System Event Mask: 1 - Unm...
    0x00000001 : co_sysev_m (00:00) -- Correctable Error Mask(CO_SYSEV_M) Correctable
Error System Event Mask: 1...

  => socket0.io0.uncore.globalieh.iehregs.gsysevtmap
    0x00000001 : co_sysev_map (01:00) -- 11: Reserved 10: Generate NMI 01 Generate SMI 00:
No system event gener...

-----
-----
Register: socket0.io0.uncore.globalieh.iehregs.gerrsrcid
  geccsid = 0x88
    ERR_COR Source Identification (ECSID) Requester ID of the source when a correctable error
    is Received.

-----
-----
...
```

5. Go/release the system. Check FTrace and DMsg. Behavior may vary based on OEM configuration. See the PCIe/CXL Device Configuration chapter for more information.

```
-----
FTrace
-----
kworker/0:0-7 [000] .N... 5263.374616: aer_event: 0000:15:00.0 PCIe Bus Error:
severity=Corrected, Advisory Non-Fatal Error, TLP Header=Not available
```

```
-----
DMsg
-----

[ 5259.493053] {1}[Hardware Error]: Hardware error from APEI Generic Hardware
Error Source: 0
[ 5259.502778] {1}[Hardware Error]: It has been corrected by h/w and requires
no further action
[ 5259.512319] {1}[Hardware Error]: event severity: corrected
[ 5259.518545] {1}[Hardware Error]: Error 0, type: corrected
```

```
[ 5259.524769] {1}[Hardware Error]: section_type: PCIe error
[ 5259.531090] {1}[Hardware Error]: port_type: 0, PCIe end point
[ 5259.537954] {1}[Hardware Error]: version: 3.0
[ 5259.543234] {1}[Hardware Error]: command: 0x0546, status: 0x0010
[ 5259.550373] {1}[Hardware Error]: device_id: 0000:15:00.0
[ 5259.556728] {1}[Hardware Error]: slot: 0
[ 5259.561512] {1}[Hardware Error]: secondary_bus: 0x00
[ 5259.567467] {1}[Hardware Error]: vendor_id: 0x1dfa, device_id: 0x01e2
[ 5259.575082] {1}[Hardware Error]: class_code: 050210
[ 5259.584519] cxl_pci 0000:15:00.0: AER: aer_status: 0x00002000, aer_mask:
0x00000000
[ 5259.593417] cxl_pci 0000:15:00.0: [13] NonFatalErr
[ 5259.600316] cxl_pci 0000:15:00.0: AER: aer_layer=Transaction Layer,
aer_agent=Receiver ID
```

6.2 PCIe/CXL.io Uncorrectable Non-Fatal Flow

Non-fatal errors are uncorrectable errors which cause a particular transaction to be unreliable, but the link is otherwise fully functional. Isolating non-fatal from fatal errors provides requester/receiver logic in a device or system management software the opportunity to recover from the error without resetting the components on the link and disturbing other transactions in progress. Devices not associated with the transaction in error are not impacted by the error.

6.2.1 PCIe/CXL.io Non-Fatal Error (UR, NF)

Test Objective

This test injects a Non-Fatal Unsupported Request Error to a PCIe/CXL2.0 Device. This triggers the PCIe/CXL device to signal a non-fatal PCIe/CXL.io error.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B step
BIOS	WW35'24 BKC
OS Version	CentOS 9
Kernel	6.10.6
HW Configuration	1 Socket
CScripts Version	BHS.2422.6001
Mode of Operation	Firmware First Mode

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18

- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration. This recipe is valid in both Operating System First Mode and Firmware First Mode.

Ensure CXL and IEH hierarchy are cleared and endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

In this recipe, inject a UR to target device 15:0.0.

Table 5. Target Device Information

Device	BDF	CScripts Node
End Point	15:0.0	-
Root Port	14:6.0	pxp0.rp2
IEH	14:0.4	sieh3 (PE0)

NOTE

See the *Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores Registers Specification*, document number 637776, for more details on selecting the correct IEH Device Configuration.

1. Halt the platform. Ensure that the relevant masks are cleared and the severity is as expected for the root port and endpoint device. Check that the platform is in a clean and healthy state.

```
>>> halt()

# Run "error.clearAllErrors()" to clear any errors before injection.
>>> error.clearAllErrors()

# Clear and check the RPPIO status
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rppiosts=-1
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rppiosts.show()
0x00000000 : memcto (18:18) -- Memory Request Completion Timeout
0x00000000 : memcac (17:17) -- Memory Request received CA Completion
0x00000000 : memurc (16:16) -- Memory Request received UR Completion
0x00000000 : iocac (09:09) -- I/O Request received CA Completion
0x00000000 : iourc (08:08) -- I/O Request received UR Completion
0x00000000 : cfgcto (02:02) -- Configuration Request Completion Timeout
0x00000000 : cfgcac (01:01) -- Configuration Request received CA Completion
0x00000000 : cfggurc (00:00) -- Configuration Request received UR Completion

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rppiomsk=0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rppiomsk.show()
0x00000000 : memcto (18:18) -- Memory Request Completion Timeout Mask
0x00000000 : memcac (17:17) -- Memory Request received CA Completion Mask
0x00000000 : memurc (16:16) -- Memory Request received UR Completion Mask
0x00000000 : ioccto (10:10) -- I/O Request Completion Timeout Mask
0x00000000 : iocac (09:09) -- I/O Request received CA Completion Mask
0x00000000 : iourc (08:08) -- I/O Request received UR Completion Mask
0x00000000 : cfgcto (02:02) -- Configuration Request Completion Timeout Mask
0x00000000 : cfgcac (01:01) -- Configuration Request received CA Completion Mask
0x00000000 : cfggurc (00:00) -- Configuration Request received UR Completion Mask
```

- Inject a non-fatal UR to a PCIe/CXL device by causing a failed memory write. This memory write is posted, with a severity mode of non-fatal.

```
>>> ei.iio.injectURError(0x15,0,0,0,access="write")
Mode: mem, selecting BAR0 with Base Address = 0x93300000 : 64-bits
Disabling memory space...
Calculated BAR: 0x93300000

Setting Severity as non-fatal

Checking for successful error injection to PCI BDF...

UR_reg value: 0x1

Error successfully injected to PCIe -> Bus:21,Device:0,Function:0!
Re-enabling memory space...
```

- Re-evaluate the PCIe status registers. RPPIOSTS should be clear since the write is posted.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rppiosts.show()
0x00000000 : memcto (18:18) -- Memory Request Completion Timeout
0x00000000 : memcac (17:17) -- Memory Request received CA Completion
0x00000000 : memurc (16:16) -- Memory Request received UR Completion
0x00000000 : iocto (10:10) -- I/O Request Completion Timeout
0x00000000 : iocac (09:09) -- I/O Request received CA Completion
0x00000000 : iourc (08:08) -- I/O Request received UR Completion
0x00000000 : cfgcto (02:02) -- Configuration Request Completion Timeout
0x00000000 : cfgcac (01:01) -- Configuration Request received CA Completion
0x00000000 : cfgurc (00:00) -- Configuration Request received UR Completion

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts.show()
0x00000000 : aemn (31:27) -- This register indicates which MSI/MSI-X vector is used for the
interrupt message genera...
0x00000000 : femr (06:06) -- Fatal Error Message Received
0x00000001 : nfemr (05:05) -- Non-Fatal Error message Received
0x00000000 : fuf (04:04) -- First Uncorrectable Fatal
0x00000000 : mefr (03:03) -- Multiple Error Fatal/Non-Fatal Received
0x00000001 : efr (02:02) -- Error Fatal/Non-Fatal Received
0x00000000 : mcer (01:01) -- Multiple Error Correctable Error Received
0x00000000 : cer (00:00) -- Correctable Error Received

>>>sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.errsrcid.show()
0x00001500 : efsid (31:16) -- Loaded with the Requester ID indicated in the received ERR_FATAL
or ERR_NONFATAL Message when the ERR_FATAL/NONFATAL Received bit is not already set.
0x00000000 : ecsid (15:00) -- Loaded with the Requester ID indicated in the received ERR_COR
Message when the ERR_COR Received bit is not already set.

>>> pci.decode(15,0,0)
0x104: Uncorrectable Error Status Register = 0x100000
    [25] TLP Prefix Blocked Error Status (Optional) = 0x0
    [24] AtomicOp Egress Blocked Status (Optional) = 0x0
    [23] MC Blocked TLP Status (Optional) = 0x0
    [22] Uncorrectable Internal Error Status (Optional) = 0x0
    [21] ACS Violation Status (Optional) = 0x0
    [20] Unsupported Request Error Status = 0x1
    [19] ECRC Error Status (Optional) = 0x0
    [18] Malformed TLP Status = 0x0
    [17] Receiver Overflow Status (Optional) = 0x0
    [16] Unexpected Completion Status = 0x0
    [15] Completer Abort Status (Optional) = 0x0
    [14] Completion Timeout Status = 0x0
    [13] Flow Control Protocol Error Status (Optional) = 0x0
    [12] Poisoned TLP Status = 0x0
    [5] Surprise Down Error Status (Optional) = 0x0
    [4] Data Link Protocol Error Status = 0x0
    [0] Undefined = 0x0
0x110: Correctable Error Status Register = 0x0000
```

```

[15] Header Log Overflow Status           = 0x0
[14] Corrected Internal Error Status      = 0x0
[13] Advisory Non-Fatal Error Status     = 0x0
[12] Replay Timer Timeout Status         = 0x0
[8]  REPLAY_NUM Rollover Status          = 0x0
[7]  Bad DLLP Status                     = 0x0
[6]  Bad TLP Status                      = 0x0
[0]  Receiver Error Status               = 0x0
0x118: Advanced Error Capabilities and Control Register = 0x02b4
[11] TLP Prefix Log Present              = 0x0
[10] Multiple Header Recording Enable    = 0x0
[9]  Multiple Header Recording Capable   = 0x1
[8]  ECRC Check Enable                   = 0x0
[7]  ECRC Check Capable                  = 0x1
[6]  ECRC Generation Enable              = 0x0
[5]  ECRC Generation Capable             = 0x1
[4:0] First Error Pointer                = 0x14
0x11c: Header Log Register (1st DW)      = 0x40000001
0x120: Header Log Register (2nd DW)      = 0x14000003
0x124: Header Log Register (3rd DW)      = 0x93300000
0x128: Header Log Register (4th DW)      = 0xfeca0000
    
```

4. Check IEH Errors. Ensure that the Global IEH and Satellite IEH correspond to the correct PCIe/CXL device. This step is not applicable if this recipe is run in OSFM.
 - Check satieh3 detected a non-fatal error status, which points to DEV4_NF_STATUS. The Registers Specification defines this bit to x16 Root Port 0 or BDF X:6.0 or pxxXXX. This should match the RP information gathered at the beginning of this recipe.
 - Check globalieh detected a non-fatal error status coming from DEV5_NF_STS. The Registers Specification defines this bit to PE0 or SATIEH3. This step is not applicable if this recipe is run in OSFM.

```

>>> error.check_ieh_errors()

...

=====
=====
IEH_3_IO0: REGISTERS AND FIELDS DESCRIPTIONS
=====
=====
Register: socket0.io0.uncore.satieh.satieh3.gerrnonsts
dev4_nf_sts = 0x1
Dev4 Error Status (DEV4_NF_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.satieh.satieh3.gerrnontime
0x00000002bce20915e : g_nfferr_time (63:00) -- Global NFFERR Time Stamp (G_NFFERR_TIME):

=> socket0.io0.uncore.satieh.satieh3.gtime
0x00000002bce20a6c1 : gtime_value (63:00) -- Global Error Timer value (GTIME_Value)

-----
Register: socket0.io0.uncore.satieh.satieh3.gsysevtsts
nf_sysev_sts = 0x1
Nonfatal Error Status - A nonfatal error was detected when set (NF_SYSEV_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.satieh.satieh3.gsysevtctl
0x000000001 : fa_sysev_m (02:02) -- Fatal Error Mask(FA_SYSEV_M) Fatal Error System
Event Mask: 1 - Unmask sy...
0x000000001 : nf_sysev_m (01:01) -- Nonfatal Error Mask(NF_SYSEV_M) NonFatal Error
System Event Mask: 1 - Unm...
0x000000001 : co_sysev_m (00:00) -- Correctable Error Mask(CO_SYSEV_M) Correctable
Error System Event Mask: 1...
    
```

```

-----
-----
Register: socket0.io0.uncore.satieh.satieh3.gerrsrcid
gefsid = 0x321
    ERR_COR Source Identification (ECSID) Requester ID of the source when a correctable error
    is received.

gecsid = 0x321
    ERR_COR Source Identification (ECSID) Requester ID of the source when a correctable error
    is received.

-----
-----
...

=====
=====
                IEH_GLOBAL: REGISTERS AND FIELDS DESCRIPTIONS
=====
=====
Register: socket0.io0.uncore.globalieh.iehregs.gerrnonsts
dev5_nf_sts = 0x1
    Dev5 Error Status (DEV5_NF_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.globalieh.iehregs.gerrnontime
0x0000031a67a0fccb : g_nfferr_time (63:00) -- Global NFFERR Time Stamp (G_NFFERR_TIME):

    => socket0.io0.uncore.globalieh.iehregs.gtime
0x0000033887e58950 : gtime_value (63:00) -- Global Error Timer value (GTIME_Value)

-----
-----
Register: socket0.io0.uncore.globalieh.iehregs.gsysevtsts
nf_sysev_sts = 0x1
    Nonfatal Error Status - A nonfatal error was detected when set (NF_SYSEV_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.globalieh.iehregs.gsysevtctl
    0x00000001 : fa_sysev_m (02:02) -- Fatal Error Mask(FA_SYSEV_M) Fatal Error System
Event Mask: 1 - Unmask sy...
    0x00000001 : nf_sysev_m (01:01) -- Nonfatal Error Mask(NF_SYSEV_M) NonFatal Error
System Event Mask: 1 - Unm...
    0x00000001 : co_sysev_m (00:00) -- Correctable Error Mask(CO_SYSEV_M) Correctable
Error System Event Mask: 1...

    => socket0.io0.uncore.globalieh.iehregs.gsysevmap
    0x00000001 : fa_sysev_map (05:04) -- 11: Reserved 10: Generate NMI 01 Generate SMI 00:
No system event gener...
    0x00000001 : nf_sysev_map (03:02) -- 11: Reserved 10: Generate NMI 01 Generate SMI 00:
No system event gener...
    0x00000001 : co_sysev_map (01:00) -- 11: Reserved 10: Generate NMI 01 Generate SMI 00:
No system event gener...

-----
-----
Register: socket0.io0.uncore.globalieh.iehregs.gerrsrcid
gefsid = 0x88
    ERR_COR Source Identification (ECSID) Requester ID of the source when a correctable error
    is received.

gecsid = 0x88
    ERR_COR Source Identification (ECSID) Requester ID of the source when a correctable error
    is received.

-----
-----
...

```

- Go/release the system. Check FTrace and DMsg. Results will differ slightly based on which device the error was injected into. See the PCIe/CXL Device Configuration chapter for more information.

```
-----
FTrace
-----
kworker/0:0-7 [000] [175.587564] aer_event: 0000:15:00.0 PCIe Bus Error: severity=Uncorrected,
non-fatal, Unsupported Request Error, TLP Header=Not available
```

```
-----
DMsg
-----

[ 3222.887580] {1}[Hardware Error]: Hardware error from APEI Generic Hardware
Error Source: 1
[ 3222.913511] {1}[Hardware Error]: event severity: recoverable
[ 3222.919945] {1}[Hardware Error]: Error 0, type: recoverable
[ 3222.926365] {1}[Hardware Error]: section_type: PCIe error
[ 3222.932688] {1}[Hardware Error]: port_type: 0, PCIe end point
[ 3222.940576] {1}[Hardware Error]: version: 3.0
[ 3222.945730] {1}[Hardware Error]: command: 0x0546, status: 0x4010
[ 3222.952726] {1}[Hardware Error]: device_id: 0000:15:00.0
[ 3222.958950] {1}[Hardware Error]: slot: 0
[ 3222.963616] {1}[Hardware Error]: secondary_bus: 0x00
[ 3222.969629] {1}[Hardware Error]: vendor_id: 0x144d, device_id: 0xa80a
[ 3222.977123] {1}[Hardware Error]: class_code: 010802
[ 3222.982927] nvme 0000:15:00.0: AER: aer_status: 0x00100000, aer_mask:
0x00000000
[ 3222.991304] nvme 0000:15:00.0: [20] UnsupReq (First)
[ 3222.998609] nvme 0000:15:00.0: AER: aer_layer=Transaction Layer,
aer_agent=Requester ID
[ 3223.007668] nvme 0000:15:00.0: AER: aer_uncor_severity: 0x00463030
[ 3223.014674] nvme 0000:15:00.0: AER: TLP Header: 40000001 14000003
93300000 feca0000
[ 3223.023562] pcieport 0000:14:06.0: AER: device recovery successful
```

6.2.2 Endpoint Cyclic Redundancy Check (ECRC)

Test Objective

To ensure end-to-end data integrity detection in systems that require high data reliability, a transaction layer end-to-end 32-bit CRC (Endpoint Cyclic Redundancy Check [ECRC]) can be placed in the TLP Digest field at the end of a TLP. ECRC is used primarily within the storage segment to accomplish higher data integrity while exchanging data between two PCIe* endpoints. ECRC provides additional data integrity on top of Link CRC (LCRC) in presence of PCIe switches. Verify a correct system behavior when a PCIe "Bad ECRC" error is injected. Bad ECRC is inserted into the PCIe packet information up-stream north bound to the root port. Evaluating if AER registers trigger with corresponding error and OS logs correct ECRC error information.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	Generic PCIe* Card
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [PEI Card](#) on page 18 (PEI 4.0 only)
- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

Review the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 to configure platform for PCIe* recipe.

- EDKII Menu > Platform Configuration > System Event Log > IIO Error Enabling >
 - IIO eDPC Support = Disabled
- EDKII Menu > Platform Configuration > System Event Log > PCIe Error Enabling >
 - PCIe ECRC Error = Enabled

Ensure the PCIe and IEH hierarchy are cleared and the endpoints are properly configured. See [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Ensure relevant masks are cleared, severity is as expected, and platform is in a clean and healthy state. Halt the platform.

```
>>> halt()

# Run "error.clearAllErrors()" to clear any errors before injection.
>>> error.clearAllErrors()

>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.erruncsev.show()
0x00000000 : pcrchfs (30:30)
0x00000000 : midetlps (29:29)
0x00000000 : idecfs (28:28)
0x00000000 : dmwrebs (27:27)
0x00000000 : ptlpebs (26:26)
0x00000000 : tpbes (25:25)
0x00000000 : aebes (24:24)
0x00000000 : mces (23:23)
0x00000001 : uies (22:22)
0x00000000 : acses (21:21)
0x00000000 : ures (20:20)
0x00000000 : ecrces (19:19)
0x00000001 : mtlpes (18:18)
```

```
0x00000001 : roes (17:17)
0x00000000 : uces (16:16)
0x00000000 : caes (15:15)
0x00000000 : ctes (14:14)
0x00000001 : fces (13:13)
0x00000001 : ptlpes (12:12)
0x00000001 : sldes (05:05)
0x00000001 : dlpes (04:04)
```

2. ECRC Error injection using the PEI 4.0 card.

```
C:\Program Files\PEI\PCIE>"C:\Python3_6_8\python.exe" pei_run.py -t TLP_GEN -
f TD=1 Length=1 -c 1
Starting TLP_GEN()
INFO: Starting PEI.run() on Endpoint image
Count TLP_US: 1
INFO: Trigger disarmed after 1.0sec.
```

NOTE

Traffic may be needed to inject the error - if the error injection will not trigger, try executing the CScripts "pcie.topology" command simultaneously with the error injection.

3. Re-evaluate the PCIe status registers. DEVSTS should trigger a non-fatal error detected. ECRCE in ERRUNCSTS will show that the ECRC error was detected.

```
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.devsts.show
0x00000000 : tp (05:05)
0x00000000 : apd (04:04)
0x00000000 : urd (03:03)
0x00000000 : fed (02:02)
0x00000001 : nfed (01:01)
0x00000000 : ced (00:00)

>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.erruncsts.show()
0x00000000 : pcrchf (30:30)
0x00000000 : midetlp (29:29)
0x00000000 : idecf (28:28)
0x00000000 : dmwreb (27:27)
0x00000000 : ptlpeb (26:26)
0x00000000 : tpbe (25:25)
0x00000000 : aebe (24:24)
0x00000000 : mce (23:23)
0x00000000 : uie (22:22)
0x00000000 : acse (21:21)
0x00000000 : ure (20:20)
0x00000001 : ecrce (19:19)
0x00000000 : mtlpe (18:18)
0x00000000 : roe (17:17)
0x00000000 : uce (16:16)
0x00000000 : cae (15:15)
0x00000000 : cte (14:14)
0x00000000 : fce (13:13)
0x00000000 : ptlpe (12:12)
0x00000000 : slde (05:05)
0x00000000 : dlpe (04:04)

>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.rooterrsts.show()
0x00000000 : aemn (31:27)
0x00000000 : femr (06:06)
0x00000001 : nfemr (05:05)
0x00000000 : fuf (04:04)
0x00000000 : mefr (03:03)
0x00000001 : efr (02:02)
0x00000000 : mcer (01:01)
```

```

0x00000000 : cer (00:00)

>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.pcists.show()
0x00000000 : dpe (15:15)
0x00000001 : sse (14:14)
0x00000000 : rma (13:13)
0x00000000 : rta (12:12)
0x00000000 : sta (11:11)
0x00000000 : dvt (10:09)
0x00000000 : mdpd (08:08)
0x00000000 : fbc (07:07)
0x00000000 : c66 (05:05)
0x00000001 : cape (04:04)
0x00000000 : ints (03:03)

>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.errsrcid.show()
0x00002910 : efsid (31:16)
0x00000000 : ecsid (15:00)

```

4. Go/release the system. Behavior may vary based on OEM configuration.

6.3 PCIe/CXL.io Uncorrectable Fatal Flow

Fatal errors are uncorrectable error conditions which render the particular link and related hardware unreliable. For fatal errors, a reset of the components on the link may be required to return to reliable operation. Platform handling of fatal errors, and any efforts to limit the effects of these errors, is platform implementation specific.

6.3.1 PCIe/CXL.io Fatal Error (UR, F)

Test Objective

This test injects an Fatal Unsupported Request Error to a PCIe/CXL 2.0 device. This triggers the PCIe/CXL device to signal a Fatal PCIe/CXL.io Error.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B step
BIOS	WW35'24 BKC
OS Version	CentOS 9
Kernel	6.10.6
HW Configuration	1 Socket
CScripts Version	BHS.2422.6001
Mode of Operation	Firmware First Mode

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration. This recipe is valid in both Operating System First Mode and Firmware First Mode.

Ensure CXL and IEH hierarchy are cleared and endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

In this recipe, inject a UR to target device 15:0.0.

Table 6. Target Device Information

Device	BDF	CScripts Node
End Point	15:0.0	-
Root Port	14:6.0	pxp0.rp2
IEH	14:0.4	sieh3 (PE0)

NOTE

See the *Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores Registers Specification*, document number 637776, for more details on selecting the correct IEH Device Configuration.

1. Halt the platform. Ensure that the relevant masks are cleared, and the severity is as expected for the root port and endpoint device. Check that the platform is in a clean and healthy state.

```
>>> halt()

# Run "error.clearAllErrors()" to clear any errors before injection.
>>> error.clearAllErrors()

# Clear and check the RPPIO status
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rppiosts=-1
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rppiosts.show()
0x00000000 : memcto (18:18) -- Memory Request Completion Timeout
0x00000000 : memcac (17:17) -- Memory Request received CA Completion
0x00000000 : memurc (16:16) -- Memory Request received UR Completion
0x00000000 : iocac (09:09) -- I/O Request received CA Completion
0x00000000 : iourc (08:08) -- I/O Request received UR Completion
0x00000000 : cfgcto (02:02) -- Configuration Request Completion Timeout
0x00000000 : cfgcac (01:01) -- Configuration Request received CA Completion
0x00000000 : cfggurc (00:00) -- Configuration Request received UR Completion

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rppiomsk=0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rppiomsk.show()
0x00000000 : memcto (18:18) -- Memory Request Completion Timeout Mask
0x00000000 : memcac (17:17) -- Memory Request received CA Completion Mask
0x00000000 : memurc (16:16) -- Memory Request received UR Completion Mask
0x00000000 : ioccto (10:10) -- I/O Request Completion Timeout Mask
0x00000000 : iocac (09:09) -- I/O Request received CA Completion Mask
0x00000000 : iourc (08:08) -- I/O Request received UR Completion Mask
```

```
0x00000000 : cfgcto (02:02) -- Configuration Request Completion Timeout Mask
0x00000000 : cfgcac (01:01) -- Configuration Request received CA Completion Mask
0x00000000 : cfgurc (00:00) -- Configuration Request received UR Completion Mask
```

2. Inject a fatal UR to a PCIe/CXL device. For a fatal UR, the command is doing a mem read with a severity of fatal.

```
>>> ei.iio.injectURError(0x15,0,0,0,"mem",severity="fatal")
Mode: mem, selecting BAR0 with Base Address = 0x93300000 : 64-bits
Disabling memory space...
Calculated BAR: 0x93300000

Setting Severity as fatal

Checking for successful error injection to PCI BDF...

UR_reg value: 0x1

Error successfully injected to PCIe -> Bus:21,Device:0,Function:0!
Re-enabling memory space...
```

3. Re-evaluate the PCIe/CXL status registers. RPPIOSTS should show a memory request received UR completion.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rppiosts.show()
0x00000000 : memcto (18:18) -- Memory Request Completion Timeout
0x00000000 : memcac (17:17) -- Memory Request received CA Completion
0x00000001 : memurc (16:16) -- Memory Request received UR Completion
0x00000000 : iocto (10:10) -- I/O Request Completion Timeout
0x00000000 : iocac (09:09) -- I/O Request received CA Completion
0x00000000 : iourc (08:08) -- I/O Request received UR Completion
0x00000000 : cfgcto (02:02) -- Configuration Request Completion Timeout
0x00000000 : cfgcac (01:01) -- Configuration Request received CA Completion
0x00000000 : cfgurc (00:00) -- Configuration Request received UR Completion

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts.show()
sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.rooterrsts.show()
0x00000000 : aemn (31:27) -- This register indicates which MSI/MSI-X vector is used for the
interrupt message generated in association with any of the status bits of this Capability.
For...
0x00000001 : femr (06:06) -- Fatal Error Message Received
0x00000000 : nfemr (05:05) -- Non-Fatal Error message Received
0x00000001 : fuf (04:04) -- First Uncorrectable Fatal
0x00000000 : mefr (03:03) -- Multiple Error Fatal/Non-Fatal Received
0x00000001 : efr (02:02) -- Error Fatal/Non-Fatal Received
0x00000000 : mcer (01:01) -- Multiple Error Correctable Error Received
0x00000000 : cer (00:00) -- Correctable Error Received

>>>sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.errsrcid.show()
0x00001500 : efsid (31:16) -- Loaded with the Requester ID indicated in the received ERR_FATAL
or ERR_NONFATAL Message when the ERR_FATAL/NONFATAL Received bit is not already set.
0x00000000 : ecsid (15:00) -- Loaded with the Requester ID indicated in the received ERR_COR
Message when the ERR_COR Received bit is not already set.

>>> pci.decode(15,0,0)
0x104: Uncorrectable Error Status Register = 0x100000
[25] TLP Prefix Blocked Error Status (Optional) = 0x0
[24] AtomicOp Egress Blocked Status (Optional) = 0x0
[23] MC Blocked TLP Status (Optional) = 0x0
[22] Uncorrectable Internal Error Status (Optional) = 0x0
[21] ACS Violation Status (Optional) = 0x0
[20] Unsupported Request Error Status = 0x1
[19] ECRC Error Status (Optional) = 0x0
[18] Malformed TLP Status = 0x0
[17] Receiver Overflow Status (Optional) = 0x0
[16] Unexpected Completion Status = 0x0
[15] Completer Abort Status (Optional) = 0x0
```

[14]	Completion Timeout Status	= 0x0
[13]	Flow Control Protocol Error Status (Optional)	= 0x0
[12]	Poisoned TLP Status	= 0x0
[5]	Surprise Down Error Status (Optional)	= 0x0
[4]	Data Link Protocol Error Status	= 0x0
[0]	Undefined	= 0x0
0x110:	Correctable Error Status Register	= 0x0000
[15]	Header Log Overflow Status	= 0x0
[14]	Corrected Internal Error Status	= 0x0
[13]	Advisory Non-Fatal Error Status	= 0x0
[12]	Replay Timer Timeout Status	= 0x0
[8]	REPLAY_NUM Rollover Status	= 0x0
[7]	Bad DLLP Status	= 0x0
[6]	Bad TLP Status	= 0x0
[0]	Receiver Error Status	= 0x0
0x118:	Advanced Error Capabilities and Control Register	= 0x02b4
[11]	TLP Prefix Log Present	= 0x0
[10]	Multiple Header Recording Enable	= 0x0
[9]	Multiple Header Recording Capable	= 0x1
[8]	ECRC Check Enable	= 0x0
[7]	ECRC Check Capable	= 0x1
[6]	ECRC Generation Enable	= 0x0
[5]	ECRC Generation Capable	= 0x1
[4:0]	First Error Pointer	= 0x14
0x11c:	Header Log Register (1st DW)	= 0x80001
0x120:	Header Log Register (2nd DW)	= 0xfd00000f
0x124:	Header Log Register (3rd DW)	= 0x93300000
0x128:	Header Log Register (4th DW)	= 0xd975564c

4. Check IEH Errors. Ensure that the Global IEH and Satellite IEH correspond to the correct PCIe/CXL Device. This step is not applicable if this recipe is run in OSFM.
 - Check satieh3 detected a fatal error status, which points to DEV4_FA_STS. The Registers Specification defines this bit to x16 Root Port 2 or BDF X:6.0 or pxxXXX. This should match the RP information gathered at the beginning of this recipe.
 - Check globalieh detected a fatal error status coming from DEV5_FA_STS. The EDS defines this bit to PE0 or SATIEH3. This step is not applicable if this recipe is run in OSFM.

```
>>> error.check_ieh_errors()

...

=====
=====
IEH_3_IO0: REGISTERS AND FIELDS DESCRIPTIONS
=====
=====
Register: socket0.io0.uncore.satieh.satieh3.gerrfatsts
dev4_fa_sts = 0x1
Dev4 Error Status (DEV4_FA_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.satieh.satieh3.gerrfatstime
0x00000099b1c67d45 : g_faferr_time (63:00) -- Global FAFERR Time Stamp (G_FAFERR_TIME)

=> socket0.io0.uncore.satieh.satieh3.gtime
0x00000099b1c692f4 : gtime_value (63:00) -- Global Error Timer value (GTIME_Value)

-----
-----
Register: socket0.io0.uncore.satieh.satieh3.gsysevtsts
fa_sysev_sts = 0x1
Fatal Error Status - A fatal error was detected when set (FA_SYSEV_STS)

ADDITIONAL INFO REGISTERS:
```

```

=> socket0.io0.uncore.satieh.satieh3.gsysevtctl
0x00000001 : fa_sysev_m (02:02) -- Fatal Error Mask(FA_SYSEV_M) Fatal Error System
Event Mask: 1 - Unmask sy...
0x00000001 : nf_sysev_m (01:01) -- Nonfatal Error Mask(NF_SYSEV_M) NonFatal Error
System Event Mask: 1 - Unm...
0x00000001 : co_sysev_m (00:00) -- Correctable Error Mask(CO_SYSEV_M) Correctable
Error System Event Mask: 1...

-----
-----
Register: socket0.io0.uncore.satieh.satieh3.gerrsrcid
gefsid = 0x321
ERR_COR Source Identification (ECSID) Requester ID of the source when a correctable error
is received.

...

=====
=====
IEH_GLOBAL: REGISTERS AND FIELDS DESCRIPTIONS
=====
=====
Register: socket0.io0.uncore.globalieh.iehregs.gerrfatsts
dev5_fa_sts = 0x1
Dev5 Error Status (DEV6_FA_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.globalieh.iehregs.gerrfatstime
0x000032c3f246dce4 : g_faferr_time (63:00) -- Global FAFERR Time Stamp (G_FAFERR_TIME)

=> socket0.io0.uncore.globalieh.iehregs.gtime
0x000032d38452fea0 : gtime_value (63:00) -- Global Error Timer value (GTIME_Value)

-----
-----
Register: socket0.io0.uncore.globalieh.iehregs.gsysevtsts
fa_sysev_sts = 0x1
Fatal Error Status - A fatal error was detected when set (FA_SYSEV_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.globalieh.iehregs.gsysevtctl
0x00000001 : fa_sysev_m (02:02) -- Fatal Error Mask(FA_SYSEV_M) Fatal Error System
Event Mask: 1 - Unmask sy...
0x00000001 : nf_sysev_m (01:01) -- Nonfatal Error Mask(NF_SYSEV_M) NonFatal Error
System Event Mask: 1 - Unm...
0x00000001 : co_sysev_m (00:00) -- Correctable Error Mask(CO_SYSEV_M) Correctable
Error System Event Mask: 1...

=> socket0.io0.uncore.globalieh.iehregs.gsysevmap
0x00000001 : co_sysev_map (01:00) -- 11: Reserved 10: Generate NMI 01 Generate SMI 00:
No system event gener...

-----
-----
Register: socket0.io0.uncore.globalieh.iehregs.gerrsrcid
gefsid = 0x88
ERR_COR Source Identification (ECSID) Requester ID of the source when a correctable error
is received.

gecsid = 0x88
ERR_COR Source Identification (ECSID) Requester ID of the source when a correctable error
is received.

-----
-----
...

```

- Go/release the system. Check DMesg. See the PCIe/CXL Device Configuration chapter for more information.

NOTE

You may not be able to see any DMesg due to the system immediately panicking upon seeing the fatal WHEA message in FFM. In OSFM it is dependent on the configuration.

6.4 Retraining and Recovery

Ability of the CPU to initiate a retraining of the PCIe* link due to certain error conditions. The point of this feature is to protect the link against signal integrity issues caused by EMI, marginal links, poor connectors, long trace lengths, and so on. The error scenarios that will induce a retrain include excessive Link Layer Replays (replay number rollover), inferred electrical idle (caused by too infrequent Skip Ordered Sets or Flow Control Updates), and excessive physical layer errors (Receiver Errors). Retraining the link involves moving the LTSSM (Link Training and Status State Machine) into the Recovery state and proceeding from there. Once in the Recovery state, the LTSSM can initiate a speed or width degradation if further errors are detected during the retraining sequence. It provides a mechanism to detect and most of the time correct issues via either replay or retraining and recovery.

6.4.1 Replay Number Rollover

Test Objective

An injection of repeated CRC errors such that a sequence is replayed at least four times. After error injection, OS should report rollover timeout and AER register should be populated for corresponding errors.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	Generic PCIe* Card
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [PEI Card](#) on page 18
- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

Ensure PCIe and IEH hierarchy are cleared and endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Ensure relevant masks are cleared, severity is as expected, and platform is in a clean and healthy state. Halt the platform.

```
>>> halt()
```

2. Error injection using the PEI 4.0 card.

```
C:\Program Files\PEI\PCIe>python pei_run.py -t REPLAY_NUM_nak -c 4
Starting REPLAY_NUM_nak()
INFO: Starting PEI_run() on Endpoint image
INFO: Trigger disarmed after 3.0sec.
Count INJ_US: 4
```

3. Re-evaluate the PCIe status registers. DEVSTS should show that the correctable error field triggered, ERRCORSTS should show the RTTE/RNRE trigger, and ROOTERRSTS should also show that the correctable error and MCER was received. ERRSRCID should show the ECSID bit set to the BDF of the RP.

```
>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.devsts.showfields()
0x00000000 : tp (05:05)
0x00000000 : apd (04:04)
0x00000000 : fed (02:02)
0x00000000 : nfed (01:01)
0x00000001 : ced (00:00)

>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.errcorsts.showfields()
0x00000000 : hloe (15:15)
0x00000000 : cie (14:14)
0x00000000 : anfe (13:13)
0x00000000 : rtte (12:12)
0x00000001 : rnre (08:08)
0x00000000 : bdllpe (07:07)
0x00000000 : btlpe (06:06)
0x00000000 : re (00:00)

>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.rooterrsts.showfields()
0x00000000 : aemn (31:27)
0x00000000 : femr (06:06)
0x00000000 : nfemr (05:05)
0x00000000 : fuf (04:04)
0x00000000 : mefr (03:03)
0x00000000 : efr (02:02)
0x00000000 : mcer (01:01)
0x00000001 : cer (00:00)

>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.errsrcid.showfields()
0x00000000 : efsid (31:16)
0x00002910 : ecsid (15:00)

>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.linksts.showfields()
```

```
0x00000000 : labs (15:15)
0x00000000 : lbms (14:14)
0x00000001 : dllla (13:13)
0x00000001 : scc (12:12)
0x00000000 : lt (11:11)
0x00000000 : undefined (10:10)
0x00000010 : nlw (09:04)
0x00000005 : cls (03:00)
```

- Go/release the system, all registers listed previously should automatically clear, there should be no need to manually clear these registers.

```
>>> go()
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.devsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.errcorsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.rooterrsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.linksts.showfields()
0x00000000 : labs (15:15)
0x00000000 : lbms (14:14)
0x00000001 : dllla (13:13)
0x00000001 : scc (12:12)
0x00000000 : lt (11:11)
0x00000000 : undefined (10:10)
0x00000010 : nlw (09:04)
0x00000005 : cls (03:00)
```

- Evaluate the OS Error Logs and confirm the PCIe error logs are captured/reported.

```
kworker/0:3-2810 [000] ..... 356.170776: aer_event: 0000:29:02.0 PCIe Bus Error:
severity=Corrected, RELAY_NUM Rollover, TLP Header=Not available
```

6.4.2 Correctable Error Threshold

Test Objective

For this scenario, use a test card to inject receiver errors, framing errors, or 8b/10b errors on inbound transactions. Inject enough errors to reach the threshold and BIOS should produce SMI. The error counters are reset once a threshold-based action is triggered

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW15 BKC
OS Version	CentOS* Stream 8
<i>continued...</i>	

Kernel	5.19.0-gnr.po.bkc.3.13.14.x86_64
HW Configuration	Generic PCIe* Card
CScripts Version	GNR 2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [PEI Card](#) on page 18
- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > IIO Error Enabling >
 - OS Native AER Support = Disable
 - IIO EDPC Support = On Fatal and Non-Fatal Errors
- EDKII > Platform Configuration > System Event Log > PCIe* Error Enabling >
 - PCIe* Corrected Error Threshold Counter = <Enable>
 - PCIe* Corrected Error Threshold = <a number larger than 1, for example 3>

Ensure PCIe and IEH hierarchy are cleared and endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Ensure platform is in a clean and healthy state. Halt the platform. Ensure the corresponding PCIe status registers and correctable error count are cleared. CORERRTH should match the value defined in the BIOS for the correctable error threshold. In this recipe, it was defined as three.

```
>>> sv.socket0.io0.pi5.pxp0.rp2.cfg.corerrcnt.show()
0x00000000 : errcnt (31:00) -- Link correctable error counter...
>>> sv.socket0.io0.pi5.pxp0.rp2.cfg.corerrth.show()
0x00000003 : errth (31:00) -- Link correctable error threshold...
```

2. Inject the PCIe correctable error a populated port. This will inject a receiver error.

```
>>> ei.utils.resetInjectorLockCheck()
>>> ei.iio.injectPcieError(socket=0, port="pxp0.rp2", errType="ce")
```

3. Re-evaluate the PCIe status registers. The correctable error count should reflect the number of errors injected. ERRCORSTS should show that RE was detected and ROOTERRSTS should also show that CE was received.

```
>>> sv.socket0.uncore.io0.uncore.pi5.pxp0.rp2.cfg.corerrcnt.show()
0x00000001 : errcnt (31:00) -- Link correctable error counter...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.errcorsts.show()
0x00000000 : hloe (15:15) -- Header Log Overflow Error
0x00000000 : cie (14:14) -- Correctable Internal Error
0x00000000 : anfe (13:13) -- Advisory Non-Fatal Error
0x00000000 : rtte (12:12) -- The PCIe sets this bit when replay...
0x00000000 : rnre (08:08) -- The PCIe sets this bit when the...
```

```
0x00000000 : bdlpe (07:07) -- The switch sets this bit on...
0x00000000 : btlpe (06:06) -- The switch sets this bit on CRC...
0x00000001 : re (00:00) -- The PCIe sets this bit when the...
```

4. Inject two additional errors so the count meets the threshold, and then check the corresponding status registers and count threshold were met. The BIOS will clear the counter when the threshold is met.

```
>>> ei.utils.resetInjectorLockCheck()
>>> ei.iio.injectPcieError(socket=0, port="pxp0.rp2", errType="ce")
>>> ei.utils.resetInjectorLockCheck()
>>> ei.iio.injectPcieError(socket=0, port="pxp0.rp2", errType="ce")
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.corerrcnt.show()
0x00000000 : errcnt (31:00) -- Link correctable error counter....
```

5. Error should be signaled when the PCIe correctable error count meets the threshold. Signaling will vary per platform.
6. Evaluate the OS Error Logs and confirm the PCIe error logs are captured/reported.

```
kernel: {4}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 0
kernel: {4}[Hardware Error]: It has been corrected by h/w and requires no further action
kernel: {4}[Hardware Error]: event severity: corrected
kernel: {4}[Hardware Error]: Error 0, type: corrected
kernel: {4}[Hardware Error]: section_type: PCIe error
kernel: {4}[Hardware Error]: port_type: 4, root port
kernel: {4}[Hardware Error]: version: 3.0
kernel: {4}[Hardware Error]: command: 0x0547, status: 0x0010
kernel: {4}[Hardware Error]: device_id: 0000:80:05.0
kernel: {4}[Hardware Error]: slot: 5
kernel: {4}[Hardware Error]: secondary_bus: 0x81
kernel: {4}[Hardware Error]: vendor_id: 0x8086, device_id: 0x352c
kernel: {4}[Hardware Error]: class_code: 060400
kernel: {4}[Hardware Error]: bridge: secondary status: 0x0000, control: 0x0003
kernel: pcieport 0000:80:05.0: AER: aer_status: 0x00000001, aer_mask: 0x00000000
kernel: pcieport 0000:80:05.0: AER: [ 0] RxErr (First)
kernel: pcieport 0000:80:05.0: AER: aer_layer=Physical Layer, aer_agent=Receiver ID

# cd /sys/kernel/debug/tracing/
# echo 1 > events/ras/aer_event/enable
# cat trace_pipe
kworker/0:2-2370 [000] .... 16002.271921: aer_event: 0000:37:01.0 PCIe Bus Error:
severity=Corrected, Receiver Error, TLP Header=Not available
```

6.5 Link CRC Error Check and Retry

6.5.1 Overview

Ability to detect the CRC error and retries the transaction. PCI Express link reliability and protection enforces via error detection and replays during link operation, especially, in low signal integrity situation. The point of this feature is to protect the link against signal integrity issues caused by EMI, marginal links, poor connectors, long trace lengths, and so forth. It provides a mechanism to detect and most of the time correct issues via replay.

Test Objective

Link CRC errors are correctable in nature and do not result in escalation if they occur frequently. When the errors occur, it causes a retransmission (replay) of the offending transaction and all others transmitted after it. The general methodology is to inject a CRC error while I/O stress traffic is running and then progressively inject more errors.

In all cases, there should be no data corruption and the system should not have any errors logged except for Bad TLP errors, Replay Timer Timeout Status, and eventually a Replay Number Rollover status.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	Generic PCIe* Card
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [PEI Card](#) on page 18
- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > PCIe Error Enabling > PCIe Corrected Error Threshold Counter = Disable

Ensure PCIe and IEH hierarchy are cleared and endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Check the masks to ensure the error is reported for CEs at the OS, specifically the bad DLLP is cleared.

```
>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.errcormsk
0x0
>>> halt()
```

2. Error injection using the PEI card.

```
C:\Program Files\PEI\PCIe>python pei_run.py -t BAD_DLLP_crc -c 1
Starting BAD_DLLP_crc()
INFO: Starting PEI.run() on Endpoint image
INFO: Trigger disarmed after 1.0sec.
Count INJ_US: 1
```

NOTE

Traffic may be needed to inject an error - if the error injection will not trigger, try executing the "pcie.topology" command simultaneously with the error injection.

3. Re-evaluate the PCIe status registers. DEVSTS should show that the correctable error field is triggered. ERRCORSTS should show the BAD DLLP trigger, and ROOTERRSTS should also show that the correctable error was received.

```
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.devsts.showfields()
0x00000000 : tp (05:05)
0x00000000 : apd (04:04)
0x00000000 : urd (03:03)
0x00000000 : fed (02:02)
0x00000000 : nfed (01:01)
0x00000001 : ced (00:00)

>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.errcorsts.showfields()
0x00000000 : hloe (15:15)
0x00000000 : cie (14:14)
0x00000000 : anfe (13:13)
0x00000000 : rtte (12:12)
0x00000000 : rnre (08:08)
0x00000001 : bdllpe (07:07)
0x00000000 : btlpe (06:06)
0x00000000 : re (00:00)

>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.rooterrsts.showfields()
0x00000000 : aemn (31:27)
0x00000000 : femr (06:06)
0x00000000 : nfemr (05:05)
0x00000000 : fuf (04:04)
0x00000000 : mefr (03:03)
0x00000000 : efr (02:02)
0x00000000 : mcer (01:01)
0x00000001 : cer (00:00)

>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.errsrcid.showfields()
0x00000000 : efsid (31:16)
0x00002910 : ecsid (15:00)

>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.linksts.showfields()
0x00000000 : labs (15:15)
0x00000000 : lbms (14:14)
0x00000001 : dl1la (13:13)
0x00000001 : scc (12:12)
0x00000000 : lt (11:11)
0x00000000 : undefined (10:10)
0x00000010 : nlw (09:04)
0x00000005 : cls (03:00)
```

4. Go/release the system. The status registers should clear. Confirm the link is up and operating at the expected speed.

```
>>> go()
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.devsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.rooterrsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.errcorsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.linksts.showfields
0x00000000 : labs (15:15)
0x00000000 : lbms (14:14)
0x00000001 : dl1la (13:13)
0x00000001 : scc (12:12)
0x00000000 : lt (11:11)
```

```
0x00000000 : undefined (10:10)
0x00000010 : nlw (09:04)
0x00000005 : cls (03:00)
```

5. Evaluate the OS Error Logs and confirm the PCIe error logs are captured/reported.

```
kworker/0:3-2998 [000] ..... 2021.033406: aer_event: 0000:29:02.0 PCIe Bus Error:
severity=Corrected, Bad DLLP, TLP Header=Not available
```

6.6 Surprise Hot-Plug

NVMe* SSD PCIe cards can be hot added/removed into the PCIe slots directly connected to the processor without going through prior notifications to the system. The flow is dependent on software interrupts be invoked. The BIOS must correctly configure the "presence_detect_changed_enable" for Hot-Plug and Link_down conditions. The processor's IIO supports termination of the outstanding transaction with the PCIe link down condition. The software stack, including the OS with Hot-Plug aware drivers, is required to properly manage the hot insertion or removal flows.

6.6.1 Surprise Hot-Add

Test Objective

Surprise PCIe Hot Plug is a variation of PCIe Card (NVMe or EDSFF Devices) Hot Plug with one main difference: the hot plug operations do not involve using the platform attention button or any software mechanism that gives warning to the hardware that a hot plug event is imminent. This scenario involves booting the system with the target HSBP slots empty and then using the Surprise Hot Plug procedure to add the card/device to the system while it continues to run. The OS should detect the newly added card, enumerate it in the PCI configuration space, and load the appropriate driver.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW25 BKC
OS Version	Cent OS* Stream 8
Kernel	5.19.0-gnr.po.bkc.7.2.21.x86_64
HW Configuration	NVMe or EDSFF device
CScripts Version	GNR2322.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [PEI Card](#) on page 18

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Socket Configuration > IIO Configuration > Global Configuration
 - Hot-Plug = Enable
 - NPEM = Enable
- Socket Configuration > IIO Configuration > Socket X Configuration > PCI Express X > Port X
 - Hot-Plug Capable = Disable
 - Surprise Hot-Plug Capable = Enable

Ensure PCIe and IEH hierarchy are cleared and endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Identify the HSBP ports that support Hot Plug -> HSBP ports.

NOTE

Check the corresponding Bus/Device/Function (BDF) of the target slot. You can use the following CScripts commands to get the BDF: `pcie.topology()` and `pcie.port_map()` to determine device and slot mapping. Also, the slot capabilities register can be used to get the physical slot number associated with a root port.

2. Boot the system with the target HSBP slot empty. Verify the LED status - power LED off.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.slotctl.show()
0x00000001 : dllsce (12:12) --
0x00000001 : hpie (05:05) --
0x00000001 : ccie (04:04) --
0x00000001 : pdcie (03:03) --
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.slotcap.show()
0x00000001 : hpc (06:06) --
0x00000001 : hps (05:05) --
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.slotsts.show()
0x00000000 : dllscs (08:08) -- This bit is set (if it is not already set) when the st...
0x00000000 : els (07:07) -- A read to this register returns the current state of the ...
0x00000000 : pds (06:06) -- This field conveys the Presence Detect status
0x00000000 : mrlss (05:05) -- This bit reports the status of an MRL sensor if it is i...
0x00000000 : ccs (04:04) -- This bit is set when the hot-plug controller completes an...
0x00000000 : pdcs (03:03) -- This bit is set when the value reported in the Presence ...
0x00000000 : mrlsc (02:02) -- This bit is set when an MRL Sensor Changed event is det...
0x00000000 : pfd (01:01) -- This bit is set when a power fault event is detected by t...
0x00000000 : abp (00:00) -- This bit is set when the attention button is pressed. It ...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.linkctl.show()
0x00000000 : drs (15:14) -- Indicates the mechanism used to report reception of a DRS...
0x00000000 : labie (11:11) -- When Set, this bit enables the generation of an interr...
0x00000000 : lbmie (10:10) -- When set, this bit enables the generation of an interr...
0x00000000 : hawd (09:09) -- Set, this bit disables hardware from changing the Link w...
0x00000000 : ecpm (08:08) -- Applicable only for Upstream Ports and with form factors...
0x00000000 : es (07:07) -- When set, this bit forces extended transmission of 4096 FT...
0x00000001 : cccfg (06:06) -- When set, this bit indicates that this component and th...
0x00000000 : rl (05:05) -- When set, this bit initiates link retraining by directing ...
0x00000000 : ld (04:04) -- This bit disables the Link by directing the LTSSM to the D...
0x00000000 : rcb (03:03) -- This bit indicates the RCB value for Root Port, Endpoints...
0x00000000 : ptmptaib (02:02) -- For a device that supports PTM, if PTM Propagation D...
```



```
0x00000000 : aspmctl (01:00) -- This field controls the level of ASPM enabled on a gi...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.linksts.show()
0x00000000 : labs (15:15) -- This bit is set by hardware to indicate that the PCIe cl...
0x00000000 : lbms (14:14) -- This bit is set by hardware to indicate that either of t...
0x00000000 : dllla (13:13) -- This bit indicates the status of the Data Link Control ...
0x00000001 : scc (12:12) -- When to 1b, this bit indicates that the component uses th...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that the Physical Laye...
0x00000000 : undefined (10:10) -- Not applicable
0x00000000 : nlw (09:04) -- This field indicates the negotiated width of the PCI Expr...
0x00000001 : cls (03:00) -- This field indicates the negotiated link speed of the giv...
```

3. Attach the U.2 NVMe* card to the target HSBP slot. Confirm the presence detect state bit is set to show that the port is populated. Also, confirm that the DLLA bit is set.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.slotsts.show()
0x00000001 : pds (06:06) -- This field conveys the Presence Detect status
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.linksts.show()
0x00000001 : labs (15:15) -- This bit is set by hardware to indicate that the PCIe cl...
0x00000001 : lbms (14:14) -- This bit is set by hardware to indicate that either of t...
0x00000001 : dllla (13:13) -- This bit indicates the status of the Data Link Control ...
0x00000001 : scc (12:12) -- When to 1b, this bit indicates that the component uses th...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that the Physical Laye...
0x00000000 : nlw (09:04) -- This field indicates the negotiated width of the PCI Expr...
0x00000003 : cls (03:00) -- This field indicates the negotiated link speed of the giv...
```

4. Verify that the card is enumerated.

```
a. in python sls command card should emnumerated "Up_L0"
>>> pcie.sls
=====
SOCKET0
=====
PXP0 is x2 x2 x2 x2 x8 -- -- --
PXP1 is x16 -- -- -- -- --
PXP2 is x16 -- -- -- -- --
PXP3 is x4 -- x4 -- x4 -- x4 --
PXP4 is x4 -- x4 -- x4 -- x4 --
PXP5 is x4 -- x4 -- x4 -- x4 --
Port pxp0.rp0 (PCIe) is x8 (ilw=8) (GEN3/dP7/uP5). LTSSM = UP_L0
```

- b. Card should enumerated in C-Script command `pcie.topology()`
- c. In OS level - check command `"lspci -tv"` to find devices and associated root ports.

6.6.2 Surprise Hot-Remove

Test Objective

Surprise PCIe Hot Plug is a variation of PCIe Card (NVMe or EDSFF Devices) Hot Plug with one main difference: the hot plug operations do not involve using the platform attention button or any software mechanism that gives warning to the hardware that a hot plug event is imminent. This scenario involves booting the system with the target PCIe slot populated and then using the Surprise Hot Remove procedure to remove the card from the system while it continues to run. Depending on the type of card, the cards driver, and the operating system, the resultant behavior could range from the device disappearing somewhat gracefully, to a fatal system error that brings the system down. However, neither the card nor the platform should be physically damaged in this scenario.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW25 BKC
OS Version	Cent OS Stream 8
Kernel	5.19.0-gnr.po.bkc.7.2.21.x86_64
HW Configuration	NVMe or EDSFF device
CScripts Version	GNR2322.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [PEI Card](#) on page 18
- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Socket Configuration > IIO Configuration > Global Configuration
 - Hot-Plug = Enable
 - NPEM = Enable
- Socket Configuration > IIO Configuration > Socket X Configuration > PCI Express X > Port X
 - Hot-Plug Capable = Disable
 - Surprise Hot-Plug Capable = Enable

Ensure PCIe and IEH hierarchy are cleared and endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Identify the HSBP ports that support Hot Plug -> HSBP ports.

NOTE

Check the corresponding Bus/Device/Function (BDF) of the target slot. You can use the following CScripts commands to get the BDF: `pcie.topology()` and `pcie.port_map()` to determine device/slot mapping. Also, the Slot Capabilities register can be used to get the physical slot number associated with a root port.

2. Boot the system with the target HSBP slot populated. Verify the LED status - power LED on.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.slotctl.show()
0x00000001 : dillsce (12:12) --
0x00000001 : hpie (05:05) --
0x00000001 : ccie (04:04) --
0x00000001 : pdcie (03:03) --
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.slotcap.show()
0x0000004b : splv (14:07) -- This field specifies the upper limit on power...
0x00000001 : hpc (06:06) -- This field defines hot-plug support capabilities...
0x00000001 : hps (05:05) -- This field indicates that a device in this slot...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.slotsts.show()
0x00000001 : pds (06:06) -- This field conveys the Presence Detect status
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.linkctl.show()
0x00000000 : drs (15:14) --
0x00000000 : labie (11:11) --
0x00000000 : lbmie (10:10) --
0x00000000 : hawd (09:09) --
0x00000000 : ecpm (08:08) --
0x00000000 : es (07:07) --
0x00000001 : cccfg (06:06) --
0x00000000 : rl (05:05) --
0x00000000 : ld (04:04) --
0x00000000 : rcb (03:03) --
0x00000000 : ptmptaib (02:02) --
0x00000000 : aspmctl (01:00) --
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.linksts.show()
0x00000000 : labs (15:15) -- This bit is set by hardware to indicate that the PCIe cl...
0x00000000 : lbms (14:14) -- This bit is set by hardware to indicate that either of t...
0x00000001 : dllla (13:13) -- This bit indicates the status of the Data Link Control ...
0x00000001 : scc (12:12) -- When to lb, this bit indicates that the component uses th...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that the Physical Laye...
0x00000000 : undefined (10:10) -- Not applicable
0x00000008 : nlw (09:04) -- This field indicates the negotiated width of the PCI Expr...
0x00000003 : cls (03:00) -- This field indicates the negotiated link speed of the giv...
```

3. Remove the U.2 NVMe* card to target the HSBP slot. Confirm the presence detect state bit is cleared to show that the port is removed. Also, confirm that the DLLA bit is cleared.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.slotsts.show()
0x00000000 : pds (06:06) -- This field conveys the Presence Detect status
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.linksts.show()
0x00000000 : labs (15:15) -- This bit is set by hardware to indicate that the PCIe cl...
0x00000000 : lbms (14:14) -- This bit is set by hardware to indicate that either of t...
0x00000000 : dllla (13:13) -- This bit indicates the status of the Data Link Control ...
0x00000001 : scc (12:12) -- When to lb, this bit indicates that the component uses th...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that the Physical Laye...
0x00000008 : nlw (09:04) -- This field indicates the negotiated width of the PCI Expr...
0x00000001 : cls (03:00) -- This field indicates the negotiated link speed of the giv...
```

4. Verify that the card is removed.

```
a. Card should not appear in sls command. This is seen as DET_SLEEP/QUIET.
>>> pcie.sls
=====
SOCKET0
=====
PXP0 is x2 x2 x2 x2 x8 -- -- --
PXP1 is x16 -- -- -- -- -- --
PXP2 is x16 -- -- -- -- -- --
PXP3 is x4 -- x4 -- x4 -- x4 --
PXP4 is x4 -- x4 -- x4 -- x4 --
PXP5 is x4 -- x4 -- x4 -- x4 --
Port pxp0.rp0 (PCIe) is x8 (ilw=8) (GEN1/-3.5dB). LTSSM = DET_QUIET

b. Card should not appear in C-Script command pcie.topology()
c. In OS level - check command "lspci | grep -i "NV"
Card should not appear and check DMESG log.
```

6.7 Enhanced Downstream Port Containment (eDPC)

Downstream Port Containment (DPC) allows halting of PCIe* traffic below a downstream port after an unmasked uncorrectable error is detected at or below the port, avoiding the potential spread of any data corruption, and permitting error recovery if supported by the software. Enhanced Downstream Port Containment (eDPC) is an enhancement to DPC thereby adding Root Port Programmable IO (RPPIO) errors. RPPIO errors enable fine-grained DPC control when non-posted requests detect certain uncorrected or advisory non-fatal errors. Its main function is to abort PCIe* traffic below a downstream port after an unmasked uncorrectable error or poison is detected at or below the port. This guarantees error containment and allows for possible recovery via the software. Once an uncorrected error is detected, an affected link is deactivated and all the transactions are gracefully handled allowing the rest of the system to operate normally. See Section 7.9.15 of the *PCI Express Base Specification*, Revision 4.0, for further details.

6.7.1 CTO Triggers DPC - FFM

Test Objective

This test uses a completion timeout error injection to trigger enhanced Downstream Port Containment in the Firmware First environment. This exercises the FFM flow where OSPM relinquishes AER and eDPC control whereas it keeps SHP control.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	Generic PCIe* Card
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > IIO Error Enabling >
 - OS Native AER Support = Disable

— IIO EDPC Support = On Fatal and Non-Fatal Errors

Ensure PCIe and IEH hierarchy are cleared and endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Check on the severity level of completion timeout and verify the mask is cleared for a completion timeout. CTO set is set to non-fatal severity with CTES=0.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncmsk.show()
0x00000000 : ptlpebm (26:26) -- Poisoned TLP Egress Blocked Mask
0x00000000 : tpbem (25:25) -- TLP Prefix Blocked Error Mask
0x00000000 : aebem (24:24) -- AtomicOp Egress Blocked Mask
0x00000000 : mcem (23:23) -- MC Blocked TLP Mask
0x00000000 : uiem (22:22) -- Uncorrectable Internal Error Mask
0x00000000 : acsem (21:21) -- ACS Violation Mask
0x00000001 : urem (20:20) -- Unsupported Request Error Mask
0x00000000 : ecr cem (19:19) -- ECRC Error Mask
0x00000000 : mtlpem (18:18) -- Malformed TLP Mask
0x00000000 : roem (17:17) -- Receiver Overflow Mask
0x00000000 : ucem (16:16) -- Unexpected Completion Mask
0x00000000 : caem (15:15) -- Completer Abort Mask
0x00000000 : ctem (14:14) -- Completion Timeout Mask
0x00000000 : fcem (13:13) -- Flow Control Protocol Error Mask
0x00000000 : ptlpem (12:12) -- Poisoned TLP Received Mask
0x00000001 : sldem (05:05) -- This bit does not apply to the...
0x00000000 : dlpebm (04:04) -- Data Link Protocol Error Mask

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsev.show()
0x00000000 : ptlpebs (26:26) -- Poisoned TLP Egress Blocked Severity
0x00000000 : tpbbs (25:25) -- TLP Prefix Blocked Error Severity
0x00000000 : aebs (24:24) -- AtomicOp Egress Blocked Severity
0x00000000 : mces (23:23) -- MC Blocked TLP Severity
0x00000001 : uies (22:22) -- Uncorrectable Internal Error Severity
0x00000000 : acses (21:21) -- ACS Violation Severity
0x00000000 : ures (20:20) -- Unsupported Request Error Severity
0x00000000 : ecr ces (19:19) -- ECRC Error Severity
0x00000001 : mtlpes (18:18) -- Malformed TLP Severity
0x00000001 : roes (17:17) -- Receiver Overflow Error Severity
0x00000000 : uces (16:16) -- Unexpected Completion Error Severity
0x00000000 : caes (15:15) -- Completer Abort Error Severity
0x00000000 : ctes (14:14) -- Completion Timeout Error Severity
0x00000001 : fces (13:13) -- Flow Control Protocol Error Severity
0x00000001 : ptlpes (12:12) -- Poisoned TLP Received Severity
0x00000000 : sldes (05:05) -- This bit does not apply to the...
0x00000001 : dlpes (04:04) -- Data Link Protocol Error Severity

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x00000000 : labs (15:15) -- This bit is set by hardware to indicate...
0x00000001 : lbms (14:14) -- This bit is set by hardware to indicate...
0x00000001 : dlla (13:13) -- This bit indicates the status of the...
0x00000001 : scc (12:12) -- When to lb, this bit indicates that the...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that...
0x00000000 : undefined (10:10) -- Not applicable
0x00000002 : nlw (09:04) -- This field indicates the negotiated...
0x00000003 : cls (03:00) -- This field indicates the negotiated...
```

2. Confirm that DPC is enabled. In Preconditions and Setup, DPC is enabled for both fatal and non-fatal errors. DPCTE is expected to be set to "2" as a result.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcctl.show
0x00000000 : dlaece (07:07) -- This bit must be RW if the DL_Active...
0x00000000 : dpcst (06:06) -- This bit must be RW if the DPC Software...
0x00000001 : ptlpebe (05:05) -- This bit must be RW if the Poisoned...
0x00000001 : dpcece (04:04) -- When Set, this bit enables the sending...
```

```
0x00000001 : dpcie (03:03) -- When Set, this bit enables the generation...
0x00000000 : dpccc (02:02) -- This bit controls the Completion Status...
0x00000002 : dpcte (01:00) -- This field enables DPC and controls...
```

3. Disable ACS.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.acsctl=0
```

4. Inject the PCIe* completion timeout error using CScripts.

```
>>> halt()
>>> ei.utils.resetInjectorLockCheck()
>>> ei.iio.injectPcieError(socket=0, port="pxp0.rp2", errType="uce")
```

5. Check DPCSTS and link status registers and confirm the correct field is triggered. DPCTS is expected to be set to "1" since DPC triggered as a result of the CTO that can be seen in the ERRUNCSTS where the CTE field is set. Device status shows that a non-fatal error was logged, as expected. The system should still be halted and as a result of DPC triggering, SUT should be operating in a down state. This is demonstrated as the DLLA field is cleared, and the CLS shows platform is operating at Gen 1 speed.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.devsts.show
0x00000000 : tp (05:05) -- When set, this bit indicates that the...
0x00000000 : apd (04:04) -- Auxiliary Power is not supported.
0x00000000 : urd (03:03) -- This bit indicates that this function...
0x00000000 : fed (02:02) -- This bit indicates that this function...
0x00000001 : nfed (01:01) -- This bit indicates that this function...
0x00000000 : ced (00:00) -- This bit indicates that this function...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsts.show
0x00000000 : ptlpeb (26:26) -- Poisoned TLP Egress Blocked Status
0x00000000 : tpbe (25:25) -- TLP Prefix Blocked Error Status
0x00000000 : aebe (24:24) -- This bit is set whenever an Atomic...
0x00000000 : mce (23:23) -- This bit is set whenever a Multicast...
0x00000000 : uie (22:22) -- This bit is set whenever an...
0x00000000 : acse (21:21) -- This bit is set whenever an ACS...
0x00000000 : ure (20:20) -- This bit is set whenever an...
0x00000000 : ecrc (19:19) -- ECRC checking is optional for project...
0x00000000 : mtlpe (18:18) -- This bit is set when it receives a...
0x00000000 : roe (17:17) -- This bit is set when the PCI Express...
0x00000000 : uce (16:16) -- This bit is set whenever a completion...
0x00000000 : cae (15:15) -- The bridge sets this bit and logs the...
0x00000001 : cte (14:14) -- For Switch Ports, this bit is set if...
0x00000000 : fce (13:13) -- This bit is set when a flow control...
0x00000000 : ptlpe (12:12) -- This bit is set and the bridge logs...
0x00000000 : slde (05:05) -- This bit is set when a surprise down...
0x00000000 : dlpe (04:04) -- This bit is set when a data link...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcsts.show
0x00000001f : rppiofep (12:08) -- The value of this field identifies...
0x00000000 : dpctre (06:05) -- This field serves as an extension to...
0x00000000 : dpcrbp (04:04) -- When the DPC Trigger Status bit is Set...
0x00000001 : dpcis (03:03) -- This bit is Set if DPC is triggered...
0x00000000 : dpctr (02:01) -- This field indicates why DPC has been...
0x00000001 : dpcts (00:00) -- When Set, this bit indicates that DPC...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x00000000 : labs (15:15) -- This bit is set by hardware to indicate...
0x00000000 : lbms (14:14) -- This bit is set by hardware to indicate...
0x00000000 : dllla (13:13) -- This bit indicates the status of the...
0x00000001 : scc (12:12) -- When to lb, this bit indicates that the...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that...
0x00000000 : undefined (10:10) -- Not applicable
0x00000004 : nlw (09:04) -- This field indicates the negotiated...
0x00000001 : cls (03:00) -- This field indicates the negotiated...
```

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts.show
0x00000000 : aemn (31:27) -- This register indicates which...
0x00000000 : femr (06:06) -- Fatal Error Message Received
0x00000000 : nfemr (05:05) -- Non-Fatal Error message Received
0x00000000 : fuf (04:04) -- First Uncorrectable Fatal
0x00000000 : mefr (03:03) -- Multiple Error Fatal/Non-Fatal Received
0x00000000 : efr (02:02) -- Error Fatal/Non-Fatal Received
0x00000000 : mcer (01:01) -- Multiple Error Correctable Error Received
0x00000001 : cer (00:00) -- Correctable Error Received
```

6. Go/release the system, status registers should clear. Confirm the link is back up and returned to the original speed - the link should train. DEVSTS and ERRUNCSTS are cleared. DPCTS is also now cleared as the system has been released, and LINKSTS shows the link is back up and operating at Gen 4 speeds.

```
>>> go()
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.devsts
0x0

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsts
0x0

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcsts.show
0x0000001f : rppiofep (12:08) -- The value of this field identifies...
0x00000000 : dpctre (06:05) -- This field serves as an extension...
0x00000000 : dpcrpb (04:04) -- When the DPC Trigger Status bit...
0x00000001 : dpcis (03:03) -- This bit is Set if DPC is triggered...
0x00000000 : dpctr (02:01) -- This field indicates why DPC has been...
0x00000000 : dpcts (00:00) -- When Set, this bit indicates that DPC...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x00000001 : labs (15:15) -- This bit is set by hardware to indicate...
0x00000000 : lbms (14:14) -- This bit is set by hardware to indicate...
0x00000001 : dllla (13:13) -- This bit indicates the status of the...
0x00000001 : scc (12:12) -- When to 1b, this bit indicates that the...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that the...
0x00000000 : undefined (10:10) -- Not applicable
0x00000004 : nlw (09:04) -- This field indicates the negotiated...
0x00000004 : cls (03:00) -- This field indicates the negotiated...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts
0x0
```

7. Evaluate the OS Error Logs and confirm the PCIe* error logs are captured/ reported.

```
# OS Error Logs
kernel: pcieport 0000:80:05.0: EDR: EDR event received
kernel: pcieport 0000:80:05.0: DPC: containment event, status:0x1f09 source:0x0000
kernel: pcieport 0000:80:05.0: DPC: unmasked uncorrectable error detected
kernel: pcieport 0000:80:05.0: AER: PCIe Bus Error: severity=Uncorrected (Non-Fatal),
type=Transaction Layer, (Requester ID)
kernel: pcieport 0000:80:05.0: AER: device [8086:352c] error status/mask=00004000/00100020
kernel: pcieport 0000:80:05.0: AER: [14] CmpltTO (First)
kernel: pci 0000:81:00.0: AER: can't recover (no error detected callback)
kernel: pcieport 0000:80:05.0: AER: device recovery successful

# FTrace Output
kworker/0:2-662 [000] .... 364.198017: aer_event: 0000:37:01.0 PCIe Bus Error:
severity=Uncorrected, non-fatal, Completion Timeout, TLP Header=Not available
```

6.7.2 CTO Triggers DPC - OS Native AER

Test Objective

This test uses a completion timeout error injection to trigger enhanced Downstream Port Containment in the OS First environment.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	Generic PCIe* Card
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [PEI Card](#) on page 18
- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > IIO Error Enabling >
 - OS Native AER Support = Enable

Ensure PCIe and IEH hierarchy are cleared and endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Check on the severity level of completion timeout and verify the mask is cleared for a completion timeout. CTO set is set to non-fatal severity with CTES=0. Ensure the link status is performing at the expected speed.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncmsk.show()
0x00000000 : ptlpebm (26:26) -- Poisoned TLP Egress Blocked Mask
0x00000000 : tpbem (25:25) -- TLP Prefix Blocked Error Mask
0x00000000 : aebem (24:24) -- AtomicOp Egress Blocked Mask
0x00000000 : mcem (23:23) -- MC Blocked TLP Mask
0x00000000 : uiem (22:22) -- Uncorrectable Internal Error Mask
0x00000000 : acsem (21:21) -- ACS Violation Mask
0x00000001 : urem (20:20) -- Unsupported Request Error Mask
0x00000000 : ecr cem (19:19) -- ECRC Error Mask
```



```

0x00000000 : mtlpem (18:18) -- Malformed TLP Mask
0x00000000 : roem (17:17) -- Receiver Overflow Mask
0x00000000 : ucem (16:16) -- Unexpected Completion Mask
0x00000000 : caem (15:15) -- Completer Abort Mask
0x00000000 : ctem (14:14) -- Completion Timeout Mask
0x00000000 : fcem (13:13) -- Flow Control Protocol Error Mask
0x00000000 : ptlpem (12:12) -- Poisoned TLP Received Mask
0x00000001 : sldem (05:05) -- This bit does not apply to the...
0x00000000 : dlpeem (04:04) -- Data Link Protocol Error Mask
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsev.show()
0x00000000 : ptlpebs (26:26) -- Poisoned TLP Egress Blocked Severity
0x00000000 : tpbbs (25:25) -- TLP Prefix Blocked Error Severity
0x00000000 : aebbs (24:24) -- AtomicOp Egress Blocked Severity
0x00000000 : mces (23:23) -- MC Blocked TLP Severity
0x00000001 : uies (22:22) -- Uncorrectable Internal Error Severity
0x00000000 : acses (21:21) -- ACS Violation Severity
0x00000000 : ures (20:20) -- Unsupported Request Error Severity
0x00000000 : ecrses (19:19) -- ECRC Error Severity
0x00000001 : mtlpes (18:18) -- Malformed TLP Severity
0x00000001 : roes (17:17) -- Receiver Overflow Error Severity
0x00000000 : uces (16:16) -- Unexpected Completion Error Severity
0x00000000 : caes (15:15) -- Completer Abort Error Severity
0x00000000 : ctes (14:14) -- Completion Timeout Error Severity
0x00000001 : fces (13:13) -- Flow Control Protocol Error Severity
0x00000001 : ptlpes (12:12) -- Poisoned TLP Received Severity
0x00000000 : sldes (05:05) -- This bit does not apply to the...
0x00000001 : dlpes (04:04) -- Data Link Protocol Error Severity
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x00000000 : labs (15:15) -- This bit is set by hardware to indicate...
0x00000001 : lbms (14:14) -- This bit is set by hardware to indicate...
0x00000001 : dllla (13:13) -- This bit indicates the status of the...
0x00000001 : scc (12:12) -- When to lb, this bit indicates that the...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that...
0x00000000 : undefined (10:10) -- Not applicable
0x00000002 : nlw (09:04) -- This field indicates the negotiated...
0x00000003 : cls (03:00) -- This field indicates the negotiated...

```

2. Confirm that DPC is enabled and the uncorrectable error mask is cleared for a completion timeout. DPCTE may not match what is defined in the BIOS as a result of operating in OS Native mode.

```

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcctl.show
0x00000000 : dlaece (07:07) -- This bit must be RW if the DL Active...
0x00000000 : dpcst (06:06) -- This bit must be RW if the DPC Software...
0x00000000 : ptlpebe (05:05) -- This bit must be RW if the Poisoned...
0x00000000 : dpcece (04:04) -- When Set, this bit enables the sending...
0x00000001 : dpcie (03:03) -- When Set, this bit enables the generation...
0x00000000 : dpccc (02:02) -- This bit controls the Completion Status...
0x00000001 : dpcte (01:00) -- This field enables DPC and controls...

```

3. Disable ACS.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.acsctl=0
```

4. Inject the PCIe* completion timeout error using the CScripts.

```

>>> halt()
>>> ei.utils.resetInjectorLockCheck()
>>> ei.iio.injectPcieError(socket=0, port="pxp0.rp2", errType="uce")

```

5. Check the DPC and link status registers and confirm the correct field is triggered. DPCTS is expected to be set to 1 since DPC triggered as a result of the CTO that can be seen in the ERRUNCSTS where the CTE field is set. Device status shows that a non-fatal error was logged, as expected. The system should still be halted

and as a result of DPC triggering, SUT should be operating in a down state. This is demonstrated as the DLLA field is cleared, and the CLS shows platform is operating at Gen 1 speed.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.devsts.show
0x00000000 : tp (05:05) -- When set, this bit indicates that the...
0x00000000 : apd (04:04) -- Auxiliary Power is not supported.
0x00000000 : urd (03:03) -- This bit indicates that this function...
0x00000000 : fed (02:02) -- This bit indicates that this function...
0x00000001 : nfed (01:01) -- This bit indicates that this function...
0x00000000 : ced (00:00) -- This bit indicates that this function...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsts.show
0x00000000 : ptlpeb (26:26) -- Poisoned TLP Egress Blocked Status
0x00000000 : tpbe (25:25) -- TLP Prefix Blocked Error Status
0x00000000 : aebe (24:24) -- This bit is set whenever an Atomic...
0x00000000 : mce (23:23) -- This bit is set whenever a Multicast...
0x00000000 : uie (22:22) -- This bit is set whenever an...
0x00000000 : acse (21:21) -- This bit is set whenever an ACS...
0x00000000 : ure (20:20) -- This bit is set whenever an...
0x00000000 : ecrce (19:19) -- ECRC checking is optional for project...
0x00000000 : mtlpe (18:18) -- This bit is set when it receives a...
0x00000000 : roe (17:17) -- This bit is set when the PCI Express...
0x00000000 : uce (16:16) -- This bit is set whenever a completion...
0x00000000 : cae (15:15) -- The bridge sets this bit and logs the...
0x00000001 : cte (14:14) -- For Switch Ports, this bit is set if...
0x00000000 : fce (13:13) -- This bit is set when a flow control...
0x00000000 : ptlpe (12:12) -- This bit is set and the bridge logs...
0x00000000 : slde (05:05) -- This bit is set when a surprise down...
0x00000000 : dlpe (04:04) -- This bit is set when a data link...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcsts.show
0x0000001f : rppiofep (12:08) -- The value of this field identifies...
0x00000000 : dpctre (06:05) -- This field serves as an extension to...
0x00000000 : dpcrpb (04:04) -- When the DPC Trigger Status bit is Set...
0x00000001 : dpcis (03:03) -- This bit is Set if DPC is triggered...
0x00000000 : dpctr (02:01) -- This field indicates why DPC has been...
0x00000001 : dpcts (00:00) -- When Set, this bit indicates that DPC...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x00000000 : labs (15:15) -- This bit is set by hardware to indicate...
0x00000000 : lbms (14:14) -- This bit is set by hardware to indicate...
0x00000000 : dllla (13:13) -- This bit indicates the status of the...
0x00000001 : scc (12:12) -- When to lb, this bit indicates that the...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that...
0x00000000 : undefined (10:10) -- Not applicable
0x00000002 : nlw (09:04) -- This field indicates the negotiated...
0x00000001 : cls (03:00) -- This field indicates the negotiated...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts.show
0x00000000 : aemn (31:27) -- This register indicates which...
0x00000000 : femr (06:06) -- Fatal Error Message Received
0x00000000 : nfemr (05:05) -- Non-Fatal Error message Received
0x00000000 : fuf (04:04) -- First Uncorrectable Fatal
0x00000000 : mefr (03:03) -- Multiple Error Fatal/Non-Fatal Received
0x00000000 : efr (02:02) -- Error Fatal/Non-Fatal Received
0x00000000 : mcer (01:01) -- Multiple Error Correctable Error Received
0x00000000 : cer (00:00) -- Correctable Error Received
```

6. Go/release the system, status registers should clear. Confirm the link is back up and returned to the original speed - the link should train. DEVSTS and ERRUNCSTS are cleared. DPCTS is also now cleared as the system has been released, and LINKSTS shows the link is back up and operating at Gen 4 speeds.

```
>>> go()
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.devsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsts
0x0
```

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcsts.show
0x00000001f : rppiofep (12:08) -- The value of this field identifies...
0x000000000 : dpctre (06:05) -- This field serves as an extension...
0x000000000 : dpcrpb (04:04) -- When the DPC Trigger Status bit...
0x000000000 : dpcis (03:03) -- This bit is Set if DPC is triggered...
0x000000000 : dpctr (02:01) -- This field indicates why DPC has been...
0x000000000 : dpcts (00:00) -- When Set, this bit indicates that DPC...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x000000001 : labs (15:15) -- This bit is set by hardware to indicate...
0x000000000 : lbms (14:14) -- This bit is set by hardware to indicate...
0x000000001 : dllla (13:13) -- This bit indicates the status of the...
0x000000001 : scc (12:12) -- When to 1b, this bit indicates that the...
0x000000000 : lt (11:11) -- For PCIe device, this bit indicates that the...
0x000000000 : undefined (10:10) -- Not applicable
0x000000002 : nlw (09:04) -- This field indicates the negotiated...
0x000000003 : cls (03:00) -- This field indicates the negotiated...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts
0x0
```

7. Evaluate the OS Error Logs and confirm the PCIe* error logs are captured/ reported.

```
kernel: pcieport 0000:80:05.0: DPC: containment event, status:0x1f01 source:0x0000
kernel: pcieport 0000:80:05.0: DPC: unmasked uncorrectable error detected
kernel: pcieport 0000:80:05.0: AER: PCIe Bus Error: severity=Uncorrected (Non-Fatal),
type=Transaction Layer, (Requester ID)
kernel: pcieport 0000:80:05.0: AER: device [8086:352c] error status/mask=00004000/00000000
kernel: pcieport 0000:80:05.0: AER: [14] CmplttTO (First)
kernel: pci 0000:81:00.0: AER: can't recover (no error_detected callback)
kernel: pcieport 0000:80:05.0: AER: device recovery successful

# cd /sys/kernel/debug/tracing/
# echo 1 > events/ras/aer_event/enable
# cat trace_pipe
kworker/0:2-662 [000] .... 364.198017: aer_event: 0000:37:01.0 PCIe Bus Error:
severity=Uncorrected, non-fatal, Completion Timeout, TLP Header=Not available
```

6.7.3 eDPC Software Trigger - FFM

Test Objective

This test uses the built-in bit in DPCCTL to trigger enhanced Downstream Port Containment through the software trigger bit.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	Generic PCIe* Card
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [PEI Card](#) on page 18
- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > IIO Error Enabling >
 - OS Native AER Support = Disable
 - IIO EDPC Support = On Fatal and Non-Fatal Errors

Ensure the PCIe and IEH hierarchy are cleared and the endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Confirm that DPC is enabled. DPC is enabled for both fatal and non-fatal errors - DPCTE is expected to be set to 2 as a result. Ensure that the root port supports the ability for the software to trigger DPC in the DPCCAP register for the DPCSTS field.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcctl.show
0x00000000 : dlaece (07:07) -- This bit must be RW if the DL_Active...
0x00000000 : dpcst (06:06) -- This bit must be RW if the DPC Software...
0x00000001 : ptlpebe (05:05) -- This bit must be RW if the Poisoned...
0x00000001 : dpcece (04:04) -- When Set, this bit enables the sending...
0x00000001 : dpcie (03:03) -- When Set, this bit enables the generation...
0x00000000 : dpccc (02:02) -- This bit controls the Completion Status...
0x00000002 : dpcte (01:00) -- This field enables DPC and controls...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpccap.show
0x00000001 : dlaecss (12:12) -- If Set, this bit indicates that the Root...
0x00000004 : rppiols (11:08) -- This field indicates how many DWORDs...
0x00000001 : dpcsts (07:07) -- If Set, this bit indicates that a Root...
0x00000001 : ptlpebs (06:06) -- If Set, this bit indicates that the...
0x00000001 : rpedpc (05:05) -- If Set, this bit indicates that a...
0x00000000 : dpcimn (04:00) -- This field indicates which...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x00000000 : labs (15:15) -- This bit is set by hardware to indicate...
0x00000001 : lbms (14:14) -- This bit is set by hardware to indicate...
0x00000001 : dllla (13:13) -- This bit indicates the status of the...
0x00000001 : scc (12:12) -- When to lb, this bit indicates that the...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that...
0x00000000 : undefined (10:10) -- Not applicable
0x00000002 : nlw (09:04) -- This field indicates the negotiated width...
0x00000003 : cls (03:00) -- This field indicates the negotiated link...
```

2. Disable ACS.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.acsctl=0
```

3. Trigger the DPC software trigger with the CScripts. This mechanism supports usage models where the software or firmware examines the RP PIO Exceptions or RP PIO advisory errors, and decides to trigger DPC based upon the situation.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcctl.dpcst=1
```

4. Check the DPCSTS and link status registers and confirm the correct field is triggered. The system should still be halted and as a result of DPC triggering, SUT should be operating in a down state. This is demonstrated as the DLLA field is cleared, and the CLS shows the platform is operating at Gen 1 speed.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcsts.show
0x00000001f : rpiofep (12:08) -- The value of this field identifies...
0x000000001 : dpctre (06:05) -- This field serves as an extension to...
0x000000000 : dpcrpb (04:04) -- When the DPC Trigger Status bit is...
0x000000001 : dpcis (03:03) -- This bit is Set if DPC is triggered...
0x000000003 : dpctr (02:01) -- This field indicates why DPC has been...
0x000000001 : dpcts (00:00) -- When Set, this bit indicates that DPC...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x000000000 : labs (15:15) -- This bit is set by hardware to indicate...
0x000000001 : lbms (14:14) -- This bit is set by hardware to indicate...
0x000000000 : dllla (13:13) -- This bit indicates the status of the...
0x000000001 : scc (12:12) -- When to lb, this bit indicates that the...
0x000000000 : lt (11:11) -- For PCIe device, this bit indicates that...
0x000000000 : undefined (10:10) -- Not applicable
0x000000002 : nlw (09:04) -- This field indicates the negotiated width...
0x000000001 : cls (03:00) -- This field indicates the negotiated link...
```

5. Go/release the system, all status registers listed previously should automatically clear, there should be no need to manually clear these registers. Confirm the link is back up and returned to the original speed - the link should train automatically.

```
>>> go()
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x000000000 : labs (15:15) -- This bit is set by hardware to indicate...
0x000000000 : lbms (14:14) -- This bit is set by hardware to indicate...
0x000000001 : dllla (13:13) -- This bit indicates the status of the...
0x000000001 : scc (12:12) -- When to lb, this bit indicates that...
0x000000000 : lt (11:11) -- For PCIe device, this bit indicates...
0x000000000 : undefined (10:10) -- Not applicable
0x000000004 : nlw (09:04) -- This field indicates the negotiated...
0x000000004 : cls (03:00) -- This field indicates the negotiated...

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcsts.show
0x00000001f : rpiofep (12:08) -- The value of this field identifies...
0x000000001 : dpctre (06:05) -- This field serves as an extension to...
0x000000000 : dpcrpb (04:04) -- When the DPC Trigger Status bit is...
0x000000001 : dpcis (03:03) -- This bit is Set if DPC is triggered...
0x000000003 : dpctr (02:01) -- This field indicates why DPC has been...
0x000000000 : dpcts (00:00) -- When Set, this bit indicates that...
```

6. Evaluate the OS Error Logs and confirm the PCIe* error logs are captured/reported.

```
kernel: pcieport 0000:80:05.0: EDR: EDR event received
kernel: pcieport 0000:80:05.0: DPC: containment event, status:0x1f2f source:0x0000
kernel: pcieport 0000:80:05.0: DPC: software trigger detected
kernel: pci 0000:81:00.0: AER: can't recover (no error_detected callback)
kernel: pcieport 0000:80:05.0: AER: device recovery successful
```

6.8 Leaky Bucket

Ability to detect link marginality during normal operation more efficiently by using HW based "Leaky Bucket" capability. "Leaky Bucket" capability will identify actionable corrected errors - errors on the receiver, that includes receiver errors as well as bad

TLP (LCRC error) or bad DLLP (bad CRC16). These errors will filter out due to link target BER (Bit Error Rate) and manage burst of errors as a single event. Available for Gen1/2/3/4 speeds. Require UEFI-FW to initialize this capability.

Leaky Bucket is a PCIe correctable error filtering mechanism that allows a platform to tolerate bursts of receiver correctable errors and let the errors deplete over time according to the link's acceptable bit error rate. This mechanism is based on a leaky bucket algorithm and allows the firmware to program the acceptable correctable error burst duration as well as the error depletion or leak rate in accordance with the design link's bit error rate target. Once the burst protected and bit error rate adjusted error counts hits a threshold the link survival actions can be requested. If the errors are not prolonged, no action is necessary. If the errors are prolonged, the link's physical layer survival actions such as re-equalization or link rate degradation may be requested to keep the link active. A recovery back to original link speed may also be initiated by the software stack.

6.8.1 PCIe Leaky Bucket

Test Objective

The objective of the recipe is to have Leaky Bucket algorithm handle burst of errors at the receiver as a single event and have enough events accumulate for the leaky bucket error count to match the leaky bucket threshold. This will trigger the link degradation process. From platform perspective, the expectation is to have the link be degraded.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	CentOS Stream release 9
Kernel	6.6.0-gnr.bkc.6.6.10.2.16.x86_64
HW Configuration	Generic PCIe* Card
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [PEI Card](#) on page 18
- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Socket Configuration > IIO Configuration > Global Configuration
 - Equalization Bypass To Highest Rate = Disable
- Platform Configuration > System Event Log > PCIe Error Enabling
 - Expected BER: 34359738367
 - Time Window (Gen1/2): 65535
 - Time Window (Gen2/3): 2
 - Error Threshold (Gen1/2): 0
 - Error Threshold (Gen3/4/5): 16
 - Gen3/4 Re-Equalization: Enable
 - Gen2 Link Degradation: Enable
 - Gen3 Link Degradation: Enable
 - Gen4 Link Degradation: Enable
 - Gen5 Link Degradation: Enable

Ensure PCIe and IEH hierarchy are cleared and endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Check the masks to ensure the error is reported for CEs at the OS, specifically that the bad DLLP is cleared.

```
>>> halt()
>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.errcormsk
0x0
```

2. Review the leaky bucket registers. LEKBLNERRCNT<x> will depend on where your PEI card is located.

```
>>>
>>> ras.iio.check_config_leakybucket(0, 'pxpl.rp0')

*****
Leaky Bucket Actions (LEKBAR)
*****
=====
| Socket | Instance | Bits | Field | Description | Value |
=====
| socket0 | pxpl.rp0 | 31:31 | lbeg5dis | When set to 1, 32GT/s and higher rates are disabled | 0x0 |
| socket0 | pxpl.rp0 | 30:30 | lbeg4dis | When set to 1, 16GT/s and higher rates are disabled | 0x0 |
| socket0 | pxpl.rp0 | 29:29 | lbeg3dis | When set to 1, 8GT/s and higher rates are disabled | 0x0 |
| socket0 | pxpl.rp0 | 28:28 | lbeg2dis | When set to 1, 5GT/s and higher rates are disabled | 0x0 |
| socket0 | pxpl.rp0 | 27:27 | lbeg5degradeen | Enable LBE to trigger a speed degrade from 32GT/s operation | 0x1 |
| socket0 | pxpl.rp0 | 26:26 | lbeg4degradeen | Enable LBE to trigger a speed degrade from 16GT/s operation | 0x1 |
| socket0 | pxpl.rp0 | 25:25 | lbeg3degradeen | Enable LBE to trigger a speed degrade from 8GT/s operation | 0x1 |
| socket0 | pxpl.rp0 | 24:24 | lbeg2degradeen | Enable LBE to trigger a speed degrade from 5GT/s operation | 0x1 |
=====

*****
Leaky Bucket Error (LEKBERR)
*****
=====
| Socket | Instance | Bits | Field | Description | Value |
=====
| socket0 | pxpl.rp0 | 31:31 | g3lbeen | Enable Leaky Bucket Error Counters for Gen3 | 0x1 |
| socket0 | pxpl.rp0 | 29:25 | g3errthresh | Leaky Bucket error threshold used for 8GT/s and above | 0x10 |
| socket0 | pxpl.rp0 | 24:20 | errthresh | Sets the threshold for leaky bucket error handling | 0x0 |
=====
```

socket0	pxpl.rp0	19:0	aggrerr	Error count for the leaky bucket algorithm increments by one when AGGRERR number of errors have been encountered during the specified time period	0xffff
---------	----------	------	---------	---	--------

Leaky Bucket Time Window (LEKBERR0)

Socket	Instance	Bits	Field	Description	Value
socket0	pxpl.rp0	31:0	exp_ber	Expected BER for leaky bucket error handling. Selects the time window that is sampled for aggregate errors	0xffffffff

Leaky Bucket Time Window (LEKBERR1)

Socket	Instance	Bits	Field	Description	Value
socket0	pxpl.rp0	31:24	g3aggrerr	Aggregate error for Gen3	0x2
socket0	pxpl.rp0	17:0	exp_ber	Expected BER for leaky bucket error handling. Selects the time window that is sampled for aggregate errors	0x7

Leaky Bucket Lane Error Count (LEKBLNERRCNT0)

Socket	Instance	Bits	Field	Description	Value
socket0	pxpl.rp0	29:25	errcnt5	Error count seen on Lane 5 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	24:20	errcnt4	Error count seen on Lane 4 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	19:15	errcnt3	Error count seen on Lane 3 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	14:10	errcnt2	Error count seen on Lane 2 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	9:5	errcnt1	Error count seen on Lane 1 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	4:0	errcnt0	Error count seen on Lane 0 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0

Leaky Bucket Lane Error Count (LEKBLNERRCNT1)

Socket	Instance	Bits	Field	Description	Value
socket0	pxpl.rp0	29:25	errcnt11	Error count seen on Lane 11 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	24:20	errcnt10	Error count seen on Lane 10 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	19:15	errcnt9	Error count seen on Lane 9 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	14:10	errcnt8	Error count seen on Lane 8 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	9:5	errcnt7	Error count seen on Lane 7 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	4:0	errcnt6	Error count seen on Lane 6 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0

Leaky Bucket Lane Error Count (LEKBLNERRCNT2)

Socket	Instance	Bits	Field	Description	Value
socket0	pxpl.rp0	19:15	errcnt15	Error count seen on Lane 15 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	14:10	errcnt14	Error count seen on Lane 14 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	9:5	errcnt13	Error count seen on Lane 13 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0
socket0	pxpl.rp0	4:0	errcnt12	Error count seen on Lane 12 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x0

Leaky Bucket Prolonged Error (LEKBPROERR)

Socket	Instance	Bits	Field	Description	Value
socket0	pxpl.rp0	31:16	g3errlnsts	Feature to check status of lane for Gen3	0x0
socket0	pxpl.rp0	15:0	errlnsts	Feature to check status of lane	0x0

Correctable Error Mask (ERRCORMSK)

Socket	Instance	Bits	Register Field	Value	Description
socket0	pxpl.rp0	15:15	hloem	Header Log Overflow Error Mask	0x0
socket0	pxpl.rp0	14:14	ciem	Correctable Internal Error Mask	0x0
socket0	pxpl.rp0	13:13	anfem	Advisory Non-Fatal Error Mask	0x0
socket0	pxpl.rp0	12:12	rttem	Replay Timer Timeout Error Mask	0x0
socket0	pxpl.rp0	8:8	rnrem	Replay Number Rollover Error Mask	0x0
socket0	pxpl.rp0	7:7	bdllpem	Bad DLLP Error Mask	0x0
socket0	pxpl.rp0	6:6	btlpem	Bad TLP Error Mask	0x0
socket0	pxpl.rp0	0:0	rem	Receiver Error Mask	0x0

Correctable Error Counter Mask (CORERRCNTMSK)

Socket	Instance	Bits	Register Field	Value	Description
socket0	pxpl.rp0	15:15	hloem	Header Log Overflow Error Counter Mask	0x1
socket0	pxpl.rp0	14:14	ciem	Correctable Internal Error Counter Mask	0x1
socket0	pxpl.rp0	13:13	anfem	Link Advisory Non-Fatal Error Counter Mask	0x1
socket0	pxpl.rp0	12:12	rttem	Link Replay Timeout Error Counter Mask	0x1
socket0	pxpl.rp0	8:8	rnrem	Link Replay Number Rollover Error Counter Mask	0x1
socket0	pxpl.rp0	7:7	bdllpem	Link Bad DLLP Error Counter Mask	0x1
socket0	pxpl.rp0	6:6	btlpem	Link Bad TLP Error Counter Mask	0x1
socket0	pxpl.rp0	0:0	rem	Link Receiver Error Counter Mask	0x1

Correctable Error Threshold (CORERRTH)

Socket	Instance	Bits	Register Field	Value	Description
socket0	pxpl.rp0	31:0	errth	Link correctable error threshold	0x0

```

>>> sv.socket0.io0.uncore.pi5.pxl.rp0.cfg.g4sts.showfields()
0x00000000 : linkeqreq16 (04:04)
0x00000001 : eql6ph3succ (03:03)
0x00000001 : eql6ph2succ (02:02)
0x00000001 : eql6ph1succ (01:01)
0x00000001 : eql6cmplt (00:00)
>>> sv.socket0.io0.uncore.pi5.pxl.rp0.cfg.linksts.showfields()
0x00000000 : labs (15:15)
0x00000000 : lbms (14:14)
0x00000001 : dlla (13:13)
0x00000001 : scc (12:12)
0x00000000 : lt (11:11)
0x00000000 : undefined (10:10)
0x00000010 : nlw (09:04)
0x00000005 : cls (03:00)

```

3. Error injection using the PEI 4.0 card.

```

C:\Program Files\PEI\PCIE>python pei_run.py -t BAD_DLLP_crc -c 75
Starting BAD_DLLP_crc()
INFO: Starting PEI.run() on Endpoint image
INFO: Trigger disarmed after 2.0sec.
Count INJ_US: 75

```

- Re-evaluate the PCIe status registers. The default threshold was set to 16 so the leaky bucket error count should have been greater than or equal to the threshold. This will result in a prolonged error. The LEKNBLNERRCNT shows a value greater than 0x10 which will trigger the G3ERRLNSTS in the LEKBPROERR register since LEKBLNERRCNT > G3ERRTHRESH. The link degradation process will begin and LEB4DIS should be set. G4STS will show the link re-equalization request. LINKSTS should show the link operating at the Gen 3 speed, and LBMS should be set showing the hardware has changed the link speed.

```

>>> ras.iio.check_config_leakybucket(0, 'pxl.rp0')

*****
Leaky Bucket Actions (LEKBAR)
*****
=====
| Socket | Instance | Bits | Field | Description | Value |
=====
| socket0 | pxl.rp0 | 31:31 | lbeg5dis | When set to 1, 32GT/s and higher rates are disabled | 0x1 |
| socket0 | pxl.rp0 | 30:30 | lbeg4dis | When set to 1, 16GT/s and higher rates are disabled | 0x0 |
| socket0 | pxl.rp0 | 29:29 | lbeg3dis | When set to 1, 8GT/s and higher rates are disabled | 0x0 |
| socket0 | pxl.rp0 | 28:28 | lbeg2dis | When set to 1, 5GT/s and higher rates are disabled | 0x0 |
| socket0 | pxl.rp0 | 27:27 | lbeg5degradeen | Enable LBE to trigger a speed degrade from 32GT/s operation | 0x1 |
| socket0 | pxl.rp0 | 26:26 | lbeg4degradeen | Enable LBE to trigger a speed degrade from 16GT/s operation | 0x1 |
| socket0 | pxl.rp0 | 25:25 | lbeg3degradeen | Enable LBE to trigger a speed degrade from 8GT/s operation | 0x1 |
| socket0 | pxl.rp0 | 24:24 | lbeg2degradeen | Enable LBE to trigger a speed degrade from 5GT/s operation | 0x1 |
=====

*****
Leaky Bucket Error (LEKBERR)
*****
=====
| Socket | Instance | Bits | Field | Description | Value |
=====
| socket0 | pxl.rp0 | 31:31 | g3lbeen | Enable Leaky Bucket Error Counters for Gen3 | 0x1 |
| socket0 | pxl.rp0 | 29:25 | g3errthresh | Leaky Bucket error threshold used for 8GT/s and above | 0x10 |
| socket0 | pxl.rp0 | 24:20 | errthresh | Sets the threshold for leaky bucket error handling | 0x0 |
| socket0 | pxl.rp0 | 19:0 | aggrerr | Error count for the leaky bucket algorithm increments by one | 0xffff |
| | | | | when AGGRERR number of errors have been encountered | |
| | | | | during the specified time period | |
=====

*****
Leaky Bucket Time Window (LEKBERR0)
*****
=====
| Socket | Instance | Bits | Field | Description | Value |
=====

```

socket0	pxpl.rp0	31:0	exp_ber	Expected BER for leaky bucket error handling.	0xffffffff	
				Selects the time window that is sampled for aggregate errors		

Leaky Bucket Time Window (LEKBERR1)

Socket	Instance	Bits	Field	Description	Value	
socket0	pxpl.rp0	31:24	g3aggrerr	Aggregate error for Gen3	0x2	
socket0	pxpl.rp0	17:0	exp_ber	Expected BER for leaky bucket error handling.	0x7	
				Selects the time window that is sampled for aggregate errors		

Leaky Bucket Lane Error Count (LEKBLNERRCNT0)

Socket	Instance	Bits	Field	Description	Value	
socket0	pxpl.rp0	29:25	errcnt5	Error count seen on Lane 5 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	
socket0	pxpl.rp0	24:20	errcnt4	Error count seen on Lane 4 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	
socket0	pxpl.rp0	19:15	errcnt3	Error count seen on Lane 3 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	
socket0	pxpl.rp0	14:10	errcnt2	Error count seen on Lane 2 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	
socket0	pxpl.rp0	9:5	errcnt1	Error count seen on Lane 1 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	
socket0	pxpl.rp0	4:0	errcnt0	Error count seen on Lane 0 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	

Leaky Bucket Lane Error Count (LEKBLNERRCNT1)

Socket	Instance	Bits	Field	Description	Value	
socket0	pxpl.rp0	29:25	errcnt11	Error count seen on Lane 11 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	
socket0	pxpl.rp0	24:20	errcnt10	Error count seen on Lane 10 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	
socket0	pxpl.rp0	19:15	errcnt9	Error count seen on Lane 9 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	
socket0	pxpl.rp0	14:10	errcnt8	Error count seen on Lane 8 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	
socket0	pxpl.rp0	9:5	errcnt7	Error count seen on Lane 7 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	
socket0	pxpl.rp0	4:0	errcnt6	Error count seen on Lane 6 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	

Leaky Bucket Lane Error Count (LEKBLNERRCNT2)

Socket	Instance	Bits	Field	Description	Value	
socket0	pxpl.rp0	19:15	errcnt15	Error count seen on Lane 15 for the current time window. Decrements progressively to 0 if no errors seen over a prolonged time period	0x1e	
socket0	pxpl.rp0	14:10	errcnt14	Error count seen on Lane 14 for the current time window.	0x1e	

					Decrements progressively to 0 if no errors seen over a prolonged time period		
socket0	pxpl.rp0	9:5	errcnt13	Error count seen on Lane 13 for the current time window.	0x1e		
				Decrements progressively to 0 if no errors seen over a prolonged time period			
socket0	pxpl.rp0	4:0	errcnt12	Error count seen on Lane 12 for the current time window.	0x1e		
				Decrements progressively to 0 if no errors seen over a prolonged time period			

Leaky Bucket Prolonged Error (LEKBPROERR)							

Socket	Instance	Bits	Field	Description	Value		
socket0	pxpl.rp0	31:16	g3errlnsts	Feature to check status of lane for Gen3	0xffff		
socket0	pxpl.rp0	15:0	errlnsts	Feature to check status of lane	0x0		

Correctable Error Mask (ERRCORMSK)							

Socket	Instance	Bits	Register Field	Value	Description		
socket0	pxpl.rp0	15:15	hloem	Header Log Overflow Error Mask	0x0		
socket0	pxpl.rp0	14:14	ciem	Correctable Internal Error Mask	0x0		
socket0	pxpl.rp0	13:13	anfer	Advisory Non-Fatal Error Mask	0x0		
socket0	pxpl.rp0	12:12	rttem	Replay Timer Timeout Error Mask	0x0		
socket0	pxpl.rp0	8:8	rnrem	Replay Number Rollover Error Mask	0x0		
socket0	pxpl.rp0	7:7	bdllpem	Bad DLLP Error Mask	0x0		
socket0	pxpl.rp0	6:6	btlpem	Bad TLP Error Mask	0x0		
socket0	pxpl.rp0	0:0	rem	Receiver Error Mask	0x0		

Correctable Error Counter Mask (CORERRCNTMSK)							

Socket	Instance	Bits	Register Field	Value	Description		
socket0	pxpl.rp0	15:15	hloem	Header Log Overflow Error Counter Mask	0x1		
socket0	pxpl.rp0	14:14	ciem	Correctable Internal Error Counter Mask	0x1		
socket0	pxpl.rp0	13:13	anfer	Link Advisory Non-Fatal Error Counter Mask	0x1		
socket0	pxpl.rp0	12:12	rttem	Link Replay Timeout Error Counter Mask	0x1		
socket0	pxpl.rp0	8:8	rnrem	Link Replay Number Rollover Error Counter Mask	0x1		
socket0	pxpl.rp0	7:7	bdllpem	Link Bad DLLP Error Counter Mask	0x1		
socket0	pxpl.rp0	6:6	btlpem	Link Bad TLP Error Counter Mask	0x1		
socket0	pxpl.rp0	0:0	rem	Link Receiver Error Counter Mask	0x1		

Correctable Error Threshold (CORERRRTH)							

Socket	Instance	Bits	Register Field	Value	Description		
socket0	pxpl.rp0	31:0	errth	Link correctable error threshold	0x0		

- Go/release the system. The leaky bucket error counter should decrement at the defined BER. The LEKBPROERR register should clear for the corresponding lane. The link equalization request bit should be set. Confirm the PCIe generation speed has downgraded.

```
>>> go()
>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.lekbproerr.showfields()
0x00000000 : g3errlnsts (31:16)
0x00000000 : errlnsts (15:00)

>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.g4sts.showfields()
0x00000001 : linkeqreq16 (04:04)
0x00000001 : eq16ph3succ (03:03)
0x00000001 : eq16ph2succ (02:02)
0x00000001 : eq16ph1succ (01:01)
0x00000001 : eq16cmplt (00:00)

>>> sv.socket0.io0.uncore.pi5.pxpl.rp0.cfg.linksts.showfields()
0x00000000 : labs (15:15)
0x00000001 : lbms (14:14)
0x00000001 : dllla (13:13)
0x00000001 : scc (12:12)
0x00000000 : lt (11:11)
0x00000000 : undefined (10:10)
0x00000010 : nlw (09:04)
0x00000004 : cls (03:00)
```

6.9 Intel® Data Streaming Accelerator (Intel® DSA)

The processor supports Intel® Data Streaming Accelerator (Intel® DSA), Version 2.0 through a data accelerator for improving the performance of storage, networking, and various other I/O applications. The DMA engine is optimized for moving data between memory. It is not backwards compatible with previous versions, but it provides the same operations.

The Intel® DSA replaces the Intel® QuickData Technology used in previous server processor generations. For more details, see the [Intel® Data Streaming Accelerator Specification](#).

6.9.1 Intel® DSA Error Injection (Advisory Non-Fatal)

The Intel® DSA is a Root Complex Integrated End Point (RCiEP) device. RCiEP devices report directly to RCEC. Errors are then reported to the satellite IEH's global Error logic. In addition, errors from RP, RCEC, and other SAT IEH(s) also report to the global Error logic of satellite IEH. After these errors are logged in the global Error logic, they are passed to the global IEH's global Error logic for signaling.

Test Objective

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Configuration

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A1
BIOS	WW05'24 BKC
OS Version	Cent OS Stream 9
<i>continued...</i>	

Kernel	6.6.0-gnr.bkc.6.6.0.44.3.6.x86_64
HW Configuration	Generic PCIe Card
CScripts Version	BHS.2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > PCIe Error Enabling > PCIe Corrected Error Threshold Counter = Disable

Ensure the PCIe and IEH hierarchy are cleared and the endpoints are properly configured. See the [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Halt the system and check to make sure DEVCTL is set to allow UR messages to send, and that ERRUNCSTS and DEVSTS are clear. Also, check that IEH is cleared.

```
>>> halt()
>>> sv.socket0.io0.uncore.dsa.dsa0.erruncsts.show
0x00000000 : ui (22:22) --
0x00000000 : ur (20:20) --
0x00000000 : uc (16:16) --
0x00000000 : ca (15:15) --
0x00000000 : cto (14:14) --
0x00000000 : ptlp (12:12) --

>>> sv.socket0.io0.uncore.dsa.dsa0.devctl.show()
0x00000000 : iflr (15:15) --
0x00000005 : mrrs (14:12) --
0x00000001 : ens (11:11) --
0x00000001 : etfe (08:08) --
0x00000002 : mps (07:05) --
0x00000001 : ero (04:04) --
0x00000001 : urre (03:03) --
0x00000001 : fere (02:02) --
0x00000001 : nere (01:01) --
0x00000001 : cere (00:00) --

>>> sv.socket0.io0.uncore.dsa.dsa0.devsts.show
0x00000000 : tp (05:05) --
0x00000000 : urd (03:03) --
0x00000000 : ned (01:01) --
0x00000000 : ced (00:00) --

>>> error.check_ieh_errors()
Checking errors in the following IPs:
  ieh_0_io0
  ieh_1_io0
  ieh_2_io0
  ieh_3_io0
  ieh_5_io0
  ieh_6_io0
  ieh_7_io0
  ieh_0_io1
```

```

ieh_1_io1
ieh_2_io1
ieh_3_io1
ieh_4_io1
ieh_5_io1
ieh_6_io1
ieh_7_io1

Please be patient...this operation might take several minutes.
\
*****
                        No errors found
*****

```

2. Enable FTrace.

```

[root@gnr-bkc /]# cd sys/kernel/debug/tracing/
[root@gnr-bkc tracing]# echo 1 > events/mce/mce_record/enable
[root@gnr-bkc tracing]# echo 1 > events/ras/aer_event/enable
[root@gnr-bkc tracing]# cat trace_pipe

```

3. Inject an Intel DSA correctable error.

```

>>> ei.iio.injectDSACorrectableError(0,0,0,err_type='correctable',bar=0)
Obtained BAR value: 33054061494272
Error successfully injected to DSA!, Status Value: 0x2000

```

4. Verify the injection by checking Intel DSA DEVSTS and ERRUNCSTS and check IEH errors.

```

>>> sv.socket0.io0.uncore.dsa.dsa0.devsts.show
0x00000000 : tp (05:05) -- When Set, this bit indicates that the Function has issued ...
0x00000001 : urd (03:03) -- This bit indicates that the Function received an Unsupported ...
0x00000000 : fed (02:02) -- This bit indicates status of Fatal errors detected. Errors are ...
0x00000000 : ned (01:01) -- This bit indicates status of Non-fatal errors detected. Errors ...
0x00000001 : ced (00:00) -- This bit indicates status of correctable errors detected.

>>> sv.socket0.io0.uncore.dsa.dsa0.erruncsts.show
0x00000000 : ui (22:22) -- Set when an Uncorrectable Internal error is detected
0x00000001 : ur (20:20) -- Set when this function sends a completion with Unsupported ...
0x00000000 : uc (16:16) -- Set when this function receives a completion that does not ...
0x00000000 : ca (15:15) -- Set when this function sends a completion with Completer Abort ...
0x00000000 : cto (14:14) -- Set when a Non-posted requested by this function is terminated ...
0x00000000 : ptlp (12:12) -- Set when a TLP received by this function is marked as poisoned

>>> sv.socket0.io0.uncore.dsa.dsa0.errcorsts.show
0x00000000 : ci (14:14) -- Set when a Corrected Internal error is detected
0x00000001 : anf (13:13) -- Set when an Error classified as Advisory Non-fatal is detected

# Check Sat IEH ROOTERRSTS and ERRSRCID
>>> sv.socket0.io0.uncore.satieh.satieh1.rooterrsts.show()
0x00000000 : aemn (31:27) --
0x00000000 : reserved_copy (26:07) --
0x00000000 : femr (06:06) --
0x00000000 : nfemr (05:05) --
0x00000000 : fuf (04:04) --
0x00000000 : mefr (03:03) --
0x00000000 : efr (02:02) --
0x00000000 : mcer (01:01) --
0x00000001 : cer (00:00) --

>>> sv.socket0.io0.uncore.satieh.satieh1.errsrcid
0x8

>>> error.check_ieh_errors()
=====
IEH_1_IO0: REGISTERS AND FIELDS DESCRIPTIONS
=====

```

```

Register: socket0.io0.uncore.satieh.satieh1.gerrcorsts
  iehlocal_co_sts = 0x1
    IEH Local Error Status (DEV0_CO_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.satieh.satieh1.gerrcortime
0x000000008f0887b2 : g_coferr_time (63:00) -- Global COFERR Time Stamp (G_COFERR_TIME)

=> socket0.io0.uncore.satieh.satieh1.gtime
0x000000008f089d14 : gtime_value (63:00) -- Global Error Timer value (GTIME_Value)

-----
Register: socket0.io0.uncore.satieh.satieh1.gsysevtsts
  co_sysev_sts = 0x1
    Correctable Error Status - A correctable error was detected when set (CO_SYSEV_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.satieh.satieh1.gsysevtctl
  0x000000001 : fa_sysev_m (02:02) -- Fatal Error Mask(FA_SYSEV_M) Fatal Error System ...
  0x000000001 : nf_sysev_m (01:01) -- Nonfatal Error Mask(NF_SYSEV_M) NonFatal Error ...
  0x000000001 : co_sysev_m (00:00) -- Correctable Error Mask(CO_SYSEV_M) Correctable ...

=====
                IEH GLOBAL: REGISTERS AND FIELDS DESCRIPTIONS
=====
Register: socket0.io0.uncore.globalieh.iehregs.gerrcorsts
  dev3_co_sts = 0x1
    Dev3 Error Status (DEV3_CO_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.globalieh.iehregs.gerrcortime
0x00000006b8a3abbfd : g_coferr_time (63:00) -- Global COFERR Time Stamp (G_COFERR_TIME)

=> socket0.io0.uncore.globalieh.iehregs.gtime
0x00000007cac813alf : gtime_value (63:00) -- Global Error Timer value (GTIME_Value)

-----
Register: socket0.io0.uncore.globalieh.iehregs.gsysevtsts
  Correctable Error Status - A correctable error was detected when set (CO_SYSEV_STS)

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.globalieh.iehregs.gsysevtctl
  0x000000001 : fa_sysev_m (02:02) -- Fatal Error Mask(FA_SYSEV_M) Fatal Error System ...
  0x000000001 : nf_sysev_m (01:01) -- Nonfatal Error Mask(NF_SYSEV_M) NonFatal Error ...
  0x000000001 : co_sysev_m (00:00) -- Correctable Error Mask(CO_SYSEV_M) Correctable ...

=> socket0.io0.uncore.globalieh.iehregs.gsysevmap
  0x000000001 : co_sysev_map (01:00) -- 11: Reserved 10: Generate NMI 01 Generate SMI ...

-----
Register: socket0.io0.uncore.globalieh.iehregs.gerrsrcid
  geccsid = 0x88
    ERR_COR Source Identification (ECSID) Requester ID of the source when a correctable ...

```

5. Check RCEC for Root Error Status and Error Source ID.

```

>>> pci.decode(0,0,4)
...
0x130: Root Error Status Register = 0x0001
[31:27] Advanced Error Interrupt Message Number = 0x00
[6] Fatal Error Messages Received = 0x0
[5] Non-Fatal Error Messages Received = 0x0
[4] First Uncorrectable Fatal = 0x0
[3] Multiple ERR_FATAL/NONFATAL Received = 0x0
[2] ERR_FATAL/NONFATAL Received = 0x0
[1] Multiple ERR_COR Received = 0x0
[0] ERR_COR Received = 0x1

# Verify that the Error Source ID (offset 0x134 in RCEC) matches the Bus # of

```



```

DSA0
>>> ei.iio.showIehMapping()
----- io0 Mapping -----
=====
| pxp/dsa# |      ieh#      |  hiop#  | bus# |
|-----|
|   pxp0   | satieh3 (PE0) |   hiop3  | 0x2 |
|   pxp1   | satieh4 (PE1) |   hiop4  | 0x3 |
|   pxp2   | satieh2 (PE2) |   hiop2  | 0x1 |
|   pxp3   | satieh5 (PE3) |   hiop5  | 0x4 |
|   dsa0   | satieh1 (HC0) | unknown | 0x8 |
|   dsa1   | satieh6 (HC1) |   unknown | 0xc |
|-----|

----- io1 Mapping -----
=====
| pxp/dsa# |      ieh#      |  hiop#  | bus# |
|-----|
|   pxp4   | satieh4 (PE4) |   hiop4  | 0x1d |
|   pxp5   | satieh3 (PE5) |   hiop3  | 0x1a |
|   dsa0   | satieh1 (HC0) |   unknown | 0x10 |
|   dsa1   | satieh6 (HC1) |   unknown | 0x14 |
|-----|

>>> pci.config(0,0,4,0x134)
0x8

```

6. Go/release the system and check FTrace / OS Serial Logs.

```

-----
OS Serial
-----
[IEH] error found at IEH(S:0x0 B:0x7E D:0x2 F:0x0) Sev: IEH CORRECT ERROR
[IEH] ErrorStatus 0x8, MaxBitIdx 0x1D
IEH CORRECT ERROR
[IEH] BitIdx 0x3, ShareIdx 0x0
[IEH] error device is (S:0x0 B:0x0 D:0x0 F:0x4) BitIdx 0x3, ShareIdx 0x0
[IEH] error found at IEH(S:0x0 B:0x0 D:0x0 F:0x4) Sev: IEH CORRECT ERROR
[IEH] ErrorStatus 0x1, MaxBitIdx 0x1
IEH CORRECT ERROR
IehRciepDevicesErrorHandler - Socket 0, Bus 0, device 0, function 4
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AE04210
WHEA: Detected PCIe Error
--Logging Corrected Error to WHEA
WHEA: Sending OS notification via SCI. Success

-----
FTrace
-----
[ 958.099131] idxd 0000:00:01.0: AER: aer_status: 0x00002000, aer_mask:
0x00000000
[ 958.108002] idxd 0000:00:01.0: AER: aer_layer=Transaction Layer,
aer_agent=Receiver ID
[ 958.740732] aer_event: 0000:00:01.0 PCIe Bus Error: severity=Corrected,
Advisory Non-Fatal Error, TLP Header=Not available

```

7. Ensure corresponding registers are cleared.

6.10 Runtime S3M Error Reporting

Feature Description

Within the S3M, interfaces are identifiable as PCI/PCIe devices. These devices utilize uncorrectable Advanced Error Reporting (AER) and System Error (SERR) messages for error reporting, which are conveyed via the Integrated Error Handler (IEH) reporting pathway.

6.10.1 Uncorrectable AER

Test Objective

The objective of this test is to observe the uncorrectable AER path from S3M.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores A step
BIOS	WW06'24 BKC
OS Version	CentOS Stream 8
Kernel	5.19.0-gnr.po.bkc.7.2.21.x86_64
HW Configuration	No particular DIMM, PCIe card, or hardware tool configuration required.
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

Recipe

1. Halt, clear any existing errors, and verify that IEH error logs were all cleared.

```
>>> halt()
>>> error.clearAllErrors()
>>> halt()
>>> error.check_ieh_errors()
```

2. Determine which satellite IEH S3M reports its AER/DO_SERR errors to. S3M and its SIEH will always be on the same bus and this external/enumerated bus will always be 0. Use `ei.iio.showIehMapping()` to view the internal bus number for

each SIEH. Then use `sv.socket0.target_info["pci_bus_map"]` to find which internal bus number maps to external bus 0; the SIEH on this bus will be the SIEH to check/modify in this recipe.

```
>>> ei.iio.showIehMapping()
>>> sv.socket0.target_info["pci_bus_map"]
```

3. Pre-injection configuration of relevant error signaling control and masks bits along the signaling path. Feel free to skip these steps if you want to test your systems default configuration (may end up not seeing the error if you have this error disabled). SIEH1 and LERR*1 are modified because this is the SIEH (depends on SKU) and local error source that S3M reports to.

```
>>> access = sv.socket0.getaccess()
>>> sv.socket0.setaccess("pcicfg")

# Configure and verify all relevant control/mask registers in SIEH's local error logic.
>>> sv.socket0.io0.uncore.satieh.satieh1.lerruncmsk1 = 0
>>> sv.socket0.io0.uncore.satieh.satieh1.lerruncmsk1 # Should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.lerrcormsk1 = 0
>>> sv.socket0.io0.uncore.satieh.satieh1.lerrcormsk1 # Should be 0

# Configure and verify all relevant control/mask registers in SIEH's RCEC.
>>> sv.socket0.io0.uncore.satieh.satieh1.errcormsk = 0
>>> sv.socket0.io0.uncore.satieh.satieh1.errcormsk # Should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.devctl = 0x7
>>> sv.socket0.io0.uncore.satieh.satieh1.devctl # Should be 0x7
>>> sv.socket0.io0.uncore.satieh.satieh1.rooterrcmd = 0x0 # Assuming firmware first mode
>>> sv.socket0.io0.uncore.satieh.satieh1.rooterrcmd # Should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.rootctl = 0x7 # Assuming firmware first mode
>>> sv.socket0.io0.uncore.satieh.satieh1.rootctl # Should be 0x7

# Configure and verify all relevant control/mask registers in SIEH's global error logic.
>>> sv.socket0.io0.uncore.satieh.satieh1.gerrcormsk = 0
>>> sv.socket0.io0.uncore.satieh.satieh1.gerrcormsk # Should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.gsysevtctl = 0x7
>>> sv.socket0.io0.uncore.satieh.satieh1.gsysevtctl # Should be 0x7

# Configure and verify all relevant control/mask registers in GIEH's global error logic.
>>> sv.socket0.io0.uncore.globalieh.iehregs.gerrcormsk = 0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gerrcormsk # Should be 0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtctl = 0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtctl # Should be 0x7

# Depending on configuration of GSYSEVTMAP and ERRPINCTRL there may be an SMI, NMI, or
ERROR[0] pin signaling. So check value of these registers to confirm signaling configuration.
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtmap.show()
>>> sv.socket0.io0.uncore.globalieh.iehregs.errpinctrl.show()
```

4. Pre-injection clearing of any relevant error status registers along the signaling path and verifying of contents.

```
# Clear and verify all relevant status registers in SIEH's local error logic.
>>> sv.socket0.io0.uncore.satieh.satieh1.lerrcorsts1 = -1
>>> sv.socket0.io0.uncore.satieh.satieh1.lerrcorsts1 # should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.lerruncsts1 = -1
>>> sv.socket0.io0.uncore.satieh.satieh1.lerruncsts1 # should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerruncfep # should be 0, if not then there are
some LERRUNCSTS# registers that should also be cleared
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerrcorcnt = 0
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerrcorcnt # should be 0

>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerrhdrlog1 # Can't be cleared by writes, ignore
any existing data at this point
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerrhdrlog2 # Can't be cleared by writes, ignore
any existing data at this point
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerrhdrlog3 # Can't be cleared by writes, ignore
any existing data at this point
```

```
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerrhdrlog4 # Can't be cleared by writes, ignore
any existing data at this point

# Clear and verify all relevant status registers in SIEH's RCEC.
>>> sv.socket0.io0.uncore.satieh.satieh1.devsts = -1
>>> sv.socket0.io0.uncore.satieh.satieh1.devsts # should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.errcorsts = -1
>>> sv.socket0.io0.uncore.satieh.satieh1.errcorsts # should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.rooterrsts = -1
>>> sv.socket0.io0.uncore.satieh.satieh1.rooterrsts # should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.errsrcid # Can't be cleared by writes, ignore any
existing data at this point

# Clear and verify all relevant status registers in SIEH's global error logic. GSYSEVTSTS
should be zero since the system was rebooted just before testing.
>>> sv.socket0.io0.uncore.satieh.satieh1.gerrcorsts = -1
>>> sv.socket0.io0.uncore.satieh.satieh1.gerrcorsts # should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.gsysevtsts # should be 0

# Clear and verify all relevant status registers in GIEH. GSYSEVTSTS should be zero since the
system was rebooted just before testing.
>>> sv.socket0.io0.uncore.globalieh.iehregs.gerrcorsts = -1
>>> sv.socket0.io0.uncore.globalieh.iehregs.gerrcorsts # should be 0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtsts # should be 0

# Set access method back to default.
>>> sv.socket0.setaccess(access)
```

5. Perform a PCI config read to a non-existent device on bus 0. S3M is the subtractive decoder and will report the unsupported request.

```
>>> pci.config(0x0, 0x1f, 0x7, 0x0)
```

6. Post-injection verification of relevant error status registers along the signaling path.

```
# Check all status registers in SIEH's local error logic are as expected.
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerruncfep
0x34
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerruncfep.show() # Will point to the LERRUNCSTS#
instance that was logged and which bit in it was logged 1st. Instance number is strapping
based but first error source should be 1 and since this is a UR injection the first error
point should be 0x14 (bit 20 is UR).
0x00000000 : reserved (15:11) -- Reserved
0x00000001 : fes (10:05) -- First Error Source: First error among 8 possible sources
0x00000014 : fep (04:00) -- First Error Pointer: Points to the particular uncorrectable error
type

>>> sv.socket0.io0.uncore.satieh.satieh1.lerrcorsts1
0x2000
>>> sv.socket0.io0.uncore.satieh.satieh1.lerrcorsts1.show() # Bit 13 should be 1 (ANF error
status)
0x00000000 : hloe (15:15) -- Header Log Overflow Status (HLOE)
0x00000000 : cie (14:14) -- Correctable Internal Error Status (CIE)
0x00000001 : anfe (13:13) -- Advisory Non-Fatal Error Status (ANFE)
0x00000000 : rtte (12:12) -- Replay Timer Timeout Status (RTTE)
0x00000000 : reserved_1 (11:09) -- Reserved
0x00000000 : rnre (08:08) -- REPLAY_NUM Rollover Status (RNRE)
0x00000000 : bdllpe (07:07) -- Bad DLLP Error Status (BDLLPE)
0x00000000 : btple (06:06) -- Bad TLP Error Status (BTLPE)
0x00000000 : reserved (05:01) -- Reserved
0x00000000 : re (00:00) -- Receiver Error Status (RE)

>>> sv.socket0.io0.uncore.satieh.satieh1.lerruncsts1
0x100000
>>> sv.socket0.io0.uncore.satieh.satieh1.lerruncsts1.show() # Bit 20 should be 1 (UR error
status)
0x00000000 : reserved_2 (31:28) -- Reserved
0x00000000 : dmwr (27:27) -- DMWR Request Egress Blocked Status (DMWR)
0x00000000 : pteb (26:26) -- Poisoned TLP Egress Blocked Status (PTEB)
```

```

0x00000000 : tpbe (25:25) -- TLP Prefix Blocked Error Status (TPBE)
0x00000000 : aeb (24:24) -- AtomicOp Egress Blocked Status (AEB)
0x00000000 : mbt (23:23) -- MC Blocked TLP Status (MBT)
0x00000000 : uie (22:22) -- Uncorrectable Internal Error Status (UIE)
0x00000000 : av (21:21) -- ACS Violation Status (AV)
0x00000000 : ure (20:20) -- Unsupported Request Error Status (URE)
0x00000000 : ee (19:19) -- ECRC Error Status (EE)
0x00000000 : mt (18:18) -- Malformed TLP Status (MT)
0x00000000 : ro (17:17) -- Receiver Overflow Status (RO)
0x00000000 : uc (16:16) -- Unexpected Completion Status (UC)
0x00000000 : ca (15:15) -- Completer Abort Status (CA)
0x00000000 : ct (14:14) -- Completion Timeout Status (CT)
0x00000000 : fcep (13:13) -- Flow Control Protocol Error Status (FCEP)
0x00000000 : pt (12:12) -- Poisoned TLP Received Status (PT)
0x00000000 : reserved_1 (11:06) -- Reserved
0x00000000 : sde (05:05) -- Surprise Down Error Status (SDE)
0x00000000 : dlpe (04:04) -- Data Link Protocol Error Status (DLPE)
0x00000000 : reserved (03:00) -- Reserved

>>> sv.socket0.io0.uncore.satieh.satiehl.reg_lerrcorcnt # Should be 1
0x1

>>> sv.socket0.io0.uncore.satieh.satiehl.reg_lerrhdrlog1 # TLP header logs, will not be
decoding and verifying contents
0x4800001
>>> sv.socket0.io0.uncore.satieh.satiehl.reg_lerrhdrlog2 # TLP header logs, will not be
decoding and verifying contents
0xfc19be0f
>>> sv.socket0.io0.uncore.satieh.satiehl.reg_lerrhdrlog3 # TLP header logs, will not be
decoding and verifying contents
0xff0000
>>> sv.socket0.io0.uncore.satieh.satiehl.reg_lerrhdrlog4 # TLP header logs, will not be
decoding and verifying contents
0x0

# Check all status registers in SIEH's RCEC are as expected.
>>> sv.socket0.io0.uncore.satieh.satiehl.devsts
0x1
>>> sv.socket0.io0.uncore.satieh.satiehl.devsts.show() # Bit 0 should be 1 (CE detected)
0x00000000 : reserved (15:06) -- Reserved Zero: Software must always write 0 to these bits.
0x00000000 : tp (05:05) -- When set, this bit indicates that the function has issued Non-
Posted REquests that have not been completed...
0x00000000 : apd (04:04) -- Auxiliary Power is not supported.
0x00000000 : urd (03:03) -- This bit indicates that the device received an Unsupported
Request. Errors are logged in this register re...
0x00000000 : fed (02:02) -- This bit indicates status of fatal errors detected. Errors are
logged in this register regardless of whet...
0x00000000 : nfd (01:01) -- This bit indicates status of non-fatal errors detected. Errors
are logged in this register regardless of...
0x00000001 : ced (00:00) -- This bit indicates status of correctable errors detected. Errors
are logged in this register regardless o...
>>> sv.socket0.io0.uncore.satieh.satiehl.errcorsts
0x4000
>>> sv.socket0.io0.uncore.satieh.satiehl.errcorsts.show() # Bit 14 should be 1 (correctable
internal error)
0x00000000 : hloe (15:15) -- Header Log Overflow Status (HLOE)
0x00000001 : cie (14:14) -- Correctable Internal Error Status (CIE)
0x00000000 : anfe (13:13) -- Advisory Non-Fatal Error Status (ANFE)
0x00000000 : rtte (12:12) -- Replay Timer Timeout Status (RTTE)
0x00000000 : reserved_1 (11:09) -- Reserved
0x00000000 : rnre (08:08) -- REPLAY_NUM Rollover Status (RNRE)
0x00000000 : bdllpe (07:07) -- Bad DLLP Error Status (BDLLPE)
0x00000000 : btlpe (06:06) -- Bad TLP Error Status (BTLPE)
0x00000000 : reserved (05:01) -- Reserved
0x00000000 : re (00:00) -- Receiver Error Status (RE)
>>> sv.socket0.io0.uncore.satieh.satiehl.rooterrsts
0x1
>>> sv.socket0.io0.uncore.satieh.satiehl.rooterrsts.show() # Bit 0 should be 1 (ERR_COR
received)
0x00000000 : aemn (31:27) -- Advanced Error Interrupt Message
0x00000000 : reserved_copy (26:07) -- Reserved

```

```
0x00000000 : femr (06:06) -- Fatal Error Messages Received (FE)
0x00000000 : nfemr (05:05) -- Non-Fatal Error Messages Receive
0x00000000 : fuf (04:04) -- First Uncorrectable Fatal (FUF) Se
0x00000000 : mefr (03:03) -- Multiple ERR_FATAL/NONFATAL Recei
0x00000000 : efr (02:02) -- ERR_FATAL/NONFATAL Received (EFR)
0x00000000 : mcer (01:01) -- Multiple ERR_COR Received (MCER)
0x00000001 : cer (00:00) -- ERR_COR Received (CER) Set when a
>>> sv.socket0.io0.uncore.satieh.satieh1.errsrcid
0x4
>>> sv.socket0.io0.uncore.satieh.satieh1.errsrcid.show() # ERR_COR source ID should match BDF
of this SIEH
0x00000000 : efsid (31:16) ...
0x00000004 : ecsid (15:00) ...

# Check all status registers in SIEH's global error logic are as expected.
>>> sv.socket0.io0.uncore.satieh.satieh1.gerrcorsts # Bit 0 should be 1 (IEH local error)
0x1
>>> sv.socket0.io0.uncore.satieh.satieh1.gsysevtsts
0x1
>>> sv.socket0.io0.uncore.satieh.satieh1.gsysevtsts.show() # Bit 0 should be 1 (correctable
error status)
0x00000000 : reserved (31:03) -- Reserved
0x00000000 : fa_sysev_sts (02:02) -- Fatal Error Status - A fa
0x00000000 : nf_sysev_sts (01:01) -- Nonfatal Error Status - A
0x00000001 : co_sysev_sts (00:00) -- Correctable Error Status

# Check all status registers in GIEH are as expected.
>>> sv.socket0.io0.uncore.globalieh.iehregs.gerrcorsts # This is strapping dependent. Purpose
of the test isn't to verify expected strapping so just verify that 1 bit between bits 1 and 31
is 1.
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtsts
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtsts.show() # Bit 0 should be 1 (correctable
error status)
```

7. Resume the system to allow platform error handling flows to occur.

```
>>> go()
```

8. Post error handling verification of relevant error status registers along the signaling path.

```
halt()
# Check all status registers in SIEH's local error logic are as expected.
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerruncfep # Should be unchanged because there's
still a non-zero LERRUNCSTS#
>>> sv.socket0.io0.uncore.satieh.satieh1.lerrcorsts1 # Should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.lerruncsts1 # Should be unchanged
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerrcorcnt # Should be 1
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerrhdrlog1 # Should be unchanged
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerrhdrlog2 # Should be unchanged
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerrhdrlog3 # Should be unchanged
>>> sv.socket0.io0.uncore.satieh.satieh1.reg_lerrhdrlog4 # Should be unchanged

# Check all status registers in SIEH's RCEC are as expected.
>>> sv.socket0.io0.uncore.satieh.satieh1.devsts # Should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.errcorsts # Should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.rooterrsts # Should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.errsrcid # Should be unchanged

# Check all status registers in SIEH's global error logic are as expected.
>>> sv.socket0.io0.uncore.satieh.satieh1.gerrcorsts # Should be 0
>>> sv.socket0.io0.uncore.satieh.satieh1.gsysevtsts # Should be 0

# Check all status registers in GIEH are as expected.
>>> sv.socket0.io0.uncore.globalieh.iehregs.gerrcorsts # Should be 0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtsts # Should be 0
```

9. There should be an aer_event in FTrace.

```
kworker/0:1-19      [000] ..... 171.633093: aer_event: 0000:00:00.4 PCIe Bus Error:
severity=Corrected, Corrected Internal Error, TLP Header=Not available
```

10. BIOS serial log:

```
[IEH] error found at IEH(S:0x0 B:0x7E D:0x2 F:0x0) Sev: IEH CORRECT ERROR
[IEH] ErrorStatus 0x8, MaxBitIdx 0x1D
IEH CORRECT ERROR
[IEH] BitIdx 0x3, ShareIdx 0x0
[IEH] error device is (S:0x0 B:0x0 D:0x0 F:0x4) BitIdx 0x3, ShareIdx 0x0 [IEH] error found at
IEH(S:0x0 B:0x0 D:0x0 F:0x4) Sev: IEH CORRECT ERROR
[IEH] ErrorStatus 0x1, MaxBitIdx 0x1
IEH CORRECT ERROR
  Clearing Local Error Status for Error Source 1, Error Severity 1, Error Status 0x2000,
Device Type S3M INTEGRATE BOOT
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7E1C5018
WHEA: Detected Ras Non Standard Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
IehRciepDevicesErrorHandler - Socket 0, Bus 0, device 0, function 4

Socket[0]Stack[0] is not a CXL Port!

Get CXL DP Port Secondary Bus# failed!
CxlDevCMErrHandler Entry
[CXL] - Not a CXL device. Exiting since FlexBusPort is not available
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7E1C0818
WHEA: Detected PCIe Error
--Logging Corrected Error to WHEA
WHEA: Sending OS notification via SCI. Success
```

6.10.2 SERR

Test Objective

The objective of this test is to observe the SERR path from S3M.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores A step
BIOS	WW06'24 BKC
OS Version	CentOS Stream 8
Kernel	5.19.0-gnr.po.bkc.7.2.21.x86_64
HW Configuration	No particular DIMM, PCIe card, or hardware tool configuration required.
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the [CScripts](#).

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

Recipe

1. Halt, clear any existing errors, and verify that IEH error logs were all cleared.

```
>>> halt()
>>> error.clearAllErrors()
>>> halt()
>>> error.check_ieh_errors()
```

2. Determine which satellite IEH S3M reports its AER/DO_SERR errors to. S3M and it's SIEH will always be on the same bus and this external/enumerated bus will always be 0. Use `ei.iio.showIehMapping()` to view the internal bus number for each SIEH. Then use `sv.socket0.target_info["pci_bus_map"]` to find which internal bus number maps to external bus 0; the SIEH on this bus will be the SIEH to check/modify in this recipe.

```
>>> ei.iio.showIehMapping()
>>> sv.socket0.target_info["pci_bus_map"]
```

3. Check the value of the following IEH and SPI registers before injecting in order to compare to their values afterwards. Clear `BIOS_SPI_UR_STS_CTL.urd` if it is asserted. The IO die 0 is selected because that is where the S3M with integrated boot logic resides and satellite IEH 1 is checked because it is strapped to S3M. Take note of the inline comments.

```
>>> sv.socket0.io0.uncore.satieh.satieh1.showsearch("err")

# SIEH RCEC Registers
erruncsts = 0x0 # Must be 0
erruncmsk = 0x180020 # uie bit (bit 22) must be 0. Which it is.
...
rooterrsts = 0x0 # Must be 0
errsrcid = 0x0

# SIEH Global Error Logic Registers
gerrfatsts = 0x0 # Must be 0
gerrfatmsk = 0x0 # Must be 0
...
gerrsrcid = 0x0
gerrfatime = 0x0

# SIEH Local Error Logic Registers
reg_lerruncfep = 0x0 # Must be 0
reg_lerrhdrlog1 = 0x40000001
reg_lerrhdrlog2 = 0xf
reg_lerrhdrlog3 = 0xfed00108
...
lerruncsts0 = 0x0 # LERRUNCSTS0.do_serr1 (bit 28) is strapped to log SERR from S3M
lerruncmsk0 = 0x0 # Must be 0

# S3M SPI Device Registers
>>> sv.socket0.io0.uncore.s3m.main_spi.ibl_arch_spi.bios_spi_ur_sts_ctl
0x1
>>> sv.socket0.io0.uncore.s3m.main_spi.ibl_arch_spi.bios_spi_ur_sts_ctl.showfields()
0x00000000 : rsvd (31:02)
0x00000000 : urd (01:01) # Must be 0
0x00000001 : urre (00:00) # Must be 1
```



```
>>> sv.socket0.io0.uncore.s3m.main_spi.ibl_arch_spi.bios_spi_sts_cmd
0x502
>>> sv.socket0.io0.uncore.s3m.main_spi.ibl_arch_spi.bios_spi_sts_cmd.showfields()
...
0x00000000 : sse (30:30) # Must be 0 (signaled SERR status bit)
0x00000001 : serren (08:08) # Must be 1
...
```

4. Execute the following command to inject an unsupported request to S3M SPI.

```
>>> ei.iio.injectS3MURError()
Disabling memory space...
Calculated BAR: 0xF6830000

Checking for successful error injection to S3M SPI...

BIOS_SPI_UR_STS_CTL value: 0x3

Error successfully injected to S3M spi!
Re-enabling memory space...
```

5. Check the registers mentioned previously again. SIEH1's RCEC logged a UIE in ERRUNCSTS and logged three bits in ROOTERRSTS related to the fatal error message received. The final registers to look at within SIEH1 are LERRUNCSTS0.do_serr_1 which indicates an SERR came from S3M and the local error header logs. Within the SPI device, note that the UR detected bit is asserted due to the detected UR and the UR reporting enable bit is asserted because CScrips modified it so an SERR would be generated. BIOS_SPI_STS_CMD reflects that a system error was signaled.

```
>>> sv.socket0.io0.uncore.satieh.satieh1.showsearch("err")
erruncsts = 0x400000
...
rooterrsts = 0x54
errsrcid = 0x40000
...
gerrfatsts = 0x1
...
gerrsrcid = 0x18f0000
gerrfatsts = 0x3214af095
...
reg_lerruncfep = 0x1c
reg_lerrhdrlog1 = 0x1
reg_lerrhdrlog2 = 0x8088858f
reg_lerrhdrlog3 = 0x0
...
lerruncsts0 = 0x10000000
...
>>> sv.socket0.io0.uncore.s3m.main_spi.ibl_arch_spi.bios_spi_ur_sts_ctl
0x3
>>> sv.socket0.io0.uncore.s3m.main_spi.ibl_arch_spi.bios_spi_ur_sts_ctl.showfields()
0x00000000 : rsvd (31:02)
0x00000001 : urd (01:01)
0x00000001 : urre (00:00)
>>> sv.socket0.io0.uncore.s3m.main_spi.ibl_arch_spi.bios_spi_sts_cmd
0x40000502
>>> sv.socket0.io0.uncore.s3m.main_spi.ibl_arch_spi.bios_spi_sts_cmd.showfields()
...
0x00000001 : sse (30:30)
...
```

6. Resume the system to allow error reporting and handling flows to complete and then check the following registers which should be cleared.

```
>>> go()
>>> sv.socket0.io0.uncore.satieh.satieh1.showsearch("err")
erruncsts = 0x0
```

```
...
rooterrsts = 0x0
...
gerrfatsts = 0x0
...
reg_lerruncfep = 0x0
...
lerruncsts0 = 0x0
...
>>> sv.socket0.io0.uncore.s3m.main_spi.ibl_arch_spi.bios_spi_sts_cmd
0x502
>>> sv.socket0.io0.uncore.s3m.main_spi.ibl_arch_spi.bios_spi_sts_cmd.showfields()
...
0x00000000 : sse (30:30)
...
```

7. BIOS serial log (IOMCA is enabled):

```
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0x4, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0x4, State = 0x2
[IEH] error found at IEH(S:0x0 B:0x7E D:0x2 F:0x0) Sev: IEH FATAL ERROR
[IEH] ErrorStatus 0x8, MaxBitIdx 0x1D
IEH FATAL ERROR
[IEH] BitIdx 0x3, ShareIdx 0x0
[IEH] error device is (S:0x0 B:0x0 D:0x0 F:0x4) BitIdx 0x3, ShareIdx 0x0 [IEH] error found at
IEH(S:0x0 B:0x0 D:0x0 F:0x4) Sev: IEH FATAL ERROR
[IEH] ErrorStatus 0x1, MaxBitIdx 0x1
IEH FATAL ERROR
    Clearing Local Error Status for Error Source 0, Error Severity 4, Error Status 0x10000000,
Device Type
ERROR: C00000002:V03071008 I0 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7E1C0A18
WHEA: Detected Ras Non Standard Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
ERROR: C00000002:V03071008 I0 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7E1C0418
LogErrorToElog: --Logging Error to EMCA
WHEA: Detected PCIe Error
    --Logging Uncorrected Error to WHEA
IehRciepDevicesErrorHandler - Socket 0, Bus 0, device 0, function 4

Socket[0]Stack[0] is not a CXL Port!

Get CXL DP Port Secondary Bus# failed!
CxlHandleIdeErr Entry
RAS Cap Base Address 0x0
No RAS Cap Base Address
CxlDevCMErrorHandler Entry
[CXL] - Not a CXL device. Exiting since FlexBusPort is not available
ERROR: C00000002:V03071008 I0 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7E1C0418
LogErrorToElog: --Logging Error to EMCA
WHEA: Detected PCIe Error
    --Logging Uncorrected Error to WHEA
S0 C0 T0 [CpuRas]MC status 0xBA000000000000E0B, class FATAL
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0x4, State = 0x4
S0 C0 T0 [CpuRas]MC status 0xBA000000000000E0B, class FATAL
S0 C0 T0 [CpuRas]MC status 0xBA000000000000E0B, class FATAL
[Mca]McaDetectAndHandle, severity = 0x1, Broadcast is 0x1
ERROR: C00000002:V03071008 I0 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7E1C0A18
WHEA: Detected Processor Error
WHEA: Ignore Processor WHEA error now.
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
ERROR: C00000002:V03071008 I0 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7E1C0A18
WHEA: Detected Ras Non Standard Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
```

```
[Mca] Clear Status  
[Mca]Find uncorrected error at CPU 0  
[Mca]Sending OS notification via MCE  
[Mca]Successfully notify OS via MCE
```

7.0 Intel® Ultra Path Interconnect (Intel® UPI) RAS

7.1 Dynamic Link Width Reduction

Dynamic Link Width Reduction is recovered from hard failure of one or more data lanes on a physical Intel UPI link, by dynamically resizing link width. Enables the system to down-shift from full to 1/2-width Intel UPI link if there are persistent errors detected on the link (example: upper or lower 8-lane wide). If possible, the link will maintain the operation over the narrow width. See the Intel UPI Specification for a description of this feature. Width reduction from full-width to x8 is supported in the event a physical lane failure. The half-width support is only for a minimal set of x8 bits to allow for any single data lane to fail. The resulting dynamic link width reduction modes supported are lanes [7:0] or [23:16].

A multi-lane failure will recover as long as all failures are not on both [7:0] and [23:16]. L0p is supported from a full-width Intel UPI link to x8 for power savings only, and L0p will be disabled if a RAS condition results in a degraded port due to Dynamic Link Width Reduction. Primary use case is to allow system to continue operating even when hard failure is detected in some of the lanes thus improving the system up-time and reliability. Link reduction can be initiated by either transmitter (Tx) or receiver (Rx) side DFT. Rx is generally the easiest of the two and tests the logic portion of the reduction. Tx initiated requires that the Rx circuits correctly drops the lane and passes the info to the logic portion of the phy in addition to testing the same logic as Rx initiated. Rx will be used during enabling with Tx providing the bulk of coverage. Intel UPI failover can only use the upper or lower x8 lane chunk, 7:0 or 23:16. If you have a lane disable in one of these ranges, the link will failover to the other range.

7.1.1 Disable Low Octet by RDC (Failover to High Lanes)

Test Objective

When the system is running with full width, program the PH_RDC register to fail single or multiple lanes in [7:0] range.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	Red Hat Enterprise Linux 9.2
<i>continued...</i>	

Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	2 socket system
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Uncore Configuration > Uncore General Configuration >
 - UPI Dynamic Link Width Reduction Support = Enable
 - Link L0p Enable = Disable
 - Link L1 Enable = Disable

Recipe

1. Verify lane failover is enabled.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl.c_failover_en
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
```

NOTE

If failover is not enabled - set failover to 1 and reset links.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl.c_failover_en = 1
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl.cp_reset=1
```

2. Ensure the Dfx lock is disabled.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutenglck=0
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutlck=0
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutenglck
socket0.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi4.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
```

```
socket1.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutlck
socket0.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io1.uncore.upi.upi4.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi4.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
```

3. Confirm that the Intel® Ultra Path Interconnect (Intel® UPI) links are running at full width.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_css.s_clm
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
```

NOTE

```
>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm.getspec
#####
# FIELD: s_clm
# Attribute: ro/v
# UpperBit: 18
# LowerBit: 16
# CurrentValue: 7
# Description:
# lane map code determined at Rx and sent in TX.Config for remote Tx; also updated on
# L0p entry/exit. Encoded as
# follows:
#
# 0b111 - lanes 0..23
# 0b001 - lanes 0..7
# 0b100 - lanes 16..23
```

4. Evaluate the Intel UPI lanes and pairs, for this example, socket0port0 and socket1port0 are used.

```
>>> upi.topology

2 sockets populated in the platform

*****
* Socket 0 (A2) UPI Config
*****

=====
| Port 4 | | Port 0 | | Port 1 | | Port 2 | | Port 3 | |
|-----|-----|-----|-----|-----|-----|
| Connected to | socket 1, port 1 | socket 1, port 0 | socket 1, port 3 | socket 1, port 2 | socket
1, port 5 | socket 1, port 4 |
|-----|-----|-----|-----|-----|-----|
| UPI LS Init Status | 0x3 - Init Done | 0x3 - Init Done | 0x3 - Init Done | 0x3 - Init Done | 0x3 - Init
Done | 0x3 - Init Done |
```

Current CRC Mode CRC16		0x0 - CRC16 0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 -
Lane Reversal No		No	No	No	No	
Retry State Local Normal		0x0 - Retry Local Normal 0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry
Tx state L0		0x0f - L0 0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Tx Lane Status*		0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000
Rx state L0		0x0f - L0 0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Rx Lane Status**		0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000
L0p Disabled		Disabled Disabled	Disabled	Disabled	Disabled	
L1 Disabled		Disabled Disabled	Disabled	Disabled	Disabled	
PPS 0x00000000		0x00000000 0x00000000	0x00000000	0x00000000	0x00000000	
Link Speed 24.0 GT/s		24.0 GT/s 24.0 GT/s	24.0 GT/s	24.0 GT/s	24.0 GT/s	

* Socket 1 (A2) UPI Config						

Port 4		Port 0	Port 1	Port 2	Port 3	
Connected to 0, port 5		socket 0, port 1 socket 0, port 4	socket 0, port 0	socket 0, port 3	socket 0, port 2	socket
UPI LS Init Status Done		0x3 - Init Done 0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init
Current CRC Mode CRC16		0x0 - CRC16 0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 -
Lane Reversal No		No	No	No	No	
Retry State Local Normal		0x0 - Retry Local Normal 0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry
Tx state L0		0x0f - L0 0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Tx Lane Status*		0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000
Rx state L0		0x0f - L0 0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Rx Lane Status**		0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000
L0p Disabled		Disabled Disabled	Disabled	Disabled	Disabled	
L1 Disabled		Disabled Disabled	Disabled	Disabled	Disabled	
PPS 0x00000000		0x00000000 0x00000000	0x00000000	0x00000000	0x00000000	
Link Speed 24.0 GT/s		24.0 GT/s 24.0 GT/s	24.0 GT/s	24.0 GT/s	24.0 GT/s	

Notes:						
* txlanedetectstat - Tx Data Lane Detection Status. A bit value of 1 indicates the TX lane is dropped.						
** rxdatareadystat - Rx Data Lane Ready Status. A bit value of 1 indicates Rx lane is dropped.						

5. Execute error injection. Disable Rx Lane 0, 1, 2, 3 on Socket0 Port 0 to force the Intel UPI Datalink Failover to the higher lanes.

```
>>> ei.utils.resetInjectorLockCheck()
>>> ei.iio.injectUPIDataLinkFailover(socket=0, port=0, direction=0, data_lanes=0xF, port_reset=1)

Checking errors in the following IPs:
  upi_0
  machine_check

=====
UPI_0: REGISTERS AND FIELDS DESCRIPTIONS
=====
Register: socket0.io0.uncore.upi.upi0.upi_regs.kti_mc_st
valid = 0x1
Set by hardware when an uncorrectable or correctable error is logged.

miscv = 0x1
KTI_MC_MISC valid flag Indicates (when set) that the MCI_MISC register contains additional
information regarding the error. See individual fields of MCI_MISC for details on which
are valid for a given error.

corr_err_sts_ind = 0x2
00 - No tracking - No hardware status tracking is provided for the structure reporting
this event. 01 - Green - Status tracking is provided for the structure posting the event;
the current status is green (below threshold). For more information, see Section
'Enhanced Cache Error reporting' of the Software Developer Manual. 10 - Yellow - Status
tracking is provided for the structure posting the event; the current status is yellow
(below threshold). For more information, see Section 'Enhanced Cache Error reporting' of
the Software Developer Manual. 11 - Reserved

cor_err_cnt = 0x2
Corrected Error Count          15-bit counter which is incremented on every corrected
error. MSB (bit 15) is a sticky overflow bit.

mscod_code = 0x22
MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based
on the standard MC_ST overwrite rules. The Error Codes and handling are defined in the
RAS. The summary per ID is: Uncorrectable (UC) 0x00 - UC Phy Initialization Failure
(NumInit) 0x01 - UC Phy Detected Drift Buffer Alarm 0x02 - UC Phy Detected Latency Buffer
Rollover (obsolete for Gen3) 0x10 - UC LL Rx detected CRC error: unsuccessful LLR (entered
Abort state) 0x11 - UC LL Rx Unsupported/Undefined packet 0x12 - UC LL or Phy Control
Error 0x13 - UC LL Rx Parameter Exception 0x14 - UC LL TDX Failure 0x15 - UC LL SGX
Failure 0x16 - UC LL Tx SDC Parity Error 0x17 - UC LL Rx SDC Parity Error 0x18 - UC LL FLE
Failure 0x1F - UC LL Detected Control Error from UPI2UPI Correctable (COR) 0x20 - COR Phy
Initialization Abort 0x21 - COR Phy Reset 0x22 - COR Phy Lane failure, recovery in x8
width 0x23 - COR Phy L0c error corrected without Phy reset 0x24 - COR Phy L0c error
triggering Phy Reset 0x25 - COR Phy L0p exit error triggering Phy Reset 0x30 - COR LL Rx
detected CRC error: successful LLR without Phy Reinit 0x31 - COR LL Rx detected CRC error:
successful LLR with Phy Reinit

mccad_int = 0x1
MCCAD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection
errors (MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-
zero) - SGX/TDX/FLE failures - SDC Parity Error Set to 1 for all other errors.
Encoding: 0 - Not Bus Interconnect Error 1 - Bus Interconnect Error

mccad_pp = 0x3
MCCAD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors
(MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-zero)
such that mccad[15:0]=0x0405. Set to 2 for all 'UC LL Rx Unsupported/Undefined Packet'
errors (MSCOD Code=0x11). Set to 2 for all SGX/TDX/FLE failures. Set to 2 for all SDC
Parity Error failures. Set to 3 for all other errors. Encoding is: 0x0 - Local Processor
Originated 0x1 - Local Processor Responded 0x2 - Local Processor Observed Error 0x3 -
Generic PP

mccad_ii = 0x3
SCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection
errors (MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-
zero) - SGX/TDX/FLE failures - SDC Parity Error Set to 3 for all other errors.
Encoding is: 0x0 - Memory Access 0x1 - Reserved II 0x2 - I/O 0x3 - Other Transaction

mccad_ll = 0x3
MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors
(MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-zero)
such that mccad[15:0]=0x0405. Set to 1 for SDC Parity Error Set to 2 for SGX/TDX/FLE
Failures Set to 3 for all other errors Encoding is: 0x0 - Memory Access 0x1 -
Reserved II 0x2 - I/O 0x3 - Other Transaction 0x0 - Level 0 0x1 - Level 1 0x2 - Level 2
0x3 - Generic LL

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.upi.upi0.upi_regs.kti_mc_misc
0x00000044 : cor_err (63:56) -- Captures Correctable Errors, indicating all correctable error cases that have occurred since last cleared. Only Correctable
Errors (as 1...
0x00000006 : phy_state (39:37) -- Link State from Physical Layer on error used for isolating lane in CRC error cases: 0x0 - 24 lane flit mux phase 0 0x1 - 24
lane flit m...
0x0000ffbe1 : flit_hdr (19:00) -- HDR bits captured: [0] - HIB (if not set other bits >4 in this log are not valid) [1] - VN [2] - VNA [3] - ACK [4] - CRD [8:5]-
Slot 0 0...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_cis
0x00000003 : s_link_up (16:15) -- 00: STxLinkUp FALSE, SRxLinkUp FALSE 01: STxLinkUp FALSE, SRxLinkUp TRUE 10: STxLinkUp TRUE, SRxLinkUp FALSE 11: STxLinkUp
TRUE, SRxLin...
0x00000001 : s_wake_detected (04:04) -- 1: Wake detected on one or more lanes.
0x00000001 : sp_detect (02:02) -- 1: remote Rx detected by Tx
0x00000001 : sp_rx_calib (01:01) -- 1: Rx calib done; sw clears by writing 1 prior to speed change to force re-calib.
0x00000001 : s_op_speed (00:00) -- 0: slow speed 1: op speed Updated at entry and exit from ResetC

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_cis
0x00000003 : s_rcvd_l0c (29:26) -- Received L0c code
0x00000003 : s_sent_l0c (25:22) -- Sent L0c code

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_pls
0x00000003 : s_rcvd_l0c_p (29:26) -- Same description as bit field in KTIREUT_PH_PLS register, but this is copy of prior to last phy reset
0x00000003 : s_sent_l0c_p (25:22) -- Same description as bit field in KTIREUT_PH_PLS register, but this is copy of prior to last phy reset.

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_wes
0x0000000f : s_rx_lane_wake_err (23:00) -- A bit value of 1 indicates Rx lane wake detect failed. Bit 0: Status of Physical Lane 0 Bit 1: Status of Physical Lane
1 ... and ...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_kes
0x0000000f : s_rx_lane_deskew_err (23:00) -- A bit value of 1 indicates Rx lane deskew failed. Bit 0: Status of Physical Lane 0 Bit 1: Status of Physical Lane
1 ... and ...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktills
0x00000001 : rx_normal_op_state (13:13) -- Reflects whether the LL Rx is in Normal Operation (i.e.. has received at least one flit from the Phy). Resets to 0 on
cold re...
0x00000003 : init_state (09:08) -- Reflects the current Initialization State of the link Layer, including any stall conditions controlled by LL_LCL[3:2]. 00 -
NOT_RDV_FO...
0x00000001 : link_layer_retry_queue_consumed (07:00) -- Link Layer Retry Queue Consumed Number of Retry Queue entries currently consumed while waiting for
ACK.

=> socket0.io0.uncore.upi.upi0.upi_regs.ktiles
0x00000001 : worst_llr_numretry_count (03:00) -- Worst Case Link Layer Retry NUM_RETRY Highest NUM_RETRY count achieved since last clear of this register or PHY
```



```
reinit i...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktilcl
0x00000003 : max_num_phy_reinit (13:12) -- Consecutive Phy ReInit's to RETRY_ABORT Note: this value is directly encoded which is different then Intel UPI
definition wher...
0x0000000f : max_num_retry (11:08) -- Consecutive LLRs to Physical Layer Reinit Note: This value is directly encoded which is different then Intel UPI definition
where v...

=> socket0.io0.uncore.upi.upi0.upi_regs.bios_kti_err_st
0x00000001 : valid (63:63) -- Set by hardware when an uncorrectable or correctable error is logged.
0x00000001 : miscv (59:59) -- BIOS_KTI_ERR_MISC valid flag Indicates (when set) that the BIOS_KTI_ERR_MISC register contains additional information regarding the
error. ...
0x00000002 : corr_err_sta_ind (54:53) -- 00 - No tracking - No hardware status tracking is provided for the structure reporting this event. 01 - Green - Status
tracking...
0x00000002 : cor_err_cnt (52:38) -- Corrected Error Count 15-bit counter which is incremented on every corrected error. MSB (bit 15) is a sticky overflow bit.
0x00000022 : mscod_code (21:16) -- MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based on the standard MC_ST overwrite
rules. The ...
0x00000001 : mcacod_int (11:11) -- MCACOD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection errors (MSCOD_Code=0x12 'UC LL or Phy
Control E...
0x00000003 : mcacod_pp (10:09) -- MCACOD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors (MSCOD_Code=0x12 'UC LL or Phy Control Error'
and KTI...
0x00000003 : mcacod_ii (03:02) -- MSCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection errors (MSCOD_Code=0x12 'UC LL or Phy
Control Err...
0x00000003 : mcacod_ll (01:00) -- MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors (MSCOD_Code=0x12 'UC LL or Phy Control
Error' a...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktierrcnt0_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes to both the 15-bit counter
and 1-bit...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktierrcnt1_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes to both the 15-bit counter
and 1-bit...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktierrcnt2_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes to both the 15-bit counter
and 1-bit...

=> socket0.io0.uncore.upi.upi0.upi_regs.kticrrcnt
0x00000001 : counter (30:00) -- Counter increments every time the Link Layer detects a CRC error.

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css
0x00000004 : s_clm (18:16) -- Lane map code determined at Rx and sent in TX.Config for remote Tx; also updated on L0p entry/exit. Encoded as follows: 0b111 -
lanes 0..23...
0x0000000f : s_rx_state (15:08) -- Rx State (encoding defined in Intel UPI specification)
0x0000000f : s_tx_state (07:00) -- Tx State (encoding defined in Intel UPI specification)

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ldc
0x00000030 : inidriftbufflatency (13:08) -- Initial Drift Buffer Latency Drift Buffer Depth refers to the initialized difference between read and writer
pointers in RX ...
0x00000004 : driftbufalarmthr (05:00) -- Drift Buffer Alarm Threshold When the difference (both underflow and overflow) between drift buffer read and write
pointers (de...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl
0x00000001 : c_rc_terms_off (29:29) -- 1: Rx pulldowns are off in ResetC
0x00000001 : c_failover_en (27:27) -- 1: Enable failover
0x00000001 : c_init_begin (05:05) -- 0: blocks exit from ResetC
0x00000001 : c_op_speed (04:04) -- 1: operational speed 0: slow speed Note: The operational speed is determined by BIOS setup of the PLL using BIOS-to-PCode
Mailbox

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_dis
0x000000dc0 : s_rx_obs_latency (19:08) -- Rx sync counter value when EIEOS of TS.Polling received at latency buffer.

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_rds
0x0000000f : rxdataradystat (23:00) -- Rx Data Lane Ready Status A bit value of 1 indicates Rx lane is dropped. Bit 0: Status of Lane 0 Bit 1:
Status of Lane 1. ...

-----
Fields with non default value
-----
=====
| IP block | # fields with non-default value |
|-----|-----|
| UPI_0 | 0x9 |
|-----|-----|
=====
```

6. Verify that socket0port0 failover to high lanes.

```
>>> go()
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_css.s_clm
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000004
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
```

7. Force system to return to full width.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_rdc=0
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl.cp_reset=1
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_css.s_clm
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
```

```
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
```

- Observe the FTrace Output in the OS. Confirm that returning to full width, the system is still functional and errors are logged/reported to the OS. Ensure MC5 is populated for DataLinkFailover injection and MSCOD shows 0x22 - COR Phy Lane failure, recovery in x8 width. Correctable errors are logged for any PHY reset - as a result you should get MC5 populated for the S0P0 and S1P1 with MSCOD 0x21, COR Phy Reset.

```
kworker/0:2-213 [000] ..... 786.994128: mce_record: CPU: 0, MCGc/s: f000c20/0, MC5:
8840008000220e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/440000c0000ffbe1/0000000000000000, RIP: 00:<0000000000000000>, TSC:
149e82f2c2b, PROCESSOR: 0:a06d0, TIME: 1679270934, SOCKET: 0, APIC: 0
kworker/0:2-213 [000] ..... 786.994276: mce_record: CPU: 72, MCGc/s: f000c20/0,
MC5: 8840004400220e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/04000000000ffbe1/0000000000000000, RIP: 00:<0000000000000000>, TSC:
149e832f297, PROCESSOR: 0:a06d0, TIME: 1679271153, SOCKET: 1, APIC: 200
kworker/72:1-1595 [072] ..... 854.358261: mce_record: CPU: 72, MCGc/s: f000c20/0,
MC5: 8800030400210e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/0600000000000000/0000000000000000, RIP: 00:<0000000000000000>, TSC:
1616f06d1bd, PROCESSOR: 0:a06d0, TIME: 1679271220, SOCKET: 1, APIC: 200
kworker/72:1-1595 [072] ..... 854.358303: mce_record: CPU: 0, MCGc/s: f000c20/0, MC5:
8800030000210e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/4600000000000000/0000000000000000, RIP: 00:<0000000000000000>, TSC:
1616f06ff6f, PROCESSOR: 0:a06d0, TIME: 1679271220, SOCKET: 0, APIC: 0
```

7.1.2 Disable High Octet by RDC (Failover to Low Lanes)

Test Objective

When the system is running with full width, program the PH_RDC register to fail single or multiple lanes in [23:16] range.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	Red Hat Enterprise Linux 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	2 socket system
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Uncore Configuration > Uncore General Configuration >
 - UPI Dynamic Link Width Reduction Support = Enable
 - Link L0p Enable = Disable
 - Link L1 Enable = Disable

Recipe

1. Verify lane failover is enabled.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl.c_failover_en
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
```

NOTE

If failover is not enabled - set failover to 1, and reset links.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl.c_failover_en = 1
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl.cp_reset=1
```

2. Ensure the Dfx lock is disabled.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutenglck=0
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutlck=0
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutenglck
socket0.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi4.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutlck
socket0.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io1.uncore.upi.upi4.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
```

```
socket0.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi4.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
```

3. Confirm that the Intel UPI links are running at full width.

```
>>> sv.sockets.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
```

NOTE

```
>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm.getspec
#####
# FIELD: s_clm
# Attribute: ro/v
# UpperBit: 18
# LowerBit: 16
# CurrentValue: 7
# Description:
# lane map code determined at Rx and sent in TX.Config for remote Tx; also updated
# on L0p entry/exit. Encoded as
# follows:
#
# 0b111 - lanes 0..23
# 0b001 - lanes 0..7
# 0b100 - lanes 16..23
```

4. Evaluate the Intel UPI lanes and pairs, for this example, socket0port0 and socket1port0 are used.

```
>>> upi.topology
2 sockets populated in the platform
*****
* Socket 0 (A2) UPI Config
*****
=====
| Port 4 | | Port 0 | | Port 1 | | Port 2 | | Port 3 | |
| | | | | | | | | | | | | |
| Connected to | socket 1, port 1 | socket 1, port 0 | socket 1, port 3 | socket 1, port 2 | socket
1, port 5 | socket 1, port 4 |
| UPI LS Init Status | 0x3 - Init Done | 0x3 - Init Done | 0x3 - Init Done | 0x3 - Init Done | 0x3 - Init
Done | 0x3 - Init Done |
| Current CRC Mode | 0x0 - CRC16 | 0x0 - CRC16 | 0x0 - CRC16 | 0x0 - CRC16 | 0x0 -
CRC16 | 0x0 - CRC16 |
| Lane Reversal | No | No | No | No | No |
| | | | | | |
| Retry State | 0x0 - Retry Local Normal | 0x0 - Retry Local Normal | 0x0 - Retry Local Normal | 0x0 - Retry Local Normal | 0x0 - Retry
Local Normal | 0x0 - Retry Local Normal |
| Tx state | 0x0f - L0 | 0x0f - L0 | 0x0f - L0 | 0x0f - L0 | 0x0f -
L0 | 0x0f - L0 |
=====
| UPI Tx Lane Status* | 0000 0000 0000 0000 0000 0000 | 0000 0000 0000 0000 0000 0000 | 0000 0000 0000 0000 0000 0000 | 0000 0000 0000 0000 0000 0000 | 0000 0000
0000 0000 0000 | 0000 0000 0000 0000 0000 0000 |
```

0x0f - L0	Rx state	0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
0x0f - L0						
UPI Rx Lane Status**	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000
0000 0000 0000 0000						
Disabled	L0p	Disabled	Disabled	Disabled	Disabled	
Disabled						
Disabled	L1	Disabled	Disabled	Disabled	Disabled	
Disabled						
0x00000000	PFS	0x00000000	0x00000000	0x00000000	0x00000000	
0x00000000						
24.0 GT/s	Link Speed	24.0 GT/s	24.0 GT/s	24.0 GT/s	24.0 GT/s	
24.0 GT/s						

* Socket 1 (A2) UPI Config *						

Port 4	Port 0	Port 1	Port 2	Port 3	Port 5	
Connected to	socket 0, port 1	socket 0, port 0	socket 0, port 3	socket 0, port 2	socket 0, port 4	socket
0, port 5						
UPI LS Init Status	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init
Done	0x3 - Init Done					
Current CRC Mode	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 -
CRC16	0x0 - CRC16					
Lane Reversal	No	No	No	No	No	
No						
Retry State	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry
Local Normal	0x0 - Retry Local Normal					
0x0f - L0	Tx state	0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
0x0f - L0						
UPI Tx Lane Status*	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000
0000 0000 0000 0000						
0x0f - L0	Rx state	0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
0x0f - L0						
UPI Rx Lane Status**	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000
0000 0000 0000 0000						
Disabled	L0p	Disabled	Disabled	Disabled	Disabled	
Disabled						
Disabled	L1	Disabled	Disabled	Disabled	Disabled	
Disabled						
0x00000000	PFS	0x00000000	0x00000000	0x00000000	0x00000000	
0x00000000						
24.0 GT/s	Link Speed	24.0 GT/s	24.0 GT/s	24.0 GT/s	24.0 GT/s	
24.0 GT/s						

Notes:						
* txlanedetectstat - Tx Data Lane Detection Status. A bit value of 1 indicates the TX lane is dropped.						
** rxdataareadystat - Rx Data Lane Ready Status. A bit value of 1 indicates Rx lane is dropped.						

5. Execute error injection. Disable Rx Lane 0, 1, 2, 3 on Socket0 Port 0 to force the Intel UPI Datalink Failover to the higher lanes.

```
>>> ei.utils.resetInjectorLockCheck()
>>> ei.io.injectUPIDatalinkFailover(socket=0, port=0, direction=0, data_lanes=0xF00000, port_reset=1)

Checking errors in the following IPs:
  upi_0
  machine_check

=====
UPI_0: REGISTERS AND FIELDS DESCRIPTIONS
=====
Register: socket0.io0.uncore.upi.upi0.upi_regs.kti_mc_st
valid = 0x1
Set by hardware when an uncorrectable or correctable error is logged.

miscv = 0x1
KTI_MC_MISC valid flag Indicates (when set) that the MCI_MISC register contains additional
information regarding the error. See individual fields of MCI_MISC for details on which
are valid for a given error.

corr_err_sts_ind = 0x2
```

```

00 - No tracking - No hardware status tracking is provided for the structure reporting
this event. 01 - Green - Status tracking is provided for the structure reporting the event;
the current status is green (below threshold). For more information, see Section
'Enhanced Cache Error reporting' of the Software Developer Manual. 10 - Yellow - Status
tracking is provided for the structure reporting the event; the current status is yellow
(below threshold). For more information, see Section 'Enhanced Cache Error reporting' of
the Software Developer Manual. 11 - Reserved

cor_err_cnt = 0x2
Corrected Error Count          15-bit counter which is incremented on every corrected
error. MSB (bit 15) is a sticky overflow bit.

mscod_code = 0x22
MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based
on the standard MC_ST overwrite rules. The Error codes and handling are defined in the
HAS. The summary per ID is: Uncorrectable (UC) 0x00 - UC Phy Initialization Failure
(NumInit) 0x01 - UC Phy Detected Drift Buffer Alarm 0x02 - UC Phy Detected Latency Buffer
Rollover (obsolete for Gen3) 0x10 - UC LL Rx detected CRC error: unsuccessful LLR (entered
Abort state) 0x11 - UC LL Rx Unsupported/Undefined packet 0x12 - UC LL or Phy Control
Error 0x13 - UC LL Rx Parameter Exception 0x14 - UC LL TDX Failure 0x15 - UC LL SGX
Failure 0x16 - UC LL Tx SDC Parity Error 0x17 - UC LL Rx SDC Parity Error 0x18 - UC LL FLE
Failure 0x1F - UC LL Detected Control Error from UFI2UPI Correctable (COR) 0x20 - COR Phy
Initialization Abort 0x21 - COR Phy Reset 0x22 - COR Phy Lane failure, recovery in x8
width 0x23 - COR Phy L0c error corrected without Phy reset 0x24 - COR Phy L0c error
triggering Phy Reset 0x25 - COR Phy L0p exit error triggering Phy Reset 0x30 - COR LL Rx
detected CRC error: successful LLR without Phy Reinit 0x31 - COR LL Rx detected CRC error:
successful LLR with Phy Reinit

mcaod_int = 0x1
MCAOD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection
errors (MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBERRST0.ctlr_txparerr=non-
zero) - SGX/TDX/FLE failures - SDC Parity Error Set to 1 for all other errors.
Encoding: 0 - Not Bus Interconnect Error 1 - Bus Interconnect Error

mcaod_pp = 0x3
MCAOD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors
(MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBERRST0.ctlr_txparerr=non-zero)
such that mcaod[15:0]=0x0405. Set to 2 for all 'UC LL Rx Unsupported/Undefined Packet'
errors (MSCOD Code=0x11). Set to 2 for all SGX/TDX/FLE failures. Set to 2 for all SDC
Parity Error Failures. Set to 3 for all other errors. Encoding is: 0x0 - Local Processor
Originated 0x1 - Local Processor Responded 0x2 - Local Processor Observed Error 0x3 -
Generic PP

mcaod_ii = 0x3
SCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection
errors (MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBERRST0.ctlr_txparerr=non-
zero) - SGX/TDX/FLE failures - SDC Parity Error Set to 3 for all other errors.
Encoding is: 0x0 - Memory Access 0x1 - Reserved II 0x2 - I/O 0x3 - Other Transaction

mcaod_ll = 0x3
MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors
(MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBERRST0.ctlr_txparerr=non-zero)
such that mcaod[15:0]=0x0405. Set to 1 for SDC Parity Error Set to 2 for SGX/TDX/FLE
Failures Set to 3 for all other errors. Encoding is: 0x0 - Memory Access 0x1 -
Reserved II 0x2 - I/O 0x3 - Other Transaction 0x0 - Level 0 0x1 - Level 1 0x2 - Level 2
0x3 - Generic LL

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.upi0.upi_regs.kti_mc_misc
0x00000044 : cor_err (63:56) -- Captures Correctable Errors, indicating all correctable error cases that have occurred since last cleared. Only Correctab...
0x00000005 : phy_state (39:37) -- Link State from Physical Layer on error used for isolating lane in CRC error cases: 0x0 - 24 lane flit mux phase 0 0x1 - ...
0x000f0f01 : flit_hdr (19:00) -- HDR bits captured: [0] - HIB (if not set other bits >4 in this log are not valid) [1] - VN [2] - VNA [3] - ACK [4] - CRD ...

=> socket0.io0.uncore.upi0.upi_regs.ktireut_ph_cis
0x00000003 : s_link_up (16:15) -- 00: STxLinkUp FALSE, SRxLinkUp FALSE 01: STxLinkUp FALSE, SRxLinkUp TRUE 10: STxLinkUp TRUE, SRxLinkUp FALSE 11: STxLink...
0x00000001 : s_wake_detected (04:04) -- 1: Wake detected on one or more lanes.
0x00000001 : sp_detect (02:02) -- 1: remote Rx detected by TX
0x00000001 : sp_tx_calib (01:01) -- 1: Rx calib done; sw clears by writing 1 prior to speed change to force re-calib.
0x00000001 : s_op_speed (00:00) -- 0: slow speed 1: op speed Updated at entry and exit from ResetC

=> socket0.io0.uncore.upi0.upi_regs.ktireut_ph_cls
0x00000003 : s_rcvd_l0c (29:26) -- Received L0c code
0x00000003 : s_sent_l0c (25:22) -- Sent L0c code

=> socket0.io0.uncore.upi0.upi_regs.ktireut_ph_pls
0x00000003 : s_rcvd_l0c_p (29:26) -- Same description as bit field in KTI_REUT_PH_PLS register, but this is copy of prior to last phy reset
0x00000003 : s_sent_l0c_p (25:22) -- Same description as bit field in KTI_REUT_PH_PLS register, but this is copy of prior to last phy reset.

=> socket0.io0.uncore.upi0.upi_regs.ktireut_ph_wes
0x0f0f0000 : s_rx_lane_wake_err (23:00) -- A bit value of 1 indicates Rx lane wake detect failed. Bit 0: Status of Physical Lane 0 Bit 1: Status of Physic...

=> socket0.io0.uncore.upi0.upi_regs.ktireut_ph_ks
0x0f0f0000 : s_rx_lane_deskew_err (23:00) -- A bit value of 1 indicates Rx lane deskew failed. Bit 0: Status of Physical Lane 0 Bit 1: Status of Physical ...

=> socket0.io0.uncore.upi0.upi_regs.ktills
0x00000001 : rx_normal_op_state (13:13) -- Reflects whether the LL Rx is in Normal Operation (i.e.. has received at least one flit from the Phy). Resets ...
0x00000003 : init_state (09:08) -- Reflects the current Initialization State of the link Layer, including any stall conditions controlled by LL_LCL[3:2]. ...

=> socket0.io0.uncore.upi0.upi_regs.ktills
0x00000001 : worst_llr_numretry_count (03:00) -- Worst Case Link Layer Retry NUM_RETRY Highest NUM_RETRY count achieved since last clear of this register ...

=> socket0.io0.uncore.upi0.upi_regs.ktilcl
0x00000003 : max_num_phy_reinit (13:12) -- Consecutive Phy ReInit's to RETRY_ABORT Note: this value is directly encoded which is different then Intel UPI ...
0x0000000f : max_num_retry (11:08) -- Consecutive LLRs to Physical Layer ReInit Note: This value is directly encoded which is different then Intel UPI def...

=> socket0.io0.uncore.upi0.upi_regs.bios_kti_err_st
0x00000001 : valid (63:63) -- Set by hardware when an uncorrectable or correctable error is logged.
0x00000001 : miscv (59:59) -- BIOS KTI_ERR_MISC valid flag Indicates (when set) that the BIOS KTI_ERR_MISC register contains additional information regard...
0x00000002 : cor_err_sta_ind (54:53) -- 00 - No tracking - No hardware status tracking is provided for the structure reporting this event. 01 - Green - ...
0x00000002 : cor_err_cnt (52:38) -- Corrected Error Count 15-bit counter which is incremented on every corrected error. MSB (bit 15) is a sticky overflow...
0x00000022 : mscod_code (21:16) -- MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based on the standard MC_ST overwr...
0x00000001 : mcaod_int (11:11) -- MCAOD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection errors (MSCOD Code=0x12 'UC LL or ...
0x00000003 : mcaod_pp (10:09) -- MCAOD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors (MSCOD Code=0x12 'UC LL or Phy Control...
0x00000003 : mcaod_ii (03:02) -- MSCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection errors (MSCOD Code=0x12 'UC LL or ...
0x00000003 : mcaod_ll (01:00) -- MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors (MSCOD Code=0x12 'UC LL or Phy C...

=> socket0.io0.uncore.upi0.upi_regs.ktierrcnt0_cntnr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes to both the 15-bit co...

=> socket0.io0.uncore.upi0.upi_regs.ktierrcnt1_cntnr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes to both the 15-bit co...

=> socket0.io0.uncore.upi0.upi_regs.ktierrcnt2_cntnr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes to both the 15-bit co...

=> socket0.io0.uncore.upi0.upi_regs.ktierrcnt
0x00000001 : counter (30:00) -- Counter increments every time the Link Layer detects a CRC error.

=> socket0.io0.uncore.upi0.upi_regs.ktireut_ph_css
0x00000001 : s_clm (18:16) -- lane map code determined at Rx and sent in TX.Config for remote Tx; also updated on L0p entry/exit. Encoded as follows: 0b11...
0x0000000f : s_rx_state (15:08) -- Rx State (encoding defined in Intel UPI specification)
0x0000000f : s_tx_state (07:00) -- Tx State (encoding defined in Intel UPI specification)

=> socket0.io0.uncore.upi0.upi_regs.ktireut_ph_ldc
0x00000030 : indriftbufflatency (13:08) -- Drift Buffer Depth refers to the initialized difference between read and writer ...
0x00000004 : driftbuffalarmthr (05:00) -- Drift Buffer Alarm Threshold When the difference (both underflow and overflow) between drift buffer read and wri...

```

```

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl
0x00000001 : c_rc_terms_off (29:29) -- 1: Rx pulldowns are off in ResetC
0x00000001 : c_failover_en (27:27) -- 1: Enable failover
0x00000001 : c_init_begin (05:05) -- 0: blocks exit from ResetC
0x00000001 : c_op_speed (04:04) -- 1: operational speed 0: slow speed Note: The operational speed is determined by BIOS setup of the PLL using BIOS-to-PCo...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_rds
0x00f00000 : rxdataradystat (23:00) -- Rx Data Lane Ready Status      A bit value of 1 indicates Rx lane is dropped.      Bit 0: Status of Lane 0      Bit 1:
Status ...

-----
*****
Fields with non default value
*****
-----
| IP block | # fields with non-default value |
-----
| UPI_0 | 0x9 |
-----
*****

```

6. Verify the socket0port0 failover to low lanes.

```

>>> go()
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_css.s_clm
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000001
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007

```

7. Force system to return to full width.

```

>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_rdc=0
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl.cp_reset=1
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_css.s_clm
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007

```

8. Observe the FTrace Output in the OS. Confirm that returning to full width, the system is still functional and errors are logged/reported to the OS. Ensure MC5 is populated for DataLinkFailover injection and MSCOD shows 0x22 - COR Phy Lane failure, recovery in x8 width. Correctable errors are logged for any PHY reset - as a result you should get MC5 populated for the S0P0 and S1P1 with MSCOD 0x21, COR Phy Reset.

```

kworker/0:0-7 [000] ..... 600.813864: mce_record: CPU: 0, MCGc/s: f000c20/0, MC5:
884008000220e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/440000a0000ffbe1/0000000000000000, RIP: 00:<0000000000000000>, TSC:
108b9016f51, PROCESSOR: 0:a06d0, TIME: 1679270966, SOCKET: 0, APIC: 0
kworker/0:0-7 [000] ..... 600.813897: mce_record: CPU: 72, MCGc/s: f000c20/0,
MC5: 8840004400220e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/04000040000ffbe1/0000000000000000, RIP: 00:<0000000000000000>, TSC:
108b901b3b5, PROCESSOR: 0:a06d0, TIME: 1679270966, SOCKET: 1, APIC: 200
kworker/0:0-7 [000] ..... 691.642879: mce_record: CPU: 0, MCGc/s: f000c20/0, MC5:
8800030000210e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/4600000000000000/0000000000000000, RIP: 00:<0000000000000000>, TSC:
12871e162f5, PROCESSOR: 0:a06d0, TIME: 1679271057, SOCKET: 0, APIC: 0

```

```
kworker/0:0-7 [000] ..... 691.642919: mce_record: CPU: 72, MCGc/s: f000c20/0,
MC5: 8800030400210e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/0600000000000000/0000000000000000, RIP: 00:<0000000000000000>, TSC:
12871e185db, PROCESSOR: 0:a06d0, TIME: 1679271057, SOCKET: 1, APIC: 200
```

7.1.3 Disable Central Octet by RDC (Failover to Low Lanes)

Test Objective

When the system is running with full width, program the PH_RDC register to fail single or multiple lanes in [15:8] range.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	Red Hat Enterprise Linux 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	2 socket system
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Uncore Configuration > Uncore General Configuration >
 - UPI Dynamic Link Width Reduction Support = Enable
 - Link L0p Enable = Disable
 - Link L1 Enable = Disable

Recipe

1. Verify lane failover is enabled.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl.c_failover_en
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_ctrl.c_failover_en - 0x00000001
```



```
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_ctrl1.c_failover_en - 0x00000001
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_ctrl1.c_failover_en - 0x00000001
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl1.c_failover_en - 0x00000001
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_ctrl1.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_ctrl1.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_ctrl1.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_ctrl1.c_failover_en - 0x00000001
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_ctrl1.c_failover_en - 0x00000001
```

NOTE

If failover is not enabled - set failover to 1, and reset links.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl1.c_failover_en = 1
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl1.cp_reset=1
```

2. Ensure the Dfx lock is disabled.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutenglck=0
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutlck=0
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutenglck
socket0.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi4.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket0.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
socket1.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutenglck - 0x00000000
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.dfx_lck_ctl_cfg.reutlck
socket0.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io1.uncore.upi.upi4.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket0.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io0.uncore.upi.upi0.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io0.uncore.upi.upi1.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi2.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi3.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi4.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
socket1.io1.uncore.upi.upi5.upi_regs.dfx_lck_ctl_cfg.reutlck - 0x00000000
```

3. Confirm that the Intel UPI links are running at full width.

```
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_css.s_clm
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
```

NOTE

```
>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm.getspec
#####
# FIELD: s_clm
# Attribute: ro/v
# UpperBit: 18
# LowerBit: 16
# CurrentValue: 7
# Description:
# lane map code determined at Rx and sent in TX.Config for remote Tx; also updated
# on L0p entry/exit. Encoded as
# follows:
#
# 0b111 - lanes 0..23
# 0b001 - lanes 0..7
# 0b100 - lanes 16..23
```

- Evaluate the Intel UPI lanes and pairs, for this example, socket0port0 and socket1port0 are used.

```
>>> upi.topology
2 sockets populated in the platform
*****
* Socket 0 (A2) UPI Config
*
*****
```

Port 4	Port 5	Port 0	Port 1	Port 2	Port 3
Connected to L, port 5	socket 1, port 1 socket 1, port 4	socket 1, port 0	socket 1, port 3	socket 1, port 2	socket
UPI LS Init Status Done	0x3 - Init Done 0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init
Current CRC Mode CRC16	0x0 - CRC16 0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 -
Lane Reversal No	No	No	No	No	No
Retry State Local Normal	0x0 - Retry Local Normal 0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry
Tx state L0	0x0f - L0 0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Tx Lane Status* 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000
Rx state L0	0x0f - L0 0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Rx Lane Status** 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000
L0p Disabled	Disabled	Disabled	Disabled	Disabled	Disabled
L1 Disabled	Disabled	Disabled	Disabled	Disabled	Disabled
PPS 0x00000000	0x00000000 0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
Link Speed 24.0 GT/s	24.0 GT/s	24.0 GT/s	24.0 GT/s	24.0 GT/s	24.0 GT/s

```
*****
* Socket 1 (A2) UPI Config
*
*****
```

Port 4	Port 5	Port 0	Port 1	Port 2	Port 3

Connected to 0, port 5	socket 0, port 1 socket 0, port 4	socket 0, port 0	socket 0, port 3	socket 0, port 2	socket
UPI LS Init Status Done	0x3 - Init Done 0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init
Current CRC Mode CRC16	0x0 - CRC16 0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 -
Lane Reversal No	No	No	No	No	No
Retry State Local Normal	0x0 - Retry Local Normal 0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry
Tx state L0	0x0f - L0 0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Tx Lane Status* 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000
Rx state L0	0x0f - L0 0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Rx Lane Status** 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000	0000 0000
L0p Disabled	Disabled	Disabled	Disabled	Disabled	Disabled
L1 Disabled	Disabled	Disabled	Disabled	Disabled	Disabled
PPS 0x00000000	0x00000000 0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
Link Speed 24.0 GT/s	24.0 GT/s	24.0 GT/s	24.0 GT/s	24.0 GT/s	24.0 GT/s

Notes:
 * txlanedetectstat - Tx Data Lane Detection Status. A bit value of 1 indicates the TX lane is dropped.
 ** rxdatareadystat - Rx Data Lane Ready Status. A bit value of 1 indicates Rx lane is dropped.

5. Execute error injection. Disable Rx Lane 0, 1, 2, 3 on Socket0 Port 0 to force the Intel UPI Datalink Failover to the higher lanes.

```
>>> ei.utils.resetInjectorLockCheck()
>>> ei.iio.injectUPIDatalinkFailover(socket=0, port=0, direction=0, data_lanes=0xF00, port_reset=1)

=====
UPI_0: REGISTERS AND FIELDS DESCRIPTIONS
=====
Register: socket0.io0.uncore.upi.upi0.upi_regs.kti_mc_st
valid = 0x1
Set by hardware when an uncorrectable or correctable error is logged.

misc = 0x1
KTI MC MISC valid flag Indicates (when set) that the MCI_MISC register contains additional
information regarding the error. See individual fields of MCI_MISC for details on which
are valid for a given error.

corr_err sts ind = 0x2
00 - No Tracking - No hardware status tracking is provided for the structure reporting
this event. 01 - Green - Status tracking is provided for the structure posting the event;
the current status is green (below threshold). For more information, see Section
'Enhanced Cache Error reporting' of the Software Developer Manual. 10 - Yellow - Status
tracking is provided for the structure posting the event; the current status is yellow
(below threshold). For more information, see Section 'Enhanced Cache Error reporting' of
the Software Developer Manual. 11 - Reserved

cor_err cnt = 0x2
Corrected Error Count 15-bit counter which is incremented on every corrected
error. MSB (bit 15) is a sticky overflow bit.

mscod_code = 0x22
MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based
on the standard MC_ST overwrite rules. The Error Codes and handling are defined in the
HAS. The summary per ID is: Uncorrectable (UC) 0x00 - UC Phy Initialization Failure
(NumInit) 0x01 - UC Phy Detected Drift Buffer Alarm 0x02 - UC Phy Detected Latency Buffer
Rollover (obsolete for Gen3) 0x10 - UC LL Rx detected CRC error: unsuccessful LLR (entered
Abort state) 0x11 - UC LL Rx Unsupported/Undefined packet 0x12 - UC LL or Phy Control
Error 0x13 - UC LL Rx Parameter Exception 0x14 - UC LL TDX Failure 0x15 - UC LL SGX
Failure 0x16 - UC LL Tx SDC Parity Error 0x17 - UC LL Rx SDC Parity Error 0x18 - UC LL FLE
Failure 0x1F - UC LL Detected Control Error from UFI2UPI Correctable (COR) 0x20 - COR Phy
Initialization Abort 0x21 - COR Phy Reset 0x22 - COR Phy Lane failure, recovery in x8
width 0x23 - COR Phy L0c error corrected without Phy Reset 0x24 - COR Phy L0c error
triggering Phy Reset 0x25 - COR Phy L0p exit error triggering Phy Reset 0x30 - COR LL Rx
detected CRC error: successful LLR without Phy Reinit 0x31 - COR LL Rx detected CRC error:
successful LLR with Phy Reinit

mcacod_int = 0x1
MCACOD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection
errors (MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_tpxarrerr=non-
zero) - SGX/TDX/FLE failures - SDC Parity Error Set to 1 for all other errors.
Encoding: 0 - Not Bus Interconnect Error 1 - Bus Interconnect Error

mcacod_pp = 0x3
MCACOD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors
(MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_tpxarrerr=non-zero)
such that mcacod[15:0]=0x0405. Set to 2 for all 'UC LL Rx Unsupported/Undefined Packet'
errors (MSCOD Code=0x11). Set to 2 for all SGX/TDX/FLE failures. Set to 2 for all SDC
Parity Error Failures. Set to 3 for all other errors. Encoding is: 0x0 - Local Processor
Originated 0x1 - Local Processor Responded 0x2 - Local Processor Observed Error 0x3 -
Generic PP
```

```

mccad0_ii = 0x3
SCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection
errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBERRST0.ctrl.txparerr=non-
zero) - SGX/TDX/FLE failures - SDC Parity Error Set to 3 for all other errors.
Encoding is: 0x0 - Memory Access 0x1 - Reserved II 0x2 - I/O 0x3 - Other Transaction

mccad0_ll = 0x3
MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBERRST0.ctrl.txparerr=non-zero)
such that mccad0[15:0]=0x0405. Set to 1 for SDC Parity Error Set to 2 for SGX/TDX/FLE
Failures Set to 3 for all other errors Encoding is: 0x0 - Memory Access 0x1 -
Reserved II 0x2 - I/O 0x3 - Other Transaction 0x0 - Level 0 0x1 - Level 1 0x2 - Level 2
0x3 - Generic LL

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.upi.upi0.upi_regs.kti_misc
0x00000044 : cor_err (63:56) -- Captures Correctable Errors, indicating all correctable error cases that have occurred since last cleared. Only Correctab...
0x00000005 : phy_state (39:37) -- Link State from Physical Layer on error used for isolating lane in CRC error cases: 0x0 - 24 lane flit mux phase 0 0x1 - ...
0x0000fbef : flit_hdr (19:00) -- HDR bits captured: [0] - HIB (if not set other bits >4 in this log are not valid) [1] - VN [2] - VNA [3] - ACK [4] - CRD ...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_cis
0x00000003 : s_link_up (16:15) -- 00: STxLinkUp FALSE, SRxLinkUp FALSE 01: STxLinkUp FALSE, SRxLinkUp TRUE 10: STxLinkUp TRUE, SRxLinkUp FALSE 11: STxLink...
0x00000001 : s_wake_detected (04:04) -- 1: Wake detected on one or more lanes.
0x00000001 : sp_rx_calib (01:01) -- 1: Rx calib done; sw clears by writing 1 prior to speed change to force re-calib.
0x00000001 : s_op_speed (00:00) -- 0: slow speed 1: op speed Updated at entry and exit from ResetC

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_cls
0x00000003 : s_rcvd_l0c (29:26) -- Received L0C code
0x00000003 : s_sent_l0c (25:22) -- Sent L0C code

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_pls
0x00000003 : s_rcvd_l0c_p (29:26) -- Same description as bit field in KTIREUT_PH_PLS register, but this is copy of prior to last phy reset
0x00000003 : s_sent_l0c_p (25:22) -- Same description as bit field in KTIREUT_PH_PLS register, but this is copy of prior to last phy reset.

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_wes
0x00000000 : s_rx_lane_wake_err (23:00) -- A bit value of 1 indicates Rx lane wake detect failed. Bit 0: Status of Physical Lane 0 Bit 1: Status of Physic...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ks
0x00000000 : s_rx_lane_deskew_err (23:00) -- A bit value of 1 indicates Rx lane deskew failed. Bit 0: Status of Physical Lane 0 Bit 1: Status of Physical ...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktills
0x00000001 : rx_normal_op_state (13:13) -- Reflects whether the LL Rx is in Normal Operation (i.e., has received at least one flit from the Phy). Resets ...
0x00000003 : init_state (09:08) -- Reflects the current Initialization State of the link Layer, including any stall conditions controlled by LL_LCL[3:2]. ...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktills
0x00000001 : worst_llr_numretry_count (03:00) -- Worst Case Link Layer Retry NUM_RETRY Highest NUM_RETRY count achieved since last clear of this register ...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktillcl
0x00000003 : max_num_phy_reinit (13:12) -- Consecutive Phy ReInit's to RETRY_ABORT Note: this value is directly encoded which is different then Intel UPI ...
0x00000000 : max_num_retry (11:08) -- Consecutive LLRs to Physical Layer ReInit Note: This value is directly encoded which is different then Intel UPI def...

=> socket0.io0.uncore.upi.upi0.upi_regs.bios_kti_err_st
0x00000001 : valid (63:63) -- Set by hardware when an uncorrectable or correctable error is logged.
0x00000001 : miscv (59:59) -- BIOS KTI_ERR_MISC valid flag Indicates (when set) that the BIOS KTI_ERR_MISC register contains additional information regard...
0x00000002 : corr_err_sts_ind (54:53) -- 00 - No tracking - No hardware status tracking is provided for the structure reporting this event. 01 - Green - ...
0x00000002 : cor_err_cnt (52:38) -- Corrected Error Count - 15-bit counter which is incremented on every corrected error. MSB (bit 15) is a sticky overflow...
0x00000022 : mscod_code (21:16) -- MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based on the standard MC ST overwr...
0x00000001 : mccad0_int (11:11) -- MCCAD0 - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection errors (MSCOD_Code=0x12 'UC LL or Phy Control...
0x00000003 : mccad0_pp (10:09) -- MCCAD0 - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors (MSCOD_Code=0x12 'UC LL or Phy Control...
0x00000003 : mccad0_ii (03:02) -- MSCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection errors (MSCOD_Code=0x12 'UC LL or Phy C...
0x00000003 : mccad0_ll (01:00) -- MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors (MSCOD_Code=0x12 'UC LL or Phy C...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktierrcnt0_cnt
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes to both the 15-bit co...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktierrcnt1_cnt
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes to both the 15-bit co...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktierrcnt2_cnt
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes to both the 15-bit co...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktierrcntcnt
0x00000001 : counter (30:00) -- Counter increments every time the Link Layer detects a CRC error.

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css
0x00000001 : s_clm (18:16) -- Lane map code determined at Rx and sent in TX.Config for remote Tx; also updated on L0p entry/exit. Encoded as follows: 0b11...
0x00000000 : s_rx_state (15:08) -- Rx State (encoding defined in Intel UPI specification)
0x00000000 : s_tx_state (07:00) -- Tx State (encoding defined in Intel UPI specification)

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ldc
0x00000030 : inidriftbufflatency (13:08) -- Initial Drift Buffer Latency Drift Buffer Depth refers to the initialized difference between read and writer ...
0x00000004 : driftbuffalarmthr (05:00) -- Drift Buffer Alarm Threshold When the difference (both underflow and overflow) between drift buffer read and wr...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctl
0x00000001 : c_rc_terma_off (29:29) -- 1: Rx pull-downs are off in ResetC
0x00000001 : c_failover_en (27:27) -- 1: Enable failover
0x00000001 : c_init_begin (05:05) -- 0: blocks exit from ResetC
0x00000001 : c_op_speed (04:04) -- 1: operational speed 0: slow speed Note: The operational speed is determined by BIOS setup of the PLL using BIOS-to-PCo...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_dis
0x000000b58 : s_rx_obs_latency (19:08) -- Rx sync counter value when EIBOS of TS.Polling received at latency buffer.

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_rds
0x00000000 : rxdataareadystat (23:00) -- Rx Data Lane Ready Status A bit value of 1 indicates Rx lane is dropped. Bit 0: Status of Lane 0 Bit 1:
Status ...

-----
*****
***** Fields with non default value *****
*****
=====
| IP block | # fields with non-default value |
=====
| UPI_0 | 1 |
=====

```

6. Verify that socket0port0 failover to low lanes.

```

>>> sv.sockets.ios.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000001
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007

```

```
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
```

7. Check the Intel UPI Errors using Cscripts:

```
>>> upi.printErrors
Errors
S1_P1 | S1_P2 | S0_P0 | S1_P3 | S0_P1 | S1_P4 | S0_P2 | S1_P5 | S0_P3 | S0_P4 | S0_P5 | S1_P0 |
=====
ktiph_rxl0c_err_s... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
ktiph_rxl0c_err_l... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
ktiph_rxl0c_err_l... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
ktiph_l0pexit_err... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
ktierdis0 (Generic) | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
ktierdis1 (Generic) | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* |
0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* |
ktireut_ph_pis (G... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
ktireut_ph_pis (G... | 0xc000000* | 0xc000000* | 0xc000000* | 0xc000000* | 0xc000000* | 0xc000000* | 0xc000000* | 0xc000000* | 0xc000000* | 0xc000000* | 0xc000000* |
0xc02000* | 0xc02000* | 0xc02000* | 0xc02000* | 0xc02000* | 0xc02000* | 0xc02000* | 0xc02000* | 0xc02000* | 0xc02000* | 0xc02000* |
ktireut_ph_pss (G... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
ktiercnt0_ctr (... | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* |
| 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* |
ktiercnt1_ctr (... | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* |
| 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* |
ktiercnt2_ctr (... | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* |
| 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* |
ktiercnt0_mask (... | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* |
0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* |
ktiercnt1_mask (... | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* |
0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* |
ktiercnt2_mask (... | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* |
0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* |
kticrcrcnt (Gen... | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* |
| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
ktiles (Generic) | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* |
0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* |
ktiviral (Generic) | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 |
| 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 |
bios_kti_err_st (... | 0x8840008000220e0f* | 0x8840008000220e0f* | 0x8840008000220e0f* | 0x8840008000220e0f* | 0x8840008000220e0f* | 0x8840008000220e0f* | 0x8840008000220e0f* | 0x8840008000220e0f* | 0x8840008000220e0f* | 0x8840008000220e0f* | 0x8840008000220e0f* |
0x8840004000220e0f* | 0x8840004000220e0f* | 0x8840004000220e0f* | 0x8840004000220e0f* | 0x8840004000220e0f* | 0x8840004000220e0f* | 0x8840004000220e0f* | 0x8840004000220e0f* | 0x8840004000220e0f* | 0x8840004000220e0f* | 0x8840004000220e0f* |
```

8. Force system to return to full width.

```
>>> go()
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_rdc=0
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_ctrl.cp_reset=1
>>> sv.sockets.ios.uncore.upi.upis.upi_regs.ktireut_ph_css.s_clm
socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket0.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io0.uncore.upi.upi1.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi2.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi3.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi4.upi_regs.ktireut_ph_css.s_clm - 0x00000007
socket1.io1.uncore.upi.upi5.upi_regs.ktireut_ph_css.s_clm - 0x00000007
```

9. Observe the FTrace Output in the OS. Confirm that returning to full width, the system is still functional and errors are logged/reported to the OS. Ensure MC5 is populated for DataLinkFailover injection and MSCOD shows 0x22 - COR Phy Lane failure, recovery in x8 width. Correctable errors are logged for any PHY reset - as a result you should get MC5 populated for the S0P0 and S1P1 with MSCOD 0x21, COR Phy Reset.

```
<...>-213 [000] ..... 545.153007: mce_record: CPU: 0, MCGc/s: f000c20/0, MC5:
8840008000220e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/440000a0000ffbe1/0000000000000000, RIP: 00:<0000000000000000>, TSC:
f066b5e26d, PROCESSOR: 0:a06d0, TIME: 1679270911, SOCKET: 0, APIC: 0
<...>-213 [000] ..... 545.153046: mce_record: CPU: 72, MCGc/s: f000c20/0, MC5:
8840004400220e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/04000040000ffbe1/0000000000000000, RIP: 00:<0000000000000000>, TSC:
f066b5decf, PROCESSOR: 0:a06d0, TIME: 1679270911, SOCKET: 1, APIC: 200
```

```
kworker/72:1-1599 [072] ..... 570.680961: mce_record: CPU: 72, MCGc/s: f000c20/0,
MC5: 8800030400210e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/0600000000000000/0000000000000000, RIP: 00:<0000000000000000>, TSC:
f9514e346f, PROCESSOR: 0:a06d0, TIME: 1679270936, SOCKET: 1, APIC: 200
kworker/72:1-1599 [072] ..... 570.680987: mce_record: CPU: 0, MCGc/s: f000c20/0, MC5:
8800030000210e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/4600000000000000/0000000000000000, RIP: 00:<0000000000000000>, TSC:
f9514f8fef, PROCESSOR: 0:a06d0, TIME: 1679270936, SOCKET: 0, APIC: 0
```

7.2 Link Level Retry

Fault detection and handling at the Intel® UPI link level. Ability to protect all data packets transmitted on the Intel® UPI link with CRC and to attempt correction via necessary replay of the transaction(s). Intel® UPI Link Level Retry feature allows keeping the link up and operational when a CRC error is detected at the receiver end. Upon detecting a CRC error, Rx sends a retry request back to the Tx. If the CRC error occurs due to a transient event, then it is expected that retry will not experience a similar CRC error and a normal operation would continue.

7.2.1 MBE and Successful LLR without Phy Reinit

Test Objective

Multi-lane transient fault (multiple-bit errors) with successful LLR and signaled CMCI.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	Red Hat Enterprise Linux 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	2 socket system
CScripts Version	GNR 2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18
- [FTrace and DMSG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

Recipe

1. Evaluate the Intel UPI lanes and pairs, for this example, socket0port0 and socket1port0 are used.

```
>>> upi.topology
```

```
2 sockets populated in the platform
```

```
*****
*                               *
*   Socket 0 (A2) UPI Config   *
*                               *
*****
```

Port 4		Port 0		Port 1		Port 2		Port 3	
Connected to		socket 1, port 1		socket 1, port 0		socket 1, port 3		socket 1, port 2	
UPI LS Init Status		0x3 - Init Done		0x3 - Init Done		0x3 - Init Done		0x3 - Init Done	
Current CRC Mode		0x0 - CRC16		0x0 - CRC16		0x0 - CRC16		0x0 - CRC16	
Lane Reversal		No		No		No		No	
Retry State		0x0 - Retry Local Normal		0x0 - Retry Local Normal		0x0 - Retry Local Normal		0x0 - Retry Local Normal	
Tx state		0x0f - L0		0x0f - L0		0x0f - L0		0x0f - L0	
UPI Tx Lane Status*		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000	
Rx state		0x0f - L0		0x0f - L0		0x0f - L0		0x0f - L0	
UPI Rx Lane Status**		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000	
L0p		Disabled		Disabled		Disabled		Disabled	
L1		Disabled		Disabled		Disabled		Disabled	
PPS		0x00000000		0x00000000		0x00000000		0x00000000	
Link Speed		24.0 GT/s		24.0 GT/s		24.0 GT/s		24.0 GT/s	

```
*****
*                               *
*   Socket 1 (A2) UPI Config   *
*                               *
*****
```

Port 4		Port 0		Port 1		Port 2		Port 3	
Connected to		socket 0, port 1		socket 0, port 0		socket 0, port 3		socket 0, port 2	
UPI LS Init Status		0x3 - Init Done		0x3 - Init Done		0x3 - Init Done		0x3 - Init Done	
Current CRC Mode		0x0 - CRC16		0x0 - CRC16		0x0 - CRC16		0x0 - CRC16	
Lane Reversal		No		No		No		No	
Retry State		0x0 - Retry Local Normal		0x0 - Retry Local Normal		0x0 - Retry Local Normal		0x0 - Retry Local Normal	
Tx state		0x0f - L0		0x0f - L0		0x0f - L0		0x0f - L0	
UPI Tx Lane Status*		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000	
Rx state		0x0f - L0		0x0f - L0		0x0f - L0		0x0f - L0	
UPI Rx Lane Status**		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000		0000 0000 0000 0000 0000 0000	



```
Notes:
* txlanedetctstat - Tx Data Lane Detection Status. A bit value of 1 indicates the TX lane is dropped.
** rxdatareadystat - Rx Data Lane Ready Status. A bit value of 1 indicates Rx lane is dropped.
```

2. Ensure errors are cleared.

[illegible]

3. Execute UPI CRC error injection a port.

```
>>> ei.util.resetInjectorLockCheck()
>>> ei.iio.injectUIPErrors(socket=0, port=0, num_crcs=2)

=====
UPI_0: REGISTERS AND FIELDS DESCRIPTIONS
=====
Register: socket0.io0.uncore.upi.upi0.upi_regs.kti_mc_st
valid = 0x1
    Set by hardware when an uncorrectable or correctable error is logged.

miscv = 0x1
    KTI_MC_MISC valid flag Indicates (when set) that the MCI_MISC register contains additional
    information regarding the error. See individual fields of MCI_MISC for details on which
    are valid for a given error.

cor_err_cnt = 0x2
    Corrected Error Count          15-bit counter which is incremented on every corrected
    error. MSB (bit 15) is a sticky overflow bit.

mscod_code = 0x30
    MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based
    on the standard MC_ST overwrite rules. The Error Codes and handling are defined in the
    HAS. The summary per ID is: Uncorrectable (UC) 0x00 - UC Phy Initialization Failure
    (NumInit) 0x01 - UC Phy Detected Drift Buffer Alarm 0x02 - UC Phy Detected Latency Buffer
    rollover (obsolete for Gen3) 0x10 - UC LL Rx detected CRC error: unsuccessful LLR (entered
    Abort state) 0x11 - UC LL Rx Unsupported/Undefined packet 0x12 - UC LL or Phy Control
    Error 0x13 - UC LL Rx Parameter Exception 0x14 - UC LL TDX Failure 0x15 - UC LL SGX
    Failure 0x16 - UC LL Tx SDC Parity Error 0x17 - UC LL Rx SDC Parity Error 0x18 - UC LL FLE
    Failure 0x1f - UC LL Detected Control Error from UFI2UPI Correctable (COR) 0x20 - COR Phy
    Initialization Abort 0x21 - COR Phv Reset 0x22 - COR Phv Lane failure, recovery in x8
```



```

width 0x23 - COR Phy L0c error corrected without Phy reset 0x24 - COR Phy L0c error
triggering Phy Reset 0x25 - COR Phy L0p exit error triggering Phy Reset 0x30 - COR LL Rx
detected CRC error: successful LLR without Phy Reinit 0x31 - COR LL Rx detected CRC error:
successful LLR with Phy Reinit

mcacod_int = 0x1
MCACOD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection
errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl.txparerr=non-
zero) - SGX/TDX/FLE failures - SDC Parity Error Set to 1 for all other errors.
Encoding: 0 - Not Bus Interconnect Error 1 - Bus Interconnect Error

mcacod_pp = 0x3
MCACOD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl.txparerr=non-zero)
such that mcacod[15:0]=0x0405. Set to 2 for all 'UC LL Rx Unsupported/Undefined Packet'
errors (MSCOD_Code=0x11). Set to 2 for all SGX/TDX/FLE failures. Set to 2 for all SDC
Parity Error Failures. Set to 3 for all other errors. Encoding is: 0x0 - Local Processor
Originated 0x1 - Local Processor Responded 0x2 - Local Processor Observed Error 0x3 -
Generic PP

mcacod_ii = 0x3
SCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection
errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl.txparerr=non-
zero) - SGX/TDX/FLE failures - SDC Parity Error Set to 3 for all other errors.
Encoding is: 0x0 - Memory Access 0x1 - Reserved II 0x2 - I/O 0x3 - Other Transaction

mcacod_ll = 0x3
MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl.txparerr=non-zero)
such that mcacod[15:0]=0x0405. Set to 1 for SDC Parity Error Set to 2 for SGX/TDX/FLE
Failures Set to 3 for all other errors Encoding is: 0x0 - Memory Access 0x1 -
Reserved II 0x2 - I/O 0x3 - Other Transaction 0x0 - Level 0 0x1 - Level 1 0x2 - Level 2
0x3 - Generic LL

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.upi0.upi_regs.kti_mc_misc
0x00000040 : cor_err (63:56) -- Captures Correctable Errors, indicating all correctable error cases that have occurred since last
cleared. Only Correctab...
0x000ffbe1 : flit_hdr (19:00) -- HDR bits captured: [0] - HIB (if not set other bits >4 in this log are not valid) [1] - VN [2] -
VNA [3] - ACK [4] - CRD ...

=> socket0.io0.uncore.upi0.upi_regs.ktireut_ph_cis
0x00000003 : s_link_up (16:15) -- 00: STxLinkUp FALSE, SRxLinkUp FALSE 01: STxLinkUp FALSE, SRxLinkUp TRUE 10: STxLinkUp TRUE,
SRxLinkUp FALSE 11: STxLinkUp...
0x00000001 : s_wake_detected (04:04) -- 1: Wake detected on one or more lanes.
0x00000001 : sp_detect (02:02) -- 1: remote Rx detected by Tx
0x00000001 : sp_rx_calib (01:01) -- 1: Rx calib done; sw clears by writing 1 prior to speed change to force re-calib.
0x00000001 : s_op_speed (00:00) -- 0: slow speed 1: op speed Updated at entry and exit from ResetC

=> socket0.io0.uncore.upi0.upi_regs.ktireut_ph_cls
0x00000003 : s_rcvd_l0c (29:26) -- Received L0c code
0x00000003 : s_sent_l0c (25:22) -- Sent L0c code

=> socket0.io0.uncore.upi0.upi_regs.ktireut_ph_pls
0x00000003 : s_rcvd_l0c_p (29:26) -- Same description as bit field in KTIREUT_PH_PLS register, but this is copy of prior to last phy
reset
0x00000003 : s_sent_l0c_p (25:22) -- Same description as bit field in KTIREUT_PH_PLS register, but this is copy of prior to last phy
reset.

=> socket0.io0.uncore.upi0.upi_regs.ktils
0x00000001 : rx_normal_op_state (13:13) -- Reflects whether the LL Rx is in Normal Operation (i.e.. has received at least one flit
from the Phy). Resets...
0x00000003 : init_state (09:08) -- Reflects the current Initialization State of the link Layer, including any stall conditions
controlled by LL LCL[3:2]....
0x00000001 : link_layer_retry_queue_consumed (07:00) -- Link Layer Retry Queue Consumed Number of Retry Queue entries currently
consumed while waiting for...

=> socket0.io0.uncore.upi0.upi_regs.ktiles
0x00000001 : worst_llr_numretry_count (03:00) -- Worst Case Link Layer Retry NUM_RETRY Highest NUM_RETRY count achieved since last
clear of this register ...

=> socket0.io0.uncore.upi0.upi_regs.ktilcl
0x00000003 : max_num_phy_reinit (13:12) -- Consecutive Phy ReInit's to RETRY_ABORT Note: this value is directly encoded which is
different then Intel UPI ...
0x0000000f : max_num_retry (11:08) -- Consecutive LLRs to Physical Layer ReInit Note: This value is directly encoded which is
different then Intel UPI def...

=> socket0.io0.uncore.upi0.upi_regs.bios_kti_err_st
0x00000001 : valid (63:63) -- Set by hardware when an uncorrectable or correctable error is logged.
0x00000001 : miscv (59:59) -- BIOS_KTI_ERR_MISC valid flag Indicates (when set) that the BIOS_KTI_ERR_MISC register contains
additional information regard...
0x00000002 : cor_err_cnt (52:38) -- Corrected Error Count 15-bit counter which is incremented on every corrected error. MSB (bit
15) is a sticky overflow...
0x00000030 : mscod_code (21:16) -- MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based on the
standard MC ST overwr...
0x00000001 : mcacod_int (11:11) -- MCACOD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or Phy Control...
0x00000003 : mcacod_pp (10:09) -- MCACOD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors (MSCOD_Code=0x12
'UC LL or Phy Control...
0x00000003 : mcacod_ii (03:02) -- MSCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or ...
0x00000003 : mcacod_ll (01:00) -- MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or Phy C...

=> socket0.io0.uncore.upi0.upi_regs.ktierrcnt0_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes
to both the 15-bit co...

=> socket0.io0.uncore.upi0.upi_regs.ktierrcnt1_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes
to both the 15-bit co...

=> socket0.io0.uncore.upi0.upi_regs.ktierrcnt2_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes
to both the 15-bit co...

```

```
=> socket0.io0.uncore.upi.upi0.upi_regs.kticrcerrcnt
0x00000002 : counter (30:00) -- Counter increments every time the Link Layer detects a CRC error.

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css
0x00000007 : s_clm (18:16) -- lane map code determined at Rx and sent in TX.Config for remote Tx; also updated on L0p entry/exit.
Encoded as follows: 0b11...
0x0000000f : s_rx_state (15:08) -- Rx State (encoding defined in Intel UPI specification)
0x0000000f : s_tx_state (07:00) -- Tx State (encoding defined in Intel UPI specification)

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ldc
0x00000030 : indriftbufflatency (13:08) -- Initial Drift Buffer Latency      Drift Buffer Depth refers to the initialized difference
between read and writer ...
0x00000004 : driftbufalarmthr (05:00) -- Drift Buffer Alarm Threshold      When the difference (both underflow and overflow) between
drift buffer read and wri...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl
0x00000001 : c_rc_terms_off (29:29) -- 1: Rx pulldowns are off in ResetC
0x00000001 : c_faillver_en (27:27) -- 1: Enable failover
0x00000001 : c_ll_en (23:23) -- 1: enables L1
0x00000001 : c_init_begin (05:05) -- 0: blocks exit from ResetC
0x00000001 : c_op_speed (04:04) -- 1: operational speed 0: slow speed Note: The operational speed is determined by BIOS setup of the
PLL using BIOS-to-PCo...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_dis
0x00000c90 : s_rx_obs_latency (19:08) -- Rx sync counter value  when EIEOS of TS.Polling received at latency buffer.

-----
*****
Fields with non default value
*****
=====
| IP block | # fields with non-default value |
-----
| UPI_0 | 0x8 |
=====
```

4. Check BIOS_KTI_ERR_ST.MSCOD_CODE to verify successful LLR without PHY Reinit, 0x30. Check COR_ERR_CNT to show the correct number of CEs.

```
>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.bios_kti_err_misc.show()
0x00000040 : cor_err (63:56)
0x00000000 : rsvd (55:54)
0x00000000 : nerr (53:41)
0x00000000 : rx_lane_reversal (40:40)
0x00000000 : phy_state (39:37)
0x00000000 : illegal_flit_field (36:36)
0x00000000 : syndrome_or_l0cbadlanes (35:20)
0x000ffbe1 : flit_hdr (19:00)

>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.bios_kti_err_st.mscod_code.getspec()
#####
# FIELD: mscod_code
# Attribute: rW/v/p
# UpperBit: 21
# LowerBit: 16
# CurrentValue: 48
# Description:
# MSCOD Error Code
# Error Code from Intel UPI Agent. This field captures the error type based on the standard MC_ST overwrite rules.
# The Error Codes and handling are defined in the HAS. The summary per ID is:
# Uncorrectable (UC)
# 0x00 - UC Phy Initialization Failure (NumInit)
# 0x01 - UC Phy Detected Drift Buffer Alarm
# 0x02 - UC Phy Reserved
# 0x10 - UC LL Rx detected CRC error: unsuccessful LLR (entered Abort state)
# 0x11 - UC LL Rx Unsupported/Undefined packet
# 0x12 - UC LL or Phy Control Error
# 0x13 - UC LL Rx Parameter Exception
# 0x14 - UC LL TDX Failure
# 0x15 - UC LL SGX Failure
# 0x16 - UC LL Tx SDC Parity Error
# 0x17 - UC LL Rx SDC Parity Error
# 0x18 - UC LL FLE Failure
# 0x1F - UC LL Detected Control Error from UFI2UPI
# Correctable (COR)
# 0x20 - COR Phy Initialization Abort
# 0x21 - COR Phy Reset
# 0x22 - COR Phy Lane failure, recovery in x8 width
# 0x23 - COR Phy L0c error corrected without Phy reset
# 0x24 - COR Phy L0c error triggering Phy Reset
# 0x25 - COR Phy L0p exit error triggering Phy Reset
# 0x30 - COR LL Rx detected CRC error: successful LLR without Phy Reinit
# 0x31 - COR LL Rx detected CRC error: successful LLR with Phy Reinit

>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.bios_kti_err_st.show()
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000000 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : corr_err_sts_ind (54:53)
0x00000002 : cor_err_cnt (52:38)
0x00000000 : fw_upd (37:37)
0x00000000 : other_info (36:32)
0x00000000 : mscod_spere (31:22)
0x00000030 : mscod_code (21:16)
```

- Execute UPI CRC error injection on a different port.

August 2025
Doc. No.: 742874, Rev.: 1.5

```

Generic PP

mccad_ii = 0x3
SCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection
errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-
zero) - SGX/TDX/FLE failures - SDC Parity Error Set to 3 for all other errors.
Encoding is: 0x0 - Memory Access 0x1 - Reserved II 0x2 - I/O 0x3 - Other Transaction

mccad_ll = 0x3
MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-zero)
such that mccad[15:0]=0x0405. Set to 1 for SDC Parity Error Set to 2 for SGX/TDX/FLE
Failures Set to 3 for all other errors Encoding is: 0x0 - Memory Access 0x1 -
Reserved II 0x2 - I/O 0x3 - Other Transaction 0x0 - Level 0 0x1 - Level 1 0x2 - Level 2
0x3 - Generic LL

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.upi.upil.upi_regs.kti_mc_misc
0x00000040 : cor_err (63:56) -- Captures Correctable Errors, indicating all correctable error cases that have occurred since last
cleared. Only Correctable...
0x000ffbe1 : flit_hdr (19:00) -- HDR bits captured: [0] - HIB (if not set other bits >4 in this log are not valid) [1] - VN [2] -
VNA [3] - ACK [4] - CRD ...

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_cis
0x00000003 : s_link_up (16:15) -- 00: STxLinkUp FALSE, SRxLinkUp FALSE 01: STxLinkUp FALSE, SRxLinkUp TRUE 10: STxLinkUp TRUE,
SRxLinkUp FALSE 11: STxLinkUp...
0x00000001 : s_wake_detected (04:04) -- 1: Wake detected on one or more lanes.
0x00000001 : sp_detect (02:02) -- 1: remote Rx detected by Tx
0x00000001 : sp_rx_calib (01:01) -- 1: Rx calib done; sw clears by writing 1 prior to speed change to force re-calib.
0x00000001 : s_op_speed (00:00) -- 0: slow speed 1: op speed Updated at entry and exit from ResetC

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_cls
0x00000003 : s_rcvd_l0c (29:26) -- Received L0c code
0x00000003 : s_sent_l0c (25:22) -- Sent L0c code

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_pls
0x00000003 : s_rcvd_l0c_p (29:26) -- Same description as bit field in KTIREUT_PH_PLS register, but this is copy of prior to last phy
reset
0x00000003 : s_sent_l0c_p (25:22) -- Same description as bit field in KTIREUT_PH_PLS register, but this is copy of prior to last phy
reset.

=> socket0.io0.uncore.upi.upil.upi_regs.ktils
0x00000001 : rx_normal_op_state (13:13) -- Reflects whether the LL Rx is in Normal Operation (i.e.. has received at least one flit
from the Phy). Resets ...
0x00000003 : init_state (09:08) -- Reflects the current Initialization State of the link Layer, including any stall conditions
controlled by LL_LCL[3:2] ...

=> socket0.io0.uncore.upi.upil.upi_regs.ktiles
0x00000001 : worst_llr_numretry_count (03:00) -- Worst Case Link Layer Retry NUM_RETRY Highest NUM_RETRY count achieved since last
clear of this register ...

=> socket0.io0.uncore.upi.upil.upi_regs.ktilcl
0x00000003 : max_num_phy_reinit (13:12) -- Consecutive Phy ReInit's to RETRY_ABORT Note: this value is directly encoded which is
different then Intel UPI ...
0x0000000f : max_num_retry (11:08) -- Consecutive LLRs to Physical Layer ReInit Note: This value is directly encoded which is
different then Intel UPI def...

=> socket0.io0.uncore.upi.upil.upi_regs.bios_kti_err_st
0x00000001 : valid (63:63) -- Set by hardware when an uncorrectable or correctable error is logged.
0x00000001 : miscv (59:59) -- BIOS_KTI_ERR_MISC valid flag Indicates (when set) that the BIOS_KTI_ERR_MISC register contains
additional information regard...
0x00000002 : cor_err_cnt (52:38) -- Corrected Error Count 15-bit counter which is incremented on every corrected error. MSB (bit
15) is a sticky overflow...
0x00000030 : mscod_code (21:16) -- MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based on the
standard MC_ST overwr...
0x00000001 : mccad_int (11:11) -- MCCAD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL o...
0x00000003 : mccad_pp (10:09) -- MCCAD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors (MSCOD_Code=0x12
'UC LL or Phy Control...
0x00000003 : mccad_ii (03:02) -- MSCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or ...
0x00000003 : mccad_ll (01:00) -- MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or Phy C...

=> socket0.io0.uncore.upi.upil.upi_regs.ktierrcnt0_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes
to both the 15-bit co...

=> socket0.io0.uncore.upi.upil.upi_regs.ktierrcnt1_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes
to both the 15-bit co...

=> socket0.io0.uncore.upi.upil.upi_regs.ktierrcnt2_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes
to both the 15-bit co...

=> socket0.io0.uncore.upi.upil.upi_regs.kticrcerrcnt
0x00000002 : counter (30:00) -- Counter increments every time the Link Layer detects a CRC error.

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_css
0x00000007 : s_clm (18:16) -- lane map code determined at Rx and sent in TX.Config for remote Tx; also updated on L0p entry/exit.
Encoded as follows: 0b11...
0x0000000f : s_rx_state (15:08) -- Rx State (encoding defined in Intel UPI specification)
0x0000000f : s_tx_state (07:00) -- Tx State (encoding defined in Intel UPI specification)

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_ldc
0x00000030 : indrftbufflatency (13:08) -- Initial Drift Buffer Latency Drift Buffer Depth refers to the initialized difference
between read and writer ...
0x00000004 : drftbuffalarmthr (05:00) -- Drift Buffer Alarm Threshold When the difference (both underflow and overflow) between
drift buffer read and wri...

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_ctrl
0x00000001 : c_rc_terms_off (29:29) -- 1: RX pull downs are off in ResetC
0x00000001 : c_failover_en (27:27) -- 1: Enable failover
0x00000001 : c_ll_en (23:23) -- 1: enables Ll

```

```
0x00000001 : c_init_begin (05:05) -- 0: blocks exit from ResetC
0x00000001 : c_op_speed (04:04) -- 1: operational speed 0: slow speed Note: The operational speed is determined by BIOS setup of the
PLL using BIOS-to-PCo...
```

```
=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_dis
0x00000c40 : s_rx_obs_latency (19:08) -- Rx sync counter value when EIEOS of TS.Polling received at latency buffer.
```

```
*****
Fields with non default value
*****
```

```
=====
| IP block | # fields with non-default value |
-----
| UPI_1 | 0x8 |
=====
```

6. Decode the IA32_MCi_MISC register for every MCA that shows the status of the KTI link. Also, check KIT_ERR_ST register to ensure correct error count and mscod_code.

```
>>> sv.socket0.io0.uncore.upi.upil.upi_regs.bios_kti_err_misc.show()
0x00000040 : cor_err (63:56)
0x00000000 : rsvd (55:54)
0x00000000 : nerr (53:41)
0x00000000 : rx_lane_reversal (40:40)
0x00000000 : phy_state (39:37)
0x00000000 : illegal_flit_field (36:36)
0x00000000 : syndrome_or_l0cbadlanes (35:20)
0x000ffbe1 : flit_hdr (19:00)
```

```
>>> sv.socket0.io0.uncore.upi.upil.upi_regs.bios_kti_err_st.show()
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000000 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : corr_err_sts_ind (54:53)
0x00000002 : cor_err_cnt (52:38)
0x00000000 : fw_upd (37:37)
0x00000000 : other_info (36:32)
0x00000000 : mscod_spare (31:22)
0x00000030 : mscod_code (21:16)
0x00000000 : mcacod_rsvd (15:12)
0x00000001 : mcacod_int (11:11)
0x00000003 : mcacod_pp (10:09)
0x00000000 : mcacod_t (08:08)
0x00000000 : mcacod_rrrr (07:04)
0x00000003 : mcacod_ii (03:02)
0x00000003 : mcacod_ll (01:00)
```

```
>>> upi.printErrors
Errors | S0_P0 | S0_P1 | S0_P2 | S0_P3 | S0_P4 | S0_P5
| S1_P0 | S1_P1 |
=====
ktiph_rx10c_err_s... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
ktiph_rx10c_err_l... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
ktiph_rx10c_err_l... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
ktiph_l0pexit_err... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
ktierrdis0 (Generic) | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
ktierrdis1 (Generic) | 0x2* | 0x2* | 0x2* | 0x2* | 0x2* | 0x2*
0x2* | 0x2* |
ktireut_ph_pis (G... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
ktireut_ph_pls (G... | 0xcc00000* | 0xcc00000* | 0xcc00000* | 0xcc00000* | 0xcc00000*
0xcc00000* | 0xc02000* | 0xc02000* |
ktireut_ph_pss (G... | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
ktierrcnt0_cntr (... | 0x2* | 0x2* | 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
ktierrcnt1_cntr (... | 0x2* | 0x2* | 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
ktierrcnt2_cntr (... | 0x2* | 0x2* | 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
ktierrcnt0_mask (... | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff*
0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* |
ktierrcnt1_mask (... | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff*
0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* |
ktierrcnt2_mask (... | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff*
0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* |
kticerrcnt (Gen... | 0x2* | 0x0 | 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
ktiles (Generic) | 0x1* | 0x1* | 0x1* | 0x1* | 0x1* | 0x1*
0x1* | 0x1* |
ktiviral (Generic) | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10
| 0x10 | 0x10 |
```

```

bios_kti_err_st (...| 0x0 | 0x8800008000300e0f*| 0x0 | 0x0 | 0x0 | 0x0
| 0x0 | 0x0 |
>>> go ()

```

7. Observe the FTrace Output in the OS. Ensure MC5 is populated for both MBE injections. Both injections should have MS_COD 0x30, COR LL Rx detected CRC error: successful LLR without Phy Reinit.

```

kworker/0:3-1903 [000] ..... 582.674018: mce_record: CPU:0, MCG/s: f000c20/0, MC5: 8800008000300e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/4000000000000000, RIP: 00:<0000000000000000>, TSC: 1017410b3a1, PROCESSOR: 0:a06d0, TIME: 1679270730, SOCKET: 0, APIC: 0
kworker/0:3-1903 [000] ..... 821.800645: mce_record: CPU:0, MCG/s: f000c20/0, MC5: 8800008000300e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/4000000000000000, RIP: 00:<0000000000000000>, TSC: 154f79dfbfd, PROCESSOR: 0:a06d0, TIME: 1679271187, SOCKET: 0, APIC: 0

```

7.2.2 MBE and Successful LLR with PHY Reinit

Test Objective

Multi-lane transient fault (multiple-bit errors) with LLR requiring successful physical layer reinitialization and signaled CMCI.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45 BKC
OS Version	Red Hat Enterprise Linux 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	2 socket system
CScripts Version	GNR 2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Uncore Configuration > Uncore General Configuration >
 - UPI Dynamic Link Width Reduction Support = Enable

Recipe

1. Evaluate the Intel UPI lanes and pairs.

```

>>> upi.topology
2 sockets populated in the platform
*****
* Socket 0 (A1) UPI Config
*

```

5	Port 0	Port 1	Port 2	Port 3	Port 4	Port
Connected to	socket 1, port 1	socket 1, port 0	socket 1, port 3	socket 1, port 2	socket 1, port 5	socket 1, port 4
UPI LS Init Status Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 -
Current CRC Mode CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 -
Lane Reversal	No	No	No	No	No	
Retry State Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 -
Tx state	0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Tx Lane Status*	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000
Rx state	0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Rx Lane Status**	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000
L0p Disabled	Disabled	Disabled	Disabled	Disabled	Disabled	
L1 Disabled	Disabled	Disabled	Disabled	Disabled	Disabled	
PPS 0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	
Link Speed GT/s	20.0 GT/s	20.0 GT/s	20.0 GT/s	20.0 GT/s	20.0 GT/s	20.0

* Socket 1 (A1) UPI Config *						

5	Port 0	Port 1	Port 2	Port 3	Port 4	Port
Connected to	socket 0, port 1	socket 0, port 0	socket 0, port 3	socket 0, port 2	socket 0, port 5	socket 0, port 4
UPI LS Init Status Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 - Init Done	0x3 -
Current CRC Mode CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 - CRC16	0x0 -
Lane Reversal	No	No	No	No	No	
Retry State Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 - Retry Local Normal	0x0 -
Tx state	0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Tx Lane Status*	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000
Rx state	0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f - L0	0x0f -
UPI Rx Lane Status**	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000
L0p Disabled	Disabled	Disabled	Disabled	Disabled	Disabled	
L1 Disabled	Disabled	Disabled	Disabled	Disabled	Disabled	
PPS 0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	



```
Notes:
* txlanedetectstat - Tx Data Lane Detection Status. A bit value of 1 indicates the TX lane is dropped.
** rxdatareadystat - Rx Data Lane Ready Status. A bit value of 1 indicates Rx lane is dropped.
```

```

>>> upi.printErrors
Errors
| S1_P5 |
=====
ktiph_rxl0c_err_...| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktiph_rxl0c_err_1...| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktiph_rxl0c_err_1...| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktiph_10pexit_err...| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktierdis0 (Generic)| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktierdis1 (Generic)| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktireut_ph_pis (G...| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktireut_ph_pls (G...| 0xcc00000*| 0xcc00000*| 0xcc00000*| 0xcc00000*| 0xcc00000*| 0xcc00000*| 0xc02000*| 0xc02000*| 0xc02000*| 0xc02000*|
0xc02000*| 0xc02000*|
ktireut_ph_pss (G...| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktierrent0_cntr (...| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktierrent1_mask (...| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktierrent2_cntr (...| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktierrent0_mask (...| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*|
0x7ffffff*|
0x7ffffff*| 0x7ffffff*|
ktiercnt1_mask (...| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*|
0x7ffffff*|
0x7ffffff*| 0x7ffffff*|
ktiercnt2_mask (...| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*| 0x7ffffff*|
0x7ffffff*|
0x7ffffff*| 0x7ffffff*|
kticreercnt (Gen...| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |
ktiles (Generic) | 0x1*| 0x1*| 0x1*| 0x1*| 0x1*| 0x1*| 0x1*| 0x1*| 0x1*| 0x1*| 0x1*| 0x1*|
0x1*|
ktiviral (Generic) | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 | 0x10 |
| 0x10 |
bios_kti_err_st (...| 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 | 0x0 |
| 0x0 |

```

```
>>> ei.io.injectUIPError(socket=0, port=0, num_crcs=2)

=====
                UPI 0: REGISTERS AND FIELDS DESCRIPTIONS
=====
Register: socket0.io0.uncore.upi.upi0.upi_regs.kti_mc_st
        valid = 0x1
        Set by hardware when an uncorrectable or correctable error is logged.

miscv = 0x1
        KTI_MC_MISC valid flag Indicates (when set) the MCI_MISC register contains additional
        information regarding the error. See individual fields of MCI_MISC for details on which
        are valid for a given error.

cor_err_cnt = 0x2
        Corrected Error Count                15-bit counter which is incremented on every corrected
        error. MSB (bit 15) is a sticky overflow bit.

mscod_code = 0x30
        MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based
        on the standard MC_ST overwrite rules. The Error Codes and handling are defined in the
        HAS. The summary per ID is: Uncorrectable (AC) 0x00 - UC Phy Initialization Failure
        (Numinit) 0x01 - UC Phy Detected Drift Buffer Alarm 0x02 - UC Phy Detected Latency Buffer
        Rollover (obsolete for Gen3) 0x10 - UC LL Rx detected CRC error: unsuccessful LLR (entered
        Abort state) 0x11 - UC LL Rx Unsupported/Undefined packet 0x12 - UC LL or Phy Control
        Error 0x13 - UC LL Rx Parameter Exception 0x14 - UC LL TDX Failure 0x15 - UC LL SGX
        Failure 0x16 - UC LL Tx SDC Parity Error 0x17 - UC LL Rx SDC Parity Error 0x18 - UC LL FLE
        Failure 0x1F - UC LL Detected Control Error from UFI2UPI Correctable (COR) 0x20 - COR Phy
        Initialization Abort 0x21 - COR Phy Reset 0x22 - COR Phy Lane failure, recovery in x8
        width 0x23 - COR Phy L0c error corrected without Phy reset 0x24 - COR Phy L0c error
        triggering Phy Reset 0x25 - COR Phy L0p exit error triggering Phy Reset 0x30 - COR LL Rx
        detected CRC error: successful LLR without Phy Reinit 0x31 - COR LL Rx detected CRC error:
        successful LLR with Phy Reinit

mccad_int = 0x1
        MCCAD - Bus Interconnect Error Set to 0 for:
        - All Transmit Path Parity Protection
        errors (MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0_ctrl.txparerr=non-
        zero)
        - SGX/TDX/FLE failures
        - SDC Parity Error Set to 1 for all other errors.
        Encoding: 0 - Not Bus Interconnect Error 1 - Bus Interconnect Error

mccad_pp = 0x3
```


August 2025
Doc. No.: 742874, Rev.: 1.5

```
=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ldc
0x00000030 : inidriftbufflatency (13:08) -- Initial Drift Buffer Latency      Drift Buffer Depth refers to the initialized difference
between read and writer ...
0x00000004 : driftbufalarmthr (05:00) -- Drift Buffer Alarm Threshold      When the difference (both underflow and overflow) between
drift buffer read and wri...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl
0x00000001 : c_rc_terms_off (29:29) -- 1: Rx pulldowns are off in ResetC
0x00000001 : c_failover_en (27:27) -- 1: Enable failover
0x00000001 : c_ll_en (23:23) -- 1: enables Ll
0x00000001 : c_init_begin (05:05) -- 0: blocks exit from ResetC
0x00000001 : c_op_speed (04:04) -- 1: operational speed 0: slow speed Note: The operational speed is determined by BIOS setup of the
PLL using BIOS-to-PCo...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_dis
0x000005d0 : s_rx_obs_latency (19:08) -- Rx sync counter value  when EIEOS of TS.Polling received at latency buffer.

-----
*****
Fields with non default value
*****
=====
| IP block | # fields with non-default value |
-----
| MACHINE_CHECK | 0x2 |
| UPI_0 | 0x8 |
=====
```

4. Execute MBE UPI CRC error injection into a different port. Decode the IA32_MCI_MISC register for every MCA that shows the status of the KTI link. Also, check KIT_ERR_ST register to ensure correct error count and mscod_code.

```
>>> go()
>>> ei.io.injectUPIError(socket=0, port=1, num_crcs=2)

=====
UPI_1: REGISTERS AND FIELDS DESCRIPTIONS
=====
Register: socket0.io0.uncore.upi.upi1.upi_regs.kti_mc_st
valid = 0x1
Set by hardware when an uncorrectable or correctable error is logged.

miscv = 0x1
KTI_MC_MISC valid flag Indicates (when set) that the MCI_MISC register contains additional
information regarding the error. See individual fields of MCI_MISC for details on which
are valid for a given error.

cor_err_cnt = 0x2
Corrected Error Count      15-bit counter which is incremented on every corrected
error. MSB (bit 15) is a sticky overflow bit.

mscod_code = 0x30
MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based
on the standard MC ST overwrite rules. The Error Codes and handling are defined in the
HAS. The summary per ID is: Uncorrectable (UC) 0x00 - UC Phy Initialization Failure
(NumInit) 0x01 - UC Phy Detected Drift Buffer Alarm 0x02 - UC Phy Detected Latency Buffer
Rollover (obsolete for Gen3) 0x10 - UC LL Rx detected CRC error: unsuccessful LLR (entered
Abort state) 0x11 - UC LL Rx Unsupported/Undefined packet 0x12 - UC LL or Phy Control
Error 0x13 - UC LL Rx Parameter Exception 0x14 - UC LL TDX Failure 0x15 - UC LL SGX
Failure 0x16 - UC LL Tx SDC Parity Error 0x17 - UC LL Rx SDC Parity Error 0x18 - UC LL FLE
Failure 0x1F - UC LL Detected Control Error from UFI2UPI Correctable (COR) 0x20 - COR Phy
Initialization Abort 0x21 - COR Phy Reset 0x22 - COR Phy Lane failure, recovery in x8
width 0x23 - COR Phy L0c error corrected without Phy reset 0x24 - COR Phy L0c error
triggering Phy Reset 0x25 - COR Phy L0p exit error triggering Phy Reset 0x30 - COR LL Rx
detected CRC error: successful LLR without Phy Reinit 0x31 - COR LL Rx detected CRC error:
successful LLR with Phy Reinit

mccad_int = 0x1
MCCAD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection
errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-
zero) - SGX/TDX/FLE failures - SDC Parity Error Set to 1 for all other errors.
Encoding: 0 - Not Bus Interconnect Error 1 - Bus Interconnect Error

mccad_pp = 0x3
MCCAD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-zero)
such that mccad[15:0]=0x0405. Set to 2 for all 'UC LL Rx Unsupported/Undefined Packet'
errors (MSCOD_Code=0x11). Set to 2 for all SGX/TDX/FLE failures. Set to 2 for all SDC
Parity Error Failures. Set to 3 for all other errors. Encoding is: 0x0 - Local Processor
Originated 0x1 - Local Processor Responded 0x2 - Local Processor Observed Error 0x3 -
Generic PP

mccad_ii = 0x3
SCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection
errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-
zero) - SGX/TDX/FLE failures - SDC Parity Error Set to 3 for all other errors.
Encoding is: 0x0 - Memory Access 0x1 - Reserved II 0x2 - I/O 0x3 - Other Transaction

mccad_ll = 0x3
MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-zero)
such that mccad[15:0]=0x0405. Set to 1 for SDC Parity Error Set to 2 for SGX/TDX/FLE
Failures Set to 3 for all other errors Encoding is: 0x0 - Memory Access 0x1 -
Reserved II 0x2 - I/O 0x3 - Other Transaction 0x0 - Level 0 0x1 - Level 1 0x2 - Level 2
0x3 - Generic LL

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.upi.upi1.upi_regs.kti_mc_misc
```

```

0x00000040 : cor_err (63:56) -- Captures Correctable Errors, indicating all correctable error cases that have occurred since last
cleared. Only Correctab...
0x000ffbe1 : flit_hdr (19:00) -- HDR bits captured: [0] - HIB (if not set other bits >4 in this log are not valid) [1] - VN [2] -
VNA [3] - ACK [4] - CRD ...

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_cis
0x00000003 : s_link_up (16:15) -- 00: STxLinkUp FALSE, SRxLinkUp FALSE 01: STxLinkUp FALSE, SRxLinkUp TRUE 10: STxLinkUp TRUE,
SRxLinkUp FALSE 11: STxLinkUp...
0x00000001 : s_wake_detected (04:04) -- 1: Wake detected on one or more lanes.
0x00000001 : sp_detect (02:02) -- 1: remote Rx detected by Tx
0x00000001 : sp_rx_calib (01:01) -- 1: Rx calib done; sw clears by writing 1 prior to speed change to force re-calib.
0x00000001 : s_op_speed (00:00) -- 0: slow speed 1: op speed Updated at entry and exit from ResetC

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_cls
0x00000003 : s_rcvd_l0c (29:26) -- Received L0c code
0x00000003 : s_sent_l0c (25:22) -- Sent L0c code

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_pls
0x00000003 : s_rcvd_l0c_p (29:26) -- Same description as bit field in KTIREUT_PH_PLS register, but this is copy of prior to last phy
reset
0x00000003 : s_sent_l0c_p (25:22) -- Same description as bit field in KTIREUT_PH_PLS register, but this is copy of prior to last phy
reset.

=> socket0.io0.uncore.upi.upil.upi_regs.ktils
0x00000001 : rx_normal_op_state (13:13) -- Reflects whether the LL Rx is in Normal Operation (i.e.. has received at least one flit
from the Phy). Resets ...
0x00000003 : init_state (09:08) -- Reflects the current Initialization State of the link Layer, including any stall conditions
controlled by LL LCL[3:2]. ...
0x00000001 : link_layer_retry_queue_consumed (07:00) -- Link Layer Retry Queue Consumed Number of Retry Queue entries currently
consumed while waiting fo...

=> socket0.io0.uncore.upi.upil.upi_regs.ktiles
0x00000001 : worst_llr_numretry_count (03:00) -- Worst Case Link Layer Retry NUM_RETRY Highest NUM_RETRY count achieved since last
clear of this register ...

=> socket0.io0.uncore.upi.upil.upi_regs.ktilcl
0x00000003 : max_num_phy_reinit (13:12) -- Consecutive Phy ReInit's to RETRY_ABORT Note: this value is directly encoded which is
different then Intel UPI ...
0x0000000f : max_num_retry (11:08) -- Consecutive LLRs to Physical Layer ReInit Note: This value is directly encoded which is
different then Intel UPI def...

=> socket0.io0.uncore.upi.upil.upi_regs.bios_kti_err_st
0x00000001 : valid (63:63) -- Set by hardware when an uncorrectable or correctable error is logged.
0x00000001 : miscv (59:59) -- BIOS_KTI_ERR_MISC valid flag Indicates (when set) that the BIOS_KTI_ERR_MISC register contains
additional information regard...
0x00000002 : cor_err_cnt (52:38) -- Corrected Error Count 15-bit counter which is incremented on every corrected error. MSB (bit
15) is a sticky overflow...
0x00000030 : mscod_code (21:16) -- MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based on the
standard MC ST overwr...
0x00000001 : mcacod_int (11:11) -- MCACOD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL o...
0x00000003 : mcacod_pp (10:09) -- MCACOD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors (MSCOD_Code=0x12
'UC LL or Phy Control...
0x00000003 : mcacod_ii (03:02) -- MSCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or ...
0x00000003 : mcacod_ll (01:00) -- MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors
(MSCOD_Code=0x12 'UC LL or Phy C...

=> socket0.io0.uncore.upi.upil.upi_regs.ktierrcnt0_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes
to both the 15-bit co...

=> socket0.io0.uncore.upi.upil.upi_regs.ktierrcnt1_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes
to both the 15-bit co...

=> socket0.io0.uncore.upi.upil.upi_regs.ktierrcnt2_cntr
0x00000002 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writing zeroes
to both the 15-bit co...

=> socket0.io0.uncore.upi.upil.upi_regs.kticrcrrcnt
0x00000002 : counter (30:00) -- Counter increments every time the Link Layer detects a CRC error.

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_css
0x00000007 : s_clm (18:16) -- lane map code determined at Rx and sent in TX.Config for remote Tx; also updated on L0p entry/exit.
Encoded as follows: 0b11...
0x0000000f : s_rx_state (15:08) -- Rx State (encoding defined in Intel UPI specification)
0x0000000f : s_tx_state (07:00) -- Tx State (encoding defined in Intel UPI specification)

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_ldc
0x00000030 : inidriftbufflatency (13:08) -- Initial Drift Buffer Latency Drift Buffer Depth refers to the initialized difference
between read and writ...
0x00000004 : driftbufalarmthr (05:00) -- Drift Buffer Alarm Threshold When the difference (both underflow and overflow) between
drift buffer read and wri...

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_ctrl
0x00000001 : c_rc_terms_off (29:29) -- 1: Rx pull downs are off in ResetC
0x00000001 : c_failover_en (27:27) -- 1: Enable failover
0x00000001 : c_ll_en (23:23) -- 1: enables LL
0x00000001 : c_init_begin (05:05) -- 0: blocks exit from ResetC
0x00000001 : c_op_speed (04:04) -- 1: operational speed 0: slow speed Note: The operational speed is determined by BIOS setup of the
PLL using BIOS-to-PCo...

=> socket0.io0.uncore.upi.upil.upi_regs.ktireut_ph_dis
0x000008d8 : s_rx_obs_latency (19:08) -- Rx sync counter value when EIEOS of TS.Polling received at latency buffer.

-----
*****
Fields with non default value
*****
=====
| IP block | # fields with non-default value |

```

```
-----
| UPI_1 |                0x8                |
|-----|

```

5. Inject a PHY re-INIT, and then check the KTIRXEINJCTL10 register.

```
>>> go()
>>> sv.socket0.io0.uncore.upi.upis.upi_regs.ktirxeinjctl0
socket0.io0.uncore.upi.upi0.upi_regs.ktirxeinjctl0 - 0xcaca0020
socket0.io0.uncore.upi.upi1.upi_regs.ktirxeinjctl0 - 0xcaca0020
>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.ktirxeinjctl0=0xcacb2143
>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.ktirxeinjctl0.show()
0x0000000c : once_stp_sel (31:28)
0x0000000a : once_str_sel (27:24)
0x0000000c : openloop_stp_sel (23:20)
0x0000000b : openloop_str_sel (19:16)
0x00000001 : einjnumphyreinit (15:13)
0x00000001 : einjnumretry (12:08)
0x00000001 : einjretryactivemode (07:06)
0x00000001 : errinjst (05:05)
0x00000001 : suppress_st (04:04)
0x00000000 : crcdis (03:03)
0x00000000 : maten (02:02)
0x00000001 : superr_hold_on_respfcn (01:01)
0x00000001 : superr_en (00:00)
```

6. Observe the FTrace Output in the OS. Ensure MC5 is populated for both MBE injections. Both injections should have MS_COD 0x30, COR LL Rx detected CRC error: successful LLR without Phy Reinit. The PHY re-init injection will trigger a MC for the S0P0 and S1P1 connection. S0P0 will show MS_COD 0x31, COR LL Rx detected CRC error: successful LLR with Phy Reinit.

```
kworker/0:0-7 [000] ..... 543.805570: mce_record: CPU: 0, MCGc/s: f000c20/0,
MC5: 8800008000300e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/40000020000ffbe1/0000000000000000, RIP: 00:<0000000000000000>, TSC:
fd63bb7d07, PROCESSOR: 0:a06d0, TIME: 1679270909, SOCKET: 0, APIC: 0
kworker/0:0-7 [000] ..... 794.419933: mce_record: CPU: 0, MCGc/s:
f000c20/0, MC5: 8800008400300e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/40
000000000ffbe1/0000000000000000, RIP: 00:<0000000000000000>, TSC: 154ea77aa65,
PROCESSOR: 0:a06d0, TIME: 1679271159, SOCKET: 0, APIC: 0
kworker/0:0-7 [000] ..... 837.004996: mce_record: CPU: 0, MCGc/s:
f000c20/0, MC5: 8800008000310e0f, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/c2
000020000ffbe1/0000000000000000, RIP: 00:<0000000000000000>, TSC: 163c9e3ca93,
PROCESSOR: 0:a06d0, TIME: 1679271202, SOCKET: 0, APIC: 0
```

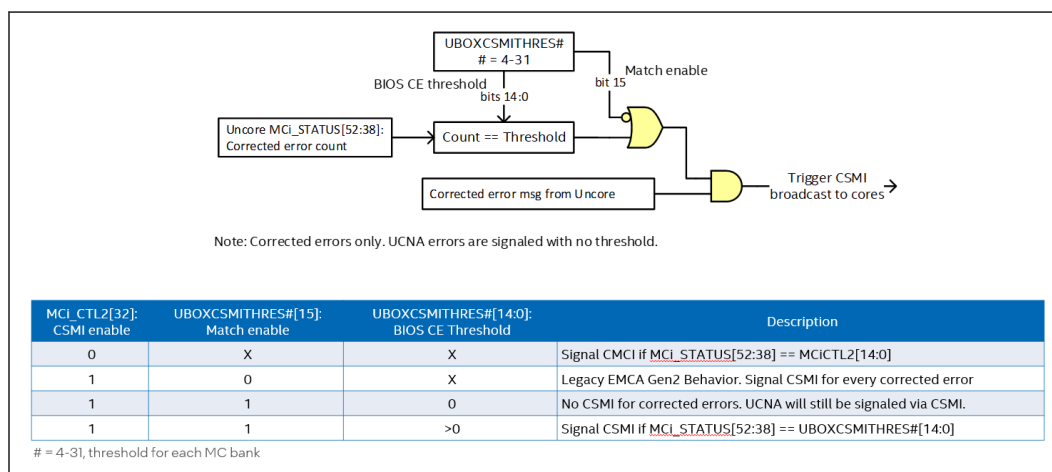
8.0 CPU RAS

8.1 Uncore Machine Check Bank CSMI Thresholding

Feature Description

The processor supports thresholding of CSMIs for each uncore machine check bank. The following diagram depicts the flow, controls, and operating modes for this feature.

Figure 2. CSMI Thresholding Flow



8.1.1 IMC Channel MC Bank

Test Objective

The objective of this test is to test CSMI thresholding for an IMC channel's machine check bank. CSMI thresholding is enabled with the threshold set to "3". The expectation is to only see CSMI signaling after three correctable ECC errors have been injected and consumed. Finally, one UCNA type error is injected to verify that the CSMI is not thresholded.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45'23 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	No particular DIMM, PCIe card, or hardware tool configuration required.
CScripts Version	BHS.2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Platform Configuration > System Event Log > eMCA Settings >
 - EMCA CMCI-SMI Threshold = 0x3
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Halt and confirm there are no preexisting memory related errors on the system. Clear any found error logs or cold boot the system.

```
>>> halt()
>>> error.check_mem_errors()
*****
No errors found
*****
```

2. CSMI thresholding is supported for all uncore MC banks and modifying the "EMCA CMCI-SMI Threshold" BIOS knob configures CSMI thresholding for all uncore banks. Check that all the UBOXCSMITHRES registers are configured the same.

```
>>> sv.socket0.io0.uncore.ubox.ncevents.showsearch("uboxcsmithres")
uboxcsmithres4 = 0x8003
uboxcsmithres5 = 0x8003
uboxcsmithres6 = 0x8003
uboxcsmithres7 = 0x8003
uboxcsmithres8 = 0x8003
uboxcsmithres9 = 0x8003
uboxcsmithres10 = 0x8003
uboxcsmithres11 = 0x8003
uboxcsmithres12 = 0x8003
uboxcsmithres13 = 0x8003
uboxcsmithres14 = 0x8003
uboxcsmithres15 = 0x8003
uboxcsmithres16 = 0x8003
uboxcsmithres17 = 0x8003
uboxcsmithres18 = 0x8003
uboxcsmithres19 = 0x8003
```

```
uboxcsmithres20 = 0x8003
uboxcsmithres21 = 0x8003
uboxcsmithres22 = 0x8003
uboxcsmithres23 = 0x8003
uboxcsmithres24 = 0x8003
uboxcsmithres25 = 0x8003
uboxcsmithres26 = 0x8003
uboxcsmithres27 = 0x8003
uboxcsmithres28 = 0x8003
uboxcsmithres29 = 0x8003
uboxcsmithres30 = 0x8003
uboxcsmithres31 = 0x8003
```

3. This test is focused on testing CSMI thresholding for the IMC MC banks because injecting memory errors is the simplest test case. Check that the UBOXCSMITHRES register for the target MC bank has match_en equal to "1" and bios_ce_threshold is equal to the programmed value.

```
>>> sv.socket0.io0.uncore.ubox.ncevents.uboxcsmithres13.show()
0x00000000 : reserved (31:16) -- reserved
0x00000001 : match_en (15:15) -- csmi enable
0x00000003 : bios_ce_threshold (14:00) -- threshold value for csmi
```

4. Check if the injectors are locked for all sockets and unlock if locked.

```
>>> ei.utils.checkInjectorLock(and_unlock=True)
```

5. Choose the target device for error injection and modify masks for a single device correctable ECC error injection.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x2222222222222222, dev1=2, dev1msk=0x0)
```

6. Inject and consume the correctable ECC error.

```
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0)
```

7. Throughout this test, keep an eye on the serial log to see when BIOS enters SMM and completes its memory error handling flow. From CScripts, the SMI_COUNT MSR (0x34) will also be checked at different times to confirm CSMI thresholding is working as intended.

```
>>> itp.threads[0].msr(0x34)
[64b] 0x000000000000009c
```

8. Read the target channel's MC bank MSRs or shadow registers to confirm injection was successful and the error was logged.

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc_status_shadow.showfields()
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000001 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : cor_err_stat (54:53)
0x00000001 : cor_err_cnt (52:38)
0x00000000 : fw_upd (37:37)
0x00000002 : other_info (36:32)
0x00000080 : mscod (31:16)
0x00000090 : mcacod (15:00)
```

9. Resume the system. Check the serial log to confirm it is unchanged from before injection and that the memory or MCA error handler were never ran because the system never entered SMM.

```
>>> go()
```

10. Halt the system and check that the SMI_COUNT is unchanged from the last time it was checked.

```
>>> halt()
>>> itp.threads[0].msr(0x34)
[64b] 0x000000000000009C
```

11. Repeat steps 6 through 10 two more times. After each injection note that the MCI_STATUS cor_err_cnt will increment. After the final injection, cor_err_cnt will be "3" and resuming the system will result in the following BIOS serial log indicating that BIOS entered and completed its SMM flow.

```
S0 C0 T0 [CpuRas]MC status 0x8C0000C200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C0000C200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0xB009409
0/13-1 RetryRdErrLogAddr1: 0x44000001
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C0000C200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C0000C200800090, class CORRECTED
TranslateSad: McTargetId is 0x0
Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh
is 0x0Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 [CpuRas]MC status 0x8C0000C200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AF0B010
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
LogErrorToElog: --Logging Error to EMCA
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T1 [CpuRas]Successfully notify OS via CMCI
S0 C133 T1 [CpuRas]Successfully notify OS via CMCI
...
S0 C1 T1 [CpuRas]Successfully notify OS via CMCI
S0 C14 T0 [CpuRas]Successfully notify OS via CMCI
```

12. Halt the system and check that the SMI_COUNT has incremented by 1.

```
>>> halt()
>>> itp.threads[0].msr(0x34)
[64b] 0x000000000000009D
```


13. The following is the ftrace output:

```
kworker/0:2-1749 [000] ..... 1050.876604: mce_record: CPU: 0, MCGc/s: f000c20/0,
MC13: 8c0000c200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000010008200/0400080008008086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
19fd014cd53, PROCESSOR: 0:a06d0, TIME: 1679271434, SOCKET: 0, APIC: 0

kworker/0:2-1749 [000] ..... 1050.876730: extlog_mem_event: {1} corrected error:
single-bit ECC physical addr: 0000000010008200 (mask lsb: ff) node:0 card:0 module:0 rank:0
bank:0 device:1 row:256 column:256 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0

kworker/0:2-1749 [000] ..... 1050.877923: mc_event: 3 Corrected errors: memory read
error on CPU SrcID#0 MC#0 Chan#0 DIMM#0 (mc:0 location:0:0:-1 address:0x10008200 grain:32
syndrome:0x00000000 err_code:0x0080:0x0090 SystemAddress:0x10008200 ProcessorSocketId:0x0
MemoryControllerId:0x0 ChannelAddress:0x4002000 ChannelId:0x0 RankAddress:0x1000800
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x100 Column:0x100 Bank:0x0 BankGroup:0x0
ChipSelect:0x0)
```

14. A future revision of this document will have the remaining steps to verify that UCNA errors are signaled without being thresholded. The desired method of generating the UCNA (patrol scrub consuming a UCECC error) is not working with the current CScripts version.

8.1.2 IMC Channel MC Bank - CSMI Threshold at Zero

Test Objective

The objective of this test is to test CSMI thresholding with thresholding enabled but threshold at "0" for an IMC channel's machine check bank. A couple correctable errors are injected to confirm that CEs are never signaled and one UCNA error is injected to confirm that UCNAs are still signaled via CSMI. CSMI thresholding is enabled with the threshold set to "3", but then the threshold is set to "0"; the reason for this is that the UBOXCSMITHRES registers should have the following bit-fields configured as such: match_en == 1 and bios_ce_threshold == 0.

The previous recipe uses the same BIOS knob configuration, so an attempt was made to minimize knob changes. Although, both recipes can be tested with only the standard testing knobs modified ([BIOS Standard System Configuration](#)) if the UBOXCSMITHRES registers are modified directly via ITP/other.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45'23 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	No particular DIMM, PCIe card, or hardware tool configuration required.
CScripts Version	BHS.2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Platform Configuration > System Event Log > eMCA Settings >
 - EMCA CMCI-SMI Threshold = 0x3
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Halt and confirm there are no preexisting memory related errors on the system. Clear any found error logs or cold boot the system.

```
>>> halt()
>>> error.check_mem_errors()
*****
No errors found
*****
```

2. This test is focused on testing CSMI thresholding for the IMC MC banks because injecting memory errors is the simplest test case. Check that the UBOXCSMITHRES register for the target MC bank has match_en equal to "1", and bios_ce_threshold is equal to the programmed value.

```
>>> sv.socket0.io0.uncore.ubox.ncevents.uboxcsmithres13.show()
0x00000000 : reserved (31:16) -- reserved
0x00000001 : match_en (15:15) -- csmi enable
0x00000003 : bios_ce_threshold (14:00) -- threshold value for csmi
```

3. Modify bios_ce_threshold to be "0" and confirm the write was successful. Access method was modified from the default (tap2sb) because tap2sb writes were unsuccessful.

```
>>> sv.socket0.io0.uncore.ubox.ncevents.setaccess("pcicfg")
>>> sv.socket0.io0.uncore.ubox.ncevents.uboxcsmithres13
0x8003

>>> sv.socket0.io0.uncore.ubox.ncevents.uboxcsmithres13.bios_ce_threshold=0
0x0

>>> sv.socket0.io0.uncore.ubox.ncevents.uboxcsmithres13
0x8000

>>> sv.socket0.io0.uncore.ubox.ncevents.uboxcsmithres13.show()
0x00000000 : reserved (31:16) -- reserved
0x00000001 : match_en (15:15) -- csmi enable
0x00000000 : bios_ce_threshold (14:00) -- threshold value for csmi

>>> sv.socket0.io0.uncore.ubox.ncevents.setaccess("tap2sb")
```

4. Check if the injectors are locked for all sockets and unlock if locked.

```
>>> ei.utils.checkInjectorLock(and_unlock=True)
```

5. Choose the target device for error injection and modify masks for a single device correctable ECC error injection.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x2222222222222222, dev1=2, dev1msk=0x0)
```

6. Inject and consume the correctable ECC error.

```
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0)
```

7. Throughout this test, keep an eye on the serial log to see when BIOS enters SMM and completes its memory error handling flow. From CScripts, the SMI_COUNT MSR (0x34) will also be checked at different times to confirm CSMI thresholding is working as intended.

```
>>> itp.threads[0].msr(0x34)
[64b] 0x0000000000000091
```

8. Check that the target's IMC machine check bank has a correctable read error logged.

```
>>> error.check_mem_errors(socket_id=0)
...
Uncore MCA from Socket: 0
=====
| Valid |      MC Bank Name      |      Status      |      Addr      |
|-----|-----|-----|-----|
|  *   | BANK #13 - MC CH0 merged | 0x8c00004200800090 | 0x00000000008004100 |
|-----|-----|-----|-----|

=====
|      Misc      |      Ctl      |      Ctl2      |
|-----|-----|-----|
| 0x0400080008008086 | 0x00003cc0ffffe5ff | 0x0000000000000000 |
|-----|-----|-----|
...

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc_status_shadow.show()
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000001 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : cor_err_stat (54:53)
0x00000001 : cor_err_cnt (52:38)
0x00000000 : fw upd (37:37)
0x00000002 : other_info (36:32)
0x00000080 : mscod (31:16)
0x00000090 : mcacod (15:00)
```

9. Resume the system. Check the serial log to confirm it is unchanged from before injection and that the memory or MCA error handler were never ran because the system never entered SMM.

```
>>> go()
```

10. Halt the system and check that the SMI_COUNT is unchanged from the last time it was checked.

```
>>> halt()
>>> itp.threads[0].msr(0x34)
[64b] 0x0000000000000091
```

11. Repeat steps 6 through 10 a couple times to confirm that none of the correctable errors are signaled and SMI_COUNT stays unchanged.
12. A future revision of this document will have the remaining steps to verify that UCNA errors are still signaled via CSMI. The desired method of generating the UCNA (patrol scrub consuming a UCECC error) is not working with the current CScripts version.

8.2 MCA Bank Error Control (Cloaking)

Feature Description

Cloaking refers to the ability to mask signaling of corrected and/or UCNA errors to OS/SW. Intel's reference BIOS implements cloaking by doing the following: modifying UBOXCSMITHRES registers so match_en==1 and bios_ce_threshold==0, modifying the CLOAKING MSR (0x52) depending on whether CE and/or UCNA cloaking was enabled, and no longer logging WHEA/elog.

8.2.1 Memory, UPI, and PCIe Correctable Errors

Test Objective

The objective of this test is to test error signaling after injecting correctable memory, UPI, and PCIe errors. The expected outcome is to see no signaling for memory and UPI CEs but PCIe CEs will still be signaled because MCA error logging is not involved; the PCIe error will not be logged to WHEA.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45'23 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	2 socket system and a generic PCIe* card
CScripts Version	BHS.2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)

- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Platform Configuration > System Event Log >
 - Corrected Error Cloaking = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Halt and check that there are no existing machine check errors. Clear any found error logs or cold boot the system.

```
>>> error.check_machine_check_errors()
*****
                                     No errors found
*****
```

2. Check that UBOXCSMITHRES registers were modified by the BIOS as expected. match_en==1 and bios_ce_threshold==0.

```
>>> sv.socket0.io0.uncore.ubox.ncevents.showsearch("uboxcsmithres")
uboxcsmithres4 = 0x8000
uboxcsmithres5 = 0x8000
uboxcsmithres6 = 0x8000
uboxcsmithres7 = 0x8000
uboxcsmithres8 = 0x8000
uboxcsmithres9 = 0x8000
uboxcsmithres10 = 0x8000
uboxcsmithres11 = 0x8000
uboxcsmithres12 = 0x8000
uboxcsmithres13 = 0x8000
uboxcsmithres14 = 0x8000
uboxcsmithres15 = 0x8000
uboxcsmithres16 = 0x8000
uboxcsmithres17 = 0x8000
uboxcsmithres18 = 0x8000
uboxcsmithres19 = 0x8000
uboxcsmithres20 = 0x8000
uboxcsmithres21 = 0x8000
uboxcsmithres22 = 0x8000
uboxcsmithres23 = 0x8000
uboxcsmithres24 = 0x8000
uboxcsmithres25 = 0x8000
uboxcsmithres26 = 0x8000
uboxcsmithres27 = 0x8000
uboxcsmithres28 = 0x8000
uboxcsmithres29 = 0x8000
uboxcsmithres30 = 0x8000
uboxcsmithres31 = 0x8000

>>> sv.socket0.io0.uncore.ubox.ncevents.uboxcsmithres4.show()
0x00000000 : reserved (31:16) -- reserved
0x00000001 : match_en (15:15) -- csmi enable
0x00000000 : bios_ce_threshold (14:00) -- threshold value for csmi
```

- Check that BIOS configured the CLOAKING MSR (0x52) as expected. This MSR is only readable in SMM mode. Bit 0 is set if CE cloaking is enabled and bit 1 is set if UCNA cloaking is enabled. Then exit SMM and halt.

```
>>> ei.utils.enter_smm()
>>> itp.threads[0].msr(0x52)
[64b] 0x0000000000000001

>>> ei.utils.exit_smm()
>>> halt()
```

- Check if the injectors are locked for all sockets and unlock if locked.

```
>>> ei.utils.checkInjectorLock(and_unlock=True)
```

- Check SMI_COUNT before testing for future reference.

```
>>> itp.threads[0].msr(0x34)
[64b] 0x000000000000009D
```

- Inject an Intel UPI CRC error.

```
>>> ei.iio.injectUPIError(socket=0, port=0, num_crcs=2, check_error_inj_lock=False, force_unlock_inj=False)
...
=====
Uncore MCA from Socket: 0
=====
| MC Bank Name | Status | Addr | Misc | Ctl1 | Ctl2 |
=====
| BANK #5 - UPI Merged | 0x8800008000300e0f | 0x0000000000000000 | 0x400000e0000ffbe1 | 0x00000000081ff0007 | 0x0000000000000000 |
=====

Bank name: UPI, Error = COR LL Rx detected CRC error: successful LLR without Phy Reinit
+ MCS_STATUS 0x8800008000300e0f

Range Field Description Value Value Description
-----
63:63 VALID Indicates that the information within this register is valid 0x1 Valid
...
61:61 UC A '1' indicates that an uncorrectable the error condition was detected. 0x0 No UC
...
59:59 MISCV If this flag is '1', the IA32_MCI_MISC register content is valid and can be read. 0x1 IA32_MCI_MISC Valid
...
31:16 MSCOD Model Specific Error Code 0x30 COR LL Rx detected CRC error: successful LLR without Phy Reinit
15:0 MCACOD Machine Check Architecture Error Code 0xe0f ---
```

- Resume the system to allow all error handling flows to complete. Check that the BIOS serial log does not indicate any SMM flow (no MCA error handler for example).

```
>>> go()
```

- Halt and check SMI_COUNT. Unchanged from before, the BIOS serial log has not printed anything, and ftrace has not printed any traces.

```
>>> >> halt()
>> itp.threads[0].msr(0x34)
[64b] 0x000000000000009D
```

- Clear the UPI MCI_STATUS MSR so there are no valid machine check errors.

```
>>> itp.threads[0].msr(0x415, 0x0)
>>> itp.threads[0].msr(0x415)
[64b] 0x0000000000000000
```

- Choose the target device for error injection and modify masks for a single device correctable ECC error injection.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x2222222222222222, dev1=2, dev1msk=0x0)
```

- Inject and consume the correctable ECC error.

```
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False, force_unlock_inj=False)
```

12. Check the target's IMC MC bank to verify that the correctable read error injection was successful. `mscod==0x80`. You can also use `error.check_machine_check_errors()`, or `error.check_mem_errors()` at this step as well.

```
>>> itp.threads[0].msr(0x435)
[64b] 0x8C00004200800090
```

13. Resume the system to allow all error handling flows to complete. Check that the BIOS serial log does not indicate any SMM flow (no MCA error handler for example).

```
>>> go()
```

14. Halt and check `SMI_COUNT`. Unchanged from before, BIOS serial log has not printed anything, and `ftrace` has not printed any traces.

```
>>> halt()
>>> itp.threads[0].msr(0x34)
[64b] 0x000000000000009D
```

15. Use the following commands to choose the root port to inject a PCIe error to. In this test, PCIe port 0e (pxp0.rp2) is chosen.

```
>>> pcie.topology()
=====
| PCIe port | config as | width | speed | slot/down | ASPM en | secbusno | subbusno | linkstate | Vendor | Vendor ID | Device ID | RevID | ASPM en | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0a | | x8 | | | | | | | | | | | | |
| 0e | | x2 | | x2 | Gen3 | slot | L1 | 15h | 15h | UpL1.1 | Intel Corporation | 8086h | 09adh | 03h | L1 |
| 0f | | x2 | | x1 | Gen1 | slot | L1 | 16h | 16h | UpL1.1 | Intel Corporation | 8086h | 1533h | 03h | L1 |
| 0g | | x2 | | x1 | Gen2 | slot | no | 17h | 18h | Up.L0 | ASPEED Technology, Inc. | 1a03h | 1150h | 06h | no |
| 0h | | x2 | | x1 | Gen2 | slot | L1 | 19h | 19h | UpL1.1 | Renesas Technology Corp. | 1912h | 0014h | 03h | L1 |
| 1a | | x16 | | x4 | Gen4 | slot | no | 2Ah | 2Ah | Up.L0 | Intel Corporation | 8086h | 0d4bh | 04h | no |
| 2a | | x16 | | | | | | | | | | | | |
| 3a | | x4 | | | | | | | | | | | | |
| 3c | | x4 | | | | | | | | | | | | |
| 3e | | x4 | | | | | | | | | | | | |
| 3g | | x4 | | | | | | | | | | | | |
| 4a | | x4 | | | | | | | | | | | | |
| 4c | | x4 | | | | | | | | | | | | |
| 4e | | x4 | | | | | | | | | | | | |
| 4g | | x4 | | | | | | | | | | | | |
| 5a | | x4 | | | | | | | | | | | | |
| 5c | | x4 | | | | | | | | | | | | |
| 5e | | x4 | | | | | | | | | | | | |
| 5g | | x4 | | | | | | | | | | | | |
=====

>>> pcie.port_map()
=====
| PCIe Port | GNR | GNR Nomenclature |
|-----|-----|-----|
| # | CScriptsII | CScriptsII |
| ... | ... | ... |
| 4 | 0e | pxp0.rp2 |
| ... | ... | ... |
=====
```

16. Clear the following status registers if not already cleared and unmask receiver error.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.errcorsts=-1
>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.errcormsk.rem=0
>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts=-1
>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.errcorsts
0x0
```

```
>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.errcormsk
0x0
>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts
0x0
```

17. Inject a correctable PCIe error to a populated port. This will inject a receiver error. A PCIe error is injected to show that cloaking only impacts MCA errors and does not cloak PCIe errors.

```
>>> ei.iio.injectPcieError(socket=0, port="pxp0.rp2", errType="ce", checkEInjLock=False,
showErrorRegs=True, force_unlock_inj=False)
...
Injecting receiver error (correctable)
Correctable error detected in socket0.io0.uncore.pi5.pxp0.rp2.cfg.errcorsts.re bit
```

18. Resume the system to allow all error handling flows to complete. Note that the BIOS serial log will have some details about BIOS' IEH error handling flow. Note that the error is NOT logged to WHEA.

```
>>> go()
```

19. Halt the system and check that the SMI_COUNT has incrementally increased from its previous value. The expectation is to see the count increment by 1, but it is alright if that is not the case, because the SMI count alone is not what is of interest and is just another data point to corroborate with the BIOS serial log and OS traces. What is of interest is whether or not the BIOS entered its IEH error handler and signaled the error to the OS. Based on the SMI count and logs shown at the bottom of this recipe it can be seen that this occurred.

```
>>> halt()
>>> itp.threads[0].msr(0x34)
[64b] 0x0000000000000009E
```

20. BIOS serial log:

```
[IEH] error found at IEH(S:0x0 B:0x7E D:0x2 F:0x0) Sev: IEH CORRECT ERROR
[IEH] ErrorStatus 0x20, MaxBitIdx 0x1D
IEH CORRECT ERROR
[IEH] BitIdx 0x5, ShareIdx 0x0
[IEH] error device is (S:0x0 B:0x14 D:0x0 F:0x4) BitIdx 0x5, ShareIdx 0x0 [IEH] error found at
IEH(S:0x0 B:0x14 D:0x0 F:0x4) Sev: IEH CORRECT ERROR
[IEH] ErrorStatus 0x10, MaxBitIdx 0x11
IEH CORRECT ERROR
[IEH] BitIdx 0x4, ShareIdx 0x0
[IEH] error device is (S:0x0 B:0x14 D:0x6 F:0x0) BitIdx 0x4, ShareIdx 0x0 [Device Error]
error on skt:0x0 Bus:0x14 Device:0x6 func:0x0
PcieRootPortErrorHandler MailBox->PcieInitPar.SerrEmuTestEn = 0x0
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AF0B610
WHEA: Detected PCIe Error
--Logging Corrected Error to WHEA
WHEA: Sending OS notification via SCI. Success
```

21. FTrace output:

```
kworker/0:3-2957 [000] ..... 883.994495: aer_event: 0000:14:06.0 PCIe Bus Error:
severity=Corrected, Receiver Error, TLP Header=Not available
```


8.2.2 UCNA Errors

Test Objective

The objective of this test is to test the machine check bank's error controls for UnCorrected No Action required (UCNA) errors, with the anticipated result being the absence of UCNA error signaling to the OS.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores A step
BIOS	WW06'24 BKC
OS Version	CentOS Stream 8
Kernel	5.19.0-gnr.po.bkc.7.2.21.x86_64
HW Configuration	No particular DIMM, PCIe card, or hardware tool configuration required.
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Platform Configuration > System Event Log >
 - UCNA Cloaking = Enable

Recipe

1. Halt and check that there are no existing memory errors. Clear any found error logs or cold boot the system.

```
>>> error.check_mem_errors()
*****
                        No errors found
*****
```

2. Note that unlike corrected error machine check bank controls, the UBOXCSMITHRES registers are not used to implement UCNA machine check bank error controls.

```
>>> sv.socket0.io0.uncore.ubox.ncevents.showsearch("uboxcsmithres")
uboxcsmithres4 = 0x0
uboxcsmithres5 = 0x0
uboxcsmithres6 = 0x0
```

```
uboxcsmithres7 = 0x0
uboxcsmithres8 = 0x0
uboxcsmithres9 = 0x0
uboxcsmithres10 = 0x0
uboxcsmithres11 = 0x0
uboxcsmithres12 = 0x0
uboxcsmithres13 = 0x0
uboxcsmithres14 = 0x0
uboxcsmithres15 = 0x0
uboxcsmithres16 = 0x0
uboxcsmithres17 = 0x0
uboxcsmithres18 = 0x0
uboxcsmithres19 = 0x0
uboxcsmithres20 = 0x0
uboxcsmithres21 = 0x0
uboxcsmithres22 = 0x0
uboxcsmithres23 = 0x0
uboxcsmithres24 = 0x0
uboxcsmithres25 = 0x0
uboxcsmithres26 = 0x0
uboxcsmithres27 = 0x0
uboxcsmithres28 = 0x0
uboxcsmithres29 = 0x0
uboxcsmithres30 = 0x0
uboxcsmithres31 = 0x0
```

3. Check that BIOS configured the CLOAKING MSR (0x52) as expected. This MSR is only readable in SMM mode. Bit 0 is set if CE cloaking is enabled and bit 1 is set if UCNA cloaking is enabled then exit SMM and halt.

```
>>> ei.utils.enter_smm()
>>> itp.threads[0].msr(0x52)
[64b] 0x0000000000000002

>>> ei.utils.exit_smm()
>>> halt()
```

4. Check if the injectors are locked for all sockets and unlock if locked.

```
>>> ei.utils.checkInjectorLock(and_unlock=True)
```

5. Inject an uncorrectable ECC error and have the patrol scrubber consume the error.

```
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0, bank=0, errType="uce", verbose=True, sub_rank=0, ps_ch=0, PatrolConsume=True, showErrorRegs=True, checkEInjLock=True)
```

6. Read the target IMC's MCI_STATUS MSR to confirm that a uncorrectable patrol scrub error (MSCOD==0x10) was logged as a UCNA MCA error.

```
>>> itp.threads[0].msr(0x435)
[64b] 0xAC000002001000C0
```

7. Resume the system and observe the serial log and ftrace window. The ftrace window should be empty (no events) and the serial log (shown next) indicates that there was an SMI due to the UCNA error, and the BIOS did not notify the OS.

```
[ProcessMcBanks] McBankBitFields 2000
S0 C26 T1 [CpuRas]MC status 0xAC000002001000C0, class UCNA
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C26 T1 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C26 T1 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C26 T1 [CpuRas]MC status 0xAC000002001000C0, class UCNA
S0 C26 T1 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C26 T1 [CpuRas]MC status 0xAC000002001000C0, class UCNA
S0 C26 T1 Entering CheckNearMemoryError.
```

```

S0 C26 T1 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x1 Node is 0x0 NodeCh
is 0x0Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C26 T1 SetPoisonRecord
S0 C26 T1 SetPoisonRecord: 0
S0 C26 T1 SetPoisonRecord: pPoisonRec->PhysicalAddress : 0x10004100
S0 C26 T1 [CpuRas]MC status 0xAC000002001000C0, class UCNA
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7E1C0018
LogErrorToElog: --Logging Error to EMCA
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x0
[Mca] Clear Status
[Mca]ClearErrLogReg

```

8.3 First and Last Corrected Error Mode of Reporting

Feature Description

First and last corrected error mode of reporting refers to the controls for overwriting corrected error logs. There are controls to choose whether to latch the first or last corrected error in the retry read error logs, IMC MC bank, B2CMI MC bank, and the UPI MC bank in the event of a burst of corrected errors. See the processors' architecture specification for more details.

8.3.1 Retry Read Error Logs and IMC Machine Check Bank

Test Objective

The objective of this test is to test the first and last corrected error mode of reporting for retry read error logs and IMC MC banks. Multiple CECC errors are injected and control bits are modified to display the two corrected error overwrite configurations.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45'23 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	No particular DIMM, PCIe card, or hardware tool configuration required.
CScripts Version	BHS.2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)

- [FTrace and DMSG for Machine Check Exception \(MCE\) Observability](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Halt and confirm there are no preexisting memory related errors on the system. Clear any found error logs or cold boot the system.

```
>>> halt()
>>> error.check_mem_errors()
*****
                        No errors found
*****
```

2. Of the four sets of Retry Read Error Logs (RREL), this test will focus on sets one and three because they are configured to log ECC error information on demand reads (en_patspr==0) and are configured to latch the first and last corrected error (see noover bit-field), respectively. Check the retry_rd_err_log registers for the target channel.

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log.en_patspr
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[0].en_patspr - 0x00000001
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[1].en_patspr - 0x00000000
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[2].en_patspr - 0x00000001
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[3].en_patspr - 0x00000000

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log.noover
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[0].noover - 0x00000001
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[1].noover - 0x00000001
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[2].noover - 0x00000000
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[3].noover - 0x00000000
```

3. Check to confirm that the control bit to configure the target channel's IMC MC bank to first corrected error mode is set.

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.mc0_dp_chkn_bit.no_over_ce
0x1
```

4. Inject a CECC error to the target channel.

```
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True)
```

5. Check that the RREL logs are valid and logged information aligning with the CECC error injection. Not shown is retry_rd_err_log_address1 which contains the DIMM address information such as failed device, chip select, bank, and so on. All of which are useful for checking that injection was successful and the RREL logs are logging information as expected. The exact system address logged is not too important for this test as it is only being checked to more easily observe the logs being overwritten or not.

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[1].showfields()
...
0x00000001 : system_addr_valid (24:24)
...
```

```

0x00000001 : noover (15:15)
0x00000000 : en_patspr (14:14)
0x00000000 : patspr (13:13)
0x00000001 : en (12:12)
...
0x00000001 : corr_correctable (10:10)
...
0x00000000 : over (02:02)
0x00000000 : uc (01:01)
0x00000001 : v (00:00)

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[3].showfields()
...
0x00000001 : system_addr_valid (24:24)
...
0x00000000 : noover (15:15)
0x00000000 : en_patspr (14:14)
0x00000000 : patspr (13:13)
0x00000001 : en (12:12)
...
0x00000001 : corr_correctable (10:10)
...
0x00000000 : over (02:02)
0x00000000 : uc (01:01)
0x00000001 : v (00:00)

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_address3[1]
0x10008200

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_address3[3]
0x10008200

```

6. Check that the IMC MC bank logged a valid corrected read ECC error and the address aligns with the RREL logs and the injection target (MCI_MISC contains the DIMM address location, not shown). All bit-fields in MCI_STATUS should match.

```

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc_status_shadow.showfields()
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000001 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : cor_err_stat (54:53)
0x00000001 : cor_err_cnt (52:38)
0x00000000 : fw_upd (37:37)
0x00000002 : other_info (36:32)
0x00000080 : mscod (31:16)
0x00000090 : mcacod (15:00)

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc8_addr_shadow
0x10008200

```

7. Inject a CECC error to the same channel but a different location. The bank parameter was changed from "0" to "1" in this injection.

```

>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=1, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True)

```

8. Check the same RREL logs in the previous step. Note that set 1 is unchanged and set 2 has over==1 and address3 contains the 2nd error injection's system address.

```

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[1].showfields()\n
...
0x00000001 : system_addr_valid (24:24)
...

```

```

0x00000001 : noover (15:15)
0x00000000 : en_patspr (14:14)
0x00000000 : patspr (13:13)
0x00000001 : en (12:12)
...
0x00000001 : corr_correctable (10:10)
...
0x00000000 : over (02:02)
0x00000000 : uc (01:01)
0x00000001 : v (00:00)

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[3].showfields()\n
...
0x00000001 : system_addr_valid (24:24)
...
0x00000000 : noover (15:15)
0x00000000 : en_patspr (14:14)
0x00000000 : patspr (13:13)
0x00000001 : en (12:12)
...
0x00000001 : corr_correctable (10:10)
...
0x00000001 : over (02:02)
0x00000000 : uc (01:01)
0x00000001 : v (00:00)

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_address3[1]
0x10008200

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_address3[3]
0x10408300
    
```

9. Note that the IMC MC bank is almost unchanged (cor_err_cnt incremented by 1) and contains information about the 1st corrected error.

```

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc_status_shadow.showfields()
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000001 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : cor_err_stat (54:53)
0x00000002 : cor_err_cnt (52:38)
0x00000000 : fw_upd (37:37)
0x00000002 : other_info (36:32)
0x00000080 : mscod (31:16)
0x00000090 : mcacod (15:00)

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc8_addr_shadow
0x10008200
    
```

10. So far the RREL logs capability to control corrected error overwrite rules has been shown but not the IMC MC bank's. Clear the following bit to configure the target IMC MC bank into last corrected error mode. Note that the access method for accessing this register needed to be modified because writing was unsuccessful with the default access method on the system used.

```

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.setaccess("mem")
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.mc0_dp_chkn_bit.no_over_ce = 0
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.setaccess("tap2sb")
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.mc0_dp_chkn_bit.no_over_ce
0x0
    
```

11. Inject a CECC error to a 3rd different location on the same channel; the bank parameter was modified from 1 to 2.

```
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=2, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True)
```

12. Check that the IMC MC bank now reflects corrected error information from the last corrected error. Because DIMM addresses were used in the injectMemError command, it will be easier to correlate the logged error information with the error injection command by checking the DIMM address information logged in MCI_MISC (not shown).

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc_status_shadow.showfields()
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000001 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : cor_err_stat (54:53)
0x00000003 : cor_err_cnt (52:38)
0x00000000 : fw_upd (37:37)
0x00000002 : other_info (36:32)
0x00000080 : mscod (31:16)
0x00000090 : mcacod (15:00)

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc8_addr_shadow
0x10008600
```

13. The following is the ftrace output after the system was un-halted:

```
kworker/0:2-1749 [000] ..... 1050.876604: mce_record: CPU: 0, MCGc/s: f000c20/0,
MC13: 8c0000c200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000010008200/0400080008008086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
19fd014cd53, PROCESSOR: 0:a06d0, TIME: 1679271434, SOCKET: 0, APIC: 0

kworker/0:2-1749 [000] ..... 1050.876730: extlog_mem_event: {1} corrected error:
single-bit ECC physical addr: 0000000010008200 (mask lsb: ff) node:0 card:0 module:0 rank:0
bank:0 device:1 row:256 column:256 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0

kworker/0:2-1749 [000] ..... 1050.877923: mc_event: 3 Corrected errors: memory read
error on CPU SrcID#0 MC#0 Chan#0 DIMM#0 (mc:0 location:0:0:-1 address:0x10008200 grain:32
syndrome:0x00000000 err_code:0x0080:0x0090 SystemAddress:0x10008200 ProcessorSocketId:0x0
MemoryControllerId:0x0 ChannelAddress:0x4002000 ChannelId:0x0 RankAddress:0x1000800
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x100 Column:0x100 Bank:0x0 BankGroup:0x0
ChipSelect:0x0)
```

14. BIOS serial log after the system was un-halted:

```
S0 C0 T0 [CpuRas]MC status 0x8C0000C200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C0000C200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0xB009409
0/13-1 RetryRdErrLogAddr1: 0x44000001
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0xB00140D
0/13-3 RetryRdErrLogAddr1: 0x44080001
0/13-3 RetryRdErrLogAddr2: 0xF0000100
[Mca]CheckMcBankErrorStatus returns TRUE
```

```

[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C0000C200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C0000C200800090, class CORRECTED
TranslateSad: McTargetId is 0x0
Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh
is 0x0Rank:0x0 SubRank:0x0 Bank:0x2 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 [CpuRas]MC status 0x8C0000C200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AF0B010
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
LogErrorToElog: --Logging Error to EMCA
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T1 [CpuRas]Successfully notify OS via CMCI
S0 C0 T0 [CpuRas]Successfully notify OS via CMCI
...
S0 C133 T1 [CpuRas]Successfully notify OS via CMCI
S0 C13 T1 [CpuRas]Successfully notify OS via CMCI

```

8.4 MCA Recovery

Software layer assisted recovery from uncorrectable data errors. Enables software layers (OS, VMM, DBMS, Application) to assist in the system recovery from errors that are not correctable at the hardware level and marked as 'corrupted data' by the CPU.

If the processor identifies an error that cannot be corrected by hardware, the processor marks the data with the corrupted data flag and the error event is handed off to the firmware and/or the operating system. If the firmware/operating system has a redundant copy of the data, it may be able to correct the error. If the error occurs in the application space, the operating system may be able to signal the application using a SIGBUS event and allow further recovery or terminate the application and keep the operating system running. If the error occurs in the Kernel space, the operating system will trigger a 'Kernel panic'. The MCA recovery feature for the execution path is to recover the SRAR type of errors detected on the data load or instruction fetch units on the Windows or Linux environments. Primary objective is to validate end-to-end flow from the time a hardware error is injected to the time an application recovers successfully.

NOTE

Multiple SRAO UCEs may cause nested MCE interrupts and become IERR. Therefore, the patrol scrub SRAO will now be downgraded to UCNA or another correctable error in logging or signaling behavior and signal the CMCI.

8.4.1 SRAR Data Poison Consumption

Test Objective

Inject an SRAR type of error to the DCU.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW48'23 BKC
OS Version	CentOS* Stream 9
Kernel	6.6.0-gnr.bkc.6.6.0.44.3.6.x86_64
HW Configuration	N/A
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)
- [FTrace and DMESG](#)
- <http://git.kernel.org/pub/scm/linux/kernel/git/aegl/ras-tools.git>
- [Kernel Tools and Scripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >

- Patrol Scrub = Disable

NOTE

With patrol scrub enabled, the tool after injecting the error will be dormant for a few seconds before triggering the error. During this, the patrol scrub will scrub the error and take the page offline before the error could be triggered. This will cause a patrol scrub error which could be seen in the MSCOD, and the tool returns no MCE. Also, since there is no poison consumption, it implies there is no SRAR.

Recipe

1. Initialize EINJ.

```
# modprobe einj
```

2. Inject a Data poison consumption error.

```
# ./einj_mem_uc -f 'single'
0: single vaddr = 0x7f56c5041400 paddr = 1310fd400
injecting ...
triggering ...
```

```
signal 7 code 4 addr 0x7f56c5041000
page not present
Saw local machine check
Test passed
```

3. Observe the following in your FTrace output:
 - UE ECC UCNA from one of the MCCHAN bank, for example, MCACOD 0x90
 - Check that the ADDR matches the einj_mem_uc addr value.
 - eLog event due to the UE ECC UCNA
 - Linux Page Offline event
 - EDAC event for the UE ECC UCNA
 - Data SRAR Poison Consumption, for example, MCACOD 134
 - For Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, this would be Bank 1.
 - For Intel® Xeon® 6900/6700-Series Processors with E-Cores, this would be Bank 3.
 - Check that the ADDR matches the einj_mem_uc addr value.
 - EDAC event for the Data poison Consumption
 - System should stay up.
4. Navigate to the original terminal opened earlier for the FTracing/log collection. Evaluate the OS logs. Ensure UCE logged in iMC. In the following example, UCE is logged in MC16 (iMC3_CH0). The address in the log collection matches the address in the einj_mem_uc tool. Since this is a DCU error, expect to see MC1 bank populated as shown in the logs.

```
# modprobe acpi_extlog
# cd /sys/kernel/debug/tracing/
# echo 1 > events/mce/mce_record/enable
# echo 1 > events/ras/enable
# cat trace_pipe

kworker/0:3-1109 [000] ..... 866.971430: mce_record: CPU: 0,
MCGc/s: f000c20/0, MC16: ac00000200a00090, IPID: 0000000000000000, ADDR/MISC/
SYND: 00000001310fd400/040000e058b9d086/0000000000000000, RIP:
00:<0000000000000000>, TSC: 14b5f032543, PROCESSOR: 0:a06d0, TIME:
1701998524, SOCKET: 0, APIC: 0
kworker/0:3-1109 [000] ..... 866.985822: memory_failure_event: pfn
0x1310fd: recovery action for dirty LRU page: Recovered
kworker/0:3-1109 [000] ..... 867.041770: extlog_mem_event: {1}
recoverable error: single-symbol chipkill ECC physical addr: 00000001310fd400
(mask lsb: ff) node:0 card:3 module:0 rank:0 bank:7 device:0 row:2839
column:928 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnD, DIMM0
kworker/0:3-1109 [000] ..... 867.042719: mc_event: 1 Fatal error:
memory read error on CPU_SrcID#0 MC#3_Chan#0_DIMM#0 (mc:3 location:0:0:-1
address:0x1310fd400 grain:32 syndrome:0x00000000 err_code:0x00a0:0x0090
SystemAddress:0x1310fd400 ProcessorSocketId:0x0 MemoryControllerId:0x3
ChannelAddress:0x2c43f500 ChannelId:0x0 RankAddress:0xb10fd00
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0xb17 Column:0x3a0
Bank:0x3 BankGroup:0x1 ChipSelect:0x0)
kworker/0:3-1109 [000] ..... 867.042744: mce_record: CPU: 92,
MCGc/s: f000c20/f, MC1: bd80000000100134, IPID: 0000000000000000, ADDR/MISC/
SYND: 00000001310fd400/0000000000000086/0000000000000000, RIP:
33:<0000000000405018>, TSC: 14b69d7c7d5, PROCESSOR: 0:a06d0, TIME:
1701998524, SOCKET: 0, APIC: 91
kworker/0:3-1109 [000] ..... 867.042749: extlog_mem_event: {2}
recoverable error: multi-bit ECC physical addr: 00000001310fd400 (mask lsb:
```

```
ff) node:0 card:3 module:0 rank:0 bank:7 device:0 row:2839 column:0 DIMM
location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnD, DIMM0
```

5. Decode the FTrace log using the `error.manualMcaDecode()` command and ensure the FTrace matches information that was decoded. For SRAR DCU, uncorrectable error for demand or underfill reads for MSCOD should be seen. Additionally, MSCOD for the DCU bank (MC bank 1) will show poison load.

```
>>> error.manualMcaDecode()
Manual Machine Check Bank Decoder
Enter MCI_STATUS, MCI_ADDR, MCI_MISC values
Enter 't' for a short demo
Enter 'Q' or 'q' to exit
>>> MCHN
>>> ac00000200a00090

>>> error.manualMcaDecode()
Manual Machine Check Bank Decoder
Enter MCI_STATUS, MCI_ADDR, MCI_MISC values
Enter 't' for a short demo
Enter 'Q' or 'q' to exit
>>> 1
>>> bd80000000100134
```

Figure 3. MCA Decode for MC Bank 1 (DCU)

Bank name: Core DCU, Error = UNCORR_LOAD_POISON_0010_0134					
+ MCI_STATUS 0x808000000100134					
Range	Field	Description	Value	Value Description	
63:63	VALID	Indicates that the information within this register is valid	0x1	Valid	
62:62	OVERFLOW	A '1' indicates that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (the VALID bit was already set)	0x0	Not Overflow	
61:61	UC	A '1' indicates that an uncorrectable the error condition was detected.	0x1	UC Detected	
60:60	EN	Error reporting enabled. A '1' indicates that error reporting is enabled by the associated flag bit(s) in the IA32_MCI_CTL register.	0x1	Enabled	
59:59	MISCV	If this flag is '1', the IA32_MCI_MISC register content is valid and can be read.	0x1	IA32_MCI_MISC Valid	
58:58	ADDRV	If this flag is '1', the IA32_MCI_ADDR register content is valid and can be read.	0x1	IA32_MCI_ADDR Valid	
57:57	PCC	Processor Context Corrupt: A '1' indicates that reliable restarting of the processor may not be possible and a reset is recommended.	0x0	Processor Context Not Corrupted	
56:56	S	Signaling an uncorrectable recoverable error.	0x1	---	
55:55	AR	Recovery action required for UCR error.	0x1	---	
54:53	THRESHOLD_BASED_STATUS	Threshold-based error status	0x0	---	
52:38	CORRECTED_ERROR_COUNT	Count of correctable errors in the DCU including sticky overflow indicator (bit 52)	0x0	---	
37:37	FW	FW Update Error Status Indicator	0x0	---	
36:32	OTHER_INFO	Not part of Machine-Check Architecture	0x0	---	
31:16	MSCOD	Model Specific Error Code	0x18	LOAD_POISON_0010_0134	
15:0	MCACOD	Machine Check Architecture Error Code	0x134	DCU data - Load Poison	

Figure 4. MCA Decode for MC Bank 16 (MCCHAN3)

Bank name: MCHW, Error = UNCORR Uncorrectable Error for Demand or Underfill Reads					
* MCCHN_STATUS 0xAC00000200A00000					
Range	Field	Description	Value	Value Description	
63:63	VALID	Indicates that the information within this register is valid	0x1	Valid	
62:62	OVERFLOW	A '1' indicates that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (the VALID bit was already set)	0x0	Not Overflow	
61:61	UC	A '1' indicates that an uncorrectable the error condition was detected.	0x1	UC Detected	
60:60	EN	Error reporting enabled. A '1' indicates that error reporting is enabled by the associated flag bit(s) in the IA32_MCI_CTL register.	0x0	Not Enabled	
59:59	MISCV	If this flag is '1', the IA32_MCI_MISC register content is valid and can be read.	0x1	IA32_MCI_MISC Valid	
58:58	ADDRV	If this flag is '1', the IA32_MCI_ADDR register content is valid and can be read.	0x1	IA32_MCI_ADDR Valid	
57:57	PCC	Processor Context Corrupt: A '1' indicates that reliable restarting of the processor may not be possible and a reset is recommended.	0x0	Processor Context Not Corrupted	
56:56	S	Signaling an uncorrectable recoverable error.	0x0	---	
55:55	AR	Recovery action required for UCR error.	0x0	---	
54:53	THRESHOLD_BASED_STATUS	Threshold-based error status	0x0	---	
52:38	CORRECTED_ERROR_COUNT	Count of correctable errors in the DCU including sticky overflow indicator (bit 52)	0x0	---	
37:37	PW	PW Update Error Status Indicator	0x0	---	
36:32	OTHER_INFO	Not part of Machine-Check Architecture	0x2	---	
31:16	MISCOD	Model Specific Error Code	0xA0	Uncorrectable Error for Demand or Underfill Reads	
15:0	MCACOD	Machine Check Architecture Error Code	0x00	MCCHAR_ERROR(1LP MODE) MEMORY_READ_ERROR CHANNEL_0	

8.4.2 SRAR Instruction Poison Consumption

Test Objective

Inject an SRAR type of error to the IFU.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW48'23 BKC
OS Version	CentOS* Stream 9
Kernel	6.6.0-gnr.bkc.6.6.0.44.3.6.x86_64
HW Configuration	N/A
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)

- [FTrace and DMESG](#)
- <http://git.kernel.org/pub/scm/linux/kernel/git/aegl/ras-tools.git>
- RAS Tools Scripts

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >

- Patrol Scrub = Disable.

NOTE

With patrol scrub enabled, there is a high chance the CPU will detect the UE ECC error before a CPU consumption occurs. The resulting OS behavior would do a page offline based off of the Patrol Scrub UCNA before the OS could even consume the error.

Recipe

1. Initialize EINJ.

```
# modprobe einj
```

2. Inject a instruction poison consumption error.

```
# ./einj_mem_uc -f 'instr'
0: instr vaddr = 0x404000 paddr = 1562ed000
injecting ...
triggering ...
signal 7 code 128 addr (nil)
Unexpected SIGBUS
page not present
Saw local machine check
Test passed
```

3. Observe the following in your ftrace output:
 - UE ECC UCNA from one of the MCCHAN bank, for example, MCACOD 0x90
 - Check that the ADDR matches the einj_mem_uc addr value.
 - eLog event due to the UE ECC UCNA
 - Linux Page Offline event
 - EDAC event for the UE ECC UCNA
 - Instruction SRAR Poison Consumption, for example, MCACOD 150
 - For Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores this would be Bank 0
 - For Intel® Xeon® 6900/6700-Series Processors with E-Cores this would be Bank 2
 - Check that the ADDR matches the einj_mem_uc addr value.
 - EDAC event for the Instruction Poison Consumption
 - System should stay up.

4. Navigate to the original terminal opened earlier for the FTracing/log collection. Evaluate the OS logs. Ensure UCE logged in the iMC. In the following example, UCE is logged in MC16 (iMC3_CH0). The address in the log collection matches the address in the einj_mem_uc tool. Since this is an IFU error, expect to see the MC0 bank populated as shown in the logs.

```
# modprobe acpi_extlog
# cd /sys/kernel/debug/tracing/
# echo 1 > events/mce/mce_record/enable
# echo 1 > events/ras/enable
# cat trace_pipe

kworker/0:2-887 [000] ..... 147.387899: mce_record: CPU: 0,
MCGC/s: f000c20/0, MC16: ac00000200a00090, IPID: 0000000000000000, ADDR/MISC/
SYND: 00000001562ed000/040003206b38d086/0000000000000000, RIP:
00:<0000000000000000>, TSC: 782b77974b, PROCESSOR: 0:a06d0, TIME: 1701999874,
SOCKET: 0, APIC: 0
kworker/0:2-887 [000] ..... 147.412394: memory_failure_event: pfn
0x1562ed: recovery action for clean LRU page: Recovered
kworker/0:2-887 [000] ..... 147.430672: extlog_mem_event: {1}
recoverable error: single-symbol chipkill ECC physical addr: 00000001562ed000
(mask lsb: ff) node:0 card:3 module:0 rank:0 bank:25 device:0 row:3431
column:416 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnD, DIMM0
kworker/0:2-887 [000] .N... 147.436299: mc_event: 1 Fatal error:
memory read error on CPU SrcID#0 MC#3 Chan#0 DIMM#0 (mc:3 location:0:0:-1
address:0x1562ed000 grain:32 syndrome:0x00000000 err_code:0x00a0:0x0090
SystemAddress:0x1562ed000 ProcessorSocketId:0x0 MemoryControllerId:0x3
ChannelAddress:0x358bb400 ChannelId:0x0 RankAddress:0xd62ed00
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0xd67 Column:0x1a0
Bank:0x1 BankGroup:0x6 ChipSelect:0x0)
kworker/0:2-887 [000] .N... 147.436323: mce_record: CPU: 91,
MCGC/s: f000c20/f, MC0: bd800000000c0150, IPID: 0000000000000000, ADDR/MISC/
SYND: 00000001562ed000/0000000000000086/0000000000000000, RIP:
33:<0000000000403fff>, TSC: 782b77ec35, PROCESSOR: 0:a06d0, TIME: 1701999874,
SOCKET: 0, APIC: 8f
kworker/0:2-887 [000] .N... 147.436329: extlog_mem_event: {2}
recoverable error: multi-bit ECC physical addr: 00000001562ed000 (mask lsb:
ff) node:0 card:3 module:0 rank:0 bank:25 device:0 row:3431 column:0 DIMM
location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnD, DIMM0
```

5. Decode the FTrace log using the `error.manualMcaDecode()` command and ensure the FTrace matches information that was decoded. For SRAR IFU, uncorrectable error for demand or underfill reads for MSCOD should be seen. Additionally, MSCOD for the IFU bank (MC bank 0) will show poison load.

```
>>> error.manualMcaDecode()
Manual Machine Check Bank Decoder
Enter MCI_STATUS, MCI_ADDR, MCI_MISC values
Enter 't' for a short demo
Enter 'Q' or 'q' to exit
>>> MCHN
>>> ac00000200a00090

>>> error.manualMcaDecode()
Manual Machine Check Bank Decoder
Enter MCI_STATUS, MCI_ADDR, MCI_MISC values
Enter 't' for a short demo
Enter 'Q' or 'q' to exit
>>> 0
>>> bd800000000c0150
```

Figure 5. MCA Decode for MC Bank 16 (MCCHAN3)

Bank name: MCHN, Error = UNCORR Uncorrectable Error for Demand or Underfill Reads					
+ MCHNR_STATUS 0xAC90000200A00000					
Range	Field	Description	Value	Value Description	
63:63	VALID	Indicates that the information within this register is valid	0x1	Valid	
62:62	OVERFLOW	A '1' indicates that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (the VALID bit was already set)	0x0	Not Overflow	
61:61	UC	A '1' indicates that an uncorrectable error condition was detected.	0x1	UC Detected	
60:60	EN	Error reporting enabled. A '1' indicates that error reporting is enabled by the associated flag bit(s) in the IA32_MCI_CTL register.	0x0	Not Enabled	
59:59	MISCV	If this flag is '1', the IA32_MCI_MISC register content is valid and can be read.	0x1	IA32_MCI_MISC Valid	
58:58	ADDRV	If this flag is '1', the IA32_MCI_ADDR register content is valid and can be read.	0x1	IA32_MCI_ADDR Valid	
57:57	PCC	Processor Context Corrupt: A '1' indicates that reliable restarting of the processor may not be possible and a reset is recommended.	0x0	Processor Context Not Corrupted	
56:56	S	Signaling an uncorrectable recoverable error.	0x0	---	
55:55	AR	Recovery action required for UCR error.	0x0	---	
54:53	THRESHOLD_BASED_STATUS	Threshold-based error status	0x0	---	
52:38	CORRECTED_ERROR_COUNT	Count of correctable errors in the DCU including sticky overflow indicator (bit 52)	0x0	---	
37:37	PW	PW Update Error Status Indicator	0x0	---	
36:32	OTHER_INFO	Not part of Machine-Check Architecture	0x2	---	
31:16	MISCOD	Model Specific Error Code	0xA0	Uncorrectable Error for Demand or Underfill Reads	
15:0	MCACOD	Machine Check Architecture Error Code	0x90	MCCHAR_ERROR(1LR MODE) MEMORY_READ_ERROR CHANNEL_0	

Figure 6. MCA Decode for MC Bank 0 (IFU)

Bank name: Core IFU, Error = UNCORR IFU_POISON_ERROR_000C_0150					
+ MCB_STATUS 0x50B00000000C0150					
Range	Field	Description	Value	Value Description	
63:63	VALID	Indicates that the information within this register is valid	0x1	Valid	
62:62	OVERFLOW	A '1' indicates that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (the VALID bit was already set)	0x0	Not Overflow	
61:61	UC	A '1' indicates that an uncorrectable error condition was detected.	0x1	UC Detected	
60:60	EN	Error reporting enabled. A '1' indicates that error reporting is enabled by the associated flag bit(s) in the IA32_MCI_CTL register.	0x1	Enabled	
59:59	MISCV	If this flag is '1', the IA32_MCI_MISC register content is valid and can be read.	0x1	IA32_MCI_MISC Valid	
58:58	ADDRV	If this flag is '1', the IA32_MCI_ADDR register content is valid and can be read.	0x1	IA32_MCI_ADDR Valid	
57:57	PCC	Processor Context Corrupt: A '1' indicates that reliable restarting of the processor may not be possible and a reset is recommended.	0x0	Processor Context Not Corrupted	
56:56	S	Signaling an uncorrectable recoverable error.	0x1	---	
54:53	THRESHOLD_BASED_STATUS	Reserved	0x0	---	
52:38	CORRECTED_ERR_CNT	Corrected error count since last clear of machine check registers	0x0	---	
37:37	PW	Firmware update error status indicator	0x0	---	
36:32	OTHER_INFO	Not part of Machine-Check Architecture	0x0	---	
31:16	MISCOD	Model Specific Error Code	0xC	IFU_POISON_ERROR_000C_0150	
15:0	MCACOD	Machine Check Architecture Error Code	0x150	IFU Icache Data Parity/Data Poison/Tag Parity Error	

8.4.3 MEC SRAR Data Poison Consumption (Intel® Xeon® 6900/6700-Series Processors with E-Cores)

Test Objective

Inject an SRAR type of error to the MEC.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processor with E-Cores C0
BIOS	WW13'24 BKC - BHSDCRB.IPC.3106.D01.2403190446
OS Version	CentOS® Stream 9
Kernel	6.8.0-31-generic x86_64
HW Configuration	N/A
CScripts Version	GNR 2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)
- [FTrace and DMESG](#)
- <http://git.kernel.org/pub/scm/linux/kernel/git/aegl/ras-tools.git>
- [Kernel Tools and Scripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >

- Patrol Scrub = Disable

NOTE

With patrol scrub enabled, there is a high chance the CPU will detect the UE ECC error before a CPU consumption occurs. The resulting OS behavior would do a page offline based off of the Patrol Scrub UCNA before the OS could even consume the error.

Recipe

1. Initialize EINJ.

```
# modprobe einj
```

2. Inject a data poison consumption error.

```
# ./einj_mem_uc -f 'single'
0: single vaddr = 0x7f03f56a6400 paddr = 167603400
injecting ...
triggering ...
signal 7 code 4 addr 0x7f03f56a6000
```



```
page not present
Saw local machine check
Test passed
```

3. Observe the following in your ftrace output:
 - UE ECC UCNA from one of the MCCHAN bank, for example, MCACOD 0x90
 - Check that the ADDR matches the einj_mem_uc addr value
 - eLog event due to the UE ECC UCNA
 - Linux Page Offline event
 - EDAC event for the UE ECC UCNA
 - Data SRAR Poison Consumption, for example, MCACOD 134
 - For Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, this would be Bank 1.
 - For Intel® Xeon® 6900/6700-Series Processors with E-Cores, this would be Bank 3
 - Check that the ADDR matches the einj_mem_uc addr value.
 - EDAC event for the Data Poison Consumption
 - System should stay up
4. Navigate to the original terminal opened earlier for the FTrace/log collection. Evaluate the OS logs. Ensure UCE logged in iMC. In the following example, UCE is logged in MC14 (iMC1_CH0). The address in the log collection matches the address in the einj_mem_uc tool. Since this is a data read error, expect to see MC3 bank populated as shown in the logs.

```
# modprobe acpi_extlog
# cd /sys/kernel/debug/tracing/
# echo 1 > events/mce/mce_record/enable
# echo 1 > events/ras/enable
# cat trace_pipe

kworker/0:1-10 [000] ..... 250.655058: mce_record: CPU: 0,
MCGc/s: f000c20/0, MC14: ac00000200a00090, IPID: 0000000000000000, ADDR/MISC/
SYND: 0000000167603400/0800018167026086/0000000000000000, RIP:
00:<0000000000000000>, TSC: bc5d2dd0c8, PROCESSOR: 0:a06f3, TIME: 1714766075,
SOCKET: 0, APIC: 0
kworker/0:1-10 [000] ..... 250.836750: memory_failure_event: pfn
0x167603: recovery action for dirty LRU page: Recovered
kworker/0:1-10 [000] ..... 250.872010: extlog_mem_event: {1}
recoverable error: single-symbol chipkill ECC physical addr: 0000000167603400
(mask lsb: ff) node:0 card:1 module:0 rank:0 bank:12 device:0 row:11488
column:1216 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnB, DIMM0
kworker/0:1-10 [000] ..... 250.872573: mc_event: 1 Fatal error:
memory read error on CPU SrcID#0 MC#1 Chan#0 DIMM#0 (mc:1 location:0:0:-1
address:0x167603400 grain:32 syndrome:0x00000000 err_code:0x00a0:0x0090
SystemAddress:0x167603400 ProcessorSocketId:0x0 MemoryControllerId:0x1
ChannelAddress:0x73b01a00 ChannelId:0x0 RankAddress:0x1cec0680
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x2ce0 Column:0x4c0
Bank:0x0 BankGroup:0x3 ChipSelect:0x0)
kworker/0:1-10 [000] ..... 250.872611: mce_record: CPU: 27,
MCGc/s: f000c20/f, MC3: bd80000000070134, IPID: 0000000000000000, ADDR/MISC/
SYND: 0000000167603400/0000000000000080/0000000000000000, RIP:
33:<00000000000405018>, TSC: bc71517262, PROCESSOR: 0:a06f3, TIME: 1714766075,
SOCKET: 0, APIC: 36
```

- Decode the FTrace log using the `error.manualMcaDecode()` command and ensure the FTrace matches information that was decoded. For SRAR Data poison consumption, uncorrectable error for demand or underfill reads for MSCOD should be seen. Additionally MSCOD for MEC bank (MC bank 3) will show Cache L1 data read.

```
>>> error.manualMcaDecode()
Manual Machine Check Bank Decoder
Enter MCI_STATUS, MCI_ADDR, MCI_MISC values
Enter 't' for a short demo
Enter 'Q' or 'q' to exit
>>> MCHN
>>> ac00000200a00090

Bank name: MCHN, Error = UNCORR Uncorrectable Error for Demand or Underfill Reads

+ MCMCHN_STATUS      0xAC00000200A00090
```

Range	Field	Value	Value Description
63:63	VALID	0x1	Indicates that the information within this register is valid
62:62	OVERFLOW	0x0	A '1' indicates that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (the VALID bit was already set)
61:61	UC	0x1	A '1' indicates that an uncorrectable the error condition UC Detected was detected.
60:60	EN	0x0	Error reporting enabled. A '1' indicates that error reporting is enabled by the associated flag bit(s) in the IA32_MCI_CTL register.
59:59	MISCV	0x1	If this flag is '1', the IA32_MCI_MISC register content is IA32_MCI_MISC Valid valid and can be read.
58:58	ADDRV	0x1	If this flag is '1', the IA32_MCI_ADDR register content is IA32_MCI_ADDR Valid valid and can be read.
57:57	PCC	0x0	Processor Context Corrupt: A '1' indicates that reliable Processor Context Not Corrupted possible and a reset restarting of the processor may not be is recommended.
56:56	S	0x0	Signaling an uncorrectable recoverable error.
55:55	AR	0x0	Recovery action required for UCR error.
54:53	THRESHOLD_BASED_STATUS	0x0	Threshold-based error status
52:38	CORRECTED_ERROR_COUNT		Count of correctable errors in the DCU

```

including sticky      0x0      ---
                        overflow indicator (bit 52)

      37:37      FW      FW Update Error Status
Indicator      0x0      ---

      36:32      OTHER_INFO      Not part of Machine-Check
Architecture      0x2      ---

      31:16      MSCOD      Model Specific Error
Code      0xa0      Uncorrectable Error for
Demand or Underfill Reads

      15:0      MCACOD      Machine Check Architecture Error
Code      0x90      MCCHAN_ERROR(1LM MODE) | MEMORY READ
ERROR | CHANNEL_0

>>> error.manualMcaDecode()
Manual Machine Check Bank Decoder
Enter MCI_STATUS, MCI_ADDR, MCI_MISC values
Enter 't' for a short demo
Enter 'Q' or 'q' to exit
>>> 3
>>> bd80000000070134

Bank name: Core MEC, Error = UNCORR 0x7

+ MC3_STATUS      0xBD80000000070134

      Range      Field
Description      -----      Value      Value Description
-----
      63:63      VALID      Indicates that the information within
this register is valid      0x1      Valid

      62:62      OVERFLOW      A '1' indicates that a machine-check
error occurred while      0x0      Not Overflow
still in the error-      the results of a previous error were
reporting register bank (the VALID bit
was already set)

      61:61      UC      A '1' indicates that an uncorrectable
the error condition      0x1      UC Detected
was detected.

      60:60      EN      Error reporting enabled. A '1' indicates
that error      0x1      Enabled
reporting is enabled by the associated
flag bit(s) in the
IA32_MCI_CTL register.

      59:59      MISCV      If this flag is '1', the IA32_MCI_MISC
register content is      0x1      IA32_MCI_MISC Valid
valid and can be read.

      58:58      ADDR_V      If this flag is '1', the IA32_MCI_ADDR
register content is      0x1      IA32_MCI_ADDR Valid
valid and can be read.

      57:57      PCC      Processor Context Corrupt: A '1'
indicates that reliable      0x0      Processor Context Not Corrupted
possible and a reset      restarting of the processor may not be
is recommended.

```

56:56	S	0x1	---	Signaling an uncorrectable recoverable error.
55:55	AR	0x1	---	Recovery action required for UCR error.
54:53	THRESHOLD_BASED_STATUS		0x0	---
52:38	CORRECTED_ERROR_COUNT	0x0	---	Count of filtered correctable MCA errors including sticky
37:37	FW	0x0	---	Firmware update error status indicator
36:32	OTHER_INFO	0x0	---	Not part of Machine-Check Architecture
31:16	MSCOD	0x7		Model Specific Error Code
15:0	MCACOD	0x134		Machine Check Architecture Error Code LEVEL_0 (CORE L1 CACHE)
				CACHE_HIERARCHY_ERROR DATA_READ DATA

8.4.4 FEC SRAR Instruction Poison Consumption (Intel® Xeon® 6900/6700-Series Processors with E-Cores)

Test Objective

Inject an SRAR type of error to the FEC.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processor with E-Cores C0
BIOS	WW13'24 BKC - BHSDCRB.IPC.3106.D01.2403190446
OS Version	CentOS* Stream 9
Kernel	6.8.0-31-generic x86_64
HW Configuration	N/A
CScripts Version	GNR 2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the :

- [CScripts](#)
- [FTrace and DMESG](#)
- <http://git.kernel.org/pub/scm/linux/kernel/git/aegl/ras-tools.git>

- [Kernel Tools and Scripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >

- Patrol Scrub = Disable

NOTE

With patrol scrub enabled, there is a high chance the CPU will detect the UE ECC error before a CPU consumption occurs. The resulting OS behavior would do a page offline based off of the Patrol Scrub UCNA before the OS could even consume the error.

Recipe

1. Initialize EINJ.

```
# modprobe einj
```

2. Inject a instruction poison consumption error.

```
# ./einj_mem_uc -f 'instr'
0: instr      vaddr = 0x404000 paddr = 13b90d000
injecting ...
triggering ...
signal 7 code 128 addr (nil)
Unexpected SIGBUS
page not present
Saw local machine check
CMCIs took ~5.398252 secs to be reported.
Test passed
```

3. Observe the following in your ftrace output:
 - UE ECC UCNA from one of the MCCHAN bank, for example, MCACOD 0x90
 - Check that the ADDR matches the einj_mem_uc addr value.
 - eLog event due to the UE ECC UCNA
 - Linux Page Offline event
 - EDAC event for the UE ECC UCNA
 - Instruction SRAR Poison Consumption, for example, MCACOD 150
 - For Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, this would be Bank 0.
 - For Intel® Xeon® 6900/6700-Series Processors with E-Cores, this would be Bank 2.
 - Check that the ADDR matches the einj_mem_uc addr value.
 - EDAC event for the Instruction Poison Consumption
 - System should stay up.

- Navigate to the original terminal opened earlier for the FTrace/log collection. Evaluate the OS logs. Ensure UCE logged in iMC. In the following example, UCE is logged in MC14 (iMC1_CH0). The address in the log collection matches the address in the einj_mem_uc tool. Since this is an FEC error, expect to see MC2 bank populated as shown in the logs.

```
# modprobe acpi_extlog
# cd /sys/kernel/debug/tracing/
# echo 1 > events/mce/mce_record/enable
# echo 1 > events/ras/enable
# cat trace_pipe

kworker/0:1-10 [000] ..... 507.699600: mce_record: CPU: 0,
MCGc/s: f000c20/0, MC14: ac00000200a00090, IPID: 0000000000000000, ADDR/MISC/
SYND: 000000013b90d000/080003813b81a086/0000000000000000, RIP:
00:<0000000000000000>, TSC: 12e134cebd2, PROCESSOR: 0:a06f3, TIME:
1714766332, SOCKET: 0, APIC: 0
kworker/0:1-10 [000] ..... 507.710794: memory_failure_event: pfn
0x13b90d: recovery action for clean LRU page: Recovered
kworker/0:1-10 [000] ..... 508.702562: extlog_mem_event: {3}
recoverable error: single-symbol chipkill ECC physical addr: 000000013b90d000
(mask lsb: ff) node:0 card:1 module:0 rank:0 bank:28 device:0 row:10096
column:832 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnB, DIMM0
kworker/0:1-10 [000] .N... 508.707898: mc_event: 1 Fatal error:
memory read error on CPU SrcID#0 MC#1 Chan#0 DIMM#0 (mc:1 location:0:0:-1
address:0x13b90d000 grain:32 syndrome:0x00000000 err_code:0x00a0:0x0090
SystemAddress:0x13b90d000 ProcessorSocketId:0x0 MemoryControllerId:0x1
ChannelAddress:0x5dc86800 ChannelId:0x0 RankAddress:0x17721a00
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x2770 Column:0x340
Bank:0x0 BankGroup:0x7 ChipSelect:0x0)
kworker/0:1-10 [000] .N... 508.707932: mce_record: CPU: 19,
MCGc/s: f000c20/f, MC2: bd80000010000150, IPID: 0000000000000000, ADDR/MISC/
SYND: 000000013b90d000/000000000000000080/0000000000000000, RIP:
33:<0000000000403fff>, TSC: 12e134bd752, PROCESSOR: 0:a06f3, TIME:
1714766332, SOCKET: 0, APIC: 26
```

- Decode the FTrace log using the `error.manualMcaDecode()` command and ensure the FTrace matches information that was decoded. For SRAR Instruction Poison, uncorrectable error for demand or underfill reads for MSCOD should be seen. Additionally, MSCOD for FEC bank (MC bank 2) will show poison load.

```
>>> error.manualMcaDecode()
Manual Machine Check Bank Decoder
Enter MCI_STATUS, MCI_ADDR, MCI_MISC values
Enter 't' for a short demo
Enter 'Q' or 'q' to exit
>>> MCHN
>>> ac00000200a00090

Bank name: MCHN, Error = UNCORR Uncorrectable Error for Demand or Underfill
Reads

+ MCMCHN_STATUS      0xAC00000200A00090

Range      Field
Description -----
-----
63:63      VALID      Indicates that the information within
this register is valid 0x1 Valid
62:62      OVERFLOW     A '1' indicates that a machine-check
error occurred while 0x0 Not Overflow
the results of a previous error were
```

still in the error- was already set)		reporting register bank (the VALID bit	
61:61 error condition	UC 0x1	UC Detected	A '1' indicates that an uncorrectable the was detected.
60:60 that error flag bit(s) in the	EN 0x0	Not Enabled	Error reporting enabled. A '1' indicates reporting is enabled by the associated IA32_MCi_CTL register.
59:59 register content is	MISCV 0x1	IA32_MCi_MISC Valid	If this flag is '1', the IA32_MCi_MISC valid and can be read.
58:58 register content is	ADDRV 0x1	IA32_MCi_ADDR Valid	If this flag is '1', the IA32_MCi_ADDR valid and can be read.
57:57 indicates that reliable possible and a reset	PCC 0x0	Processor Context Corrupt: A '1' Processor Context Not Corrupted	Processor Context Corrupt: A '1' Processor Context Not Corrupted restarting of the processor may not be is recommended.
56:56 error.	S 0x0	---	Signaling an uncorrectable recoverable
55:55 error.	AR 0x0	---	Recovery action required for UCR
54:53 status	THRESHOLD_BASED_STATUS	0x0	Threshold-based error ---
52:38 including sticky	CORRECTED_ERROR_COUNT 0x0	---	Count of correctable errors in the DCU overflow indicator (bit 52)
37:37 Indicator	FW	0x0	FW Update Error Status ---
36:32 Architecture	OTHER_INFO	0x2	Not part of Machine-Check ---
31:16 Code Demand or Underfill Reads	MSCOD	0xa0	Model Specific Error Uncorrectable Error for
15:0 Code ERROR CHANNEL_0	MCACOD 0x90	Machine Check Architecture Error MCCHAN_ERROR(1LM MODE) MEMORY READ	
<pre>>>> error.manualMcaDecode() Manual Machine Check Bank Decoder Enter MCI_STATUS, MCI_ADDR, MCI_MISC values Enter 't' for a short demo Enter 'Q' or 'q' to exit >>> 2 >>> bd80000010000150 Bank name: Core FEC, Error = UNCORR Poisoned Data + MC2_STATUS 0xBD80000010000150 Range Field</pre>			

Description	Value	Value Description
63:63 this register is valid	VALID 0x1	Indicates that the information within Valid
62:62 error occurred while still in the error- was already set)	OVERFLOW 0x0	A '1' indicates that a machine-check Not Overflow the results of a previous error were reporting register bank (the VALID bit
61:61 the error condition	UC 0x1	A '1' indicates that an uncorrectable UC Detected was detected.
60:60 that error flag bit(s) in the	EN 0x1	Error reporting enabled. A '1' indicates Enabled reporting is enabled by the associated IA32_MCi_CTL register.
59:59 register content is	MISCV 0x1	If this flag is '1', the IA32_MCi_MISC IA32_MCi_MISC Valid valid and can be read.
58:58 register content is	ADDRV 0x1	If this flag is '1', the IA32_MCi_ADDR IA32_MCi_ADDR Valid valid and can be read.
57:57 indicates that reliable possible and a reset	PCC 0x0	Processor Context Corrupt: A '1' Processor Context Not Corrupted restarting of the processor may not be is recommended.
56:56 (UCR) error.	S 0x1	Signaling an uncorrected recoverable ---
55:55 error.	AR 0x1	Recovery action required for UCR ---
54:53 Reserved	THRESHOLD_BASED_STATUS 0x0	---
52:38 including sticky	CORRECTED_ERROR_COUNT 0x0	Count of filtered correctable MCA errors --- overflow indicator (bit 52)
37:37 indicator	FW 0x0	Firmware update error status ---
36:32 Architecture	OTHER_INFO 0x0	Not part of Machine-Check ---
31:16 Code	MSCOD 0x1000	Model Specific Error Poisoned Data
15:0 Code	MCACOD 0x150	Machine Check Architecture Error CACHE_HIERARCHY_ERROR INSTRUCTION_FETCH INSTRUCTION LEVEL_0 (CORE L1 CACHE)

9.0 System RAS

9.1 Viral: IIO - Crash Dump

Test Objective

Upon an unrecoverable error condition, minimizing further error propagation is highly desired. Viral provides a method of providing high level of error containment. This test is to ensure IIO uncorrectable error is contained and prevented from reaching permanent storage or network.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW5 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	Generic PCIe Card
CScripts Version	GNR 2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18
- [CrashDump](#) on page 19
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log >
 - Viral Status = Enable

Recipe

1. Ensure that the system has viral mode enabled and is not already in viral state.

```
>>> ras.sys.viral_config_check()
```

Socket#	Description	Value
socket0	Software Error Recovery Support	Y
socket1	Software Error Recovery Support	Y
socket0	Poison Enable Bit Support	Y
socket1	Poison Enable Bit Support	Y
socket0	Viral Enable Bit	Y
socket1	Viral Enable Bit	Y
socket0-io0	IIO Viral Config	0x40000003
socket0-io1	IIO Viral Config	0x40000003
socket1-io0	IIO Viral Config	0x40000003
socket1-io1	IIO Viral Config	0x40000003
socket0-io0	HIOP Global Error Config	0x0
socket0-io1	HIOP Global Error Config	0x0
socket1-io0	HIOP Global Error Config	0x0
socket1-io1	HIOP Global Error Config	0x0
socket0-io0	UPI Viral Config	0x40000017
socket0-io1	UPI Viral Config	0x40000017
socket1-io0	UPI Viral Config	0x40000017
socket1-io1	UPI Viral Config	0x40000017
socket0	Ubox Viral Config	0x19fc0000
socket1	Ubox Viral Config	0x19fc0000
socket0	PCU Viral Control	0xf04
socket1	PCU Viral Control	0xf04
socket0-mc0	IMC Viral Config	0x0
socket0-mc1	IMC Viral Config	0x0
socket1-mc0	IMC Viral Config	0x0
socket1-mc1	IMC Viral Config	0x0
socket0-mc0	IMC Viral Status	0x0
socket0-mc1	IMC Viral Status	0x0
socket1-mc0	IMC Viral Status	0x0

```

| socket1-mc1 |          IMC Viral Status          |          0x0          |
=====
>>> sv.sockets.io0.uncore.hiop.hiop1.hiop_reg.viral_1_5_2_cfg.show()
0x00000000 : iio_viral_state (31:31) --
0x00000000 : iio_viral_status (30:30) --
0x00000000 : iio_global_viral_mask (02:02) --
0x00000001 : iio_signal_global_fatal (01:01) --
0x00000001 : iio_fatal_viral_alert_enable (00:00) --

0x00000000 : iio_viral_state (31:31) --
0x00000000 : iio_viral_status (30:30) --
0x00000000 : iio_global_viral_mask (02:02) --
0x00000001 : iio_signal_global_fatal (01:01) --
0x00000001 : iio_fatal_viral_alert_enable (00:00) --

>>> sv.sockets.io0.uncore.hiop.hiop1.hiop_reg.r2glerr_1_5_2_cfg.show()
0x00000000 : a2ffatalstatus (18:18) --
0x00000000 : a2fviralstatus (17:17) --
0x00000000 : f2aviralstatus (16:16) --
0x00000000 : maska2fviralout2f2aviralin (14:14) --
0x00000000 : maska2ffatalout2f2aviralin (13:13) --
0x00000000 : maska2fviralout (10:10) --
0x00000000 : maskf2aviralin (09:09) --
0x00000000 : maska2ffataltoviralout (08:08) -- c
0x00000000 : maska2ffatalout (07:07) --
0x00000000 : maskexternfatalin (06:06) --
0x00000000 : maskexternviralin (05:05) --
0x00000000 : resetglobalviral (04:04) --
0x00000000 : resetglobalfatalerror (03:03) --

0x00000000 : a2ffatalstatus (18:18) --
0x00000000 : a2fviralstatus (17:17) --
0x00000000 : f2aviralstatus (16:16) --
0x00000000 : maska2fviralout2f2aviralin (14:14) --
0x00000000 : maska2ffatalout2f2aviralin (13:13) --
0x00000000 : maska2fviralout (10:10) --
0x00000000 : maskf2aviralin (09:09) --
0x00000000 : maska2ffataltoviralout (08:08) --
0x00000000 : maska2ffatalout (07:07) --
0x00000000 : maskexternfatalin (06:06) --
0x00000000 : maskexternviralin (05:05) --
0x00000000 : resetglobalviral (04:04) --
0x00000000 : resetglobalfatalerror (03:03) --

```

2. Inject Fatal CTO error to the root port.

```

>>> ei.iio.injectPcieError(socket=0, port="pxp0.rp2", errType="uce", severity="fatal", showErrorRegs="True",
verbose="true")

#####
Socket: 0 io0

Uncore MCA from Socket: 0
Socket 0 : No errors logged in any Uncore-specific Machine Check Banks

#####
Socket: 0 iol

Uncore MCA from Socket: 0
Socket 0 : No errors logged in any Uncore-specific Machine Check Banks

#####
Socket: 0 soc

Uncore MCA from Socket: 0
Socket 0 : No errors logged in any Uncore-specific Machine Check Banks

#####
Socket: 0 Compute0

```

```

Socket 0 : No errors logged in any Core-specific Machine Check Banks

#####
Socket: 0 Compute1

Socket 0 : No errors logged in any Core-specific Machine Check Banks

#####
Socket: 0 Compute2

Socket 0 : No errors logged in any Core-specific Machine Check Banks

=====
PCIE_PXP0: REGISTERS AND FIELDS DESCRIPTIONS
=====
Register: socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsts
cte = 0x1
For Switch Ports, this bit is set if the Switch Port issues Non-Posted Requests on its own
behalf (vs. only forwarding such as Requests generated by other devices).

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsev
0x00000001 : uies (22:22) -- Uncorrectable Internal Error Severity
0x00000001 : mtlpes (18:18) -- Malformed TLP Severity
0x00000001 : roes (17:17) -- Receiver Overflow Error Severity
0x00000001 : ctes (14:14) -- Completion Timeout Error Severity
0x00000001 : fces (13:13) -- Flow Control Protocol Error Severity
0x00000001 : ptlpes (12:12) -- Poisoned TLP Received Severity
0x00000001 : dlpes (04:04) -- Data Link Protocol Error Severity

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncmsk
0x00000001 : urem (20:20) -- Unsupported Request Error Mask
0x00000001 : ecrsem (19:19) -- (for VFs) RO-Zero\n(for non-VFs with ECRC Check Capable [AERCAPCTL.ECC] is 1)...
0x00000001 : sldem (05:05) -- This bit does not apply to the upstream port or End point

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.aerhdrlog1
0xffffffff : tlphdrlog (31:00) -- As soon as an error is logged in this register, it remains locked for furt...

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.aerhdrlog2
0xffffffff : tlphdrlog (31:00) -- As soon as an error is logged in this register, it remains locked for furt...

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.aerhdrlog3
0xffffffff : tlphdrlog (31:00) -- As soon as an error is logged in this register, it remains locked for furt...

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.aerhdrlog4
0xffffffff : tlphdrlog (31:00) -- As soon as an error is logged in this register, it remains locked for furt...

-----
Register: socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts
femr = 0x1
Fatal Error Message Received

fuf = 0x1
First Uncorrectable Fatal

efr = 0x1
Error Fatal/Non-Fatal Received

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.secsts
0x00000001 : rma (13:13) -- This bit is set when the secondary side for Type 1 configuration space header fu...

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.bctl
0x00000001 : se (01:01) -- This bit controls the forwarding of PCI Express ERR_COR, ERR_NONFATAL and ERR_FAT...
0x00000001 : pere (00:00) -- This bit controls the response to poisoned TLPs in the PCI Express port.\nlf: En...

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.errsrcid
0x00001430 : efsid (31:16) -- Loaded with the Requester ID indicated in the received ERR_FATAL or ERR_NONFAT...

-----
Register: socket0.io0.uncore.pi5.pxp0.rp2.cfg.pcists
sse = 0x1
This bit is Set when a Function sends an ERR_FATAL or ERR_NONFATAL Message, and the SERR#
Enable bit in the Command Register is 1b.\nFunctions with a Type 0 Configuration Space
Header that do not send ERR_FATAL or ERR_NONFATAL Messages are permitted to hardwire this
bit to 0b.\nDefault value of this bit is 0b.

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.devctl
0x00000005 : mrrs (14:12) -- This field sets the maximum Read Requests size of the function as a requester. ...
0x00000001 : etfe (08:08) -- When set, this bit enables a function to use an 8-bit tag field as a Requester....
0x00000002 : mps (07:05) -- This field sets maximum TLP payload size for the function. As a receiver, the fu...
0x00000001 : fere (02:02) -- When this bit is set, generation of the ERR_FATAL message is enabled.
0x00000001 : nfer (01:01) -- When this bit is set, generation of the ERR_NONFATAL message is enabled.
0x00000001 : cere (00:00) -- When this bit is set, generation of the ERR_CORR message is enabled.

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.pciCmd
0x00000001 : intxd (10:10) -- This bit controls the ability of the PCI-Express Function to generate legacy I...
0x00000001 : see (08:08) -- When Set, this bit enables reporting upstream of Non-fatal and Fatal errors dete...

```

```

0x00000001 : pere (06:06) -- This bit controls the logging of poisoned TLPs in the Master Data Parity Error ...
0x00000001 : bme (02:02) -- Controls the ability of a Function to issue Memory and I/O Read/Write Requests, \...
0x00000001 : mse (01:01) -- Controls a Function's response to Memory Space accesses. When this bit is \nClear...
0x00000001 : iose (00:00) -- Controls a Function's response to I/O Space accesses. When this bit is Clear, a...

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.rootctl
0x00000001 : crssve (04:04) -- When Set, this bit enables the Root Port to indicate to software when Request...
0x00000001 : pmeie (03:03) -- This field controls the generation of interrupts for PME messages. \n1: Enable...
0x00000001 : sefee (02:02) -- This field controls generation of system errors in the PCI-Express port hierar...
0x00000001 : senfee (01:01) -- This field controls generation of system errors in the PCI-Express port hiera...
0x00000001 : secee (00:00) -- This field controls generation of system errors in the PCI-Express port hierar...

-----
Register: socket0.io0.uncore.pi5.pxp0.rp2.cfg.secsts
rma = 0x1
This bit is set when the secondary side for Type 1 configuration space header function
(for requests initiated by the Type 1 header function itself) receives a completion with
Unsupported Requests Completion Status.

-----
Register: socket0.io0.uncore.pi5.pxp0.rp2.cfg.devsts
fed = 0x1
This bit indicates that this function has detected a Fatal error. Errors are logged in
this register regardless of whether error reporting is enabled or not in the Device
Control register.

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts
0x00000001 : lbms (14:14) -- This bit is set by hardware to indicate that either of the following has occur...
0x00000001 : dllla (13:13) -- This bit indicates the status of the Data Link Control and Management Status M...
0x00000001 : scc (12:12) -- When set to 1b, this bit indicates that the component uses the same physical referen...
0x00000002 : nlw (09:04) -- This field indicates the negotiated width of the PCI Express link.\n00 0001b: xl...
0x00000003 : cls (03:00) -- This field indicates the negotiated link speed of the given PCI Express link.\nT...

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts2
0x00000001 : eqph3succ (04:04) -- When set to 1b, this bit indicates that Phase 3 of the 8.0 GT/s Transmitt...
0x00000001 : eqph2succ (03:03) -- When set to 1b, this bit indicates that Phase 2 of the 8.0 GT/s Transmitt...
0x00000001 : eqph1succ (02:02) -- When set to 1b, this bit indicates that Phase 1 of the 8.0 GT/s Transmitt...
0x00000001 : eqcmplt (01:01) -- When set to 1b, this bit indicates that the Transmitter Equalization procedu...

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.ltssmsmsts
0x00000009 : ltssmstatemain (31:28) -- LTSSM State Main.
0x00000001 : ltssmstatesub (27:24) -- LTSSM State Sub State.
0x00000004 : lnknum (15:08) -- Link Number.
0x00000003 : rcvratesup (07:03) -- Rate support received from link partner. Bit 0 = 5GT/s, Bit 1 = 8GT/s, B...
0x00000001 : dlactive (01:01) -- DL Active
0x00000001 : ltssmlnkup (00:00) -- LTSSM Link UP.

=> socket0.io0.uncore.pi5.pxp0.rp2.cfg.slotsts
0x00000001 : dllscs (08:08) -- This bit is set (if it is not already set) when the state of the Data Link La...
0x00000001 : pds (06:06) -- This field conveys the Presence Detect status of an adapter in the slot. It refl...
0x00000001 : pdcs (03:03) -- This bit is set when the value reported in the Presence Detect State bit is cha...

-----
Register: socket0.io0.uncore.pi5.pxp0.rp2.cfg.rppiosts
cfgurc = 0x1
Configuration Request received UR Completion

-----
Register: socket0.io0.uncore.pi5.pxp0.rp3.cfg.secsts
rma = 0x1
This bit is set when the secondary side for Type 1 configuration space header function
(for requests initiated by the Type 1 header function itself) receives a completion with
Unsupported Requests Completion Status.

-----
Register: socket0.io0.uncore.pi5.pxp0.rp3.cfg.rppiosts
cfgurc = 0x1
Configuration Request received UR Completion

-----
Register: socket0.io0.uncore.pi5.pxp0.rp6.cfg.secsts
rma = 0x1
This bit is set when the secondary side for Type 1 configuration space header function
(for requests initiated by the Type 1 header function itself) receives a completion with
Unsupported Requests Completion Status.

-----
Register: socket0.io0.uncore.pi5.pxp0.rp6.cfg.rppiosts
cfgurc = 0x1
Configuration Request received UR Completion

-----
Register: socket0.io0.uncore.pi5.pxp0.rp7.cfg.secsts
rma = 0x1
This bit is set when the secondary side for Type 1 configuration space header function
(for requests initiated by the Type 1 header function itself) receives a completion with
Unsupported Requests Completion Status.

-----

```

```
Register: socket0.io0.uncore.pi5.pxp0.rp7.cfg.rppiosts
cfgurc = 0x1
Configuration Request received UR Completion
```

```
*****
Fields with non default value
*****
```

```
=====
| IP block | # fields with non-default value |
|-----|-----|
| PCIE_PXP0 | 0xe |
|-----|-----|
```

```
*****
The Error registers Dump CSV and JSON files are located in: C:\Users\dcgpae\PythonSv
=====
```

STAGE	CHECKPOINT	STATUS
Setup	Access Validation	Passed
Setup	Input Validation - socket	Passed
Setup	Halt Validation	Passed
Setup	Unlock injectors	Passed
Setup	Input Validation - port	Passed
Setup	Error Register Access	Passed
Setup	Input Validation - Clrerrtype	Passed
Setup	Injector Enabling	Passed
Injection	Inject PCIe/CXL.io Error	Passed
Verification	Error Check	Passed

- Evaluate viral error logs. The platform should be in a viral state and the status should be set. With IOMCA enabled and EMCAGEN2 enabled, it will result in MSMI_MCERR_INTERNAL and MSMI_IERR_INTERNAL with viral states enabled.

```
>>> sv.sockets.io0.uncore.hiop.hiop1.hiop_reg.viral_1_5_2_cfg.show()
0x00000001 : iio_viral_state (31:31) --
0x00000001 : iio_viral_status (30:30) --
0x00000000 : iio_global_viral_mask (02:02) --
0x00000001 : iio_signal_global_fatal (01:01) --
0x00000001 : iio_fatal_viral_alert_enable (00:00) --

0x00000001 : iio_viral_state (31:31) --
0x00000001 : iio_viral_status (30:30) --
0x00000000 : iio_global_viral_mask (02:02) --
0x00000001 : iio_signal_global_fatal (01:01) --
0x00000001 : iio_fatal_viral_alert_enable (00:00) --

>>> sv.sockets.io0.uncore.hiop.hiop1.hiop_reg.r2glerr_1_5_2_cfg.show()
0x00000000 : a2ffatalstatus (18:18) --
0x00000000 : a2fviralstatus (17:17) --
0x00000001 : f2aviralstatus (16:16) --
0x00000000 : maska2fviralout2f2aviralin (14:14) --
0x00000000 : maska2ffatalout2f2aviralin (13:13) --
0x00000000 : maska2fviralout (10:10) --
0x00000000 : maskf2aviralin (09:09) --
0x00000000 : maska2ffataloutviralout (08:08) --
0x00000000 : maska2ffatalout (07:07) --
0x00000000 : maskexternfatalin (06:06) --
0x00000000 : maskexternviralin (05:05) --
0x00000000 : resetglobalviral (04:04) --
0x00000000 : resetglobalfatalerror (03:03) --

0x00000000 : a2ffatalstatus (18:18) --
0x00000000 : a2fviralstatus (17:17) --
0x00000001 : f2aviralstatus (16:16) --
0x00000000 : maska2fviralout2f2aviralin (14:14) --
0x00000000 : maska2ffatalout2f2aviralin (13:13) --
0x00000000 : maska2fviralout (10:10) --
0x00000000 : maskf2aviralin (09:09) --
0x00000000 : maska2ffataloutviralout (08:08) --
0x00000000 : maska2ffatalout (07:07) --
0x00000000 : maskexternfatalin (06:06) --
0x00000000 : maskexternviralin (05:05) --
0x00000000 : resetglobalviral (04:04) --
0x00000000 : resetglobalfatalerror (03:03) --

>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) --
0x00000000 : ierr (30:30) --
0x00000000 : mcerr (29:29) --
0x00000000 : caterr_internal (28:28) --
0x00000000 : ierr_internal (27:27) --
0x00000000 : mcerr_internal (26:26) --
0x00000000 : cam_parity_err (25:25) --
0x00000000 : msmi_mcp_smbus (24:24) --
0x00000000 : msmi (23:23) --
0x00000000 : msmi_ierr (22:22) --
0x00000000 : msmi_mcerr (21:21) --
0x00000001 : msmi_internal (20:20) --
0x00000001 : msmi_ierr_internal (19:19) --
0x00000001 : msmi_mcerr_internal (18:18) --
0x00000000 : rmca (17:17) --
0x00000000 : rsmi (16:16) --
0x00000000 : rmca_internal (15:15) --
0x00000000 : rsmi_internal (14:14) --
0x00000000 : first_core_ierr (04:00) --

0x00000000 : caterr (31:31) --
0x00000000 : ierr (30:30) --
0x00000000 : mcerr (29:29) --
0x00000000 : caterr_internal (28:28) --
0x00000000 : ierr_internal (27:27) --
0x00000000 : mcerr_internal (26:26) --
0x00000000 : cam_parity_err (25:25) --
```

```

0x00000000 : msmi_mcp_smbus (24:24) --
0x00000000 : msmi (23:23) --
0x00000000 : msmi_ierr (22:22) --
0x00000000 : msmi_mcerr (21:21) --
0x00000001 : msmi_internal (20:20) --
0x00000001 : msmi_ierr internal (19:19) --
0x00000000 : msmi_mcerr internal (18:18) --
0x00000000 : rmca (17:17) --
0x00000000 : rsmmi (16:16) --
0x00000000 : rmca_internal (15:15) --
0x00000000 : rsmmi_internal (14:14) --
0x00000000 : first_core_ierr (04:00) --

>>> ras.sys.viral_error_check()
=====
| Socket# | Register | Value |
| --- | --- | --- |
| *** | VIRAL Registers | *** |
| --- | --- | --- |
| socket0 | iio_viral_state | 0x1 |
| socket0 | iio_viral_status | 0x1 |
| socket0 | iio_global_viral_mask | 0x0 |
| socket0 | iio_signal_global_fatal | 0x1 |
| socket0 | iio_fatal_viral_alert_enable | 0x1 |
| --- | --- | --- |
| socket1 | iio_viral_state | 0x1 |
| socket1 | iio_viral_status | 0x1 |
| socket1 | iio_global_viral_mask | 0x0 |
| socket1 | iio_signal_global_fatal | 0x1 |
| socket1 | iio_fatal_viral_alert_enable | 0x1 |
| --- | --- | --- |
| *** | R2GLERRCFG Registers | *** |
| --- | --- | --- |
| socket0 | a2ffatalstatus | 0x0 |
| socket0 | a2fviralstatus | 0x0 |
| socket0 | f2aviralstatus | 0x1 |
| socket0 | maska2fviralout2f2aviralin | 0x0 |
| socket0 | maska2ffatalout2f2aviralin | 0x0 |
| socket0 | maska2fviralout | 0x0 |
| socket0 | maskf2aviralin | 0x0 |
| socket0 | maska2ffataltoviralout | 0x0 |
| socket0 | maska2ffatalout | 0x0 |
| socket0 | maskexternfatalin | 0x0 |
| socket0 | maskexternviralin | 0x0 |
| socket0 | resetglobalviral | 0x0 |
| socket0 | resetglobalfatalerror | 0x0 |
| socket1 | a2ffatalstatus | 0x0 |
| socket1 | a2fviralstatus | 0x0 |
| socket1 | f2aviralstatus | 0x1 |
| socket1 | maska2fviralout2f2aviralin | 0x0 |
| socket1 | maska2ffatalout2f2aviralin | 0x0 |
| socket1 | maska2fviralout | 0x0 |
| socket1 | maskf2aviralin | 0x0 |
| socket1 | maska2ffataltoviralout | 0x0 |
| socket1 | maska2ffatalout | 0x0 |
| socket1 | maskexternfatalin | 0x0 |
| socket1 | maskexternviralin | 0x0 |
| socket1 | resetglobalviral | 0x0 |
| socket1 | resetglobalfatalerror | 0x0 |
| --- | --- | --- |
| *** | KTIVIRAL Registers | *** |
| --- | --- | --- |
| socket0 | kti_viral_state | 0x1 |
| socket0 | kti_viral_status | 0x1 |
| socket0 | kti_pkt_viral_set | 0x1 |
| socket0 | kti_unc_err_set | 0x0 |
| socket0 | kti_remote_viral_lpbk_en | 0x1 |
| socket0 | kti_global_viral_mask | 0x1 |
| socket0 | kti_signal_global_fatal | 0x1 |
| socket0 | kti_viral_alert_enable | 0x1 |
| --- | --- | --- |
| socket1 | kti_viral_state | 0x1 |
| socket1 | kti_viral_status | 0x1 |
| socket1 | kti_pkt_viral_set | 0x1 |
| socket1 | kti_unc_err_set | 0x0 |
| socket1 | kti_remote_viral_lpbk_en | 0x1 |
| socket1 | kti_global_viral_mask | 0x1 |
| socket1 | kti_signal_global_fatal | 0x1 |
| socket1 | kti_viral_alert_enable | 0x1 |

```

***	UBOXGLERRCFG Registers	***
socket0	maskbadfusepull	0x1
socket0	maskincorrectbankindex	0x1
socket0	maskieherr2	0x0
socket0	maskuboxmsmi	0x0
socket0	maskextviral	0x1
socket0	maskpcuproxysmi	0x3
socket0	maskpcuproxymc	0x3
socket0	maskpcuproxyierr	0x3
socket0	maskmasterlocktimeout	0x0
socket0	masksmtimeout	0x0
socket0	maskcfgwraddrmisalign	0x0
socket0	maskcfrdaddrmisalign	0x0
socket0	maskpoisonreception	0x0
socket0	maskunsupportopcode	0x0
socket0	maskuboxierr	0x0
socket0	maskuboxumc	0x0
socket0	resetuboxviral	0x0
socket0	resetuboxfatalerror	0x0
socket0	statusuboxviral	0x0
socket0	statusuboxfatalerror	0x1
socket1	maskbadfusepull	0x1
socket1	maskincorrectbankindex	0x1
socket1	maskieherr2	0x0
socket1	maskuboxmsmi	0x0
socket1	maskextviral	0x1
socket1	maskpcuproxysmi	0x3
socket1	maskpcuproxymc	0x3
socket1	maskpcuproxyierr	0x3
socket1	maskmasterlocktimeout	0x0
socket1	masksmtimeout	0x0
socket1	maskcfgwraddrmisalign	0x0
socket1	maskcfrdaddrmisalign	0x0
socket1	maskpoisonreception	0x0
socket1	maskunsupportopcode	0x0
socket1	maskuboxierr	0x0
socket1	maskuboxumc	0x0
socket1	resetuboxviral	0x0
socket1	resetuboxfatalerror	0x0
socket1	statusuboxviral	0x0
socket1	statusuboxfatalerror	0x1
***	VIRAL_CONTROL Registers	***
socket0	en_punit_agg_network_error	0x1
socket0	en_pcu_fw_error	0x1
socket0	en_pcu_uc_error	0x1
socket0	en_pcu_hw_error	0x1
socket0	viral_log	0x0
socket0	viral_log_disable	0x0
socket0	emca_mode	0x1
socket0	viral_fast_pin_dis	0x0
socket0	viral_reset	0x0
socket1	en_punit_agg_network_error	0x1
socket1	en_pcu_fw_error	0x1
socket1	en_pcu_uc_error	0x1
socket1	en_pcu_hw_error	0x1
socket1	viral_log	0x0
socket1	viral_log_disable	0x0
socket1	emca_mode	0x1
socket1	viral_fast_pin_dis	0x0
socket1	viral_reset	0x0
***	DDR4_VIRAL_CTL Registers	***
socket0	dis_trk_addr_rf_par_err	0x0
socket0	dis_wdb_rd_ecc_persist	0x0


```

| socket0 | dis_ha_wr_data_mac_par_err | 0x0 |
|-----|-----|-----|
| socket0 | dis_wdb_rd_ecc_uc | 0x0 |
| socket0 | dis_internal_error | 0x0 |
| socket0 | dis_ddrt_uc_ecc_err | 0x0 |
| socket0 | dis_cmi_illegal_opcode | 0x0 |
| socket0 | dis_poison_err | 0x0 |
|-----|-----|-----|
| socket0 | dis_cmiucdataerr | 0x0 |
| socket0 | dis_hbmcaper | 0x0 |
| socket0 | dis_hbmwrddqparerr | 0x0 |
| socket0 | dis_link_fail | 0x0 |
| socket0 | dis_uncorr_ufill_rd_err | 0x0 |
| socket0 | dis_uncorr_demand_rd_err | 0x0 |
| socket0 | dis_app_err | 0x0 |
| socket0 | dis_uncorr_spr_err | 0x0 |
| socket0 | dis_uncorr_pat_err | 0x0 |
| socket0 | dis_rp0_rpa_par_err | 0x0 |
| socket0 | dis_wdb_par_err | 0x0 |
| socket0 | dis_ha_wr_be_par_err | 0x0 |
| socket0 | dis_ha_wr_data_par_err | 0x0 |
| socket1 | dis_trk_addr_rf_par_err | 0x0 |
| socket1 | dis_wdb_rd_ecc_persist | 0x0 |
| socket1 | dis_ha_wr_data_mac_par_err | 0x0 |
| socket1 | dis_wdb_rd_ecc_uc | 0x0 |
| socket1 | dis_internal_error | 0x0 |
| socket1 | dis_ddrt_uc_ecc_err | 0x0 |
| socket1 | dis_cmi_illegal_opcode | 0x0 |
| socket1 | dis_poison_err | 0x0 |
| socket1 | dis_cmiucdataerr | 0x0 |
| socket1 | dis_hbmcaper | 0x0 |
| socket1 | dis_hbmwrddqparerr | 0x0 |
| socket1 | dis_link_fail | 0x0 |
| socket1 | dis_uncorr_ufill_rd_err | 0x0 |
| socket1 | dis_uncorr_demand_rd_err | 0x0 |
| socket1 | dis_app_err | 0x0 |
| socket1 | dis_uncorr_spr_err | 0x0 |
| socket1 | dis_uncorr_pat_err | 0x0 |
| socket1 | dis_rp0_rpa_par_err | 0x0 |
| socket1 | dis_wdb_par_err | 0x0 |
| socket1 | dis_ha_wr_be_par_err | 0x0 |
| socket1 | dis_ha_wr_data_par_err | 0x0 |
|-----|-----|-----|
| *** | IMC_VIRALCTL Registers | *** |
|-----|-----|-----|
| socket0 | viral_ch0 | 0x0 |
| socket0 | viral_clear | 0x0 |
| socket1 | viral_ch0 | 0x0 |
| socket1 | viral_clear | 0x0 |
=====

```

4. Run the Crashdump either using Cscripts or ACD tool and extract the crashdump summary. The logs should reflect the error logs collected on CScripts. Refer to [CrashDump](#) section for more information on how to run and extract Crashdump.

```

>>> error.crashdump_summary()

HW.MCE.UBOX:Generic_I/O_Error_On_Bus[0x14]_Dev[0x6]_Fun[0x0]_From_Seg[0x0].incomplete_dump.mca_same_value

FaultCPU= GNR
FaultTimeStamp= 2024-02-06 02:29:34Z
FaultPPIN= 0x0ba23bc5d0a79669
FaultCPUID= 0xa06d0
FaultMicrocode= 0x80000594
FaultReason= HW.MCE.UBOX:Generic_I/O_Error_On_Bus[0x14]_Dev[0x6]_Fun[0x0]_From_Seg[0x0].incomplete_dump.mca_same_value
FaultDevice= N/A
FaultInstance= N/A
FaultCode= CPU0.Ubox.MSCOD.0x0
FaultDescription= Generic IO error
FaultAction= CPU0: Contact your OEM debug team
FaultActionCode= hw.contact_oem_debug_team

```

```
##### Signature by Sockets #####
=====
|socket|Signature|
|-----|
|0      |HW.MCE.UBOX:Generic_I/O_Error_On_Bus[0x14]_Dev[0x6]_Fun[0x0]_From_Seg[0x0].incomplete_dump.mca_same_value|
|1      |INCOMPLETE_DUMP.MCA_SAME_VALUE|
|-----|

##### Status[63]=1 Machine Check summary #####
=====
|skt|die|cor|thr|bank|Status|Address|Misc|CTL|
|Error|
|-----|
|0 |io0|- |4 |0xba0000000000000b|0x0000000000000000|0x0000000014300000|0x0000000000000001|Ubox:Generic_I/
O_Error_On_Bus[0x14]_Dev[0x6]_Fun[0x0]_from_Seg[0x0]|
|0 |io0|- |4 |UPI0|0x88000004000300e0f|0x0000000000000000|0x40000020200ffbe1|0x0000000081ff0007|
Unmerged UPI0:COR_LL Rx detected CRC error: successful LLR without Phy Reinit|
|0 |io0|- |4 |UPI1|0x88000004000300e0f|0x0000000000000000|0x40000020200ffbe1|0x0000000081ff0007|
Unmerged UPI1:COR_LL Rx detected CRC error: successful LLR without Phy Reinit|
|0 |io1|- |4 |UPI0|0x88000004000300e0f|0x0000000000000000|0x40000020200ffbe1|0x0000000081ff0007|
Unmerged UPI0:COR_LL Rx detected CRC error: successful LLR without Phy Reinit|
|0 |io1|- |4 |UPI1|0x88000004000300e0f|0x0000000000000000|0x40000040200ffbe1|0x0000000081ff0007|
Unmerged UPI1:COR_LL Rx detected CRC error: successful LLR without Phy Reinit|
|1 |io0|- |4 |UPI0|0x88000004000300e0f|0x0000000000000000|0x40000000200ffbe1|0x0000000081ff0007|
Unmerged UPI0:COR_LL Rx detected CRC error: successful LLR without Phy Reinit|
|1 |io0|- |4 |UPI1|0x88000004000300e0f|0x0000000000000000|0x40000000200ffbe1|0x0000000081ff0007|
Unmerged UPI1:COR_LL Rx detected CRC error: successful LLR without Phy Reinit|
|1 |io1|- |4 |UPI0|0x88000004000300e0f|0x0000000000000000|0x40000020200ffbe1|0x0000000081ff0007|
Unmerged UPI0:COR_LL Rx detected CRC error: successful LLR without Phy Reinit|
|1 |io1|- |4 |UPI1|0x88000004000300e0f|0x0000000000000000|0x40000000200ffbe1|0x0000000081ff0007|
Unmerged UPI1:COR_LL Rx detected CRC error: successful LLR without Phy Reinit|
|-----|

##### Internal and UBOX Errors per socket #####
First Error Assertion SUMMARY by socket
Legend: 1 means yes
=====
|CPU socket|socket0 io0|socket0 io1|socket0 compute0|socket0 compute1|socket0 compute2|
|socket0 soc|socket1 io0|socket1 io1|socket1 compute0|socket1 compute1|socket1 compute2|
|socket1 soc|
|-----|

|Die|soc|io0|io1|compute0|compute1|compute2|
|soc|io0|io1|compute0|compute1|compute2|
|soc|
|First ierr [16:0]|---|---|---|---|---|
|---|---|---|---|---|---|
|---|
|First mcerr [16:0]|0x88 Global IEH/Ubox io0|---|---|---|---|
|---|---|---|---|---|---|
|---|
|Asserted Caterr#|0|---|---|---|---|
|---|0|---|---|---|---|
|---|
|Asserted ierr|0|---|---|---|---|
|---|0|---|---|---|---|
|---|
|Asserted mcerr|0|---|---|---|---|
|---|0|---|---|---|---|
|---|
|Asserted MSMI#|1|---|---|---|---|
|---|1|---|---|---|---|
|---|
|Asserted MSMI_ierr|1|---|---|---|---|
|---|1|---|---|---|---|
|---|
|Asserted MSMI_mcerr|1|---|---|---|---|
|---|0|---|---|---|---|
|---|
|SideBandTrkTO|0|---|---|---|---|
|---|0|---|---|---|---|
|---|
|SideBank_Err|0|---|---|---|---|
|---|0|---|---|---|---|
|---|
|SMITimeOut|0|---|---|---|---|
|---|0|---|---|---|---|
|---|
|PPIN|0x0ba24045f236bc4d|0x0ba24045f236bc4d|0x0ba24045f236bc4d|0x0ba24045f236bc4d|0x0ba24045f236bc4d|
0x0ba24045f236bc4d|0x0ba24045f236bc4d|0x0ba23bc5d0a79669|0x0ba23bc5d0a79669|0x0ba23bc5d0a79669|0x0ba23bc5d0a79669|
0x0ba23bc5d0a79669|0x0ba23bc5d0a79669|
|-----|
|-----|
|-----|
```

```
##### IERR and MCEERR Logs #####
```

```
***cpu0***
```

```
=====
|Die|Register          |Value      |Description
|-----|-----|-----|
|io0|pcu_cr_mca_err_src_log|0x001c0000|MSMI_IERR# Asserted
|   |   |   |MSMI_MCERR# Asserted
|io0|ierrloggingreg      |0x00000000|
|io0|mcerrloggingreg      |0x00010088|Bits [15:0] First MCERR Src Valid --> 0x88 Global IEH/Ubox io0 <--
|io0|uboxerrstatus       |0x00000000|
|io0|rmcerrloggingreg    |0x00000000|
=====
```

9.2 Viral: Intel UPI - Crash Dump

Test Objective

Upon an unrecoverable error condition, minimizing further error propagation is highly desired. Viral provides a method of providing high level of error containment. This test is to ensure that the Intel UPI uncorrectable error is contained and prevented from reaching permanent storage or network.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, IIntel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW5 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	Generic PCIe Card
CScripts Version	GNR 2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [CrashDump](#) on page 19
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log >
 - Viral Status = Enable

Recipe

NOTE

This recipe may result in CHA TOR Time-out/Three-Strike Timeout.

1. Ensure that the system is in viral mode. Ensure that the signal global fatal and viral alert enable are set.

```
>>> >>> ras.sys.viral_config_check()
=====
| Socket# | Description | Value |
|-----|-----|-----|
| socket0 | Software Error Recovery Support | Y |
|-----|-----|-----|
| socket1 | Software Error Recovery Support | Y |
|-----|-----|-----|
| socket0 | Poison Enable Bit Support | Y |
|-----|-----|-----|
| socket1 | Poison Enable Bit Support | Y |
|-----|-----|-----|
| socket0 | Viral Enable Bit | Y |
|-----|-----|-----|
| socket1 | Viral Enable Bit | Y |
|-----|-----|-----|
| socket0-io0 | IIO Viral Config | 0x3 |
|-----|-----|-----|
| socket0-io1 | IIO Viral Config | 0x3 |
|-----|-----|-----|
| socket1-io0 | IIO Viral Config | 0x3 |
|-----|-----|-----|
| socket1-io1 | IIO Viral Config | 0x3 |
|-----|-----|-----|
| socket0-io0 | HIOP Global Error Config | 0x0 |
|-----|-----|-----|
| socket0-io1 | HIOP Global Error Config | 0x0 |
|-----|-----|-----|
| socket1-io0 | HIOP Global Error Config | 0x0 |
|-----|-----|-----|
| socket1-io1 | HIOP Global Error Config | 0x0 |
|-----|-----|-----|
| socket0-io0 | UPI Viral Config | 0x17 |
|-----|-----|-----|
| socket0-io1 | UPI Viral Config | 0x17 |
|-----|-----|-----|
| socket1-io0 | UPI Viral Config | 0x17 |
|-----|-----|-----|
| socket1-io1 | UPI Viral Config | 0x17 |
|-----|-----|-----|
| socket0 | Ubox Viral Config | 0x19fc0000 |
|-----|-----|-----|
| socket1 | Ubox Viral Config | 0x19fc0000 |
|-----|-----|-----|
| socket0 | PCU Viral Control | 0xf04 |
|-----|-----|-----|
| socket1 | PCU Viral Control | 0xf04 |
|-----|-----|-----|
| socket0-mc0 | IMC Viral Config | 0x0 |
|-----|-----|-----|
| socket0-mc1 | IMC Viral Config | 0x0 |
|-----|-----|-----|
```

socket0-mc2	IMC Viral Config	0x0
socket0-mc3	IMC Viral Config	0x0
socket1-mc0	IMC Viral Config	0x0
socket1-mc1	IMC Viral Config	0x0
socket1-mc2	IMC Viral Config	0x0
socket1-mc3	IMC Viral Config	0x0
socket0-mc0	IMC Viral Status	0x0
socket0-mc1	IMC Viral Status	0x0
socket0-mc2	IMC Viral Status	0x0
socket0-mc3	IMC Viral Status	0x0
socket1-mc0	IMC Viral Status	0x0
socket1-mc1	IMC Viral Status	0x0
socket1-mc2	IMC Viral Status	0x0
socket1-mc3	IMC Viral Status	0x0

```

>>> sv.sockets.showsearch("ktiviral")
io0.uncore.upi.upi0.upi_regs.ktiviral = 0x17
io0.uncore.upi.upi1.upi_regs.ktiviral = 0x17
io1.uncore.upi.upi2.upi_regs.ktiviral = 0x17
io1.uncore.upi.upi4.upi_regs.ktiviral = 0x17
io1.uncore.upi.upi5.upi_regs.ktiviral = 0x17
io1.uncore.upi.upi3.upi_regs.ktiviral = 0x17
io0.uncore.upi.upi0.upi_regs.ktiviral = 0x17
io0.uncore.upi.upi1.upi_regs.ktiviral = 0x17
io1.uncore.upi.upi2.upi_regs.ktiviral = 0x17
io1.uncore.upi.upi4.upi_regs.ktiviral = 0x17
io1.uncore.upi.upi5.upi_regs.ktiviral = 0x17
io1.uncore.upi.upi3.upi_regs.ktiviral = 0x17
>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.ktiviral.show()
0x00000000 : kti_viral_state (31:31) --
0x00000000 : kti_viral_status (30:30) --
0x00000000 : kti_pkt_viral_set (28:28) --
0x00000000 : kti_unc_err_set (26:26) --
0x00000001 : kti_remote_viral_lpbk_en (04:04) --
0x00000001 : kti_global_viral_mask (02:02) --
0x00000001 : kti_signal_global_fatal (01:01) --
0x00000001 : kti_viral_alert_enable (00:00) --
>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.upiulviralctrl.show()
0x00000000 : upiul_fatal_set (28:28) --
0x00000000 : upiul_viral_set (27:27) --
0x00000000 : upiul_unc_err_set (26:26) --
0x00000001 : ufi_global_fatal_viral_mask (02:02) --
0x00000001 : ufi_fatal_alert_enable (01:01) --
0x00000001 : ufi_viral_alert_enable (00:00) --

```

2. Inject an error to generate a viral condition.

```

>>> ei.io.injectUPIError(socket=0, port=0, num_crcs=0, stop_injection=True)
*****
Socket: 0 io0

```

```

Uncore MCA from Socket: 0
=====
| Valid | DIE | MC Bank Name | Status | Addr | Misc | Ctl | Ctl2 |
=====
| * | io0 | UPI0 | 0xba00010000100e0f | 0x0 | 0x2000020000ffbe1 | 0x81ff0007 | 0x540000001 |
| * | io0 | UPI1 | 0x8800004000300e0f | 0x0 | 0x40000020200ffbe1 | 0x81ff0007 | 0x540000001 |
=====

#####
Socket: 0 io1

Uncore MCA from Socket: 0
=====
| Valid | DIE | MC Bank Name | Status | Addr | Misc | Ctl | Ctl2 |
=====
| * | io0 | UPI2 | 0xba00010000100e0f | 0x0 | 0x2000020000ffbe1 | 0x81ff0007 | 0x540000001 |
| * | io0 | UPI3 | 0x8800004000300e0f | 0x0 | 0x40000020200ffbe1 | 0x81ff0007 | 0x540000001 |
=====

#####
Socket: 0 soc

Uncore MCA from Socket: 0
Socket 0 : No errors logged in any Uncore-specific Machine Check Banks

#####
Socket: 0 Compute0

Socket 0 : No errors logged in any Core-specific Machine Check Banks

#####
Socket: 0 Compute1

Socket 0 : No errors logged in any Core-specific Machine Check Banks

#####
Socket: 0 Compute2

Socket 0 : No errors logged in any Core-specific Machine Check Banks

#####
Socket: 1 io0

Uncore MCA from Socket: 1
Socket 1 : No errors logged in any Uncore-specific Machine Check Banks

#####
Socket: 1 io1

Uncore MCA from Socket: 1
=====
| Valid | DIE | MC Bank Name | Status | Addr | Misc | Ctl | Ctl2 |
=====
| * | io0 | UPI2 | 0x8800004000300e0f | 0x0 | 0x40000020200ffbe1 | 0x81ff0007 | 0x540000001 |
| * | io0 | UPI3 | 0xba00010000100e0f | 0x0 | 0x2000000000ffbe1 | 0x81ff0007 | 0x540000001 |
=====

#####
Socket: 1 soc

Uncore MCA from Socket: 1
Socket 1 : No errors logged in any Uncore-specific Machine Check Banks

#####
Socket: 1 Compute0

Socket 1 : No errors logged in any Core-specific Machine Check Banks

#####
Socket: 1 Compute1

Socket 1 : No errors logged in any Core-specific Machine Check Banks

#####
Socket: 1 Compute2

Socket 1 : No errors logged in any Core-specific Machine Check Banks

=====
UPI_0: REGISTERS AND FIELDS DESCRIPTIONS
=====
Register: socket0.io0.uncore.upi0.upi_regs.kti_mc_st
valid = 0x1
Set by hardware when an uncorrectable or correctable error is logged.

uc = 0x1
Error Uncorrected Set for error which was not corrected.

en = 0x1
Error Enable Flag Set by hardware when MCA (uncorrectable) is signaled.

miscv = 0x1
KTI_MC_MISC valid flag Indicates (when set) that the MCI_MISC register contains additional
information regarding the error. See individual fields of MCI_MISC for details on which
are valid for a given error.

pcc = 0x1
Processor Context Corrupt Flag Bit set on all uncorrectable errors

cor_err_cnt = 0x4
Corrected Error Count 15-bit counter which is incremented on every corrected
error. MSB (bit 15) is a sticky overflow bit.

mscod_code = 0x10
MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based

```

```

on the standard MC ST overwrite rules. The Error Codes and handling are defined in the
HAS. The summary per ID is: Uncorrectable (UC) 0x00 - UC Phy Initialization Failure
(NumInit) 0x01 - UC Phy Detected Drift Buffer Alarm 0x02 - UC Phy Detected Latency Buffer
Rollover (obsolate for Gen5) 0x10 - UC LL Rx detected CRC error: unsuccessful LLR (entered
Abort state) 0x11 - UC LL Rx Unsupported/Undefined packet 0x12 - UC LL or Phy Control
Error 0x13 - UC LL Rx Parameter Exception 0x14 - UC LL TDX Failure 0x15 - UC LL SGX
Failure 0x16 - UC LL Tx SDC Parity Error 0x17 - UC LL Rx SDC Parity Error 0x18 - UC LL FLE
Failure 0x1F - UC LL Detected Control Error from UPI2UPI Correctable (COR) 0x20 - COR Phy
Initialization Abort 0x21 - COR Phy Reset 0x22 - COR Phy Lane failure, recovery in x8
width 0x23 - COR Phy L0c error corrected without Phy reset 0x24 - COR Phy L0c error
triggering Phy Reset 0x25 - COR Phy L0p exit error triggering Phy Reset 0x30 - COR LL Rx
detected CRC error: successful LLR without Phy Reinit 0x31 - COR LL Rx detected CRC error:
successful LLR with Phy Reinit

mccad int = 0x1
MCCAD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection
errors (MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_tpxarer=non-zero)
- SGX/TDX/FLE failures - SDC Parity Error Set to 1 for all other errors.
Encoding: 0 - Not Bus Interconnect Error 1 - Bus Interconnect Error

mccad pp = 0x3
MCCAD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors
(MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_tpxarer=non-zero)
such that mcaod[15:0]=0x0405. Set to 2 for all 'UC LL Rx Unsupported/Undefined Packet',
errors (MSCOD Code=0x11). Set to 2 for all SGX/TDX/FLE failures. Set to 2 for all SDC
Parity Error failures. Set to 3 for all other errors. Encoding is: 0x0 - Local Processor
Originated 0x1 - Local Processor Responded 0x2 - Local Processor Observed Error 0x3 -
Generic PP

mccad ii = 0x3
SCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection
errors (MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_tpxarer=non-zero)
- SGX/TDX/FLE failures - SDC Parity Error Set to 3 for all other errors.
Encoding is: 0x0 - Memory Access 0x1 - Reserved II 0x2 - I/O 0x3 - Other Transaction

mccad ll = 0x3
MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors
(MSCOD Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_tpxarer=non-zero)
such that mcaod[15:0]=0x0405. Set to 2 for all SDC Parity Error Set to 2 for SGX/TDX/FLE
Failures Set to 3 for all other errors Encoding is: 0x0 - Memory Access 0x1 -
Reserved II 0x2 - I/O 0x3 - Other Transaction 0x0 - Level 0 0x1 - Level 1 0x2 - Level 2
0x3 - Generic LL

ADDITIONAL INFO REGISTERS:
=> socket0.io0.uncore.upi.upi0.upi_regs.kti_mc_misc
0x00000002 : cor_err (63:56) -- Captures Correctable Errors, indicating all correctable error cases that have occurred since...
0x00000001 : phy_state (39:37) -- Link State from Physical Layer on error used for isolating lane in CRC error cases: 0x0 - ...
0x0000ffbe1 : flit_hdr (19:00) -- HDR bits captured: [0] - HIB (if not set other bits >4 in this log are not valid) [1] - VM ...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_cis
0x00000003 : s_link_up (16:15) -- 00: STxLinkUp FALSE, SRxLinkUp FALSE 01: STxLinkUp FALSE, SRxLinkUp TRUE 10: STxLinkUp TRU...
0x00000001 : s_wake_detected (04:04) -- 1: Wake detected on one or more lanes.
0x00000001 : sp_detect (02:02) -- 1: remote Rx detected by Tx
0x00000001 : sp_rx_calib (01:01) -- 1: Rx calib done; sw clears by writing 1 prior to speed change to force re-calib.
0x00000001 : c_op_speed (00:00) -- 0: slow speed 1: op speed Updated at entry and exit from ResetC

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_cls
0x00000003 : s_rcvd_l0c (29:26) -- Received L0c code
0x00000003 : s_sent_l0c (25:22) -- Sent L0c code

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_pls
0x00000003 : s_sent_l0c_p (25:22) -- Same description as bit field in KTIUREUT_PH_PLS register, but this is copy of prior to ...
0x00000001 : s_rst_rcvd_p (13:13) -- Same description as bit field in KTIUREUT_PH_PLS register, but this is copy of prior to ...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktils
0x00000001 : rx_normal_op_state (13:13) -- Reflects whether the LL Rx is in Normal Operation (i.e. has received at least on...
0x00000004 : retry_state (12:10) -- Reflects the current state of Local Retry State Machine 000 - Retry_Local_Normal 001 - R...
0x00000003 : init_state (09:08) -- Reflects the current Initialization State of the Link Layer, including any stall conditio...
0x00000001 : link_layer_retry_queue_consumed (07:00) -- Link Layer Retry Queue Consumed Number of Retry Queue entries curre...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktiles
0x00000003 : worst_llr_num_phyreinit_count (09:08) -- Worst Case Retry NUM PHY REINIT Highest NUM PHY REINIT count achieved ...
0x0000000f : worst_llr_numretry_count (03:00) -- Worst Case Link Layer Retry NUM RETRY Highest NUM RETRY count achieved sinc...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktilcl
0x00000003 : max_num_phy_reinit (13:12) -- Consecutive Phy ReInit? to RETRY ABORT Note: this value is directly encoded which...
0x0000000f : max_num_retry (11:08) -- Consecutive LLRs to Physical Layer Reinit Note: This value is directly encoded which 1...

=> socket0.io0.uncore.upi.upi0.upi_regs.bios_kti_err_st
0x00000001 : valid (63:63) -- Set by hardware when an uncorrectable or correctable error is logged.
0x00000001 : uc (61:61) -- Error Uncorrected Set for error which was not corrected.
0x00000001 : en (60:60) -- Error Enable Flag Set by hardware when MCA (uncorrectable) is signaled.
0x00000001 : miscv (59:59) -- BIOS_KTI_ERR_MISC valid flag Indicates (when set) that the BIOS_KTI_ERR_MISC register contains...
0x00000001 : pcc (57:57) -- Processor Context Corrupt Flag Bit set on all uncorrectable errors
0x00000004 : cor_err_cnt (52:38) -- Corrected Error Count 15-bit counter which is incremented on every corrected error. MSB...
0x00000010 : mscod_code (21:16) -- MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type base...
0x00000001 : mscod_int (11:11) -- MCCAD - Bus Interconnect Error Set to 0 for: - All Transmit Path Parity Protection erro...
0x00000003 : mscod_pp (10:09) -- MCCAD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors (MSCOD.C...
0x00000003 : mscod_ii (03:02) -- MSCOD Bus Interconnect Error II Set to 1 for: - All Transmit Path Parity Protection erro...
0x00000003 : mscod_ll (01:00) -- MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors (M...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktierrcnt0_cntr
0x00000005 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writin...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktierrcnt1_cntr
0x00000005 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writin...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktierrcnt2_cntr
0x00000005 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writin...

=> socket0.io0.uncore.upi.upi0.upi_regs.kticrrcnt
0x5cbfe23e : counter (30:00) -- Counter increments every time the Link Layer detects a CRC error.

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css
0x00000007 : s_clm (18:16) -- lane map code determined at Rx and sent in TX.Config for remote Tx; also updated on L0p entry/...
0x0000000f : s_rx_state (15:08) -- Rx State (encoding defined in Intel UPI specification)
0x0000000f : s_tx_state (07:00) -- Tx State (encoding defined in Intel UPI specification)

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ldc
0x00000030 : indrftbufflatency (13:08) -- Initial Drift Buffer Latency Drift Buffer Depth refers to the initialized diffe...
0x00000004 : drftbufalarmthr (05:00) -- Drift Buffer Alarm Threshold When the difference (both underflow and overflow) bet...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl
0x00000001 : c_rc_terms_off (29:29) -- 1: Rx pull downs are off in ResetC
0x00000001 : c_fallover_en (27:27) -- 1: Enable fallover
0x00000001 : c_ll_en (23:23) -- 1: enables LL
0x00000001 : c_init_begin (05:05) -- 0: blocks exit from ResetC
0x00000001 : c_op_speed (04:04) -- 1: operational speed 0: slow speed Note: The operational speed is determined by BIOS setu...

=> socket0.io0.uncore.upi.upi0.upi_regs.ktireut_ph_dis
0x00000380 : s_rx_obs_latency (19:08) -- Rx sync counter value when EIEOS of TS.Polling received at latency buffer.

-----
Register: socket0.io0.uncore.upi.upi0.upi_regs.ktiviral
kti_viral_state = 0x1
Indicates that the Intel UPI Agent is currently asserting Viral flow outgoing (Tx).

```

```

kti_viral_status = 0x1
    Indicates previous detection of a Viral condition sent.

kti_unc_err_set = 0x1
    Status that indicates the Intel UPI Agent has detected an uncorrectable error.
    Note 1 - If this bit is set, the Intel UPI Agent will continue to assert Global
    Fatal/Global Viral (if enabled)      Note 2 - uncorrectable error used here includes
    KTIERRDIS masking, but does NOT include KTI_MC_CTL masking

kti_global_viral_mask = 0x1
    Enables generation of Tx (outgoing) Viral Flow 0 - Block Injection of Viral on Intel UPI 1
    - Allow Injection of Viral on Intel UPI Note: polarity of this bit is treated as an enable

kti_signal_global_fatal = 0x1
    Enables assertion of Global Fatal by Intel UPI Agent due to internal detection of an
    uncorrectable error. 0 - Disabled 1 - Enabled

kti_viral_alert_enable = 0x1
    Enables assertion of Global Viral by Intel UPI Agent. 0 - Disabled 1 - Enabled

-----
Register: socket1.io0.uncore.upi.upi0.upi_regs.kti_mc_st
valid = 0x1
    Set by hardware when an uncorrectable or correctable error is logged.

miscv = 0x1
    KTI_MC_MISC valid flag indicates (when set) that the MCI_MISC register contains additional
    information regarding the error. See individual fields of MCI_MISC for details on which
    are valid for a given error.

cor_err_cnt = 0x1
    Corrected Error Count      15-bit counter which is incremented on every corrected
    error. MSB (bit 15) is a sticky overflow bit.

mscod_code = 0x30
    MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type based
    on the standard MC_ST overwrite rules. The Error Codes and handling are defined in the
    RAS. The summary ID is: Uncorrectable (UC) 0x00 - UC Phy Initialization Failure
    (NumInit) 0x01 - UC Phy Detected Drift Buffer Alarm 0x02 - UC Phy Detected Latency Buffer
    Rollover (obsolete for Gen3) 0x10 - UC LL Rx detected CRC error: unsuccessful LLR (entered
    Abort state) 0x11 - UC LL Rx Unsupported/Undefined packet 0x12 - UC LL or Phy Control
    Error 0x13 - UC LL Rx Parameter Exception 0x14 - UC LL TDX Failure 0x15 - UC LL SGX
    Failure 0x16 - UC LL Tx SDC Parity Error 0x17 - UC LL Rx SDC Parity Error 0x18 - UC LL FLE
    Failure 0x1F - UC LL Detected Control Error from UFI2UPI Correctable (COR) 0x20 - COR Phy
    Initialization Abort 0x21 - COR Phy Reset 0x22 - COR Phy Lane failure, recovery in x8
    width 0x23 - COR Phy L0C error corrected without Phy reset 0x24 - COR Phy L0C error
    triggering Phy Reset 0x25 - COR Phy L0p exit error triggering Phy Reset 0x30 - COR LL Rx
    detected CRC error: successful LLR without Phy Reinit 0x31 - COR LL Rx detected CRC error:
    successful LLR with Phy Reinit

mccod_int = 0x1
    MCCOD - Bus Interconnect Error Set to 0 for:
    errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-
    zero)
    - SGX/TDX/FLE failures
    - SDC Parity Error Set to 1 for all other errors.
    Encoding: 0 - Not Bus Interconnect Error 1 - Bus Interconnect Error

mccod_pp = 0x3
    MCCOD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors
    (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-zero)
    such that mccod[15:0]=0x0405. Set to 2 for all 'UC LL Rx Unsupported/Undefined Packet'
    errors (MSCOD_Code=0x11). Set to 2 for all SGX/TDX/FLE failures. Set to 2 for all SDC
    Parity Error Failures. Set to 3 for all other errors. Encoding is: 0x0 - Local Processor
    Originated 0x1 - Local Processor Responded 0x2 - Local Processor Observed Error 0x3 -
    Generic PP

mccod_ii = 0x3
    SCCOD Bus Interconnect Error II Set to 1 for:
    errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-
    zero)
    - SGX/TDX/FLE failures
    - SDC Parity Error Set to 3 for all other errors.
    Encoding is: 0x0 - Memory Access 0x1 - Reserved II 0x2 - I/O 0x3 - Other Transaction

mccod_ll = 0x3
    MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors
    (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-zero)
    such that mccod[15:0]=0x0405. Set to 1 for SDC Parity Error Set to 2 for SGX/TDX/FLE
    Failures Set to 3 for all other errors. Encoding is: 0x0 - Memory Access 0x1 -
    Reserved II 0x2 - I/O 0x3 - Other Transaction 0x0 - Level 0 0x1 - Level 1 0x2 - Level 2
    0x3 - Generic LL

ADDITIONAL INFO REGISTERS:
=> socket1.io0.uncore.upi.upi0.upi_regs.kti_mc_misc
0x00000000 : cor_err (63:56) -- Captures Correctable Errors, indicating all correctable error cases that have occurred since...
0x00000001 : phy_state (39:37) -- Link State from Physical Layer on error used for isolating lane in CRC error cases: 0x0 - ...
0x00000200 : syndrome_or_10cbadlanes (35:20) -- Captures Syndrome for CRC check, or in L0C error captures 8-bit vector of b...
0x0000fbfe : flit_hdr (19:00) -- HDR bits captured: [0] - HTB (if not set other bits >4 in this log are not valid) [1] - VN ...

=> socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_cis
0x00000003 : s_link_up (16:15) -- 00: STxLinkUp FALSE, SRxLinkUp FALSE 01: STxLinkUp FALSE, SRxLinkUp TRUE 10: STxLinkUp TRU...
0x00000001 : s_wake_detected (04:04) -- 1: Wake detected on one or more lanes.
0x00000001 : sp_detect (02:02) -- 1: remote Rx detected by TX
0x00000001 : sp_tx_calib (01:01) -- 1: Rx calib done; sw clears by writing 1 prior to speed change to force re-calib.
0x00000001 : s_op_speed (00:00) -- 0: slow speed 1: op speed Updated at entry and exit from ResetC

=> socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_cls
0x00000003 : s_rcvd_l0c (29:26) -- Received L0C code
0x00000003 : s_sent_l0c (25:22) -- Sent L0C code

=> socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_pls
0x00000003 : s_sent_l0c_p (25:22) -- Same description as bit field in KTI_REUT_PH_PLS register, but this is copy of prior to ...
0x00000001 : s_rst_rcvd_p (13:13) -- Same description as bit field in KTI_REUT_PH_PLS register, but this is copy of prior to ...

=> socket1.io0.uncore.upi.upi0.upi_regs.ktils
0x00000001 : rx_normal_op_state (13:13) -- Reflects whether the LL Rx is in Normal Operation (i.e., has received at least on...
0x00000003 : init_state (09:08) -- Reflects the current Initialization State of the link Layer, including any stall conditio...

=> socket1.io0.uncore.upi.upi0.upi_regs.ktils
0x00000001 : worst_llr_numretry_count (03:00) -- Worst Case Link Layer Retry NUM_RETRY Highest NUM_RETRY count achieved sinc...

=> socket1.io0.uncore.upi.upi0.upi_regs.ktilcl
0x00000003 : max_num_phy_reinit (13:12) -- Consecutive Phy ReInit's to RETRY_ABORT Note: this value is directly encoded whic...
0x0000000f : max_num_retry (11:08) -- Consecutive LLRs to Physical Layer ReInit Note: This value is directly encoded which i...

=> socket1.io0.uncore.upi.upi0.upi_regs.bios_kti_err_st
0x00000001 : valid (63:63) -- Set by hardware when an uncorrectable or correctable error is logged.
0x00000001 : miscv (59:59) -- BIOS_KTI_ERR_MISC valid flag indicates (when set) that the BIOS_KTI_ERR_MISC register contains...
0x00000001 : cor_err_cnt (52:38) -- Corrected Error Count      15-bit counter which is incremented on every corrected error. MSB...
0x00000030 : mscod_code (21:16) -- MSCOD Error Code Error Code from Intel UPI Agent. This field captures the error type base...
0x00000001 : mccod_int (11:11) -- MCCOD - Bus Interconnect Error Set to 0 for:
    - All Transmit Path Parity Protection errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-zero)
    such that mccod[15:0]=0x0405. Set to 2 for all 'UC LL Rx Unsupported/Undefined Packet' errors (MSCOD_Code=0x11). Set to 2 for all SGX/TDX/FLE failures. Set to 2 for all SDC Parity Error Failures. Set to 3 for all other errors. Encoding is: 0x0 - Local Processor Originated 0x1 - Local Processor Responded 0x2 - Local Processor Observed Error 0x3 - Generic PP
0x00000003 : mccod_pp (10:09) -- MCCOD - Bus Int Error PP Set to 2 for all Transmit Path Parity Protection errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-zero) such that mccod[15:0]=0x0405. Set to 2 for all 'UC LL Rx Unsupported/Undefined Packet' errors (MSCOD_Code=0x11). Set to 2 for all SGX/TDX/FLE failures. Set to 2 for all SDC Parity Error Failures. Set to 3 for all other errors. Encoding is: 0x0 - Local Processor Originated 0x1 - Local Processor Responded 0x2 - Local Processor Observed Error 0x3 - Generic PP
0x00000003 : mccod_ii (03:02) -- SCCOD Bus Interconnect Error II Set to 1 for:
    errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-zero)
    - SGX/TDX/FLE failures
    - SDC Parity Error Set to 3 for all other errors.
    Encoding is: 0x0 - Memory Access 0x1 - Reserved II 0x2 - I/O 0x3 - Other Transaction
0x00000003 : mccod_ll (01:00) -- MSCOD Bus Interconnect Error LL Set to 1 for all Transmit Path Parity Protection errors (MSCOD_Code=0x12 'UC LL or Phy Control Error' and KTIDBGERRST0.ctrl_txparerr=non-zero) such that mccod[15:0]=0x0405. Set to 1 for SDC Parity Error Set to 2 for SGX/TDX/FLE Failures Set to 3 for all other errors. Encoding is: 0x0 - Memory Access 0x1 - Reserved II 0x2 - I/O 0x3 - Other Transaction 0x0 - Level 0 0x1 - Level 1 0x2 - Level 2 0x3 - Generic LL

=> socket1.io0.uncore.upi.upi0.upi_regs.ktiercnt0_cnt
0x00000001 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writin...

```



```

=> socket1.io0.uncore.upi.upi0.upi_regs.ktierrcnt1 cntr
0x00000001 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writin...

=> socket1.io0.uncore.upi.upi0.upi_regs.ktierrcnt2 cntr
0x00000001 : counter (14:00) -- Counter increments based on error enabled in the Mask register. Counter is cleared by writin...

=> socket1.io0.uncore.upi.upi0.upi_regs.kticrrcnt
0x00000001 : counter (30:00) -- Counter increments every time the Link Layer detects a CRC error.

=> socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_css
0x00000007 : s_cim (18:16) -- Lane map code determined at Rx and sent in TX.Config for remote Tx; also updated on L0p entry/...
0x0000000f : s_rx_state (15:08) -- Rx State (encoding defined in Intel UPI specification)
0x0000000f : s_tx_state (07:00) -- Tx State (encoding defined in Intel UPI specification)

=> socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ldc
0x00000030 : inidriftbufflatency (13:08) -- Initial Drift Buffer Latency      Drift Buffer Depth refers to the initialized diffe...
0x00000004 : driftbufalarmthr (05:00) -- Drift Buffer Alarm Threshold      When the difference (both underflow and overflow) bet...

=> socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_ctrl
0x00000001 : c_rc_terms_off (29:29) -- 1: Rx pulldowns are off in ResetC
0x00000001 : c_fallover_en (27:27) -- 1: Enable failover
0x00000001 : c_ll_en (23:23) -- 1: enables LL
0x00000001 : c_det_slave (09:09) -- 0: Determinism master 1: Determinism slave
0x00000001 : c_init_begin (05:05) -- 0: blocks exit from ResetC
0x00000001 : c_op_speed (04:04) -- 1: operational speed 0: slow speed Note: The operational speed is determined by BIOS setu...

=> socket1.io0.uncore.upi.upi0.upi_regs.ktireut_ph_dis
0x000000c20 : s_rx_obs_latency (19:08) -- Rx sync counter value      when EIEOS of TS.Polling received at latency buffer.

-----
Register: socket1.io0.uncore.upi.upi0.upi_regs.ktiviral
kti_viral_state = 0x1
Indicates that the Intel UPI Agent is currently asserting Viral flow outgoing (Tx).

kti_viral_status = 0x1
Indicates previous detection of a Viral condition sent.

kti_pkt_viral_set = 0x1
Status that indicates the Intel UPI Agent has received Viral bit received in the LLR.Ack
message. Note: If this bit is set, the Intel UPI Agent will continue to assert Global
Viral (if enabled)

kti_global_viral_mask = 0x1
Enables generation of Tx (outgoing) Viral Flow 0 - Block Injection of Viral on Intel UPI 1
- Allow Injection of Viral on Intel UPI Note: polarity of this bit is treated as an enable

kti_signal_global_fatal = 0x1
Enables assertion of Global Fatal by Intel UPI Agent due to internal detection of an
uncorrectable error. 0 - Disabled 1 - Enabled

kti_viral_alert_enable = 0x1
Enables assertion of Global Viral by Intel UPI Agent. 0 - Disabled 1 - Enabled

-----
Fields with non default value
=====
| IP block | # fields with non-default value |
|-----|-----|
| MACHINE_CHECK | 0x6 |
| UPI_D | 0x1f |
=====

```

3. Ensure that the viral state, status, and unc_err_set are set on the corresponding port where the injection took place.

```

>>> sv.sockets.showsearch("ktiviral")
io0.uncore.upi.upi0.upi_regs.ktiviral = 0xc4000017
io0.uncore.upi.upi1.upi_regs.ktiviral = 0xd0000017
io1.uncore.upi.upi2.upi_regs.ktiviral = 0xd0000017
io1.uncore.upi.upi4.upi_regs.ktiviral = 0xd0000017
io1.uncore.upi.upi5.upi_regs.ktiviral = 0xd0000017
io1.uncore.upi.upi3.upi_regs.ktiviral = 0xd0000017
io0.uncore.upi.upi0.upi_regs.ktiviral = 0xd0000017
io0.uncore.upi.upi1.upi_regs.ktiviral = 0xc4000017
io1.uncore.upi.upi2.upi_regs.ktiviral = 0xd0000017
io1.uncore.upi.upi4.upi_regs.ktiviral = 0xd0000017
io1.uncore.upi.upi5.upi_regs.ktiviral = 0xd0000017
io1.uncore.upi.upi3.upi_regs.ktiviral = 0xd0000017
>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.ktiviral.show()
0x00000001 : kti_viral_state (31:31) --
0x00000001 : kti_viral_status (30:30) --
0x00000000 : kti_pkt_viral_set (28:28) --
0x00000001 : kti_unc_err_set (26:26) --
0x00000001 : kti_remote_viral_lpbk_en (04:04) --
0x00000001 : kti_global_viral_mask (02:02) --
0x00000001 : kti_signal_global_fatal (01:01) --
0x00000001 : kti_viral_alert_enable (00:00) --
>>> sv.socket0.io0.uncore.upi.upi0.upi_regs.upiulviralctrl.showfields
0x00000001 : upiul_fatal_set (28:28)
0x00000001 : upiul_viral_set (27:27)

```

```

0x00000000 : upiul_unc_err_set (26:26)
0x00000001 : ufi_global_fatal_viral_mask (02:02)
0x00000001 : ufi_fatal_alert_enable (01:01)
0x00000001 : ufi_viral_alert_enable (00:00)
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.showfields()
0x00000000 : caterr (31:31)
0x00000000 : ierr (30:30)
0x00000000 : mcerr (29:29)
0x00000000 : caterr_internal (28:28)
0x00000000 : ierr_internal (27:27)
0x00000000 : mcerr_internal (26:26)
0x00000000 : cam_parity_err (25:25)
0x00000000 : msmi_mcp_smbus (24:24)
0x00000000 : msmi (23:23)
0x00000000 : msmi_ierr (22:22)
0x00000000 : msmi_mcerr (21:21)
0x00000001 : msmi_internal (20:20)
0x00000001 : msmi_ierr_internal (19:19)
0x00000001 : msmi_mcerr_internal (18:18)
0x00000000 : rmca (17:17)
0x00000000 : rmsmi (16:16)
0x00000000 : rmca_internal (15:15)
0x00000000 : rmsmi_internal (14:14)
0x00000000 : first_core_ierr (04:00)

0x00000000 : caterr (31:31)
0x00000000 : ierr (30:30)
0x00000000 : mcerr (29:29)
0x00000000 : caterr_internal (28:28)
0x00000000 : ierr_internal (27:27)
0x00000000 : mcerr_internal (26:26)
0x00000000 : cam_parity_err (25:25)
0x00000000 : msmi_mcp_smbus (24:24)
0x00000000 : msmi (23:23)
0x00000000 : msmi_ierr (22:22)
0x00000000 : msmi_mcerr (21:21)
0x00000001 : msmi_internal (20:20)
0x00000001 : msmi_ierr_internal (19:19)
0x00000001 : msmi_mcerr_internal (18:18)
0x00000000 : rmca (17:17)
0x00000000 : rmsmi (16:16)
0x00000000 : rmca_internal (15:15)
0x00000000 : rmsmi_internal (14:14)
0x00000000 : first_core_ierr (04:00)

```

```

>>> ras.sys.viral_error_check()
=====
| Socket# | Register | Value |
|-----|-----|-----|
| *** | VIRAL Registers | *** |
|-----|-----|-----|
| socket0 | iio_viral_state | 0x1 |
|-----|-----|-----|
| socket0 | iio_viral_status | 0x1 |
|-----|-----|-----|
| socket0 | iio_global_viral_mask | 0x0 |
|-----|-----|-----|
| socket0 | iio_signal_global_fatal | 0x1 |
|-----|-----|-----|
| socket0 | iio_fatal_viral_alert_enable | 0x1 |
|-----|-----|-----|
| socket1 | iio_viral_state | 0x1 |
|-----|-----|-----|

```

socket1	iio_viral_status	0x1
socket1	iio_global_viral_mask	0x0
socket1	iio_signal_global_fatal	0x1
socket1	iio_fatal_viral_alert_enable	0x1
***	R2GLERRCFG Registers	***
socket0	a2ffatalstatus	0x0
socket0	a2fviralstatus	0x0
socket0	f2aviralstatus	0x1
socket0	maska2fviralout2f2aviralin	0x0
socket0	maska2ffatalout2f2aviralin	0x0
socket0	maska2fviralout	0x0
socket0	maskf2aviralin	0x0
socket0	maska2fffataltoviralout	0x0
socket0	maska2fffatalout	0x0
socket0	maskexternfatalin	0x0
socket0	maskexternviralin	0x0
socket0	resetglobalviral	0x0
socket0	resetglobalfatalerror	0x0
socket1	a2ffatalstatus	0x0
socket1	a2fviralstatus	0x0
socket1	f2aviralstatus	0x1
socket1	maska2fviralout2f2aviralin	0x0
socket1	maska2fffatalout2f2aviralin	0x0
socket1	maska2fviralout	0x0
socket1	maskf2aviralin	0x0
socket1	maska2fffataltoviralout	0x0
socket1	maska2fffatalout	0x0
socket1	maskexternfatalin	0x0
socket1	maskexternviralin	0x0
socket1	resetglobalviral	0x0
socket1	resetglobalfatalerror	0x0
***	KTIVIRAL Registers	***

socket0	kti_viral_state	0x1
socket0	kti_viral_status	0x1
socket0	kti_pkt_viral_set	?
socket0	kti_unc_err_set	?
socket0	kti_remote_viral_lpbk_en	0x1
socket0	kti_global_viral_mask	0x1
socket0	kti_signal_global_fatal	0x1
socket0	kti_viral_alert_enable	0x1
socket1	kti_viral_state	0x1
socket1	kti_viral_status	0x1
socket1	kti_pkt_viral_set	?
socket1	kti_unc_err_set	?
socket1	kti_remote_viral_lpbk_en	0x1
socket1	kti_global_viral_mask	0x1
socket1	kti_signal_global_fatal	0x1
socket1	kti_viral_alert_enable	0x1
***	UBOXGLERRCFG Registers	***
socket0	maskbadfusepull	0x1
socket0	maskincorrectbankindex	0x1
socket0	maskieherr2	0x0
socket0	maskuboxmsmi	0x0
socket0	maskextviral	0x1
socket0	maskpcuproxymsmi	0x3
socket0	maskpcuproxyumc	0x3
socket0	maskpcuproxyierr	0x3
socket0	maskmasterlocktimeout	0x0
socket0	masksmitimeout	0x0
socket0	maskcfgwraddrmisalign	0x0
socket0	maskcfgrdaddrmisalign	0x0
socket0	maskpoisonreception	0x0
socket0	maskunsupportopcode	0x0

socket0	maskuboxierr	0x0
socket0	maskuboxumc	0x0
socket0	resetuboxviral	0x0
socket0	resetuboxfatalerror	0x0
socket0	statusuboxviral	0x0
socket0	statusuboxfatalerror	0x1
socket1	maskbadfusepull	0x1
socket1	maskincorrectbankindex	0x1
socket1	maskieherr2	0x0
socket1	maskuboxmsmi	0x0
socket1	maskextviral	0x1
socket1	maskpcuproxymsmi	0x3
socket1	maskpcuproxyumc	0x3
socket1	maskpcuproxyierr	0x3
socket1	maskmasterlocktimeout	0x0
socket1	masksmitimeout	0x0
socket1	maskcfgwraddrmisalign	0x0
socket1	maskcfggrdaddrmisalign	0x0
socket1	maskpoisonreception	0x0
socket1	maskunsupportopcode	0x0
socket1	maskuboxierr	0x0
socket1	maskuboxumc	0x0
socket1	resetuboxviral	0x0
socket1	resetuboxfatalerror	0x0
socket1	statusuboxviral	0x0
socket1	statusuboxfatalerror	0x1
***	VIRAL_CONTROL Registers	***
socket0	en_punit_agg_network_error	0x1
socket0	en_pcu_fw_error	0x1
socket0	en_pcu_uc_error	0x1
socket0	en_pcu_hw_error	0x1
socket0	viral_log	0x0

socket0	viral_log_disable	0x0
socket0	emca_mode	0x1
socket0	viral_fast_pin_dis	0x0
socket0	viral_reset	0x0
socket1	en_punit_agg_network_error	0x1
socket1	en_pcu_fw_error	0x1
socket1	en_pcu_uc_error	0x1
socket1	en_pcu_hw_error	0x1
socket1	viral_log	0x0
socket1	viral_log_disable	0x0
socket1	emca_mode	0x1
socket1	viral_fast_pin_dis	0x0
socket1	viral_reset	0x0
***	DDR4_VIRAL_CTL Registers	***
socket0	dis_trk_addr_rf_par_err	0x0
socket0	dis_wdb_rd_ecc_persist	0x0
socket0	dis_ha_wr_data_mac_par_err	0x0
socket0	dis_wdb_rd_ecc_uc	0x0
socket0	dis_internal_error	0x0
socket0	dis_ddrt_uc_ecc_err	0x0
socket0	dis_cmi_illegal_opcode	0x0
socket0	dis_poison_err	0x0
socket0	dis_cmiucdataerr	0x0
socket0	dis_hbmcaperr	0x0
socket0	dis_hbmwrddqparerr	0x0
socket0	dis_link_fail	0x0
socket0	dis_uncorr_ufill_rd_err	0x0
socket0	dis_uncorr_demand_rd_err	0x0
socket0	dis_apppp_err	0x0
socket0	dis_uncorr_spr_err	0x0
socket0	dis_uncorr_pat_err	0x0

socket0	dis_rpq0_rpa_par_err	0x0
socket0	dis_wdb_par_err	0x0
socket0	dis_ha_wr_be_par_err	0x0
socket0	dis_ha_wr_data_par_err	0x0
socket1	dis_trk_addr_rf_par_err	0x0
socket1	dis_wdb_rd_ecc_persist	0x0
socket1	dis_ha_wr_data_mac_par_err	0x0
socket1	dis_wdb_rd_ecc_uc	0x0
socket1	dis_internal_error	0x0
socket1	dis_ddrt_uc_ecc_err	0x0
socket1	dis_cmi_illegal_opcode	0x0
socket1	dis_poison_err	0x0
socket1	dis_cmiucdataerr	0x0
socket1	dis_hbmcaperr	0x0
socket1	dis_hbmwrddqparerr	0x0
socket1	dis_link_fail	0x0
socket1	dis_uncorr_ufill_rd_err	0x0
socket1	dis_uncorr_demand_rd_err	0x0
socket1	dis_app_err	0x0
socket1	dis_uncorr_spr_err	0x0
socket1	dis_uncorr_pat_err	0x0
socket1	dis_rpq0_rpa_par_err	0x0
socket1	dis_wdb_par_err	0x0
socket1	dis_ha_wr_be_par_err	0x0
socket1	dis_ha_wr_data_par_err	0x0
***	IMC_VIRALCTL Registers	***
socket0	viral_ch0	0x0
socket0	viral_clear	0x0
socket1	viral_ch0	0x0
socket1	viral_clear	0x0

- Run the Crashdump either using Cscripts or ACD tool and extract the crashdump summary. The logs should reflect the error logs collected on CScripts. Refer to [CrashDump](#) section for more information on how to run and extract Crashdump. Then AC cycle the system.

```
>>> error.crashdump_summary()
HW.MCE.UPI:Unmerged_UPI0:Uc_Ll_Rx_Detected_Crc_Error.incomplete_dump.mca_zero_value

FaultCPU= GNR
FaultTimeStamp= 2024-02-19 19:13:25Z
FaultPPIN= 0x0ba87f429cc50b7d
FaultCPUID= 0xa06d0
FaultMicrocode= 0x80000594
FaultReason= HW.MCE.UPI:Unmerged_UPI0:Uc_Ll_Rx_Detected_Crc_Error.incomplete_dump.mca_zero_value
FaultDevice= N/A
FaultInstance= N/A
FaultCode= CPU0.UPI0.MSCOD.0x10
FaultDescription= Possible Voltage Issue, Board Socket Contamination or Part Related Issue. Verify system have latest microcode; it could be contamination on pins or
wrong Torque Setting/Installation. Remove CPU clean socket, apply correct torque.
FaultAction= CPU0: Monitor Failure Rate; Clean CPUx; Remove and Inspect CPUx; Inspect Motherboard pins. Contact your Intel representative on repeat error on same CPUx.
FaultActionCode= hw.check_margins

##### Signature by Sockets #####
=====
|socket|Signature|
=====
|0|HW.MCE.UPI:Unmerged_UPI0:Uc_Ll_Rx_Detected_Crc_Error.incomplete_dump.mca_zero_value|
|1|HW.MCE.UPI:Unmerged_UPI1:Uc_Ll_Rx_Detected_Crc_Error.incomplete_dump.mca_zero_value|
=====

##### Status[63]=1 Machine Check summary #####
=====
|skt|die|cor|thr|bank|Status|Address|Misc|CTL|
Error|
=====
-----
|0|io0|-|0|[UPI0]0xba00010000100e0f|0x0000000000000000|0x02000020000fffbf|0x0000000081ff0007|
Unmerged_UPI0:UC_LL_Rx detected CRC error|
|0|io0|-|0|[UPI1]0x8800004000300e0f|0x0000000000000000|0x40000020200fffbf|0x0000000081ff0007|
Unmerged_UPI1:COR_LL_Rx detected CRC error: successful LLR without Phy Reinit|
|0|io1|-|0|[UPI0]0x8800004000300e0f|0x0000000000000000|0x40000020200fffbf|0x0000000081ff0007|
Unmerged_UPI0:COR_LL_Rx detected CRC error: successful LLR without Phy Reinit|
|0|io1|-|0|[UPI1]0x8800004000300e0f|0x0000000000000000|0x40000020200fffbf|0x0000000081ff0007|
Unmerged_UPI1:COR_LL_Rx detected CRC error: successful LLR without Phy Reinit|
|0|io1|-|0|[UPI2]0x8800004000300e0f|0x0000000000000000|0x40000020200fffbf|0x0000000081ff0007|
Unmerged_UPI2:COR_LL_Rx detected CRC error: successful LLR without Phy Reinit|
|0|io1|-|0|[UPI3]0x8800004000300e0f|0x0000000000000000|0x40000020200fffbf|0x0000000081ff0007|
Unmerged_UPI3:COR_LL_Rx detected CRC error: successful LLR without Phy Reinit|
|1|io0|-|0|[UPI0]0x8800004000300e0f|0x0000000000000000|0x40000020200fffbf|0x0000000081ff0007|
Unmerged_UPI0:COR_LL_Rx detected CRC error: successful LLR without Phy Reinit|
|1|io0|-|0|[UPI1]0x8800004000300e0f|0x0000000000000000|0x40000020200fffbf|0x0000000081ff0007|
Unmerged_UPI1:UC_LL_Rx detected CRC error|
|1|io1|-|0|[UPI0]0x8800004000300e0f|0x0000000000000000|0x40000020200fffbf|0x0000000081ff0007|
Unmerged_UPI0:COR_LL_Rx detected CRC error: successful LLR without Phy Reinit|
|1|io1|-|0|[UPI1]0x8800004000300e0f|0x0000000000000000|0x40000020200fffbf|0x0000000081ff0007|
Unmerged_UPI1:COR_LL_Rx detected CRC error: successful LLR without Phy Reinit|
|1|io1|-|0|[UPI2]0x8800004000300e0f|0x0000000000000000|0x40000020200fffbf|0x0000000081ff0007|
Unmerged_UPI2:COR_LL_Rx detected CRC error: successful LLR without Phy Reinit|
|1|io1|-|0|[UPI3]0x8800004000300e0f|0x0000000000000000|0x40000020200fffbf|0x0000000081ff0007|
Unmerged_UPI3:COR_LL_Rx detected CRC error: successful LLR without Phy Reinit|
=====

##### Internal and UBOX Errors per socket #####
First Error Assertion SUMMARY by socket
Legend: 1 means yes
=====
|CPU socket|socket0 io0|socket1 io0|
=====
|Die|io0|io0|
|First ierr [16:0]|1|1|
|First mcerr [16:0]|0x117 UPI io0|0x617 UPI io0|
|Asserted Caterr#|0|0|
|Asserted ierr|0|0|
|Asserted mcerr|0|0|
|Asserted MSMI|1|1|
|Asserted MSMI ierr|1|1|
|Asserted MSMI mcerr|1|1|
|SideBandTrkTO|0|0|
|SideBank_Err|0|0|
|SWTTimeOut|0|0|
|PPIN|0x0ba8734351171c30|0x0ba87f429cc50b7d|
=====

##### IERR and MCEERR Logs #####

***cpu0***
=====
|Die|Register|Value|Description|
=====
|io0|pcu_cr_mca_err_src_log|0x001c0000|MSMI_IERR# Asserted
| | | |MSMI_MCERR# Asserted
|io0|ierrloggingreg|0x00000000| |
|io0|mcerrloggingreg|0x00010117|Bits [15:0] First MCERR Src Valid --> 0x117 UPI io0 <--|
|io0|uboxerrstatus|0x00000000|
|io0|rmcerrloggingreg|0x00000000|
=====

=====
|Die|Register|Value|
=====
|io0|pcu_first_ierr_tsc_lo_cfg|0x79e622f0|
|io0|pcu_first_ierr_tsc_hi_cfg|0x00000012|
|io0|pcu_first_mcerr_tsc_lo_cfg|0x79e62308|
|io0|pcu_first_mcerr_tsc_hi_cfg|0x00000012|
|io0|pcu_first_mca_tsc_lo_cfg|0x00000000|
|io0|pcu_first_mca_tsc_hi_cfg|0x00000000|
=====

***cpu1***
=====
|Die|Register|Value|Description|
=====
|io0|pcu_cr_mca_err_src_log|0x001c0000|MSMI_IERR# Asserted
| | | |MSMI_MCERR# Asserted
|io0|ierrloggingreg|0x00000000| |
|io0|mcerrloggingreg|0x00010617|Bits [15:0] First MCERR Src Valid --> 0x617 UPI io0 <--|
|io0|uboxerrstatus|0x00000000|
|io0|rmcerrloggingreg|0x00000000|
=====
```



```
=====
|Die|Register |Value |
=====
|io|pcu_first_ierr_tsc_lo_cfg |0x79e622fc|
|io|pcu_first_ierr_tsc_hi_cfg |0x00000012|
|io|pcu_first_mcerf_tsc_lo_cfg|0x79e64f20|
|io|pcu_first_mcerf_tsc_hi_cfg|0x00000012|
|io|pcu_first_rmca_tsc_lo_cfg |0x00000000|
|io|pcu_first_rmca_tsc_hi_cfg |0x00000000|
=====
>>> itp.pulsepwrgood()
```

9.3 TOR Timeout and Three-Strike Timeout

Test Objective

To facilitate implementation and validation of error handling flows in the event of a system hang, a core MLC three-strike timeout error injection function, `ei.injectThreeStrike()`, is available as part of the CScripts package. This test verifies the three-strike error injection and reporting. Typically, TOR Timeout will accompany a three-strike timeout which is reported by the MLC MCA banks.

Configuration

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW20 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+ on x86_64
HW Configuration	N/A
CScripts Version	GNR 2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [CrashDump](#) on page 19
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

Recipe

1. Boot to the OS and inject a three strike error.

```
>>> halt()
>>> ei.sys.injectThreeStrike(socket=0, verbose="True")
```

2. Run the Crashdump either using Cscripts or ACD tool and extract the crashdump summary. The logs should reflect the error logs collected on CScripts. Refer to [CrashDump](#) section for more information on how to run and extract Crashdump. Parse the Crash Dump summary file. Read through MCA ERROR SOURCE LOG to evaluate which CPU asserted IERR first. Read through MC3 to confirm a 3-strike timeout. Read CBO MCA logs to isolate offending transaction.

```
>>> error.crashdump_summary()
HW.MCE.B2CMI:Timeout_Error.incomplete_dump.mca_zero_value

FaultCPU= GNR
FaultTimeStamp= 2024-06-21 09:07:42
FaultPPIN= 0x0000000000000000
FaultCPUID= 0x0
FaultMicrocode= 0x00000000
FaultReason= HW.MCE.B2CMI:Timeout_Error.incomplete_dump.mca_zero_value
FaultDevice= N/A
FaultInstance= N/A
FaultCode= CPU0.B2CMI0.MSCOD.0x9
FaultDescription= M2M time out. This is a fatal error type
FaultAction= Monitor Failure Rate: On repeated error same CPUx isolated DIMM. If "MISCV" is valid: the MCI_MISC register contains additional information regarding
channel involved in the error. On subsequent Errors, Contact your Intel representative.
FaultActionCode= hw.contact_intel

##### Signature by Sockets #####
-----
|socket|Signature|
-----
|0|HW.MCE.B2CMI:Timeout_Error.incomplete_dump.mca_zero_value|
|1|HW.STUCK_TRANSACTION.TOR:Tor_Timeout.incomplete_dump.mca_zero_value|
-----
##### Status[63]=1 Machine Check summary #####
=====
---
|s|kt|die| |cor|thr|bank| |Status| |Address| |Misc| |CTL| |
|Error| | | | | | | | |
-----
---
|0| |io|0| | | |4| |0xba00000000000e0b|0x0000000000000000|0x0000000014380000|0x000000000000001f|Ubox:Generic_I/
0|Error_On_Bus[0x14]_Dev[0x7]_Fun[0x0]_frcs[0x0]|
|1| |compute2|16| |1| |3| |0xbe000000e18c0400|0x0000000000000000|0x0000000078218000|0x000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xe18c_0x0400|
|1| |compute2|17| |1| |3| |0xbe000000e18c0400|0x0000000000000000|0x000000007821c000|0x000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xe18c_0x0400|
|1| |compute2|19| |1| |3| |0xbe000000e18c0400|0x0000000000000000|0x0000000078220000|0x000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xe18c_0x0400|
|1| |compute2|19| |1| |3| |0xbe000000e18c0400|0x0000000000000000|0x0000000078224000|0x000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xe18c_0x0400|
|1| |compute2|20| |1| |3| |0xbe000000e18c0400|0x0000000000000000|0x0000000078228000|0x000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xe18c_0x0400|
|1| |compute2|22| |1| |3| |0xbe000000e18c0400|0x0000000000000000|0x0000000078230000|0x000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xe18c_0x0400|
|1| |compute2|23| |1| |3| |0xbe000000e18c0400|0x0000000000000000|0x0000000078234000|0x000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xe18c_0x0400|
|0| |soc| | | | |B2CMI0|0xba00000000009040|0x0000000000000000|0x0000000000001000|0x0000000000000eff3|
Unmerged_B2CMI0:timeout_error
|0| |soc| | | | |B2CMI1|0xba00000000009040|0x0000000000000000|0x0000000100000100|0x0000000000000eff3|
Unmerged_B2CMI1:timeout_error
|0| |soc| | | | |CHA0|0xfe0000000000c017a|0x000000001950ffff80|0x00000a4800016f086|0x0000003fffffffffff|
Unmerged_CHA0:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA1|0xfe0000000000c017a|0x00000000177e7ff00|0x0100a4800016f086|0x0000003fffffffffff|
Unmerged_CHA1:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA2|0xfe0000000000c017a|0x000000001a5e3ff00|0x0200a4800016f086|0x0000003fffffffffff|
Unmerged_CHA2:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA3|0xfe0000000000c017a|0x000000001777bfff40|0x0300a4800016f086|0x0000003fffffffffff|
Unmerged_CHA3:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA4|0xfe0000000000c017a|0x000000001621bfec0|0x0400a4800016f086|0x0000003fffffffffff|
Unmerged_CHA4:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA5|0xfe0000000000c017a|0x000000001aa0ffff80|0x0500a4800016f086|0x0000003fffffffffff|
Unmerged_CHA5:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA6|0xfe0000000000c017a|0x00000000156f3ff40|0x0600a4800016f086|0x0000003fffffffffff|
Unmerged_CHA6:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA7|0xfe0000000000c017a|0x0000000019b7bfff80|0x0700a4800016f086|0x0000003fffffffffff|
Unmerged_CHA7:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA8|0xfe0000000000c017a|0x0000000016683ff00|0x0800a4800016f086|0x0000003fffffffffff|
Unmerged_CHA8:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA9|0xfe0000000000c017a|0x00000000146c7ff80|0x0900a4800016f086|0x0000003fffffffffff|
Unmerged_CHA9:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA10|0xfe0000000000c017a|0x0000000017ab7ff40|0x0a00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA10:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA11|0xfe0000000000c017a|0x0000000017487fec0|0x0b00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA11:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA12|0xfe0000000000c017a|0x0000000016f4bfff80|0x0c00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA12:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA13|0xfe0000000000c017a|0x0000000014c3bffe40|0x0d00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA13:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA14|0xfe0000000000c017a|0x00000000183f7ff80|0x0e00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA14:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA15|0xfe0000000000c017a|0x00000000001a77fff80|0x0f00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA15:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA16|0xfe0000000000c017a|0x000000001147fff80|0x1000a4800016f086|0x0000003fffffffffff|
Unmerged_CHA16:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA17|0xfe0000000000c017a|0x0000000019ea3ff80|0x1100a4800016f086|0x0000003fffffffffff|
Unmerged_CHA17:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA18|0xfe0000000000c017a|0x0000000011af7ff80|0x1200a4800016f086|0x0000003fffffffffff|
Unmerged_CHA18:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA19|0xfe0000000000c017a|0x000000001211bfec0|0x1300a4800016f086|0x0000003fffffffffff|
Unmerged_CHA19:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA20|0xfe0000000000c017a|0x0000000019ea3ff40|0x1400a4800016f086|0x0000003fffffffffff|
Unmerged_CHA20:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA21|0xfe0000000000c017a|0x0000000019ea3fec0|0x1500a4800016f086|0x0000003fffffffffff|
Unmerged_CHA21:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA22|0xfe0000000000c017a|0x0000000014d2bfec0|0x1600a4800016f086|0x0000003fffffffffff|
Unmerged_CHA22:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA23|0xfe0000000000c017a|0x000000001a8c3ff80|0x1700a4800016f086|0x0000003fffffffffff|
Unmerged_CHA23:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA24|0xfe0000000000c017a|0x00000000120d7ff40|0x1800a4800016f086|0x0000003fffffffffff|
Unmerged_CHA24:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA25|0xfe0000000000c017a|0x0000000019d1bffe40|0x1900a4800016f086|0x0000003fffffffffff|
Unmerged_CHA25:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA26|0xfe0000000000c017a|0x0000000018797ff00|0x1a00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA26:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA27|0xfe0000000000c017a|0x000000001854bfff00|0x1b00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA27:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA28|0xfe0000000000c017a|0x00000000194c3ff00|0x1c00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA28:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA29|0xfe0000000000c017a|0x0000000014247ff40|0x1d00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA29:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA30|0xfe0000000000c017a|0x000000001853ff40|0x1e00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA30:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA31|0xfe0000000000c017a|0x00000000160d3ff80|0x1f00a4800016f086|0x0000003fffffffffff|
Unmerged_CHA31:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA32|0xfe0000000000c017a|0x000000001a60ffff00|0x2000a4800016f086|0x0000003fffffffffff|
Unmerged_CHA32:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA33|0xfe0000000000c017a|0x0000000015083ff40|0x2100a4800016f086|0x0000003fffffffffff|
Unmerged_CHA33:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA34|0xfe0000000000c017a|0x0000000019467ff80|0x2200a4800016f086|0x0000003fffffffffff|
Unmerged_CHA34:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA35|0xfe0000000000c017a|0x000000001503bffe40|0x2300a4800016f086|0x0000003fffffffffff|
Unmerged_CHA35:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
|0| |soc| | | | |CHA36|0xfe0000000000c017a|0x0000000014913ff40|0x2400a4800016f086|0x0000003fffffffffff|
Unmerged_CHA36:TOR_TIMEOUT_OriginalReq[WbInvd]_TorID[0]
```

```
[0] |soc| | | |CHA37| 0xfe00000000c017a|0x00000001a577ff80|0x2500a4800016f086|0x0000003fffffffff|
Unmerged_CHA37:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA38| 0xfe00000000c017a|0x00000001a727ff80|0x2600a4800016f086|0x0000003fffffffff|
Unmerged_CHA38:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA39| 0xfe00000000c017a|0x00000001a29bff00|0x2700a4800016f086|0x0000003fffffffff|
Unmerged_CHA39:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA40| 0xfe00000000c017a|0x000000018cbbff00|0x2800a4800016f086|0x0000003fffffffff|
Unmerged_CHA40:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA41| 0xfe00000000c017a|0x00000001848fff00|0x2900a4800016f086|0x0000003fffffffff|
Unmerged_CHA41:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA42| 0xfe00000000c017a|0x000000017907fe80|0x2a00a4800016f086|0x0000003fffffffff|
Unmerged_CHA42:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA43| 0xfe00000000c017a|0x000000015e07ff80|0x2b00a4800016f086|0x0000003fffffffff|
Unmerged_CHA43:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA44| 0xfe00000000c017a|0x000000016c23ff80|0x2c00a4800016f086|0x0000003fffffffff|
Unmerged_CHA44:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA45| 0xfe00000000c017a|0x00000001749fff40|0x2d00a4800016f086|0x0000003fffffffff|
Unmerged_CHA45:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA46| 0xfe00000000c017a|0x000000017fdfff40|0x2e00a4800016f086|0x0000003fffffffff|
Unmerged_CHA46:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA47| 0xfe00000000c017a|0x000000018a77ff40|0x2f00a4800016f086|0x0000003fffffffff|
Unmerged_CHA47:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA48| 0xfe00000000c017a|0x0000000155bfff80|0x3000a4800016f086|0x0000003fffffffff|
Unmerged_CHA48:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA49| 0xfe00000000c017a|0x000000019aefff40|0x3100a4800016f086|0x0000003fffffffff|
Unmerged_CHA49:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA50| 0xfe00000000c017a|0x0000000130fff80|0x3200a4800016f086|0x0000003fffffffff|
Unmerged_CHA50:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA51| 0xfe00000000c017a|0x000000016a53ff80|0x3300a4800016f086|0x0000003fffffffff|
Unmerged_CHA51:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA52| 0xfe00000000c017a|0x000000017cfbfff80|0x3400a4800016f086|0x0000003fffffffff|
Unmerged_CHA52:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA53| 0xfe00000000c017a|0x0000000104107ff00|0x3500a4800016f086|0x0000003fffffffff|
Unmerged_CHA53:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA54| 0xfe00000000c017a|0x00000001a167ff00|0x3600a4800016f086|0x0000003fffffffff|
Unmerged_CHA54:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA55| 0xfe00000000c017a|0x000000017a8bfff80|0x3700a4800016f086|0x0000003fffffffff|
Unmerged_CHA55:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA56| 0xfe00000000c017a|0x00000001819ffec0|0x3800a4800016f086|0x0000003fffffffff|
Unmerged_CHA56:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA57| 0xfe00000000c017a|0x000000018aabfff80|0x3900a4800016f086|0x0000003fffffffff|
Unmerged_CHA57:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA58| 0xfe00000000c017a|0x00000001589bffc0|0x3a00a4800016f086|0x0000003fffffffff|
Unmerged_CHA58:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA59| 0xfe00000000c017a|0x000000018e53ff80|0x3b00a4800016f086|0x0000003fffffffff|
Unmerged_CHA59:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA60| 0xfe00000000c017a|0x0000000199bfff80|0x3c00a4800016f086|0x0000003fffffffff|
Unmerged_CHA60:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA61| 0xfe00000000c017a|0x0000000140cfff80|0x3d00a4800016f086|0x0000003fffffffff|
Unmerged_CHA61:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA62| 0xfe00000000c017a|0x000000018b57ff80|0x3e00a4800016f086|0x0000003fffffffff|
Unmerged_CHA62:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA63| 0xfe00000000c017a|0x000000016277ff00|0x3f00a4800016f086|0x0000003fffffffff|
Unmerged_CHA63:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA64| 0xfe00000000c017a|0x000000017a9bfff80|0x4000a4800016f086|0x0000003fffffffff|
Unmerged_CHA64:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA65| 0xfe00000000c017a|0x000000015017ff80|0x4100a4800016f086|0x0000003fffffffff|
Unmerged_CHA65:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA66| 0xfe00000000c017a|0x0000000198fbf40|0x4200a4800016f086|0x0000003fffffffff|
Unmerged_CHA66:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA67| 0xfe00000000c017a|0x00000001a543ff00|0x4300a4800016f086|0x0000003fffffffff|
Unmerged_CHA67:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA68| 0xfe00000000c017a|0x000000016a57ff00|0x4400a4800016f086|0x0000003fffffffff|
Unmerged_CHA68:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA69| 0xfe00000000c017a|0x000000013127ff80|0x4500a4800016f086|0x0000003fffffffff|
Unmerged_CHA69:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA70| 0xfe00000000c017a|0x000000015d73ff80|0x4600a4800016f086|0x0000003fffffffff|
Unmerged_CHA70:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[0] |soc| | | |CHA71| 0xfe00000000c017a|0x000000018cefff40|0x4700a4800016f086|0x0000003fffffffff|
Unmerged_CHA71:TOR TIMEOUT OriginalReq[WbInvd]_TorID[0] |
[1] |soc| | | |CHA12| 0xfe00000000c1136|0x000000007e0c1100|0x0c3c817800100886|0x0000003fffffffff|
Unmerged_CHA12:TOR TIMEOUT OriginalReq[Drd]_TorID[30] |
[1] |soc| | | |CHA17| 0xfe00000000c1136|0x000000007e0c1080|0x113c817800100886|0x0000003fffffffff|
Unmerged_CHA17:TOR TIMEOUT OriginalReq[Drd]_TorID[30] |
[1] |soc| | | |CHA18| 0xfe00000000c1136|0x000000007e0c1180|0x123c817800100886|0x0000003fffffffff|
Unmerged_CHA18:TOR TIMEOUT OriginalReq[Drd]_TorID[30] |
[1] |soc| | | |CHA22| 0xfe00000000c1136|0x000000007e0c10c0|0x163c817800100886|0x0000003fffffffff|
Unmerged_CHA22:TOR TIMEOUT OriginalReq[Drd]_TorID[30] |
[1] |soc| | | |CHA23| 0xfe00000000c1136|0x000000007e0c1140|0x173c817800100886|0x0000003fffffffff|
Unmerged_CHA23:TOR TIMEOUT OriginalReq[Drd]_TorID[30] |
=====
```

=====
 ##### Internal and UBOX Errors per socket #####
 First Error Assertion SUMMARY by socket
 Legend: 1 means yes

CPU socket	socket0 io0	socket1 io0
Die	io0	io0
First ierr [16:0]	Core0 compute0	Core164 compute2
First mcerr [16:0]	B2CMI1 compute1	CHA57 compute0
Asserted Caterr#	1	1
Asserted ierr	0	0
Asserted mcerr	1	1
Asserted MSMI#	1	1
Asserted MSMI_ierr	1	1
Asserted MSMI_mcerr	1	1
SideBandTrkTO	0	0
SideBank Err	0	0
SMITimeOut	0	0
PPIN	0x0000000000000000	0x0000000000000000

=====
 ##### IERR and MCEERR Logs #####
 cpu0

Die	Register	Value	Description
	io0 pcu_cr_mca_err_src_log 0x141c0000		CATERR# Asserted
			MCERR# Asserted
			MSMI_IERR# Asserted
			MSMI_MCERR# Asserted
	io0 ierrloggingreg	0x3480000034800 Bits [15:0] First Ierr Src Valid --> Core0 compute0 <--	
		Bits [17] FirstIerrSrcID is from a CORE	
	io0 mcerrloggingreg	0x1536f00015376 Bits [15:0] First MCERR Src Valid --> B2CMI1 compute1 <--	
	io0 uboxerrstatus	0x00000000	
	io0 rmccerrloggingreg	0x00000000	
	Die Register	Value	
	io0 pcu_first_ierr_tsc_lo_cfg	0x620a886c	
	io0 pcu_first_ierr_tsc_hi_cfg	0x00000011	
	io0 pcu_first_mcerr_tsc_lo_cfg	0x58877e8	
	io0 pcu_first_mcerr_tsc_hi_cfg	0x00000010	
	io0 pcu_first_rmca_tsc_lo_cfg	0x00000000	

```

|io0|pcu_first_rmca_tsc_hi_cfg |0x00000000|
=====
***cpul***
=====
|Die|Register |Value |Description |
=====
|io0|pcu_cr_mca_err_src_log|0x14fc0000 |CATERR# Asserted |
| | | |MCERR# Asserted |
| | | |MSMI_IERR# Asserted |
| | | |MSMI_MCERR# Asserted |
|io0|ierrloggingreg |0x3581600035824|Bits [15:0] First Ierr Src Valid --> Core164 compute2 <--|
| | | |Bits [17] FirstIerrSrcID is from a CORE |
|io0|mcerrloggingreg |0x14ea600014f46|Bits [15:0] First MCERR Src Valid --> CHA57 compute0 <--|
|io0|uboxerrstatus |0x00000000 | |
|io0|rmcerrloggingreg |0x00000000 | |
=====
|Die|Register |Value |
=====
|io0|pcu_first_ierr_tsc_lo_cfg |0x10f72a9c|
|io0|pcu_first_ierr_tsc_hi_cfg |0x00000012|
|io0|pcu_first_mcerr_tsc_lo_cfg |0x978f1628|
|io0|pcu_first_mcerr_tsc_hi_cfg |0x00000011|
|io0|pcu_first_rmca_tsc_lo_cfg |0x00000000|
|io0|pcu_first_rmca_tsc_hi_cfg |0x00000000|
=====

```

3. Release the system and AC cycle.

```

>>> go()
**ac cycle**

```

9.4 Inject MCERR

Test Objective

To facilitate implementation and validation of error handling flows in the event of a system hang, an inject MCERR function is available as part of the CScripts package. This test triggers a machine check error for testing and debugging.

Configuration

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores B1 step
BIOS	BHSDCRB1.IPC.3544.P02.2409050016
OS Version	CentOS Stream 9
Kernel	5.14.0
HW Configuration	N/A
CScripts Version	GNR 2422.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [CrashDump](#) on page 19

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > IIO Error Enabling >
 - IIO MCA Support = Enable
 - IIO EDPC Support = Disable

NOTE

Ensure PCIe card is populated on the port and IOMCA is enabled. Ensure PCIe and IEH hierarchy are cleared and endpoints are properly configured. See [PCI Express* \(PCIe*\) Device Configuration](#) on page 31 for more information.

Recipe

1. Boot to the OS and inject MCERR.

```
>>> halt()
>>> ei.sys.injectMCERR(verbose=True)

#####
#
#          DOING MCERR ERROR INJECTION          #
#
#####
*****
*
*   Injecting (MCERR)                           *
*
*****
Injecting uncorrectable PCIE error to trigger MCERR.
Finding first available populated PCIe port.
Halting to do check on injector lock
INFO: BIOS knob 'Disable BIOS Done' is enabled!
Injectors already UNLOCKED on socket0.cpu.core0.thread0
Injectors already UNLOCKED on socket1.cpu.core0.thread0
Injectors already UNLOCKED, skipping SMM entry
INFO: Socket 0 'pxp0.rp0' is operating in PCIe mode!

Uncorrectable error detected in socket0.io0.uncore.pi5.pxp0.rp0.cfg.erruncsts.cte
bit!
*****
*
*   Verifying MCERR error was injected successfully...
*
*****
INFO: MCERR is detected on the system!
INFO: MCERR is detected on the system!
***      SOCKET 0      ***
*** MCA ERR SRC LOG ***

socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log
=====
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for any
reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected error
```

```

on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is O...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted to
assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

INFO: MCERR is detected on the system!
*** SOCKET 1 ***
*** MCA ERR SRC LOG ***
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log
=====
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for any
reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected error
on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is O...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted to
assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

INFO: MCERR is detected on the system!
[GPC core group] Resuming on 256 devices -- 16:47:15.035152 2024-09-12
[GPC core group] Halt on 256 devices -- 16:47:15.553597 2024-09-12
=====
|STAGE          |CHECKPOINT          |STATUS|
-----
|Setup          |Validate Injection Type|Pass  |
|Setup          |Validate Access      |Pass  |

```

```
|Setup          |Validate Sockets          |Pass |
|Setup          |PCIe injection            |Pass |
|Verification|Injection Verification    |Pass |
=====
```

2. Run the Crashdump either using Cscripts or ACD tool and extract the crashdump summary. The logs should reflect the error logs collected on CScripts. Refer to [CrashDump](#) section for more information on how to run and extract Crashdump. Parse the Crash Dump summary file. Read through MCA ERROR SOURCE LOG to evaluate which CPU asserted MCERR first. The test injected an uncorrectable memory error that was consumed by a memory read operation. The system signaled this error because the error was enabled in the control register of the MC Bank.

```
>>> error.crashdump_summary()
HW.MCE.UBOX:Generic_I/O_Error_On_Bus[0X14]_Dev[0X2]_Fun[0X0]_From_Seg[0X0].incomplete_dump.mca_zero_value

FaultCPU= GNR
FaultTimeStamp= 2024-09-12 16:51:48
FaultPPIN= 0x326fb15d872eb1af
FaultCPUID= 0x000a06d1
FaultMicrocode= 0x81000311
FaultReason= HW.MCE.UBOX:Generic_I/O_Error_On_Bus[0X14]_Dev[0X2]_Fun[0X0]_From_Seg[0X0].incomplete_dump.mca_zero_value
FaultDevice= N/A
FaultInstance= N/A
FaultCode= CPU0.UBOX.MSCOD.0x0
FaultDescription= Generic IO error
FaultAction= CPU0: Contact your OEM debug team
FaultActionCode= hw.contact_oem_debug_team

##### Signature by Sockets #####
=====
|socket|Signature|
-----|
|0|HW.MCE.UBOX:Generic_I/O_Error_On_Bus[0X14]_Dev[0X2]_Fun[0X0]_From_Seg[0X0].incomplete_dump.mca_zero_value|
|1|INCOMPLETE_DUMP.MCA_SAME_VALUE|
=====
##### Status[63]=1 Machine Check summary #####
=====
|s|kt|die|cor|thr|bank|Status|Address|Misc|CTL|Error|
-----|
|0|io0|-|-|4|0xb80000000000000b|0x0000000000000000|0x0000000014100000|0x0000000000000001f|Ubox:Generic_I/O_Error_On_Bus[0X14]_Dev[0X2]_Fun[0X0]_From_Seg[0X0]|
=====
##### Internal and UBOX Errors per socket #####
First Error Assertion SUMMARY by socket
Legend: 1 means yes
=====
|CPU socket|socket0 io0|socket1 io0|
-----|
|Die|io0|io0|
|First ierr [16:0]|---|---|
|First mcerr [16:0]|0x88 Global IEH/Ubox io0|---|
|Asserted Catter#|0|0|
|Asserted ierr|0|0|
|Asserted mcerr|0|0|
|Asserted MSMT#|1|0|
|Asserted MSMT ierr|0|0|
|Asserted MSMT mcerr|1|0|
|SideBandTrkTO|0|0|
|SideBand Err|0|0|
|SMITimeOut|0|0|
|PPIN|0x326fb15d872eb1af|0x326ea85dc508cccc|
=====
##### IERR and MCERR Logs #####
***cpu0***
=====
|Die|Register|Value|Description|
-----|
|io0|pcu_cr_mca_err_src_log|0x00140000|MSMT_MCERR# Asserted|
|io0|ierloggingreg|0x00000000|
|io0|mcerrloggingreg|0x00010088|Bits [15:0] First MCERR Src Valid --> 0x88 Global IEH/Ubox io0 <--|
|io0|uboxerrstatus|0x00000000|
|io0|rmccerrloggingreg|0x00000000|
=====
|Die|Register|Value|
-----|
|io0|pcu_first_ierr_tsc_lo_cfg|0x00000000|
|io0|pcu_first_ierr_tsc_hi_cfg|0x00000000|
|io0|pcu_first_mcerr_tsc_lo_cfg|0xa37029ac|
|io0|pcu_first_mcerr_tsc_hi_cfg|0x00000009|
|io0|pcu_first_rmca_tsc_lo_cfg|0x00000000|
|io0|pcu_first_rmca_tsc_hi_cfg|0x00000000|
=====
>>> go()
```

3. AC Cycle the platform.

NOTE

If PCIE card is not populated or IOMCA is not enabled, refer to Spoofing Recipe to verify MCERR

9.5 Inject IERR

Test Objective

To facilitate implementation and validation of error handling flows in the event of a system hang, an inject IERR function, is available as part of the CScripts package. This test injects a serious internal error and observe how system responds to it.

NOTE

Cscript uses UPI Link Layer uncorrectable error to generate IERR, UPI Link Layer uncorrectable error itself is MCERR, IERR source will depend on the traffic after UPILL uce injected. Mca_err_src_log and the first IERR in crashdump summary are only for reference, various results might be observed.

Configuration

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores B1 step
BIOS	BHSDCRB1.IPC.3544.P02.2409050016
OS Version	CentOS Stream 9
Kernel	5.14.0
HW Configuration	N/A
CScripts Version	GNR 2422.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [CrashDump](#) on page 19

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

Recipe

1. Boot to the OS and inject an IERR in the system.

```
In : halt()

[GPC core group] Halt on 256 devices -- 17:29:01.426924 2024-09-12
In : ei.sys.injectIERR()

#####
#
# DOING IERR ERROR INJECTION
#
#####
*****
*
* Injecting (IERR)
*
```



```

*
*****
Checking for available IERR methods...
Detected more than 1 socket present. Triggering IERR using ei.iio.injectUpiLLError.
Injecting UPI link layer disable on socket0, io0, ipi0

*****
*
*   Verifying IERR error was injected successfully...
*
*****
Wait for IERR trigger (5 secs)!
[GPC core group] Resuming on 256 devices -- 17:31:01.359460 2024-09-12
Warning - All targeted threads are already running...
[GPC core group] Resuming on 256 devices -- 17:31:01.359460 2024-09-12
Warning - All targeted threads are already running...

>>>In : sv.socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for any
reason). It is or(bit 30, bit29); ...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000001 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is 0...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000001 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected error
on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); func...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (b...
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted to
assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

>>> sv.socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for any
reason). It is or(bit 30, bit29); ...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000001 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is 0...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000001 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical

```

```
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected error
on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); func...
0x00000001 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (b...
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted to
assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

>>> error.check_machine_check_errors()

Checking errors in the following IPs:
machine_check
#####
Socket: 0 io0
Uncore MCA from Socket: 0
=====
=====
| Valid | Die | MC Bank Name | STATUS | ADDR
| MISC | CTL |
-----
| * | io0 | BANK #4 - UBOX | 0xba0000000000e0b | 0x0000000000000000 |
0x0000000029300000 | 0x000000000000001f |
| * | io0 | BANK #5 - UPI Merged | 0xa200000007000000 | 0x0000000000000001 |
0x0,CC:0x93,RC:0x0 | 0x0,CC:0x93,RC:0x0 |
| * | io0 | UPI0 | 0xfa00000020120e0f | 0x0 |
0x60000000000000 | 0x81ff0007 |
=====
#####
Socket: 1 io0
Uncore MCA from Socket: 1
=====
=====
| Valid | Die | MC Bank Name | STATUS | ADDR
| MISC | CTL |
-----
| * | io0 | BANK #5 - UPI Merged | 0xa200000007010000 | 0x0000000000000002 |
0x0,CC:0x93,RC:0x0 | 0x0,CC:0x93,RC:0x0 |
| * | io0 | UPI1 | 0xfa00000100120e0f | 0x0 |
0x80000000000000 | 0x81ff0007 |
=====
=====
```

2. Run the Crashdump either using Cscripts or ACD tool and extract the crashdump summary. The logs should reflect the error logs collected on CScripts. Refer to [CrashDump](#) section for more information on how to run and extract Crashdump.

Parse the Crash Dump summary file. Read through MCA ERROR SOURCE LOG to evaluate which CPU asserted MCERR and IERR first. The IERR injection allows to see how system responds when it observes a fatal error condition. The IERR will be asserted until system is reset to ensure the recovery.

```
>>> error.crashdump_summary()
HW.MCE.UBOX:Generic_I/O_Error_On_Bus[0X29]_Dev[0X6]_Fun[0X0]_From_Seg[0X0].incomplete_dump.mca_zero_value
FaultCPU= GNR
FaultTimeStamp= 2024-09-12 17:34:53
FaultPPIN= 0x326fb15d872eb1af
FaultCPUID= 0x000a06d1
FaultMicrocode= 0x8100311
FaultReason= HW.MCE.UBOX:Generic_I/O_Error_On_Bus[0X29]_Dev[0X6]_Fun[0X0]_From_Seg[0X0].incomplete_dump.mca_zero_value
FaultDevice= N/A
FaultInstance= N/A
FaultCode= CPU0.UBOX.MSCOD.0x0
FaultDescription= Generic IO error
FaultAction= CPU0: Contact your OEM debug team
FaultActionCode= hw.contact_oem_debug_team
##### Signature by Sockets #####
=====
|socket|Signature|
=====
|0|HW.MCE.UBOX:Generic_I/O_Error_On_Bus[0X29]_Dev[0X6]_Fun[0X0]_From_Seg[0X0].incomplete_dump.mca_zero_value|
|1|HW.STUCK_TRANSACTION_TORFor_Timeout.incomplete_dump.mca_zero_value|
=====
##### Internal and UBOX Errors per socket #####
First Error Assertion SUMMARY by socket
Legend: 1 means yes
=====
|CPU socket|socket0 io0|socket1 io0|
=====
|Die|io0|io0|
|First ierr [16:0]|Core91 computel|Core106 computel|
|First mcerr [16:0]|0x117 UPI io0|0x617 UPI io0|
|Asserted CATerr#|1|1|
|Asserted ierr|0|0|
|Asserted mcerr|1|1|
|Asserted MSMI#|1|1|
|Asserted MSMI_ierr|1|1|
|Asserted MSMI_mcerr|1|1|
|SideBandTrkTO|0|0|
|SideBank Err|0|0|
|SMTimeOut|0|0|
|PPIN|0x326fb15d872eb1af|0x326ea85dc508cccc|
=====
##### IERR and MCERR Logs #####
***cpu0***
=====
|Die|Register|Value|Description|
=====
|io0|pcu_cr_mca_err_src_log|0x14bc0000|CATERR# Asserted|
| | | |MCERR# Asserted|
| | | |MSMI_IERR# Asserted|
| | | |MSMI_MCERR# Asserted|
|io0|ierrloggingreg|0x348260003501b|Bits [15:0] First Ierr Src Valid --> Core91 computel <--|
| | | |Bits [17] FirstIerrSrcID is from a CORE|
|io0|mcerrloggingreg|0x1011700010117|Bits [15:0] First MCERR Src Valid --> 0x117 UPI io0 <--|
|io0|uboxerrstatus|0x00000000| |
|io0|rmcerrloggingreg|0x00000000| |
=====
|Die|Register|Value|
=====
|io0|pcu_first_ierr_tsc_lo_cfg|0xab3cd860|
|io0|pcu_first_ierr_tsc_hi_cfg|0x00000009|
|io0|pcu_first_mcerr_tsc_lo_cfg|0xa37029ac|
|io0|pcu_first_mcerr_tsc_hi_cfg|0x00000009|
|io0|pcu_first_rmca_tsc_lo_cfg|0x00000000|
|io0|pcu_first_rmca_tsc_hi_cfg|0x00000000|
=====
***cpu1***
=====
|Die|Register|Value|Description|
=====
|io0|pcu_cr_mca_err_src_log|0x14fc0000|CATERR# Asserted|
| | | |MCERR# Asserted|
| | | |MSMI_IERR# Asserted|
| | | |MSMI_MCERR# Asserted|
|io0|ierrloggingreg|0x3502b0003502a|Bits [15:0] First Ierr Src Valid --> Core106 computel <--|
| | | |Bits [17] FirstIerrSrcID is from a CORE|
|io0|mcerrloggingreg|0x1061700010617|Bits [15:0] First MCERR Src Valid --> 0x617 UPI io0 <--|
|io0|uboxerrstatus|0x00000000| |
|io0|rmcerrloggingreg|0x00000000| |
=====
|Die|Register|Value|
=====
|io0|pcu_first_ierr_tsc_lo_cfg|0xab3cd8b0|
|io0|pcu_first_ierr_tsc_hi_cfg|0x00000009|
|io0|pcu_first_mcerr_tsc_lo_cfg|0x96b55e34|
|io0|pcu_first_mcerr_tsc_hi_cfg|0x00000006|
|io0|pcu_first_rmca_tsc_lo_cfg|0x00000000|
|io0|pcu_first_rmca_tsc_hi_cfg|0x00000000|
=====
>>> go()
```

3. AC Cycle the platform.

9.6 Error Spoofing (AMEI)

Error Spoofing, also called Asynchronous MCA Error Injection (AMEI), is a mechanism by which the UEFI FW can write to MCA Banks and trigger an event simulating an error without injecting a real error within the HW. It is primarily useful for SW, OS, and FW verification.

NOTE

The error event happens in SMM and calls the BIOS SMM handler. May not work for use cases where the customer wants to validate OS handler.

9.6.1 AWR Flow: Spoofed AMEI CHA TOR TO

Test Objective

To facilitate implementation and validation of error handling flows in the event of a system hang, a spoofed CHA TOR timeout error injection function, is available as part of the CScripts package. This test verifies TOR TO injection and reporting.

Configuration

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW15 BKC
OS Version	CentOS* Stream 8
Kernel	5.19.0-gnr.po.bkc.3.13.14.x86_64
HW Configuration	N/A
CScripts Version	GNR 2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMSG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [CrashDump](#) on page 19

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

Recipe

1. Boot to the OS and inject the spoofed CHA TOR TO error.

```
>>> halt()
[GPC core group] Halt on 224 devices -- 11:05:18.823375 2024-04-30
>>> ei.sys.AMEI(verbose="True", validation="True")

*****
GNR AMEI Tool
*****
Select Input Mode [1,2]
Enter Q or q to exit

1: Error Event Selection mode
2: Raw Data Input mode

>>> 1
0:IFU Parity Error (MCA)
```

```

1:IFU Trusted Paths Error (MCA)
2:DCU data load poison (MCA)
3:MLC 3 Strike Timeout (MCA)
4:CHA TOR Timeout (MCA)
5:UPI UC Rx detected CRC error (MCA)
6:LLC Uncorrectable data error (MCA)

*****
GNR AMEI Tool (Error Event Input Mode)
*****
Enter Error Event Number
Enter Q or q to exit

>>> 4
>>> 0
Bank7, (CHA)
Initiating an SMI
[GPC core group] Resuming on 224 devices -- 11:06:52.946850 2024-04-30
Enter SMM completed
Section doesn't exist: Register definition
Section doesn't exist: Register definition
Section doesn't exist: Register definition
Trigger MCA Error signal...
Clear AMEI control registers after the error injection...
[GNR0_C0_T0] SMM entry Break at [0x10:0000000000008000] -- 11:07:10.037252
2024-04-30
[GNR1_C87_T1] Resuming -- 11:07:19.199600 2024-04-30
Exit SMM Mode...

Exit SMM completed
Procedure completed succesfully, use error.check_machine_check_errors() or
crashdump() to verify injection

=====
|STAGE          |CHECKPOINT                               |STATUS|
|-----|
|Setup          |Halt Validation                         |Pass  |
|Setup          |Advanced RAS Validation                 |Pass  |
|Setup          |Access Validation                      |Pass  |
|Setup          |Enable AMEI                           |Pass  |
|Setup          |Error Signal Validation                 |Pass  |
|Verification   |Error Injection Verification            |Pass  |
|Verification   |Lock MCA/CMCI generation                |Pass  |
|-----|

{'results': [{'STAGE': 'Setup',
  'CHECKPOINT': 'Halt Validation',
  'STATUS': 'Pass'},
  {'STAGE': 'Setup',
  'CHECKPOINT': 'Advanced RAS Validation',
  'STATUS': 'Pass'},
  {'STAGE': 'Setup',
  'CHECKPOINT': 'Access Validation',
  'STATUS': 'Pass'},
  {'STAGE': 'Setup',
  'CHECKPOINT': 'Enable AMEI',
  'STATUS': 'Pass'},
  {'STAGE': 'Setup',
  'CHECKPOINT': 'Error Signal Validation',
  'STATUS': 'Pass'},
  {'STAGE': 'Verification',
  'CHECKPOINT': 'Error Injection Verification',
  'STATUS': 'Pass'}],

```

```
{'STAGE': 'Verification',
'CHECKPOINT': 'Lock MCA/CMCI generation',
'STATUS': 'Pass'}}}
```

- Run the Crashdump either using Cscripts or ACD tool and extract the crashdump summary. The logs should reflect the error logs collected on CScripts. Refer to [CrashDump](#) section for more information on how to run and extract Crashdump. Parse the Crash Dump summary file. Read through MCA ERROR SOURCE LOG to evaluate which CPU asserted MCERR first. There should be no 3-strike timeout. Socket 0's CHA0 TOR Timeout OriginalReq should neither show lock nor IO type. CHA MC 7 should populate.

```
>>> error.crashdump_summary()
HW.STUCK_TRANSACTION.TOR:Tor_Timeout.incomplete_dump.mca_zero_value

FaultCPU= GNR
FaultTimeStamp= 2024-04-30 18:07:10Zf
FaultPPIN= 0x0bfd9432b536c27
FaultCPUID= 0xa06d0
FaultMicrocode= 0x80000620
FaultReason= HW.STUCK_TRANSACTION.TOR:Tor_Timeout.incomplete_dump.mca_zero_value
FaultDevice= N/A
FaultInstance= N/A
FaultCode= CPU0.CHA0.MSCOD.0xc
FaultDescription= A stuck transaction in Cbo TOR. The Cbo Table of Request (TOR) timer has expired on a transaction. This status value should be accompanied by valid
MISC and ADDR bank register values. The transaction address and type are encoded in those registers. The MISC register will log the: Core ID, Thread ID, Cacheability,
Original Request and Opcode. The Original Request field should be used to note the request type. The opcode field will likely state a TOR Timeout and may not be as
useful. The ADDR register contains the target physical address. It should be noted that the address field does not have byte granularity (lower 6 bits are not decoded).
this will make exact access difficult, but we get into the right area. The process for this error is to isolate the Original request, core/thread id and address and to
map them back via a system memory mapping function. This is normally taken from the system when it boots up again. This decode is normally accompanied by an MLC (three
strike timeout) machine check. The address logged in that bank will log the EIP value at the time of the three strike. The core id logged in the Cbo timeout MISC
register should align with the core MLC machine check. In the case of multiple hits this may be used to correlate multiple stuck transactions to the cores that allocated
these transactions.
FaultAction= CPU0: Decode TOR_DUMP; Decode Address Dest; TorDump will have more details
FaultActionCode= hw.contact_intel

##### Signature by Sockets #####
=====
|socket|Signature|
|-----|-----|
|0|HW.STUCK_TRANSACTION.TOR:Tor_Timeout.incomplete_dump.mca_zero_value|
|1|INCOMPLETE_DUMP.MCA_ZERO_VALUE|
|-----|-----|

##### Status[63]=1 Machine Check summary #####
=====
|skt|die|cor|thr|bank|Status|Address|Misc|CTL|Error|
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
|0|soc|-|-|CHA0|0xfe20000000c017a|0x0000000063298e40|0x0805240000169086|0x00000003ffffffff|Unmerged_CHA0:TOR_TIMEOUT_OriginalReq[UNKNOWN: 584]_TorID[0]|
|-----|-----|-----|-----|-----|-----|-----|-----|-----|

##### Internal and UBOX Errors per socket #####
First Error Assertion SUMMARY by socket
Legend: 1 means yes
=====
|CPU socket|socket0.io0|socket1.io0|
|-----|-----|-----|
|Die|io0|io0|
|First ierr [16:0]|---|---|
|First mcerr [16:0]|Core0 compute0|---|
|Asserted Caterr#|1|0|
|Asserted ierr|0|0|
|Asserted mcerr|1|0|
|Asserted MSMI#|0|0|
|Asserted MSMI_ierr|0|0|
|Asserted MSMI_mcerr|0|0|
|SideBankTrkTO|0|0|
|SideBank_Err|0|0|
|SMITimeOut|0|0|
|PPIN|0x0bfdc841b39d335a|0x0bfd9432b536c27|
|-----|-----|-----|

##### IERR and MCEERR Logs #####
***cpu0***
=====
|Die|Register|Value|Description|
|-----|-----|-----|-----|
|io0|pcu_cr_mca_err_src_log|0x14000000|CATERR# Asserted|
| | | |MCERR# Asserted|
|io0|ierrloggingreg|0x00000000| |
|io0|mcerrloggingreg|0x00034800|Bits [15:0] First MCERR Src Valid --> Core0 compute0 <--|
| | | |Bits [17] FirstMcerrSrcID is from a CORE|
|io0|uboxerrstatus|0x00000000| |
|io0|rmcerrloggingreg|0x00000000| |
|-----|-----|-----|-----|

>>> go()
```

- Collect serial log and confirm AWR triggered.

```
CollectPrevBootFatalErrors: inAWR: 1
Socket: 0, Machine Check Bank 7: Status: 0xFE20000000C017A, Address: 0x0000000063298E40,
Misc: 0x0805240000169086
```

- AC Cycle the platform.

9.6.2 Manually Spoofed AMEI DCU

Test Objective

To facilitate implementation and validation of error handling flows in the event of a system hang, a spoofed DCU data load poison (coming from MLC), is available as part of the CScripts package. This test verifies error injection in DCU and reporting.

Configuration

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW35 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+ on x86_64
HW Configuration	N/A
CScripts Version	GNR 2422.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [CrashDump](#) on page 19

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

Recipe

1. Boot to the OS and inject the spoofed DCU with Data - loading poison (coming from MLC).
 - Valid: 1
 - UC: 1
 - EN: 1
 - PCC: 1
 - MSCOD: 0x0010
 - MCACOD: 0x0134

```
>>> halt()
[GPC core group] Halt on 288 devices -- 09:16:46.447987 2024-09-16
>>> ei.sys.AMEI(verbose="True", validation="True")

*****
GNR AMEI Tool
*****
Select Input Mode [1,2]
Enter Q or q to exit

1: Error Event Selection mode
```

```

2: Raw Data Input mode

Enter input mode [1, 2]:2

*****
GNR AMEI Tool (RAW Data Input Mode)
*****
Enter Socket, Bank #, MCi_STATUS, MCi_ADDR, MCi_MISC values
Enter Q or q to exit

Enter socket [0, 1]:0
Enter Bank number {0: 'IFU', 1: 'DCU', 2: 'DTLB', 3: 'MLC', 4: 'UBOX', 5: 'UPI', 6:
'PCU', 7: 'CHA', 9: 'LCC', 11: 'MSE', 12: 'B2CMI', 13: 'MCCHAN0', 14: 'MCCHAN1', 15:
'MCCHAN2', 16: 'MCCHAN3', 17: 'MCCHAN4', 18: 'MCCHAN5', 19: 'MCCHAN6', 20:
'MCCHAN7', 21: 'MCCHAN8', 22: 'MCCHAN9', 23: 'MCCHAN10', 24: 'MCCHAN11'}:1
Enter core number [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18,
19, 20, 21, 22, 23, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75,
76, 77, 78, 79, 80, 81, 82, 83, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129,
130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143]:0
Enter error signal number {1: 'mca', 2: 'cmci', 3: 'eMCA2 - mcerr/msmi', 4: 'eMCA2 -
cmci/csmi'}:1
Bank1, (DCU)
Enter MCi_STATUS value:b200000000100134
Enter MCi_ADDR value:0
Enter MCi_MISC value:0
Initiating an SMI

[GPC core group] Resuming on 288 devices -- 09:16:46.447987 2024-09-16
Enter SMM completed
WARNING: This type of error signal can cause OS to hang

[GNR0_C0_T0] SMM entry Break at [0x10:0000000000008000] -- 09:17:31.906720
2024-09-16
[GNR1_C143_T1] SMM entry Break at [0x10:0000000000008000] -- 09:17:32.213506
2024-09-16
Trigger MCA Error signal...
Clear AMEI control registers after the error injection ...

[GNR0_C0_T0] Resuming -- 09:17:44.543455 2024-09-16
[GNR1_C143_T1] Resuming -- 09:17:45.082436 2024-09-16
Exit SMM Mode...

Exit SMM completed
Procedure completed succesfully, use error.check_machine_check_errors() or
crashdump() to verify injection

*****
GNR AMEI Tool
*****
Select Input Mode [1,2]
Enter Q or q to exit

1: Error Event Selection mode
2: Raw Data Input mode

Enter input mode [1, 2]:q
=====
|STAGE|CHECKPOINT|STATUS|
-----|-----|-----|
|Setup|Halt Validation|Pass|
|Setup|Advanced RAS Validation|Pass|
|Setup|Access Validation|Pass|
|Setup|Enable AMEI|Pass|

```


Setup	Register definition	Pass	
Setup	Error Signal Validation	Pass	
Verification	Error Injection Verification	Pass	
Verification	Lock MCA/CMCI generation	Pass	
=====			

2. Read through registers to evaluate which CPU asserted MCERR first. DCU MC Bank 1 should populate.

```
>>> sv.socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for any reason). It is or(bit 30, bit29); functions a...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable MCERR.
0x00000001 : caterr_internal (28:28) -- Internal error: This socket asserted or attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : mcerf_internal (26:26) -- Internal error: This socket asserted unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any reason). It is or(bit 22, bit21); functions as a ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed unrecoverable MSMI_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or attempted to assert a MSMI#. This is OR (bit 19,18).
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted to assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

>>> sv.socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for any reason). It is or(bit 30, bit29); functions a...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to assert IERR.
0x00000000 : mcerf_internal (26:26) -- Internal error: This socket asserted unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any reason). It is or(bit 22, bit21); functions as a ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed unrecoverable MSMI_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or attempted to assert a MSMI#. This is OR (bit 19,18).
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted to assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

>>> sv.socket0.io0.uncore.showsearch("mcerrloggingreg")
ubox.ncevents.mcerrloggingreg = 0x34800

>>> sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1' if the SecondMCerrSrcID is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with the second MCERR
0x00000001 : firstmcerrsrcfromcore (17:17) -- Set to '1' if the FirstMCerrSrcID is from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00004800 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with the first MCERR

>>> sv.socket1.io0.uncore.showsearch("mcerrloggingreg")
ubox.ncevents.mcerrloggingreg = 0x0

>>> sv.socket0.io0.uncore.showsearch("ierrloggingreg")
ubox.ncevents.ierrloggingreg = 0x0

>>> sv.socket1.io0.uncore.showsearch("ierrloggingreg")
ubox.ncevents.ierrloggingreg = 0x0

>>> go()
```

3. AC Cycle the platform.

9.6.3 Spoofed AMEI UPI

Test Objective

To facilitate implementation and validation of error handling flows in the event of a system hang, a spoofed UPI UC Rx detected CRC error injection function, is available as part of the CScripts package. This test verifies CRC error injection on UPI UC-LL-Rx and reporting.

Configuration

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW20 BKC
OS Version	RHEL 9.2
<i>continued...</i>	

Kernel	6.7.0-rc4-2023-12-04-intel-next+ on x86_64
HW Configuration	N/A
CScripts Version	GNR 2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMSG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [CrashDump](#) on page 19

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

Recipe

1. Boot to the OS and inject the spoofed UPI UC Rx detected CRC error.

```
>>> halt()
[GPC core group] Halt on 288 devices -- 09:38:44.284394 2024-06-13
>>> ei.sys.AMEI(verbose="True", validation="True")

*****
GNR AMEI Tool
*****
Select Input Mode [1,2]
Enter Q or q to exit

1: Error Event Selection mode
2: Raw Data Input mode

>>> 1
0:IFU Parity Error (MCA)
1:IFU Trusted Paths Error (MCA)
2:DCU data load poison (MCA)
3:MLC 3 Strike Timeout (MCA)
4:CHA TOR Timeout (MCA)
5:UPI UC Rx detected CRC error (MCA)
6:LLC Uncorrectable data error (MCA)

*****
GNR AMEI Tool (Error Event Input Mode)
*****
Enter Error Event Number
Enter Q or q to exit

>>> 5
>>> 0
Bank5, (UPI)
Initiating an SMI
[GPC core group] Resuming on 288 devices -- 09:39:13.584344 2024-06-13
Enter SMM completed
Section doesn't exist: Register definition
Section doesn't exist: Register definition
Section doesn't exist: Register definition
Trigger MCA Error signal...
Clear AMEI control registers after the error injection ...
```

```
[GNR0_C0_T0] SMM entry Break at [0x10:0000000000008000] -- 09:39:52.929608
2024-06-13
```

```
[GNR1_C143_T1] Resuming -- 09:40:04.090232 2024-06-13
```

```
Exit SMM Mode...
```

```
Exit SMM completed
```

```
=====
|STAGE          |CHECKPOINT                               |STATUS|
=====
|Setup          |Halt Validation                         |Pass  |
|Setup          |Advanced RAS Validation                 |Pass  |
|Setup          |Access Validation                       |Pass  |
|Setup          |Enable AMEI                            |Pass  |
|Setup          |Error Signal Validation                 |Pass  |
|Verification   |Error Injection Verification            |Pass  |
|Verification   |Lock MCA/CMCI generation                |Pass  |
=====
```

```
{'results': [{'STAGE': 'Setup',
'CHECKPOINT': 'Halt Validation',
'STATUS': 'Pass'},
{'STAGE': 'Setup',
'CHECKPOINT': 'Advanced RAS Validation',
'STATUS': 'Pass'},
{'STAGE': 'Setup',
'CHECKPOINT': 'Access Validation',
'STATUS': 'Pass'},
{'STAGE': 'Setup',
'CHECKPOINT': 'Enable AMEI',
'STATUS': 'Pass'},
{'STAGE': 'Setup',
'CHECKPOINT': 'Error Signal Validation',
'STATUS': 'Pass'},
{'STAGE': 'Verification',
'CHECKPOINT': 'Error Injection Verification',
'STATUS': 'Pass'},
{'STAGE': 'Verification',
'CHECKPOINT': 'Lock MCA/CMCI generation',
'STATUS': 'Pass'}]}
```

2. Run the Crashdump either using Cscripts or ACD tool and extract the crashdump summary. The logs should reflect the error logs collected on CScripts. Refer to [CrashDump](#) section for more information on how to run and extract Crashdump. Parse the Crash Dump summary file. Read through MCA ERROR SOURCE LOG to evaluate which CPU asserted MCERR first. UPI MC Bank 5 should populate.

```
>>> error.crashdump_summary()
HW.MCE.UPI:Unmerged_UPI0:Uc_Ll_Rx_Detected_Crc_Error.incomplete_dump.mca_zero_value

FaultCPU= GNR
FaultTimeStamp= 2024-06-13 16:39:54
FaultPPIN= 0x0000000000000000
FaultCPUID= 0x0
FaultMicrocode= 0x00000000
FaultReason= HW.MCE.UPI:Unmerged_UPI0:Uc_Ll_Rx_Detected_Crc_Error.incomplete_dump.mca_zero_value
FaultDevice= N/A
FaultInstance= N/A
FaultCode= CPU0.UPI0.MSCOD.0x10
FaultDescription= Possible Voltage Issue, Board Socket Contamination or Part Related Issue. Verify system have latest microcode; it could be contamination on pins or
wrong Torque Setting/Installation. Remove CPU clean socket, apply correct torque.
FaultAction= CPU0; Monitor Failure Rate; Clean CPUx; Remove and inspect CPUx; Inspect Motherboard pins. Contact your Intel representative on repeat error on same CPUx.
FaultActionCode= hw.check_margins

##### Signature by Sockets #####
|socket|Signature|
-----|-----|
|0|HW.MCE.UPI:Unmerged_UPI0:Uc_Ll_Rx_Detected_Crc_Error.incomplete_dump.mca_zero_value|
|1|INCOMPLETE_DUMP.MCA_ZERO_VALUE|

##### Status[63]=1 Machine Check summary #####
|s|t|d|e|c|o|t|t|h|r|b|a|n|k|s|t|a|t|u|s|a|d|d|r|e|s|s|m|i|s|c|c|t|l|e|r|r|o|r|
-----|-----|-----|-----|-----|-----|
|0||i|o|0|-|-|UPI0|0xba00014000100000|0x0000000000000000|0x20c000800000ffbe1|0x000000000081ff0007|Unmerged_UPI0:Uc_Ll_Rx_detected_crc_error|

##### Internal and UBOX Errors per socket #####
First Error Assertion SUMMARY by socket
Legend: 1 means yes
|CPU socket|socket0 io0|socket1 io0|
-----|-----|-----|
```

```
-----
|Die|      |io0|      |io0|      |
|First ierr [16:0]|----|----|
|First mcerr [16:0]|----|----|
|Asserted Caterr#| 1| 10|
|Asserted ierr| 10| 10|
|Asserted mcerr| 11| 10|
|Asserted MSMI#| 10| 10|
|Asserted MSMI_ierr| 10| 10|
|Asserted MSMI_mcerr| 10| 10|
|SideBandTrkTO| 10| 10|
|SideBank_Err| 10| 10|
|SMITimeOut| 10| 10|
|FPIN| 10x0000000000000000|10x0000000000000000|
=====
##### IERR and MCEERR Logs #####
***cpu0***
=====
|Die|Register|Value|Description|
-----
|io0|pcu_cr_mca_err_src_log|0x14000000|CATERR# Asserted
| | | |MCERR# Asserted
|io0|ierrloggingreg|0x00000000|
|io0|mcerrloggingreg|0x00034800|Bits [15:0] First MCERR Src Valid --> Core0 compute0 <--
| | | |Bits [17] FirstMcerrSrcID is from a CORE
|io0|uboxerrstatus|0x00000000|
|io0|rmcerrloggingreg|0x00000000|
=====
|Die|Register|Value|
-----
|io0|pcu_first_ierr_tsc_lo_cfg|0x00000000|
|io0|pcu_first_ierr_tsc_hi_cfg|0x00000000|
|io0|pcu_first_mcerr_tsc_lo_cfg|0xcd7b1cf8|
|io0|pcu_first_mcerr_tsc_hi_cfg|0x0000000e|
|io0|pcu_first_rmca_tsc_lo_cfg|0x00000000|
|io0|pcu_first_rmca_tsc_hi_cfg|0x00000000|
=====
>>> go()
```

3. AC Cycle the platform.

9.6.4 Spoofed AMEI LLC

Test Objective

To facilitate implementation and validation of error handling flows in the event of a system hang, a spoofed LLC Uncorrectable data error injection function, is available as part of the CScripts package. This test verifies LLC UC data error injection and reporting.

Configuration

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW20 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+ on x86_64
HW Configuration	N/A
CScripts Version	GNR 2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [CrashDump](#) on page 19

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

Recipe

1. Boot to the OS and inject the spoofed LLC UC Data error.

```
>>> halt()
[GPC core group] Halt on 288 devices -- 08:28:27.987402 2024-06-18
>>> ei.sys.AMEI(verbose="True", validation="True")

*****
GMR AMEI Tool
*****
Select Input Mode [1,2]
Enter Q or q to exit

1: Error Event Selection mode
2: Raw Data Input mode

>>> 1
0:IFU Parity Error (MCA)
1:IFU Trusted Paths Error (MCA)
2:DCU data load poison (MCA)
3:MLC 3 Strike Timeout (MCA)
4:CHA TOR Timeout (MCA)
5:UPI UC Rx detected CRC error (MCA)
6:LLC Uncorrectable data error (MCA)

*****
GMR AMEI Tool (Error Event Input Mode)
*****
Enter Error Event Number
Enter Q or q to exit

>>> 6
>>> 0
Bank9, (LCC)
Initiating an SMI
[GPC core group] Resuming on 288 devices -- 08:29:01.619564 2024-06-18
Enter SMM completed
Section doesn't exist: Register definition
Section doesn't exist: Register definition
Section doesn't exist: Register definition
Trigger MCA Error signal...
Clear AMEI control registers after the error injection ...
[GNR0_C0_T0] SMM entry Break at [0x10:0000000000008000] -- 08:29:35.843626
2024-06-18
[GNR1_C143_T1] Resuming -- 08:29:46.802871 2024-06-18
Exit SMM Mode...
Exit SMM completed
Procedure completed successfully, use error.check_machine_check_errors() or
crashdump() to verify injection
=====
|STAGE|CHECKPOINT|STATUS|
=====
|Setup|Halt Validation|Pass|
|Setup|Advanced RAS Validation|Pass|
|Setup|Access Validation|Pass|
|Setup|Enable AMEI|Pass|
|Setup|Error Signal Validation|Pass|
|Verification|Error Injection Verification|Pass|
|Verification|Lock MCA/CMCI generation|Pass|
=====
{'results': [{'STAGE': 'Setup',
'CHECKPOINT': 'Halt Validation',
```

```
'STATUS': 'Pass'},
{'STAGE': 'Setup',
'CHECKPOINT': 'Advanced RAS Validation',
'STATUS': 'Pass'},
{'STAGE': 'Setup',
'CHECKPOINT': 'Access Validation',
'STATUS': 'Pass'},
{'STAGE': 'Setup',
'CHECKPOINT': 'Enable AMEI',
'STATUS': 'Pass'},
{'STAGE': 'Setup',
'CHECKPOINT': 'Error Signal Validation',
'STATUS': 'Pass'},
{'STAGE': 'Verification',
'CHECKPOINT': 'Error Injection Verification',
'STATUS': 'Pass'},
{'STAGE': 'Verification',
'CHECKPOINT': 'Lock MCA/CMCI generation',
'STATUS': 'Pass']}]}
```

2. Run the Crashdump either using Cscripts or ACD tool and extract the crashdump summary. The logs should reflect the error logs collected on CScripts. Refer to [CrashDump](#) section for more information on how to run and extract Crashdump. Parse the Crash Dump summary file. Read through MCA ERROR SOURCE LOG to evaluate which CPU asserted MCERR first. LLC MC Bank 9 should populate.

```
>>> error.crashdump_summary()
HW.MCE.LLC:Uncorrectable_Data_Error.incomplete_dump.mca_zero_value
FaultCPU= GNR
FaultTimeStamp= 2024-06-18 09:49:04
FaultPPIN= 0x0000000000000000
FaultCPUID= 0x0
FaultMicrocode= 0x00000000
FaultReason= HW.MCE.LLC:Uncorrectable_Data_Error.incomplete_dump.mca_zero_value
FaultDevice= W/A
FaultInstances= W/A
FaultCode= CPU0.LLC0.MSCOD.0x1
FaultDescription= Uncorrectable Timeout error
FaultAction= CPU0: Internal Error. Contact your Intel representative.
FaultActionCode= hw.contact_intel
##### Signature by Sockets #####
=====
|socket|Signature|
-----
|0|HW.MCE.LLC:Uncorrectable_Data_Error.incomplete_dump.mca_zero_value|
|1|INCOMPLETE_DUMP.MCA_ZERO_VALUE|
=====
##### Status(63)= Machine Check summary #####
=====
|skt|die|cor|thr|bank|Status|Address|Misc|CTL|Error|
-----
|0|soc|-|1|LLC0|0xf200000000010000|0x0000000000000000|0x0000000000000000|0x000000000000007f|Unmerged_LLC0:UNCORRECTABLE_DATA_ERROR|
=====
##### Internal and UBEX Errors per socket #####
First Error Assertion SUMMARY by socket
Legend: 1 means yes
=====
|CPU socket|socket0 io0|socket1 io0|
-----
|Die|Register|Value|Description|
-----
|io0|pcu_cr_mca_err_src_log|0x14000000|CATERR# Asserted|
|io0|ierrloggingreg|0x00000000|MCERR# Asserted|
|io0|mcerrloggingreg|0x00034800|Bits [15:0] First MCERR Src Valid --> Core0 compute0 <--|
|io0|uboxerrstatus|0x00000000|Bits [17] FirstMcerrSrcID is from a CORE|
|io0|rmcerrloggingreg|0x00000000||
=====
|Die|Register|Value|
-----
|io0|pcu_first_ierr_tsc_lo_cfg|0x00000000|
|io0|pcu_first_ierr_tsc_hi_cfg|0x00000000|
|io0|pcu_first_mcerr_tsc_lo_cfg|0x47c31294|
|io0|pcu_first_mcerr_tsc_hi_cfg|0x00000009|
|io0|pcu_first_tmca_tsc_lo_cfg|0x00000000|
|io0|pcu_first_tmca_tsc_hi_cfg|0x00000000|
=====
>>> go()
```

3. AC Cycle the platform.

10.0 Memory RAS

10.1 Single Device Data Correction (SDDC)

Feature Description

The processor's IMC module implements error check and correction technique. Processor also implements a "read retry" algorithm to recover from a failure (for example, transient errors). This entire flow of error detection, correction, is defined as Single Device Data Correction (SDDC). SDDC feature allows managing DRAM failures where the whole DRAM device is failed.

Permanent Fault Detection (PFD) is a feature that enhances the IMC's error detection and correction capabilities. PFD identifies permanent faults and in the case of ECC uncorrectable single-device permanent faults PFD may be able to correct the error.

The following figure is useful for understanding the scope or coverage of ECC. For 10x4 DRAM device based DIMMs ECC covers an entire device and for 5x8 DIMMs it is the left or right half of the device. If multiple bits have been flipped in a cacheline and all these bits fall within the boundary just mentioned, **nearly** 100% of such error patterns are correctable. If the multiple flipped bits were detected to occur across different devices, or on the left and right half of a device for 5x8 DIMMs, then the ECC error is uncorrectable.

A retry read error log's parity register indicates which of the 64 bits from a device (10x4) or left/right half of a device (5x8) were detected to have flipped.

Figure 7. Cacheline of Data from 10x4 and 5x8 DIMMs

Cacheline, Device, and Burst Bit Mapping for 10x4 and 5x8 DDR5 DIMMs															
	x8	4		3		2		1		0					
	x4	9	8	7	6	5	4	3	2	1	0				
Burst	0														
	1														
	2														
	3														
	4														
	5														
	6														
	7														
	8														
	9														
	10														
	11														
	12														
	13														
	14														
	15														

10.1.1 Single Device Multi-bit Correctable ECC Error

Test Objective

The objective of this test is to test the IMC's ability to detect and correct multi-bit single device ECC errors and explore key memory error logs.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45'23 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	No particular DIMM, PCIe card, or hardware tool configuration required
CScripts Version	BHS.2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#)
- [FTrace and DMESE for Machine Check Exception \(MCE\) Observability](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is highly recommended to have a POR memory population.

Recipe

1. Halt and confirm there are no preexisting memory related errors on the system.

```
>>> halt()
>>> error.check_mem_errors()
*****
                        No errors found
*****
```

2. Check the current ECC mode as the results of injection can differ based on ECC mode, injection target, and error mask used. SDDC coverage differs depending on x4 or x8 DRAM based DIMMs so be aware of the DRAM topology of the installed DIMMs. Also, some enabled features impact the best possible ECC mode that the

memory can operate in. For this test, directory mode was disabled and any other feature that may require metadata bits is not enabled. Therefore, the DDR memory is operating in 128b ECC mode.

```
>>> ras.mem.show ecc_mode()
Discovering Populated Channels
Discovering Channels
Ecc Modes
```

```
=====
| Channel | SDDC |
|-----|
| socket0 Mc0 Ch0 | 128b |
| socket0 Mc1 Ch0 | 128b |
| socket0 Mc2 Ch0 | 128b |
| socket0 Mc3 Ch0 | 128b |
=====
```

```
Ecc Modes
=====
| Channel | SDDC |
|-----|
| socket1 Mc0 Ch0 | 128b |
| socket1 Mc1 Ch0 | 128b |
| socket1 Mc2 Ch0 | 128b |
| socket1 Mc3 Ch0 | 128b |
=====
```

3. Check if the injectors are locked for all sockets and unlock if locked.

```
>>> ei.utils.checkInjectorLock(and_unlock=True)
```

4. Choose the target device for error injection and modify masks for a single device correctable ECC error injection. A very high percentage of single device masks are correctable so there are many acceptable masks for this test. For this test, all the bits from DQ1 of device 1 are flipped.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x2222222222222222, dev1=2, dev1msk=0x0)
```

5. Inject and consume the correctable ECC error.

```
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0)
```

6. Check that the target's IMC machine check bank has an error logged. The error should be a correctable demand read error (uc=0, mscod=0x80).

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc_status_shadow
0x8c00004200800090

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc8_addr_shadow
0x10008200

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc_misc_shadow
0x400080008008086

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc_status_shadow.show()
0x00000001 : valid (63:63)
...
0x00000001 : miscv (59:59)
0x00000001 : addrv (58:58)
...
0x00000001 : cor_err_cnt (52:38)
...
0x00000080 : mscod (31:16)
0x00000090 : mcacod (15:00)
```

7. The error was detected by a demand read, so check one of the sets of retry read error logs that was configured to log extra error information on demand read retries. There are four sets of retry read errors logs and the en_patspr bit-field indicates whether the set of logs will log patrol/sparing errors. Based on this output, sets one and three will log demand/ufill read errors.

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log.en_patspr
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[0].en_patspr - 0x00000001
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[1].en_patspr - 0x00000000
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[2].en_patspr - 0x00000001
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[3].en_patspr - 0x00000000
```

8. Read each of the registers in set one of the retry read error logs. This is for reference if needed. The next steps decode and points out key bit-fields.

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[1]
0xb009409

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_address1[1]
0x44000001

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_address2[1]
0xf0000100

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_address3[1]
0x8004100

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_misc[1]
0x0

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_parity[1]
0x2222222222222222
```

9. retry_rd_err_log. The ECC mode is 128b so PFD results for any single device errors are not interesting or useful because there is 100% detection and correction of single device errors. The most interesting bit-fields that should match are shown next. For this register and all subsequent register reads, read the descriptions for each bit-field (not shown) or the processor's register specification for more details.

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[1].show()
...
0x00000002 : corr_fsm_errtype (27:26)
...
0x00000001 : system_addr_valid (24:24)
...
0x00000001 : corr_correctable (10:10)
...
0x00000000 : uc (01:01)
0x00000001 : v (00:00)
```

10. retry_rd_err_log_address1/2/3. These registers log the error location and address. All details should match with the error injection location specified and the IMC MC bank logs.

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_address1[1].show()
0x00000000 : channel_addr_valid (31:31)
0x00000001 : failed_dev_valid (30:30)
0x00000040 : col (29:20)
0x00000000 : ba (19:18)
0x00000000 : bg (17:14)
0x00000000 : cbit (13:09)
0x00000000 : chip_select (08:06)
0x00000001 : failed_dev (05:00)

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_address2[1].show()
...
```

```
0x00000100 : row (17:00)
```

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_address3[1].show()
0x0000010008200 : sysaddress (51:00)
```

11. retry_rd_err_log_parity. Matches the mask specified in memDevs.

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log_parity[1].show()
0x2222222222222222 : par_syn (63:00) -- This register holds the parity syndrome(mask) from
correction path for the current ECC mode.
```

12. Based on the logical rank that was injected to, check that the rank's corrected ECC error counter incremented by 1. For DDR5, the logical rank/chip-select is encoded as: {DIMM, rank. sub-channel}. There is a counter and threshold register for each of the eight possible ranks on the channel.

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.correrrcnt
correrrcnt[0]=0x1
correrrcnt[1]=0x0
correrrcnt[2]=0x0
correrrcnt[3]=0x0
correrrcnt[4]=0x0
correrrcnt[5]=0x0
correrrcnt[6]=0x0
correrrcnt[7]=0x0

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.correrrthrshld
correrrthrshld[0]=0x7fff
correrrthrshld[1]=0x7fff
correrrthrshld[2]=0x7fff
correrrthrshld[3]=0x7fff
correrrthrshld[4]=0x7fff
correrrthrshld[5]=0x7fff
correrrthrshld[6]=0x7fff
correrrthrshld[7]=0x7fff
```

13. Resume the system to allow all error handling flows to complete.

```
>>> go()
```

14. BIOS serial log:

```
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0xB009409
0/13-1 RetryRdErrLogAddr1: 0x44000001
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
TranslateSad: McTargetId is 0x0
Entering CheckNearMemoryError.
```

```

S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh
is 0x0Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AF0B010
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
LogErrorToElog: --Logging Error to EMCA
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T0 [CpuRas]Successfully notify OS via CMCI
S0 C81 T0 [CpuRas]Successfully notify OS via CMCI
...
S0 C75 T0 [CpuRas]Successfully notify OS via CMCI
S0 C23 T1 [CpuRas]Successfully notify OS via CMCI

```

15. ftrace output:

```

<...>-8      [000] ..... 616.289085: mce_record: CPU: 0, MCGc/s: f000c20/0,
MC13: 8c00004200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000010008200/0400080008008086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
12783ee7d0f, PROCESSOR: 0:a06d0, TIME: 1679270783, SOCKET: 0, APIC: 0

kworker/0:0-8      [000] .N... 616.305142: extlog_mem_event: {1} corrected error:
single-bit ECC physical addr: 0000000010008200 (mask lsb: ff) node:0 card:0 module:0 rank:0
bank:0 device:1 row:256 column:256 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0

kworker/0:0-8      [000] ..... 616.312659: mc_event: 1 Corrected error: memory read
error on CPU_SrcID#0_MC#0_Chan#0_DIMM#0 (mc:0 location:0:0:-1 address:0x10008200 grain:32
syndrome:0x00000000_err_code:0x0080:0x0090 SystemAddress:0x10008200 ProcessorSocketId:0x0
MemoryControllerId:0x0 ChannelAddress:0x4002000 ChannelId:0x0 RankAddress:0x1000800
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x100 Column:0x100 Bank:0x0 BankGroup:0x0
ChipSelect:0x0)

```

10.1.2 Permanent Fault Detection - 125 Bit ECC Mode

Test Objective

This section demonstrates a single DRAM device error that PFD will assist ECC to correct. The final ECC result logged in the `retry_rd_err_log` and machine check bank will demonstrate PFD success in assisting ECC. It also needs to be noted, PFD can assist ECC only on permanent faults. If the underlying DRAM device is healthy, then only continuous error injections can create the permanent fault condition for PFD to detect.

The error injection command for PFD test will inject the error continuously. As a result, the injector will also inject during the final scrubbed data to DRAM (good data to DRAM) making an Uncorrectable error. Therefore, any subsequent reads from the injected location will encounter an Uncorrectable error that PFD will not be able to correct unless the underlying DRAM is bad.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A1
BIOS	WW23 BKC
OS Version	CentOS* Stream 8
Kernel	5.19.0-gnr.p0.bkc.3.13.14.x86_64
HW Configuration	1 DIMM per socket on channel 0 for both sockets
CScripts Version	GNR2322.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - PFD = Enable
- EDKII > Socket Configuration > Uncore Configuration > Uncore General Configuration >
 - Directory Mode Enable = Enable

Recipe

1. Halt the system and verify that the platform is operating in 125 bit ECC mode:

```
>>> halt()
>>> ras.mem.show_ecc_modes()
Discovering Populated Channels
Discovering Channels
Ecc Modes
=====
| Channel | SDDC |
|-----|
| socket0 Mc0 Ch0 | 125b |
|-----|
Ecc Modes
=====
| Channel | SDDC |
|-----|
| socket1 Mc0 Ch0 | 125b |
|-----|
```

2. Set the device mask for 125-bit ECC mode.

```
>>> halt()
>>> ei.utils.resetInjectorLockCheck()
>>> ei.mem.memDevs(dev0msk=0x0000_7013_0000_0000, dev1msk=0x0)
dev0 = 0x1, dev0msk = 0x701300000000, dev1 = 2, dev1msk = 0x0. Customized injectors may not
work correctly in RAS modes
```

3. Inject a continuous uncorrectable error and immediately consume it.

```
>>> ei.mem.injectMemError(0x8000, immConsume=True, oneInjection=False, errType="uce")
```

4. Observe that the error was corrected and logged in the `retry_rd_err_logs` and iMC Status register. Observe the corrected error count was incremented.

```
>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.imc0_mc_status_shadow.show()
0x00000001 : valid (63:63) --
0x00000000 : over (62:62) --
0x00000000 : uc (61:61) --
0x00000000 : en (60:60) --
0x00000001 : miscv (59:59) --
0x00000001 : addrv (58:58) --
0x00000000 : pcc (57:57) --
0x00000000 : s (56:56) --
0x00000000 : ar (55:55) --
0x00000000 : cor_err_stat (54:53) --
0x00000001 : cor_err_cnt (52:38) --
0x00000000 : fw_upd (37:37) --
0x00000002 : other_info (36:32) --
0x00000080 : mscod (31:16) --
0x00000090 : mcacod (15:00) --

>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.retry_rd_err_log[1].show()
0x00000000 : lock (31:31) --
0x00000000 : ucna (30:30) --
0x00000002 : ecc_bist_errtype (29:28) --
0x00000001 : corr_fsm_errtype (27:26) --
0x00000001 : ecc_bist_valid (25:25) --
0x00000001 : system_addr_valid (24:24) --
0x00000001 : persistent_error (23:23) --
0x00000000 : transfer_number (22:19) --
0x00000000 : single_bit_error (18:18) --
0x00000000 : col_correction (17:17) --
0x00000001 : bist_corr (16:16) --
0x00000001 : noover (15:15) --
0x00000000 : en_patspr (14:14) --
0x00000000 : patspr (13:13) --
0x00000001 : en (12:12) --
0x00000000 : corr_correctable_par_dev (11:11) --
0x00000001 : corr_correctable (10:10) --
0x00000000 : corr_noerr (09:09) --
0x00000000 : corr_err (08:08) --
0x00000000 : system_addr_err (07:07) --
0x00000002 : mode (06:03) --
0x00000000 : over (02:02) --
0x00000000 : uc (01:01) --
0x00000001 : v (00:00) --
```

5. Observe in serial log that the error was signaled as corrected.

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
RasEventHndlrEntry index: 0x2
RasEventHndlrEntry index: 0x3
RasEventHndlrEntry index: 0x4
RasEventHndlrEntry index: 0x5
[PERF] start
[PERF] end
S0 C8 T1 [GetChannelErrorInfo]:raw data EccMode is 0x2. Uncorrect is 0x0 Rank:0x0 SubRank:0x0
Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x0 Row:0x4 SingleBit:0x0 Transfer:0x0
PersistentError:0x1 InterLocSyn:0x0
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C8 T1 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C8 T1 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
```

```

S0 C8 T1 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C8 T1 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C8 T1 [CpuRas]MC status 0x8C00004200800090, class CORRECTED

Forward Address Translation start for 0x8000
[GetMcAndChId] SingeWay is set, McId:0 ChId:0
McTargetId is 0x0
MemType is MemType11mDdr
0x8000 belongs to DDR5
TranslatedAddress->SocketId is 0x0
TranslatedAddress->MemoryControllerId is 0x0
TranslatedAddress->ChannelId is 0x0
[Forward11mDecodeGetCa] TAD skt:0 Mc:0 Chn:0 TadIndex:0
Sub Channel address:4000
Pseudo Channel address:2000
rank :0
rank address:2000
[ForwardAmap] DDR5 X4 FGBG_INT2
RankAddress:0x2000 ChipSelect:0x0 PhysicalRankId:0x0 DimmSlot:0x0
Col:0x0 Row:0x4 BankGroup:0x0 Bank:0x0

Forward Address Translation Successful!!
Entering CheckNearMemoryError.
S0 C8 T1 [GetChannelErrorInfo]:raw data EccMode is 0x2. Uncorrect is 0x0 Rank:0x0 SubRank:0x0
Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x0 Row:0x4 SingleBit:0x0 Transfer:0x0
PersistentError:0x1 InterLocSyn:0x0
S0 C8 T1 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AAF6810
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x11 Bank 0xD, Socket 0x0 Core 0x8
S0 C1 T0 [CpuRas]Successfully notify OS via CMCI
S0 C1 T1 [CpuRas]Successfully notify OS via CMCI
S0 C9 T1 [CpuRas]Successfully notify OS via CMCI
S0 C9 T0 [CpuRas]Successfully notify OS via CMCI
S0 C6 T0 [CpuRas]Successfully notify OS via CMCI
S0 C4 T0 [CpuRas]Successfully notify OS via CMCI
S0 C4 T1 [CpuRas]Successfully notify OS via CMCI
S0 C0 T1 [CpuRas]Successfully notify OS via CMCI
S0 C2 T0 [CpuRas]Successfully notify OS via CMCI
S0 C2 T1 [CpuRas]Successfully notify OS via CMCI
S0 C10 T1 [CpuRas]Successfully notify OS via CMCI
S0 C6 T1 [CpuRas]Successfully notify OS via CMCI
S0 C7 T1 [CpuRas]Successfully notify OS via CMCI
S0 C11 T0 [CpuRas]Successfully notify OS via CMCI
S0 C11 T1 [CpuRas]Successfully notify OS via CMCI
S0 C0 T0 [CpuRas]Successfully notify OS via CMCI
S0 C7 T0 [CpuRas]Successfully notify OS via CMCI
S0 C3 T0 [CpuRas]Successfully notify OS via CMCI
S0 C5 T1 [CpuRas]Successfully notify OS via CMCI
S0 C8 T1 [CpuRas]Successfully notify OS via CMCI
S0 C5 T0 [CpuRas]Successfully notify OS via CMCI
S0 C10 T0 [CpuRas]Successfully notify OS via CMCI
S0 C3 T1 [CpuRas]Successfully notify OS via CMCI
S0 C8 T0 [CpuRas]Successfully notify OS via CMCI

```

6. Observe the FTrace logs.

```

cd /sys/kernel/debug/tracing
echo 1 > events/mce/mce_record/enable
echo 1 > events/ras/enable
cat trace_pipe
<...>-307 [000] ..... 849.436476: mce_record: CPU: 0, MCGc/s: f000c20/0,
MC13: 8c00004200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000008000/0800084000010086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
1a9b64e83fb, PROCESSOR: 0:a06d0, TIME: 1688582121, SOCKET: 0, APIC: 0
kworker/0:2-307 [000] ..... 849.437239: mc_event: 1 Corrected error: memory read

```



```
error on CPU_SrcID#0_MC#0_Chan#0_DIMM#0 (mc:0 location:0:0:-1 address:0x00008000 grain:32
syndrome:0x00000000_err_code:0x0080:0x0090 SystemAddress:0x8000 ProcessorSocketId:0x0
MemoryControllerId:0x0 ChannelAddress:0x4000 ChannelId:0x0 RankAddress:0x1000
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x0 Column:0x200 Bank:0x2 BankGroup:0x0
ChipSelect:0x0)
```

10.2 Adaptive Double Device Data Correction (ADDDC)

Feature Description

Adaptive Double Device Data Correction (ADDDC) is a runtime DRAM sparing feature which is implemented via adaptive Virtual Lockstep (VLS).

ADDDC invocation results in a map out of a failing DRAM device during runtime. It is the platform (BIOS/BMC) that monitors the health of DRAM devices through a pattern of errors that the memory controller detects and corrects. Once the BIOS is convinced of the DRAM device failing, or predicts a given failure pattern can lead to uncorrectable errors, then the BIOS configures the hardware to perform a device map out through the algorithm called VLS. With VLS, the hardware re-arranges cachelines stored in the DRAM device stored within a rank to ideally across two logical ranks; a single rank DIMM on the channel can have one or two VLS regions established but the VLS bandwidth impact is worse compared to having two or four ranks on the channel. Such data rearrangement allows the hardware to move data from a failing device to a spare device and continues to correct errors from a single DRAM device. ADDDC requires the DIMMs to be built on x4 DRAM devices.

The following test cases use error injectors at the iMC to corrupt the data being delivered to the DDR5 DIMMs. When DDR5 on-die ECC is enabled, DRAM device further protects the incoming data with its own ECC code before saving it on to the DRAM array. Also, on the read path, DDR5's on-die ECC can detect the error in data protected with its own generated ECC code. iMC has no means of corrupting the raw bits being saved on the DRAM by DDR5 device; therefore, it cannot trigger error cases for DDR5's on-die ECC to correct, all error detections and corrections will occur at the iMC.

After hardware and/or software thresholding has determined a bank qualifies for RAS action there are various flows the BIOS can take regarding setting up VLS regions which depend on: DIMM population and the number and type of VLS regions existing at that point in time.

Intel BKC BIOS ADDDC Actions

NOTE

Any reference to the BIOS in this section refers to Intel's current BKC BIOS. The BIOS actions are implementation specific whenever they are discussed. This section describes the BIOS behavior as it is convenient, but it should not be considered thorough or used as a BIOS writer's guide.

The following table shows the current BKC BIOS implementation of the buddy rank selection order list. Here is a simple example to help read this table: two dual rank DDR5 DIMMs on the channel, no VLS regions on the channel initially, then a bank is qualified for ADDDC, this bank is on chip-select/rank 3, BIOS will choose rank 5 as the

buddy rank. If rank 5 is not available or a valid choice then rank 7 will be selected. If none of the other ranks on the sub-channel are valid to be the buddy rank, then the BIOS will buddy the failed rank with itself.

The BIOS will choose the buddy bank by flipping bit 0 of the failed bank.

Figure 8. DDR5 - Intel BKC BIOS VLS Buddy Rank Selection Order

DDR Type	Ranks/ DIMM	DIMMs Per Channel	Failed Rank	Decoded Failed Rank (DIMM, Rank, Sub-ch)	Buddy Rank Selection Order
RDIMM	1	1	0	(0, 0, 0)	0
			0	(0, 0, 0)	4, 0
		2	1	(0, 0, 1)	5, 1
			4	(1, 0, 0)	0, 4
			5	(1, 0, 1)	1, 5
	2	1	0	(0, 0, 0)	2, 0
			1	(0, 0, 1)	3, 1
			2	(0, 1, 0)	0, 2
			3	(0, 1, 1)	1, 3
		2	0	(0, 0, 0)	4, 6, 2, 0
			1	(0, 0, 1)	5, 7, 3, 1
			2	(0, 1, 0)	4, 6, 0, 2
			3	(0, 1, 1)	5, 7, 1, 3
			4	(1, 0, 0)	0, 2, 6, 4
			5	(1, 0, 1)	1, 3, 7, 5
			6	(1, 1, 0)	0, 2, 4, 6
			7	(1, 1, 1)	1, 3, 5, 7

Even if multiple systems have the same bank qualified for bank VLS, the current BKC BIOS implementation takes a variety of different actions depending on the number of ranks on the channel (as you can see on the previous table). After this bank, the VLS region is established, with ADDDC there is another VLS region on the channel. In the event, a second bank has been qualified for the VLS. The action BIOS will take depends on where the second bank is located and the number of ranks on the channel.

One Rank on the Channel

The following figure shows a high level reference of the cacheline data organization for a region in the VLS. Specifically, the case where there is one rank on the channel and bank 0 on device 0 was qualified to be mapped out.

Figure 9. One Rank on the Channel - One Bank VLS Cacheline Data Organization

Rank A		DID									
		9	8	7	6	5	4	3	2	1	0
Bank	0										
	1										
	2										
	3										
	4										
	5										
	6										
	7										
	8										
	9										
	...										

The following figure depicts the five possible 2nd strike locations for this configuration. "2nd strike" refers to a 2nd bank that has been qualified to be mapped out.

Figure 10. One Rank on the Channel - Possible 2nd Strike Locations

Rank A		DID									
		9	8	7	6	5	4	3	2	1	0
Bank	0	a									
	1	c									b
	2	e									d
	3										
	4										
	5										
	6										
	7										
	8										
	9										
	...										

Here is a list of some example test cases for this 1 rank on the channel configuration:

- One bank VLS region.
- Scenario 'a' - 1 Bank VLS then 2nd Strike to Region in VLS, Different Device Number, Same Bank
 - No ADDDC action expected from BIOS.
- Scenario 'b' - 1 Bank VLS then 2nd Strike to Region in VLS, Same Device Number, Different Bank
 - BIOS is expected to reverse the 1st bank VLS region and rebuddy with a different bank then create a 2nd bank VLS region for the 2nd strike.
- Scenario 'c' - 1 Bank VLS then 2nd Strike to Region in VLS, Different Device Number, Different Bank
 - BIOS is expected to reverse the 1st bank VLS region and rebuddy with a different bank then create a 2nd bank VLS region for the 2nd strike.
- Scenario 'd' - 1 Bank VLS then 2nd Strike to Region not in VLS, Same Device Number, Different Bank
 - No ADDDC action expected from BIOS.
- Scenario 'e' - 1 Bank VLS then 2nd Strike to Region not in VLS, Different Device Number, Different Bank
 - BIOS is expected to create a 2nd independent VLS region.

Two Ranks on the Channel

The following figure depicts a high level view of the cacheline data organization for a region in VLS. Specifically, the case where there are two ranks on the channel and bank 0 on device 0 of rank A was qualified to be mapped out.

Figure 11. Two Ranks on the Channel - One Bank VLS Cacheline Data Organization

Rank A		DID										Rank B		DID									
		9	8	7	6	5	4	3	2	1	0			9	8	7	6	5	4	3	2	1	0
Bank	0											Bank	0										
	1												1										
	2												2										
	3												3										
	4												4										
	5												5										
	6												6										
	7												7										
	8												8										
	...												...										

The following figure shows the five possible 2nd strike locations for this configuration. "2nd strike" refers to a 2nd bank that has been qualified to be mapped out.

Figure 12. Two Ranks on the Channel - Possible 2nd Strike Locations

Rank A		DID										Rank B		DID									
		9	8	7	6	5	4	3	2	1	0			9	8	7	6	5	4	3	2	1	0
Bank	0											Bank	0										
	1												1										
	2												2										
	3												3										
	4												4										
	5												5										
	6												6										
	7												7										
	8												8										
	...												...										

Here is a list of some example test cases for this 2 ranks on the channel configuration:

1. One bank VLS region.
2. Scenario 'a' - 1 Bank VLS then 2nd Strike to Failed Rank, Region in VLS, Same Bank, Different Device.
 - a. No ADDDC action expected from BIOS.
3. Scenario 'b' - 1 Bank VLS then 2nd Strike to Failed Rank, Region not in VLS, Same Device, Different Bank.
 - a. BIOS is expected to promote the bank VLS region to rank VLS.
4. Scenario 'c' - 1 Bank VLS then 2nd Strike to Failed Rank, Region not in VLS, Different Device, Different Bank.
 - a. BIOS is expected to create a 2nd independent bank VLS region.
5. Scenario 'd' - 1 Bank VLS then 2nd Strike to Buddy Rank, Region in VLS.
 - a. BIOS is expected to reverse the 1st bank VLS region and rebuddy with a different bank (rank if 4 ranks on the channel) then create a 2nd bank VLS region for the 2nd strike.
6. Scenario 'e' - 1 Bank VLS then 2nd Strike to Buddy Rank, Region not in VLS.
 - a. The BIOS is expected to create a 2nd independent bank VLS region.

Four Ranks on the Channel

See the "Two Ranks on the Channel" section. With this configuration, there will be differences in the selection of buddy ranks but the diagrams and scenario descriptions are the same.

There are some unique test cases that are only possible with four ranks on the channel. Namely, reverse rank VLS and rebuddy flows. While the previously mentioned scenarios "a" through "e" refer to scenarios that occur after 1 bank VLS region has been established. Next scenarios that will be discussed will be after 1 rank VLS region has been established. With two ranks on the channel and 1 rank VLS region, there are not any other ranks that BIOS could use to rebuddy the failed rank with in the event where the buddy rank was deemed to be unhealthy. With four ranks on the channel, there are two other ranks which can be used.

1. Scenario "f" - 1 Rank VLS then 2nd Strike to Failed Rank, Any Bank, Non-failed Device.
 - a. No ADDDC action expected from BIOS.
2. Scenario "g" - 1 Rank VLS then 2nd Strike to Buddy Rank, Any Device, Any Bank.
 - a. The BIOS is expected to reverse the rank VLS region, rebuddy with a different rank, and then create a bank VLS region for the 2nd strike.

Hardware and Software Thresholding

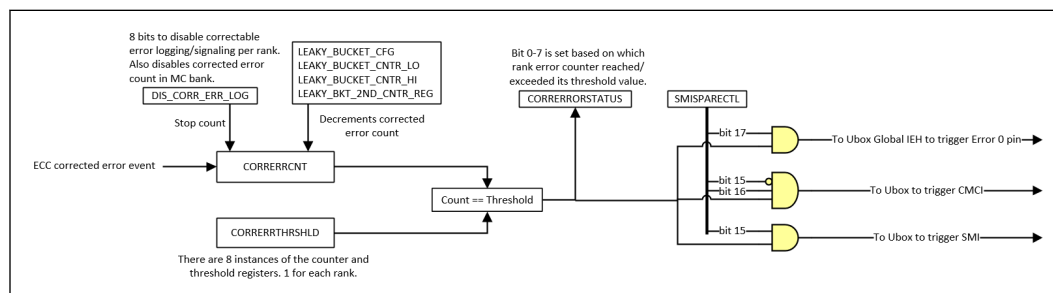
Hardware thresholding here refers to the per rank correctable ECC (CECC) hardware counters and thresholds, that is, CORRERRCNT and CORRERRTHRSHLD. This is NOT CSMI or CMCI thresholding. Reaching or exceeding this threshold with the current BIOS implementation will qualify a row for PPR or trigger VLS.

Software thresholding refers to the BIOS per bank and per row counters and thresholds for CECC errors. This is another thresholding layer on top of hardware thresholding if they are both enabled. Crossing the hardware threshold will cause the

BIOS to take that last error as a sample for its row and bank counters. Assuming all these features are enabled, when the row threshold is met then runtime sPPR action is taken or once the bank threshold is met then VLS action is taken.

The following figure shows a high level view of the hardware thresholding scheme.

Figure 13. Correctable ECC Thresholding



By default, SMISPAECTL is configured to generate an SMI interrupt.

10.2.1 One Rank on the Channel

This section covers the 6 test cases with one rank on the channel that were mentioned previously.

10.2.1.1 One Bank VLS Region

Test Objective

The objective of this test is to test one bank VLS region on a channel. Multiple correctable ECC errors are injected until the HW threshold is met to trigger ADDDC flow and various CSRs are read to confirm that BIOS and HW are behaving as expected.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45'23 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	4 DIMMs per socket, 10x4 single rank, 32 GB DIMMs on slots A1, B1, C1, and D1. Having 1 single rank DIMM on a channel is the main requirement for this specific test. A different number of ranks on the channel will result in a different buddy rank.
CScripts Version	BHS.2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is highly recommended to have a POR memory population.

Recipe

1. Halt and confirm there are no preexisting memory related errors on the system. Clear any found error logs or cold boot the system.

```
>>> halt()
>>> error.check_mem_errors()
*****
                                     No errors found
*****
```

- a. Considering multiple CECC error injections are performed while halted, the BIOS and OS actions/reporting that would occur if the test were done while the system was running do not occur. One way to modify this test is to inject errors while the system is running to allow the BIOS/OS actions for each CECC error to occur.

2. Check if the injectors are locked for all sockets and unlock if locked.

```
>>> ei.utils.checkInjectorLock(and_unlock=True)
```

3. Modify memDevs for a single device error injection. Aside from a few DUE patterns and depending on ECC mode, nearly 100% of single device non-permanent ECC errors are correctable. Therefore, dev0msk value can vary and the test will still be valid, but here 0x3 is used to flip bits 0 and 1 on burst 0. This step is optional because the default memDevs will work but this step ensures that any previous memDevs modifications that may have been done do not ruin the test.

```
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
dev0 = 0x0, dev0msk = 0x3, dev1 = 1, dev1msk = 0x0. Customized injectors may not work correctly in RAS modes
```

4. The hardware per rank CECC error thresholds were set to "2". So inject two CECC errors to the same location to reach the threshold value.


```
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

5. Use the following command to verify that the CORRERRCNT counter for that rank has reached the CORRERRTHRSOLD threshold value and that the CORRERRORSTATUS overflow bit for that rank is set. As shown next, the CECC count for rank 0 on MC 0 is 0x2 and that the overflow bit is set. Instead of using the adddc_status_check function, alternative options would be to read the registers mentioned directly or use RAS.mem.memCorrEccCount() so only this table is printed. The adddc_status_check is used because later in the test to verify that the VLS region has been established and it is useful to compare the output of the function from before and after VLS.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x2 | | 0x2 | Y |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
=====
```

6. Resume the system so BIOS can perform its SMM flow and start the VLS sparing operation.

```
>>> go()
```

7. **Optional:** You can read the following copy_in_progress bit to check if VLS sparing is in progress. The VLS sparing operation varies in time based on the configuration so this bit may not be seen to be "1" because sparing completed before the CSR was read.

```
>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.sparing_patrol_status.show()
0x00000000 : patrol_complete (02:02)
0x00000000 : copy_complete (01:01)
0x00000001 : copy_in_progress (00:00)
```

8. Verify that the ADDDC_REGION and ADDDC_REGION_CONTROL registers were modified by BIOS and indicate the failed and non-failed (buddy) ranks. The information printed for the failed bank and device shown next align with the previous error injection. Considering there is only one rank on the channel, the failed and non-failed rank are the same. BIOS selects the buddy bank by flipping bit 0 of the failed bank; in this case, the buddy bank is "1". Also, observe that the per rank CECC counter and overflow bit were cleared.

```
>>> ras.mem.adddc_status_check(0,0)
...
=====
| Register Dump |           | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x0 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | non-failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x1 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 0 | socket0 | mc0-ch0 | Failed | Device | = | 0x0 | Rank0 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 1 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 2 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 3 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
```

9. ftrace output:

```
<...>-2963 [000] ..... 601.246952: mce_record: CPU: 0, MCGc/s: f000c20/0,
MC13: 8c00004200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000010008200/0400000008008086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
121dd6126d3, PROCESSOR: 0:a06d0, TIME: 1679270984, SOCKET: 0, APIC: 0

<...>-2963 [000] ..... 601.253609: extlog_mem_event: {1} corrected error:
single-bit ECC physical addr: 0000000010008200 (mask lsb: ff) node:0 card:0 module:0 rank:0
bank:0 device:0 row:256 column:256 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0

kworker/0:3-2963 [000] ..... 601.373098: mc_event: 1 Corrected error: memory read
error on CPU SrcID#0_MC#0_Chann#0_DIMM#0 (mc:0 location:0:0:-1 address:0x10008200 grain:32
syndrome:0x00000000_err_code:0x0080:0x0090 SystemAddress:0x10008200 ProcessorSocketId:0x0
MemoryControllerId:0x0_ChannelAddress:0x4002000_ChannelId:0x0_RankAddress:0x1000800
PhysicalRankId:0x0_DimmSlotId:0x0_DimmRankId:0x0_Row:0x100_Column:0x100_Bank:0x0_BankGroup:0x0
LockStepRank:0x0_LockStepPhysicalRank:0x0_LockStepBank:0x1_LockStepBankGroup:0x0
ChipSelect:0x0)
```

10. BIOS serial log for the SMI to handle the memory error and activate VLS sparing:

```
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0xB009409
0/13-1 RetryRdErrLogAddr1: 0x44000000
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[i]mc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
```

```

McSsrState Node:0 :PatrolSPAMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!
S1 C6 T1 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh
is 0x0Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, Rank 0, SubRank 0, Bank 0, BankGrp 0, Device 0.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N N
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!

[ADDDC]NonRankVlsMode: Call HandleNonRankVlsStrike()...
[ADDDC]: HandleNonRankVlsStrike() start...
[ADDDC]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0, Bank 0, Device 0.
[ADDDC]: BankMode is 0.
[ADDDC]: No previous error in device 0.
[ADDDC]: Call HandleBankVlsEvent()...
[ADDDC]: HandleBankVlsEvent()... Start
[GetAvailableRegion]: NodeId 0, ChOnNode 0, Region 0, Status Success.
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0 Bank 0.
:No available pair rank for the failed rank 0 in channel 0, so select itself as buddy
rank for the bank vls event.
[GetNewNonFailedRankBank]: Rank 0 - Buddy Rank 0, Bank 0 - Buddy Bank 1.
AdddcEvent0:
  Type 3, Region 0 (PRank 0 : PBank 0) - (BRank 0 : BBank 1), FailedDevice 0.
AdddcEvent1:
  Type 3, Region 0 (PRank 0 : PBank 0) - (BRank 0 : BBank 1), FailedDevice 0.
[ADDDC]: HandleBankVlsEvent()... End
[ADDDC]: HandleNonRankVlsStrike() Status Success.
[ADDDC]: The event has been successfully handled!
[ADDDC]: Call SchedulePendingList()
DisableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:80
[SSR] disable C6 & GV
[SSR] stop and save patrol scrub info
[ADDDC]: ShedulePendingList()... Start
[ActionOnBankVls] Region 0, NodeId 0, Ch 0, FailedRank 0 - NonFailedRank 0, FailedBank 0 -
NonFailedBank 1.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AF0B610
WHEA: Detected Ras Non Standard Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
[ADDDC]: Event type 3, Action Status Success.

```

```
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleNewAdddcEvent() return, Status = Success.
[ADDDC]: Main handler return, Status Success.
[GetRRLErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0
Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0
UpdateMcChanSsrState ch:1:PatrolSPAMode
McSsrState Node:1 :PatrolSPAMode
UpdateMcChanSsrState ch:2:PatrolSPAMode
McSsrState Node:2 :PatrolSPAMode
UpdateMcChanSsrState ch:3:PatrolSPAMode
McSsrState Node:3 :PatrolSPAMode
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
TranslateSad: McTargetId is 0x0
Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh
is 0x0Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
Ssr copy inprogress
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AC99810
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
LogErrorToElog: --Logging Error to EMCA
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T1 [CpuRas]Successfully notify OS via CMCI
S0 C22 T0 [CpuRas]Successfully notify OS via CMCI
...
S0 C86 T0 [CpuRas]Successfully notify OS via CMCI
S0 C83 T0 [CpuRas]Successfully notify OS via CMCI
```

11. BIOS serial log for the SMI at VLS sparing completion:

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AF0B610
WHEA: Detected Ras Non Standard Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
[ADDDC]: ShedulePendingList()... Start
[CleanPreVlsEvent]: NodeId 0.
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:81
[SSR] restore and restart patrol scrub info
[SSR] enable C6
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: P B N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
```

```

26 27 28 29 30 31
-   Bank Mode:  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N  N
N  N  N  N  N  N  N
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!

[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
UpdateMcChanSsrState ch:1:PatrolSPAMode
McSsrState Node:1 :PatrolSPAMode
UpdateMcChanSsrState ch:2:PatrolSPAMode
McSsrState Node:2 :PatrolSPAMode
UpdateMcChanSsrState ch:3:PatrolSPAMode
McSsrState Node:3 :PatrolSPAMode
[imc] memory error handler end!

```

10.2.1.2 Scenario 'a' - 1 Bank VLS then 2nd Strike to Region in VLS, Different Device Number, Same Bank

Test Objective

The objective of this test is to test scenario "a" as described in [this section](#). The scenario is as follows: bank is qualified for RAS action, bank VLS region is established, second bank is qualified for RAS action. The second bank location is in a region in VLS, different device as the failed device, and the same bank number as the failed bank. Expected outcome is no action taken.

Configuration

The following table details the system configuration used to develop this recipe. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW07'24 BKC
OS Version	n/a - test was done at UEFI shell
Kernel	n/a - test was done at UEFI shell
HW Configuration	1 DIMM per socket, 10x4 single rank, 32 GB DIMM
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable

Recipe

1. Establish 1 bank VLS region as described in [this section](#).
2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2". Therefore, inject two CECC errors to a different device from before.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

4. Use the following command to verify that the CORRERRCNT counter for that rank has reached the CORRERRTHRSHTD threshold value and that the CORRERRSTATUS overflow bit for that rank is set. As shown next, the CECC count for rank 0 on MC 0 is 0x2 and that the overflow bit is set. Instead of using the adddc_status_check function, alternative options would be to read the registers mentioned directly or use ras.mem.memCorrEccCount() so only this table is printed. The adddc_status_check is used because later in the test to verify that the VLS region has been established and it is useful to compare the output of the function from before and after VLS.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
```

```
...
***** Memory Correctable ECC Error Count *****
```

Socket	iMC-Channel	Rank	Count	Count Overflow	Threshold	Thrshtd Overflow
socket0	mc0-ch0	0	0x2		0x2	Y
socket0	mc0-ch0	1	0x0		0x2	
socket0	mc0-ch0	2	0x0		0x2	
socket0	mc0-ch0	3	0x0		0x2	
socket0	mc0-ch0	4	0x0		0x2	
socket0	mc0-ch0	5	0x0		0x2	
socket0	mc0-ch0	6	0x0		0x2	

```
=====
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | | |
=====
```

5. Resume the system so BIOS can perform its SMM flow.

```
>>> go()
```

6. Verify that no action was taken and there is still only 1 bank VLS region. Also observe that the per rank CECC counter and overflow bit were cleared.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
| Register Dump | | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x0 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | non-failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x1 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 0 | socket0 | mc0-ch0 | Failed | Device | = | 0x0 | Rank0 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 1 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 2 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 3 | socket0 | mc0-ch0 | - | - | - | - | disabled |
=====

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
=====
```

7. BIOS serial log:

```
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0x9009429
0/13-1 RetryRdErrLogAddr1: 0x44000001
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[imc] memory error handler start!
PS Disabled :UpdateMcChanSsrState ch:0:SSRStopMode
McSsrState Node:0 :SSRStopMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!
S1 C142 T1 [GetChannelErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x0
SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
InterLocSyn:0x0
```

```

*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, Rank 0, SubRank 0, BankGrp 0, Device 1.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P B N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!

[ADDDC]NonRankVlsMode: Call HandleNonRankVlsStrike()...
[ADDDC]: HandleNonRankVlsStrike() start...
[ADDDC]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0, Bank 0, Device 1.
[ADDDC]: BankMode is 1.
[ADDDC]: Failure in Primary VLS Bank. This error can not be handled
[ADDDC]: HandleNonRankVlsStrike() Status Unsupported.
[ADDDC]: HandleNewAdddcEvent() return, Status = Unsupported.
[ADDDC]: Main handler return, Status Unsupported.
RasEventHndlrEntry index: 0x2
RasEventHndlrEntry index: 0x3
RasEventHndlrEntry index: 0x4
RasEventHndlrEntry index: 0x5
[PERF] start
[PERF] end
[GetRRLErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x0 SubRank:0x0 Bank:0x0
BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:2:SSRStopMode
McSsrState Node:2 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:3:SSRStopMode
McSsrState Node:3 :SSRStopMode
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
TranslateSad: McTargetId is 0x0
Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x0 SubRank:0x0
Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x0
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AC82810
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C148 T1 [CpuRas]CMCI is disabled, skip CMCI signal
...
S0 C142 T0 [CpuRas]CMCI is disabled, skip CMCI signal

```

10.2.1.3 Scenario 'b' - 1 Bank VLS then 2nd Strike to Region in VLS, Same Device Number, Different Bank

Test Objective

The objective of this test is to test scenario "b" as described in [this section](#). The scenario is as follows: bank is qualified for RAS action, bank VLS region is established, second bank is qualified for RAS action. The second bank location is in a region in VLS, same device as the failed device, and a different bank number from the failed bank. Expected action is for BIOS to reverse the 1st bank VLS, re-buddy, and create a second bank VLS region with the failing second bank.

Configuration

The following table details the system configuration used to develop this recipe. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW07'24 BKC
OS Version	n/a - test was done at UEFI shell
Kernel	n/a - test was done at UEFI shell
HW Configuration	1 DIMM per socket, 10x4 single rank, 32 GB DIMM
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable

Recipe

1. Establish onebank VLS region as described in [this section](#).
2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2". Therefore, inject two CECC errors to a different bank that is in VLS (only option is the buddy bank).

```
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=1, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=1, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

- Use the following command to verify that the CORRERRCNT counter for that rank has reached the CORRERRTHRSHTD threshold value and that the CORRERRSTATUS overflow bit for that rank is set. As shown next, the CECC count for rank 0 on MC 0 is 0x2 and that the overflow bit is set. Instead of using the adddc_status_check function, alternative options would be to read the registers mentioned directly or use ras.mem.memCorrEccCount() so only this table is printed. The adddc_status_check is used because later in the test to verify that the VLS region has been established and it is useful to compare the output of the function from before and after VLS.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x2 | | 0x2 | Y |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
=====
```

- Resume the system so BIOS can perform its SMM flow.

```
>>> go()
```

- Note that the original failed bank is now buddied with a different bank and a second bank VLS region has been created for the second failing bank.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
| Register Dump | | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x0 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | non-failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x1 | 0x0 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 0 | socket0 | mc0-ch0 | Failed | Device | = | 0x0 | Rank0 |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | failed | 1 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x1 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | non-failed | 1 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x1 | 0x1 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 1 | socket0 | mc0-ch0 | Failed | Device | = | 0x0 | Rank0 |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 2 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 3 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|
=====

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
=====
```

socket0	mc0-ch0	5	0x0				0x2			
socket0	mc0-ch0	6	0x0				0x2			
socket0	mc0-ch0	7	0x0				0x2			

7. BIOS serial log:

```

S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0x9009429
0/13-1 RetryRdErrLogAddr1: 0x44040000
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[imc] memory error handler start!
PS Disabled :UpdateMcChanSsrState ch:0:SSRStopMode
McSsrState Node:0 :SSRStopMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!
S1 C68 T0 [GetChannelErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x0 SubRank:0x0
Bank:0x1 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, Rank 0, SubRank 0, Bank 1, BankGrp 0, Device 0.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P B N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!

[ADDDC]NonRankVlsMode: Call HandleNonRankVlsStrike()...
[ADDDC]: HandleNonRankVlsStrike() start...
[ADDDC]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0, Bank 1, Device 0.
[ADDDC]: BankMode is 2.
  FindBankVlsPairRank(Node 0, Ch 0, Rank 0, RegionNum = 0) = Success
[ADDDC]: BankVlsCnt = 1.
[ADDDC]: Try BANK VLS Reverse...
[ADDDC]: HandleBankVlsRevsEvent()... Start
  FindRegionWithRankBank(Node 0, Ch 0, Rank 0, Bank 1, RegionNum = 0) = Success
[FillRegionStruct]: Region 0, PRank 0, PBank 0, BRank 0, BBank 1, FailedDevice 0.
  AdddcEvent0:
    Type 5, Region 0 (PRank 0 : PBank 0) - (BRank 0 : BBank 1), FailedDevice 0.
[ADDDC]: Trying Bank VLS for the Primary Bank
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0 Bank 0.
  :No available pair rank for the failed rank 0 in channel 0, so select itself as buddy rank for the bank vls
event.
[GetNewNonFailedRankBank]: Rank 0 - Buddy Rank 0, Bank 0 - Buddy Bank 4.
  AdddcEvent1:
    Type 3, Region 0 (PRank 0 : PBank 0) - (BRank 0 : BBank 4), FailedDevice 0.
[ADDDC]: Trying Bank VLS for the buddy bank (current failed bank)
[GetAvailableRegion]: NodeId 0, ChOnNode 0, Region 1, Status Success.
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0 Bank 1.
  :No available pair rank for the failed rank 0 in channel 0, so select itself as buddy rank for the bank vls
event.
[GetNewNonFailedRankBank]: Rank 0 - Buddy Rank 0, Bank 1 - Buddy Bank 5.
  AdddcEvent2:
    Type 3, Region 1 (PRank 0 : PBank 1) - (BRank 0 : BBank 5), FailedDevice 0.
  AdddcEvent3:

```

```

Type 3, Region 1 (PRank 0 : PBank 1) - (BRank 0 : BBank 5), FailedDevice 0.
[ADDDC]: HandleBankVlsRevsEvent()... End
[ADDDC]: HandleNonRankVlsStrike() Status Success.
[ADDDC]: The event has been successfully handled!
[ADDDC]: Call SchedulePendingList()
DisableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:80
[SSR] disable C6 & GV
[SSR] stop and save patrol scrub info
[ADDDC]: ShedulePendingList()... Start
[ActionOnBankVlsReverse] Region 0, NodeId 0, Ch 0, PRank 0 - BRank 0, PBank 0 - BBank 1.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: Event type 5, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleNewAdddcEvent() return, Status = Success.
[ADDDC]: Main handler return, Status Success.
[GetRRLErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x0 SubRank:0x0 Bank:0x1
BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:2:SSRStopMode
McSsrState Node:2 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:3:SSRStopMode
McSsrState Node:3 :SSRStopMode
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
TranslateSad: McTargetId is 0x0
Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x0 SubRank:0x0
Bank:0x1 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
Ssr copy inprogress
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x0
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AC82810
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C130 T1 [CpuRas]CMCI is disabled, skip CMCI signal
...
S0 C151 T1 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C19 T0 [CpuRas]CMCI is disabled, skip CMCI signal

[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: ShedulePendingList()... Start
[ActionOnBankVls] Region 0, NodeId 0, Ch 0, FailedRank 0 - NonFailedRank 0, FailedBank 0 - NonFailedBank 4.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:2:SSRStopMode
McSsrState Node:2 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:3:SSRStopMode
McSsrState Node:3 :SSRStopMode
[imc] memory error handler end!
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: ShedulePendingList()... Start
[ActionOnBankVls] Region 1, NodeId 0, Ch 0, FailedRank 0 - NonFailedRank 0, FailedBank 1 - NonFailedBank 5.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode

```

```
McSsrState Node:1 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:2:SSRStopMode
McSsrState Node:2 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:3:SSRStopMode
McSsrState Node:3 :SSRStopMode
[imc] memory error handler end!
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: SchedulePendingList()... Start
[CleanPreVlsEvent]: NodeId 0.
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:81
[SSR] restore and restart patrol scrub info
[SSR] enable C6
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P P N N B B N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!

[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:2:SSRStopMode
McSsrState Node:2 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:3:SSRStopMode
McSsrState Node:3 :SSRStopMode
[imc] memory error handler end!
```

10.2.1.4 Scenario 'c' - 1 Bank VLS then 2nd Strike to Region in VLS, Different Device Number, Different Bank

Test Objective

The objective of this test is to test scenario "c" as described in [this section](#). The scenario is as follows: bank is qualified for RAS action, bank VLS region is established, second bank is qualified for RAS action. The second bank location is in a region in VLS, different device and bank versus the failed bank. Expected action is for BIOS to reverse the 1st bank VLS, re-buddy, and create a second bank VLS region with the failing second bank.

Configuration

The following table details the system configuration used to develop this recipe. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW07'24 BKC
OS Version	n/a - test was done at UEFI shell
continued...	

Kernel	n/a - test was done at UEFI shell
HW Configuration	1 DIMM per socket, 10x4 single rank, 32 GB DIMM
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable

Recipe

1. Establish one bank VLS region as described in [this section](#).
2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2". Therefore, inject two CECC errors to a different device and bank from before. Since the objective is to target a VLS region, the only bank to choose is the buddy bank and the choice for device can be anything other than the failed device (device one was chosen in this example).

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=1, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=1, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

4. Use the following command to verify that the CORRERRCNT counter for that rank has reached the CORRERRTHRSHLD threshold value and that the CORRERRORSTATUS overflow bit for that rank is set. As shown next, the CECC count for rank 0 on MC 0 is 0x2 and that the overflow bit is set. Instead of using the adddc_status_check function, alternative options would be to read the registers mentioned directly or use ras.mem.memCorrEccCount() so only this table is printed. The adddc_status_check is used because later in the test to verify that the VLS region has been established and it is useful to compare the output of the function from before and after VLS.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x2 | | 0x2 | Y |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
=====
```

5. Resume the system so BIOS can perform its SMM flow.

```
>>> go()
```

6. Note that the original failed bank is now buddied with a different bank and a second bank VLS region has been created for the second failing bank (different device compared to scenario 'b').

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
| Register Dump | | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x0 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | non-failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x1 | 0x0 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 0 | socket0 | mc0-ch0 | Failed | Device | = | 0x0 | Rank0 |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | failed | 1 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x1 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | non-failed | 1 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x1 | 0x1 | Bank |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 1 | socket0 | mc0-ch0 | Failed | Device | = | 0x1 | Rank0 |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 2 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 3 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|
=====

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
=====
```

7. BIOS serial log:

```

S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0x9009429
0/13-1 RetryRdErrLogAddr1: 0x44040001
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[imc] memory error handler start!
PS Disabled :UpdateMcChanSsrState ch:0:SSRStopMode
McSsrState Node:0 :SSRStopMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!
S1 C70 T1 [GetChannelErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x0 SubRank:0x0
Bank:0x1 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, Rank 0, SubRank 0, Bank 1, BankGrp 0, Device 1.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P B N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!

[ADDDC]NonRankVlsMode: Call HandleNonRankVlsStrike()...
[ADDDC]: HandleNonRankVlsStrike() start...
[ADDDC]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0, Bank 1, Device 1.
[ADDDC]: BankMode is 2.
FindBankVlsPairRank(Node 0, Ch 0, Rank 0, RegionNum = 0) = Success
[ADDDC]: BankVlsCnt = 1.
[ADDDC]: Try BANK VLS Reverse...
[ADDDC]: HandleBankVlsRevsEvent()... Start
FindRegionWithRankBank(Node 0, Ch 0, Rank 0, Bank 1, RegionNum = 0) = Success
[FillRegionStruct]: Region 0, PRank 0, PBank 0, BRank 0, BBank 1, FailedDevice 0.
AdddcEvent0:
Type 5, Region 0 (PRank 0 : PBank 0) - (BRank 0 : BBank 1), FailedDevice 0.
[ADDDC]: Trying Bank VLS for the Primary Bank
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0 Bank 0.
:No available pair rank for the failed rank 0 in channel 0, so select itself as buddy rank for the bank vls
event.
[GetNewNonFailedRankBank]: Rank 0 - Buddy Rank 0, Bank 0 - Buddy Bank 4.
AdddcEvent1:
Type 3, Region 0 (PRank 0 : PBank 0) - (BRank 0 : BBank 4), FailedDevice 0.
[ADDDC]: Trying Bank VLS for the buddy bank (current failed bank)
[GetAvailableRegion]: NodeId 0, ChOnNode 0, Region 1, Status Success.
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0 Bank 1.
:No available pair rank for the failed rank 0 in channel 0, so select itself as buddy rank for the bank vls
event.
[GetNewNonFailedRankBank]: Rank 0 - Buddy Rank 0, Bank 1 - Buddy Bank 5.
AdddcEvent2:
Type 3, Region 1 (PRank 0 : PBank 1) - (BRank 0 : BBank 5), FailedDevice 1.
AdddcEvent3:
Type 3, Region 1 (PRank 0 : PBank 1) - (BRank 0 : BBank 5), FailedDevice 1.
[ADDDC]: HandleBankVlsRevsEvent()... End
[ADDDC]: HandleNonRankVlsStrike() Status Success.
[ADDDC]: The event has been successfully handled!
[ADDDC]: Call SchedulePendingList()
DisableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:80
[SSR] disable C6 & GV
[SSR] stop and save patrol scrub info
[ADDDC]: ShedulePendingList()... Start

```



```
[ActionOnBankVlsReverse] Region 0, NodeId 0, Ch 0, PRank 0 - BRank 0, PBank 0 - BBank 1.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: Event type 5, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleNewAdddcEvent() return, Status = Success.
[ADDDC]: Main handler return, Status Success.
[GetRRLErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x0 SubRank:0x0 Bank:0x1
BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:2:SSRStopMode
McSsrState Node:2 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:3:SSRStopMode
McSsrState Node:3 :SSRStopMode
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
TranslateSad: McTargetId is 0x0
Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x0 SubRank:0x0
Bank:0x1 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
Ssr copy inprogress
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x0
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AC82810
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C9 T1 [CpuRas]CMCI is disabled, skip CMCI signal
...
S0 C80 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C86 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C149 T0 [CpuRas]CMCI is disabled, skip CMCI signal

[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: SchedulePendingList()... Start
[ActionOnBankVls] Region 0, NodeId 0, Ch 0, FailedRank 0 - NonFailedRank 0, FailedBank 0 - NonFailedBank 4.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:2:SSRStopMode
McSsrState Node:2 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:3:SSRStopMode
McSsrState Node:3 :SSRStopMode
[imc] memory error handler end!
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: SchedulePendingList()... Start
[ActionOnBankVls] Region 1, NodeId 0, Ch 0, FailedRank 0 - NonFailedRank 0, FailedBank 1 - NonFailedBank 5.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:2:SSRStopMode
McSsrState Node:2 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:3:SSRStopMode
McSsrState Node:3 :SSRStopMode
[imc] memory error handler end!
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
```

```
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
[ADDDC]: ShedulePendingList()... Start
[CleanPreVlsEvent]: NodeId 0.
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:81
[SSR] restore and restart patrol scrub info
[SSR] enable C6
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000003 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P P N N B B N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!

[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: ShedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:2:SSRStopMode
McSsrState Node:2 :SSRStopMode
PS Disabled :UpdateMcChanSsrState ch:3:SSRStopMode
McSsrState Node:3 :SSRStopMode
[imc] memory error handler end!
```

10.2.1.5 Scenario 'd' - 1 Bank VLS then 2nd Strike to Region not in VLS, Same Device Number, Different Bank

Test Objective

The objective of this test is to test scenario "d" as described in [this section](#). The scenario is as follows: bank is qualified for RAS action, bank VLS region is established, second bank is qualified for RAS action. Where the second bank location is to a region not in VLS, same device as the failed device, but a different bank. Expected outcome is no action taken. If there were more ranks on the channel then rank VLS action would have been taken.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45'23 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	4 DIMMs per socket, 10x4 single rank, 32 GB DIMMs on slots A1, B1, C1, and D1. Having 1 single rank DIMM on a channel is the main requirement for this test. Different number of ranks on the channel will change the buddy rank selected and RAS action taken once the threshold is met.
CScripts Version	BHS.2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is highly recommended to have a POR memory population.

Recipe

1. Establish one bank VLS region as described in [this section](#).
2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2". Therefore, inject two CECC errors to the same device (memDevs is unchanged) but to a bank not in VLS (bank two was chosen).

```
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=2, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=2, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

4. Use the following command to verify that the CORRERRCNT counter for that rank has reached the CORRERRTHRSHTD threshold value and that the CORRERRORSTATUS overflow bit for that rank is set. As shown next, the CECC count for rank 0 on MC 0 is 0x2 and that the overflow bit is set. Instead of using the adddc_status_check function, alternative options would be to read the registers mentioned directly or use ras.mem.memCorrEccCount() so only this table is printed. The adddc_status_check is used because later in the test to verify that the VLS region has been established and it is useful to compare the output of the function from before and after VLS.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
```

Socket	iMC-Channel	Rank	Count	Count Overflow	Threshold	Thrshld Overflow
socket0	mc0-ch0	0	0x2		0x2	Y
socket0	mc0-ch0	1	0x0		0x2	
socket0	mc0-ch0	2	0x0		0x2	
socket0	mc0-ch0	3	0x0		0x2	
socket0	mc0-ch0	4	0x0		0x2	
socket0	mc0-ch0	5	0x0		0x2	
socket0	mc0-ch0	6	0x0		0x2	
socket0	mc0-ch0	7	0x0		0x2	

- Resume the system so BIOS can perform its SMM flow and start the VLS sparing operation.

```
>>> go()
```

- Verify that no action was taken and there is still only one bank VLS region. Also observe that the per rank CECC counter and overflow bit were cleared. If the channel has two or four ranks, then it is expected to see one rank VLS region.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
| Register Dump |          | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
|-----|
| SVLS | failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x0 | Bank |
|-----|
| SVLS | non-failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x1 | Bank |
|-----|
| SVLS | --- | 0 | socket0 | mc0-ch0 | Failed | Device | = | 0x0 | Rank0 |
|-----|
| SVLS | --- | 1 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|
| SVLS | --- | 2 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|
| SVLS | --- | 3 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|
=====

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
|-----|
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
|-----|
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
|-----|
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
|-----|
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
|-----|
=====
```

- ftrace output:

```
kworker/0:3-2963 [000] ..... 870.770196: mce_record: CPU: 0, MCGc/s: f000c20/0, MC13: 8c00004200800090, IPID:
0000000000000000, ADDR/MISC/SYND: 0000000010008600/0400004008008086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
17ffe9d589f, PROCESSOR: 0:a06d0, TIME: 1679271254, SOCKET: 0, APIC: 0

kworker/0:3-2963 [000] ..... 870.770261: extlog_mem_event: {2} corrected error: single-bit ECC physical addr:
0000000010008600 (mask lsb: ff) node:0 card:0 module:0 rank:0 bank:2 device:0 row:256 column:256 DIMM location: not
present. DMI handle: 0x0000 FRU: 00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0

kworker/0:3-2963 [000] ..... 870.771355: mc_event: 1 Corrected error: memory read error on
CPU_SrcID#0_MC#0_Chan#0_DIMM#0 (mc:0 location:0:0:-1 address:0x10008600 grain:32 syndrome:0x00000000)
```

```
err_code:0x0080:0x0090 SystemAddress:0x10008600 ProcessorSocketId:0x0 MemoryControllerId:0x0 ChannelAddress:0x4002100
ChannelId:0x0 RankAddress:0x1000800 PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x100 Column:0x100 Bank:0x2
BankGroup:0x0 ChipSelect:0x0)
```

8. BIOS serial log:

```
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0xB009409
0/13-1 RetryRdErrLogAddr1: 0x44080000
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!
S1 C8 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x0 SubRank:0x0
Bank:0x2 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, Rank 0, SubRank 0, Bank 2, BankGrp 0, Device 0.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P B N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!

[ADDDC]NonRankVlsMode: Call HandleNonRankVlsStrike()...
[ADDDC]: HandleNonRankVlsStrike() start...
[ADDDC]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0, Bank 2, Device 0.
[ADDDC]: BankMode is 0.
[ADDDC]: Previous error in device 0!
FindBankVlsPairRank(Node 0, Ch 0, Rank 0, RegionNum = 0) = Success
[ADDDC]: BankVlsCnt is 1.
[ADDDC]: Promote BANK to RANK VLS...
[ADDDC]: HandleBank2RankVlsEvent()... Start
FindRegionWithRank(Node 0, Ch 0, Rank 0, RegionNum = 0) = Success
[FillRegionStruct]: Region 0, PRank 0, PBank 0, BRank 0, BBank 1, FailedDevice 0.
BANK VLS in single rank, skip BANK to RANK VLS promotion.
[ADDDC]: HandleBank2RankVlsEvent() Status = Unsupported, try next memory sparing feature...
[ADDDC]: HandleNonRankVlsStrike() Status = Unsupported.
[ADDDC]: HandleNewAdddcEvent() return, Status = Unsupported.
[ADDDC]: Main handler return, Status = Unsupported.
RasEventHndlrEntry index: 0x2
RasEventHndlrEntry index: 0x3
RasEventHndlrEntry index: 0x4
RasEventHndlrEntry index: 0x5
[PERF] start
[PERF] end
[GetRRLErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x0 SubRank:0x0 Bank:0x2
BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
UpdateMcChanSsrState ch:1:PatrolSPAMode
McSsrState Node:1 :PatrolSPAMode
UpdateMcChanSsrState ch:2:PatrolSPAMode
McSsrState Node:2 :PatrolSPAMode
UpdateMcChanSsrState ch:3:PatrolSPAMode
McSsrState Node:3 :PatrolSPAMode
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
```

```
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
TranslateSad: McTargetId is 0x0
Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x0 SubRank:0x0
Bank:0x2 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AC99810
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
LogErrorToElog: --Logging Error to EMCA
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T0 [CpuRas]Successfully notify OS via CMCI
S0 C146 T1 [CpuRas]Successfully notify OS via CMCI
S0 C69 T1 [CpuRas]Successfully notify OS via CMCI
S0 C23 T1 [CpuRas]Successfully notify OS via CMCI
...
```

10.2.1.6 Scenario 'e' - 1 Bank VLS then 2nd Strike to Region not in VLS, Different Device Number, Different Bank

Test Objective

The objective of this test is to test scenario "e" as described in [this section](#). The scenario is as follows: bank is qualified for RAS action, bank VLS region is established, second bank is qualified for RAS action. Where the second bank location is to a region not in VLS, different device compared to the failed device, different bank. Expected outcome is to observe a second independent bank VLS region established.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW45'23 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	4 DIMMs per socket, 10x4 single rank, 32 GB DIMMs on slots A1, B1, C1, and D1. Having 1 single rank DIMM on a channel is the main requirement for this test. Different number of ranks on the channel will impact the buddy rank selected.
CScripts Version	BHS.2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is highly recommended to have a POR memory population.

Recipe

1. Establish one bank VLS region as described in [this section](#).
2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2". Therefore, inject two CECC errors to a different device (memDevs was changed, dev0 is now 1) but to a bank not in VLS (bank two was chosen).

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=2, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=2, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

4. Use the following command to verify that the CORRERRCNT counter for that rank has reached the CORRERRTHRSHLD threshold value and that the CORRERRORSTATUS overflow bit for that rank is set. As shown next, the CECC count for rank 0 on MC 0 is 0x2 and that the overflow bit is set. Instead of using the adddc_status_check function, alternative options would be to read the registers mentioned directly or use ras.mem.memCorrEccCount() so only this table is printed. The adddc_status_check is used because later in the test to verify that the VLS region has been established and it is useful to compare the output of the function from before and after VLS.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x2 | | 0x2 | Y |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
```

socket0	mc0-ch0	3	0x0			0x2		
socket0	mc0-ch0	4	0x0			0x2		
socket0	mc0-ch0	5	0x0			0x2		
socket0	mc0-ch0	6	0x0			0x2		
socket0	mc0-ch0	7	0x0			0x2		

- Resume the system so BIOS can perform its SMM flow and start the VLS sparing operation.

```
>>> go()
```

- Verify that a second bank VLS region was established and the first region remains the same. The second region's failed bank should align with the previous error injections (socket, channel, cs, device, and so on). Also observe that the per rank CECC counter and overflow bit were cleared.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
| Register Dump |          | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
|-----|
| SVLS | failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x0 | Bank |
| SVLS | non-failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x1 | Bank |
| SVLS | --- | 0 | socket0 | mc0-ch0 | Failed | Device | = | 0x0 | Rank0 |
| SVLS | failed | 1 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x2 | Bank |
| SVLS | non-failed | 1 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x3 | Bank |
| SVLS | --- | 1 | socket0 | mc0-ch0 | Failed | Device | = | 0x1 | Rank0 |
| SVLS | --- | 2 | socket0 | mc0-ch0 | - | - | - | - | disabled |
| SVLS | --- | 3 | socket0 | mc0-ch0 | - | - | - | - | disabled |
=====

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
=====
```

- fttrace output:

```
kworker/0:0-8 [000] ..... 813.446617: mce_record: CPU: 0, MCGc/s: f000c20/0,
MC13: 8c00004200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000010008600/0400084008008086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
167dc61c167, PROCESSOR: 0:a06d0, TIME: 1679270999, SOCKET: 0, APIC: 0

kworker/0:0-8 [000] ..... 813.446721: extlog_mem_event: {2} corrected error:
single-bit ECC physical addr: 0000000010008600 (mask lsb: ff) node:0 card:0 module:0 rank:0
bank:0 device:1 row:256 column:256 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0

kworker/0:0-8 [000] ..... 813.448007: mc_event: 1 Corrected error: memory read
error on CPU_SrcID#0_MC#0_Chan#0_DIMM#0 (mc:0 location:0:0:-1 address:0x10008600 grain:32
```



```
syndrome:0x00000000 err_code:0x0080:0x0090 SystemAddress:0x10008600 ProcessorSocketId:0x0
MemoryControllerId:0x0 ChannelAddress:0x4002100 ChannelId:0x0 RankAddress:0x1000800
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x100 Column:0x100 Bank:0x2 BankGroup:0x0
ChipSelect:0x0)
```

8. BIOS serial log for the SMI that activates VLS sparing and handles the memory error:

```
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0xB009409
0/13-1 RetryRdErrLogAddr1: 0x44080001
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!
S1 C74 T1 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh
is 0x0Rank:0x0 SubRank:0x0 Bank:0x2 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, Rank 0, SubRank 0, Bank 2, BankGrp 0, Device 1.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: P B N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
  S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!

[ADDDC]NonRankVlsMode: Call HandleNonRankVlsStrike()...
[ADDDC]: HandleNonRankVlsStrike() start...
[ADDDC]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0, Bank 2, Device 1.
[ADDDC]: BankMode is 0.
[ADDDC]: No previous error in device 1.
[ADDDC]: Call HandleBankVlsEvent()...
```

```
[ADDDC]: HandleBankVlsEvent()... Start
[GetAvailableRegion]: NodeId 0, ChOnNode 0, Region 1, Status Success.
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0 Bank 2.
:No available pair rank for the failed rank 0 in channel 0, so select itself as buddy
rank for the bank vls event.
[GetNewNonFailedRankBank]: Rank 0 - Buddy Rank 0, Bank 2 - Buddy Bank 3.
AdddcEvent0:
    Type 3, Region 1          (PRank 0 : PBank 2) - (BRank 0 : BBank 3), FailedDevice 1.
AdddcEvent1:
    Type 3, Region 1          (PRank 0 : PBank 2) - (BRank 0 : BBank 3), FailedDevice 1.
[ADDDC]: HandleBankVlsEvent()... End
[ADDDC]: HandleNonRankVlsStrike() Status Success.
[ADDDC]: The event has been successfully handled!
[ADDDC]: Call SchedulePendingList()
DisableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:80
[SSR] disable C6 & GV
[SSR] stop and save patrol scrub info
[ADDDC]: ShedulePendingList()... Start
    [ActionOnBankVls] Region 1, NodeId 0, Ch 0, FailedRank 0 - NonFailedRank 0, FailedBank 2 -
    NonFailedBank 3.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AF0B610
WHEA: Detected Ras Non Standard Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleNewAdddcEvent() return, Status = Success.
[ADDDC]: Main handler return, Status Success.
[GetRRLErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0
Rank:0x0 SubRank:0x0 Bank:0x2 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0
UpdateMcChanSsrState ch:1:PatrolSPAMode
McSsrState Node:1 :PatrolSPAMode
UpdateMcChanSsrState ch:2:PatrolSPAMode
McSsrState Node:2 :PatrolSPAMode
UpdateMcChanSsrState ch:3:PatrolSPAMode
McSsrState Node:3 :PatrolSPAMode
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
TranslateSad: McTargetId is 0x0
Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh
is 0x0Rank:0x0 SubRank:0x0 Bank:0x2 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
Ssr copy inprogress
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AC99810
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
LogErrorToElog: --Logging Error to EMCA
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T1 [CpuRas]Successfully notify OS via CMCI
S0 C150 T0 [CpuRas]Successfully notify OS via CMCI
S0 C13 T1 [CpuRas]Successfully notify OS via CMCI
S0 C73 T1 [CpuRas]Successfully notify OS via CMCI
...
```

9. BIOS serial log for the SMI at sparing completion:

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AF0B610
WHEA: Detected Ras Non Standard Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
[ADDDC]: ShedulePendingList()... Start
[CleanPreVlsEvent]: NodeId 0.
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:81
[SSR] restore and restart patrol scrub info
[SSR] enable C6
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000003 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: P B P B N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N N
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!

[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
UpdateMcChanSsrState ch:1:PatrolSPAMode
McSsrState Node:1 :PatrolSPAMode
UpdateMcChanSsrState ch:2:PatrolSPAMode
McSsrState Node:2 :PatrolSPAMode
UpdateMcChanSsrState ch:3:PatrolSPAMode
McSsrState Node:3 :PatrolSPAMode
[imc] memory error handler end!
```

10.2.2 Four Ranks on the Channel

This section covers the eight test cases with four ranks on the channel that were mentioned [previously](#).

10.2.2.1 One Bank VLS Region

Test Objective

The objective of this test is to test one bank VLS region on a channel. Multiple correctable ECC errors are injected until the HW threshold is met to trigger ADDDC flow and various CSRs are read to confirm that BIOS and HW are behaving as expected.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores A step
BIOS	WW15'24 BKC
OS Version	n/a - Test executed and logs collected at UEFI shell
Kernel	n/a - Test executed and logs collected at UEFI shell
HW Configuration	4 DIMMs per socket, 10x4 dual rank, 64 GB DIMMs on slots A1, A2, B1, and B2. A non-POR configuration was used to develop this recipe but POR configurations are expected to be used by the audience of this document.
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Halt and confirm there are no preexisting memory related errors on the system. Clear any found error logs or cold boot the system.

```
>>> halt()
>>> error.check_mem_errors()
*****
                        No errors found
*****
```

- a. Multiple CECC error injections are performed while halted, the BIOS and OS actions/reporting that would occur if the test were done while the system was running do not occur. One way to modify this test is to inject errors while the system is running to allow the BIOS/OS actions for each CECC error to occur.

2. Verify that ADDDC is enabled (see the following ADDDC row).

```
>>> mc.dimminfoNoSPD()
INFO: BIOS knob 'Disable BIOS Done' is enabled!
INFO: BIOS knob 'Enable DFX Features' is enabled!
Discovering Populated Channels
Discovering Channels
```

	Socket 0	Socket 1	System
Active Memory	256.0GB	256.0GB	512.0GB
Expected Memory	NotAvail	NotAvail	NotAvail
Clustering Mode	Quad	Quad	Quad
1LM Memory Size	256.00GB	256.00GB	512.00GB
2LM Memory Size	0.00 B	0.00 B	0.00 B
AppDirect Memory Size	0.00 B	0.00 B	0.00 B
DDR Frequency	5200MHz	5200MHz	5200MHz
Max DDR Frequency	6400MHz	6400MHz	
MC0 CL-RCD-RP-CMD	42-42-43-1n	42-42-43-1n	
MC1 CL-RCD-RP-CMD	42-42-43-1n	42-42-43-1n	
MC0 CL-RCD-RP-CMD	42-42-43-1n	42-42-43-1n	
MC1 CL-RCD-RP-CMD	42-42-43-1n	42-42-43-1n	
ECC Checking / PFD	On / On	On / On	On / On
ECC Mode	128b	128b	128b
CAP/WrCRC	On / Disable	On / Disable	On / Disable
Patrol/DemandScrub	Off / On	Off / On	Off/On
Memory Mode	1LM	1LM	1LM
Channel Mode	Independent	Independent	Independent
Refresh Mode	SameBankFine	SameBankFine	SameBankFine
RFM	Off	Off	Off
ZQCal/DQSOSC/RxRt	On / On / Off	On / On / Off	On / On / Off
ADDDC	Enabled	Enabled	Enabled
Encryption	Disabled	Disabled	Disabled
Paging Policy	AdaptOpen	AdaptOpen	AdaptOpen
Directory Mode	Off	Off	Off
DDR Scrambling	Enabled	Enabled	Enabled
CKE Throttling	PPD	PPD	PPD
Thermal Throttling	CLTT	CLTT	CLTT
NUMA			On

3. Check if the injectors are locked for all sockets and unlock if locked.

```
>>> ei.utils.checkInjectorLock(and_unlock=True)
```

4. Modify memDevs for a single device error injection. Aside from a few DUE patterns and depending on ECC mode, nearly 100% of single device non-permanent ECC errors are correctable. Therefore, the dev0msk value can vary and the test will still be valid, but here 0x3 is used to flip bits 0 and 1 on burst 0. This step is optional because the default memDevs will work but this step ensures that any previous memDevs modifications that may have been done do not ruin the test.

```
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
dev0 = 0x0, dev0msk = 0x3, dev1 = 1, dev1msk = 0x0. Customized injectors may not work
correctly in RAS modes
```

5. The hardware per rank CECC error thresholds were set to "2". So inject two CECC errors to the same location to reach the threshold value.

```
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
```

```
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

- Use the following command to verify that the CORRERRCNT counter for that rank has reached the CORRERRTHSHLD threshold value and that the CORRERRORSTATUS overflow bit for that rank is set. As shown next, the CECC count for rank 0 on MC 0 is 0x2 and that the overflow bit is set. Instead of using the adddc_status_check function, alternative options would be to read the registers mentioned directly or use ras.mem.memCorrEccCount() so only this table is printed. The adddc_status_check is used because later in the test to verify that the VLS region has been established and it is useful to compare the output of the function from before and after VLS.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
=====
| socket0 | mc0-ch0 | 0 | 0x2 | | 0x2 | Y |
=====
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
=====
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
=====
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
=====
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
=====
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
=====
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
=====
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
=====
```

- Resume the system so BIOS can perform its SMM flow and start the VLS sparing operation.

```
>>> go()
```

- Read the following copy_in_progress bit to check if VLS sparing is in progress. The VLS sparing operation varies in time based on the configuration so this bit may not be seen to be "1" because sparing completed before the CSR was read. CECC logging and signaling is disabled during ADDDC sparing. If further error injections are desired then it is necessary to wait until copy_in_progress is 0 and BIOS has reenabled logging and signaling.

```
>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.sparing_patrol_status.show()
0x00000000 : patrol_complete (02:02)
0x00000000 : copy_complete (01:01)
0x00000001 : copy_in_progress (00:00)
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.dis_corr_err_log
0xff
```

- Verify that the ADDDC_REGION and ADDDC_REGION_CONTROL registers were modified by BIOS and indicate the failed and non-failed (buddy) ranks. The information printed for the failed bank and device shown next align with the previous error injection. The selected buddy rank (CS == 0x4) and bank (Bank == 0x1) match the expectations described in [Intel BKC BIOS ADDDC Actions](#) on page 225 and the per rank ECC error counter has been cleared (Count == 0x0).

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
| Register Dump |          | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x0 | Bank |
| SVLS | non-failed | 0 | socket0 | mc0-ch0 | 0x4 | 0x0 | 0x0 | 0x1 | Bank |
| SVLS | --- | 0 | socket0 | mc0-ch0 | Failed | Device | = | 0x0 | Rank0 |
| SVLS | --- | 1 | socket0 | mc0-ch0 | - | - | - | - | disabled |
| SVLS | --- | 2 | socket0 | mc0-ch0 | - | - | - | - | disabled |
| SVLS | --- | 3 | socket0 | mc0-ch0 | - | - | - | - | disabled |
=====

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
=====
```

10. BIOS serial log for the SMI to handle the memory error and activate VLS sparing:

```
[ProcessMcBanks] McBankBitFields 2000
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0xB009409
0/13-1 RetryRdErrLogAddr1: 0x44000000
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[imc] memory error handler start!
PS Disabled :UpdateMcChanSsrState ch:0:SSRStopMode
McSsrState Node:0 :SSRStopMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
S1 C83 T1 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh
is 0x0Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, SubRank 0, Bank 0, BankGrp 0, Device 0.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N
```

```

N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 4, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 6, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N

[ADDDC]NonRankVlsMode: Call HandleNonRankVlsStrike()...
[ADDDC]: HandleNonRankVlsStrike() start...
[ADDDC]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0, Bank 0, Device 0.
[ADDDC]: BankMode is 0.
[ADDDC]: No previous error in device 0.
[ADDDC]: Call HandleBankVlsEvent()...
[ADDDC]: HandleBankVlsEvent()... Start
[GetAvailableRegion]: NodeId 0, ChOnNode 0, Region 0, Status Success.
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0 Bank 0.
FindBankVlsPairRank(Node 0, Ch 0, Rank 0, RegionNum = 255) = Not Found
FindBankVlsPairRank(Node 0, Ch 0, Rank 4, RegionNum = 255) = Not Found
Check buddy rank 4.
The buddy rank 4 is valid.
[GetNewNonFailedRankBank]: Rank 0 - Buddy Rank 4, Bank 0 - Buddy Bank 1.
AdddcEvent0:
Type 3, Region 0 (PRank 0 : PBank 0) - (BRank 4 : BBank 1), FailedDevice 0.
AdddcEvent1:
Type 3, Region 0 (PRank 0 : PBank 0) - (BRank 4 : BBank 1), FailedDevice 0.
[ADDDC]: HandleBankVlsEvent()... End
[ADDDC]: HandleNonRankVlsStrike() Status Success.
[ADDDC]: The event has been successfully handled!
[ADDDC]: Call SchedulePendingList()
DisableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:80
[SSR] disable C6 & GV
[SSR] stop and save patrol scrub info
[ADDDC]: SchedulePendingList()... Start

```



```
[ActionOnBankVls] Region 0, NodeId 0, Ch 0, FailedRank 0 - NonFailedRank 4, FailedBank 0 -
NonFailedBank 1.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleNewAdddcEvent() return, Status = Success.
[ADDDC]: Main handler return, Status Success.
[GetRRLErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0
Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh
is 0x0 Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 Ssr copy inprogress
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x0
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64018
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T0 [CpuRas]CMCI is disabled, skip CMCI signal
...
S0 C91 T1 [CpuRas]CMCI is disabled, skip CMCI signal
```

11. BIOS serial log for the SMI at VLS sparing completion:

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: SchedulePendingList()... Start
[CleanPreVlsEvent]: NodeId 0.
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:81
[SSR] restore and restart patrol scrub info
[SSR] enable C6
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: P N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
```

```

- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 4, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N B N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 6, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N

[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!

```

10.2.2.2 Scenario 'a' - 1 Bank VLS then 2nd Strike to Failed Rank, Region in VLS, Same Bank, Different Device

Test Objective

The objective of this test is to test scenario 'a' as described in [this section](#).

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores A step
BIOS	WW15'24 BKC
OS Version	n/a - Test executed and logs collected at UEFI shell
<i>continued...</i>	

Kernel	n/a - Test executed and logs collected at UEFI shell
HW Configuration	4 DIMMs per socket, 10x4 dual rank, 64 GB DIMMs on slots A1, A2, B1, and B2. A non-POR configuration was used to develop this recipe but POR configurations are expected to be used by the audience of this document.
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Establish 1 bank VLS region as described in [One Bank VLS Region](#) on page 259.
2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2" so inject two CECC errors to a different device compared to the location/target in step 1.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

4. Check that the per rank CECC error threshold has been met.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x2 | | 0x2 | Y |
```

	socket0		mc0-ch0		1		0x0				0x2			
	socket0		mc0-ch0		2		0x0				0x2			
	socket0		mc0-ch0		3		0x0				0x2			
	socket0		mc0-ch0		4		0x0				0x2			
	socket0		mc0-ch0		5		0x0				0x2			
	socket0		mc0-ch0		6		0x0				0x2			
	socket0		mc0-ch0		7		0x0				0x2			

5. Resume the system so BIOS can perform its SMM flow.

```
>>> go()
```

6. Not expecting any VLS sparing operations to occur so copy_in_progress should be 0.

```
>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.sparing_patrol_status.show()
0x00000000 : patrol_complete (02:02)
0x00000000 : copy_complete (01:01)
0x00000000 : copy_in_progress (00:00)
```

7. Verify that the initial bank VLS region remains the same and the per rank CECC error counter was cleared.

```
>>> ras.mem.addcc_status_check(socket=0, mc=0)
...
=====
| Register Dump |           | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
|-----|
| SVLS | failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x0 | Bank |
| SVLS | non-failed | 0 | socket0 | mc0-ch0 | 0x4 | 0x0 | 0x0 | 0x1 | Bank |
| SVLS | --- | 0 | socket0 | mc0-ch0 | Failed | Device | = | 0x0 | Rank0 |
| SVLS | --- | 1 | socket0 | mc0-ch0 | - | - | - | - | disabled |
| SVLS | --- | 2 | socket0 | mc0-ch0 | - | - | - | - | disabled |
| SVLS | --- | 3 | socket0 | mc0-ch0 | - | - | - | - | disabled |
=====

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
=====
```

10.2.2.3 Scenario 'b' - 1 Bank VLS then 2nd Strike to Failed Rank, Region not in VLS, Same Device, Different Bank

Test Objective

The objective of this test is to test scenario 'b' as described in [this section](#).

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores A step
BIOS	WW15'24 BKC
OS Version	n/a - Test executed and logs collected at UEFI shell
Kernel	n/a - Test executed and logs collected at UEFI shell
HW Configuration	4 DIMMs per socket, 10x4 dual rank, 64 GB DIMMs on slots A1, A2, B1, and B2. A non-POR configuration was used to develop this recipe but POR configurations are expected to be used by the audience of this document.
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold = Disabled
 - ADDDC Sparing = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Establish 1 bank VLS region as described in [One Bank VLS Region](#) on page 259.

2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2" so inject two CECC errors to a different bank compared to the location/target in step 1.

```
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=1, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=1, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

4. Check that the per rank CECC error threshold has been met.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x2 | | 0x2 | Y |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
=====
```

5. Resume the system so the BIOS can perform its SMM flow.

```
>>> go()
```

6. Expecting VLS sparing operations to occur to promote the bank VLS region to rank VLS so copy_in_progress should be 1.

```
>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.sparing_patrol_status.show()
0x00000000 : patrol_complete (02:02)
0x00000000 : copy_complete (01:01)
0x00000001 : copy_in_progress (00:00)
```

7. Check the "Region" column to verify that the bank VLS region has been promoted to rank VLS and check that the per rank CECC error counters have been cleared.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
| Register Dump | | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x0 | Rank |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | non-failed | 0 | socket0 | mc0-ch0 | 0x4 | 0x0 | 0x0 | 0x1 | Rank |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 0 | socket0 | mc0-ch0 | Failed | Device | = | 0x0 | Rank0 |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 1 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 2 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|
| SVLS | --- | 3 | socket0 | mc0-ch0 | - | - | - | - | disabled |
|-----|-----|-----|-----|-----|-----|-----|-----|
```

***** Memory Correctable ECC Error Count *****

Socket	iMC-Channel	Rank	Count	Count Overflow	Threshold	Threshld Overflow
socket0	mc0-ch0	0	0x0		0x2	
socket0	mc0-ch0	1	0x0		0x2	
socket0	mc0-ch0	2	0x0		0x2	
socket0	mc0-ch0	3	0x0		0x2	
socket0	mc0-ch0	4	0x0		0x2	
socket0	mc0-ch0	5	0x0		0x2	
socket0	mc0-ch0	6	0x0		0x2	
socket0	mc0-ch0	7	0x0		0x2	

8. BIOS serial log for the SMM flow after step 5. Reverse bank VLS and corrected memory error reporting flows:

```
[ProcessMcBanks] McBankBitField 2000
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0xB009409
0/13-1 RetryRdErrLogAddr1: 0x44040000
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[imc] memory error handler start!
PS Disabled :UpdateMcChanSsrState ch:0:SSRStopMode
McSsrState Node:0 :SSRStopMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
S1 C74 T1 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x0 SubRank:0x0
Bank:0x1 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, Rank 0, SubRank 0, Bank 1, BankGrp 0, Device 0.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 4, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N B N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 6, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
```

9. BIOS serial log for the SMM flow when reverse bank VLS has completed and rank VLS sparing begins:

10. BIOS serial log for the SMM flow when rank VLS sparing completes:


```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: SchedulePendingList()... Start
[CleanPreVlsEvent]: NodeId 0.
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:81
[SSR] restore and restart patrol scrub info
[SSR] enable C6
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode P, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 4, RMode B, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 6, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
[ADDDC]: Event type 2, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
```

10.2.2.4 Scenario 'c' - 1 Bank VLS then 2nd Strike to Failed Rank, Region not in VLS, Different Device, Different Bank

Test Objective

The objective of this test is to test scenario 'c' as described in [this section](#).

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores A step
BIOS	WW15'24 BKC
OS Version	n/a - Test executed and logs collected at UEFI shell
continued...	

Kernel	n/a - Test executed and logs collected at UEFI shell
HW Configuration	4 DIMMs per socket, 10x4 dual rank, 64 GB DIMMs on slots A1, A2, B1, and B2. A non-POR configuration was used to develop this recipe but POR configurations are expected to be used by the audience of this document.
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Establish 1 bank VLS region as described in [One Bank VLS Region](#) on page 259.
2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2" so inject two CECC errors to a different device and bank compared to the location/target in step 1.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=1, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=1, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

4. Check that the per rank CECC error threshold has been met.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
=====
| socket0 | mc0-ch0 | 0 | 0x2 | | 0x2 | Y |
```

socket0	mc0-ch0	1	0x0				0x2			
socket0	mc0-ch0	2	0x0				0x2			
socket0	mc0-ch0	3	0x0				0x2			
socket0	mc0-ch0	4	0x0				0x2			
socket0	mc0-ch0	5	0x0				0x2			
socket0	mc0-ch0	6	0x0				0x2			
socket0	mc0-ch0	7	0x0				0x2			

- Resume the system so BIOS can perform its SMM flow.

```
>>> go()
```

- Expecting VLS sparing operations to occur to create a 2nd bank VLS region so copy_in_progress should be 1.

```
>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.sparing_patrol_status.show()
0x00000000 : patrol_complete (02:02)
0x00000000 : copy_complete (01:01)
0x00000001 : copy_in_progress (00:00)
```

- Verify that the initial bank VLS region remains the same, there's a 2nd bank VLS region which aligns with expectations, and the per rank CECC error counter was cleared.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
| Register Dump |           | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
|-----|
| SVLS          | failed   | 0        | socket0 | mc0-ch0     | 0x0 | 0x0     | 0x0    | 0x0  | Bank    |
|-----|
| SVLS          | non-failed | 0        | socket0 | mc0-ch0     | 0x4 | 0x0     | 0x0    | 0x1  | Bank    |
|-----|
| SVLS          | ---      | 0        | socket0 | mc0-ch0     | Failed | Device  | =      | 0x0  | Rank0   |
|-----|
| SVLS          | failed   | 1        | socket0 | mc0-ch0     | 0x0 | 0x0     | 0x0    | 0x1  | Bank    |
|-----|
| SVLS          | non-failed | 1        | socket0 | mc0-ch0     | 0x4 | 0x0     | 0x0    | 0x0  | Bank    |
|-----|
| SVLS          | ---      | 1        | socket0 | mc0-ch0     | Failed | Device  | =      | 0x1  | Rank0   |
|-----|
| SVLS          | ---      | 2        | socket0 | mc0-ch0     | -    | -       | -      | -    | disabled |
|-----|
| SVLS          | ---      | 3        | socket0 | mc0-ch0     | -    | -       | -      | -    | disabled |
|-----|

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|
| socket0 | mc0-ch0     | 0    | 0x0   |                | 0x2       |                  |
|-----|
| socket0 | mc0-ch0     | 1    | 0x0   |                | 0x2       |                  |
|-----|
| socket0 | mc0-ch0     | 2    | 0x0   |                | 0x2       |                  |
|-----|
| socket0 | mc0-ch0     | 3    | 0x0   |                | 0x2       |                  |
|-----|
| socket0 | mc0-ch0     | 4    | 0x0   |                | 0x2       |                  |
|-----|
| socket0 | mc0-ch0     | 5    | 0x0   |                | 0x2       |                  |
|-----|
| socket0 | mc0-ch0     | 6    | 0x0   |                | 0x2       |                  |
|-----|
| socket0 | mc0-ch0     | 7    | 0x0   |                | 0x2       |                  |
|-----|
```

- BIOS serial log for the SMM flow after step 5. Setup bank VLS reigon and corrected memory error reporting flows:

```
[ProcessMcBanks] McBankBitFields 2000
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0xB009409
0/13-1 RetryRdErrLogAddr1: 0x44040001
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[imc] memory error handler start!
PS Disabled :UpdateMcChanSsrState ch:0:SSRStopMode
McSsrState Node:0 :SSRStopMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
S1 C80 T1 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x0 SubRank:0x0
Bank:0x1 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, Rank 0, SubRank 0, Bank 1, BankGrp 0, Device 1.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 4, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N B N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 6, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N

[ADDDC]NonRankVlsMode: Call HandleNonRankVlsStrike()...
[ADDDC]: HandleNonRankVlsStrike() start...
[ADDDC]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0, Bank 1, Device 1.
[ADDDC]: BankMode is 0.
[ADDDC]: No previous error in device 1.
[ADDDC]: Call HandleBankVlsEvent()...
[ADDDC]: HandleBankVlsEvent()... Start
[GetAvailableRegion]: NodeId 0, ChOnNode 0, Region 1, Status Success.
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0 Bank 1.
FindBankVlsPairRank(Node 0, Ch 0, Rank 0, RegionNum = 0) = Success
There is already BankVls between rank 0 and rank 4, so select rank 4 as buddy rank .
[GetNewNonFailedRankBank]: Rank 0 - Buddy Rank 4, Bank 1 - Buddy Bank 0.
AdddcEvent0:
Type 3, Region 1 (PRank 0 : PBank 1) - (BRank 4 : BBank 0), FailedDevice 1.
AdddcEvent1:
Type 3, Region 1 (PRank 0 : PBank 1) - (BRank 4 : BBank 0), FailedDevice 1.
[ADDDC]: HandleBankVlsEvent()... End
[ADDDC]: HandleNonRankVlsStrike() Status Success.
[ADDDC]: The event has been successfully handled!
[ADDDC]: Call SchedulePendingList()
DisableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:80
[SSR] disable C6 & GV
[SSR] stop and save patrol scrub info
[ADDDC]: SchedulePendingList()... Start
```

```
[ActionOnBankVls] Region 1, NodeId 0, Ch 0, FailedRank 0 - NonFailedRank 4, FailedBank 1 - NonFailedBank 0.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleNewAdddcEvent() return, Status = Success.
[ADDDC]: Main handler return, Status Success.
[GetRRLErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x0 SubRank:0x0 Bank:0x1
BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x0 SubRank:0x0
Bank:0x1 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 Ssr copy inprogress
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x0
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64018
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C9 T0 [CpuRas]CMCI is disabled, skip CMCI signal
...
```

9. BIOS serial log for the SMM flow after bank VLS sparing completes:

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: SchedulePendingList()... Start
[CleanPreVlsEvent]: NodeId 0.
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:81
[SSR] restore and restart patrol scrub info
[SSR] enable C6
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000003 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P P N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 4, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: B B N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 6, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
```

```
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
```

10.2.2.5 Scenario 'd' - 1 Bank VLS then 2nd Strike to Buddy Rank, Region in VLS

Test Objective

The objective of this test is to test scenario 'd' as described in [this section](#).

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores A step
BIOS	WW15'24 BKC
OS Version	n/a - Test executed and logs collected at UEFI shell
Kernel	n/a - Test executed and logs collected at UEFI shell
HW Configuration	4 DIMMs per socket, 10x4 dual rank, 64 GB DIMMs on slots A1, A2, B1, and B2. A non-POR configuration was used to develop this recipe but POR configurations are expected to be used by the audience of this document.
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Establish 1 bank VLS region as described in [One Bank VLS Region](#) on page 259.
2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2" so inject two CECC errors to the buddy rank and bank. In step 1, a bank VLS region was established with the buddy rank as chip-select 0x4 and bank 0x1. Chip-select decodes to {DIMM, rank, sub-channel}. Use these DIMM, rank, sub-channel, and bank values as the new error injection target.

```
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=1, rank=0, sub_channel=0, bank_group=0,
bank=1, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=1, rank=0, sub_channel=0, bank_group=0,
bank=1, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

4. Check that the per rank CECC error threshold has been met.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 4 | 0x2 | | 0x2 | Y
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
=====
```

5. Resume the system so BIOS can perform its SMM flow.

```
>>> go()
```

6. Reverse bank VLS and two bank VLS sparing flows are expected so copy_in_progress should be 1.

```
>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.sparing_patrol_status.show()
0x00000000 : patrol_complete (02:02)
0x00000000 : copy_complete (01:01)
0x00000001 : copy_in_progress (00:00)
```

7. Verify the 2 bank VLS regions. Note that the 1st failed bank has buddied with rank 0x6 which aligns with the expected buddy rank selection order and that the 2nd failed bank was buddied with the expected rank and next available bank according to the BIOS algorithm.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
```

Register Dump		Region #	Socket	iMC-Channel	CS	SubRank	BankGp	Bank	Region
SVLS	failed	0	socket0	mc0-ch0	0x0	0x0	0x0	0x0	Bank
SVLS	non-failed	0	socket0	mc0-ch0	0x6	0x0	0x0	0x1	Bank
SVLS	---	0	socket0	mc0-ch0	Failed	Device	=	0x0	Rank0
SVLS	failed	1	socket0	mc0-ch0	0x4	0x0	0x0	0x1	Bank
SVLS	non-failed	1	socket0	mc0-ch0	0x0	0x0	0x0	0x1	Bank
SVLS	---	1	socket0	mc0-ch0	Failed	Device	=	0x0	Rank4
SVLS	---	2	socket0	mc0-ch0	-	-	-	-	disabled
SVLS	---	3	socket0	mc0-ch0	-	-	-	-	disabled

***** Memory Correctable ECC Error Count *****

Socket	iMC-Channel	Rank	Count	Count Overflow	Threshold	Threshld Overflow
socket0	mc0-ch0	0	0x0		0x2	
socket0	mc0-ch0	1	0x0		0x2	
socket0	mc0-ch0	2	0x0		0x2	
socket0	mc0-ch0	3	0x0		0x2	
socket0	mc0-ch0	4	0x0		0x2	
socket0	mc0-ch0	5	0x0		0x2	
socket0	mc0-ch0	6	0x0		0x2	
socket0	mc0-ch0	7	0x0		0x2	

8. BIOS serial log for the SMM flow after step 5. Reverse bank VLS and corrected memory error reporting flows:

```
[ProcessMcBanks] McBankBitField 2000
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0x9009429
0/13-1 RetryRdErrLogAddr1: 0x44040100
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[imc] memory error handler start!
PS Disabled :UpdateMcChanSsrState ch:0:SSRStopMode
McSsrState Node:0 :SSRStopMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
S1 C21 T1 [GetChannelErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x4 SubRank:0x0
Bank:0x1 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, Rank 4, SubRank 0, Bank 1, BankGrp 0, Device 0.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
```



```

- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 4, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N B N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 6, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N

[ADDDC]NonRankVlsMode: Call HandleNonRankVlsStrike()...
[ADDDC]: HandleNonRankVlsStrike() start...
[ADDDC]: NodeId 0, ChOnNode 0, Rank 4, SubRank 0, Bank 1, Device 0.
[ADDDC]: BankMode is 2.
        FindBankVlsPairRank(Node 0, Ch 0, Rank 4, RegionNum = 0) = Success
[ADDDC]: BankVlsCnt = 1.
[ADDDC]: Try BANK VLS Reverse...
[ADDDC]: HandleBankVlsRevsEvent()... Start
        FindRegionWithRankBank(Node 0, Ch 0, Rank 4, Bank 1, RegionNum = 0) = Success
[FillRegionStruct]: Region 0, PRank 0, PBank 0, BRank 4, BBank 1, FailedDevice 0.
        AdddcEvent0:
                Type 5, Region 0 (PRank 0 : PBank 0) - (BRank 4 : BBank 1), FailedDevice 0.
[ADDDC]: Trying Bank VLS for the Primary Bank
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0 Bank 0.
        FindBankVlsPairRank(Node 0, Ch 0, Rank 0, RegionNum = 0) = Success
                BuddyRank 4 is not healthy. Skip it.
        FindBankVlsPairRank(Node 0, Ch 0, Rank 6, RegionNum = 255) = Not Found
                Check buddy rank 6.
                The buddy rank 6 is valid.
[GetNewNonFailedRankBank]: Rank 0 - Buddy Rank 6, Bank 0 - Buddy Bank 1.
        AdddcEvent1:
                Type 3, Region 0 (PRank 0 : PBank 0) - (BRank 6 : BBank 1), FailedDevice 0.
[ADDDC]: Trying Bank VLS for the buddy bank (current failed bank)
[GetAvailableRegion]: NodeId 0, ChOnNode 0, Region 1, Status Success.
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 4, SubRank 0 Bank 1.
        FindBankVlsPairRank(Node 0, Ch 0, Rank 4, RegionNum = 0) = Success
                There is already BankVls between rank 4 and rank 0, so select rank 0 as buddy rank .
[GetNewNonFailedRankBank]: Rank 4 - Buddy Rank 0, Bank 1 - Buddy Bank 1.
        AdddcEvent2:
                Type 3, Region 1 (PRank 4 : PBank 1) - (BRank 0 : BBank 1), FailedDevice 0.
        AdddcEvent3:
                Type 3, Region 1 (PRank 4 : PBank 1) - (BRank 0 : BBank 1), FailedDevice 0.
[ADDDC]: HandleBankVlsRevsEvent()... End
[ADDDC]: HandleNonRankVlsStrike() Status Success.
[ADDDC]: The event has been successfully handled!
[ADDDC]: Call SchedulePendingList()
DisableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:80
[SSR] disable C6 & GV
[SSR] stop and save patrol scrub info
[ADDDC]: SchedulePendingList()... Start
        [ActionOnBankVlsReverse] Region 0, NodeId 0, Ch 0, PRank 0 - BRank 4, PBank 0 - BBank 1.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: Event type 5, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleNewAdddcEvent() return, Status = Success.
[ADDDC]: Main handler return, Status Success.
[GetRRLErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x4 SubRank:0x0 Bank:0x1
BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
PS Disabled :UpdateMcChanSsrState ch1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x4 SubRank:0x0
Bank:0x1 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 Ssr copy inprogress
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x0
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64018
[Mca] Clear Status

```

```
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C68 T1 [CpuRas]CMCI is disabled, skip CMCI signal
...
```

9. BIOS serial log for the SMM flow after reverse bank VLS sparing completes. Bank VLS sparing begins again to rebuddy the 1st failed bank:

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: SchedulePendingList()... Start
[ActionOnBankVls] Region 0, NodeId 0, Ch 0, FailedRank 0 - NonFailedRank 6, FailedBank 0 - NonFailedBank 1.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
```

10. BIOS serial log for the SMM flow after bank VLS sparing completes. Bank VLS sparing begins for the 2nd failed bank:

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: SchedulePendingList()... Start
[ActionOnBankVls] Region 1, NodeId 0, Ch 0, FailedRank 4 - NonFailedRank 0, FailedBank 1 - NonFailedBank 1.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
```

11. BIOS serial log for the SMM flow after bank VLS sparing completes:

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: SchedulePendingList()... Start
[CleanPreVlsEvent]: NodeId 0.
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:81
[SSR] restore and restart patrol scrub info
[SSR] enable C6
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P B N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 4, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N P N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
```

```

- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 6, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N B N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N

[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!

```

10.2.2.6 Scenario 'e' - 1 Bank VLS then 2nd Strike to Buddy Rank, Region not in VLS

Test Objective

The objective of this test is to test scenario 'e' as described in [this section](#).

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores A step
BIOS	WW15'24 BKC
OS Version	n/a - Test executed and logs collected at UEFI shell
Kernel	n/a - Test executed and logs collected at UEFI shell
HW Configuration	4 DIMMs per socket, 10x4 dual rank, 64 GB DIMMs on slots A1, A2, B1, and B2. A non-POR configuration was used to develop this recipe but POR configurations are expected to be used by the audience of this document.
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled

- ADDDC Sparing = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Establish 1 bank VLS region as described in [One Bank VLS Region](#) on page 259.
2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2" so inject two CECC errors to the buddy rank and bank. In step 1, a bank VLS region was established with the buddy rank as chip-select 0x4 and bank 0x1. Chip-select decodes to {DIMM, rank, sub-channel}. Use these DIMM, rank, and sub-channel values as the new error injection target BUT use a bank other than 0x1.

```
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=1, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=1, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

4. Check that the per rank CECC error threshold has been met.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 4 | 0x2 | | 0x2 | Y
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
=====
```

5. Resume the system so BIOS can perform its SMM flow.

```
>>> go()
```

6. Expecting bank VLS sparing to occur so copy_in_progress should be 1.

```
>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.sparing_patrol_status.show()
0x00000000 : patrol_complete (02:02)
0x00000000 : copy_complete (01:01)
0x00000001 : copy_in_progress (00:00)
```

7. Verify that the initial bank VLS region remains the same, the per rank CECC error counter was cleared, and the 2nd bank VLS region was established.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
| Register Dump |          | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
=====
|   SVLS       | failed   |    0     | socket0 | mc0-ch0     | 0x0 | 0x0     | 0x0    | 0x0  | Bank   |
=====
|   SVLS       | non-failed |    0     | socket0 | mc0-ch0     | 0x4 | 0x0     | 0x0    | 0x1  | Bank   |
=====
|   SVLS       | ---      |    0     | socket0 | mc0-ch0     | Failed | Device  | =     | 0x0  | Rank0  |
=====
|   SVLS       | failed   |    1     | socket0 | mc0-ch0     | 0x4 | 0x0     | 0x0    | 0x0  | Bank   |
=====
|   SVLS       | non-failed |    1     | socket0 | mc0-ch0     | 0x0 | 0x0     | 0x0    | 0x1  | Bank   |
=====
|   SVLS       | ---      |    1     | socket0 | mc0-ch0     | Failed | Device  | =     | 0x0  | Rank4  |
=====
|   SVLS       | ---      |    2     | socket0 | mc0-ch0     | -     | -       | -     | -    | disabled |
=====
|   SVLS       | ---      |    3     | socket0 | mc0-ch0     | -     | -       | -     | -    | disabled |
=====

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
=====
| socket0 | mc0-ch0    | 0    | 0x0   |                | 0x2       |                  |
=====
| socket0 | mc0-ch0    | 1    | 0x0   |                | 0x2       |                  |
=====
| socket0 | mc0-ch0    | 2    | 0x0   |                | 0x2       |                  |
=====
| socket0 | mc0-ch0    | 3    | 0x0   |                | 0x2       |                  |
=====
| socket0 | mc0-ch0    | 4    | 0x0   |                | 0x2       |                  |
=====
| socket0 | mc0-ch0    | 5    | 0x0   |                | 0x2       |                  |
=====
| socket0 | mc0-ch0    | 6    | 0x0   |                | 0x2       |                  |
=====
| socket0 | mc0-ch0    | 7    | 0x0   |                | 0x2       |                  |
=====
```

8. BIOS serial log for the SMM flow after step 5. Setup bank VLS region and corrected memory error reporting flows:

```
[ProcessMcBanks] McBankBitField 2000
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0xB009409
0/13-1 RetryRdErrLogAddr1: 0x44000100
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[imc] memory error handler start!
PS Disabled :UpdateMcChanSsrState ch:0:SSRStopMode
McSsrState Node:0 :SSRStopMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
S1 C24 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x4 SubRank:0x0
Bank:0x0 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, Rank 4, SubRank 0, Bank 0, BankGrp 0, Device 0.
*****
```

```
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 4, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N B N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 6, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N

[ADDDC]NonRankVlsMode: Call HandleNonRankVlsStrike()...
[ADDDC]: HandleNonRankVlsStrike() start...
[ADDDC]: NodeId 0, ChOnNode 0, Rank 4, SubRank 0, Bank 0, Device 0.
[ADDDC]: BankMode is 0.
[ADDDC]: No previous error in device 0.
[ADDDC]: Call HandleBankVlsEvent()...
[ADDDC]: HandleBankVlsEvent()... Start
[GetAvailableRegion]: NodeId 0, ChOnNode 0, Region 1, Status Success.
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 4, SubRank 0 Bank 0.
FindBankVlsPairRank(Node 0, Ch 0, Rank 4, RegionNum = 0) = Success
There is already BankVls between rank 4 and rank 0, so select rank 0 as buddy rank .
[GetNewNonFailedRankBank]: Rank 4 - Buddy Rank 0, Bank 0 - Buddy Bank 1.
AdddcEvent0:
Type 3, Region 1 (PRank 4 : PBank 0) - (BRank 0 : BBank 1), FailedDevice 0.
AdddcEvent1:
Type 3, Region 1 (PRank 4 : PBank 0) - (BRank 0 : BBank 1), FailedDevice 0.
[ADDDC]: HandleBankVlsEvent()... End
[ADDDC]: HandleNonRankVlsStrike() Status Success.
[ADDDC]: The event has been successfully handled!
[ADDDC]: Call SchedulePendingList()
DisableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:80
[SSR] disable C6 & GV
[SSR] stop and save patrol scrub info
[ADDDC]: SchedulePendingList()... Start
[ActionOnBankVls] Region 1, NodeId 0, Ch 0, FailedRank 4 - NonFailedRank 0, FailedBank 0 - NonFailedBank 1.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleNewAdddcEvent() return, Status = Success.
[ADDDC]: Main handler return, Status Success.
[GetRRLErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x4 SubRank:0x0 Bank:0x0
BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x4 SubRank:0x0
Bank:0x0 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 Ssr copy inprogress
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x0
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64018
[Mca] Clear Status
[Mca]ClearErrLogReg
```

```
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C77 T0 [CpuRas]CMCI is disabled, skip CMCI signal
...
```

9. BIOS serial log for the SMM flow after bank VLS sparing completes:

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 I0 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: SchedulePendingList()... Start
[CleanPreVlsEvent]: NodeId 0.
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:81
[SSR] restore and restart patrol scrub info
[SSR] enable C6
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P B N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 4, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P B N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 6, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
```

10.2.2.7 Scenario 'f' - 1 Rank VLS then 2nd Strike to Failed Rank, Any Bank, Non-failed Device

Test Objective

The objective of this test is to test scenario 'f' as described in [this section](#).

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores A step
BIOS	WW15'24 BKC
OS Version	n/a - Test executed and logs collected at UEFI shell
Kernel	n/a - Test executed and logs collected at UEFI shell
HW Configuration	4 DIMMs per socket, 10x4 dual rank, 64 GB DIMMs on slots A1, A2, B1, and B2. A non-POR configuration was used to develop this recipe but POR configurations are expected to be used by the audience of this document.
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Establish 1 rank VLS region as described in [Scenario 'b' - 1 Bank VLS then 2nd Strike to Failed Rank, Region not in VLS, Same Device, Different Bank](#) on page 269.
2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2" so inject two CECC errors to the failed rank, any bank, and any device other than the failed device. The failed rank and device are 0x0 and 0x0.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=0, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```


4. Check that the per rank CECC error threshold has been met.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
=====
| socket0 | mc0-ch0 | 0 | 0x2 | | 0x2 | Y |
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
=====
```

5. Resume the system so BIOS can perform its SMM flow.

```
>>> go()
```

6. No ADDDC action is expected to be taken by BIOS so copy_in_progress should be 0.

```
>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.sparing_patrol_status.show()
0x00000000 : patrol_complete (02:02)
0x00000000 : copy_complete (01:01)
0x00000000 : copy_in_progress (00:00)
```

7. Note that the existing VLS regions are unchanged and the CECC error counters are cleared.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
| Register Dump | | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
=====
| SVLS | failed | 0 | socket0 | mc0-ch0 | 0x0 | 0x0 | 0x0 | 0x0 | Rank |
| SVLS | non-failed | 0 | socket0 | mc0-ch0 | 0x4 | 0x0 | 0x0 | 0x1 | Rank |
| SVLS | --- | 0 | socket0 | mc0-ch0 | Failed | Device | = | 0x0 | Rank0 |
| SVLS | --- | 1 | socket0 | mc0-ch0 | - | - | - | - | disabled |
| SVLS | --- | 2 | socket0 | mc0-ch0 | - | - | - | - | disabled |
| SVLS | --- | 3 | socket0 | mc0-ch0 | - | - | - | - | disabled |
=====

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
=====
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 4 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
=====
```

	socket0		mc0-ch0		7		0x0				0x2			
=====		=====		=====		=====		=====		=====		=====		=====

10.2.2.8 Scenario 'g' - 1 Rank VLS then 2nd Strike to Buddy Rank, Any Device, Any Bank

Test Objective

The objective of this test is to test scenario 'g' as described in [this section](#).

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700/6500-Series Processors with P-Cores A step
BIOS	WW15'24 BKC
OS Version	n/a - Test executed and logs collected at UEFI shell
Kernel	n/a - Test executed and logs collected at UEFI shell
HW Configuration	4 DIMMs per socket, 10x4 dual rank, 64 GB DIMMs on slots A1, A2, B1, and B2. A non-POR configuration was used to develop this recipe but POR configurations are expected to be used by the audience of this document.
CScripts Version	BHS.2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used.

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log > Memory Error Enabling >
 - Spare Interrupt = SMI
- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 2
 - Trigger SW Error Threshold= Disabled
 - ADDDC Sparing = Enable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used to create this recipe. It is recommended to have a POR memory population.

Recipe

1. Establish 1 rank VLS region as described in [Scenario 'b' - 1 Bank VLS then 2nd Strike to Failed Rank, Region not in VLS, Same Device, Different Bank](#) on page 269.
2. Halt the system for error injections.

```
>>> halt()
```

3. The hardware per rank CECC error thresholds were set to "2" so inject two CECC errors to the buddy rank, any device, and any bank. In step 1, a bank VLS region was established with the buddy rank as chip-select 0x4. Chip-select decodes to {DIMM, rank, sub-channel}. Use these DIMM, rank, and sub-channel values as the new error injection target.

```
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=1, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
>>> ei.mem.memDevs(dev0=0, dev0msk=0x3, dev1=1, dev1msk=0x0)
>>> ei.mem.injectMemError(socket=0, channel=0, dimm=1, rank=0, sub_channel=0, bank_group=0,
bank=0, errType="ce", verbose=True, sub_rank=0, ps_ch=0, showErrorRegs=True,
checkEInjLock=False)
```

4. Check that the per rank CECC error threshold has been met.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
|-----|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 0 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 1 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 2 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 3 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 4 | 0x2 | | 0x2 | Y
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 5 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 6 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | 7 | 0x0 | | 0x2 | |
|-----|-----|-----|-----|-----|-----|
=====
```

5. Resume the system so BIOS can perform its SMM flow.

```
>>> go()
```

6. Reverse rank VLS, establishing a another rank VLS region (rebuddy), and a new bank VLS region for the failed bank on the buddy are expected so copy_in_progress should be 1. It took more than 7 minutes for all the sparing copy flows to complete when collecting the logs for this recipe.

```
>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.sparing_patrol_status.show()
0x00000000 : patrol_complete (02:02)
0x00000000 : copy_complete (01:01)
0x00000001 : copy_in_progress (00:00)
```

7. Verify the expected rank VLS region for the failed device on rank 0x0 and the bank VLS region for the failed bank on rank 0x4.

```
>>> ras.mem.adddc_status_check(socket=0, mc=0)
...
=====
| Register Dump |          | Region # | Socket | iMC-Channel | CS | SubRank | BankGp | Bank | Region |
=====
| SVLS         | failed  | 0        | socket0 | mc0-ch0     | 0x0 | 0x0     | 0x0    | 0x0  | Rank   |
| SVLS         | non-failed | 0        | socket0 | mc0-ch0     | 0x6 | 0x0     | 0x0    | 0x1  | Rank   |
| SVLS         | ---     | 0        | socket0 | mc0-ch0     | Failed | Device  | =      | 0x0  | Rank0  |
| SVLS         | failed  | 1        | socket0 | mc0-ch0     | 0x4 | 0x0     | 0x0    | 0x0  | Bank   |
| SVLS         | non-failed | 1        | socket0 | mc0-ch0     | 0x2 | 0x0     | 0x0    | 0x1  | Bank   |
| SVLS         | ---     | 1        | socket0 | mc0-ch0     | Failed | Device  | =      | 0x0  | Rank4  |
| SVLS         | ---     | 2        | socket0 | mc0-ch0     | -    | -       | -      | -    | disabled |
| SVLS         | ---     | 3        | socket0 | mc0-ch0     | -    | -       | -      | -    | disabled |
=====

***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld Overflow |
=====
| socket0 | mc0-ch0    | 0    | 0x0   |                | 0x2       |                  |
| socket0 | mc0-ch0    | 1    | 0x0   |                | 0x2       |                  |
| socket0 | mc0-ch0    | 2    | 0x0   |                | 0x2       |                  |
| socket0 | mc0-ch0    | 3    | 0x0   |                | 0x2       |                  |
| socket0 | mc0-ch0    | 4    | 0x0   |                | 0x2       |                  |
| socket0 | mc0-ch0    | 5    | 0x0   |                | 0x2       |                  |
| socket0 | mc0-ch0    | 6    | 0x0   |                | 0x2       |                  |
| socket0 | mc0-ch0    | 7    | 0x0   |                | 0x2       |                  |
=====
```

8. BIOS serial log for the SMM flow after step 5. Reverse rank VLS and corrected memory error reporting flows:

```
[ProcessMcBanks] McBankBitFields 2000
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/13/0 McStatusReg: 0x8C00004200800090
0/13/0 McMiscReg: 0x0
0/13-0 RetryRdErrLog: 0xD000
0/13-0 RetryRdErrLogAddr1: 0x0
0/13-0 RetryRdErrLogAddr2: 0x0
0/13-1 RetryRdErrLog: 0x9009429
0/13-1 RetryRdErrLogAddr1: 0x44000100
0/13-1 RetryRdErrLogAddr2: 0xF0000100
0/13-2 RetryRdErrLog: 0x0
0/13-2 RetryRdErrLogAddr1: 0x0
0/13-2 RetryRdErrLogAddr2: 0x0
0/13-3 RetryRdErrLog: 0x0
0/13-3 RetryRdErrLogAddr1: 0x0
0/13-3 RetryRdErrLogAddr2: 0x0
[imc] memory error handler start!
PS Disabled :UpdateMcChanSsrState ch:0:SSRStopMode
McSsrState Node:0 :SSRStopMode
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
SwErrorThreshold is not enabled
S1 C76 T1 [GetChannelErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0Rank:0x4 SubRank:0x0
Bank:0x0 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
*****
[GetFailedDeviceInfo]: Node: 0, Ch 0, Rank 4, SubRank 0, Bank 0, BankGrp 0, Device 0.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode P, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
```

```
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 4, RMode B, DevSts 0x00000000 -
- SubRank 00:
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 6, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank      : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N

[ADDDC]RankVlsBuddyMode: Call HandleRankVlsRevsEvent()...
[ADDDC]: HandleRankVlsRevsEvent()... Start
FindRegionWithRank(Node 0, Ch 0, Rank 4, RegionNum = 0) = Success
[FillRegionStruct]: Region 0, PRank 0, PBank 0, BRank 4, BBank 1, FailedDevice 0.
AdddcEvent0:
Type 4, Region 0 (PRank 0 : PBank 0) - (BRank 4 : BBank 1), FailedDevice 0.
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0 Bank 0.
FindBankVlsPairRank(Node 0, Ch 0, Rank 0, RegionNum = 0) = Success
BuddyRank 4 is not healthy. Skip it.
FindBankVlsPairRank(Node 0, Ch 0, Rank 6, RegionNum = 255) = Not Found
Check buddy rank 6.
The buddy rank 6 is valid.
[GetNewNonFailedRankBank]: Rank 0 - Buddy Rank 6, Bank 0 - Buddy Bank 1.
AdddcEvent1:
Type 2, Region 0 (PRank 0 : PBank 0) - (BRank 6 : BBank 1), FailedDevice 0.
[GetAvailableRegion]: NodeId 0, ChOnNode 0, Region 1, Status Success.
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 4, SubRank 0 Bank 0.
FindBankVlsPairRank(Node 0, Ch 0, Rank 4, RegionNum = 0) = Success
FindBankVlsPairRank(Node 0, Ch 0, Rank 0, RegionNum = 0) = Success
Check buddy rank 0.
FindBankVlsPairRank(Node 0, Ch 0, Rank 2, RegionNum = 255) = Not Found
Check buddy rank 2.
The buddy rank 2 is valid.
[GetNewNonFailedRankBank]: Rank 4 - Buddy Rank 2, Bank 0 - Buddy Bank 1.
AdddcEvent2:
Type 3, Region 1 (PRank 4 : PBank 0) - (BRank 2 : BBank 1), FailedDevice 0.
AdddcEvent3:
Type 3, Region 1 (PRank 4 : PBank 0) - (BRank 2 : BBank 1), FailedDevice 0.
[ADDDC]: HandleRankVlsRevsEvent()... End
[ADDDC]: The event has been successfully handled!
[ADDDC]: Call SchedulePendingList()
DisableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:80
[SSR] disable C6 & GV
[SSR] stop and save patrol scrub info
[ADDDC]: SchedulePendingList()... Start
[ActionOnRankVlsReverse] Region 0, NodeId 0, Ch 0, PRank 0 - BRank 4.
VlsSparingCopy: Rank AdddcSparingMR
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: Event type 4, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleNewAdddcEvent() return, Status = Success.
[ADDDC]: Main handler return, Status Success.
[GetRRLErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x4 SubRank:0x0 Bank:0x0
BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x5. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0 Rank:0x4 SubRank:0x0
Bank:0x0 BankGroup:0x0 Dev:0x0 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 Ssr copy inprogress
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x0
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64018
[Mca] Clear Status
[Mca]ClearErrLogReg
```

```
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C2 T1 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C78 T0 [CpuRas]CMCI is disabled, skip CMCI signal
...
```

9. BIOS serial log for the SMM flow after reverse rank VLS sparing completes. Rank VLS sparing begins again to rebuddy the 1st failed rank:

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: SchedulePendingList()... Start
[ActionOnRankVls] Region 0, NodeId 0, Ch 0, PRank 0 - BRank 6.
VlsSparingCopy: Rank AdddcSparingMR
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: Event type 2, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
```

10. BIOS serial log for the SMM flow after rank VLS sparing completes. Bank VLS sparing begins for the 2nd failed bank:

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: SchedulePendingList()... Start
[ActionOnBankVls] Region 1, NodeId 0, Ch 0, FailedRank 4 - NonFailedRank 2, FailedBank 0 - NonFailedBank 1.
VlsSparingCopy: Bank AdddcSparingMR
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!
```

11. BIOS serial log for the SMM flow after bank VLS sparing completes:

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:SpareCopyCPLMode
McSsrState Node:0 :SpareCopyCPLMode
[ADDDC]: Main handler start...
[ADDDC]: HandleInProgressAdddcEvent() Start...
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DF64E18
[ADDDC]: SchedulePendingList()... Start
[CleanPreVlsEvent]: NodeId 0.
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:81
[SSR] restore and restart patrol scrub info
[SSR] enable C6
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode P, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 2, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N B N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 3, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 4, RMode N, DevSts 0x00000001 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: P N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 5, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
```

```

- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 6, RMode B, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N
- Rank 7, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N N

[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleInProgressAdddcEvent() Status Success...
[ADDDC]: Main handler return, Status Success.
PS Disabled :UpdateMcChanSsrState ch:1:SSRStopMode
McSsrState Node:1 :SSRStopMode
[imc] memory error handler end!

```

10.3 Address Range/Partial Memory Mirroring

Memory mirroring is a method of keeping a duplicate (secondary or mirrored) copy of the contents of memory as a redundant backup for use if the primary memory fails. The feature is implemented in the hardware/software and with the option for the end user to enable the system operation in mirror mode. Total available memory is reduced in the mirror mode as all the memory at the slave is now used as duplicate memory. This affects the overall system performance, but it is a known compromise for end user that is more concerned about data availability. Mirroring may be configured for a portion of the address space (address range mirroring/partial mirroring). Mirroring is handled by the M2Mem (Mesh 2 Memory) logic in the IMC (Integrated Memory Controller).

10.3.1 Uncorrectable Error in a Mirrored Region

Test Objective

Inject an uncorrectable error into a specified address located within the mirrored address region of memory, Read the target cache-line, then verify that the system stays operation and B2CMI's MCA bank logs a correct error and MIRRORCORR=1 is set in IA32_MCI_MISC register.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	A1
BIOS	WW05'24 BKC
OS Version	RHEL9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	4 DIMMs per socket
CScripts Version	686308 GNR 2406.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [Enhanced MCA Logging \(ELOG\)](#) on page 28

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - UEFI ARM Mirror = Enable
 - ARM Mirror Percentage = 10
 - Patrol Scrub = Enable at End of Post
 - Mirror TAD0 = Enable
- EDKII > Platform Configuration > System Event Log >
 - RAS Log Level = MID (BASIC_FLOW, FUNC_FLOW)
- EDKII > Platform Configuration > Miscellaneous Configuration >
 - Serial Debug Message Level > Normal

Recipe

1. Halt the system and verify that mirroring is enabled by reading mode.mirror

```
>>> halt()
>>> sv.sockets.compute0.uncore.memss.b2cmis.mode.mirror
socket0.compute0.uncore.memss.b2cmi0.mode.mirror - 0x00000001
socket0.compute0.uncore.memss.b2cmi1.mode.mirror - 0x00000001
socket1.compute0.uncore.memss.b2cmi0.mode.mirror - 0x00000001
socket1.compute0.uncore.memss.b2cmi1.mode.mirror - 0x00000001
```

2. Check the memory configuration to verify that the system is in mirror mode. Directory mode should be disabled and ECC should be enabled. Verify channel mode has P/P or ParMirr which indicates address range memory mirroring.

```
>>> mc.dimminfoNoSPD()
=====
| Socket 0 | Socket 1 | System |
=====
Active Memory | 61.0GB | 63.0GB | 124.0GB |
Expected Memory | NotAvail | NotAvail | NotAvail |
Active Channels | 0/0 | 0/0 | |
Clustering Mode | Hemi | Hemi | Hemi |
DDR Frequency | 4800MHz | 4800MHz | 4800MHz |
Max DDR Frequency | 6400MHz | 6400MHz | |
0/0 CL-RCD-RP-CMD | 20-20-21-1n | 20-20-21-1n | |
1/0 CL-RCD-RP-CMD | 20-20-21-1n | 20-20-21-1n | |
ECC Checking / PFD | On / On | On / On | On / On |
ECC Mode | 128b | 128b | 128b |
CAP/WrCRC | On / Disable | On / Disable | On / Disable |
Patrol/DemandScrub | On / On | On / On | On/On |
Memory Mode | 1LM | 1LM | 1LM |
Channel Mode | P/P | P/P | ParMirr |
Refresh Mode | AllBankNormal | AllBankNormal | AllBankNormal |
RFM | Off | Off | Off |
ZQCal/DQSOSC | On / On | On / On | On/On |
Mirror Size | 3.0GB | 0.0GB | 0GB |
```


ADDDC	Disabled	Disabled	Disabled
Encryption	Disabled	Disabled	Disabled
Paging Policy	AdaptOpen	AdaptOpen	AdaptOpen
Directory Mode	Off	Off	Off
DDR Scrambling	Enabled	Enabled	Enabled
CKE Throttling	PPD	PPD	PPD
Thermal Throttling	CLTT	CLTT	CLTT
NUMA			On

- Use `mc.addTran()` to print all valid TAD array entries to find the mirrored address range.

```
>>> mc.addTran()
```

- Use the following command to translate how a few address within the specified system address range map to its physical DIMM address. Take note of the memory controller ("mc") column. From this table, system address can be selected to test injecting errors into mirrored regions on different memory controllers.

```
>>> ras.mem.sa2da_table(0x100000000, 0x100000400, 64)
Section doesn't exist: Validate address
Halting to read memory content
```

Figure 14. Command Table

[GPC core group] Halt on 48 devices -- 16:04:52.832360 2023-06-29															
System Address	Socket	mc	Channel	Sub channel	DIMM	Rank	Sub rank	Bank group	Bank	Row	Column	Mem type	Rank Address	Sub-Channel address	
0x100000000	0	0	0	0	0	0	0	4	0	4096	0	ddr5	536870912	1073741824	
0x100000040	0	0	0	0	0	0	0	4	0	4096	16	ddr5	536870976	1073741888	
0x100000080	0	0	0	1	0	0	0	4	0	4096	0	ddr5	536870912	1073741824	
0x1000000c0	0	0	0	1	0	0	0	4	0	4096	16	ddr5	536870976	1073741888	
0x100000100	0	1	0	0	0	0	0	4	0	4096	0	ddr5	536870912	1073741824	
0x100000140	0	1	0	0	0	0	0	4	0	4096	16	ddr5	536870976	1073741888	
0x100000180	0	1	0	1	0	0	0	4	0	4096	0	ddr5	536870912	1073741824	
0x1000001c0	0	1	0	1	0	0	0	4	0	4096	16	ddr5	536870976	1073741888	
0x100000200	0	0	0	0	0	0	0	4	2	4096	0	ddr5	536870912	1073741952	
0x100000240	0	0	0	0	0	0	0	4	2	4096	16	ddr5	536870976	1073742016	
0x100000280	0	0	0	1	0	0	0	4	2	4096	0	ddr5	536870912	1073741952	
0x1000002c0	0	0	0	1	0	0	0	4	2	4096	16	ddr5	536870976	1073742016	
0x100000300	0	1	0	0	0	0	0	4	2	4096	0	ddr5	536870912	1073741952	
0x100000340	0	1	0	0	0	0	0	4	2	4096	16	ddr5	536870976	1073742016	
0x100000380	0	1	0	1	0	0	0	4	2	4096	0	ddr5	536870912	1073741952	
0x1000003c0	0	1	0	1	0	0	0	4	2	4096	16	ddr5	536870976	1073742016	

- Inject an uncorrectable error to an address within the mirrored address region.

```
>>> ei.utils.resetInjectorLockCheck()
>>> ei.mem.memDevs(dev0msk=0x3, dev1msk=0x3)
>>> ei.mem.injectMemError(0x100000300, errType="uce")
```

- Verify the error was corrected. Check `B2CMI_STATUS` to verify a corrected error was logged. Alternatively, you can read the `MCI_STATUS_SHADOW` register.

```
>>> sv.socket0.compute0.uncore.memss.b2cmi1.mci_status_shadow
0x8c00004001010090
>>> sv.socket0.compute0.uncore.memss.b2cmi1.mci_status_shadow.show()
0x00000001 : valid (63:63) --
0x00000000 : over (62:62) --
0x00000000 : uc (61:61) --
0x00000000 : en (60:60) --
0x00000001 : miscv (59:59) --
0x00000001 : addrv (58:58) --
0x00000000 : pcc (57:57) --
```

```
0x00000000 : s (56:56) --
0x00000000 : ar (55:55) --
0x00000000 : corrstatus (54:53) --
0x00000001 : corrcount (52:38) --
0x00000000 : otherinfo (37:32) --
0x00000001 : mscodddrtype (25:24) --
0x00000001 : mscod (23:16) --
0x00000090 : mcacod (15:00) --
```

7. Verify in the B2CMI_MISC register that the error was corrected during mirroring and mirror scrub was successful. Alternatively you can read the MCI_MISC_SHADOW register.

```
>>> sv.socket0.compute0.uncore.memss.b2cmil.mci_misc_shadow
0x4800101000206086
>>> sv.socket0.compute0.uncore.memss.b2cmil.mci_misc_shadow.show()
0x00000000 : mirrorfailover (63:63) -- Mirror Failover Error (MirrorFailover)
0x00000001 : mirrorcorrerr (62:62) -- Mirror Channel Corrected Error
0x00000000 : mccmdmirrfo (61:61) -- MC command was to an already permanently failed
0x00000000 : mccmdmirrsec (60:60) -- Error on secondary mirror channel (McCmdMirrSec)
0x00000001 : mccmdmirrregion (59:59) -- MC command was to a mirror region
0x00000000 : resvd_unused0 (58:57) -- Reserved for future use.
0x00000000 : mccmdchnl (56:55) -- Memory channel (McCmdChnl): Physical
0x00000000 : mccmdmemregion (54:51) -- Memory region type (McCmdMemRegion)
0x00000000 : mccmdopcode (50:45) -- Memory Controller Command Opcode (McCmdOpcode)
0x00000001 : mccmdvld (44:44) -- Memory Controller Command Valid (McCmdVld)
0x00000001 : bankindex (43:36) -- Bank Index (BankIndex) BankIndex of M2mem
0x00000000 : smiopcode (35:32) -- SMI Opcode (SmiOpcode): Decoding in case
0x00000000 : resvd_unused1 (31:30) -- Reserved for future use
0x00000001 : trkid (29:21) -- Mesh2mem Tracker Identifier (TrkId): bits[7:6] = physical
0x0000000c : errortype (20:11) -- Error type attribute (ErrorType): Error type
0x00000000 : resvd_unused2 (10:10) -- Reserved for future use
0x00000000 : hbmregion (09:09) -- 1: HBM 0: DDR4 Used in conjunction with
0x00000002 : addrmode (08:06) -- Address Mode (AddrMode): Address mode for the
0x00000006 : addrlsb (05:00) -- Address LSB (AddrLsb): Recoverable address LSB
```

8. Resume the system to allow all error handling flows to complete.

```
>>> go()
```

9. FTrace Output. Verify that an ELog record was generated indicated by extlog_mem_event.

```
cd /sys/kernel/debug/tracing
echo 1 > events/mce/mce_record/enable
echo 1 > events/ras/enable
cat trace_pipe
<...>-2240 [000] ..... 489.760413: mce_record: CPU: 0, MCGc/s: f000c20/0, MC12:
8c00004001010090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000100000300/4800103007006086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
e754f381ld, PROCESSOR: 0:a06d0, TIME: 1708029149, SOCKET: 0, APIC: 0
kworker/0:3-2240 [000] ..... 489.777042: extlog_mem_event: {1} corrected error:
single-bit ECC physical addr: 0000000100000300 (mask lsb: ff) node:0 card:3 module:0 rank:0
bank:0 row:8192 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnD, DIMM0
kworker/0:3-2240 [000] ..... 489.783482: mc_event: 1 Corrected error: memory read
error on CPU_SrcID#0_MC#3_Chan#0_DIMM#0 (mc:3 location:0:0:-1 address:0x100000300 grain:32
syndrome:0x00000000_err_code:0x0101:0x0090 SystemAddress:0x100000300 ProcessorSocketId:0x0
MemoryControllerId:0x3 ChannelAddress:0x40000000 ChannelId:0x0 RankAddress:0x10000000
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x2000 Column:0x0 Bank:0x0 BankGroup:0x0
ChipSelect:0x0)
```

10. Serial Log Output

```
[ECS] CheckAndHandleECSEvent Enter
[ECS] Timestamp - skt:0 mc:0 ch:0 duration: 40 minutes
[ECS] Timestamp - skt:0 mc:1 ch:0 duration: 40 minutes
[ECS] CheckAndHandleECSEvent Exit
[MirrorFailoverHandler] BIOS assisted mirroring
```

```
[MirrorFailoverHandler] mci status shadow; N0:0x1010090; N1:0x8C000040
[MirrorFailoverHandler] BIOS assisted mirroring;MirrorFailover:0
[ECS] CheckAndHandleECSEvent Enter
[ECS] Timestamp - skt:1 mc:0 ch:0 duration: 40 minutes
[ECS] Timestamp - skt:1 mc:1 ch:0 duration: 40 minutes
[ECS] CheckAndHandleECSEvent Exit
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]ProcessMcBankError: Scope Uncore
S0 C0 T0 [Mca]EmcaReadMcBankBitField: Scope Uncore, BitField 0x1000
S0 C0 T0 [Mca]McBankBitField: 0x1000
S0 C0 T0 [Mca]EmcaClearMcBankBitField: Scope Uncore, BitField 0x1000
S0 C0 T0 [Mca]McBankErrorHandler: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x2
S0 C0 T0 [Mca]McBankErrorHandler: IoMcaErrorHandler Skt = 0x0, McBank = 0xC, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
S0 C0 T0 [Mca]MapErrorSeverity: MCE class is 0x5
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x4
S0 C0 T0 [Mca]McBankErrorHandler: Section Type = EMCA_SECTYPE_PLATMEM
FillPlatformMemoryErrorData Start!
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
FillPlatformMemoryErrorData: BankType:0x100
Error Source is M2M or IMC (MCCHAN)BankType:0x100
MatchMcBankMcaCode Start!
MatchMcBankMcaCode End!
FillPlatformMemoryErrorData End!
S0 C0 T0 [Mca]McBankErrorHandler: Report platform memory error log
ReportPlatMemErrorLog Entry
S0 C0 T0 CreatePlatformMemoryErrorRecord Entry,McBank: 12,Socket: 0,Severity: 2
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
...
```

10.3.2 Persistent Uncorrectable Error in a Mirrored Region

Test Objective

Inject an uncorrectable error into a specified address located within the mirrored address region of memory, read the target cache-line, then verify that the system stays operation and B2CMI's MCA bank logs a correct error and MIRRORCORR=1 is set in IA32_MCI_MISC register.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	A1
BIOS	WW15 BKC
OS Version	CentOS Stream 8
<i>continued...</i>	

Kernel	5.19.0-gnr.po.bkc.3.13.14.x86_64
HW Configuration	4 DIMMs per socket
CScripts Version	GNR2314.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMSG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [Enhanced MCA Logging \(ELOG\)](#) on page 28

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - UEFI ARM Mirror = Enable
 - ARM Mirror Percentage = 10
 - Patrol Scrub = Enable at End of Post
 - Mirror TAD0 = Enable
- EDKII > Platform Configuration > System Event Log >
 - RAS Log Level = MID (BASIC_FLOW, FUNC_FLOW)
- EDKII > Platform Configuration > Miscellaneous Configuration >
 - Serial Debug Message Level > Normal

Recipe

1. Halt the system and verify that mirroring is enabled by reading mode.mirror.

```
>>> halt()
>>> sv.sockets.compute0.uncore.memss.b2cmis.mode.mirror
socket0.compute0.uncore.memss.b2cmi0.mode.mirror - 0x00000001
socket0.compute0.uncore.memss.b2cmi1.mode.mirror - 0x00000001
socket1.compute0.uncore.memss.b2cmi0.mode.mirror - 0x00000001
socket1.compute0.uncore.memss.b2cmi1.mode.mirror - 0x00000001
```

2. Check the memory configuration to verify that the system is in mirror mode. Directory mode should be disabled and ECC should be enabled. Verify channel mode has P/P or ParMirr which indicates address range memory mirroring.

```
>>> mc.dimminfoNoSPD()
=====
| Socket 0 | Socket 1 | System |
=====
Active Memory | 61.0GB | 63.0GB | 124.0GB |
Expected Memory | NotAvail | NotAvail | NotAvail |
Active Channels | 0/0 | 0/0 | 0/0 |
Clustering Mode | Hemi | Hemi | Hemi |
DDR Frequency | 4800MHz | 4800MHz | 4800MHz |
Max DDR Frequency | 6400MHz | 6400MHz | 6400MHz |
0/0 CL-RCD-RP-CMD | 20-20-21-1n | 20-20-21-1n | 20-20-21-1n |
1/0 CL-RCD-RP-CMD | 20-20-21-1n | 20-20-21-1n | 20-20-21-1n |
ECC Checking / PFD | On / On | On / On | On / On |
ECC Mode | 128b | 128b | 128b |
=====
```

CAP/WrCRC	On / Disable	On / Disable	On / Disable
Patrol/DemandScrub	On / On	On / On	On/On
Memory Mode	1LM	1LM	1LM
Channel Mode	P/P	P/P	ParMirr
Refresh Mode	AllBankNormal	AllBankNormal	AllBankNormal
RFM	Off	Off	Off
ZQCal/DQSOSC	On / On	On / On	On/On
Mirror Size	3.0GB	0.0GB	0GB
ADDDC	Disabled	Disabled	Disabled
Encryption	Disabled	Disabled	Disabled
Paging Policy	AdaptOpen	AdaptOpen	AdaptOpen
Directory Mode	Off	Off	Off
DDR Scrambling	Enabled	Enabled	Enabled
CKE Throttling	PPD	PPD	PPD
Thermal Throttling	CLTT	CLTT	CLTT
NUMA			On

- Use `mc.addTran()` to print all valid TAD array entries to find the mirrored address range.

```
>>> mc.addTran()
```

- Use the following command to translate how a few address within the specified system address range map to its physical DIMM address. Take note of the memory controller ("mc") column. From this table, system address can be selected to test injecting errors into mirrored regions on different memory controllers.

```
>>> ras.mem.sa2da_table(0x100000000, 0x100000400, 64)
Section doesn't exist: Validate address
Halting to read memory content
```

Figure 15. Command Table

[GPC core group] Halt on 4B devices -- 16:04:52.832360 2023-06-29														
System Address	Socket	mc	Channel	Sub channel	DIMM	Rank	Sub rank	Bank group	Bank	Row	Column	Mem type	Rank Address	Sub-Channel address
0x100000000	0	0	0	0	0	0	0	4	0	4096	0	ddr5	536870912	1073741824
0x100000000	0	0	0	0	0	0	0	4	0	4096	16	ddr5	536870976	1073741888
0x100000000	0	0	0	1	0	0	0	4	0	4096	0	ddr5	536870912	1073741824
0x100000000	0	0	0	1	0	0	0	4	0	4096	16	ddr5	536870976	1073741888
0x100000100	0	1	0	0	0	0	0	4	0	4096	0	ddr5	536870912	1073741824
0x100000140	0	1	0	0	0	0	0	4	0	4096	16	ddr5	536870976	1073741888
0x100000180	0	1	0	1	0	0	0	4	0	4096	0	ddr5	536870912	1073741824
0x1000001c0	0	1	0	1	0	0	0	4	0	4096	16	ddr5	536870976	1073741888
0x100000200	0	0	0	0	0	0	0	4	2	4096	0	ddr5	536870912	1073741952
0x100000240	0	0	0	0	0	0	0	4	2	4096	16	ddr5	536870976	1073742016
0x100000280	0	0	0	1	0	0	0	4	2	4096	0	ddr5	536870912	1073741952
0x1000002c0	0	0	0	1	0	0	0	4	2	4096	16	ddr5	536870976	1073742016
0x100000300	0	1	0	0	0	0	0	4	2	4096	0	ddr5	536870912	1073741952
0x100000340	0	1	0	0	0	0	0	4	2	4096	16	ddr5	536870976	1073742016
0x100000380	0	1	0	1	0	0	0	4	2	4096	0	ddr5	536870912	1073741952
0x1000003c0	0	1	0	1	0	0	0	4	2	4096	16	ddr5	536870976	1073742016

- Inject a persistent uncorrectable error to an address within the mirrored address region.

```
>>> ei.utils.resetInjectorLockCheck()
>>> ei.mem.memDevs(dev0msk=0x3, dev1msk=0x3)
>>> ei.mem.injectMemError(0x100000300, oneInjection=False, errType="uce")
```

- Verify the error was corrected. Check `B2CMI_STATUS` to verify a corrected error was logged. Alternatively, you can read the `MCI_STATUS_SHADOW` register.

```
>>> sv.socket0.compute0.uncore.memss.b2cmil.mci_status_shadow
0x8c00000001010090
>>> sv.socket0.compute0.uncore.memss.b2cmil.mci_status_shadow.show
0x00000001 : valid (63:63) --
0x00000000 : over (62:62) --
0x00000000 : uc (61:61) --
0x00000000 : en (60:60) --
0x00000001 : miscv (59:59) --
0x00000001 : addrv (58:58) --
0x00000000 : pcc (57:57) --
0x00000000 : s (56:56) --
0x00000000 : ar (55:55) --
0x00000000 : corrstatus (54:53) --
0x00000000 : corrcount (52:38) --
0x00000000 : otherinfo (37:32) --
0x00000001 : mscodddrtype (25:24) --
0x00000001 : mscod (23:16) --
0x00000090 : mcacod (15:00) --
```

7. Verify in the B2CMI_MISC register that the error was corrected during mirroring and mirror scrub was successful. Alternatively you can read the MCI_MISC_SHADOW register.

```
>>> sv.socket0.compute0.uncore.memss.b2cmil.mci_misc_shadow
0x880010101d60a086
>>> sv.socket0.compute0.uncore.memss.b2cmil.mci_misc_shadow.show()
0x00000001 : mirrorfailover (63:63) --
0x00000000 : mirrorcorrerr (62:62) --
0x00000000 : mccmdmirrfo (61:61) --
0x00000000 : mccmdmirrsec (60:60) --
0x00000001 : mccmdmirrregion (59:59) --
0x00000000 : resvd_unused0 (58:57) --
0x00000000 : mccmdchnl (56:55) --
0x00000000 : mccmdmemregion (54:51) --
0x00000000 : mccmdopcode (50:45) --
0x00000001 : mccmdvld (44:44) --
0x00000001 : bankindex (43:36) --
0x00000000 : smiopcode (35:32) --
0x00000000 : resvd_unused1 (31:30) --
0x000000eb : trkid (29:21) --
0x00000014 : errortype (20:11) --
0x00000000 : resvd_unused2 (10:10) --
0x00000000 : hbmregion (09:09) --
0x00000002 : addrmode (08:06) --
0x00000006 : addr1sb (05:00) --
```

8. A DDR channel failure should be logged and the failed channel should be identified for the mirror failover. Bits [1:0] correspond to channels 1 - 0. In this case, bit 0 of the ddr4chnlfailed is set. Meaning that channel 0 of memory controller 0 failed.

```
>>> ei.utils.clearMemInjectors(socket=0)
>>> go()
>>> sv.socket0.compute0.uncore.memss.b2cmil.mirrorfailover.show()
0x00000000 : ddr4frcmirrwrstfailed (08:08) --
0x00000000 : buddychnlfailed (01:01) --
0x00000001 : ddr4chnlfailed (00:00) --
```

9. FTrace Output. Verify an ELog record was generated indicated by extlog_mem_event.

```
cd /sys/kernel/debug/tracing
echo 1 > events/mce/mce_record/enable
echo 1 > events/ras/enable
cat trace_pipe
<...>-14872 [000] ..... 1094.056011: mce_record: CPU: 0, MCGc/s: f000c20/0, MC12:
8c00004001010090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000100000300/8800103016e0a086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
1ba5f477507, PROCESSOR: 0:a06d0, TIME: 1708030937, SOCKET: 0, APIC: 0
```

```

kworker/0:0-14872 [000] .N... 1094.060742: extlog_mem_event: {1} corrected error:
mirror Broken physical addr: 0000000100000300 (mask lsb: ff) node:0 card:3 module:0 rank:0
bank:0 row:8192 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnD, DIMM0
kworker/0:0-14872 [000] ..... 1094.077183: mc_event: 1 Corrected error: memory read
error on CPU_SrcID#0_MC#3_Chcn#0_DIMM#0 (mc:3 location:0:0:-1 address:0x100000300 grain:32
syndrome:0x00000000 err_code:0x0101:0x0090 SystemAddress:0x100000300 ProcessorSocketId:0x0
MemoryControllerId:0x3 ChannelAddress:0x40000000 ChannelId:0x0 RankAddress:0x10000000
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x2000 Column:0x0 Bank:0x0 BankGroup:0x0
ChipSelect:0x0)

```

10. Serial Log Output:

```

[ECS] CheckAndHandleECSEvent Enter
[ECS] Timestamp - skt:0 mc:0 ch:0 duration: 7 minutes
[ECS] Timestamp - skt:0 mc:1 ch:0 duration: 7 minutes
[ECS] CheckAndHandleECSEvent Exit
[MirrorFailoverHandler] BIOS assisted mirroring
[MirrorFailoverHandler] mci status shadow; N0:0x1010090; N1:0x8C000000
[MirrorFailoverHandler] BIOS assisted mirroring;MirrorFailover:1
[Mirror] Mirror Failover on DDR4
[SSR] SSR state Save! Flag:0
[SSR] stop and save patrol scrub info
[Mirror] Mirror Failover mailbox call successful
S1 C10 TO RETRY_RD_ERR_LOG_MCDDC_DP_REG is not valid on Skt 0 Sktch 1 PseudoCh 0

Forward Address Translation start for 0x100000300
[GetMcAndChId] RtNum:2 RtInterleaveShift:7 InterleaveWays:2 RtShiftAddress:3 RtIndex:1
McTargetId is 0x1
MemType is MemTypeDdr
0x100000300 belongs to DDR5
TranslatedAddress->SocketId is 0x0
TranslatedAddress->MemoryControllerId is 0x1
TranslatedAddress->ChannelId is 0x0
[ForwardDlmmDecodeGetCa] TAD skt:0 Mc:1 Chn:0 TadIndex:1
[MirrorPriCa] mirror support, insert BIT18[0] to CA, CA 0x80000100
Sub Channel address:40000080
Pseudo Channel address:20000000
rank :0
rank address:20000000
[ForwardAmap] DDR5 X4 FGBG_INT2
RankAddress:0x20000000 ChipSelect:0x0 PhysicalRankId:0x0 DimmSlot:0x0
Col:0x0 Row:0x1000 BankGroup:0x4 Bank:0x2

[TranslateMirror] Starting translate mirror
Sub Channel address:40020080
Pseudo Channel address:20010000
rank :0
rank address:20010000
[ForwardAmap] DDR5 X4 FGBG_INT2
[TranslateMirror] Sec RankAddress:0x20010000 ChipSelect:0x0 PhysicalRankId:0x0
DimmSlot:0x0
[TranslateMirror] Sec Col:0x0 Row:0x1000 BankGroup:0x6 Bank:0x2

Forward Address Translation Successful!!
ReportMemErrorLog Entry
CreateMemoryErrorRecord Entry
MemRecord.Header SignatureStart: 0x52455043
MemRecord.Header Revision: 0x300
MemRecord.Header SignatureEnd: 0xFFFFFFFF
MemRecord.Header SectionCount: 0x1
MemRecord.Header ErrorSeverity: 0x2
MemRecord.Header ValidationBits: 0x0
MemRecord.Header RecordLength: 0x118
MemRecord.Header TimeStamp: 0x0
MemRecord.Header PlatformID: 0x00000000-0000-0000-0000-000000000000
MemRecord.Header CreatorID: 0x00000000-0000-0000-0000-000000000000
MemRecord.Header NotificationType: 0x2DCE8BB1-BDD7-450E-B9AD-9CF4EBD4F890
MemRecord.Header RecordID: 0x0
MemRecord.Header PersistenceInfo: 0x0

```

```

ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AB0CE10
LogErrorToWhea BIOS ID: BHSDCRB1.86B.0024.D04.2306101807
[WheaLog]LogErrorToWhea++, CodeType: 2
[WheaLog] Erasing all status blocks
[WheaLog] Erasing all status blocks
[WheaLog]RestartAllWheaElog: Success

```

10.4 Full Memory Mirroring

Memory mirroring is a method of keeping a duplicate (secondary or mirrored) copy of the contents of memory as a redundant backup for use if the primary memory fails. The feature is implemented in the hardware/software and with the option for the end user to enable the system operation in mirror mode. Total available memory is reduced in the mirror mode as all the memory at the secondary is now used as duplicate memory. This affects the overall system performance, but it is a known compromise for end user that is more concerned about data availability. In full memory mirroring, half of all installed memory is reserved for redundancy and not reported in total system memory size. Mirroring is handled by the B2CMI logic in the IMC.

10.4.1 Correctable Error on Primary Channel

Test Objective

The objective of this test is the correctable ECC error flow on the primary channel for a mirrored region. Particularly, to point out how the flow is not any different compared to a non-mirrored region.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW41'23 BKC
OS Version	n/a - test was done at UEFI shell
Kernel	n/a - test was done at UEFI shell
HW Configuration	4 DIMMs per socket, 10x4 single rank, 32 GB DIMMs on slots A1, B1, C1, and D1 (Recommended to use 1 or 2 DPC because they are the only POR prime configurations which support mirroring)
CScripts Version	GNR2334.6000

Preconditions and Setup

NOTE

This test was executed at UEFI shell. A future revision will have this test ran when OS is fully booted.

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Mirror Mode = Full Mirror Mode
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used. It is highly recommended to have a POR memory population.

Recipe

1. Halt and confirm there are no preexisting memory related errors on the system. Clear any found error logs or cold boot the system.

```
>>> halt()
>>> error.check_mem_errors()
```

2. Verify that mirroring is enabled by reading the following bit-field.

```
>>> sv.socket0.soc.memss.b2cmi0.mode.mirror
0x1
```

3. Check that the memory configuration aligns with what is expected. Full memory mirroring, therefore, active memory capacity is equal to half the installed capacity; all channels are operating in mirror mode and the mirror size is equal to the active memory size. With the current CScripts version there were issues seen with the reported mirror size values but everything else aligns with expectations.

```
>>> mc.dimminfoNoSPD()
...
=====
| Socket 0 | Socket 1 | System |
=====
Active Memory | 64.0GB | 64.0GB | 128.0GB |
Expected Memory | NotAvail | NotAvail | NotAvail |
Clustering Mode | Hex | Hex | Hex |
1LM Memory Size | 64.00GB | 64.00GB | 128.00GB |
2LM Memory Size | 0.00 B | 0.00 B | 0.00 B |
AppDirect Memory Size | 0.00 B | 0.00 B | 0.00 B |
DDR Frequency | 4800MHz | 4800MHz | 4800MHz |
Max DDR Frequency | 6400MHz | 6400MHz | |
MC0 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
MC1 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
MC2 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
MC3 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
MC0 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
MC1 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
MC2 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
MC3 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
ECC Checking / PFD | On / On | On / On | On / On |
ECC Mode | 128b | 128b | 128b |
CAP/WrCRC | On / Disable | On / Disable | On / Disable |
Patrol/DemandScrub | On / On | On / On | On/On |
Memory Mode | 1LM | 1LM | 1LM |
Channel Mode | Mirror | Mirror | Mirror |
Refresh Mode | AllBankNormal | AllBankNormal | AllBankNormal |
RFM | Off | Off | Off |
```

ZQCal/DQSOSC/RxRt	On / On / Off	On / On / Off	On / On / Off
Mirror Size	64.0GB	64.0GB	0GB
ADDDC	Disabled	Disabled	Disabled
Encryption	Disabled	Disabled	Disabled
Paging Policy	AdaptOpen	AdaptOpen	AdaptOpen
Directory Mode	Off	Off	Off
DDR Scrambling	Enabled	Enabled	Enabled
CKE Throttling	PPD	PPD	PPD
Thermal Throttling	CLTT	CLTT	CLTT
NUMA			On

4. Print the valid TAD arrays to check that they show they are mirrored.

```
>>> mc.addTran()
...
TAD @ Socket0 Mc0

TAD[ S0 ][ MC0 ][ 0 ]: Mirrored
Range      : 0.00 B up to 2.00 GB (0x0 - 0x7fff_ffff)
Span       : 2.00 GB
Ch Offset   : 0x0 (0.00 )
target_ways : 0 (1-way) PA[12] - 4KB granularity
chan_ways   : 2 (4-way) PA[8] - 256B granularity
ddr5_Ch0 Range : 0x0 (0.00 ) - 0x3fff_ffff (1024.00 M)
ddr5 Sub-Ch Range : 0x0 (0.00 ) - 0x1fff_ffff (512.00 M)

TAD[ S0 ][ MC0 ][ 1 ]: Mirrored
Range      : 2.00 GB up to 66.00 GB (0x8000_0000 - 0x10_7fff_ffff)
Span       : 64.00 GB
Ch Offset   : 0x8000_0000 (2.00 G)
target_ways : 0 (1-way) PA[12] - 4KB granularity
chan_ways   : 2 (4-way) PA[8] - 256B granularity
ddr5_Ch0 Range : 0x0 (0.00 ) - 0x7_ffff_ffff (32.00 G)
ddr5 Sub-Ch Range : 0x0 (0.00 ) - 0x3_ffff_ffff (16.00 G)
```

5. Use the following command to translate how a few addresses, within the specified system address range, map to its physical DIMM address. Take note of the memory controller ("mc") column. From this table, system addresses can be selected to test injecting errors into mirrored regions on different memory controllers.

```
>>> ras.mem.sa2da_table(0x100000000, 0x100000400, 64)
=====
| System Address | Socket | mc | Channel | Sub channel | DIMM | Rank | Sub rank | Bank group | Bank | Row | Column | Mem type | Rank Address | Sub-Channel address |
=====
| 0x100000000 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x100000040 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000080 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x1000000c0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000100 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x100000140 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000180 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x1000001c0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000200 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x100000240 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000280 | 0 | 0 | 2 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x1000002c0 | 0 | 0 | 2 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000300 | 0 | 0 | 3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x100000340 | 0 | 0 | 3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000380 | 0 | 0 | 3 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x1000003c0 | 0 | 0 | 3 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
=====
```

6. Check if the injectors are locked for all sockets and unlock if locked.

```
>>> ei.utils.checkInjectorLock(and_unlock=True)
```

- Choose the target device for error injection and modify masks for a single device correctable ECC error injection.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=2, dev1msk=0x0)
dev0 = 0x1, dev0msk = 0x3, dev1 = 2, dev1msk = 0x0.
```

- Inject and consume the correctable ECC error.

```
>>> ei.mem.injectMemError(addr=0x000100000000, errType="ce", verbose=True)
```

- Check that the target's IMC machine check bank has a correctable read error logged. The CScripts version used has some issues with its MC bank decoding features and manual decoding may be necessary if more information is desired. Checking that MSCOD == 0x80 is the key detail here. Note that the logged address is incorrect due to a sighting that will be fixed in a future stepping.

```
>>> error.check_mem_errors(socket_id=0)
Uncore MCA from Socket: 0
=====
| Valid | MC Bank Name | Status | Addr | Misc | Ctl | Ctl2 |
=====
| * | BANK #13 - MC CH0 merged | 0x8c00004200800090 | 0x0000000180000000 | 0x04000a0080000086 | 0x00003c0ffffe5ff | 0x0000000000000000 |
=====
Bank name: MCHN, Error = CORR Transient or correctable Error for Demand or Underfill Reads
...

>>> error.manualMcaDecode()

Manual Machine Check Bank Decoder
Enter MCI_STATUS, MCI_ADDR, MCI_MISC values
Enter 't' for a short demo
Enter 'Q' or 'q' to exit

Enter MCA Bank Number [0,1,2,3,4,5,6,MSE,MCHN,B2CMI,CBO,LLC,UPI] or q for quit: MCHN
Enter MCI_STATUS [0x0]: 0x8c00004200800090
Enter MCI_ADDR [0x0]: 0x0000000180000000
Enter MCI_MISC [0x0]: 0x04000a0080000086

Bank name: MCHN, Error = CORR Transient or correctable Error for Demand or Underfill Reads

+ MCMCHN_STATUS 0x8c00004200800090

Range Field Description Value Value Description
-----
63:63 VALID Indicates that the information within this register is valid 0x1 Valid
62:62 OVERFLOW A '1' indicates that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (the VALID bit was already set) 0x0 Not Overflow
61:61 UC A '1' indicates that an uncorrectable the error condition was detected. 0x0 No UC
60:60 EN Error reporting is enabled. A '1' indicates that error reporting is enabled by the associated flag bit(s) in the IA32_MCI_CTL register. 0x0 Not Enabled
59:59 MISCV If this flag is '1', the IA32_MCI_MISC register content is valid and can be read. 0x1 IA32_MCI_MISC Valid
58:58 ADDRv If this flag is '1', the IA32_MCI_ADDR register content is valid and can be read. 0x1 IA32_MCI_ADDR Valid
57:57 PCC Processor Context Corrupt: A '1' indicates that reliable restarting of the processor may not be possible and a reset is recommended. 0x0 Processor Context Not Corrupted
56:56 S Signaling an uncorrectable recoverable error. 0x0 ---
55:55 AR Recovery action required for UCR error. 0x0 ---
54:53 THRESHOLD_BASED_STATUS Threshold-based error status 0x0 ---
52:38 CORRECTED_ERROR_COUNT Count of correctable errors in the DCU including sticky overflow indicator (bit 52) 0x1 ---
37:37 FW FW Update Error Status Indicator 0x0 ---
36:32 OTHER_INFO Not part of Machine-Check Architecture 0x2 ---
31:16 MSCOD Model Specific Error Code 0x80 Transient or correctable Error for Demand or Underfill Reads
15:0 MCACOD Machine Check Architecture Error Code 0x90 MCHAN_ERROR(1IM MODE)|MEMORY_READ_ERROR|CHANNEL_0
```

- Resume the system to allow all error handling flows to complete.

```
>>> go()
```

- For reference if needed, here is the BIOS serial log:

```
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
```

```

S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
TranslateSad: McTargetId is 0x0
Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Rank:0x0 SubRank:0x0
Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x0 Row:0x2000 SingleBit:0x0 Transfer:0x0
PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AC27810
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T1 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C81 T0 [CpuRas]CMCI is disabled, skip CMCI signal
...
S0 C130 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C82 T1 [CpuRas]CMCI is disabled, skip CMCI signal

```

10.4.2 UCE on Primary Channel

Test Objective

The objective of this test is the uncorrectable ECC error flow on the primary channel for a mirrored region. Expected outcome is for B2CMI to request the same data from the secondary channel to correct the error and scrub the primary channel with good data.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW41'23 BKC
OS Version	CentOS* Stream 8
Kernel	5.19.0-gnr.po.bkc.3.13.14.x86_64
HW Configuration	4 DIMMs per socket, 10x4 single rank, 32 GB DIMMs on slots A1, B1, C1, and D1 (Recommended to use 1 or 2 DPC because they are the only POR prime configurations which support mirroring)
CScripts Version	GNR2334.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Socket Configuration > Memory Configuration > Memory RAS Configuration >

- Mirror Mode = Full Mirror Mode
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used. It is highly recommended to have a POR memory population.

Recipe

1. Complete steps 1-6 in the [CE recipe](#) to check on the state of the system, find an address to inject to, and to unlock the injectors.
2. Choose the target device for error injection and modify masks for a multi-device uncorrectable ECC error injection.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=2, dev1msk=0x3)
dev0 = 0x1, dev0msk = 0x3, dev1 = 2, dev1msk = 0x3.
```

3. Inject and consume the uncorrectable ECC error.

```
>>> ei.mem.injectMemError(addr=0x000100000000, errType="uce", verbose=True)
```

4. Check the B2CMI MC bank to verify that the injection was successful. For easier decoding, read the B2CMI shadow registers. For MCI_STATUS check that all the bit-fields are the same as shown next. MCI_ADDR should align with the injection address in the previous step. For MCI_MISC, the bit-fields of interest which should be logged as shown next are: mirrorcorrerr (mirror corrected the error and mirror scrub was successful), mccmdmirrregion (transaction was to a mirrored region), and bankindex (value should align with the memory controller of the injection target).

```
>> sv.socket0.soc.memss.b2cmi0.mci_status_shadow
0x8c00004001010090

>> sv.socket0.soc.memss.b2cmi0.mci_addr_shadow
0x100000000

>> sv.socket0.soc.memss.b2cmi0.mci_misc_shadow
0x4800100023806086

>>> sv.socket0.soc.memss.b2cmi0.mci_status_shadow.show()
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000001 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : corrstatus (54:53)
0x00000001 : corrcount (52:38)
0x00000000 : otherinfo (37:32)
0x00000001 : mscodddrtype (25:24)
0x00000001 : mscod (23:16)
0x00000090 : mcacod (15:00)

>>> sv.socket0.soc.memss.b2cmi0.mci_misc_shadow.show()
0x00000000 : mirrorfailover (63:63)
0x00000001 : mirrorcorrerr (62:62)
0x00000000 : mccmdmirrfo (61:61)
0x00000000 : mccmdmirrsec (60:60)
0x00000001 : mccmdmirrregion (59:59)
0x00000000 : resvd_unused0 (58:57)
```

```
0x00000000 : mccmdchnl (56:55)
0x00000000 : mccmdmemregion (54:51)
0x00000000 : mccmdopcode (50:45)
0x00000001 : mccmdvld (44:44)
0x00000000 : bankindex (43:36)
0x00000000 : smiopcode (35:32)
0x00000000 : resvd_unused1 (31:30)
0x0000011c : trkid (29:21)
0x0000000c : errorrtyp (20:11)
0x00000000 : resvd_unused2 (10:10)
0x00000000 : hbmregion (09:09)
0x00000002 : addrmode (08:06)
0x00000006 : addr1sb (05:00)
```

5. Resume the system to allow all error handling flows to complete.

```
>>> go()
```

6. For reference if needed, here is the BIOS serial log:

```
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED

Forward Address Translation start for 0x100000000
[GetMcAndChId] RtNum:2 RtInterleaveShift:7 InterleaveWays:4 RtShiftAddress:0 RtIndex:0
McTargetId is 0x0
MemType is MemTypellmDdr
0x100000000 belongs to DDR5
TranslatedAddress->SocketId is 0x0
TranslatedAddress->MemoryControllerId is 0x0
TranslatedAddress->ChannelId is 0x0
[ForwardDlmmDecodeGetCa] TAD skt:0 Mc:0 Chn:0 TadIndex:1
[MirrorPriCa] mirror support, insert BIT18[0] to CA, CA 0x40000000
    Sub Channel address:20000000
    Pseudo Channel address:10000000
    rank :0
    rank address:10000000
[ForwardAmap] DDR5 X4 FGBG_INT2
    RankAddress:0x10000000 ChipSelect:0x0 PhysicalRankId:0x0 DimmSlot:0x0
    Col:0x0 Row:0x2000 BankGroup:0x0 Bank:0x0

[TranslateMirror] Starting translate mirror
    Sub Channel address:20020000
    Pseudo Channel address:10010000
    rank :0
    rank address:10010000
[ForwardAmap] DDR5 X4 FGBG_INT2
[TranslateMirror] Sec RankAddress:0x10010000 ChipSelect:0x0 PhysicalRankId:0x0
DimmSlot:0x0
[TranslateMirror] Sec Col:0x0 Row:0x2000 BankGroup:0x2 Bank:0x0

Forward Address Translation Successful!!
RETRY_RD_ERR_LOG_MCDDC_DP_REG is not valid
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
System is in 128bit ECC mode with Mirror Mode enabled
Entering CheckNearMemoryError.
S0 C0 T0 RETRY_RD_ERR_LOG_MCDDC_DP_REG is not valid on Skt 0 Sktch 0 PseudoCh 0
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AD54810
```

```

WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xC, Socket 0x0 Core 0x0
S0 C0 T1 [CpuRas]Successfully notify OS via CMCI
S0 C131 T1 [CpuRas]Successfully notify OS via CMCI
...
S0 C1 T0 [CpuRas]Successfully notify OS via CMCI
S0 C5 T0 [CpuRas]Successfully notify OS via CMCI

```

10.4.3 Persistent UCE on Primary Channel

Test Objective

The objective of this test mirror failover by injecting a persistent UCECC error on the primary channel. Expected outcome is for B2CMI to request the same data from the secondary channel to correct the error and scrub the primary channel with good data BUT mirror scrub fails and the primary channel fails over to the secondary channel.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW41'23 BKC
OS Version	CentOS* Stream 8
Kernel	5.19.0-gnr.po.bkc.3.13.14.x86_64
HW Configuration	4 DIMMs per socket, 10x4 single rank, 32 GB DIMMs on slots A1, B1, C1, and D1 (Recommended to use 1 or 2 DPC because they are the only POR prime configurations which support mirroring)
CScripts Version	GNR2334.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Mirror Mode = Full Mirror Mode
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used. It is highly recommended to have a POR memory population.

Recipe

1. Complete steps 1-6 in the [CE recipe](#) to check on the state of the system, find an address to inject to, and to unlock the injectors.
2. Choose the target device for error injection and modify masks for a multi-device uncorrectable ECC error injection.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=2, dev1msk=0x3)
dev0 = 0x1, dev0msk = 0x3, dev1 = 2, dev1msk = 0x3.
```

3. Inject a persistent uncorrectable ECC error and consume the error.

```
>>> ei.mem.injectMemError(addr=0x000100000000, errType="uce", oneInjection=False, verbose=True)
```

4. Check the target B2CMI MC bank to verify that the injection was successful. For easier decoding, read the B2CMI shadow registers. For MCI_STATUS check that all the bit-fields are the same as shown next. MCI_ADDR should align with the injection address in the previous step. For MCI_MISC, the bit-fields of interest which should be logged as shown next are: mirrorfailover (mirror scrub failed and failover must occur), mccmdmirrregion (transaction was to a mirrored region), and bankindex (value should align with the memory controller of the injection target).

```
>>> sv.socket0.soc.memss.b2cmi0.mci_status_shadow
0x8c0000001010090

>>> sv.socket0.soc.memss.b2cmi0.mci_addr_shadow
0x100000000

>>> sv.socket0.soc.memss.b2cmi0.mci_misc_shadow
0x880010000480a086

>>> sv.socket0.soc.memss.b2cmi0.mci_status_shadow.show()
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000001 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : corrstatus (54:53)
0x00000000 : corrcount (52:38)
0x00000000 : otherinfo (37:32)
0x00000001 : mscodddrtype (25:24)
0x00000001 : mscod (23:16)
0x00000090 : mcacod (15:00)

>>> sv.socket0.soc.memss.b2cmi0.mci_misc_shadow.show()
0x00000001 : mirrorfailover (63:63)
0x00000000 : mirrorcorrerr (62:62)
0x00000000 : mccmdmirrfo (61:61)
0x00000000 : mccmdmirrsec (60:60)
0x00000001 : mccmdmirrregion (59:59)
0x00000000 : resvd_unused0 (58:57)
0x00000000 : mccmdchnl (56:55)
0x00000000 : mccmdmemregion (54:51)
0x00000000 : mccmdopcode (50:45)
0x00000001 : mccmdvld (44:44)
0x00000000 : bankindex (43:36)
0x00000000 : smiopcode (35:32)
0x00000000 : resvd_unused1 (31:30)
0x00000024 : trkid (29:21)
0x00000014 : errortype (20:11)
0x00000000 : resvd_unused2 (10:10)
```



```
0x00000000 : hbmregion (09:09)
0x00000002 : addrmode (08:06)
0x00000006 : addr1sb (05:00)
```

5. Since the system is halted and mirror failover has not been configured by BIOS, ddr4chnlfailed is 0.

```
>>> sv.socket0.soc.memss.b2cmi0.mirrorfailover.show()
0x00000000 : ddr4frcmirrwrstfailed (08:08)
0x00000000 : buddychnlfailed (01:01)
0x00000000 : ddr4chnlfailed (00:00)
```

6. Resume the system to allow all error handling flows to complete.

```
>>> go()
```

7. After go(), MIRRORFAILOVER.ddr4chnlfailed on the primary channel should be "1" to indicate that the channel failed over. Also check that MIRRORFAILOVER on the secondary channel shows the primary channel has failed over to the secondary channel; buddychnlfailed should be "1".

```
>>> sv.socket0.soc.memss.b2cmi0.mirrorfailover.show()
0x00000000 : ddr4frcmirrwrstfailed (08:08)
0x00000000 : buddychnlfailed (01:01)
0x00000001 : ddr4chnlfailed (00:00)
```

```
>>> sv.socket0.soc.memss.b2cmi1.mirrorfailover.show()
0x00000000 : ddr4frcmirrwrstfailed (08:08)
0x00000001 : buddychnlfailed (01:01)
0x00000000 : ddr4chnlfailed (00:00)
```

8. For reference if needed, here is the BIOS serial log:

```
[Mirror] Mirror Failover on DDR4
[SSR] SSR state Save! Flag:0
[SSR] stop and save patrol scrub info
[Mirror] Mirror Failover mailbox call successful
S1 C9 T0 RETRY_RD_ERR_LOG_MCDDC_DP_REG is not valid on Skt 0 Sktch 0 PseudoCh 0

Forward Address Translation start for 0x100000000
[GetMcAndChId] RtNum:2 RtInterleaveShift:7 InterleaveWays:4 RtShiftAddress:0 RtIndex:0
McTargetId is 0x0
MemType is MemTypeLlmDdr
0x100000000 belongs to DDR5
TranslatedAddress->SocketId is 0x0
TranslatedAddress->MemoryControllerId is 0x0
TranslatedAddress->ChannelId is 0x0
[ForwardLlmDecodeGetCa] TAD skt:0 Mc:0 Chn:0 TadIndex:1
[MirrorPriCa] mirror support, insert BIT18[0] to CA, CA 0x40000000
Sub Channel address:20000000
Pseudo Channel address:10000000
rank :0
rank address:10000000
[ForwardAmap] DDR5 X4 FGBG_INT2
RankAddress:0x10000000 ChipSelect:0x0 PhysicalRankId:0x0 DimmSlot:0x0
Col:0x0 Row:0x2000 BankGroup:0x0 Bank:0x0

[TranslateMirror] Starting translate mirror
Sub Channel address:20020000
Pseudo Channel address:10010000
rank :0
rank address:10010000
[ForwardAmap] DDR5 X4 FGBG_INT2
[TranslateMirror] Sec RankAddress:0x10010000 ChipSelect:0x0 PhysicalRankId:0x0
DimmSlot:0x0
[TranslateMirror] Sec Col:0x0 Row:0x2000 BankGroup:0x2 Bank:0x0

Forward Address Translation Successful!!
```

```

ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AF89210
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
[Mirror] GetChannelMirrorStatus - Ch 0 on Socket 0 is FullyMirrored
[SSR] SSR state Save! Flag:1
[SSR] restore and restart patrol scrub info
Faillover happend in socket 0, mc 0, Saving failed channel 0
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED

Forward Address Translation start for 0x100000000
[GetMcAndChId] RtNum:2 RtInterleaveShift:7 InterleaveWays:4 RtShiftAddress:0 RtIndex:0
McTargetId is 0x0
MemType is MemTypeIllumDdr
0x100000000 belongs to DDR5
TranslatedAddress->SocketId is 0x0
TranslatedAddress->MemoryControllerId is 0x0
TranslatedAddress->ChannelId is 0x0
[ForwardIllumDecodeGetCa] TAD skt:0 Mc:0 Chn:0 TadIndex:1
[MirrorPriCa] mirror support, insert BIT18[0] to CA, CA 0x40000000
IsMirrorFailover TRUE

[TranslateMirror] Starting translate mirror
    Sub Channel address:20020000
    Pseudo Channel address:10010000
    rank :0
    rank address:10010000
[ForwardAmap] DDR5 X4 FGBG INT2
[TranslateMirror] Sec RankAddress:0x10010000    ChipSelect:0x0    PhysicalRankId:0x0
DimmSlot:0x0
[TranslateMirror] Sec    Col:0x0    Row:0x2000    BankGroup:0x2    Bank:0x0

Forward Address Translation Successful!!
RETRY_RD_ERR_LOG_MCDDC_DP_REG is not valid
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
System is in 128bit ECC mode with Mirror Mode enabled
Entering CheckNearMemoryError.
S0 C0 T0 RETRY_RD_ERR_LOG_MCDDC_DP_REG is not valid on Skt 0 Sktch 1 PseudoCh 0
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AD54810
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xC, Socket 0x0 Core 0x0
S0 C0 T1 [CpuRas]Successfully notify OS via CMCI
S0 C0 T0 [CpuRas]Successfully notify OS via CMCI
S0 C66 T1 [CpuRas]Successfully notify OS via CMCI
S0 C15 T1 [CpuRas]Successfully notify OS via CMCI
...
    
```

10.4.4 UCE on Primary, CE on Secondary

Test Objective

The objective of this test is to test the scenario where there is an uncorrectable ECC error on the primary channel and the mirrored data has a correctable ECC error. Expected outcome is to see the secondary channel's IMC perform ECC correction and for the primary channel's B2CMI to use this good data to correct and scrub the UC ECC error on the primary channel.

Configuration

Stepping	Intel® Xeon® 6700-Series Processor with E-Cores C step
BIOS	WW34'24 BKC
OS Version	n/a - Test executed and logs collected at UEFI shell
Kernel	n/a - Test executed and logs collected at UEFI shell
HW Configuration	4 DIMMs per socket, 10x4 dual rank, 64GB DIMMs on slots A1, A2, B1, and B2 (Recommended to use 1 or 2 DPC because they are the only POR prime configurations which support mirroring)
CScripts Version	BHS.2431.6000

Preconditions and Setup

Validation tools used in this recipe. See for more information about each tool.

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Mirror Mode = Full Mirror Mode
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used. It is highly recommended to have a POR memory population.

Recipe

1. Complete steps 1-6 in the [CE recipe](#) to check on the state of the system, find an address to inject to, and to unlock the injectors.
2. Using the desired target address, arm the injectors on the secondary channel for a correctable ECC error and do not inject or consume.

```
>>> ei.mem.injectMemError(addr=0x000100000000, errType="ce", immInject=False,
immConsume=False, Inj2ndCh=True, verbose=True, showErrorRegs=True, checkEInjLock=False)
```

3. Using the desired target address, arm the injectors on the primary channel for an uncorrectable ECC error and do not inject or consume. The order of this step and the previous step does not matter.

```
>>> ei.mem.injectMemError(addr=0x000100000000, errType="uce", immInject=False,
immConsume=False, Inj2ndCh=False, verbose=True, showErrorRegs=True, checkEInjLock=False)
```

4. Check the state of the memory error injectors to verify that they are armed as expected. For this particular target address, the primary channel is {socket 0, mc 0, channel 0 (only 1 channel per IMC so redundant)} and it's shown that the injector is armed for a multi-device ECC error which is uncorrectable. Also shown is the injector on the secondary channel (mc 1) which is armed for a single device single bit ECC error which is correctable.

```
>>> ei.mem.showMemInjectors(socket=0)
Socket0 MC mc0 Channel 0
Injector 0
=====
|           |           Device0           |           Device1           |
|-----|-----|-----|
| Device Status | Currently ARMED | Currently ARMED |
|-----|-----|-----|
| Mask (injector) | 00000000 00000000 | 00000000 00000000 |
|-----|-----|-----|
| Match (injector) | 000a0000 10000000 | 000a0000 10000000 |
|-----|-----|-----|
| memDev        | 0x1              | 0x5              |
|-----|-----|-----|
| XOR Mask      | 0x4              | 0x1              |
|-----|-----|-----|
=====

Socket0 MC mc1 Channel 0
Injector 0
=====
|           |           Device0           |           Device1           |
|-----|-----|-----|
| Device Status | Currently ARMED | Currently unarmed |
|-----|-----|-----|
| Mask (injector) | 00000000 00000000 | 00000000 00000000 |
|-----|-----|-----|
| Match (injector) | 000a0000 10008000 | 000a0000 10008000 |
|-----|-----|-----|
| memDev        | 0x1              | 0x1              |
|-----|-----|-----|
| XOR Mask      | 0x2              | 0x0              |
|-----|-----|-----|
=====
```

5. Write to the target address. 0xcafecafecafecafe was chosen as an example.

```
>>> itp.threads[0].mem("0x000100000000p", 8, 0xcafecafecafecafe)
```

6. Write back invalidate all the caches so modified cachelines will be written back to DDR5 memory. This action will inject the errors to memory.

```
>>> itp.threads[0].wbinvd()
```

7. Check that the memory error injectors are now unarmed since the write back invalidate injected the errors.

```
>>> ei.mem.showMemInjectors(socket=0)
Socket0 MC mc0 Channel 0
Injector 0
=====
|           |           Device0           |           Device1           |
|-----|-----|-----|
| Device Status | Currently unarmed | Currently unarmed |
|-----|-----|-----|
| Mask (injector) | 00000000 00000000 | 00000000 00000000 |
|-----|-----|-----|
| Match (injector) | 00080000 10000000 | 00080000 10000000 |
|-----|-----|-----|
=====
```

memDev	0x1	0x5
XOR Mask	0x4	0x1
=====		
Socket0 MC mcl Channel 0		
Injector 0		
=====		
	Device0	Device1
Device Status	Currently unarmed	Currently unarmed
Mask (injector)	00000000 00000000	00000000 00000000
Match (injector)	00080000 10008000	00080000 10008000
=====		
memDev	0x1	0x1
XOR Mask	0x2	0x0
=====		

8. Read the target address to consume the errors. The primary data will be read first, an uncorrectable ECC error will be detected, secondary (mirrored) data will be read, a correctable ECC error will be detected, corrected, logged, mirror scrub and check on the primary channel will occur, and the good data will be delivered to the requester.

```
>>> itp.threads[0].mem('0x000100000000p', 8)
```

9. Check the primary channel's B2CMI MCA bank. There should be a valid (valid==1) corrected (uc==0) MCA error indicating a read ECC error (mscod==1) with a corrected error count (corrcount) of 1.

```
>>> sv.socket0.soc.memss.b2cmi0.mci_status_shadow
0x8c00004001010090
>>> sv.socket0.soc.memss.b2cmi0.mci_status_shadow.showfields()\n
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000001 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : corrstatus (54:53)
0x00000001 : corrcount (52:38)
0x00000000 : otherinfo (37:32)
0x00000000 : unused (26:26)
0x00000001 : mscodddrtype (25:24)
0x00000001 : mscod (23:16)
0x00000090 : mcacod (15:00)
```

10. Primary B2CMI's MCI_ADDR should match the target address.

```
>>> sv.socket0.soc.memss.b2cmi0.mci_addr_shadow
0x100000000
```

11. Primary B2CMI's MCI_MISC should indicate that the following: error was corrected during mirroring and mirror scrub was successful (mirrorcorrerr==1), transaction was to a mirrored region (mccmdmirrregion==1), mscmdvld==1 to indicate the other mscmd fields are valid, mirror scrub check read returned good data (errortype[3]==1), and the transaction couldn't be corrected by ECC (errortype[2]==1).

```
>>> sv.socket0.soc.memss.b2cmi0.mci_misc_shadow
0x480010001b006086
>>> sv.socket0.soc.memss.b2cmi0.mci_misc_shadow.showfields()\n
0x00000000 : mirrorfailover (63:63)
0x00000001 : mirrorcorrerr (62:62)
0x00000000 : mccmdmirrfo (61:61)
0x00000000 : mccmdmirrsec (60:60)
0x00000001 : mccmdmirrregion (59:59)
0x00000000 : resvd_unused0 (58:57)
0x00000000 : mccmdchnl (56:55)
0x00000000 : mccmdmemregion (54:51)
0x00000000 : mccmdopcode (50:45)
0x00000001 : mccmdvld (44:44)
0x00000000 : bankindex (43:36)
0x00000000 : smiopcode (35:32)
0x00000000 : resvd_unused1 (31:30)
0x000000d8 : trkid (29:21)
0x0000000c : errortype (20:11)
0x00000000 : resvd_unused2 (10:10)
0x00000000 : hbmregion (09:09)
0x00000002 : addrmode (08:06)
0x00000006 : addr1sb (05:00)
```

12. The secondary channel's IMC should have a valid (valid==1) corrected (UC==0) MCA error indicating that there was a corrected demand read ECC error (mscod==0x80) with a corrected error count (corrcount) of 1.

```
>>> sv.socket0.soc.memss.mcl.ch0.mcchan.imc0_mc_status_shadow
0x8c00004200800090
>>> sv.socket0.soc.memss.mcl.ch0.mcchan.imc0_mc_status_shadow.showfields()\n
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000001 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : cor_err_stat (54:53)
0x00000001 : cor_err_cnt (52:38)
0x00000000 : fw_upd (37:37)
0x00000002 : other_info (36:32)
0x00000080 : mscod (31:16)
0x00000090 : mcacod (15:00)
```

13. Secondary IMC's MCI_ADDR should match the target address.

```
>>> sv.socket0.soc.memss.mcl.ch0.mcchan.imc0_mc8_addr_shadow
0x100000000
```

14. Secondary IMC's MCI_MISC contains FRU identification details which are also logged in the retry read error logs. See the processor register specification for the decoding of IMC MCI_MISC.

```
>>> sv.socket0.soc.memss.mcl.ch0.mcchan.imc0_mc_misc_shadow
0x400082040080086
```

15. Secondary IMC's retry read error logs configured to log demand read ECC errors should have valid data. Not going into fine detail here about what to expect because there's another recipe that goes into detail about what to expect in the case of a corrected ECC error and the register definitions should be clear.

```
>>> sv.socket0.soc.memss.mcl.ch0.mcchan.retry_rd_err_log[1]
0xb049409
>>> sv.socket0.soc.memss.mcl.ch0.mcchan.retry_rd_err_log[1].showfields()\n
0x00000000 : lock (31:31)
```

```

0x00000000 : ucna (30:30)
0x00000000 : ecc_bist_errtype (29:28)
0x00000002 : corr_fsm_errtype (27:26)
0x00000001 : ecc_bist_valid (25:25)
0x00000001 : system_addr_valid (24:24)
0x00000000 : persistent_error (23:23)
0x00000000 : transfer_number (22:19)
0x00000001 : single_bit_error (18:18)
0x00000000 : col_correction (17:17)
0x00000000 : bist_corr (16:16)
0x00000001 : noover (15:15)
0x00000000 : en_patspr (14:14)
0x00000000 : patspr (13:13)
0x00000001 : en (12:12)
0x00000000 : corr_correctable_par_dev (11:11)
0x00000001 : corr_correctable (10:10)
0x00000000 : corr_noerr (09:09)
0x00000000 : corr_err (08:08)
0x00000000 : system_addr_err (07:07)
0x00000001 : mode (06:03) -- enum 'retry_rd_eccmode' was not found
0x00000000 : over (02:02)
0x00000000 : uc (01:01)
0x00000001 : v (00:00)

>>> sv.socket0.soc.memss.mc1.ch0.mcchan.retry_rd_err_log_address1[1]
0x40040001
>>> sv.socket0.soc.memss.mc1.ch0.mcchan.retry_rd_err_log_address1[1].showfields()\n
0x00000000 : channel_addr_valid (31:31)
0x00000001 : failed_dev_valid (30:30)
0x00000000 : col (29:20)
0x00000001 : ba (19:18)
0x00000000 : bg (17:14)
0x00000000 : cbit (13:09)
0x00000000 : chip_select (08:06)
0x00000001 : failed_dev (05:00)

>>> sv.socket0.soc.memss.mc1.ch0.mcchan.retry_rd_err_log_address2[1]
0x88000801
>>> sv.socket0.soc.memss.mc1.ch0.mcchan.retry_rd_err_log_address2[1].showfields()\n
0x00000008 : meta_ecc_failed_dev (31:28)
0x00000002 : meta_ecc_errtype (27:26)
0x00000000 : meta_ecc_valid (25:25)
0x00000000 : ecc_bist_corr_err_x4_boundary_fault (24:23)
0x00000000 : ecc_bist_corr_err_devnum (22:18)
0x00000801 : row (17:00)

>>> sv.socket0.soc.memss.mc1.ch0.mcchan.retry_rd_err_log_address3[1]
0x100000000

>>> sv.socket0.soc.memss.mc1.ch0.mcchan.retry_rd_err_log_parity[1]
0x2

```

16. Resume the system to allow all error handling flows to complete.

```
>>> go()
```

17. For reference if needed, here is the BIOS serial log:

```

[ProcessMcBanks] McBankBitFields 4000
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
0/14/1 McStatusReg: 0x8C00004200800090
0/14/1 McMiscReg: 0x0
0/14-0 RetryRdErrLog: 0x0
0/14-0 RetryRdErrLogAddr1: 0x0
0/14-0 RetryRdErrLogAddr2: 0x0
0/14-1 RetryRdErrLog: 0xB049409
0/14-1 RetryRdErrLogAddr1: 0x40040001
0/14-1 RetryRdErrLogAddr2: 0x88000801
0/14-2 RetryRdErrLog: 0x0
0/14-2 RetryRdErrLogAddr1: 0x0

```

```

0/14-2 RetryRdErrLogAddr2: 0x0
0/14-3 RetryRdErrLog: 0x0
0/14-3 RetryRdErrLogAddr1: 0x0
0/14-3 RetryRdErrLogAddr2: 0x0
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xE, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xE, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xE, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 Entering CheckNearMemoryError.
S0 C0 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x1 NodeCh
is 0x0Rank:0x0 SubRank:0x0 Bank:0x1 BankGroup:0x0 Dev:0x1 Col:0x0 Row:0x801 SingleBit:0x1
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xC, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
S0 C0 T0 RETRY_RD_ERR_LOG_MCDDC_DP_REG is not valid
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
System is in 128bit ECC mode with Mirror Mode enabled
S0 C0 T0 Entering CheckNearMemoryError.
S0 C0 T0 RETRY_RD_ERR_LOG_MCDDC_DP_REG is not valid on Skt 0 Sktch 0 PseudoCh 0
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
S0 C0 T0 [CpuRas]MC status 0x8C00004001010090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x0
ERROR: C00000002:V03071008 I0 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DC8B818
ERROR: C00000002:V03071008 I0 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7DC8B818
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xC, Socket 0x0 Core 0x0
S0 C0 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C10 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C58 T0 [CpuRas]CMCI is disabled, skip CMCI signal
...
S0 C60 T0 [CpuRas]CMCI is disabled, skip CMCI signal
S0 C111 T0 [CpuRas]CMCI is disabled, skip CMCI signal
    
```

10.5 DDR Command/Address Parity Check and Retry

This feature is for fault detection and handling on the command and address lines in the DDR memory interface domain during read and write transactions. CMD/ADDR parity errors on the CMD/ADDR bus are signaled on the Alert pin and are recoverable. If an Alert signal is asserted by any of the ranks (the Alert signal is a single signal per channel; Alert signals from all DRAMs and ranks are wired-or connection). When an Alert signal is asserted, iMC will be unable to identify which rank/DIMM asserted the error signal or for which cmd. The error source will be identified in error recovery process after the assertion of an Alert signal. The high level DDR CMD/ADDR parity recovery flow shows below:

1. DDR Register device detects a CMD/ADDR Parity error. Drops the transaction. Logs Status bit and Address info and asserts DDRn_ALERT_N. Puts DIMMs in SelfRefresh. Wait for iMC to send channel RESTART command.
2. IMC holds the pending transactions and performs following steps: a. IMC logs the ADDR/CMD Parity error in Machine-check bank as a corrected error (UC=0, PCC=0, ADDRv=0, MSCOD=0x0200, MCACOD=0x00Bn, CorrErr+). Triggers CMCI if threshold reached. Trigger CSMI if EMCA2 is enabled.

3. IMC clears the DIMM's Register device error log and resumes the normal transactions including all the previously pending transactions.
4. In case of persistent fault:
 - a. IMC will make multiple attempts to read the register device info (default is 16 attempts), if not successful then declare a persistent fault. IMC will trigger MCERR with Link_fail condition (UC=1, PCC=1, ADDR=0, MISCV=1, MSCOD=0x108, MCACOD=0x008n). IMC will also update the LINK_MCA_CTL.err0_log (bit1) register bit if enabled by setting LINK_MCA_CTL.err0_en (bit 0) register bit. IMC will not update CORRERRSTATUS register.

10.5.1 DDR Transient C/A Parity Error

Test Objective

Primary usage model for this feature is to detect and handle faults on the DDR memory command and address bus during read, write, and various other DDR protocol specific transactions. Following are the key target use-cases:

- Transient event - the host memory controller (actor) retries the transaction at fault (with successful recovery) upon receiving an ALERT signal indicating parity errors detected on the corresponding command and address lines in the DDR memory interface.
- Upon receiving an ALERT signal indicating uncorrected parity errors detected on the corresponding command and address lines in the DDR4 memory interface on both primary and secondary channels, fail-over occurs.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	A1
BIOS	WW05'24 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	4 DIMMs on both sockets
CScripts Version	GNR2406.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESE for Machine Check Exception \(MCE\) Observability](#) on page 27
- [Enhanced MCA Logging \(ELOG\)](#) on page 28

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory DFX Configuration >
 - C/A Parity Enable = Enable

Recipe

1. Verify that the iMC machine check status and alert signal registers are clear prior to testing.

```
>>> halt()
>>> sv.socket0.soc.memss.showsearch("mc_status_shadow")
mc2.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc3.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc8.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc9.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc0.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc1.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc6.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc7.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc4.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc5.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc10.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc11.ch0.mcchan.imc0_mc_status_shadow = 0x0

>>> sv.socket0.soc.memss.showsearch("alert")
mc2.ch0.mcchan.alertsignal = 0x0
mc3.ch0.mcchan.alertsignal = 0x0
mc8.ch0.mcchan.alertsignal = 0x0
mc9.ch0.mcchan.alertsignal = 0x0
mc0.ch0.mcchan.alertsignal = 0x0
mc1.ch0.mcchan.alertsignal = 0x0
mc6.ch0.mcchan.alertsignal = 0x0
mc7.ch0.mcchan.alertsignal = 0x0
mc4.ch0.mcchan.alertsignal = 0x0
mc5.ch0.mcchan.alertsignal = 0x0
mc10.ch0.mcchan.alertsignal = 0x0
mc11.ch0.mcchan.alertsignal = 0x0
```

2. OS: Open a new terminal, enable MCE Record event for Ftrace and start reading the trace live.

```
# cd /sys/kernel/debug/tracing/
# echo 1 > events/mce/mce_record/enable
# cat trace_pipe
```

3. CScripts: Inject the CMD parity error - use the corresponding configuration that works for your platform.

```
>>> ei.utils.resetInjectorLockCheck()
>>> ei.mem.injectCMDParity(socket=0, mc=0, ch=0)
...
Successfully injected CMD Parity error to mc0 ch0!
```

4. Verify that the C/A parity error was logged and that the alert signal bit was set.

```
>>> sv.socket0.soc.memss.showsearch("mc_status_shadow")
mc2.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc3.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc8.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc9.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc0.ch0.mcchan.imc0_mc_status_shadow = 0x80000040020000b0
mc1.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc6.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc7.ch0.mcchan.imc0_mc_status_shadow = 0x0
```

```
mc4.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc5.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc10.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc11.ch0.mcchan.imc0_mc_status_shadow = 0x0

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc_status_shadow.show()
0x00000001 : valid (63:63) -- Error logged in this bank is valid
0x00000000 : over (62:62) -- (Overflow) Indicates (when set) that a machine check error
occurred while the results o...
0x00000000 : uc (61:61) -- Error logged in this banks is an uncorrected error.
0x00000000 : en (60:60) -- Indicates (when set) that signaling of the machine check exception
for this error was ena...
0x00000000 : miscv (59:59) -- Indicates (when set) that the MCiMISC register contains
additional information regardi...
0x00000000 : addrv (58:58) -- indicates (when set) that the MCiADDR register contains the
address where the error oc...
0x00000000 : pcc (57:57) -- Indicates (when set) that the state of the processor might have
been corrupted by the er...
0x00000000 : s (56:56) -- S - Signaling. When set it indicates that the error was signaled
with an MCERR. When clear...
0x00000000 : ar (55:55) -- AR - Action REquired. When set it indicates that SW needs to take
immediate recovery acti...
0x00000000 : cor_err_stat (54:53) -- These bits are used to indicate when the number of
corrected errors has exceede...
0x00000001 : cor_err_cnt (52:38) -- A 15 bit counter that is incremented each time a corrected
(or uncorrected but i...
0x00000000 : fw_upd (37:37) -- Indicates that the BIOS has updated the contents of the MC
Bank. HW should set this t...
0x00000000 : other_info (36:32) -- The functions of the bits in this field are implementation
specific and are not p...
0x00000200 : mscod (31:16) -- Specifies a model specific error code that uniquely identifies
the machine check error...
0x000000b0 : mcacod (15:00) -- Specifies the machine check architecture defined error code for
the machine check err...

>>> sv.socket0.soc.memss.showsearch("alert")
mc2.ch0.mcchan.alertsignal = 0x0
mc3.ch0.mcchan.alertsignal = 0x0
mc8.ch0.mcchan.alertsignal = 0x0
mc9.ch0.mcchan.alertsignal = 0x0
mc0.ch0.mcchan.alertsignal = 0x1
mc1.ch0.mcchan.alertsignal = 0x0
mc6.ch0.mcchan.alertsignal = 0x0
mc7.ch0.mcchan.alertsignal = 0x0
mc4.ch0.mcchan.alertsignal = 0x0
mc5.ch0.mcchan.alertsignal = 0x0
mc10.ch0.mcchan.alertsignal = 0x0
mc11.ch0.mcchan.alertsignal = 0x0

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.alertsignal.show()
0x00000001 : seen (00:00) -- If 1, alert signal (DDR5 parity error or Wrcrc error, if enabled)
has been observed. Th...
```

5. Resume the system to allow all error handling flows to complete.

```
go()
```

6. Serial Log Output

```
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x80000040020000B0, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x80000040020000B0, class CORRECTED
S0 C0 T0 [CpuRas]MC status 0x80000040020000B0, class CORRECTED
```

```
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AC6A010
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0
S0 C0 T1 [CpuRas]Successfully notify OS via CMCI
S0 C6 T0 [CpuRas]Successfully notify OS via CMCI
S0 C6 T1 [CpuRas]Successfully notify OS via CMCI
S0 C7 T0 [CpuRas]Successfully notify OS via CMCI
S0 C2 T1 [CpuRas]Successfully notify OS via CMCI
S0 C4 T0 [CpuRas]Successfully notify OS via CMCI
S0 C0 T0 [CpuRas]Successfully notify OS via CMCI
S0 C3 T1 [CpuRas]Successfully notify OS via CMCI
S0 C5 T0 [CpuRas]Successfully notify OS via CMCI
S0 C9 T1 [CpuRas]Successfully notify OS via CMCI
S0 C5 T1 [CpuRas]Successfully notify OS via CMCI
S0 C11 T1 [CpuRas]Successfully notify OS via CMCI
S0 C8 T1 [CpuRas]Successfully notify OS via CMCI
S0 C1 T0 [CpuRas]Successfully notify OS via CMCI
S0 C2 T0 [CpuRas]Successfully notify OS via CMCI
S0 C10 T1 [CpuRas]Successfully notify OS via CMCI
S0 C10 T0 [CpuRas]Successfully notify OS via CMCI
S0 C1 T1 [CpuRas]Successfully notify OS via CMCI
S0 C8 T0 [CpuRas]Successfully notify OS via CMCI
S0 C9 T0 [CpuRas]Successfully notify OS via CMCI
S0 C4 T1 [CpuRas]Successfully notify OS via CMCI
S0 C3 T0 [CpuRas]Successfully notify OS via CMCI
S0 C11 T0 [CpuRas]Successfully notify OS via CMCI
S0 C7 T1 [CpuRas]Successfully notify OS via CMCI
```

7. OS: Check the terminal that is reading the Ftrace trace. The details to take note of are the MC bank (13) and MC status(). This aligns with what was seen in the serial log and in CScripts. Verify an ELog record was generated indicated by `extlog_mem_event`.

```
cd /sys/kernel/debug/tracing
echo 1 > events/mce/mce_record/enable
echo 1 > events/ras/enable
cat trace_pipe
kworker/0:1-10 [000] ..... 618.100889: mce_record: CPU: 0, MCGc/s: f000c20/0,
MC13: 80000040020000b0, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000000000/0000000000000000/0000000000000000, RIP: 00:<0000000000000000>, TSC:
126f9252331, PROCESSOR: 0:a06d0, TIME: 1707952860, SOCKET: 0, APIC: 0
kworker/0:1-10 [000] ..... 618.100933: extlog_mem_event: {1} corrected error:
parity error physical addr: ffffffff (mask lsb: ff) node:0 card:0 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA
```

10.6 Memory Demand and Patrol Scrub

Patrol Scrubbing

Patrol scrubbing pro-actively searches the system memory repairing Correctable errors preventing accumulation of single-bit errors and turning it into an uncorrected error. Patrol scrubbing is accomplished by using an engine that generates requests to memory addresses in a stride. The engine will generate a memory request at the preprogrammed frequency, and the demand scrubbing flow corrects the error, if any. Patrol scrubbing finds opportunities on idle cycles to scrub the memory and get rid of any detectable Correctable errors. Patrol scrubs are intended to ensure that data with a Correctable error does not remain in DRAM long enough to stand a significant chance of further corruption to an Uncorrectable error due to a high energy particle error. The iMC will issue a Patrol Scrub at a rate sufficient to write every line once a day. For a maximum channel capacity of 192 GB, this would be one scrub every 26.8 micro-sec. The Patrol Scrub rate is configurable via 16b scrub interval timer.

Demand Scrubbing

Demand scrubbing is the ability to write corrected data back to the memory once a Correctable error is detected on a read transaction. This allows data correction in memory at the time of detection, and decreases the chances of a second error on the same address accumulating and causing a multi-bit error condition.

PFD (Permanent Fault Detection)

PFD (Permanent Fault Detection) is new with 4th Gen Intel® Xeon® Processor Scalable Family, Codename Sapphire Rapids. The logic determines if given fault from DIMM is confined to single device (Correctable), multi devices (Uncorrectable), or if the fault was transient. The ECC logic makes use of this information to correct the error from faulty DRAM device.

10.6.1 Patrol Scrub

Test Objective

UEFI-FW enables the patrol scrubbing engine during boot time and sets the scrubbing interval. Once enabled, it reads every cache line and checks for any error and writes back the data after correction. In case the patrol scrubber detects an uncorrected error, it writes back a poison signature and triggers a recoverable machine-check event. The patrol scrubber should detect the corrected error and scrub the memory location with correct data.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, IIntel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A Step
BIOS	WW07'24 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	4 DIMMs on both sockets
CScripts Version	GNR2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [Enhanced MCA Logging \(ELOG\)](#) on page 28

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Patrol Scrub = Enable at End of Post
 - Patrol Scrub Interval = 1
 - DDR5 ECS = Enable
- EDKII > Socket Configuration > Memory Configuration >
 - PTRR DDR Enable = Disable
 - If PTRR is enabled there's a chance that PTRR may take over the transaction and the patrol scrub will not take place.

Recipe

1. Initialize ftrace and ELog according to the [Validation Tools Overview](#) on page 18 section.
2. Ensure that patrol scrub is enabled on the DIMM that will be encountering the memory CECC error.

```
>>> sv.socket0.soc.memss.mcs.chs.mcchan.scrubctl
socket0.soc.memss.mc0.ch0.mcchan.scrubctl - 0x90000591
socket0.soc.memss.mc1.ch0.mcchan.scrubctl - 0x90000591
socket0.soc.memss.mc2.ch0.mcchan.scrubctl - 0x90000591
socket0.soc.memss.mc3.ch0.mcchan.scrubctl - 0x90000591
socket1.soc.memss.mc0.ch0.mcchan.scrubctl - 0x90000591
socket1.soc.memss.mc1.ch0.mcchan.scrubctl - 0x90000591
socket1.soc.memss.mc2.ch0.mcchan.scrubctl - 0x90000591
socket1.soc.memss.mc3.ch0.mcchan.scrubctl - 0x90000591
>>> sv.socket0.soc.memss.mc2.ch0.scrubctl.showfields()
0x00000001 : scrub_en (31:31)
0x00000000 : stop_on_cmpl (30:30)
0x00000000 : ptl_cmpl (29:29)
0x00000001 : scrubissued (28:28)
0x00000000 : issueonce (27:27)
0x00000000 : startscrub (26:26)
0x00000591 : scrubinterval (23:00)
```

3. Ensure that the machine check status register and retry_rd_err_logs are clear. Check that en_patspr is enabled in the corresponding DIMM location to log patrol scrub errors.

```
>>> sv.socket0.soc.memss.showsearch("mc_status_shadow")
memss.mc2.ch0.mcchan.imc0_mc_status_shadow = 0x0
memss.mc3.ch0.mcchan.imc0_mc_status_shadow = 0x0
memss.mc0.ch0.mcchan.imc0_mc_status_shadow = 0x0
memss.mc1.ch0.mcchan.imc0_mc_status_shadow = 0x0
>>> sv.socket0.soc.memss.mcs.chs.mcchan.retry_rd_err_log
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log - 0x00000000
socket0.soc.memss.mc1.ch0.mcchan.retry_rd_err_log - 0x00000000
socket0.soc.memss.mc2.ch0.mcchan.retry_rd_err_log - 0x00000000
socket0.soc.memss.mc3.ch0.mcchan.retry_rd_err_log - 0x00000000
>>> sv.socket0.soc.memss.mc2.ch0.mcchan.retry_rd_err_log[0].showfields()
0x00000000 : lock (31:31)
0x00000000 : ucna (30:30)
0x00000000 : ecc_bist_errtype (29:28)
0x00000000 : corr_fsm_errtype (27:26)
0x00000000 : ecc_bist_valid (25:25)
0x00000000 : system_addr_valid (24:24)
0x00000000 : persistent_error (23:23)
0x00000000 : transfer_number (22:19)
0x00000000 : single_bit_error (18:18)
0x00000000 : col_correction (17:17)
0x00000000 : bist_corr (16:16)
0x00000001 : noover (15:15)
```

```

0x00000001 : en_patspr (14:14)
0x00000000 : patspr (13:13)
0x00000001 : en (12:12)
0x00000000 : corr_correctable_par_dev (11:11)
0x00000000 : corr_correctable (10:10)
0x00000000 : corr_noerr (09:09)
0x00000000 : corr_err (08:08)
0x00000000 : system_addr_err (07:07)
0x00000000 : mode (06:03) -- enum 'retry_rd_eccmode' was not found
0x00000000 : over (02:02)
0x00000000 : uc (01:01)
0x00000000 : v (00:00)

```

4. Inject a correctable patrol scrub error.

```

>>> halt()
>>> ei.utils.resetInjectorLockCheck()
>>> ei.mem.injectMemError(0x8000, errType="ce", PatrolConsume=True)

```

5. Review the machine check bank registers. Verify that a correctable patrol scrub error was logged in the MSCOD.

```

>>> sv.socket0.soc.memss.showsearch("mc_status_shadow")
mc2.ch0.mcchan.imc0_mc_status_shadow = 0x8c000042000800c0
mc3.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc0.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc1.ch0.mcchan.imc0_mc_status_shadow = 0x0
>>> sv.socket0.soc.memss.mc2.ch0.mcchan.imc0_mc_status_shadow.showfields()
0x00000001 : valid (63:63)
0x00000000 : over (62:62)
0x00000000 : uc (61:61)
0x00000000 : en (60:60)
0x00000001 : miscv (59:59)
0x00000001 : addrv (58:58)
0x00000000 : pcc (57:57)
0x00000000 : s (56:56)
0x00000000 : ar (55:55)
0x00000000 : cor_err_stat (54:53)
0x00000001 : cor_err_cnt (52:38)
0x00000000 : fw_upd (37:37)
0x00000002 : other_info (36:32)
0x00000008 : mscod (31:16)
0x000000c0 : mcacod (15:00)

```

6. The retry_rd_err_log will also indicate a patrol scrub error was logged.

```

>>> sv.socket0.soc.memss.mc2.ch0.mcchan.retry_rd_err_log.showfields()
0x00000000 : lock (31:31)
0x00000000 : ucna (30:30)
0x00000000 : ecc_bist_errtype (29:28)
0x00000002 : corr_fsm_errtype (27:26)
0x00000001 : ecc_bist_valid (25:25)
0x00000001 : system_addr_valid (24:24)
0x00000000 : persistent_error (23:23)
0x00000000 : transfer_number (22:19)
0x00000001 : single_bit_error (18:18)
0x00000000 : col_correction (17:17)
0x00000000 : bist_corr (16:16)
0x00000001 : noover (15:15)
0x00000001 : en_patspr (14:14)
0x00000001 : patspr (13:13)
0x00000001 : en (12:12)
0x00000000 : corr_correctable_par_dev (11:11)
0x00000001 : corr_correctable (10:10)
0x00000000 : corr_noerr (09:09)
0x00000000 : corr_err (08:08)
0x00000000 : system_addr_err (07:07)
0x00000001 : mode (06:03) -- enum 'retry_rd_eccmode' was not found

```

```
0x00000000 : over (02:02)
0x00000000 : uc (01:01)
0x00000001 : v (00:00)
```

7. Resume the system and check the serial log that BIOS handled the error.

```
[ProcessMcBanks] McBankBitFields 8000
S0 C21 T0 [CpuRas]MC status 0x8C000042000800C0, class CORRECTED
0/15/2 McStatusReg: 0x8C000042000800C0
0/15/2 McMiscReg: 0x0
0/15-0 RetryRdErrLog: 0xB04F409
0/15-0 RetryRdErrLogAddr1: 0x44000001
0/15-0 RetryRdErrLogAddr2: 0xF0000000
0/15-1 RetryRdErrLog: 0x9000
0/15-1 RetryRdErrLogAddr1: 0x0
0/15-1 RetryRdErrLogAddr2: 0x0
0/15-2 RetryRdErrLog: 0x0
0/15-2 RetryRdErrLogAddr1: 0x0
0/15-2 RetryRdErrLogAddr2: 0x0
0/15-3 RetryRdErrLog: 0x0
0/15-3 RetryRdErrLogAddr1: 0x0
0/15-3 RetryRdErrLogAddr2: 0x0
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C21 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xF, State = 0x1
S0 C21 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xF, State = 0x2
S0 C21 T0 [CpuRas]MC status 0x8C000042000800C0, class CORRECTED
S0 C21 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xF, State = 0x4
S0 C21 T0 [CpuRas]MC status 0x8C000042000800C0, class CORRECTED
TranslateSad: McTargetId is 0x2
Entering CheckNearMemoryError.
S0 C21 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x2 NodeCh
is 0x0Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x0 SingleBit:0x1
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
S0 C21 T0 [CpuRas]MC status 0x8C000042000800C0, class CORRECTED
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8010
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
LogErrorToElog: --Logging Error to EMCA
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x0
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x2A Bank 0xF, Socket 0x0 Core 0x15
S0 C1 T0 [CpuRas]Successfully notify OS via CMCI
```

8. Verify that an ELog record was generated indicated by extlog_mem_event.

```
<...>-2241 [000] ..... 564.041704: mce_record: CPU: 0, MCGc/s:
f000c20/0, MC15: 8c000042000800c0, IPID: 0000000000000000, ADDR/MISC/SYND:
00000000000008000/04000800000008086/0000000000000000, RIP:
00:<0000000000000000>, TSC: 113ba12aedf, PROCESSOR: 0:a06d0, TIME:
1708487536, SOCKET: 0, APIC: 0
<...>-2241 [000] ..... 564.041783: extlog_mem_event: {1}
corrected error: scrub corrected error physical addr: 00000000000008000 (mask
lsb: ff) node:0 card:2 module:0 rank:0 bank:0 device:1 row:0 column:256 DIMM
location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnC, DIMM0
kworker/0:5-2241 [000] ..... 564.043088: mc_event: 1 Corrected
error: memory scrubbing error on CPU_SrcID#0_MC#2_Chan#0_DIMM#0 (mc:2
location:0:0:-1 address:0x00008000 grain:32 syndrome:0x00000000
err_code:0x0008:0x00c0 SystemAddress:0x8000 ProcessorSocketId:0x0
MemoryControllerId:0x2 ChannelAddress:0x2000 ChannelId:0x0 RankAddress:0x800
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x0 Column:0x100
Bank:0x0 BankGroup:0x0 ChipSelect:0x0)
```


10.6.2 Demand Scrub

Test Objective

The processor's memory sub-system scrubs the memory location when a corrected error is detected during demand read by writing back the corrected data. Next time if the same memory location is accessed, then same error will not be detected thus improving the reliability of the memory sub-system. This also minimizes the risk of accumulation of single device errors and potentially resulting in an uncorrected error.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Granite Rapids, Sierra Forest, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A1
BIOS	WW05'24 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	4 DIMMs on both sockets
CScripts Version	GNR2406.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [Enhanced MCA Logging \(ELOG\)](#) on page 28

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - DDR5 ECS = Enable

Recipe

1. Set up FTrace and ELog recording from the OS based on the FTrace and ELog sections in this document.
2. Ensure the machine check status register and retry_rd_err_log valid bits are cleared.

```
>>> sv.socket0.soc.memss.showsearch("mc_status_shadow")
mc0.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc1.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc2.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc3.ch0.mcchan.imc0_mc_status_shadow = 0x0
```

```
>>> sv.socket0.soc.memss.mcs.chs.mcchan.retry_rd_err_log.v
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[0].v - 0x00000000
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[1].v - 0x00000000
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[2].v - 0x00000000
socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[3].v - 0x00000000
socket0.soc.memss.mc1.ch0.mcchan.retry_rd_err_log[0].v - 0x00000000
socket0.soc.memss.mc1.ch0.mcchan.retry_rd_err_log[1].v - 0x00000000
socket0.soc.memss.mc1.ch0.mcchan.retry_rd_err_log[2].v - 0x00000000
socket0.soc.memss.mc1.ch0.mcchan.retry_rd_err_log[3].v - 0x00000000
socket0.soc.memss.mc2.ch0.mcchan.retry_rd_err_log[0].v - 0x00000000
socket0.soc.memss.mc2.ch0.mcchan.retry_rd_err_log[1].v - 0x00000000
socket0.soc.memss.mc2.ch0.mcchan.retry_rd_err_log[2].v - 0x00000000
socket0.soc.memss.mc2.ch0.mcchan.retry_rd_err_log[3].v - 0x00000000
socket0.soc.memss.mc3.ch0.mcchan.retry_rd_err_log[0].v - 0x00000000
socket0.soc.memss.mc3.ch0.mcchan.retry_rd_err_log[1].v - 0x00000000
socket0.soc.memss.mc3.ch0.mcchan.retry_rd_err_log[2].v - 0x00000000
socket0.soc.memss.mc3.ch0.mcchan.retry_rd_err_log[3].v - 0x00000000

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[1].show()
0x00000000 : lock (31:31) -- This register is locked for UC error update; if ucna...
0x00000000 : ucna (30:30) -- See description for locked bit.
0x00000000 : ecc_bist_errtype (29:28) -- Indicates the err type for the PFD result...
0x00000000 : corr_fsm_errtype (27:26) -- Indicates the err type for the Corection...
0x00000000 : ecc_bist_valid (25:25) -- Indicates that PFD is enabled for the logged...
0x00000000 : system_addr_valid (24:24) -- Indicates there is valid system address...
0x00000000 : persistent_error (23:23) -- Indicates persistent error detected by PFD.
0x00000000 : transfer_number (22:19) -- Indicates Transfer number of the single bit...
0x00000000 : single_bit_error (18:18) -- Indicates if there was a single bit...
0x00000000 : col_correction (17:17) -- This field indicate a column correction...
0x00000000 : bist_corr (16:16) -- Correction is done by PFD. Also set to 1 if...
0x00000001 : noover (15:15) -- Lock register after the first correctable error...
0x00000000 : en_patspr (14:14) -- If en_patspr=1: Log patrol/sparing errors...
0x00000000 : patspr (13:13) -- Indicates if we have logged a patrol/spare error...
0x00000001 : en (12:12) -- Enable error logging. Will log on every retry if...
0x00000000 : corr_correctable_par_dev (11:11) -- During the last log, entered...
0x00000000 : corr_correctable (10:10) -- During the last log, entered...
0x00000000 : corr_noerr (09:09) -- During the last log, entered correction...
0x00000000 : corr_err (08:08) -- During the last log, entered correction...
0x00000000 : system_addr_err (07:07) -- System address translation error
0x00000000 : mode (06:03) -- Indicate the ecc mode this register is logging...
0x00000000 : over (02:02) -- The log has been overwritten.
0x00000000 : uc (01:01) -- Uncorrectable error. Logs cannot be overwritten...
0x00000000 : v (00:00) -- The contents of this register have valid data as well...
```

3. Inject a correctable error.

```
>>> halt()
>>> ei.utils.resetInjectorLockCheck()
>>> ei.mem.injectMemError(0x10000, errType="ce", immConsume=False)
```

4. Consume the error by reading the memory address.

```
>>> mem(0x10000)
0x10000
```

5. Check the machine check bank registers and check a memory read error was detected.

```
>>> sv.socket0.soc.memss.showsearch("mc_status_shadow")
mc0.ch0.mcchan.imc0_mc_status_shadow = 0x8c00004200800090
mc1.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc2.ch0.mcchan.imc0_mc_status_shadow = 0x0
mc3.ch0.mcchan.imc0_mc_status_shadow = 0x0

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.imc0_mc_status_shadow.show()
0x00000001 : valid (63:63) -- Error logged in this bank is valid
0x00000000 : over (62:62) -- (Overflow) Indicates (when set) that a machine check error...
0x00000000 : uc (61:61) -- Error logged in this banks is an uncorrected error.
0x00000000 : en (60:60) -- Indicates (when set) that signaling of the machine...
0x00000001 : miscv (59:59) -- Indicates (when set) that the MCI_MISC register contains...
```

```

0x00000001 : addrv (58:58) -- indicates (when set) that the MCiADDR register contains...
0x00000000 : pcc (57:57) -- Indicates (when set) that the state of the processor might...
0x00000000 : s (56:56) -- S - Signaling. When set it indicates that the error was...
0x00000000 : ar (55:55) -- AR - Action REquired. When set it indicates that SW needs...
0x00000000 : cor_err_stat (54:53) -- These bits are used to indicate when the number...
0x00000001 : cor_err_cnt (52:38) -- A 15 bit counter that is incremented each time...
0x00000000 : fw_upd (37:37) -- Indicates that the BIOS has updated the contents of...
0x00000002 : other_info (36:32) -- The functions of the bits in this field are...
0x00000080 : mscod (31:16) -- Specifies a model specific error code that uniquely...
0x00000090 : mcacod (15:00) -- Specifies the machine check architecture defined...

```

6. Check the retry_rd_err_logs that the error was logged.

```

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.retry_rd_err_log[1].show()
0x00000000 : lock (31:31) -- This register is locked for UC error update; if ucna bit is...
0x00000000 : ucna (30:30) -- See description for locked bit.
0x00000000 : ecc_bist_errtype (29:28) -- Indicates the err type for the PFD result.
0x00000002 : corr_fsm_errtype (27:26) -- Indicates the err type for the Corection fsm...
0x00000001 : ecc_bist_valid (25:25) -- Indicates that PFD is enabled for the logged...
0x00000001 : system_addr_valid (24:24) -- Indicates there is valid system address in the...
0x00000000 : persistent_error (23:23) -- Indicates persistent error detected by PFD.
0x00000000 : transfer_number (22:19) -- Indicates Transfer number of the single...
0x00000000 : single_bit_error (18:18) -- Indicates if there was a single bit...
0x00000000 : col_correction (17:17) -- This field indicate a column correction done...
0x00000000 : bist_corr (16:16) -- Correction is done by PFD. Also set to 1 if correction...
0x00000001 : noover (15:15) -- Lock register after the first correctable error, do not...
0x00000000 : en_patspr (14:14) -- If en_patspr=1: Log patrol/sparing errors...
0x00000000 : patspr (13:13) -- Indicates if we have logged a patrol/spare error...
0x00000001 : en (12:12) -- Enable error logging. Will log on every retry if type matches.
0x00000000 : corr_correctable_par_dev (11:11) -- During the last log, entered...
0x00000001 : corr_correctable (10:10) -- During the last log, entered correction...
0x00000000 : corr_noerr (09:09) -- During the last log, entered correction but...
0x00000000 : corr_err (08:08) -- During the last log, entered correction but...
0x00000000 : system_addr_err (07:07) -- System address translation error
0x00000001 : mode (06:03) -- Indicate the ecc mode this register is logging...
0x00000000 : over (02:02) -- The log has been overwritten.
0x00000000 : uc (01:01) -- Uncorrectable error. Logs cannot be overwritten...
0x00000001 : v (00:00) -- The contents of this register have valid data as...

```

7. Resume the system and check the serial log that BIOS handled the error.

```

[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED

Forward Address Translation start for 0x10000
[GetMcAndChId] RtNum:2 RtInterleaveShift:7 InterleaveWays:2 RtShiftAddress:4 RtIndex:0
McTargetId is 0x0
MemType is MemTypeDdr
0x10000 belongs to DDR5
TranslatedAddress->SocketId is 0x0
TranslatedAddress->MemoryControllerId is 0x0
TranslatedAddress->ChannelId is 0x0
[ForwardDdrDecodeGetCa] TAD skt:0 Mc:0 Chn:0 TadIndex:0
  Sub Channel address:2000
  Pseudo Channel address:1000
  rank :0
  rank address:1000
[ForwardAmap] DDR5 X4 FGBG_INT2
  RankAddress:0x1000 ChipSelect:0x0 PhysicalRankId:0x0 DimmSlot:0x0
  Col:0x200 Row:0x0 BankGroup:0x0 Bank:0x2

Forward Address Translation Successful!!

```

```

Entering CheckNearMemoryError.
[GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Rank:0x0 SubRank:0x0 Bank:0x2
BankGroup:0x0 Dev:0x1 Col:0x200 Row:0x0 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
InterLocSyn:0x0
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
[Mca]McaDetectAndHandle, severity = 0x2, Broadcast is 0x1
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AC6B010
WHEA: Detected Memory Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
[Mca] Clear Status
[Mca]ClearErrLogReg
[Mca]Sending CMCI for CPU 0x0 Bank 0xD, Socket 0x0 Core 0x0

```

8. Check the FTrace logs to ensure that an ELog record was logged indicated by extlog_mem_event.

```

[root@gnr-bkc ~]# cd /sys/kernel/debug/tracing/
[root@gnr-bkc tracing]# echo 1 > events/mce/mce_record/enable
[root@gnr-bkc tracing]# cat trace_pipe
<...>-8 [000] ..... 436.694699: mce_record: CPU: 0, MCGc/s: f000c20/0, MC13:
8c00004200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000010000/0400084000010086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
e3db41967f, PROCESSOR: 0:a06d0, TIME: 1707951635, SOCKET: 0, APIC: 0
<...>-8 [000] ..... 436.694809: extlog_mem_event: {1} corrected error:
single-bit ECC physical addr: 0000000000010000 (mask lsb: ff) node:0 card:0 module:0 rank:0
bank:2 device:1 row:0 column:512 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0
kworker/0:0-8 [000] ..... 436.696034: mc_event: 1 Corrected error: memory read
error on CPU SrcID#0 MC#0 Chan#0 DIMM#0 (mc:0 location:0:0:-1 address:0x00010000 grain:32
syndrome:0x00000000_err_code:0x0080:0x0090 SystemAddress:0x10000 ProcessorSocketId:0x0
MemoryControllerId:0x0 ChannelAddress:0x4000 ChannelId:0x0 RankAddress:0x1000
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x0 Column:0x200 Bank:0x2 BankGroup:0x0
ChipSelect:0x0)

```

10.7 Runtime Post Package Repair (Runtime PPR)

PPR (Runtime Post Package Repair) is invoked during runtime by BIOS. It maps out what platform identifies as bad rows with spare rows in DDR DIMMs. The bad row can be identified based on software thresholding, which helps BIOS identify which row is the most affected thus repairing the row with the most correctable errors. At least one row per bank group can be repaired. Maximum rows per bank group is vendor dependent. Once the bad row has been repaired at runtime, it is qualified for repair by boot time hard PPR by the BIOS, which logs the error in NVRAM.

10.7.1 Runtime Soft PPR

Test Objective

Inject correctable errors, observe BIOS qualify lines for PPR, and repair the affected rows at runtime utilizing runtime soft PPR.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A1
BIOS	WW05'24 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	4 DIMMs per socket
CScripts Version	GNR2406.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [Enhanced MCA Logging \(ELOG\)](#) on page 28

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Runtime PPR/Row Sparing = Enable
 - Correctable Error Threshold = 1
 - Trigger SW Error Threshold = Enable
 - SW Per Row Threshold = 2
- EDKII > Platform Configuration > System Event Log >
 - RAS Log Level = MID (BASIC_FLOW, FUNC_FLOW)
- EDKII > Platform Configuration > Miscellaneous Configuration >
 - Serial Debug Message Level = Normal

NOTE

sPPR is experiencing issues such as sightings report issue #120. BIOS/BMC developers can continue with development by temporarily bypassing the issue by turning off data scrambling.

Recipe

1. Set up FTrace and ELog recording from the OS based on the FTrace and ELog sections in this document.
2. Halt the system and reset the injectors.

```
>>> halt()
>>> ei.utils.resetInjectorLockCheck()
```

3. Inject a correctable error then resume the system so BIOS can log the DIMM error record.

```
>>> ei.mem.injectMemError(0x8000, errType="ce")
>>> go()
```

4. Observe in serial log that the SW Row Threshold counter is at 1/2.

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
[Updated] Policy bank level threshold: 3, row threshold: 2, time window: 24.
[Updated] UpdateEnhancedDimmErrRecord started.
[GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Rank:0x0 SubRank:0x0 Bank:0x0
BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x0 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
InterLocSyn:0x0
Check Error is in VLS record structure.
AddNewVlsFailureSample Started.
[Updated] UpdateVlsFailureSample Entry.
[Updated] CurrSWBank Counter/Bank Threshold:1/3.
Check error is in PPR record structure.
AddNewPprFailureSample Started.
[Updated] UpdatePprFailureSample Entry.
[Updated] CurrSWRow_Counter/Row_Threshold:1/2.
[Updated] ErrorThReached=0
[Updated] UpdateEnhancedDimmErrRecord ended.
UpdateMcChanSsrState ch:1:PatrolSPAMode
McSsrState Node:1 :PatrolSPAMode
UpdateMcChanSsrState ch:6:PatrolSPAMode
McSsrState Node:6 :PatrolSPAMode
UpdateMcChanSsrState ch:7:PatrolSPAMode
McSsrState Node:7 :PatrolSPAMode
[imc] memory error handler end!
[ECS] CheckAndHandleECSEvent Enter
[ECS] CheckAndHandleECSEvent Exit
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]ProcessMcBankError: Scope Uncore
S0 C0 T0 [Mca]EmcaReadMcBankBitField: Scope Uncore, BitField 0x2000
S0 C0 T0 [Mca]McBankBitField: 0x2000
S0 C0 T0 [Mca]EmcaClearMcBankBitField: Scope Uncore, BitField 0x2000
S0 C0 T0 [Mca]McBankErrorHandler: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [Mca]McBankErrorHandler: IoMcaErrorHandler Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]MapErrorSeverity: MCE class is 0x5
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [Mca]McBankErrorHandler: Section Type = EMCA_SECTYPE_PLATMEM
FillPlatformMemoryErrorData Start!
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
FillPlatformMemoryErrorData: BankType:0x200
Error Source is M2M or IMC (MCCHAN)BankType:0x200
```

5. Verify in FTrace logs that an ELog event was recorded indicated by extlog_mem_event.

```
kworker/0:7-2239 [000] ..... 434.249253: mce_record: CPU: 0, MCGc/s: f000c20/0, MC15:
8c00004200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000008000/0400080000008086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
ef870c4d09, PROCESSOR: 0:a06d0, TIME: 1708036462, SOCKET: 0, APIC: 0
kworker/0:7-2239 [000] .N... 434.266192: extlog_mem_event: {1} corrected error:
single-bit ECC physical addr: 0000000000008000 (mask lsb: ff) node:0 card:2 module:0 rank:0
bank:0 device:1 row:0 column:256 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnC, DIMM0
kworker/0:7-2239 [000] ..... 434.321341: mc_event: 1 Corrected error: memory read
error on CPU SrcID#0_MC#2_Chan#0_DIMM#0 (mc:2 location:0:0:-1 address:0x00008000 grain:32
syndrome:0x00000000_err_code:0x0080:0x0090 SystemAddress:0x8000 ProcessorSocketId:0x0
MemoryControllerId:0x2 ChannelAddress:0x2000 ChannelId:0x0 RankAddress:0x800
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x0 Column:0x100 Bank:0x0 BankGroup:0x0
ChipSelect:0x0)
```

6. Repeat the last step so you reach the SW row threshold counter.

```
>>> ei.mem.injectMemError(0x8000, errType="ce")
>>> go()
```

7. Confirm in serial log that the SW row threshold was reached.

```
[ECS] CheckAndHandleECSEvent Exit
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
[Updated] Policy bank level threshold: 3, row threshold: 2, time window: 24.
[Updated] UpdateEnhancedDimmErrRecord started.
[GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Rank:0x0 SubRank:0x0 Bank:0x0
BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x0 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
InterLocSyn:0x0
Check Error is in VLS record structure.
[Updated] Error is recorded in vls structure!
[Updated] UpdateVlsFailureSample Entry.
[Updated] CurrSWBank Counter/Bank_Threshold:2/3.
Check error is in PPR record structure.
[Updated] Error is recorded in sPPR structure!
[Updated] UpdatePprFailureSample Entry.
[Updated] CurrSWRow_Counter/Row_Threshold:2/2.
[Updated] Current Row Error Time=BBD26B, failure time=BBD26A
[Updated] Row threshold reached and in time window.
[Updated] ErrorThReached=1
[Updated] UpdateEnhancedDimmErrRecord ended.
```

8. Observe in serial log that BIOS invoked runtime soft PPR to complete the repair.

```
RasEventHndlrEntry index: 0x0
RasEventHndlrEntry index: 0x1
CheckAndHandleRtRowSparing: Node: 0
Socket = 0, Ch = 0, SubCh = 0, Rank = 0
IsSpprResourceAvailable: For MR54/55/56/57: 54 data FFFFFFFF
Adding PPR entry at Index= 0
[SSR] SSR state Save! Flag:0
[SSR] stop and save patrol scrub info
UpdatePPRSpareRegion: NodeId[0].McCh[0].R[0].SubR[0].BankBG[0].Row[0].FailedDevice[0x1]
CopyClSize[32].Loops[4]
UpdatePPRSpareRegion Status=0
gMst CPU number=30
RuntimeSPPRCopy Status=0
PrintPprEvent: Address-
Socket[0].MC[0].CH[0].D[0].PseudoCh[0].SubCh[0].R[0].CID[0].Bank[0].Row[0].FailDevMask = 0x2
ReportsPPRErrorLog Entry
CreateMemoryErrorRecord Entry
ERROR: C00000002:V03071009 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AC6AC10
LogErrorToPprVls BIOS ID: BHSDCRB1.86B.0022.D33.2304050137
LogErrorToPprVls++, CodeType: 2, Value:3071009
ErrorRecordHeader->SectionCount: 1
ErrorRecordDescription->SectionType: A5BC1114-6F64-4EDE-B863-3E83ED7C83B1
ErrorRecordHeader->RecordLength: 118
sizeof(MEM_ERROR_RECORD): 118, sizeof(EMCA_MEM_ERROR_MULTI_RECORD): 500
MemoryErrorData->ValidFields: 3C3F9
MemoryErrorData->Node: 0
MemoryErrorData->Card: 0
MemoryErrorData->ModuleRank: 0
MemoryErrorData->Bank: 0
MemoryErrorData->Device: 1
MemoryErrorData->Row: 0
MemoryErrorData->Row[17:16]: 0
MemoryErrorData->Column: 100
MemoryErrorData->RankNum: 0
MemoryErrorData->CardHandle: 15
MemoryErrorData->ModuleHandle: 0
LogErrorToPprVls--, Success
ReportsPPRErrorLog Exit,Success
[SSR] SSR state Save! Flag:1
[SSR] restore and restart patrol scrub info
```

```

EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
CheckAndHandleRtRowSparing Exit:
[GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Rank:0x0 SubRank:0x0 Bank:0x0
BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x0 SingleBit:0x0 Transfer:0x0 PersistentError:0x0
InterLocSyn:0x0
UpdateMcChanSsrState ch:1:PatrolSPAMode
McSsrState Node:1 :PatrolSPAMode
UpdateMcChanSsrState ch:6:PatrolSPAMode
McSsrState Node:6 :PatrolSPAMode
UpdateMcChanSsrState ch:7:PatrolSPAMode
McSsrState Node:7 :PatrolSPAMode
[imc] memory error handler end!
[ECS] CheckAndHandleECSEvent Enter
[ECS] CheckAndHandleECSEvent Exit
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 TO [Mca]ProcessMcBankError: Scope Uncore
S0 C0 TO [Mca]EmcaReadMcBankBitField: Scope Uncore, BitField 0x2000
S0 C0 TO [Mca]McBankBitField: 0x2000
S0 C0 TO [Mca]EmcaClearMcBankBitField: Scope Uncore, BitField 0x2000
S0 C0 TO [Mca]McBankErrorHandler: Inside the function
S0 C0 TO [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 TO [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 TO [Mca]McBankErrorHandler: IoMcaErrorHandler Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 TO [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 TO [Mca]MapErrorSeverity: MCE class is 0x5
S0 C0 TO [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 TO [Mca]McBankErrorHandler: Section Type = EMCA_SECTYPE_PLATMEM
FillPlatformMemoryErrorData Start!
S0 C0 TO [CpuRas]MC status 0x8C00004200800090, class CORRECTED
    
```

9. Verify in FTrace logs that an ELog event was recorded indicated by `extmem_log_event`.

```

kworker/0:7-2239 [000] ..... 687.252241: mce_record: CPU: 0, MCGc/s:
f000c20/0, MC15: 8c00004200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000000008000/0400080000008086/0000000000000000, RIP:
00:<0000000000000000>, TSC: 147e33a6ccb, PROCESSOR: 0:a06d0, TIME:
1708036835, SOCKET: 0, APIC: 0
kworker/0:7-2239 [000] .N... 687.268132: extlog_mem_event: {2}
corrected error: single-bit ECC physical addr: 0000000000008000 (mask lsb:
ff) node:0 card:2 module:0 rank:0 bank:0 device:1 row:0 column:256 DIMM
location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnC, DIMM0
kworker/0:7-2239 [000] ..... 687.323377: mc_event: 1 Corrected
error: memory read error on CPU_SrcID#0_MC#2_Chan#0_DIMM#0 (mc:2
location:0:0:-1 address:0x00008000 grain:32 syndrome:0x00000000
err_code:0x0080:0x0090 SystemAddress:0x8000 ProcessorSocketId:0x0
MemoryControllerId:0x2 ChannelAddress:0x2000 ChannelId:0x0 RankAddress:0x800
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x0 Column:0x100
Bank:0x0 BankGroup:0x0 ChipSelect:0x0)
    
```

10.8 Boot Time Post Package Repair (Boot Time PPR)

PPR is invoked at power up with MRC. It maps out what platform identifies as bad rows with spare rows in DDR DIMMs. The PPR mode provides a simple and easy repair method of the device after placed in the system. At least one row per bank group can be repaired. Maximum rows per bank group is vendor dependent. sPPR (Soft PPR) quickly and temporarily does a row repair, the soft repair will revert to the un-repaired state if DRAM power is removed or DRAM is reset. hPPR (Hard PPR) is a permanent

row repair method where the repair persists through power removal. For lab case uses, verify that soft PPR is enabled on the system, otherwise repairs done on rows through hPPR will be permanently lost.

10.8.1 Boot Time Post Package Repair (PPR) with Correctable Errors

Test Objective

Inject correctable errors to meet the hardware and software thresholds. Observe BIOS qualify lines for Post Package Repair (PPR). On reboot, observe that BIOS applies PPR to repair bad rows.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	A1
BIOS	WW05'24 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	4 DIMMs per socket
CScripts Version	686308 BHS CScripts GNR2406.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#)
- [FTrace and DMESG](#)
- [Enhanced MCA Logging \(ELOG\)](#) on page 28

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Runtime PPR > Row Sparing = Enable
 - This knob is not required to be enabled to test boot time PPR. It's only mentioned here because it was enabled when collecting the logs for this recipe.
 - Correctable Error Threshold = 1
 - Trigger SW Error Threshold = Enable
 - SW Per Row Threshold = 2
- EDKII > Socket Configuration > Memory Configuration >

- DDR PPR Type = Soft PPR
- Attempt Fast Boot = Enable
- Attempt Fast Cold Boot = Enable
 - If fast boot knobs are not enabled then BIOS will skip loading NVRAM and there'll be no PPR entries for boot time PPR.
- EDKII > Platform Configuration > System Event Log >
 - RAS Log Level = MID (BASIC_FLOW, FUNC_FLOW)

Recipe

1. Set up the system OS to capture FTrace/ELog record output per the ELOG BKM.

```
>>> halt()
>>> ei.utils.resetInjectorLockCheck()
```

2. Set the memDevs for a single-bit correctable error. Inject a correctable error then resume the system so BIOS can log the DIMM error record.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1msk=0x0)
dev0 = 0x1, dev0msk = 0x3, dev1 = 2, dev1msk = 0x0. Customized injectors may not work
correctly in RAS modes
>>> ei.mem.injectMemError(socket=0, channel=0, sub_channel=0, dimm=0, rank=0, sub_rank=0,
bank_group=0, bank=0, ps_ch=0, errType="ce")
>>> go()
```

3. Observe in serial log that the SW Row Threshold counter is at 1/2.

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
[Updated] Policy bank level threshold: 3, row threshold: 2, time window: 24.
[Updated] UpdateEnhancedDimmErrRecord started.
S1 C0 T1 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Rank:0x0 SubRank:0x0
Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0
PersistentError:0x0 InterLocSyn:0x0
Check Error is in VLS record structure.
AddNewVlsFailureSample Started.
[Updated] UpdateVlsFailureSample Entry.
[Updated] CurrSWBank_Counter/Bank_Threshold:1/3.
Check error is in PPR record structure.
AddNewPprFailureSample Started.
[Updated] UpdatePprFailureSample Entry.
[Updated] CurrSWRow_Counter/Row_Threshold:1/2.
[Updated] ErrorThReached=0
[Updated] UpdateEnhancedDimmErrRecord ended.
UpdateMcChanSsrState ch:1:PatrolSPAMode
McSsrState Node:1 :PatrolSPAMode
[imc] memory error handler end!
[ECS] CheckAndHandleECSEvent Enter
[ECS] Timestamp - skt:1 mc:0 ch:0 duration: 123 minutes
[ECS] Timestamp - skt:1 mc:1 ch:0 duration: 123 minutes
[ECS] CheckAndHandleECSEvent Exit
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]ProcessMcBankError: Scope Uncore
S0 C0 T0 [Mca]EmcaReadMcBankBitField: Scope Uncore, BitField 0x2000
S0 C0 T0 [Mca]McBankBitField: 0x2000
S0 C0 T0 [Mca]EmcaClearMcBankBitField: Scope Uncore, BitField 0x2000
S0 C0 T0 [Mca]McBankErrorHandler: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
```

```
S0 C0 T0 [Mca]McBankErrorHandler: IoMcaErrorHandler Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]MapErrorSeverity: MCE class is 0x5
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [Mca]McBankErrorHandler: Section Type = EMCA_SECTYPE_PLATMEM
FillPlatformMemoryErrorData Start!
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
FillPlatformMemoryErrorData: BankType:0x200
Error Source is M2M or IMC(MCCHAN)BankType:0x200
```

4. Verify an ELog record was generated indicated by extlog_mem_event.

```
kworker/0:2-2118 [000] ..... 727.379004: mce_record: CPU: 0, MCGc/s: f000c20/0, MC13:
8c00004200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000010008200/0400080008008086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
155d8cac6f7, PROCESSOR: 0:a06d0, TIME: 1708388046, SOCKET: 0, APIC: 0
kworker/0:2-2118 [000] ..... 727.379062: extlog_mem_event: {1} corrected error:
single-bit ECC physical addr: 0000000010008200 (mask lsb: ff) node:0 card:0 module:0 rank:0
bank:0 device:1 row:256 column:256 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0
kworker/0:2-2118 [000] ..... 727.379980: mc event: 1 Corrected error: memory read
error on CPU_SrcID#0_MC#0_Chan#0_DIMM#0 (mc:0 location:0:0:-1 address:0x10008200 grain:32
syndrome:0x00000000_err_code:0x0080:0x0090 SystemAddress:0x10008200 ProcessorSocketId:0x0
MemoryControllerId:0x0 ChannelAddress:0x4002000 ChannelId:0x0 RankAddress:0x1000800
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x100 Column:0x100 Bank:0x0 BankGroup:0x0
ChipSelect:0x0)
```

5. Repeat the last step so you reach the SW row threshold counter.

```
>>> ei.mem.injectMemError(socket=0, channel=0, sub_channel=0, dimm=0, rank=0, sub_rank=0,
bank_group=0, bank=0, ps_ch=0, errType="ce")
>>> go()
```

6. Confirm in serial log that the SW row threshold was reached.

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
[Updated] Policy bank level threshold: 3, row threshold: 2, time window: 24.
[Updated] UpdateEnhancedDimmErrRecord started.
S1 C10 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Rank:0x0
SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0
Transfer:0x0 PersistentError:0x0 InterLocSyn:0x0
Check Error is in VLS record structure.
[Updated] Error is recorded in vls structure!
[Updated] UpdateVlsFailureSample Entry.
[Updated] CurrSWBank_Counter/Bank_Threshold:2/3.
Check error is in PPR record structure.
[Updated] Error is recorded in sPPR structure!
[Updated] UpdatePprFailureSample Entry.
[Updated] CurrSWRow_Counter/Row_Threshold:2/2.
[Updated]Current Row_Error Time=BDE8A3, failure time=BDE8A0
[Updated] Row threshold reached and in time window.
[Updated] ErrorThReached=1
[Updated] UpdateEnhancedDimmErrRecord ended.
```

7. Observe in serial log that BIOS qualified the bad row for PPR.

```
LogErrorToPprVls++, CodeType: 2, Value:3071009
ErrorRecordHeader->SectionCount: 1
ErrorRecordDescription->SectionType: A5BC1114-6F64-4EDE-B863-3E83ED7C83B1
ErrorRecordHeader->RecordLength: 118
sizeof(MEM_ERROR_RECORD): 118, sizeof(EMCA_MEM_ERROR_MULTI_RECORD): 500
MemoryErrorData->ValidFields: 3C3F9
MemoryErrorData->Node: 0
MemoryErrorData->Card: 0
MemoryErrorData->ModuleRank: 0
MemoryErrorData->Bank: 0
MemoryErrorData->Device: 1
MemoryErrorData->Row: 100
```

```
MemoryErrorData->Row[17:16]: 0
MemoryErrorData->Column: 100
MemoryErrorData->RankNum: 0
MemoryErrorData->CardHandle: 15
MemoryErrorData->ModuleHandle: 0
[PPR] Error logged Address-
Socket[0].MC[0].CH[0].D[0].PseudoCh[0].SubCh[0].R[0].CID[0].Bank[0].Row[256].FailDevMask =
0x2.ErrorType[2]
[PPR] Error logged - Entry: 0
UpdatePprAddress: NodeId: 0, ChannelId: 0, DimmSlot: 0, DimmRank: 0, Bank: 0, Row: 100,
SubRank: 0, DramMask: 2, Value: 3071009, Success
LogErrorToPprVls--, Success
ReportsPPRErrorLog Exit,Success
[SSR] SSR state Save! Flag:1
[SSR] restore and restart patrol scrub info
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
CheckAndHandleRtRowSparing Exit:
S1 C10 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Rank:0x0
SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x100 Row:0x100 SingleBit:0x0 Transfer:0x0
PersistentError:0x0 InterLocSyn:0x0
UpdateMcChanSsrState ch:1:PatrolSPAMode
McSsrState Node:1 :PatrolSPAMode
[imc] memory error handler end!
```

8. Verify that an ELog record was generated indicated by extmem_event_log.

```
kworker/0:1-12058 [000] ..... 953.891103: mce_record: CPU: 0, MCGc/s: f000c20/0, MC13:
8c00004200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
0000000010008200/0400080008008086/0000000000000000, RIP: 00:<0000000000000000>, TSC:
1a4f4913677, PROCESSOR: 0:a06d0, TIME: 1708388273, SOCKET: 0, APIC: 0
kworker/0:1-12058 [000] ..... 953.891334: extlog_mem_event: {2} corrected error:
single-bit ECC physical addr: 0000000010008200 (mask lsb: ff) node:0 card:0 module:0 rank:0
bank:0 device:1 row:256 column:256 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0
kworker/0:1-12058 [000] ..... 953.892033: mc_event: 1 Corrected error: memory read
error on CPU SrcID#0_MC#0_Chan#0_DIMM#0 (mc:0 location:0:0:-1 address:0x10008200 grain:32
syndrome:0x00000000_err_code:0x0080:0x0090 SystemAddress:0x10008200 ProcessorSocketId:0x0
MemoryControllerId:0x0 ChannelAddress:0x4002000 ChannelId:0x0 RankAddress:0x1000800
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x100 Column:0x100 Bank:0x0 BankGroup:0x0
ChipSelect:0x0)
```

9. Reboot the system and verify PPR action in the serial log. Observe that the DramMask is a non-zero value indication detection/correction of correctable errors.

```
>>> itp.resettarget

[SkctId: 0] PPR Flow -- Started
N0.C00.D0.SC0.R0: PS0 PPR Addr Entry - SubRank: 0, Bank: 0, NibbleMask: 0x2, Row: 0x100
N0:
ExecutePpr - START
N0.C00.D0.SC0.R0: PS0 ExecutePPR - Row: 0x100, Bank: 0, CID: 0, DramMask: 0x00000002
N0.C00.D0.SC0.R0: ExecutePPR - Check PPR Resource
N0.C00.D0.SC0.R0: PS0 PPR Step 1 or 7 - Pre-charge before PPR
N0.C00.D0.SC0.R0: PPR Step 2 - Write MR23 to enable PPR
N0.C00.D0.SC0.R0: PS0 PPR Step 3 - Issue 4 MR24 guard key sequence
N0.C00.D0.SC0.R0: PS0 PPR Step 4,5,6 - Issue ACT and WRA for Row 0x100 Repaire
N0.C00.D0.SC0.R0: PS0 PPR Step 1 or 7 - Pre-charge before PPR
N0: PPR Step 8 - Wait tPGM_Exit(min) after PRE command
N0.D0.SC0.R0: PPR Step 9 - Clear MR23 to disable PPR
N0: ExecutePpr - END

Enhanced warning of type 31 logged:
Major Warning Code = 0x30, Minor Warning Code = 0x16,
Major Checkpoint: 0xB9
Minor Checkpoint: 0x05
Socket 0
Channel 0
SubChannel 0
Dimm 0
Rank 0
```

```

Memtest of type 255
Dat0S: 000000F0
Dat1S: 00000000
Dat2S: 000000F0
Dat3S: 00000000
EccS: 00000000
Chunk: 00000000
Column: 00000000
ColumnExt: 00000000
Row: 00000100
RowExt: 00000000
Bank: 00000000
Rank: 00000000
Subrank: 00000000
Count: 1
PprRepairStatus: Passed, Error Type: CE

```

10.8.2 PPR with Uncorrectable Errors

Test Objective

Inject an uncorrectable error and observe how BIOS qualifies the line for PPR. With a UC error, the system will reboot, observe within the serial log that PPR successfully repairs the UC error.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Granite Rapids, Sierra Forest, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	A1
BIOS	WW50'23 BKC
OS Version	CentOS Stream 8
Kernel	5.19.0-gnr.po.bkc.3.13.14.x86_64
HW Configuration	2 DIMMs per socket
CScripts Version	686308 BHS CScripts GNR2344.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#)
- [FTrace and DMESG](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration >
 - DDR PPR Type = Soft PPR
 - Force PPR On All DRAM For UCE = Enabled
 - Attempt Fast Boot = Enable

- Attempt Fast Cold Boot = Enable
 - If fast boot knobs are not enabled then BIOS will skip loading NVRAM and there'll be no PPR entries for boot time PPR.
- EDKII > Platform Configuration > System Event Log >
 - RAS Log Level = MID (BASIC_FLOW, FUNC_FLOW)
- EDKII > Platform Configuration > Miscellaneous Configuration >
 - Serial Debug Message Level = Normal

Recipe

1. Halt the system and reset the injectors.

```
>>> halt()
>>> ei.utils.resetInjectorLockCheck()
```

2. Inject an uncorrectable error then resume the system so the BIOS can log the DIMM error record. The system should reset on its own due to UC error.

```
>>> ei.mem.injectMemError(0x0, errType="uce")
>>> go()
```

3. Observe in serial log that BIOS qualified the bad row for PPR.

```
LogErrorToPprVls BIOS ID: BHSDCRB1.86B.2833.D02.2312010054
LogErrorToPprVls++, CodeType: 2, Value:3071008
ErrorRecordHeader->SectionCount: 1
ErrorRecordDescription->SectionType: A5BC1114-6F64-4EDE-B863-3E83ED7C83B1
ErrorRecordHeader->RecordLength: 500
sizeof(MEM_ERROR_RECORD): 118, sizeof(EMCA_MEM_ERROR_MULTI_RECORD): 500
MemoryErrorData->ValidFields: 3C3FB
MemoryErrorData->Node: 0
MemoryErrorData->Card: 0
MemoryErrorData->ModuleRank: 0
MemoryErrorData->Bank: 0
MemoryErrorData->Device: 0
MemoryErrorData->Row: 0
MemoryErrorData->Row[17:16]: 0
MemoryErrorData->Column: 0
MemoryErrorData->RankNum: 0
MemoryErrorData->CardHandle: 15
MemoryErrorData->ModuleHandle: 0
[PPR] Error logged Address-
Socket[0].MC[0].CH[0].D[0].PseudoCh[0].SubCh[0].R[0].CID[0].Bank[0].Row[0].FailDevMask = 0x0.ErrorType[1]
[PPR] Error logged - Entry: 0
PPR: PPR index: 0
UpdatePprAddress: NodeId: 0, ChannelId: 0, DimmSlot: 0, DimmRank: 0, Bank: 0, Row: 0, SubRank: 0, DramMask: 0, Value: 3071008, Success
LogErrorToPprVls--, Success
...
```

4. The system should now have rebooted, then observe that a DDR PPR request is present in the serial log; fast boot is disabled now.

```
Socket: 0, Machine Check Bank 1: Status: 0xBD80000000100134, Address:
0x0000000000000000, Misc: 0x00000000000000086
START_MRC_RUN
New DDR PPR request found. Disable fast boot
N0: I3C Instance 3: SETAASA - Failed!
N0: I3C Instance 5: SETAASA - Failed!
N0: I3C Instance 3: RSTDAA - Failed!
N0: I3C Instance 5: RSTDAA - Failed!
```

```

N0: setupChanged: 1
N0: Clearing the MRC NVRAM structure.
N0: bootMode = NormalBoot
N0: subBootMode = WarmBoot --> ColdBoot
bootMode = NormalBoot
subBootMode = ColdBoot
[SketId: 0] Parallel Mode Dispatch -- Started
Dispatch N1 for MemInit
N1 Entering MRC
N1: Detect ColdBootSlowRequiredFlag and will force slow cold boot.

```

5. Observe in serial log that PPR took action on the UC error and that the PprRepairStatus passed.

```

[SketId: 0] PPR Flow -- Started

N0.C00.D0.SC0.R0.PS0: PPR Addr Entry - SubRank: 0, Bank: 0, NibbleMask: 0x0, Row: 0x0
N0:
ExecutePpr - START
N0.C00.D0.SC0.R0: ExecutePPR - Input DramMask is 0x00 but cannot detect the failed device,
forcing it as:0x000001FF
N0.C00.D0.SC0.R0.PS0: ExecutePPR - Row: 0x0, Bank: 0, CID: 0, DramMask: 0x000001FF
N0.C00.D0.SC0.R0: ExecutePPR - Check PPR Resource
N0.C00.D0.SC0.R0.PS0: PPR Step 1 or 7 - Pre-charge before PPR
N0.C00.D0.SC0.R0: PPR Step 2 - Write MR23 to enable PPR
N0.C00.D0.SC0.R0.PS0: PPR Step 3 - Issue 4 MR24 guard key sequence
N0.C00.D0.SC0.R0.PS0: PPR Step 4,5,6 - Issue ACT and WRA for Row 0x0 Repaire
N0.C00.D0.SC0.R0.PS0: PPR Step 1 or 7 - Pre-charge before PPR
N0: PPR Step 8 - Wait tPGM_Exit(min) after PRE command
N0.D0.SC0.R0: PPR Step 9 - Clear MR23 to disable PPR
N0: ExecutePpr - END

Enhanced warning of type 31 logged:
Major Warning Code = 0x31, Minor Warning Code = 0x16,
Major Checkpoint: 0xB9
Minor Checkpoint: 0x05
Socket 0
Channel 0
SubChannel 0
Dimm 0
Rank 0
Memtest of type 255
Dat0S:      FFFFFFFF
Dat1S:      00000000
Dat2S:      FFFFFFFF
Dat3S:      00000000
EccS:       0000000F
Chunk:      00000000
Column:     00000000
ColumnExt:  00000000
Row:        00000000
RowExt:     00000000
Bank:       00000000
Rank:       00000000
Subrank:    00000000
Count:      1
PprRepairStatus: Passed, Error Type: UCE
[SketId: 0] PPR Flow - 153ms

```

10.9 PMIC

When the system supports DDR5 PMIC, BIOS will support DIMM Failure Isolation during BIOS MRC. The DDR5 power fail isolation procedure is divided into two phases:

1. Phase 1 - DIMM Discovery, isolate bad PMIC and disable related DIMM channel.

2. Phase 2 - After VR Enable, check the status to ensure all PMICs expected to power-on the DIMM can do so successfully.

BIOS should detect the faulty PMIC and perform following steps:

1. Floats CAMP signal to prevent PMIC interference

NOTE

After PMIC Spec Rev 1.4, it re-labeled PWR_GOOD pin with CAMP (Control AND Monitor Port)

2. Disable all the DIMMs in the same channel if faulty PMIC is detected.

10.9.1 PMIC Failure Isolation

Test Objective

Inject a critical temperature shutdown event to the DDR5 PMIC during runtime. After AC cycling BIOS should isolate the DIMM failure and show an EWL for the critical temperature power failure and more EWLs for each of the ranks on the channel that are disabled. System should successfully boot with all the DIMMs on the target channel disabled; target channel being the channel which contains the target DIMM.

NOTE

PMIC error injection behavior will depend on PMIC error registers behavior. PMIC error injection may vary dependent on the PMIC vendor. If PMIC is not following the PMIC spec to write the error register, the error from the injection might not be captured by BIOS.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6700-Series Processor with E-Cores C step
BIOS	WW39'24 BKC
OS Version	n/a - tested at UEFI shell
Kernel	n/a - tested at UEFI shell
HW Configuration	4 DIMMs per socket - 64GB dual rank DIMMs
CScripts Version	BHS 2448.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration
 - Memory Reference Code (MRC) Promote Warnings = Disable
 - Promote Warnings = Disable
 - If promotion knobs are enabled then BIOS will not disable the DIMMs on the channel and continue booting as mentioned previously but will instead consider the bad PMIC as a fatal error and hang the system.
- EDKII > Socket Configuration > Memory Configuration > Memory DFX Configuration
 - Power Management Integrated Circuit (PMIC) Failure Isolation = Enable
 - PMIC Secure Mode = Disable
- Socket Configuration > Memory Configuration >
 - Enforce Population POR = Disable
 - Disabling this knob is not required but was disabled because a non-POR memory population was used. It is highly recommended to have a POR memory population.

Recipe

1. Verify that the target PMIC has no history of errors or currently logged errors. Following values should all be zero.

```
In : ras.mem.PMICErrorStatus(socket=0, mc=0, channel=0, slot=0)
PMIC register: 0x4
```

Field	Value	Description
R04 [7]: GLOBAL_ERROR_COUNT Global Error Count Since Last Erase Operation	0x0	No Error or Only 1 Error since last Erase operation
R04 [6]: GLOBAL_ERROR_LOG_BUCK_OV_OR_UV Global Error Log History for Buck Regulator Output Over or Under Voltage	0x0	No Error Occurred
R04 [5]: GLOBAL_ERROR_LOG_VIN_BULK_OVER_VOLTAGE Global Error Log History for VIN_Bulk Over Voltage	0x0	No Error Occurred
R04 [4]: GLOBAL_ERROR_LOG_CRITICAL_TEMPERATURE Global Error Log History for Critical Temperature	0x0	No Error Occurred

```
PMIC register: 0x5
```

Field	Value	Description
R05 [6]: SWA_POWER_GOOD PMIC Power On - SWA Power Not Good	0x0	Normal Power On
R05 [5]: SWB_POWER_GOOD PMIC Power On - SWB Power Not Good	0x0	Normal Power On
R05 [4]: SWC_POWER_GOOD PMIC Power On - SWC Power Not Good	0x0	Normal Power On
R05 [3]: SWD_POWER_GOOD PMIC Power On - SWD Power Not Good	0x0	Normal Power On
R05 [2:0]: PMIC_ERROR_LOG PMIC Power On - High Level Status Bit to Previous Last Known Power Cycle	0x0	Normal Power On

```
PMIC register: 0x6
```

Field	Value	Description
R06 [7]: SWA_UNDER_VOLTAGE_LOCKOUT PMIC Power On - SWA Under Voltage Lockout	0x0	Normal Power On
R06 [6]: SWB_UNDER_VOLTAGE_LOCKOUT PMIC Power On - SWB Under Voltage Lockout	0x0	Normal Power On
R06 [5]: SWC_UNDER_VOLTAGE_LOCKOUT PMIC Power On - SWC Under Voltage Lockout	0x0	Normal Power On
R06 [4]: SWD_UNDER_VOLTAGE_LOCKOUT PMIC Power On - SWD Under Voltage Lockout	0x0	Normal Power On
R06 [3]: SWA_OVER_VOLTAGE PMIC Power On - SWA Over Voltage	0x0	Normal Power On
R06 [2]: SWB_OVER_VOLTAGE PMIC Power On - SWB Over Voltage	0x0	Normal Power On
R06 [1]: SWC_OVER_VOLTAGE PMIC Power On - SWC Over Voltage	0x0	Normal Power On
R06 [0]: SWD_OVER_VOLTAGE PMIC Power On - SWD Over Voltage	0x0	Normal Power On

- Inject a "Critical Temperature Shutdown" on the target PMIC by writing 0x82 to register 0x35. Python exception print statements are expected with the CScripts version used and were omitted from the logs below. PMIC behavior is not uniform across all PMICs so the CScripts command below should check the PMIC vendor ID and revision for known problematic PMICs. See note.

NOTE

Some observations show that the PMIC error injection behavior may vary dependent on the PMIC vendor. For example, some PMIC vendors or specific PMIC date code will not write the error data into the Non-Volatile Memory (NVM) error registers (for example, R04, R05, R06, R07) right after the error injection. In that case, the error reflected/logged in the Volatile-Memory registers (for example, R08, R09, R0A, R0B) would be cleared by a PMIC reset (or power cycle) and might not be able to be captured by BIOS. (For more detail information, contact your DIMM/PMIC vendor for assistance.

```
In : ei.mem.injectPMICError(socket=0, mc=0, channel=0, slot=0, raw_value=0x82, verbose=True)
PMIC Vendor ID: 0xb380
PMIC Revision: 0x19
```

<PYTHON EXCEPTIONS WERE OMITTED>

```
TargetEvent: PowerDomain : CPU : Off      -- 19:09:14.210762 2024-12-10
TargetEvent: PowerDomain : S0 : Off       -- 19:09:14.212757 2024-12-10
TargetEvent: Reset : Started              -- 19:09:14.238323 2024-12-10
TargetEvent: Power : Off                  -- 19:09:14.242805 2024-12-10
TargetEvent: PowerDomain : PCH : Off      -- 19:09:14.242805 2024-12-10
TargetEvent: Reset : Started              -- 19:09:14.263160 2024-12-10
```

- At this point, the system has shut off. AC power cycle the system and analyze the BIOS serial log as described in the follow steps.
- As seen in the BIOS serial log, BIOS found PMIC register 0x05 to be non-zero and 0x3 means it was a "Critical Temperature" error. An EWL of type 1 is logged with a major/minor warning code of 0x46/0x5 which decodes to WARN_POWER_FAILURE/WARN_DDR5_POWER_FAILURE_CRIT_TEMP.

```
N0.C00.D0: Bad Pmic Power status in phase 1! R05 = 0x3
```

```
Enhanced warning of type 1 logged:
Major Warning Code = 0x46, Minor Warning Code = 0x05,
Major Checkpoint: 0xB0
Minor Checkpoint: 0x02
Socket 0
Channel 0
Dimm 0
N0.C00.D0: FPT: DisableDIMM()
N0.C00.D0.R0: FPT: DisableRank()
```

5. Also shown in the BIOS serial log, note that further EWLs of type 1 were logged for each rank on the target channel since they're all disabled. The major/minor warning code of 0xA/0x3 decodes to WARN_USER_DIMM_DISABLE/
WARN_USER_DIMM_DISABLE_RANK_DISABLED.

```
Enhanced warning of type 1 logged:
Major Warning Code = 0x0A, Minor Warning Code = 0x03,
Major Checkpoint: 0xB0
Minor Checkpoint: 0x02
Socket 0
Channel 0
Dimm 0
Rank 0
N0.C00.D0.R1: FPT: DisableRank()
```

```
Enhanced warning of type 1 logged:
Major Warning Code = 0x0A, Minor Warning Code = 0x03,
Major Checkpoint: 0xB0
Minor Checkpoint: 0x02
Socket 0
Channel 0
Dimm 0
Rank 1
N0.C00.D1: FPT: DisableDIMM()
N0.C00.D1.R0: FPT: DisableRank()
```

```
Enhanced warning of type 1 logged:
Major Warning Code = 0x0A, Minor Warning Code = 0x03,
Major Checkpoint: 0xB0
Minor Checkpoint: 0x02
Socket 0
Channel 0
Dimm 1
Rank 0
N0.C00.D1.R1: FPT: DisableRank()
```

```
Enhanced warning of type 1 logged:
Major Warning Code = 0x0A, Minor Warning Code = 0x03,
Major Checkpoint: 0xB0
Minor Checkpoint: 0x02
Socket 0
Channel 0
Dimm 1
Rank 1
```

6. Also note that the active memory has been reduced since the two 64GB DIMMs on socket 0 channel 0 have been disabled.

```
=====
START_DIMMINFO_SYSTEM_TABLE
=====
| Socket 0 | Socket 1 | Socket 2 | Socket 3 | System |
=====
```

Active Memory	128GB	256GB	N/A	N/A	384GB
DDR Freq					DDR5-5200
Ch0 CL-RCD-RP-CMD		42-42-42-1n			
Ch1 CL-RCD-RP-CMD	42-42-42-1n	42-42-42-1n			
DDR Vdd	1.115V	1.115V	N/A	N/A	
ECC Checking					On
Patrol/Demand Scrub					Off
RAS Mode					Indep
Paging Policy					Adapt Open
Data Scrambling					On
CCMRC Revision					00.50.00
RC Version					2.0.2.000B

=====

STOP_DIMMINFO_SYSTEM_TABLE

=====

- The system should successfully boot.
- Check the target DIMMs PMIC registers and confirm that register 0x4 is equal to 0x10 indicating that this PMIC has saved the critical temperature event to its error log history. Also confirm that register 0x5's PMIC_ERR_LOG bitfield indicates that there was a critical temperature event.

```
In : ras.mem.PMICErrorStatus(socket=0, mc=0, channel=0, slot=0)
PMIC register: 0x4
```

Field	Value	Description
R04 [7]: GLOBAL_ERROR_COUNT Global Error Count Since Last Erase Operation	0x0	No Error or Only 1 Error since last Erase operation
R04 [6]: GLOBAL_ERROR_LOG_BUCK_OV_OR_UV Global Error Log History for Buck Regulator Output Over or Under Voltage	0x0	No Error Occurred
R04 [5]: GLOBAL_ERROR_LOG_VIN_BULK_OVER_VOLTAGE Global Error Log History for VIN_Bulk Over Voltage	0x0	No Error Occurred
R04 [4]: GLOBAL_ERROR_LOG_CRITICAL_TEMPERATURE Global Error Log History for Critical Temperature	0x1	Error Occurred

```
PMIC register: 0x5
```

Field	Value	Description
R05 [6]: SWA_POWER_GOOD PMIC Power On - SWA Power Not Good	0x0	Normal Power On
R05 [5]: SWB_POWER_GOOD PMIC Power On - SWB Power Not Good	0x0	Normal Power On
R05 [4]: SWC_POWER_GOOD PMIC Power On - SWC Power Not Good	0x0	Normal Power On
R05 [3]: SWD_POWER_GOOD PMIC Power On - SWD Power Not Good	0x0	Normal Power On
R05 [2:0]: PMIC_ERROR_LOG PMIC Power On - High Level Status Bit to Previous Last Known Power Cycle	0x3	Critical Temperature

- To bring the system back to a good working state it's recommended to clear the target DIMMs PMIC error registers by doing the following command and then reflashing the IFWI or clearing CMOS since otherwise BIOS will track the disabled DIMMs on subsequent boots and continue to disable them.

```
In : ras.mem.clearPMICErrorStatus(socket=0, mc=0, channel=0, slot=0, verbose=True)
```

When '1' is written to any of the clear registers, the PMIC updates the status registers to default state and removes the interrupt condition on GSI_n and PWR_GOOD output signal assuming that event is no longer present. If the failing condition is still present, the status register will still remain at '1'

```

Clearing registers: [0x04, 0x05, 0x06]...
Writing 1 to registers: ['0x10', '0x11', '0x12', '0x13', '0x14']
Checking if Status register value are zero...
0x8 register value is 0x0, register was successfully cleared
0x9 register value is 0x0, register was successfully cleared
0xa register value is 0x0, register was successfully cleared
0xb register value is 0x0, register was successfully cleared
0x33 register value is 0x0, register was successfully cleared
=====
| STAGE          | CHECKPOINT                                     | STATUS |
|-----|-----|
| Verification | Halt Check                                   | Pass   |
| Verification | Validate Parameters                         | Pass   |
| Verification | Write Protected Mode Disabled Check       | Pass   |
| Verification | Error Clear                               | Pass   |
=====

```

10.10 SMBus Hang Recovery

Memory SMBus Hang Recovery allows system recovery in case SMBus fails to respond during run-time thus preventing system crash. UEFI FW is notified via an SMI and it may be able to recover the SMBus by resetting and re-activating the link. Memory SMBus is actively used by the processor for thermal monitoring and implementing CLTT.

10.10.1 SMBus Hang Recovery

Test Objective

Invoke a hang condition on the I3C controller, and allow BIOS to use SMI to perform an SMBus error recovery during runtime improving system reliability and data integrity.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW03'24 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	4 DIMMs per socket
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [Enhanced MCA Logging \(ELOG\)](#) on page 28

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Processor Configuration >
 - Smbus Error Recovery = SMI

Recipe

1. Halt the system and reset the injectors.

```
>>> halt()
>>> ei.utils.resetInjectorLockCheck()
```

2. Verify that bit 9 (transfer_err_stat) is set to 0 prior to injection in the intr_status register.

```
>>> sv.socket0.compute0.uncore.spdi3cbus.spdi3cbus0.sb_i3c_0.intr_status.show()
0x00000000 : reserved_16_31 (31:16) --
0x00000000 : bus_reset_done_stat (15:15) --
0x00000000 : ppoll_cmd_miss_stat (14:14) --
0x00000000 : reserved_11_13 (13:11) --
0x00000000 : hc_internal_err_stat (10:10) --
0x00000000 : transfer_err_stat (09:09) --
0x00000000 : reserved_6_8 (08:06) --
0x00000000 : transfer_abort_stat (05:05) --
0x00000000 : resp_ready_stat (04:04) --
0x00000001 : cmd_queue_ready_stat (03:03) --
0x00000000 : ibi_status_thld_stat (02:02) --
0x00000000 : rx_thld_stat (01:01) --
0x00000001 : tx_thld_stat (00:00) --
```

3. Inject an error to the I3C* controller. Error type 0x13 is for a HALT error.

```
ei.sys.injectI3CError(socket=0, spdi3c=0, err_type=0x13)
```

4. Verify that the error injection passed.

```
Checking for healthy i3c controllers on socket0
Discovering Populated Mcs
Discovering Populated Channels
Discovering Channels
WARNING:socket0 compute0 spdi3cbus0 Interrupt enable bits are not set correctly (expecting 0x463f)!

===== SPD Controller State =====
=====|
Status      |      Speed (Khz)      | Ppoll  ||      |      | Mstr|Bus Pin|      |      | Intr
| State |poll| hc|trf|trf|rsp|cmd|ibi| rx| tx|      | I3C   | I2C   | Cmd   ||      |      | Trf
Name      |En |Mode |State|SDA|SCL|miss|err|err|abt|rdy|rdy|thd|thd|thd|  pp  |  od  |
od |  En
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 0 | 1 | 0 | 0 | 1 | 9090 | 1000 | 1000 | 0x1515 |
=====

STEP1: PASS
STEP2: Inject error
socket0.spdi3cbus0 in Halt state.
STEP2: PASS
STEP3: Check if error was injected
ERROR: socket0 compute0 spdi3cbus0 transfer state machine is in Halt (0x13) state!
ERROR: socket0 compute0 spdi3cbus0 Unexpected interrupt bits asserted.
```

```
WARNING:socket0 compute0 spd3c3bus0 Interrupt enable bits are not set correctly (expecting
0x463f)!

SPD Controller State
=
|
| Speed (Khz)      | Ppoll |      |      | Mstr |Bus Pin|      | Intr Status      |
|
| tx|      I3C      | I2C   | Cmd  |      | Trf   | State |poll| hc|trf|trf|rsp|cmd|ibi| rx|
|
|      Name        |En |Mode |State |SDA|SCL|miss|err|err|abt|rdy|rdy|thd|thd|thd|
pp | od | od | En |
-----
|socket0 compute0 spd3c3bus0| 1 | i3c | 0x13 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 |
9090 | 1000 | 1000 | 0x1515 |
=====
PASS: Error type 0x13 was injected to target socket0 successfully!
STEP3: PASS
```

5. Verify that bit 9 (transfer_err_stat) is set in the intr_status register. This bit is set to 1'b1 when any transfer error occurs on the I3C bus.

```
>>> sv.socket0.compute0.uncore.spd3c3bus.spd3c3bus0.sb_i3c_0.intr_status.show()
0x00000000 : reserved_16_31 (31:16) --
0x00000000 : bus_reset_done_stat (15:15) --
0x00000000 : ppoll_cmd_miss_stat (14:14) --
0x00000000 : reserved_11_13 (13:11) --
0x00000000 : hc_internal_err_stat (10:10) --
0x00000001 : transfer_err_stat (09:09) --
0x00000000 : reserved_6_8 (08:06) --
0x00000000 : transfer_abort_stat (05:05) --
0x00000000 : resp_ready_stat (04:04) --
0x00000001 : cmd_queue_ready_stat (03:03) --
0x00000000 : ibi_status_thld_stat (02:02) --
0x00000000 : rx_thld_stat (01:01) --
0x00000001 : tx_thld_stat (00:00) --
```

6. Resume the system and verify in the serial log output that I3C Bus recovery occurred.

```
>>> go()
Socket:0x0 BiosLockByPassScratchPad8:0x3
SmbInstance for Index 0 is 0
Pcode kick off I3C Bus recovery on Socket:0x0, Instance:0x0
Semaphore is acquired on SktId: 0x0
SaveTSODPolling on Socket:0x0, Inst:0x0 - TsodCmd:0x1515, DisPoll:0x0
Resetting I3C BUS on Socket:0x0, Inst:0x0
starting PeriodicTimer
PeriodicTimer SMI is checking I3C bus reset done...
I3C BUS Reset Done Event on Socket 0x0, Instance 0x0
RestoreTsodPolling on Socket:0x0, Inst:0x0 - TsodCmd:0x1515, DisPoll:0x0
Release Semaphore
Socket:0x0 BiosLockByPassScratchPad8:0x1
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AEF8610
WHEA: Detected Ras Non Standard Error
```

11.0 CXL RAS

11.1 CXL 101

CXL is a relatively new technology. Keep in mind that because this is a new technology. The OS, Kernel, and tools will be updated frequently. The version of the OS and tools used in this chapter may be outdated. The most popular CXL devices in the industry is CXL Type 3 devices or CXL memory (CXL.mem) devices. This will be the card type used in this chapter.

Recipes in this section are done using Linux. For the time being, Intel recommends using the "latest stable" upstream Kernel or a Kernel with all the CXL patches from "latest" upstream Kernel. The main CXL tools are from the ndctl package, which encompasses tools and libraries for CXL, NVDIMM, DAX, memory tiering and other platform memory device topics. Also recommend using the latest version of ndctl package possible. The key package names from ndctl are "cxl-cli" and "daxctl". Fixes and enhancements are ongoing.

Next is a small list of website that may be helpful in understanding CXL:

- NDCTL Github for latest build and availability: <https://github.com/pmem/ndctl>
- PMEM.io is the home for NDCTL:
 - <https://docs.pmem.io/ndctl-user-guide/>
 - <https://pmem.io/blog/2023/05/exploring-the-software-ecosystem-for-compute-express-link-cxl-memory/>

Normally you should go through your linux distribution's package manager, but if that fails you can find generic instructions on how to install NDCTL here: <https://docs.pmem.io/ndctl-user-guide/installing-ndctl>.

Some NDCTL/Linux distro status:

- Fedora*.
 - The latest ndctl package is always available.
- CentOS* Stream 8/9.
 - See "Fedora CPOR": <https://copr.fedorainfracloud.org/coprs/djbw/ndctl/> for a [repo download](#)
- Other Linux distros:
 - Contact your OS distributor or compile ndctl from the previous GitHub link.
 - You can find instructions for installing from source here: <https://docs.pmem.io/ndctl-user-guide/installing-ndctl/installing-ndctl-from-source-on-linux>

Key Terminology

Terminology	Definition
System Physical Address (SPA)	Hardware addresses of physical memory
Host Physical Address (HPA)	On Intel platforms HPA is SPA.
Device Physical Address (DPA)	DPA forms a device-scoped flat address space. An LD-FAM device presents a distinct DPA space per LD. A G-FAM device presents the same DPA space to all hosts. The CXL HDM decoders or GFD decoders map HPA into DPA space.
BDF	Bus Device Function number, basic "address" for a PCIe/CXL device
Component Command Interface (CCI)	A CXL CCI represents a command target that is used to process management and configuration commands. In this chapter, mostly CCIs are issued through the primary mailbox.
Memory Error Firmware Notification (MEFN)	An EFN is used to report memory errors.
Vendor Defined Message (VDM)	These are message that are vendor specific, or not documented in the PCIe spec. CXL MEFN are PCIe VDMs.
Virtual Address (VA)	Virtual addresses refer to the virtual store viewed by the process.
Target Physical Address	Address that the test is trying to read or write in the 2LM flows that will trigger errors.
Alias Physical Address	In Intel Flat Memory Mode, 2 SPA addresses can occupy the same DPA. An alias address is paired address from SPA.
Intel Flat Memory Mode	Mode where local attached DDR and CXL DDR are in a 1:1 ratio in a exclusive cache environment. Where the DDR acts like near memory and CXL acts like far memory.
1LM + Vol Mode	Local attached DDR and CXL DDR are assigned different address spaces with different numa nodes.
Hetero Mode	Heterogeneous interleave is the interleaving of memory requests across a combination of the local attached DDR and CXL DDR.
Cache	A hardware component that stores data so that future requests for that data can be served faster.
Cache Hit	When a search in a cache for an address is found.
Cache Miss	When a search in a cache for an address is not found.
Cache Line	The unit of memory that is copied to and is individually tracked in a cache. Specifically, 64 bytes of data or instructions aligned on a 64-byte physical address boundary.
Cache Line Read	A read to a cache where data size and address aligns with the cache.
Cache Line Write	A write to a cache where data size and address aligns with the cache line.
Dirty/Modified Cache Line	If a cache line has been modified and needs to be flushed back to lower memory, it is called "dirty".
Partial Cache Line Write	A write to a cache where data size is less than a cache line.
System RAM mode	CXL memory is part as OS main memory.
devdax mode	"Device DAX" provides a means for applications to directly access storage, without the involvement of a file system. CXL memory is exposed in the OS as a /dev/dax* node. "devdax" is not part of the system ram mode.

11.1.1 Finding Address

For many of the CXL RAS tests, you will need to find an address associated with a CXL target card.

Once you know the BDF of the device, there are few methods to find what address are associated with the CXL device.

Type of memory configuration

CXL Type 3 cards can be mapped into three different modes:

- 1LM + Volatile [1LM+Vol]
- Heterogeneous [Hetero]
- Intel Flat Memory Mode

From a RAS flow perspective, 1LM+Vol and Hetero are the same. One can use either mode for your tests. What differs is address translation. Intel Flat Memory Mode have significantly different error flows.

Using CScripts' `cxl.addMap()`

A simple way to find out the address range of a CXL device is to use CScripts' `cxl.addMap()`.

```
>>> cxl.topology()
Halting for function execution...

[GPC core group] Halt on 72 devices -- 12:27:30.778069 2023-10-09
=====
|Socket|PCIe Port |Dev BDF |Config as|Width|Speed|ASPM En|io linkSts|Vendor|VendorID|DeviceID|CXL Version|CXL Type|
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
|1|2a/pxp2.rp0|0xa8,0,0|CXL|x16|Gen5|no|Up.L0|0xldfa|ldfah|01e0h|2.0|Type 3|
=====

>>> cxl.addMap()
=====
|SKT|PORT |CXL TYPE|DECODER|BASE |LIMIT |SPAN |WAYS |GRANULARITY|
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
|1|pxp2.rp0|Type 3|0|0x3ce40000000|0x3d63ffffffff|32.0 GB|1 way|256 Bytes|
=====
```

Using CXL CLI

Use `cxl list` to see the Port, Mem Device, and Decoder to find basic information. For example, "`cxl list -PMDu`".

If the OS sees a CXL device and it has been configured, you should see a port with a "`decoder`" and "`memdev`".

- Under "`decoders`", "`resource`" is the base SPA to this device and "`dpa_address`" is where the SPA maps to on this device's DPA. For example:
 - "`resource`": "0x30e40000000",
 - "`dpa_resource`": "0",
- Under "`memdevs`", find the root port's BDF under "`host`". For example:
 - "`host`": "0000:a8:00.0"

Use `cxl list` to see the Region and Dax to find DAX memory mode and NUMA proximity domain. For example, "`cxl list -RXu`".

Find the region where the "`resource`" matches the previous settings.

- Under "`daxregion`", "`devices`", "`mode`" is the describing if memory mode is "`system-ram`" or "`devdax`". For example:
 - "`mode`": "system-ram"

- Under *"daxregion"*, *"devices"*, *"target_node"* is the memory proximity domain. For example:
 - *"target_node":2*

In the next example you will find:

- BDF of CXl device is ae:0.0 or BUS=0xae, Device=0x0, Function=0x0.
- SPA Start is 0x30e40000000, which is mapped to DPA 0x0.
- Size is 32 GiB.
- CXl is configured as *"system-ram"*.
- Memory Proximity Domain is 2.

```
# cxi list -PMDu
[
  {
    "ports": [
      {
        "port": "port4",
        "host": "pci0000:a7",
        "depth": 1,
        "decoders:port4": [
          {
            "decoder": "decoder7.0",
            "resource": "0x30e40000000",
            "size": "32.00 GiB (34.36 GB)",
            "interleave_ways": 1,
            "region": "region6",
            "dpa_resource": "0",
            "dpa_size": "32.00 GiB (34.36 GB)",
            "mode": "ram"
          },
          {
            "decoder": "decoder4.0",
            "resource": "0x30e40000000",
            "size": "32.00 GiB (34.36 GB)",
            "interleave_ways": 1,
            "region": "region6",
            "nr_targets": 1
          }
        ]
      },
      {
        "memdevs:port4": [
          {
            "memdev": "mem0",
            "ram_size": "32.00 GiB (34.36 GB)",
            "serial": "0x2023653c044aed1d",
            "numa_node": 1,
            "host": "0000:a8:00.0"
          }
        ]
      }
    ]
  },
  ...
]

# cxi list -RXu
{
  "region": "region6",
  "resource": "0x30e40000000",
  "size": "32.00 GiB (34.36 GB)",
  "type": "ram",
  "interleave_ways": 1,
  "interleave_granularity": 256,
  "decode_state": "commit",
  "daxregion": {
    "id": 6,
    "size": "32.00 GiB (34.36 GB)",
    "align": 2097152,
    "devices": [
      {
        "chardev": "dax6.0",
        "size": "32.00 GiB (34.36 GB)",
        "target_node": 2,
        "align": 2097152,
        "mode": "system-ram",
        "online_memblocks": 15,
        "total_memblocks": 15,
        "movable": true
      }
    ]
  }
}
```

```

}
}

# dmesg | grep ACPI | grep SRAT
[ 0.022653] ACPI: SRAT 0x0000000077772000 019118 (v03 INTEL INTEL ID 00000002 INTL 01000013)
[ 0.022721] ACPI: Reserving SRAT table memory at [mem 0x77772000-0x7778b117]
[ 0.023188] ACPI: SRAT: Node 0 PXM 0 [mem 0x00000000-0x7fffffff]
[ 0.023191] ACPI: SRAT: Node 0 PXM 0 [mem 0x10000000-0x87fffffff]
[ 0.023194] ACPI: SRAT: Node 1 PXM 1 [mem 0x3064000000-0x30e3fffffff]
[ 0.023302] ACPI: SRAT: Node 2 PXM 8 [mem 0x30e4000000-0x3163fffffff]

```

Using daxctl

Use *daxctl list* to see Region, Device, and Mapping. For example, "*daxctl list -RDMu*".

If the OS sees a CXL device and it has been configured as 1lm+vol, one should see one or more regions with a device in "devdax" or "system-ram" mode.

For the purpose of RAS testing and creating a "target app" that uses CXL memory:

- If the device is in devdax mode, applications will typically only access CXL memory via `/dev/dax*`. Devdax memory is **not** part of the general system memory pool.
- If the device is in system-ram mode, applications may access CXL memory via standard allocation methodology. If you configure CXL memory from devdax to system-RAM mode, you will see an increase in "free" memory. System-RAM mode memory will be in its own numa node and proximity domain, which allows you to bind an application to that numa domain and force it to use CXL memory versus native DDR memory.

In the next example (using 1lm+vol) you will find:

- 2 regions:
 - 1 in devdax "mode", but disabled "state".
 - 1 in system-ram "mode", but active.
- dev0.0 has a "size" of 254.74 GiB.
- "start" is at 0x2080000000, matching the address in "cxl list"-.
- Looking at the ACPI SRAT, via grepping dmesg:
 - SRAT Numa "Node" 1, proximity domain(PXM)5 matches the base address.
- Looking at the BIOS e820 table:
 - CXL memory region is "soft-reserved".
- "Free" system memory (can also use `/proc/meminfo`) shows 379 GiB:
 - This system has 8 x 16 GiB native DDR for a total of 128 GiB.
 - CXL card added 254.74 GiB.
 - Native DDR + CXL system-ram roughly totals the total memory of 379 GiB.
- numactl shows there are 2 membinding:
 - 0 would be using native DDR.
 - 1 would be using CXL memory.

```

# daxctl list -RDMu
[
{
  "path": "\platform\hmem.1",
  "id": 1,
  "size": "254.75 GiB (273.54 GB)",
  "align": 2097152,

```

```

"devices":[
  {
    "chardev":"dax1.0",
    "size":"254.75 GiB (273.54 GB)",
    "target_node":1,
    "align":2097152,
    "mode":"devdax",
    "state":"disabled",
    "mappings":[
      {
        "page_offset":"0",
        "start":"0x2080000000",
        "end":"0x602ffffffff",
        "size":"254.75 GiB (273.54 GB)"
      }
    ]
  }
],
{
  "path":"\\platform\\hmem.0",
  "id":0,
  "size":"254.75 GiB (273.54 GB)",
  "align":2097152,
  "devices":[
    {
      "chardev":"dax0.0",
      "size":"254.75 GiB (273.54 GB)",
      "target_node":1,
      "align":2097152,
      "mode":"system-ram",
      "online_memblocks":127,
      "total_memblocks":127,
      "movable":true,
      "mappings":[
        {
          "page_offset":"0",
          "start":"0x2080000000",
          "end":"0x602ffffffff",
          "size":"254.75 GiB (273.54 GB)"
        }
      ]
    }
  ]
}
]

# dmesg | grep ACPI | grep SRAT
[ 0.327357] ACPI: SRAT 0x0000000077729000 017CD8 (v03 INTEL INTEL ID 00000002 INTL 01000013)
[ 0.385723] ACPI: Reserving SRAT table memory at [mem 0x77729000-0x77740cd7]
[ 0.396435] ACPI: SRAT: Node 0 PXM 0 [mem 0x00000000-0x7ffffffff]
[ 0.397831] ACPI: SRAT: Node 0 PXM 0 [mem 0x100000000-0x207ffffffff]
[ 0.399348] ACPI: SRAT: Node 1 PXM 5 [mem 0x2080000000-0x602ffffffff]

# dmesg | grep BIOS-e820
...
[ 0.000000] BIOS-e820: [mem 0x0000000207f000000-0x0000000207ffffffff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000002080000000-0x0000000602ffffffff] soft reserved

# free -h
              total          used          free      shared  buff/cache   available
Mem:          379Gi          4.4Gi          374Gi          10Mi          262Mi          373Gi
Swap:           0B           0B           0B

# numactl -show
policy: default
preferred node: current
physcpubind: 0 1 2 3
cpubind: 0
nodebind: 0
membind: 0 1

```

DEVDAx Mode versus System-Ram Mode

This section provides only a quick and simple guide to switching to and from devdax versus system-RAM.

When you boot your system with a CXL card, you should automatically see a device under *daxctl list*. If not, try *daxctl crate-device*. If that does not work, it is likely there is either an OS, BIOS, or CXL card issue.

Reconfigure from system-RAM mode to devdax mode. In the next example you will find:

- CXL originally in system-RAM mode.
- *Free* had a value 97144412.
- Use *daxctl reconfigure-device* to reconfigure to devdax.
- CXL now in devdax mode.
- *Free* has a new lower value of 65687132 [~ 32 GiB less].

```
# daxctl list -RDMu
{
  "path":"/platform/ACPI0017:00/root0/decoder0.6/region6/dax_region6",
  "id":6,
  "size":"32.00 GiB (34.36 GB)",
  "align":2097152,
  "devices":[
    {
      "chardev":"dax6.0",
      "size":"32.00 GiB (34.36 GB)",
      "target_node":2,
      "align":2097152,
      "mode":"system-ram",
      "online_memblocks":15,
      "total_memblocks":15,
      "movable":true,
      "mappings":[
        {
          "page_offset":"0",
          "start":"0x3ce40000000",
          "end":"0x3d63ffffffff",
          "size":"32.00 GiB (34.36 GB)"
        }
      ]
    }
  ]
}
# free
      total        used        free      shared  buff/cache   available
Mem:    97144412    2160776    95223140       12800       374660    94983636
Swap:   32923644         0     32923644
# daxctl reconfigure-device -u --mode=devdax --force dax6.0
{
  "chardev":"dax6.0",
  "size":"32.00 GiB (34.36 GB)",
  "target_node":2,
  "align":2097152,
  "mode":"devdax"
}
reconfigured 1 device
# daxctl list -RDMu
{
  "path":"/platform/ACPI0017:00/root0/decoder0.6/region6/dax_region6",
  "id":6,
  "size":"32.00 GiB (34.36 GB)",
  "align":2097152,
  "devices":[
    {
      "chardev":"dax6.0",
      "size":"32.00 GiB (34.36 GB)",
      "target_node":2,
      "align":2097152,
      "mode":"devdax",
      "mappings":[
        {
          "page_offset":"0",
          "start":"0x3ce40000000",
          "end":"0x3d63ffffffff",
          "size":"32.00 GiB (34.36 GB)"
        }
      ]
    }
  ]
}
```

```

    }
  ]
}
# free

```

	total	used	free	shared	buff/cache	available
Mem:	65687132	1672556	64192832	12800	371312	64014576
Swap:	32923644	0	32923644			

Reconfigure from devdax mode to system-RAM mode:

- CXL originally in devdax mode.
- *Free* had a value 62 GiB.
- Use `daxctl reconfigure-device` to reconfigure to devdax.
- CXL now in system-RAM mode.
- *Free* has a new lower value of 92 GiB [~32 GiB more].

```

# daxctl list -RDMu
{
  "path":"/platform/ACPI0017:00/root0/decoder0.6/region6/dax_region6",
  "id":6,
  "size":"32.00 GiB (34.36 GB)",
  "align":2097152,
  "devices":[
    {
      "chardev":"dax6.0",
      "size":"32.00 GiB (34.36 GB)",
      "target_node":2,
      "align":2097152,
      "mode":"devdax",
      "mappings":[
        {
          "page_offset":"0",
          "start":"0x3ce40000000",
          "end":"0x3d63ffffffff",
          "size":"32.00 GiB (34.36 GB)"
        }
      ]
    }
  ]
}
# free -h

```

	total	used	free	shared	buff/cache	available
Mem:	62Gi	1.6Gi	61Gi	12Mi	362Mi	61Gi
Swap:	31Gi	0B	31Gi			

```

# daxctl reconfigure-device -u --mode=system-ram --force dax6.0
{
  "chardev":"dax6.0",
  "size":"32.00 GiB (34.36 GB)",
  "target_node":2,
  "align":2097152,
  "mode":"system-ram",
  "online_memblocks":15,
  "total_memblocks":15,
  "movable":true
}
reconfigured 1 device
# free -h

```

	total	used	free	shared	buff/cache	available
Mem:	92Gi	2.0Gi	90Gi	12Mi	362Mi	90Gi
Swap:	31Gi	0B	31Gi			

```

# daxctl list -RDMu
{
  "path":"/platform/ACPI0017:00/root0/decoder0.6/region6/dax_region6",
  "id":6,
  "size":"32.00 GiB (34.36 GB)",
  "align":2097152,
  "devices":[
    {
      "chardev":"dax6.0",

```

```

    "size": "32.00 GiB (34.36 GB)",
    "target_node": 2,
    "align": 2097152,
    "mode": "system-ram",
    "online_memblocks": 15,
    "total_memblocks": 15,
    "movable": true,
    "mappings": [
      {
        "page_offset": "0",
        "start": "0x3ce40000000",
        "end": "0x3d63ffffffff",
        "size": "32.00 GiB (34.36 GB)"
      }
    ]
  }
}
}

```

Creating a Target App (System-RAM Edition)

For many RAS testing, you want to choose a specific app to inject poison, but you need to an application that uses CXL memory. In 1lm+Vol mode, use numctl to bind the application to CXL memory. For other modes, have an application that allocates more than the interleave size so that some memory will be using CXL memory. For convenience, the recipes in this chapter uses mca-recover from [ras-tools](#). Slight modification to mca-recover.c may be used to increase the mmap size to cross over memory interleave.

In the next example:

- Observe that mca-recover allocates an address in numa node 1.
 - This would be an address between [0x3ce40000000-0x3d63ffffffff].
 - mca-recover mmmaps a buffer starting at 0x3d580091000, which is within the CXL memory space being targeted.

```

# dmesg | grep ACPI | grep SRAT
[ 0.021647] ACPI: SRAT 0x0000000007777F000 00BF18 (v03 INTEL INTEL ID 00000002 INTL 01000013)
[ 0.021727] ACPI: Reserving SRAT table memory at [mem 0x7777f000-0x7778af17]
[ 0.022060] ACPI: SRAT: Node 0 PXM 0 [mem 0x00000000-0x7fffffffff]
[ 0.022063] ACPI: SRAT: Node 0 PXM 0 [mem 0x100000000-0x87fffffffff]
[ 0.022066] ACPI: SRAT: Node 1 PXM 1 [mem 0x3c640000000-0x3ce3fffffffff]
[ 0.022087] ACPI: SRAT: Node 2 PXM 8 [mem 0x3ce40000000-0x3d63fffffffff]

# numactl -show
policy: default
preferred node: current
physcpubind: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68
69 70 71
cpubind: 0 1
nodebind: 0 1
membind: 0 1 2

# numactl --membind=2 ./mca-recover
vtop(7f663e20a000) = 3d580091000
Use /sys/kernel/debug/apei/einj/... to inject
Then press <ENTER> to access:

```

You can then do a HPA->DPA address translation for a targeted error injection.

```

>>> cxl.addTran()
Choose one option
1) HPA ==> DPA
2) DPA ==> HPA
3) quit

Select one option (type a number): 1

```



```
Set the Host Physical Address (HPA): 0x3d580091000
```

```
Translation > Host Physical Address to Device Physical Address
HPA=0x3d580091000 ==> Socket=1 ==> Port=pxp2.rp0/2a ==> DPA ==> 0x740091000
```

Creating a Target App (dev-dax Edition)

An alternative to a target app is to directly read devdax via `/dev/dax*` as a character device.

First find the "dax" device, you want to access and note the "resource"/base address. In the next Python* example, doing a `m.seek(0)` means reading the devdax base address. Doing a `m.seek(0x2000)` means reading to devdax base address + 0x2000. ... and so on. This method requires "less" translating. Before running the sample Python script, write "0xdeadbeef" to the dev dax's base address.

```
>>> import mmap

# Open devdax for read+write
# 0 buffering, this is very important for our experiment.
>>> f = open("/dev/dax0.0", "r+b", 0)

# Check that you are able to open /dev/dax*
>>> f.fileno()
3

# MMap /dev/dax with Offset=0.
# This means basically ends up mapping to DPA=0 on first CXL Device in the CXL Region.
>>> m = mmap.mmap(f.fileno(), length=1024*1024*1024, offset=0)

# Read 4 bytes of offset 0
>>> m.seek(0)
>>> data = m.read(4)

# print out data in a more readable format and notice it says 0xdeadbeef.
>>> [hex(i) for i in data]
['0xef', '0xbe', '0xad', '0xde']
```

11.1.2 CXl Type 3 Address Translation

Once in a while, you will need to do an HPA<->DPA translation. Tools are constantly being developed, but here are the known available APIs/methods. *CXL* Memory Device SW Guide*, document number 643805, also has an explanation for HPA<->DPA translation.

- CScripts: `cxl.addTran` for both HPA<->DPA.
- BIOS: `CXLfwdAddressTranslation()` for HPA->DPA.
 - The Intel specific `_DSM` Address Translation does not support CXL. See *Birch Stream Platform BIOS Writer's Guide (BWG)*, document number 647898.
- Linux: `cxl_dpa_to_hpa()` in `cxl/core/trace.c` for DPA->HPA since upstream Kernel 6.4.
 - There is no standalone CLI at the moment to access this translation.
 - The translation is mainly used for FTrace logging for poison injection.
 - CXL Event logs may contain HPA entry in the ftrace output.

11.1.1.3 CXL Mailbox and Event Logs

CXL Type 3 Memory Device RAS is dependent on issuing commands such as "inject poison" and "get/clear event records" via mailbox and/or another component command interface. This document will only use the CXL specification commands as documented in the CXL Specification's "Component Command Interface". Note that not all the commands are supported on every device, for instance "inject poison". Check the vendor's CXL card support capabilities before initiating "inject poison" testing.

Event logs can be signaled via the three following methods:

- MSI:
 - Standard PCIe MSIs for OS First Error handling
 - Linux observability via ftrace, no dmesg outputs. See CXL FTrace chapter later.
- MEFN VDM:
 - MEFN VDM turn into SMIs for firmware first mode. However, with end to end solution with the Intel BIOS reference code, MEFN VDMs can only be used when a secondary mailbox is present on the CXL card.
 - See "RAS Deep Dive Technical Training" slides for more information about MEFN VDM and Firmware First Mode requirements. *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644.
- MCTP Notification:
 - MCTP Notifications typically go to the BMC for BMC handling.
 - How BMC handles CXL events is out of scope for this document.

Key CCI Commands

Command	Reason
Get/Clear Event Records	Observe and clear CXL Event Logs.
Get/Set Event Interrupt	Change Interrupt policy between MSI and MEFN VDM.
Get/Set MCTP Event Interrupt Policy	Enable MCTP event notification.
Get Supported Logs	Used mostly in conjunction with Get Log, to see what CCI commands are supported by device.
Get Log	Used to see what CCI commands are supported by device.
Get Health Info	Used for alert recipes.
Get/Set Alert Configuration	Used to alert recipes.
Get Poison List	Optionally used to see what poisons are in the CXL card so they can be cleared.
Inject/Clear Poison	Inject and clear poisons, actually UECC errors, into the CXL card.
Identify Memory	Mostly to find event log sizes, poison list size and inject poison limit

Key CCI Command Sequences

CXL CCI Commands are accessed through mailboxes or via MCTP. This document will only execute the CCI commands via the primary mailbox through CScripts. There is no *official* OS tool to access the mailbox via CLI, this is by design. The BMC can execute CCI commands via MCTP. MCTP CCI access is out of scope for this document, but more information is in the *UEFI DSP0281 CXL Type 3 Dev CCI over MCTP Specification* and the *Compute Express Link (CXL) MCTP Manageability Quick Start Validation Guide* (document number 784530).

How To Find Supported Commands

Not all the CCI commands are mandatory, like *poison injection*. Also as the industry is still young, not all the mandatory CCI commands may have been implemented in your card yet.

CScripts has a command called *cxl.cxl_device_command*, which supports only a subset of available CXL spec defined CCI commands. When you run *cxl.cxl_device_command*, it will show a list of CScripts-supported command supported by the CXL card. If you want to know what list of commands are supported by the CXL card, you can follow the next recipe.

1. Use the Get Supported Logs command:
 - a. Make sure that Command Effect Logs [CEL] logs are supported.
 - b. Find CEL Size.
 - i. The example has a size of 0x34.
2. Use the Get Log command:
 - a. Plug in CEL GUID and Size of CEL log.
 - i. GUID is : 0da9c0b5-bf41-4b78-8f79-96b1623b3f17.
 - ii. Offset is : 0x0.
 - iii. Length is : 0x34 (from "Get Supported Logs" output).

```
>>> cxl.cxl_device_command(1, "2a")
Commands list:
MemCommands:
1) identifyMemory
2) getHealthInfo
3) getAlertConfiguration
4) setAlertConfiguration
5) getPoisonList
6) injectPoison
7) clearPoison
DevCommands:
8) getEventRecords
9) clearEventRecords
10) getEventInterruptPolicy
11) setEventInterruptPolicy
12) getSupportedLogs
13) getLog

q) Exit

Select one option (type a number): 12
Command selected: getSupportedLogs

...

Detailed: Output Payload table
=====
|getSupportedLogs: Output Payload Fields|Output Value|units|
=====
|Number_of_SupportedLogEntries|1| |
|Reserved|236226805760|
|SupportedLogEntries|0x420000000034173f3b62b196798f784b41bfb5c0a90d|
|-->Log Identifier#1|0da9c0b5-bf41-4b78-8f79-96b1623b3f17 (CEL)|
|-->Log Size#1|0x34|bytes|
|-->Log Identifier#2|None|
|-->Log Size#2|0x0|bytes|
=====
>>> cxl.cxl_device_command(1, "2a")
```

```

Commands list:
MemCommands:
1) identifyMemory
2) getHealthInfo
3) getAlertConfiguration
4) setAlertConfiguration
5) getPoisonList
6) injectPoison
7) clearPoison
DevCommands:
8) getEventRecords
9) clearEventRecords
10) getEventInterruptPolicy
11) setEventInterruptPolicy
12) getSupportedLogs
13) getLog

g) Exit

Select one option (type a number): 13
Command selected: getLog

Input payload is required!!

Options for LogIdentifier:
1) 0da9c0b5-bf41-4b78-8f79-96b1623b3f17 (CEL)
2) 5e1819d9-11a9-400c-811f-d60719403d86 (VDL)
3) b3fab4cf-01b6-4332-943e-5e9962f23567 (CSDL)
Select one option (type a number): 1

Options for Offset:
Type numeric value for input Field Payload(0x0-0xFFFFFFFF): 0

Options for Length:
Type numeric value for input Field Payload(0x0-0xFFFFFFFF): 0x34
Input Payload: 0x3400000000173f3b62b196798f784b41bfb5c0a90d

....

Detailed: Output Payload table
=====
|getLog: Output Payload Fields|Output Value
=====
...
|-->Opcode#11          |Cmb. OpCode: 0x4300 -> Command: getPoisonList
|-->Command Effect#11   |0
|-->Opcode#12          |Cmb. OpCode: 0x4301 -> Command: injectPoison
|-->Command Effect#12   |0
|-->Opcode#13          |Cmb. OpCode: 0x4302 -> Command: clearPoison
|-->Command Effect#13   |0
=====

Detailed: Input Payload table
=====
|getLog: Input Payload Fields|Input Value                                     |units|
=====
|LogIdentifier              |0da9c0b5-bf41-4b78-8f79-96b1623b3f17 (CEL) |bytes|
|Offset                     |0                                           |bytes|
|Length                     |52                                          |bytes|
=====

```

How To Inject Poison/Uncorrectable ECC

Check if your CXL Card vendor for how the CXL "poison inject" command is implemented. Behavior may be different than the documented in this IVG. The CXL spec only specifies that the address is poisoned and an Information Event is logged.

CScripts Method

1. Use the Inject Poison command:
 - a. Input is DPA address. See the section on CXL Finding Address and CXL Address Translation for a "good" DPA.
 - b. (Optional) Check getPoisonList.

NOTE

Decoding of getPoisonList will be improved in the future version of CScripts.

```

>>> cxl.cxl_device_command(1, "2a")
Commands list:
MemCommands:
1) identifyMemory
2) getHealthInfo
3) getAlertConfiguration

```

```

4) setAlertConfiguration
5) getPoisonList
6) injectPoison
7) clearPoison
DevCommands:
8) getEventRecords
9) clearEventRecords
10) getEventInterruptPolicy
11) setEventInterruptPolicy
12) getSupportedLogs
13) getLog

q) Exit

Select one option (type a number): 6
Command selected: injectPoison

Input payload is required!!
Options for Inject_Poison_Physical_Address:
Type numeric value for input Field Payload(0x0-0xFFFFFFFFFFFFFFFF): 0x100
Input Payload: 0x100

RAW table
=====
|Command      |Combined Opcode|Input Payload Size[Bytes]|Output Payload Size[Bytes]|Input Payload Value|Output Payload Value|
=====
|injectPoison|0x4301         |0x8                     |0x0                     |0x100              |0x0                  |
=====

Detailed: Input Payload table
...

>>> cxl.cxl_device_command(1, "2a")
Commands list:
MemCommands:
1) identifyMemory
2) getHealthInfo
3) getAlertConfiguration
4) setAlertConfiguration
5) getPoisonList
6) injectPoison
7) clearPoison
DevCommands:
8) getEventRecords
9) clearEventRecords
10) getEventInterruptPolicy
11) setEventInterruptPolicy
12) getSupportedLogs
13) getLog

q) Exit

Select one option (type a number): 5
Command selected: getPoisonList

Input payload is required!!
Options for Get_Poison_List_Physical_Address:
Type numeric value for input Field Payload(0x0-0xFFFFFFFFFFFFFFFF): 0
Options for Get_Poison_List_Physical_Address_Length:
Type numeric value for input Field Payload(0x0-0xFFFFFFFFFFFFFFFF): 0x10000
Input Payload: 0x100000000000000000000000000000000

RAW table
...

Detailed: Output Payload table
=====
|getPoisonList: Output Payload Fields|Output Value|units|
=====
|Poison_List_Flags|Disabled All|~|
|Reserved1|0|~|
|Overflow_Timestamp|0|ns|
|Media_Error_Record_Count|1|# records|
|Reserved2|0|~|
|Media_Error_Records|36893488147419103491|error records|
|-->Media Error Address#1|Reserved = 3, DPA...|~|
|-->Media Error Length#1|2|#Adj. DPAs|
=====

Detailed: Input Payload table
=====
|getPoisonList: Input Payload Fields|Input Value|units|
=====
|Get_Poison_List_Physical_Address|Reserved = 0, DPA...|~|
|Get_Poison_List_Physical_Address_Length|4194304|Bytes|
=====

```

Manually decoding the previous list using the CXL spec results in the following. Future revisions of CScripts will do a better job decoding.

```

Poison List Flags : 0x0[0]
Overflow Timestamp : 0x0[0]
                  : 1969-12-31 18:00:00 (19620 days ago)
Media Error Record Count : 0x1[1]record(s)
Media Error Records

```

```
Media Error Records 0:
Media Error Address : 0x103[259]
                   : Injected
                   : 0x100[256]
Media Error Length : 0x2[2]
```

Linux Method

If you are using the latest stable Linux kernel, see <https://www.kernel.org/>, you can inject poison via debugfs

```
[root@gnr-bkc ~]# cxl list -M
[
  {
    "memdev": "mem2",
    "ram_size": 137438953472,
    "serial": 2458346470421228829,
    "numa_node": 0,
    "host": "0000:3e:00.0"
  },
  {
    "memdev": "mem1",
    "ram_size": 137438953472,
    "serial": 2458263362736942365,
    "numa_node": 0,
    "host": "0000:8a:00.0"
  },
  {
    "memdev": "mem0",
    "ram_size": 137438953472,
    "serial": 2458271785234918685,
    "numa_node": 0,
    "host": "0000:64:00.0"
  }
]
[root@gnr-bkc ~]# ls /sys/kernel/debug/cxl/
0000:3d:02.0 0000:63:02.0 0000:89:02.0 einj_types mbox mem0 mem1 mem2
[root@gnr-bkc ~]# echo <dpa> > /sys/kernel/debug/cxl/mem0/inject_poison
```

How To Clear Poison Addresses

1. Get a list of poisoned addresses:
 - a. See past event logs or use the "get poison list" command.
2. Use the Poison Clear command:
 - a. Input is DPA address. See the section on CXL Finding Address and CXL Address Translation.
 - b. Input some kind of data. Example uses 0xcafecafe.
 - c. Alternatively is just to reset/power cycle the device if the CXL card only has volatile memory.
3. Read the poisoned address back and see if successful with not poison consumption.

CScripts Method

```
>>> cxl.cxl_device_command(1, "2a")
Commands list:
MemCommands:
1) identifyMemory
2) getHealthInfo
3) getAlertConfiguration
4) setAlertConfiguration
5) getPoisonList
6) injectPoison
```

```

7) clearPoison
DevCommands:
8) getEventRecords
9) clearEventRecords
10) getEventInterruptPolicy
11) setEventInterruptPolicy
12) getSupportedLogs
13) getLog

q) Exit

Select one option (type a number): 7
Command selected: clearPoison

Input payload is required!!
Options for Clear_Poison_Physical_Address:
Type numeric value for Input Field Payload(0x0-0xFFFFFFFFFFFFFFFF): 0x100
Options for Clear_Poison_WriteData:
Type numeric value for Input Field Payload(0x0-0xFFFFFFFFFFFFFFFF...): 0xcafecafe
Input Payload: 0xcafecafe00000000000000100

RAW table
=====
|Command|Combined Opcode|Input Payload Size[Bytes]|Output Payload Size[Bytes]|Input Payload Value|Output Payload Value|
|-----|-----|-----|-----|-----|-----|
|clearPoison|0x4302|0x48|0x0|0xcafecafe00000000000000100|0x0|
=====

Detailed: Input Payload table
=====
|clearPoison: Input Payload Fields|Input Value|units|
|-----|-----|-----|
|Clear_Poison_Physical_Address|Reserved = 0, DPA...|-|
|Clear_Poison_WriteData|3405695742|-|
=====

>>> mem(0x3ce40000100)
0xcafecafe
>>> msr(0x405)
0x0

```

Linux Method

Future version of CXL Cli will have a poison list command.

```
echo <dpa> > /sys/kernel/debug/cxl/mem0/clear_poison
```

How To Clear Event Logs

1. Check the Event Status register to see the event logs.
2. Use Get Event Records command to get the appropriate Event Log.
 - a. Save the "Event Record Handle" value. The example shows 0x10.

NOTE

Decoding of getPoisonList will be improved in the future version of CScripts.

3. Use the Clear Event Record command:
 - a. Input Event Record Handle.

```

# Note: Ability to check "Event Status" is TBD using cxl.capabilities

Select one option (type a number): 8
Command selected: getEventRecords

Input payload is required!!
Options for EventLog:
1) Informational Event Log
2) Warning Event Log
3) Failure Event Log
4) Fatal Event Log
Select one option (type a number): 1
Input Payload: 0x0

RAW table
...

Detailed: Output Payload table

```

Manually decoding the previous log using the CXL spec results in the following. Future CScripts revisions will do a better job decoding.

```

Flags : 0x0[0]
Overflow Error Count : 0x0[0]
First Overflow Event Timestamp : 0x0[0]
                                : 1969-12-31 18:00:00 (19620 days ago)
Last Overflow Event Timestamp : 0x0[0]
                                : 1969-12-31 18:00:00 (19620 days ago)
Event Record Count : 0x1[1]
Event Record Identifier : 601DCBB3-9C06-4EAB-B8AF-4E9BFB5C9624
                        : DRAM Event Record
Event Record Length : 0x80[128]
Event Record Flags : 0x0[0]
Severity : 0x0[0]
Event Record Handle : 0x10[16]
Related Event Record Handle : 0x0[0]
Event Record Timestamp : 0x53804e88f49[5738158657353]
Time is not valid
Maintenance Operation Class : 0x0[0]

```

August 2025
Doc. No.: 742874, Rev.: 1.5


```
Options for Number of Event Record Handles:
Type numeric value for input Field Payload(0x0-0xFF): 1

Options for Event Record Handles:
Type numeric value for input Field Payload(0x0-varies): 0x10
Input Payload: 0x10000000010000

RAW table
=====
|Command|Combined Opcode|Input Payload Size[Bytes]|Output Payload Size[Bytes]|Input Payload Value|Output Payload Value|
=====
|clearEventRecords|0x101|0x8|0x0|0x10000000010000|0x0|
=====

Detailed: Input Payload table
=====
|clearEventRecords: Input Payload Fields|Input Value|units|
=====
|EventLog|Informational Event Log|-|
|ClearEventFlags|No action|-|
|Number of Event Record Handles|1|# event records|
|Event Record Handles|16|# events|
=====
```

11.1.4 CXL Firmware First Versus OS First

Figure 16. CXL Error Logging and Signaling

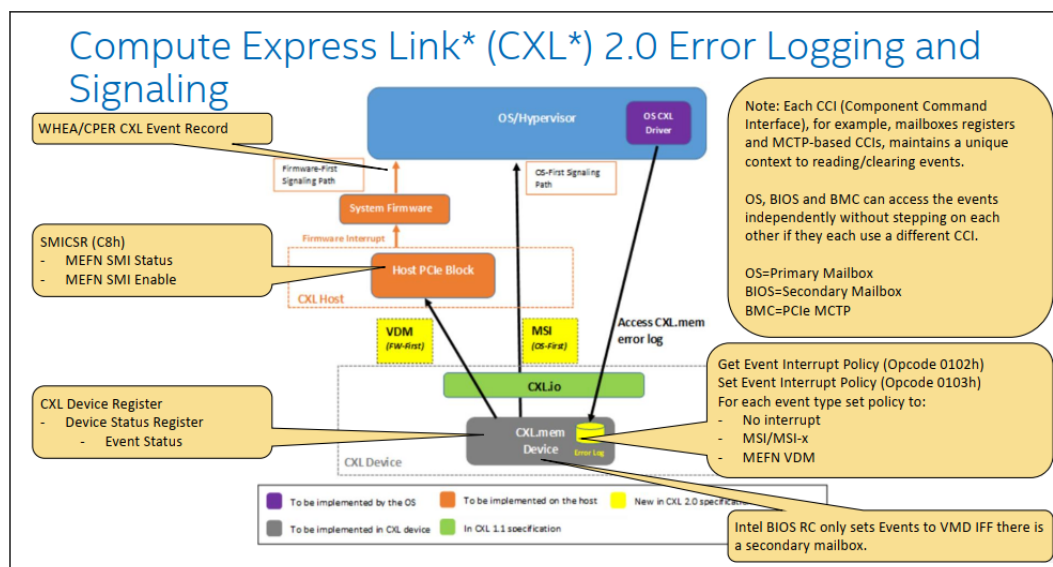
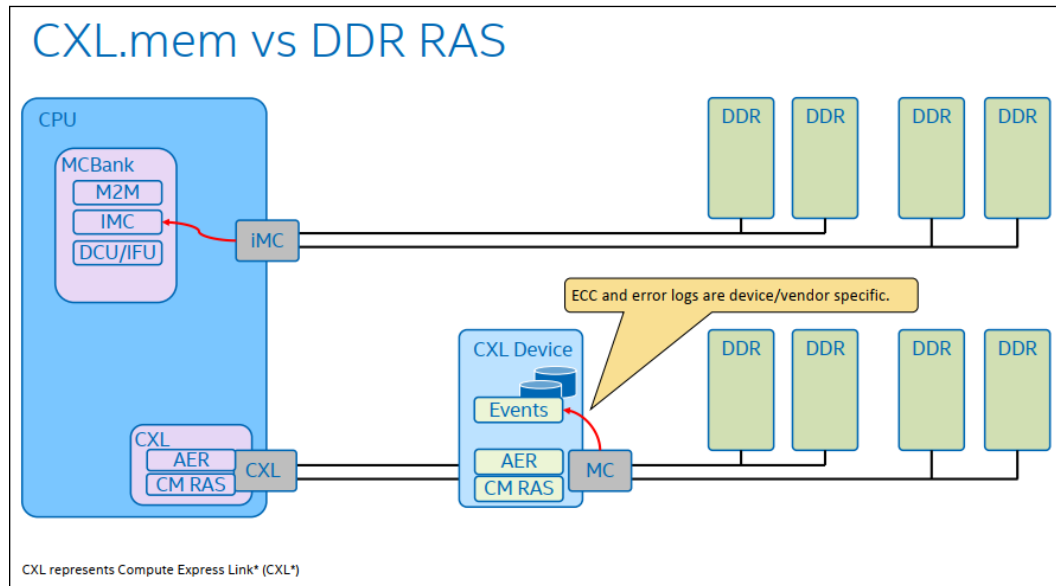


Figure 17. CXL.mem Versus DDR RAS



For CXL, there are 2 different types of errors and error flow paths.

- CXL.IO (basically PCIe AER) and CXL.CM Protocol Errors (see CXL RAS Capability Structure in the CXL Spec):
 - These errors are signaled via standard PCIe AER.
 - These are errors detected between the CXL Root Port and CXL Device.
- CXL Event Logs:
 - These logs are signaled via MSI [for OS First], MEFN VFM [for Firmware First], or MCTP [for BMC].
 - These are errors detected between the CXL Memory Controller and CXL attached memory.

See the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for more information on this topic.

To Enable Firmware First mode on CXL:

- EDKII > Platform Configuration > System Event Log > IIO Error Enabling > OS Native AER Support = Disable.
- EDKII > Platform Configuration > System Event Log > CXL Error Enabling > CXL Memory Event Fw Notification(MEFN) Support = Enable.
 - All CXL Cards on the system MUST support a secondary mailbox. Otherwise system will revert back to OS first.

To Enable OS First mode on CXL:

- EDKII > Platform Configuration > System Event Log > IIO Error Enabling > OS Native AER Support = Enable.

- EDKII > Platform Configuration > System Event Log > CXL Error Enabling > CXL Memory Event Fw Notification(MEFN) Support = Disable.

NOTE

You can mix and match Firmware First and OS First mode between AER Handling and CXL Memory Event Handling.

FTrace is designed to capture logs and events consistently, whether you are operating in Firmware First mode or OS First mode. The distinction lies in the origin of the FTrace data. In Firmware First mode, the data for FTrace comes directly from the BIOS, specifically in the form of ACPI Generic Hardware Error Source (GHES) and Common Platform Error Record (CPER) logs. On the other hand, in OS First mode, FTrace gathers its records through the operating system itself.

At the time this information was written, based on the latest stable kernel available at <http://kernel.org>, there are certain CPER records that are not yet fully interpreted by the system. These records are identified as "non_standard_event" within the FTrace events:

- CXL Protocol Errors - See UEFI Spec for more information on this record format
- CXL Memory Sparing Event Record
- CXL Physical Switch Event Record
- CXL Virtual Switch Event Record
- CXL MLD Port Event Record
- CXL Dynamic Capacity Event Record

The following CXL related CPER records are supported:

- PCIe AER
- CXL General Media Event Record
- CXL DRAM Event Record
- CXL Memory Module Event Record

Example ACPI GHES/CPER record in Linux DMESG

```
[ 8025.841184] {27}[Hardware Error]: Hardware error from APEI Generic Hardware
Error Source: 0
[ 8025.851329] {27}[Hardware Error]: It has been corrected by h/w and requires no
further action
[ 8025.861248] {27}[Hardware Error]: event severity: corrected
[ 8025.867701] {27}[Hardware Error]: Error 0, type: corrected
[ 8025.874150] {27}[Hardware Error]: section_type: CXL Memory Module Event
```

11.1.5 CXL FTrace

See the [FTrace](#) section for more information on how to use FTrace. For CXL, next is the list of events that are used for testing.

- mce/mce_record
 - For all machine check events.
- ras/aer_event

- For all PCIe AER events.
- `ras/memory_failure_event`
 - For observation into page offlining events.
- `cxl/*`
 - `cxl_aer_uncorrectable_error` and `cxl_aer_correctable_error` are for CXL.CM protocol errors in the CXL RAS Capability registers.
 - These will also be accompanied with standard AER Uncorrectable/Correctable Internal Error from `ras/aer_event`.
 - `cxl_poison` are for poison inject/clear events.
 - The rest are related to CXL Event Logs.
 - *cxl monitor* can also be used for `cxl/*`.

```
cd /sys/kernel/debug/tracing/
echo 1 > events/mce/mce_record/enable
echo 1 > events/ras/aer_event/enable
echo 1 > events/ras/memory_failure_event/enable
echo 1 > events/cxl/enable
```

Example CXL Event Log output. Using `cat trace` or `cat trace_pipe`.

NOTE

`trace_pipe` may be broken in some Kernel versions.

```
# cat trace
# tracer: nop
#
# entries-in-buffer/entries-written: 1/1   #P:72
#
#          -----> irqs-off/BH-disabled
#          /-----> need-resched
#          | /-----> hardirq/softirq
#          || /-----> preempt-depth
#          ||| /-----> migrate-disable
#          |||| /-----> delay
#
#          TASK-PID   CPU#  | ||||  TIMESTAMP  FUNCTION
#          | |       | |   | ||||  |         |
irq/181-0000:a8-1435   [055]  ....  81342.821593: cxl_dram: memdev=mem0 host=0000:a8:00.0
serial=2315805941843029277 log=Informational : time=82964223536009 uuid=601dcbb3-9c06-4eab-
b8af-4e9bfb5c9624 len=128 flags='' handle=10 related handle=0 maint_op_class=0 : dpa=2000
dpa_flags='' descriptor='UNCORRECTABLE_EVENT' type='ECC Error' transaction_type='Host Inject
Poison' channel=0 rank=0 nibble_mask=0 bank_group=0 bank=0 row=0 column=64 cor_mask=00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
validity_flags='CHANNEL|RANK|BANK GROUP|BANK|ROW|COLUMN'
```

11.2 CXL Link/Protocol Errors

The goal in this chapter is to test signal paths versus the actual error itself. Ultimately, these errors shows up as an AER Uncorrectable/Correctable Internal Error in the RCEC associated with the CXL Root Port. Thus creating a correctable or uncorrectable error "flow" via the RCEC.

11.2.1 CXL Corrected Error "Flow" Injection

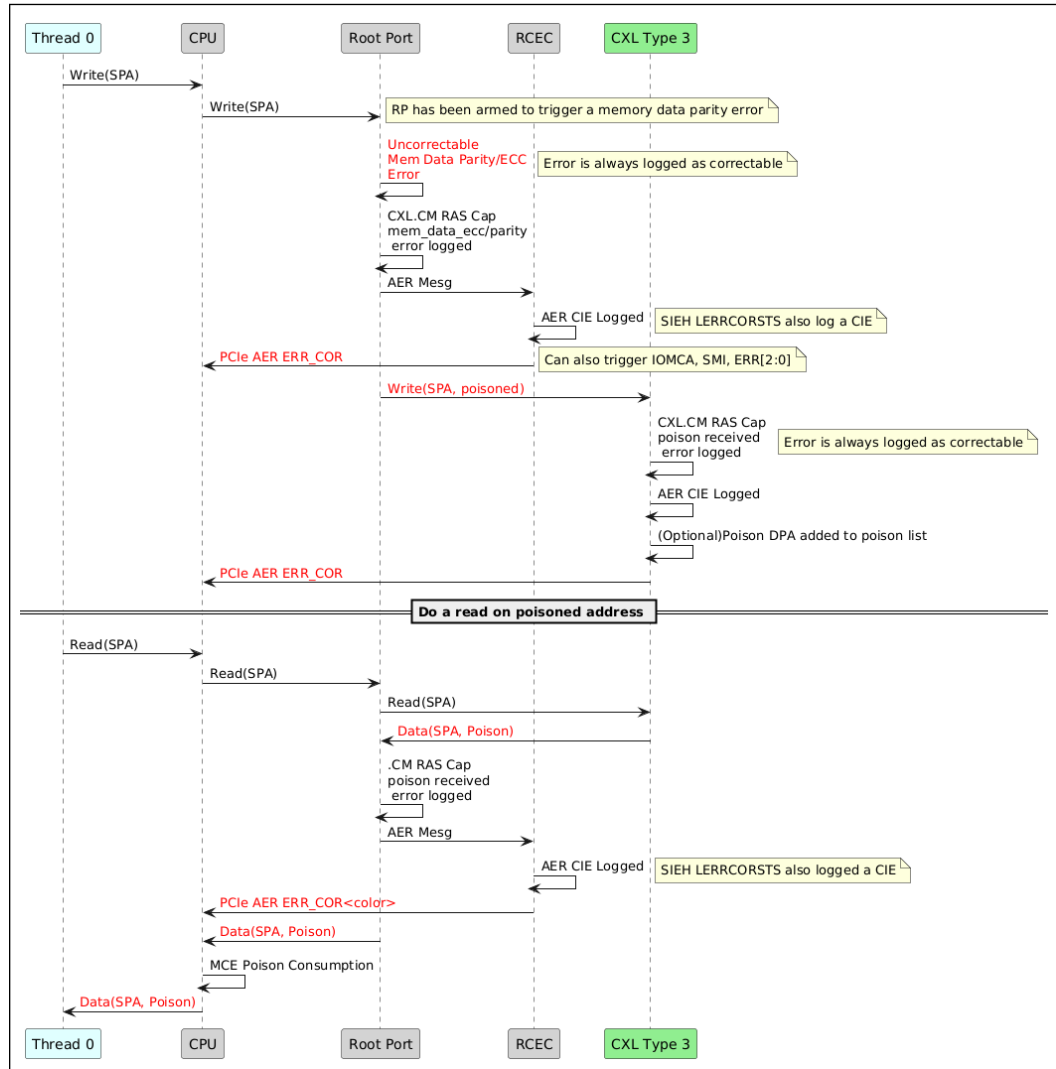
Test Objective

This test injects a corrected error "flow". The prime example used in this recipe injects an uncorrectable mem_data_parity error into a CXL 2.0 Device that gets downgraded into a correctable error due to poison enabling. A bonus side effect of mem_data_parity is that it results in a poisoned address in the CXL ecosystem. This allows a CXL poison consumption test to occur. See the "Error Capabilities and Control Register" section in the CXL spec for more info about poison enabling. See the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for more information about CXL error reporting and flows.

If you want a pure CE error injection, refer to the [ce_crc_error](#) recipe.

The next figure summarizes the error flow of the recipe in this chapter:

Figure 18. CXL Correctable flow



Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B0
BIOS	BHSDCRB1.IPC.0034.D82.2408060239
OS Version	CentOS Stream 9
Kernel	6.10.6
<i>continued...</i>	

HW Configuration	1 Socket
CScripts Version	BHS.2422.6001
Mode of operation	Firmware First Mode

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- ["CScripts on page 18"](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability on page 27](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration. [CXL Firmware First vs OS First](#) on how to set the knobs:

- Test is valid for either OS First Mode or Firmware First Mode.

Recipe

1. Inject a correctable error into CXL memory using the address from the [target app](#).

• Pseudo Script

```
# Variables so this semi-pseudo test code can be copy/pastable with minor modifications.
test_socket = 0
test_io = 0
test_port = "pxp1"
test_cxlcm_port = "cxlcm_port_regs0"
test_cxl_port = "1a"
test_ta = 0x6000000000 # get the address from the target app

test_node = sv.sockets[test_socket].ios[test_io].uncore.pi5.get_by_path(test_port)
cxlcm_node = test_node.get_by_path(test_cxlcm_port)

halt()

# Check CXLCM Error Status and Mask Registers to make sure it is clean
# - cxl.cxl_cmем_err_check()
# - cxl.cxl_ieh_error_dump()

# Check Error Capabilities and Control Register.
# If mcg_contain.poisоn_en=1, expect CE otherwise expect UE.
# System default is mcg_contain.poisоn_en=1
cxldump = cxl.capabilities(test_socket, test_cxl_port, False)
cxldump.rootport.comp_regs.cachemem_caps.ras.errctl.show()
cxlrp_poisоn_enabled = cxldump.rootport.comp_regs.cachemem_caps.ras.errctl.value
cxldump.Endpoint1.comp_regs.cachemem_caps.ras.errctl.show()
cxlep_poisоn_enabled = cxldump.Endpoint1.comp_regs.cachemem_caps.ras.errctl.value

# Clear cache
itp.threads.wbinvd()

# Arm the error injection
orig_access = cxlcm_node.get_access()
cxlcm_node.set_access("mem")
cxlcm_node.error_injection_register.mem_data_par_errinj=1

# confirm that mem_data_par_errinj is actually set.
# If not check to make sure you did set access()
if (not cxlcm_node.error_injection_register.mem_data_par_errinj):
    print("Error Injection Failed")

# Read from the CXL Target Address to make sure it's valid.
# Write to CXL Target Address to trigger error injection.
```

```
# Make sure to wbinvd to flush the data from cache
itp.threads[0].mem("%sp" % hex(test_ta), 4)
itp.threads[0].mem("%sp" % hex(test_ta), 4, 0xdeadbeef)
itp.threads.wbinvd()

# Confirm that error injection was triggered by reading "done"
# "done" will be cleared when error injection is done
cxlcm_node.error_injection_register.mem_data_par_errinj_done
cxlcm_node.error_injection_register.mem_data_par_errinj=0
if (cxlcm_node.error_injection_register.mem_data_par_errinj_done):
    print("Error Injection Failed")

# Confirm that error was detected
# If "poison_enabled=1", error will be a correctable, else uncorrectable
if (cxlrp_poison_enabled == 0x1):
    cxlcm_node.correctable_error_status_register.show()
    if (not cxlcm_node.correctable_error_status_register.mem_data_ecc):
        print("Error not detected on RP.")
else:
    cxlcm_node.uncorrectable_error_status_register.show()
    if (not cxlcm_node.uncorrectable_error_status_register.mem_data_parity):
        print("Error not detected on RP.")

cxlep_dump = cxldump.EndPoint1.comp_regs
if (cxlep_poison_enabled == 0x1):
    if (not cxlep_dump.cachemem_caps.ras.corerrsts.mem_poison_received.value):
        print("Error not detected on EP.")
else:
    if (not cxlep_dump.cachemem_caps.ras.uncerrsts.poison_received.value):
        print("Error not detected on EP.")

# Run cscript cxl error check functions for more info.
cxl.cxl_cmern_err_check(test_socket, test_cxl_port)
cxl.cxl_ieh_error_dump(test_socket, test_cxl_port)

itp.go()
```

• CScripts Output

```
>>> test_socket = 0
>>> test_io = 0
>>> test_port = "pxpl"
>>> test_cxlcm_port = "cxlcm_port_regs0"
>>> test_cxl_port = "1a"
>>> test_ta = 0x6000000000 # get the address from the target app

>>> test_node = sv.sockets[test_socket].ios[test_io].uncore.pi5.get_by_path(test_port)
>>> cxlcm_node = test_node.get_by_path(test_cxlcm_port)

>>> halt()
[GPC core group] Halt on 144 devices -- 14:25:34.059385 2024-08-22

# Check CXLCM Error Status and Mask Registers to make sure it is clean
# - cxl.cxl_cmern_err_check()
# - cxl.cxl_ieh_error_dump()

# Check Error Capabilities and Control Register.
# If mcg_contain.poison_en=1, expect CE otherwise expect UE.
# System default is mcg_contain.poison_en=1

>>> cxldump = cxl.capabilities(test_socket, test_cxl_port, False)
>>> cxldump.rootport.comp_regs.cachemem_caps.ras.errctl.show()

WARNING: CAP ID = 0x007 decoder is not supported by CScripts, EXIT...
WARNING: CAP ID = 0x007 decoder is not supported by CScripts, EXIT...
WARNING: CAP ID = 0x3 decoder is not supported by CScripts, EXIT...
WARNING: CAP ID = 0x3 decoder is not supported by CScripts, EXIT...
WARNING: CAP ID = 0x4000 decoder is not supported by CScripts, EXIT...
WARNING: CAP ID = 0x4000 decoder is not supported by CScripts, EXIT...
Base Address = 0x1e5ffe101100

[0x014] Error Capabilities and Control Register (addr = 0x1e5ffe101114, val = 0x2000)
```



```
-----
[ 3: 0] First_Error_Pointer..... 0x0
[ 9: 9] Multiple_Header_Recording_Capability..... 0x0
[13:13] Poison_Enabled..... 0x1

>>> cxlrp_poison_enabled = cxldump.rootport.comp_regs.cachemem_caps.ras.errctl.value
>>> cxldump.EndPoint1.comp_regs.cachemem_caps.ras.errctl.show()

Base Address = 0xle5ffe001020

[0x014] Error Capabilities and Control Register (addr = 0xle5ffe001034, val = 0x2000)
-----
[ 3: 0] First_Error_Pointer..... 0x0
[ 9: 9] Multiple_Header_Recording_Capability..... 0x0
[13:13] Poison_Enabled..... 0x1
0x00000000 : cxl_idc_rx_error (16:16) -- Crypto Log of any errors encountered on Rx side
0x00000000 : cxl_idc_tx_error (15:15) -- Crypto Log of any errors encountered on Tx side
0x00000000 : internal_error (14:14) -- Internal_Error. Currently we only have one type of error that is
logged as error which is illegal opcode on M2S Req, RWD, U2C Rsp, U2C Req . R...
0x00000000 : duplicate_retry_ack (13:13) -- Retry.Ack received when LRSM is in Normal State
0x00000000 : llrb_crc (12:12) -- LLRB bad CRC condition
0x00000000 : receiver_overflow (11:11) -- Receiver Overflow. First 3b of the Header Log are relevant and
should be interpreted as such: 0= cacheReq_Overflow 1 = Cache Resp_Overflow ...
0x00000000 : poison_received (10:10) -- Received Poison from Device. Header Log contains the entire flit
received.
0x00000000 : rsvd_encoding_violation (09:09) -- Received unrecognized encoding. Header Log contains the
entire flit received. This bit is shared by a number of conditions such as st...
0x00000000 : reinit_threshold (08:08) -- REINIT Threshold Hit. Header Log not applicable.
0x00000000 : mem_data_ecc (07:07) -- Data ECC on CXL.mem. Header Log contains M2S Rwd Data Header.No
support for ECC in gen3, hence this error is NA
0x00000000 : mem_be_parity (06:06) -- BE Parity on CXL.mem. Header Log contains M2S Rwd Data Headerfor
host and S2M DRS Data header for device
0x00000000 : mem_address_parity (05:05) -- Address Parity on CXL.mem. If Bit 0 of Header Log is '0, rest
of Header Log contains M2S Req. If Bit 0 of Header Log is '1, rest of Header...
0x00000000 : mem_data_parity (04:04) -- Data Parity on CXL.mem. Header Log contains M2S Rwd Data Header
for host and S2M DRS Data header for device
0x00000000 : cache_data_ecc (03:03) -- Data ECC on IDI/CXL.cache. Header Log contains H2D Data Header. No
support for ECC in gen3, hence this error is NA
0x00000000 : cache_be_parity (02:02) -- BE Parity on IDI/CXL.cache. Header Log contains H2D Data Header
for Host and D2H data header for device
0x00000000 : cache_address_parity (01:01) -- Address Parity on IDI/CXL.cache. Header Log contains H2D
Request for Host and D2H request for device
0x00000000 : cache_data_parity (00:00) -- Data Parity on IDI/CXL.cache. Header Log contains H2D Data
Header for Host and D2H data header for device

>>> cxlep_poison_enabled = cxldump.EndPoint1.comp_regs.cachemem_caps.ras.errctl.value

# Clear cache
>>> itp.threads.wbinvd()

# Arm the error injection
>>> orig_access = cxlcm_node.get_access()
>>> cxlcm_node.set_access("mem")
>>> cxlcm_node.error_injection_register.mem_data_par_errinj=1

# confirm that mem_data_par_errinj is actually set.
# If not check to make sure you did set_access()
>>> if (not cxlcm_node.error_injection_register.mem_data_par_errinj):
>>>     print("Error Injection Failed")

# Read from the CXL Target Address to make sure it's valid.
# Write to CXL Target Address to trigger error injection.
# Make sure to wbinvd to flush the data from cache
>>> itp.threads[0].mem("%sp" % hex(test_ta), 4)
>>> itp.threads[0].mem("%sp" % hex(test_ta), 4, 0xdeadbeef)
>>> itp.threads.wbinvd()

# Confirm that error injection was triggered by reading "done"
# "done" will be cleared when error injection is done
>>> cxlcm_node.error_injection_register.mem_data_par_errinj_done
>>> cxlcm_node.error_injection_register.mem_data_par_errinj=0
>>> if (cxlcm_node.error_injection_register.mem_data_par_errinj_done):
>>>     print("Error Injection Failed")

# Confirm that error was detected
# If "poison_enabled=1", error will be a correctable, else uncorrectable
>>> if (cxlrp_poison_enabled == 0x1):
>>>     cxlcm_node.correctable_error_status_register.show()
>>>     if (not cxlcm_node.correctable_error_status_register.mem_data_ecc):
```

```
>>> print("Error not detected on RP.")
>>> else:
>>>     cxlcm_node.uncorrectable_error_status_register.show()
>>>     if (not cxlcm_node.uncorrectable_error_status_register.mem_data_parity):
>>>         print("Error not detected on RP.")

Error not detected on RP.

>>> cxlep_dump = cxldump.EndPoint1.comp_regs
>>> if (cxlep_poison_enabled == 0x1):
>>>     if (not cxlep_dump.cachemem_caps.ras.corerrsts.mem_poison_received.value):
>>>         print("Error not detected on EP.")
>>> else:
>>>     if (not cxlep_dump.cachemem_caps.ras.uncerrsts.poison_received.value):
>>>         print("Error not detected on EP.")

Error not detected on EP.

# Run cscript cxl error check functions for more info.

>>> cxl.cxl_cmern_err_check(test_socket, test_cxl_port)

++ HOST ERROR INFORMATION ++
INFO: Socket 0 'pxpl.rp0' is operating in CXL2.0 mode!

*****
*   CXL 2.0 PortUncorrectable Error Dump on Socket0 Port = '1a'   *
*****

Uncorrectable Error Status Register   : 0x00000000
Uncorrectable Error Mask Register     : 0x00003000
Uncorrectable Error Severity Register : 0x0001cfff
-----
| Name | Bit# | Mask | Severity | Status |
-----
| Cache_Data_Parity | 0 | 0 | 1 | 0 |
| Cache_Address_Parity | 1 | 0 | 1 | 0 |
| Cache_BE_Parity | 2 | 0 | 1 | 0 |
| Cache_Data_ECC | 3 | 0 | 1 | 0 |
| Mem_Data_Parity | 4 | 0 | 1 | 0 |
| Mem_Address_Parity | 5 | 0 | 1 | 0 |
| Mem_BE_Parity | 6 | 0 | 1 | 0 |
| Mem_Data_ECC | 7 | 0 | 1 | 0 |
| REINIT_Threshold | 8 | 0 | 1 | 0 |
| Rsvd_Encoding_Violation | 9 | 0 | 1 | 0 |
| Poison_Received | 10 | 0 | 1 | 0 |
| Receiver_Overflow | 11 | 0 | 1 | 0 |
| LLRB_bad_CRC_condition | 12 | 1 | 0 | 0 |
| Retry_Ack_received_when_LRSM_is_in_Normal_State | 13 | 1 | 0 | 0 |
| Internal_Error | 14 | 0 | 1 | 0 |
-----

Found 0 error(s)...

*****
*   CXL 2.0 PortCorrectable Error Dump on Socket0 Port = '1a'   *
*****

Correctable Error Status Register : 0x00000002
Correctable Error Mask Register   : 0x00000000
-----
| Name | Bit# | Mask | Status |
-----
| Cache_Data_ECC | 0 | 0 | 0 |
| Mem_Data_ECC | 1 | 0 | 1 |
| CRC_Threshold | 2 | 0 | 0 |
| Retry_Threshold | 3 | 0 | 0 |
| Cache_Poison_Received | 4 | 0 | 0 |
| Mem_Poison_Received | 5 | 0 | 0 |
| Physical_Layer_Error | 6 | 0 | 0 |
-----

Found 1 error(s)...
0 : Mem_Data_ECC is detected!

*****
```

```
* CXL 2.0 Port CXL cm.mem Error Capabilities and Control Register on Socket0 Port = '1a' *
*****
```

```
Error Capabilities and Control Register : 0x00002000
```

Error Capabilities and Control Register	Bit#	Control
First_Error_Pointer	3: 0	0
Multiple_Header_Recording_Capability	9: 9	0
Poison_Enabled	13:13	1

```
=====
++ DEVICE ERROR INFORMATION ++
```

```
*****
* CXL2.0 Port Uncorrectable Error Dump on Socket0 Port = '1a' *
*****
```

```
Uncorrectable Error Status Register : 0x00000000
Uncorrectable Error Mask Register : 0x00000000
Uncorrectable Error Severity Register : 0x0001cfff
```

Name	Bit#	Mask	Severity	Status
Cache_Data_Parity	0	0	1	0
Cache_Address_Parity	1	0	1	0
Cache_BE_Parity	2	0	1	0
Cache_Data_ECC	3	0	1	0
Mem_Data_Parity	4	0	1	0
Mem_Address_Parity	5	0	1	0
Mem_BE_Parity	6	0	1	0
Mem_Data_ECC	7	0	1	0
REINIT_Threshold	8	0	1	0
Rsvd_Encoding_Violation	9	0	1	0
Poison_Received	10	0	1	0
Receiver_Overflow	11	0	1	0
LLRB_bad_CRC_condition	12	0	0	0
Retry.Ack received when LRSM is in Normal State	13	0	0	0
Internal_Error	14	0	1	0

```
Found 0 error(s)...
```

```
*****
* CXL2.0 Port Correctable Error Dump on Socket0 Port = '1a' *
*****
```

```
Correctable Error Status Register : 0x00000020
Correctable Error Mask Register : 0x00000000
```

Name	Bit#	Mask	Status
Cache_Data_ECC	0	0	0
Mem_Data_ECC	1	0	0
CRC_Threshold	2	0	0
Retry_Threshold	3	0	0
Cache_Poison_Received	4	0	0
Mem_Poison_Received	5	0	1
Physical_Layer_Error	6	0	0

```
Found 1 error(s)...
```

```
0 : Mem_Poison_Received is detected!
```

```
*****
* CXL 2.0 Port CXL cm.mem Error Capabilities and Control Register on Socket0 Port = '1a' *
*****
```

```
Error Capabilities and Control Register : 0x00002000
```

Error Capabilities and Control Register	Bit#	Control
First_Error_Pointer	3: 0	0
Multiple_Header_Recording_Capability	9: 9	0
Poison_Enabled	13:13	1

```
>>> cxl.cxl_ieh_error_dump(test_socket, test_cxl_port)
INFO: Socket 0 'pxpl.rp0' is operating in CXL2.0 mode!
```

```
*****
* IEH4 Local 2 Uncorrectable Error Dump on Socket0 Port = '1a' *
*****
```

```
Uncorrectable Error Status Register : 0x00000000
Uncorrectable Error Mask Register : 0x00180020
Uncorrectable Error Severity Register : 0x00463010
```

Name	Bit#	Mask	Severity	Status
Poisoned TLP Egress Blocked	26	0	0	0
TLP Prefix Blocked Error	25	0	0	0
AtomicOp Egress Blocked	24	0	0	0
MC Blocked TLP	23	0	0	0
Uncorrectable Internal Error	22	0	1	0
ACS Violation	21	0	0	0
Unsupported Request Error	20	1	0	0
ECRC Error	19	1	0	0
Malformed TLP	18	0	1	0
Receiver Overflow Error	17	0	1	0
Unexpected Completion Error	16	0	0	0
Completer Abort Error	15	0	0	0
Completion Timeout Error	14	0	0	0
Flow Control Protocol Error	13	0	1	0
Poisoned TLP Received	12	0	1	0
Data Link Protocol Error	4	0	1	0

Found 0 error(s)...

```
*****
* IEH4 Local 2 Correctable Error Dump on Socket0 Port = '1a' *
*****
```

```
Correctable Error Status Register : 0x00004000
Correctable Error Mask Register : 0x00004000
```

Name	Bit#	Mask	Status
Header Log Overflow Error	15	0	0
Correctable Internal Error	14	1	1
Advisory Non-Fatal Error	13	0	0
Replay timer time-out occurs	12	0	0
Replay number rolls over from 11 to 00	8	0	0
CRC errors on DLLP	7	0	0
CRC errors on TLP	6	0	0
receiver error	0	0	0

Found 1 error(s)...

0 : Correctable Internal Error is detected!

```
*****
* IEH4 Root Error Dump on Socket0 Port = '1a' *
*****
```

```
Root Error Command Register : 0x00000000
```

Root Error Command Register	Bit#	Control
Enables the interrupt generation when a Fatal error is reported	2: 2	0
Enables the interrupt generation when a Non-Fatal error is reported	1: 1	0
Enables the interrupt generation when a Correctable error is reported	0: 0	0

```
Root Error Status Register : 0x00000001
```

Root Error Status Register	Bit#	Status
MSI/MSI-X vector used for the interrupt message	31:27	0
Fatal Error Message Received	6: 6	0
Non-Fatal Error message Received	5: 5	0

```
| First Uncorrectable Fatal          | 4: 4 | 0 |
| Multiple Error Fatal/Non-Fatal Received | 3: 3 | 0 |
| Error Fatal/Non-Fatal Received      | 2: 2 | 0 |
| Multiple Error Correctable Error Received | 1: 1 | 0 |
| Correctable Error Received          | 0: 0 | 1 |
```

```
Found 1 error(s)...
0 : Correctable Error Received is detected!
```

```
>>> itp.go()
```

2. Observe the FTrace output in the OS.

NOTE

See [CXl Firmware First vs OS First](#) for any known issues with OS First Mode or Firmware First Mode.

Pass / Fail	Criteria
PASS	Check for PCIe Correctable Internal Error on the associated RCEC: - Event type: aer_event - BDF info (can vary): 0000:3d:00.4 (RCEC) - Error type : PCIe Bus Error - Severity=Corrected, Corrected Internal Error
PASS	Check for CXl.io Correctable Internal Error on EP: - Event type: aer_event - BDF info (can vary): 0000:3e:00.0 (EP) - Error type : PCIe Bus Error - Severity= Advisory Non-Fatal Error Corrected Internal Error
OS FAIL	Check for CXl Protocol Error Mem Poison Received on the RP. Look for 1, but found 0 cxl_aer_correctable_error ftrace event: Due to the above mentioned OS bug, Event type is shown as a Non-standard event - Check: "host" == "0000:3d:02.0" (RP) (can vary) - Severity= 1 - Check: "status" & 0x2 : Mem Data ECC
OS FAIL	Check for CXl Protocol Error Mem Poison Received on the EP. Look for 1, but found 0 cxl_aer_correctable_error ftrace event: Due to the above mentioned OS bug, Event type is shown as a Non-standard event - Check: "host" == "0000:3e:00.0" (EP) (can vary) - Severity= 1 - Check: "status" & 0x20 : Mem Received Poison
PASS	No unexpected events seen. Check with your CXl card vendor for implementation specific behavior.

```
# cat trace
# tracer: nop
#
# entries-in-buffer/entries-written: 3/3   #P:72
#
#          -----> irqs-off/BH-disabled
#          /-----> need-resched
#          | /-----> hardirq/softirq
#          || /-----> preempt-depth
#          ||| /-----> migrate-disable
#          |||| /-----> delay
#
# TASK-PID   CPU#  | TIMESTAMP | FUNCTION
# |-----|-----|-----|
kworker/0:2-1105 [000] d..1.   290.443353: non_standard_event: severity: 1; sec type:b4efb980-
b552-e34d-a777-68784b771048; FRU: 00000000-0000-0000-0000-000000000000 ; data len:260; raw data:67 00 00
```

```
00 00 00 00 00 05 00 00 00 00 00 00 00 00 02 3d 00 00 00 00 98 1e b0 0d 5f 1e 00 00 04 06 30 00 00 00
00 00 00 00 00 00 00 00 00 00 10 a0 42 01 22 80 00 00 27 51 00 00 05 49 7a 00 40 00 85 70 80 25 30 00 c0
03 48 01 1f 00 01 00 00 00 00 f7 1f b3 10 49 14 00 00 3e 00 80 01 05 00 1e 00 01 00 00 00 00 00 00 00
38 00 58 00 00 00 00 00 23 00 01 00 98 1e 41 01 07 00 27 00 be 00 26 00 17 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 ff cf 01 00 02 00 00 00 00 00 00 00 00 20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
kworker/0:2-1105 [000] d..l. 290.443357: non_standard_event: severity: 1; sec type:b4efb980-
b552-e34d-a777-68784b771048; FRU: 00000000-0000-0000-0000-000000000000 ; data len:260; raw data:67 00 00
00 00 00 00 02 00 00 00 00 00 00 00 00 00 00 00 3e 00 00 00 00 00 98 1e e2 01 00 00 00 00 02 05 00 00 00
00 00 1d ed 4a 08 17 cd 1d 22 10 b0 02 00 c1 8f 2c 11 37 50 09 00 05 61 43 00 40 00 85 10 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 10 00 b1 00 00 00 00 00 3e 00 80 01 05 00 1e 01 00 00 00 00 00 00 00
38 00 58 00 00 00 00 00 23 00 c1 40 98 1e 81 03 00 00 9e e8 06 40 00 00 00 00 00 00 01 00 00 00 20 00 00
00 0b 81 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 48 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 ff cf 01 00 20 00 00 00 00 00 00 00 00 20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
kworker/0:1-10 [000] .N... 290.472386: aer_event: 0000:3d:00.4 PCIe Bus Error:
severity=Corrected, Corrected Internal Error, TLP Header=Not available

kworker/0:1-10 [000] .N... 290.504453: aer_event: 0000:3e:00.0 PCIe Bus Error:
severity=Corrected, Advisory Non-Fatal Error|Corrected Internal Error, TLP Header=Not available
```

3. Linux dmesg output

NOTE

The dmesg output depends on several factors like OS distro and kernel. Information is used for debugging purposes only.

```
[ 288.701979] {1}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 0
[ 288.728927] {1}[Hardware Error]: It has been corrected by h/w and requires no further action
[ 288.738665] {1}[Hardware Error]: event severity: corrected
[ 288.744897] {1}[Hardware Error]: Error 0, type: corrected
[ 288.751132] {1}[Hardware Error]: section_type: CXL Protocol Error
[ 288.758268] {1}[Hardware Error]: agent_type: 5, CXL Root Port
[ 288.765866] {1}[Hardware Error]: agent_address: 0000:3d:02.0
[ 288.772573] {1}[Hardware Error]: slot: 6
[ 288.777321] {1}[Hardware Error]: vendor_id: 0x1e98, device_id: 0x0db0
[ 288.784907] {1}[Hardware Error]: sub_vendor_id: 0x1e5f, sub_device_id: 0x0000
[ 288.793279] {1}[Hardware Error]: class_code: 0604
[ 288.798912] {1}[Hardware Error]: DVSEC length: 0x0038
[ 288.804939] {1}[Hardware Error]: CXL DVSEC:
[ 288.809989] {1}[Hardware Error]: 00000000: 00010023 01411e98 00270007 002600be
[ 288.818361] {1}[Hardware Error]: 00000010: 00000017 00000000 00000000 00000000
[ 288.826726] {1}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000000
[ 288.835244] {1}[Hardware Error]: 00000030: 00000000 00000000
[ 288.841985] {1}[Hardware Error]: Error log length: 0x0058
[ 288.848517] {1}[Hardware Error]: CXL Error Log:
[ 288.854085] {1}[Hardware Error]: cxl_ras_uncor_status: 0x00000000
[ 288.854086] {1}[Hardware Error]: cxl_ras_uncor_mask: 0x00003000
[ 288.868518] {1}[Hardware Error]: cxl_ras_uncor_severity: 0x0001cfff
[ 288.876018] {1}[Hardware Error]: cxl_ras_cor_status: 0x00000002
[ 288.876019] {1}[Hardware Error]: cxl_ras_cor_mask: 0x00000000
[ 288.890043] {1}[Hardware Error]: cap_control: 0x00002000
[ 288.896456] {1}[Hardware Error]: Header Log Registers:
[ 288.902673] {1}[Hardware Error]: 00000000: 00000000 00000000 00000000 00000000
[ 288.911230] {1}[Hardware Error]: 00000010: 00000000 00000000 00000000 00000000
[ 288.919797] {1}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000000
[ 288.928376] {1}[Hardware Error]: 00000030: 00000000 00000000 00000000 00000000
[ 288.936965] {1}[Hardware Error]: Error 1, type: corrected
[ 288.943297] {1}[Hardware Error]: section_type: PCIe error
[ 288.949721] {1}[Hardware Error]: port_type: 10, root complex event collector
[ 288.958113] {1}[Hardware Error]: version: 3.0
[ 288.963384] {1}[Hardware Error]: command: 0x0504, status: 0x0010
[ 288.970501] {1}[Hardware Error]: device_id: 0000:3d:00.4
[ 288.976830] {1}[Hardware Error]: slot: 0
[ 288.981597] {1}[Hardware Error]: secondary_bus: 0x00
[ 288.987533] {1}[Hardware Error]: vendor_id: 0x8086, device_id: 0x0b23
[ 288.995122] {1}[Hardware Error]: class_code: 080700
[ 289.000957] {1}[Hardware Error]: Error 2, type: corrected
[ 289.007273] {1}[Hardware Error]: section_type: CXL Protocol Error
[ 289.014471] {1}[Hardware Error]: agent_type: 2, CXL Device
```

```
[ 289.021090] {1}[Hardware Error]: agent_address: 0000:3e:00.0
[ 289.027902] {1}[Hardware Error]: slot: 0
[ 289.032757] {1}[Hardware Error]: vendor_id: 0x1e98, device_id: 0x01e2
[ 289.040439] {1}[Hardware Error]: sub_vendor_id: 0x0000, sub_device_id: 0x0000
[ 289.049012] {1}[Hardware Error]: class_code: 0502
[ 289.054756] {1}[Hardware Error]: DVSEC length: 0x0038
[ 289.060887] {1}[Hardware Error]: CXL DVSEC:
[ 289.066036] {1}[Hardware Error]: 00000000: 40c10023 03811e98 e89e0000 00004006
[ 289.074609] {1}[Hardware Error]: 00000010: 00000000 00000001 00000020 0000810b
[ 289.083181] {1}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000048
[ 289.091753] {1}[Hardware Error]: 00000030: 00000000 00000000
[ 289.098472] {1}[Hardware Error]: Error log length: 0x0058
[ 289.104982] {1}[Hardware Error]: CXL Error Log:
[ 289.110527] {1}[Hardware Error]: cxl_ras_uncor_status: 0x00000000
[ 289.110527] {1}[Hardware Error]: cxl_ras_uncor_mask: 0x00000000
[ 289.124920] {1}[Hardware Error]: cxl_ras_uncor_severity: 0x0001cfff
[ 289.132403] {1}[Hardware Error]: cxl_ras_cor_status: 0x00000020
[ 289.132404] {1}[Hardware Error]: cxl_ras_cor_mask: 0x00000000
[ 289.146388] {1}[Hardware Error]: cap_control: 0x00002000
[ 289.152794] {1}[Hardware Error]: Header Log Registers:
[ 289.159008] {1}[Hardware Error]: 00000000: 00000000 00000000 00000000 00000000
[ 289.167562] {1}[Hardware Error]: 00000010: 00000000 00000000 00000000 00000000
[ 289.176129] {1}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000000
[ 289.184706] {1}[Hardware Error]: 00000030: 00000000 00000000 00000000 00000000
[ 289.193295] {1}[Hardware Error]: Error 3, type: corrected
[ 289.199619] {1}[Hardware Error]: section_type: PCIe error
[ 289.206043] {1}[Hardware Error]: port_type: 0, PCIe end point
[ 289.212864] {1}[Hardware Error]: version: 3.0
[ 289.218116] {1}[Hardware Error]: command: 0x0546, status: 0x0010
[ 289.225228] {1}[Hardware Error]: device_id: 0000:3e:00.0
[ 289.231555] {1}[Hardware Error]: slot: 0
[ 289.236319] {1}[Hardware Error]: secondary_bus: 0x00
[ 289.242253] {1}[Hardware Error]: vendor_id: 0x1dfa, device_id: 0x01e2
[ 289.249838] {1}[Hardware Error]: class_code: 050210
[ 289.259238] pcieport 0000:3d:00.4: AER: aer_status: 0x00004000, aer_mask: 0x00000000
[ 289.268224] pcieport 0000:3d:00.4: [14] CorrIntErr
[ 289.275212] pcieport 0000:3d:00.4: AER: aer_layer=Transaction Layer, aer_agent=Receiver ID
[ 289.284720] cxl_pci 0000:3e:00.0: AER: aer_status: 0x00006000, aer_mask: 0x00000000
[ 289.293542] cxl_pci 0000:3e:00.0: [13] NonFatalErr
[ 289.300455] cxl_pci 0000:3e:00.0: [14] CorrIntErr
[ 289.307350] cxl_pci 0000:3e:00.0: AER: aer_layer=Transaction Layer, aer_agent=Receiver ID
```

NOTE

The Advisory Non-Fatal error returned along with the Corrected error is due to an OS bug and can be ignored.

4. [Optional] To Create a "Poison Consumption", refer to [Poison Consumption](#) chapter for the next steps.

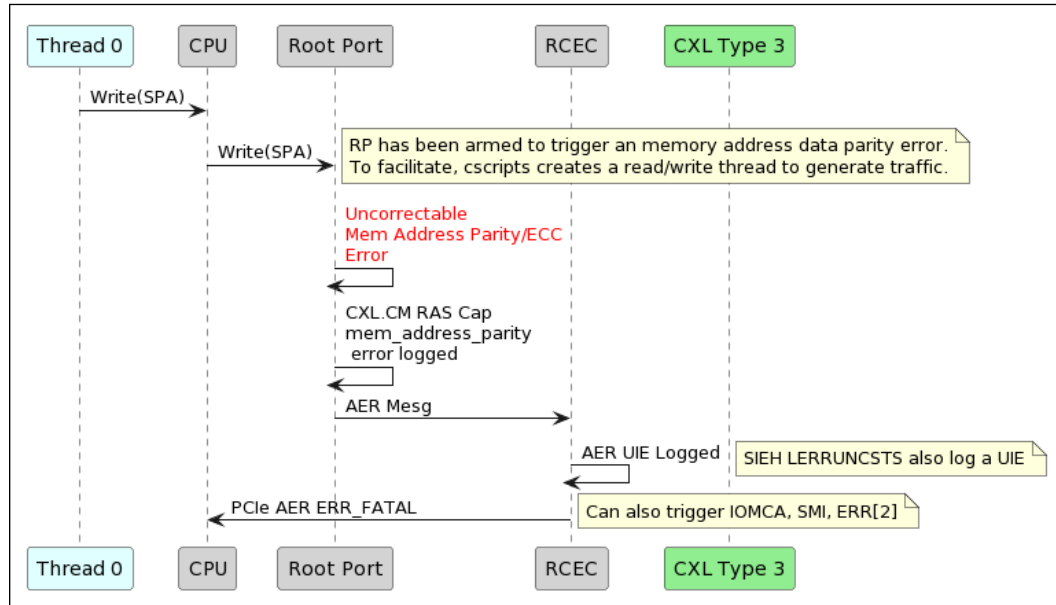
11.2.2 CXL Uncorrected Error "Flow" Injection

Test Objective

This test injects an uncorrected error "flow". The prime example used in this recipe injects an uncorrectable mem_addr_parity error into a CXL 2.0 Root Port. See the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for more information about CXL error reporting and flows.

The next figure summarizes the error flow of the recipe in this chapter:

Figure 19. Uncorrected Error Flow



Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, IIntel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B0
BIOS	BHSDCRB1.IPC.0034.D82.2408060239
OS Version	CentOS Stream 9
Kernel	6.10.6
HW Configuration	1 Socket
CScripts Version	BHS.2422.6001
Mode of operation	Firmware First Mode

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- "CScripts on page 18"
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration. [CXL Firmware First vs OS First](#) on how to set the knobs:

- Test is valid for either OS First Mode or Firmware First Mode

- In Firmware First, because this error results in a Uncorrectable Internal Error (UIE) in the RCEC, the PCIe CPER records sent to the OS has a "FATAL" severity, which will result in the Kernel panicking in the ACPI GHES handler before sending the telemetry to FTrace.

Recipe

1. Inject an address parity error into CXl memory.

• Pseudo Script

```
# Variables so this semi-pseudo test code can be copy/pastable with minor modifications.
test_socket = 0
test_cxl_port = "1a"

halt()

# Inject an address parity error
ei.mem.injectCxlError(test_socket, test_cxl_port, "uce_mem_addr_par", Severity = "nonfatal",
Verbose = True)

# Run cscript cxl error check functions for more info.
cxl.cxl_cmern_err_check(test_socket, test_cxl_port)
cxl.cxl_ieh_error_dump(test_socket, test_cxl_port)

itp.go()
```

• CScripts Output

```
>>> # Variables so this semi-pseudo test code can be copy/pastable with minor modifications.
>>> test_socket = 1
>>> test_cxl_port = "1a"
>>> test_cxlcm_port = "cxlcm_port_regs0"
>>>
>>> halt()
>>>
>>> # Inject an address parity error
>>> ei.mem.injectCxlError(test_socket, test_cxl_port, "uce_mem_addr_par", Severity = "nonfatal", Verbose
= True)
INFO: Socket 0 'pxpl.rp0' is operating in CXl2.0 mode!
Changing register access to 'mem'
Resetting injectors
Getting HDM information...
Memory Range: 0x6080000000-0x807ffffff
Starting Memory Stress transactions...
Generate traffic at Host-managed Device Memory (HDM) address (reads)
Generate traffic at Host-managed Device Memory (HDM) address (writes)
Clear Error Done
Setting Severity for error injection: nonfatal
Trigger Error Injector
Re-establishing default register access
Waiting for error injection done
Error detected at
socket0.io0.uncore.pi5.pxpl.cxlcm_port_regs0.uncorrectable_error_status_register.mem_address_parity bit
Stop traffic (write) at HDM address
Stop traffic (read) at HDM address
Stopping Memory Stress transactions. The test finished successfully
=====
|STAGE|CHECKPOINT|STATUS|
-----|-----|-----|
|Setup|Access Validation|Pass|
|Setup|Input Validation - socket|Pass|
|Setup|Setup Configuration - halt|Pass|
|Setup|Setup Validation - BIOS knobs|Pass|
|Setup|Input Validation - port|Pass|
|Setup|Input Validation - errType|Pass|
|Setup|Input Validation - memory address|N/A|
|Setup|Error Register Access|Pass|
|Setup|Clear Errors|Pass|
|Setup|Get HDM Info|Pass|
|Setup|Start Memory Stress|Pass|
|Setup|Injector Enabling|Pass|
|Injection|Inject CXl.cache/CXl.mem Error|Pass|
```

```

|Verification|Error Check          |Pass |
|Setup       |Stop Memory Stress          |Pass |
=====

# Run cscript cxl error check functions for more info.
>>> cxl.cxl_cmем_err_check(test_socket, test_cxl_port)

++ HOST ERROR INFORMATION ++
INFO: Socket 0 'pxpl.rp0' is operating in CXL2.0 mode!

*****
* CXL 2.0 PortUncorrectable Error Dump on Socket0 Port = '1a' *
*****

Uncorrectable Error Status Register : 0x00000020
Uncorrectable Error Mask Register   : 0x00000000
Uncorrectable Error Severity Register : 0x0001cfd

-----
| Name | Bit# | Mask | Severity | Status |
-----
| Cache_Data_Parity | 0 | 0 | 1 | 0 |
| Cache_Address_Parity | 1 | 0 | 1 | 0 |
| Cache_BE_Parity | 2 | 0 | 1 | 0 |
| Cache_Data_ECC | 3 | 0 | 1 | 0 |
| Mem_Data_Parity | 4 | 0 | 1 | 0 |
| Mem_Address_Parity | 5 | 0 | 0 | 1 |
| Mem_BE_Parity | 6 | 0 | 1 | 0 |
| Mem_Data_ECC | 7 | 0 | 1 | 0 |
| REINIT_Threshold | 8 | 0 | 1 | 0 |
| Rsvd_Encoding_Violation | 9 | 0 | 1 | 0 |
| Poison_Received | 10 | 0 | 1 | 0 |
| Receiver_Overflow | 11 | 0 | 1 | 0 |
| LLRB_bad_CRC_condition | 12 | 0 | 0 | 0 |
| Retry_Ack_received_when_LRSM_is_in_Normal_State | 13 | 0 | 0 | 0 |
| Internal_Error | 14 | 0 | 1 | 0 |
-----

Found 1 error(s)...
0 : socket0.io0.uncore.pi5.pxpl.cxlcm_port_regs0.uncorrectable_error_status_register.mem_address_parity
= 1, Mem_Address_Parity

*****
* CXL 2.0 PortCorrectable Error Dump on Socket0 Port = '1a' *
*****

Correctable Error Status Register : 0x00000000
Correctable Error Mask Register : 0x00000000

-----
| Name | Bit# | Mask | Status |
-----
| Cache_Data_ECC | 0 | 0 | 0 |
| Mem_Data_ECC | 1 | 0 | 0 |
| CRC_Threshold | 2 | 0 | 0 |
| Retry_Threshold | 3 | 0 | 0 |
| Cache_Poison_Received | 4 | 0 | 0 |
| Mem_Poison_Received | 5 | 0 | 0 |
| Physical_Layer_Error | 6 | 0 | 0 |
-----

Found 0 error(s)...

*****
* CXL 2.0 Port CXL cm.mem Error Capabilities and Control Register on Socket0 Port = '1a' *
*****

Error Capabilities and Control Register : 0x00002005

-----
| Error Capabilities and Control Register | Bit# | Control |
-----
| First_Error_Pointer | 3:0 | 5 |
| Multiple_Header_Recording_Capability | 9:9 | 0 |
| Poison_Enabled | 13:13 | 1 |
-----
=====
=====

```

++ DEVICE ERROR INFORMATION ++

```
*****
* CXL2.0 Port Uncorrectable Error Dump on Socket0 Port = '1a' *
*****
```

```
Uncorrectable Error Status Register : 0x00000000
Uncorrectable Error Mask Register : 0x00000000
Uncorrectable Error Severity Register : 0x0001cfff
```

Name	Bit#	Mask	Severity	Status
Cache_Data_Parity	0	0	1	0
Cache_Address_Parity	1	0	1	0
Cache_BE_Parity	2	0	1	0
Cache_Data_ECC	3	0	1	0
Mem_Data_Parity	4	0	1	0
Mem_Address_Parity	5	0	1	0
Mem_BE_Parity	6	0	1	0
Mem_Data_ECC	7	0	1	0
REINIT_Threshold	8	0	1	0
Rsvd_Encoding_Violation	9	0	1	0
Poison_Received	10	0	1	0
Receiver_Overflow	11	0	1	0
LLRB_bad_CRC_condition	12	0	0	0
Retry.Ack received when LRSM is in Normal State	13	0	0	0
Internal_Error	14	0	1	0

Found 0 error(s)...

```
*****
* CXL2.0 Port Correctable Error Dump on Socket0 Port = '1a' *
*****
```

```
Correctable Error Status Register : 0x00000000
Correctable Error Mask Register : 0x00000000
```

Name	Bit#	Mask	Status
Cache_Data_ECC	0	0	0
Mem_Data_ECC	1	0	0
CRC_Threshold	2	0	0
Retry_Threshold	3	0	0
Cache_Poison_Received	4	0	0
Mem_Poison_Received	5	0	0
Physical_Layer_Error	6	0	0

Found 0 error(s)...

```
*****
* CXL2.0 Port CXL cm.mem Error Capabilities and Control Register on Socket0 Port = '1a' *
*****
```

```
Error Capabilities and Control Register : 0x00002000
```

Error Capabilities and Control Register	Bit#	Control
First_Error_Pointer	3: 0	0
Multiple_Header_Recording_Capability	9: 9	0
Poison_Enabled	13:13	1

```
>>> cxl.cxl_ieh_error_dump(test_socket, test_cxl_port)
```

```
INFO: Socket 0 'pxpl.rp0' is operating in CXL2.0 mode!
```

```
*****
* IEH4 Local 2 Uncorrectable Error Dump on Socket0 Port = '1a' *
*****
```

```
Uncorrectable Error Status Register : 0x00400000
Uncorrectable Error Mask Register : 0x00180020
Uncorrectable Error Severity Register : 0x00463010
```

Name	Bit#	Mask	Severity	Status
Poisoned TLP Egress Blocked	26	0	0	0
TLP Prefix Blocked Error	25	0	0	0
AtomicOp Egress Blocked	24	0	0	0
MC Blocked TLP	23	0	0	0
Uncorrectable Internal Error	22	0	1	1
ACS Violation	21	0	0	0
Unsupported Request Error	20	1	0	0
ECRC Error	19	1	0	0
Malformed TLP	18	0	1	0
Receiver Overflow Error	17	0	1	0
Unexpected Completion Error	16	0	0	0
Completer Abort Error	15	0	0	0
Completion Timeout Error	14	0	0	0
Flow Control Protocol Error	13	0	1	0
Poisoned TLP Received	12	0	1	0
Data Link Protocol Error	4	0	1	0

Found 1 error(s)...

0 : Uncorrectable Internal Error is detected!

```
*****
* IEH4 Local 2 Correctable Error Dump on Socket0 Port = '1a' *
*****
```

Correctable Error Status Register : 0x00000000
Correctable Error Mask Register : 0x00000000

Name	Bit#	Mask	Status
Header Log Overflow Error	15	0	0
Correctable Internal Error	14	0	0
Advisory Non-Fatal Error	13	0	0
Replay timer time-out occurs	12	0	0
Replay number rolls over from 11 to 00	8	0	0
CRC errors on DLLP	7	0	0
CRC errors on TLP	6	0	0
receiver error	0	0	0

Found 0 error(s)...

```
*****
* IEH4 Root Error Dump on Socket0 Port = '1a' *
*****
```

Root Error Command Register : 0x00000000

Root Error Command Register	Bit#	Control
Enables the interrupt generation when a Fatal error is reported	2: 2	0
Enables the interrupt generation when a Non-Fatal error is reported	1: 1	0
Enables the interrupt generation when a Correctable error is reported	0: 0	0

Root Error Status Register : 0x00000054

Root Error Status Register	Bit#	Status
MSI/MSI-X vector used for the interrupt message	31:27	0
Fatal Error Message Received	6: 6	1
Non-Fatal Error message Received	5: 5	0
First Uncorrectable Fatal	4: 4	1
Multiple Error Fatal/Non-Fatal Received	3: 3	0
Error Fatal/Non-Fatal Received	2: 2	1
Multiple Error Correctable Error Received	1: 1	0
Correctable Error Received	0: 0	0

Found 3 error(s)...

0 : Fatal Error Message Received is detected!
1 : First Uncorrectable Fatal is detected!
2 : Error Fatal/Non-Fatal Received is detected!

```
>>> go()
```

2. Observe the FTrace output in the OS.

NOTE

FFM the system will panic before it had a chance to send/convert the APEI GHES message to FTrace.

NOTE

See [CXL Firmware First vs OS First](#) for any known issues with OS First Mode or Firmware First Mode.

3. Linux dmesg output

NOTE

The dmesg output depends on several factors like OS distro and kernel. Information is used for debugging purposes only.

```
[IEH] error found at IEH(S:0x0 B:0xFE D:0x2 F:0x0) Sev: IEH FATAL ERROR
[IEH] ErrorStatus 0x40, MaxBitIdx 0x1D
IEH FATAL ERROR
[IEH] BitIdx 0x6, ShareIdx 0x0
[IEH] error device is (S:0x0 B:0x3D D:0x0 F:0x4) BitIdx 0x6, ShareIdx 0x0 [IEH] error found at IEH(S:0x0
B:0x3D D:0x0 F:0x4) Sev: IEH FATAL ERROR
[IEH] ErrorStatus 0x1, MaxBitIdx 0x11
IEH FATAL ERROR
    Clearing Local Error Status for Error Source 2, Error Severity 4, Error Status 0x400000, Device Type
CXL 1p1/2p0 CM
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7E6BAC18
WHEA: Detected Ras Non Standard Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
CxlDevCMErrorHandler Entry
[CXL] - Device FlexBusPortStatus = 26
RAS Cap Base Address: 0x1E5FFE101100
ReportCxlProtocolErrorLog Entry
RAS Cap Base Address: 0x1E5FFE101100
    CXL Uncorrectable Error Status = 0x20
[PCI] 00:3D:02.0: Serial Number capability not found
[PCI] 00:3D:02.0: Serial Number capability not found
ReportCxlProtocolErrorLog Exit,Success
RAS Cap Base Address: 0x1E5FFE101100
CxlDevCMErrorHandler Exit
CxlHandleIdeErr Entry
RAS Cap Base Address 0x1E5FFE101100
CxlHandleIdeErr Exit
IehRciepDevicesErrorHandler - Socket 0, Bus 3D, device 0, function 4

Get CXL DP Port Secondary Bus# failed!
CxlHandleIdeErr Entry
RAS Cap Base Address 0x0
No RAS Cap Base Address
CxlDevCMErrorHandler Entry
[CXL] - Not a CXL device. Exiting since FlexBusPort is not available
ERROR: C00000002:V03071008 IO 515DFD4E-2D7E-40D1-8C22-8AD3CD224325 7E6BA418
WHEA: Detected CXL Protocol Error
--Logging Uncorrected Error to WHEA
WHEA: Detected PCIe Error
--Logging Uncorrected Error to WHEA
[ 784.085314] {1}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 1
WheaElogSwSmiCallback Enter
WheaElogSwSmiCallback Enter
WheaElogSwSmiCallback Enter
WheaElogSwSmiCallback Enter
[ 784.085318] {1}[Hardware Error]: event severity: fatal
```

```

[ 784.085319] {1}[Hardware Error]: Error 0, type: recoverable
[ 784.085320] {1}[Hardware Error]: section_type: CXL Protocol Error
[ 784.085321] {1}[Hardware Error]: agent_type: 5, CXL Root Port
[ 784.085322] {1}[Hardware Error]: agent_address: 0000:3d:02.0
[ 784.085323] {1}[Hardware Error]: slot: 6
[ 784.085324] {1}[Hardware Error]: vendor_id: 0x1e98, device_id: 0x0db0
[ 784.085325] {1}[Hardware Error]: sub_vendor_id: 0x1e5f, sub_device_id: 0x0000
[ 784.085325] {1}[Hardware Error]: class_code: 0604
[ 784.085326] {1}[Hardware Error]: DVSEC_length: 0x0038
[ 784.085327] {1}[Hardware Error]: CXL DVSEC:
[ 784.085328] {1}[Hardware Error]: 00000000: 00010023 01411e98 00270007 002600be
[ 784.085329] {1}[Hardware Error]: 00000010: 00000017 00000000 00000000 00000000
[ 784.085330] {1}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000000
[ 784.085331] {1}[Hardware Error]: 00000030: 00000000 00000000
[ 784.085332] {1}[Hardware Error]: Error log length: 0x0058
[ 784.085332] {1}[Hardware Error]: CXL Error Log:
[ 784.085333] {1}[Hardware Error]: cxl_ras_uncor_status: 0x00000020
[ 784.085333] {1}[Hardware Error]: cxl_ras_uncor_mask: 0x00000000
[ 784.085334] {1}[Hardware Error]: cxl_ras_uncor_severity: 0x0001cfd5
[ 784.085335] {1}[Hardware Error]: cxl_ras_cor_status: 0x00000000
[ 784.085335] {1}[Hardware Error]: cxl_ras_cor_mask: 0x00000000
[ 784.085336] {1}[Hardware Error]: cap_control: 0x00002005
[ 784.085336] {1}[Hardware Error]: Header Log Registers:
[ 784.085337] {1}[Hardware Error]: 00000000: 00000000 e8b5e703 30400000 00000000
[ 784.085338] {1}[Hardware Error]: 00000010: 00000000 00000000 00000000 00000000
[ 784.085339] {1}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000000
[ 784.085340] {1}[Hardware Error]: 00000030: 00000000 00000000 00000000 00000000
[ 784.085340] {1}[Hardware Error]: Error 1, type: fatal
[ 784.085341] {1}[Hardware Error]: section_type: PCIe error
[ 784.085341] {1}[Hardware Error]: port_type: 10, root complex event collector
[ 784.085342] {1}[Hardware Error]: version: 3.0
[ 784.085343] {1}[Hardware Error]: command: 0x0504, status: 0x4010
[ 784.085343] {1}[Hardware Error]: device_id: 0000:3d:00.4
[ 784.085344] {1}[Hardware Error]: slot: 0
[ 784.085345] {1}[Hardware Error]: secondary_bus: 0x00
[ 784.085345] {1}[Hardware Error]: vendor_id: 0x8086, device_id: 0x0b23
[ 784.085346] {1}[Hardware Error]: class_code: 080700
[ 784.085346] {1}[Hardware Error]: aer_uncor_status: 0x00400000, aer_uncor_mask: 0x00180020
[ 784.085347] {1}[Hardware Error]: aer_uncor_severity: 0x00463010
[ 784.085347] {1}[Hardware Error]: TLP Header: 00000000 00000000 00000000 00000000
[ 784.085351] Kernel panic - not syncing: Fatal hardware error!
[ 784.085352] CPU: 0 PID: 0 Comm: swapper/0 Not tainted 6.10.6 #1
[ 784.085354] Hardware name: Intel Corporation AvenueCity/AvenueCity, BIOS
BHSDCRB1.IPC.0034.D82.2408060239 08/06/2024
[ 784.085355] Call Trace:
[ 784.085359] <NMI>
[ 784.085361] panic+0x32f/0x350
[ 784.085369] __ghes_panic+0x69/0x70
[ 784.085375] ghes_in_nmi_queue_one_entry.constprop.0+0x1d9/0x2b0
[ 784.085378] ghes_notify_nmi+0x5d/0xd0
[ 784.085380] nmi_handle+0x5f/0x170
[ 784.085384] ? intel_idle+0x5c/0xa0
[ 784.085388] default_do_nmi+0x76/0x190
[ 784.085391] exc_nmi+0xf0/0x130
[ 784.085392] end_repeat_nmi+0xf/0x53
[ 784.085397] RIP: 0010:intel_idle+0x5c/0xa0
[ 784.085399] Code: f0 48 89 d1 0f 01 c8 48 8b 06 a8 08 75 1c 8b 05 02 86 d9 01 85 c0 7e 07 0f 00 2d fb
6b 48 00 b9 01 00 00 00 48 89 f8 0f 01 c9 <f0> 80 66 02 df f0 83 44 24 fc 00 48 8b 06 a8 08 74 0b 65 81
25 df
[ 784.085400] RSP: 0018:ffffffff83203e28 EFLAGS: 00010046
[ 784.085402] RAX: 00000000000000021 RBX: 0000000000000002 RCX: 0000000000000001
[ 784.085403] RDX: 0000000000000000 RSI: ffffffff8320c940 RDI: 0000000000000021
[ 784.085403] RBP: ffd1ffbfcc223948 R08: 0000000000000002 R09: 0000000000000018
[ 784.085404] R10: 00000000000009057 R11: fff11001ffe42eb64 R12: ffffffff8348b180
[ 784.085404] R13: ffffffff8348b268 R14: 0000000000000002 R15: 0000000000000000
[ 784.085406] ? intel_idle+0x5c/0xa0
[ 784.085408] ? intel_idle+0x5c/0xa0
[ 784.085410] </NMI>
[ 784.085410] <TASK>
[ 784.085410] cpuidle_enter_state+0x82/0x400
[ 784.085413] cpuidle_enter+0x2d/0x40
[ 784.085416] cpuidle_idle_call+0xfc/0x160
[ 784.085419] do_idle+0x73/0xb0
[ 784.085421] cpu_startup_entry+0x29/0x30
[ 784.085422] rest_init+0xcd/0xd0
[ 784.085424] start_kernel+0x412/0x650
[ 784.085429] x86_64_start_reservations+0x18/0x30
[ 784.085433] x86_64_start_kernel+0x92/0xa0
[ 784.085435] common_startup_64+0x13e/0x148

```

```
[ 784.085439] </TASK>
[ 784.094718] Dumping ftrace buffer:
[ 784.094734] (ftrace buffer empty)
[ 784.111846] Kernel Offset: disabled
```

Pass / Fail	Criteria
PASS	Look for 1 CXL Protocol Error with a CXL uncorrectable status of Mem_Address_Parity on the CXL Root Port. Check: - section_type: CXL Protocol Error - agent_type: CXL Root Port - agent_address: 0000:3d:02.0 (CXL RP) - cxl_ras_uncor_status: 0x00000020
PASS	Look for 1 PCIe Uncorrectable Internal Error on the associated RCEC. Check: - Error type: fatal - section_type: PCIe error - port_type: root complex event collector - aer_uncor_status: 0x00400000 - device_id: 0000:3d:00.4 (RCEC)
PASS	No unexpected events seen. Check with your CXL card vendor for implementation specific behavior.

11.2.3 CXL Correctable Data Path/Protocol Error (Host to Device)

Test Objective

This test injects a correctable protocol error from host to device. The prime example used in this recipe injects a correctable link CRC errors, and the EP will set the CRC threshold in the Correctable Status register. See the *Birch Stream Platform: Granite Rapids, Sierra Forest, and Clearwater Forest Error Architecture and RAS*, document number 728644, for more information about CXL error reporting and flows.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B0
BIOS	BHSDCRB1.IPC.0034.D82.2408060239
OS Version	CentOS Stream 9
Kernel	6.10.6
HW Configuration	1 Socket
CScripts Version	BHS.2422.6001
Mode of operation	Firmware First Mode

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- ["CScripts on page 18"](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability on page 27](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration. [CXL Firmware First vs OS First](#) on how to set the knobs:

- Test is valid for either OS First Mode or Firmware First Mode

Recipe

1. Inject multiple correctable link CRC errors. A CRC threshold is detected by the CXL device.

- **Pseudo Script**

```
# Variables so this semi-pseudo test code can be copy/pastable with minor modifications.
test_socket = 0
test_cxl_port = "3a"

halt()

# Inject a Correctable CRC error into the CXL device
ei.mem.injectCxlError(test_socket, test_cxl_port, "ce_crc_error", verbose = True)

# Run cscript cxl error check functions for more info.
cxl.cxl_cmemo_err_check(test_socket, test_cxl_port)

go()
```

- **CScripts Output**

NOTE

The traceback output error is an execution result which has no impact on the test and can be ignored.

```
# Variables so this semi-pseudo test code can be copy/pastable with minor modifications.
>>> test_socket = 0
>>> test_cxl_port = "3a"

>>> halt()

# Inject a Correctable CRC error into the CXL device

>>> ei.mem.injectCxlError(test_socket, test_cxl_port, "ce_crc_error", verbose = True)
INFO: Socket 0 'pxp3.rp0' is operating in CXL2.0 mode!
Clearing previous Errors
Getting HDM information...
Memory Range: 0x6000000000-0xbfffffffff
Starting Memory Stress transactions...
Generate traffic at Host-managed Device Memory (HDM) address (reads)
Generate traffic at Host-managed Device Memory (HDM) address (writes)
Exception in thread Thread-330 (_cxl_mem_cache_transactions_producer):
Traceback (most recent call last):
  File "C:\Python310\lib\threading.py", line 1009, in _bootstrap_inner
    Injecting Error ...
    self.run()
  File "C:\Python310\lib\threading.py", line 946, in run
    self._target(*self._args, **self._kwargs)
  File "C:\cscripts\commlibs\error\base\mem.py", line 983, in _cxl_mem_cache_transactions_producer
    for addr in list(reversed(range(mem_range['lo'], mem_range['hi'], offset))):
MemoryError
Waiting until getting CRC Threshold..
Threshold Detected in:
```



```
CRC_Threshold
Stop traffic (read) at HDM address
Stopping Memory Stress transactions. The test finished successfully
```

STAGE	CHECKPOINT	STATUS
Setup	Access Validation	Pass
Setup	Input Validation - socket	Pass
Setup	Setup Configuration - halt	Pass
Setup	Setup Validation - BIOS knobs	Pass
Setup	Input Validation - port	Pass
Setup	Input Validation - errType	Pass
Setup	Input Validation - memory address	N/A
Setup	Error Register Access	Pass
Setup	Clear Errors	Pass
Setup	Get HDM Info	Pass
Setup	Start Memory Stress	Pass
Setup	Injector Enabling	Pass
Injection	Inject CXL.cache/CXL.mem Error	Pass
Verification	Error Check	Pass
Setup	Stop Memory Stress	Pass

```
# Run cscript cxl error check functions for more info.
>>> cxl.cxl_cmemo_err_check(test_socket, test_cxl_port)
```

```
++ HOST ERROR INFORMATION ++
INFO: Socket 0 'p3p3.rp0' is operating in CXL2.0 mode!
```

```
*****
* CXL 2.0 PortUncorrectable Error Dump on Socket0 Port = '3a' *
*****
```

```
Uncorrectable Error Status Register : 0x00000000
Uncorrectable Error Mask Register : 0x00003000
Uncorrectable Error Severity Register : 0x0001cfff
```

Name	Bit#	Mask	Severity	Status
Cache_Data_Parity	0	0	1	0
Cache_Address_Parity	1	0	1	0
Cache_BE_Parity	2	0	1	0
Cache_Data_ECC	3	0	1	0
Mem_Data_Parity	4	0	1	0
Mem_Address_Parity	5	0	1	0
Mem_BE_Parity	6	0	1	0
Mem_Data_ECC	7	0	1	0
REINIT_Threshold	8	0	1	0
Rsvd_Encoding_Violation	9	0	1	0
Poison_Received	10	0	1	0
Receiver_Overflow	11	0	1	0
LLRB_bad_CRC_condition	12	1	0	0
Retry.Ack received when LRSM is in Normal State	13	1	0	0
Internal_Error	14	0	1	0

```
Found 0 error(s)...
```

```
*****
* CXL 2.0 PortCorrectable Error Dump on Socket0 Port = '3a' *
*****
```

```
Correctable Error Status Register : 0x00000000
Correctable Error Mask Register : 0x00000000
```

Name	Bit#	Mask	Status
Cache_Data_ECC	0	0	0
Mem_Data_ECC	1	0	0
CRC_Threshold	2	0	0
Retry_Threshold	3	0	0
Cache_Poison_Received	4	0	0
Mem_Poison_Received	5	0	0
Physical_Layer_Error	6	0	0

```
Found 0 error(s)...
```

```
*****
* CXL 2.0 Port CXL cm.mem Error Capabilities and Control Register on Socket0 Port = '3a' *
*****
```

Error Capabilities and Control Register : 0x00002000

Error Capabilities and Control Register	Bit#	Control
First_Error_Pointer	3: 0	0
Multiple_Header_Recording_Capability	9: 9	0
Poison_Enabled	13:13	1

```
=====
++ DEVICE ERROR INFORMATION ++
```

```
*****
* CXL2.0 Port Uncorrectable Error Dump on Socket0 Port = '3a' *
*****
```

```
Uncorrectable Error Status Register : 0x00000000
Uncorrectable Error Mask Register : 0x00000000
Uncorrectable Error Severity Register : 0x0001cfff
```

Name	Bit#	Mask	Severity	Status
Cache_Data_Parity	0	0	1	0
Cache_Address_Parity	1	0	1	0
Cache_BE_Parity	2	0	1	0
Cache_Data_ECC	3	0	1	0
Mem_Data_Parity	4	0	1	0
Mem_Address_Parity	5	0	1	0
Mem_BE_Parity	6	0	1	0
Mem_Data_ECC	7	0	1	0
REINIT_Threshold	8	0	1	0
Rsvd_Encoding_Violation	9	0	1	0
Poison_Received	10	0	1	0
Receiver_Overflow	11	0	1	0
LLRB_bad_CRC_condition	12	0	0	0
Retry.Ack received when LRSM is in Normal State	13	0	0	0
Internal_Error	14	0	1	0

Found 0 error(s)...

```
*****
* CXL2.0 Port Correctable Error Dump on Socket0 Port = '3a' *
*****
```

```
Correctable Error Status Register : 0x00000004
Correctable Error Mask Register : 0x00000000
```

Name	Bit#	Mask	Status
Cache_Data_ECC	0	0	0
Mem_Data_ECC	1	0	0
CRC_Threshold	2	0	1
Retry_Threshold	3	0	0
Cache_Poison_Received	4	0	0
Mem_Poison_Received	5	0	0
Physical_Layer_Error	6	0	0

Found 1 error(s)...

0 : CRC_Threshold is detected!

```
*****
* CXL2.0 Port CXL cm.mem Error Capabilities and Control Register on Socket0 Port = '3a' *
*****
```

Error Capabilities and Control Register : 0x00002000

Error Capabilities and Control Register	Bit#	Control
First_Error_Pointer	3: 0	0

```
| Multiple_Header_Recording_Capability | 9: 9 | 0 |
| Poison_Enabled | 13:13 | 1 |
-----
```

```
>>> go()
```

2. Observe the FTrace output in the OS.

NOTE

See [CXL Firmware First vs OS First](#) for any known issues with OS First Mode or Firmware First Mode.

Pass / Fail	Criteria
PASS	Check for CXL.io Correctable Internal Error on EP: - Check Event type: aer_event - Check BDF info (can vary): 0000:8a:00.0 (EP) - Check Error type : PCIe Bus Error - Check Severity= Advisory Non-Fatal Error Corrected Internal Error
OS FAIL	Check for CXL Protocol Error Corrected CRC error received on the EP. Look for 1, but found 0 cxl_aer_correctable_error ftrace event: Due to the above mentioned OS bug, Event type is shown as a Non-standard event - Check: "host" == "0000:8a:00.0" (EP) (can vary) - Severity= 1 - Check: "status" & 0x4 : Correctable CRC error
PASS	No unexpected events seen. Check with your CXL card vendor for implementation specific behavior.

```
# cat trace
# tracer: nop
#
# entries-in-buffer/entries-written: 3/3   #P:72
#
#          -----> irqs-off/BH-disabled
#          /-----> need-resched
#          | /-----> hardirq/softirq
#          || /-----> preempt-depth
#          ||| /-----> migrate-disable
#          |||| /-----> delay
#
#          TASK-PID   CPU#  | TIMESTAMP | FUNCTION
#          | |       |   |   |   |   |
# kworker/0:1-10     [000] d..1. 3041.049343: non_standard_event:
# severity: 1; sec type:b4efb980-b552-e34d-a777-68784b771048; FRU:
# 00000000-0000-0000-0000-000000000000 ; data len:260; raw data:67 00 00 00 00
# 00 00 00 02 00 00 00 00 00 00 00 00 00 00 00 8a 00 00 00 00 00 98 1e e2 01 00 00 00
# 00 02 05 00 00 00 00 00 00 00 1d ed 4a 04 81 81 1d 22 10 b0 02 00 c1 8f 2c 11 37
# 50 09 00 05 61 43 00 40 00 85 10 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
# 00 10 00 b1 00 00 00 00 00 3e 00 80 01 05 00 1e 01 00 00 00 00 00 00 00 00 00 38
# 00 58 00 00 00 00 00 23 00 c1 40 98 1e 81 03 00 00 9e e8 06 40 00 00 00 00 00 00
# 00 01 00 00 00 20 00 00 00 0b 81 00 00 00 00 00 00 00 00 00 00 00 00 00 00 48
# 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ff cf 01 00 04 00 00
# 00 00 00 00 00 00 20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
# 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
# 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
# kworker/0:1-10     [000] .N... 3041.074448: aer_event: 0000:8a:00.0
# PCIe Bus Error: severity=Corrected, Advisory Non-Fatal Error|Corrected
# Internal Error, TLP Header=Not available
```

3. Linux dmesg output

NOTE

The dmesg output depends on several factors like the OS distro and kernel. Information is used for debugging purposes only.

```
[ 3039.954054] {1}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 0
[ 3039.965693] systemd-coredump[4050]: Resource limits disable core dumping for process 1887 (systemd-journal).
[ 3039.981355] {1}[Hardware Error]: It has been corrected by h/w and requires no further action
[ 3039.981358] {1}[Hardware Error]: event severity: corrected
[ 3039.981360] {1}[Hardware Error]: Error 0, type: corrected
[ 3039.981361] {1}[Hardware Error]: section_type: CXL Protocol Error
[ 3039.981362] {1}[Hardware Error]: agent_type: 2, CXL Device
[ 3039.981363] {1}[Hardware Error]: agent_address: 0000:8a:00.0
[ 3039.981366] {1}[Hardware Error]: slot: 0
[ 3039.992576] systemd-coredump[4050]: Process 1887 (systemd-journal) of user 0 dumped core.
[ 3040.002143] {1}[Hardware Error]: vendor_id: 0x1e98, device_id: 0x01e2
[ 3040.002146] {1}[Hardware Error]: sub_vendor_id: 0x0000, sub_device_id: 0x0000
[ 3040.002147] {1}[Hardware Error]: class_code: 0502
[ 3040.010726] systemd[1]: systemd-journald.service: Main process exited, code=dumped, status=6/ABRT
[ 3040.016030] {1}[Hardware Error]: DVSEC length: 0x0038
[ 3040.016031] {1}[Hardware Error]: CXL DVSEC:
[ 3040.016034] {1}[Hardware Error]: 00000000: 40c10023 03811e98 e89e0000 00004006
[ 3040.016035] {1}[Hardware Error]: 00000010: 00000000 00000001 00000020 0000810b
[ 3040.016036] {1}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000048
[ 3040.016037] {1}[Hardware Error]: 00000030: 00000000 00000000
[ 3040.016037] {1}[Hardware Error]: Error log length: 0x0058
[ 3040.016038] {1}[Hardware Error]: CXL Error Log:
[ 3040.016038] {1}[Hardware Error]: cxl_ras_uncor_status: 0x00000000
[ 3040.016039] {1}[Hardware Error]: cxl_ras_uncor_mask: 0x00000000
[ 3040.016040] {1}[Hardware Error]: cxl_ras_uncor_severity: 0x0001cfff
[ 3040.016040] {1}[Hardware Error]: cxl_ras_cor_status: 0x00000004
[ 3040.016041] {1}[Hardware Error]: cxl_ras_cor_mask: 0x00000000
[ 3040.016041] {1}[Hardware Error]: cap_control: 0x00002000
[ 3040.016042] {1}[Hardware Error]: Header Log Registers:
[ 3040.016042] {1}[Hardware Error]: 00000000: 00000000 00000000 00000000 00000000
[ 3040.016043] {1}[Hardware Error]: 00000010: 00000000 00000000 00000000 00000000
[ 3040.016043] {1}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000000
[ 3040.016044] {1}[Hardware Error]: 00000030: 00000000 00000000 00000000 00000000
[ 3040.016044] {1}[Hardware Error]: Error 1, type: corrected
[ 3040.016045] {1}[Hardware Error]: section_type: PCIe error
[ 3040.016046] {1}[Hardware Error]: port_type: 0, PCIe end point
[ 3040.016047] {1}[Hardware Error]: version: 3.0
[ 3040.016048] {1}[Hardware Error]: command: 0x0546, status: 0x0010
[ 3040.016048] {1}[Hardware Error]: device_id: 0000:8a:00.0
[ 3040.016049] {1}[Hardware Error]: slot: 0
[ 3040.016049] {1}[Hardware Error]: secondary_bus: 0x00
[ 3040.016050] {1}[Hardware Error]: vendor_id: 0x1dfa, device_id: 0x01e2
[ 3040.016050] {1}[Hardware Error]: class_code: 050210
[ 3040.016253] cxl_pci 0000:8a:00.0: AER: aer_status: 0x00006000, aer_mask: 0x00000000
[ 3040.029682] cxl_pci 0000:8a:00.0: [13] NonFatalErr
[ 3040.041150] cxl_pci 0000:8a:00.0: [14] CorrIntErr
[ 3040.041152] cxl_pci 0000:8a:00.0: AER: aer_layer=Transaction Layer, aer_agent=Receiver ID
```

NOTE

The Advisory Non-Fatal error returned along with the Corrected error is due to an OS bug and can be ignored.

11.3 Event Log Notifications

Event log notifications are notifications from the "media" attached behind the CXL device. For instance, the memory, DDR, cache, and so on. These are logs retrieved via CCI via mailboxes or MCTP. The system is notified of these event logs via MSI for OS First Mode, MEFN VDM for Firmware First Mode, or MCTP Notification for BMC observation.

OS should see the same FTrace outputs regardless if the system is in OS First Mode or Firmware First mode.

In OS first mode:

1. OS will process the MSI.
2. "Get Event Log" via primary mailbox.
3. "Clear Event Log" via primary mailbox.
4. Send results to CXL Ftrace.

In Firmware first mode:

1. BIOS will process the SMI/MEFN VDM.
2. "Get Event Log" via primary mailbox.
3. "Clear Event Log" via primary mailbox.
4. Send ACPI CXL CPER Record to OS.
5. OS will process ACPI CXL CPER record to CXL FTrace.

The primary objective of these recipes are to assist in confirming that both BIOS and OS are accurately detecting and harvesting CXL Event Records. It is important to note that the specific event records logged by a CXL endpoint can differ between vendors. Therefore, it is recommended to consult with your CXL vendor to understand the expected behaviors and characteristics of their event records.

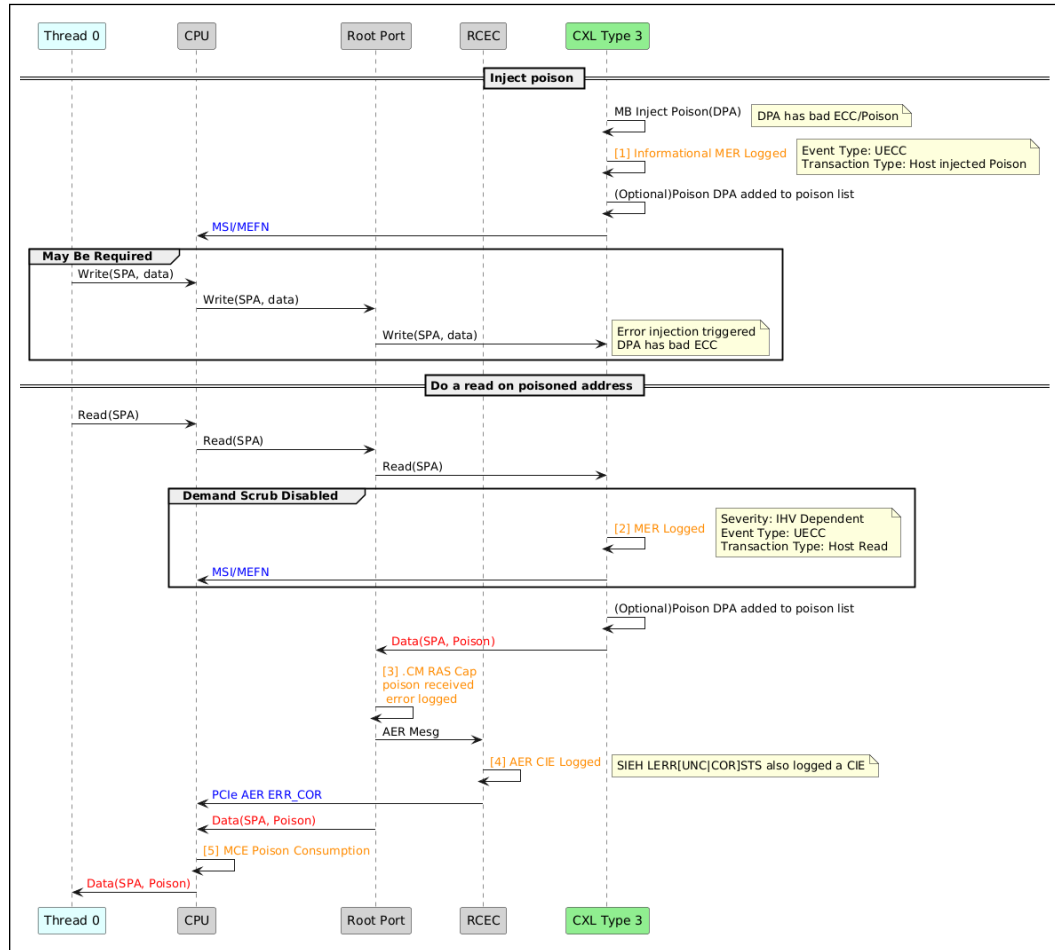
11.3.1 Poison Injection (CScripts)

Test Objective

Inject an uncorrectable ECC error into a CXL Type 3 device using CScripts.

Warning: There are many different types of implementations. Our current setup has patrol scrub disabled and demand scrub disabled. You may need to write to force the error injection, which may also end up clearing the ECC and Poison.

Figure 20. Poison Injection Flow



Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B0
BIOS	BHSDCRB1.IPC.0034.D82.2408060239
OS Version	CentOS Stream 9
Kernel	6.10.6
HW Configuration	1 Socket
CScripts Version	BHS.2422.6001
Mode of Operation	Firmware First Mode

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- ["CScripts on page 18"](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability on page 27](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration. [CXL Firmware First vs OS First](#) on how to set the knobs:

- Test is valid for either OS First Mode or Firmware First Mode

Recipe

1. Issue an "Inject Poison" CCI Command in CScripts:

NOTE

Inject poison behavior may differ from one CXL Card to another. Check with your IHV vendor regarding the behavior of inject poison, as you may need a memory write to the address followed by a wbinvd to trigger the error

• Pseudo Script

```
# Populate the variables below:
dpa_ta = 0x5000 # DPA Address to "inject poison"
test_socket = 0 # socket of CXL device
test_port = "1a" # port of CXL device

halt
itp.threads.wbinvd()

# Injecting poison will usually cause a "informational" Event Log.

cxl.cxl_device_command(test_socket, test_port, "injectPoison", dpa_ta)

# Check Device Register->Event Status Register to see what type of Event Log was logged
# and if there was an Event Log

cxldump = cxl.capabilities(test_socket, test_port, False)
cxldump.Endpoint1.cxl_dev_regs.dev_sts_regs.event_status.show()

# Clear cache so OS reads will go to CXL using separate program.
# If your IHV or CXL card does not support Demand Scrubbing, everytime you do a read it will
result in a new event log
# If your IHV or CXL card supports Patrol Scrubbing, you may get an error before you do a read
itp.threads.wbinvd()

go
```

• CScripts Output

```
>>> # Populate the variables below:
>>> dpa_ta = 0x3000 # DPA Address to "inject poison"
>>> test_socket = 0 # socket of CXL device
>>> test_port = "1a" # port of CXL device
>>>
>>> halt
>>> itp.threads.wbinvd()
>>>
>>> # Injecting poison will usually cause a "informational" Event Log.
>>>
>>> cxl.cxl_device_command(test_socket, test_port, "injectPoison", dpa_ta)

Reading available Opcodes/commands for Socket:0, Port:1a device ...
```

```

RAW table
=====
|Command      |Combined Opcode|Input Payload Size[Bytes]|Output Payload Size[Bytes]|Input
Payload Value|Output Payload Value|
-----
|injectPoison|0x4301          |0x8                      |0x0                      |
0x5000      |0x0              |                          |                          |
=====

Detailed: Input Payload table
=====
|injectPoison: Input Payload Fields|Input
Value                                |units|
-----
|Inject Poison Physical Address    |Reserved = 0x0, DPA[7:6] = 0x0, DPA[15:8] = 0x50,
DPA[23:16] = 0x0, DPA[31:24] = 0x0, DPA[39:32] = 0x0, DPA[47:40] = 0x0, DPA[55:48] = 0x0,
DPA[63:56] = 0x0, |-              |
=====

# Check Device Register->Event Status Register to see what type of Event Log was logged and if
there was an Event Log

>>> cxldump = cxl.capabilities(test_socket, test_port, False)
WARNING: CAP ID = 0x007 decoder is not supported by CScripts, EXIT...
WARNING: CAP ID = 0x007 decoder is not supported by CScripts, EXIT...
WARNING: CAP ID = 0x3 decoder is not supported by CScripts, EXIT...
WARNING: CAP ID = 0x3 decoder is not supported by CScripts, EXIT...
WARNING: CAP ID = 0x4000 decoder is not supported by CScripts, EXIT...
WARNING: CAP ID = 0x4000 decoder is not supported by CScripts, EXIT...

>>> cxldump.EndPoint1.cxl dev_regs.dev_sts_regs.event_status.show()
Base Address = 0x1e5ffd011000

[0x000] Event Status (addr = 0x1e5ffd011000, val = 0x1)
-----
[ 0: 0] Informational Event Log..... 0x1
[ 1: 1] Warning Event Log..... 0x0
[ 2: 2] Failure Event Log..... 0x0
[ 3: 3] Fatal Event Log..... 0x0
[ 4: 4] Fatal Event Log..... 0x0

>>>
>>>
>>> # Clear cache so OS reads will go to CXL using separate program.
>>> itp.threads.wbinvd()
>>>
>>> go
>>>

[GPC core group] Resuming on 144 devices -- 14:07:23.242395 2024-09-03

```

2. Observe the FTrace output in the OS.

NOTE

See [CXL Firmware First vs OS First](#) for any known issues with OS First Mode or Firmware First Mode.

Pass / Fail	Criteria
PASS	Check for a Memory Event Record. Look for 1 CXL_DRAM event - log = 0 : Informational - Descriptor= 1 : 'UNCORRECTABLE_EVENT' - type= 0 : 'ECC Error' - Transaction type =4 : 'Host Inject Poison'
PASS	<i>Note:</i> Additional event(s) are seen with this CXL device. Check with your CXL card vendor for implementation specific behavior.

```
# cat /sys/kernel/debug/tracing/trace
# tracer: nop
#
# entries-in-buffer/entries-written: 4/4      #P:72
#
#          -----> irqs-off/BH-disabled
#          /-----> need-resched
#          | /-----> hardirq/softirq
#          || /-----> preempt-depth
#          ||| /-----> migrate-disable
#          |||| /-----> delay
#
# TASK-PID      CPU#  | TIMESTAMP | FUNCTION
# |-----|-----|-----|-----|
# <...>-10      [000] .N...   301.566192: cxl_dram: memdev=mem1 host=0000:3e:00.0
serial=2458346470421228829 log=Informational : time=1725393918906485576
uuid=601dcbb3-9c06-4eab-b8af-4e9bfb5c9624 len=128 flags='' handle=10 related_handle=0
maint_op_class=0 : dpa=5000 dpa_flags='' descriptor='UNCORRECTABLE_EVENT' type='ECC Error'
transaction_type='Host Inject Poison' channel=0 rank=0 nibble_mask=0 bank_group=0 bank=0 row=0
column=640 cor_mask=00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 validity_flags='CHANNEL|RANK|BANK GROUP|BANK|ROW|COLUMN' hpa=600000f000
region=region0 region_uuid=00000000-0000-0000-0000-000000000000
kworker/0:1-10      [000] .N...   301.578803: cxl_dram: memdev=mem1 host=0000:3e:00.0
serial=2458346470421228829 log=Informational : time=1725393918906512319
uuid=601dcbb3-9c06-4eab-b8af-4e9bfb5c9624 len=128 flags='' handle=f related_handle=0
maint_op_class=0 : dpa=5040 dpa_flags='' descriptor='UNCORRECTABLE_EVENT' type='ECC Error'
transaction_type='Host Inject Poison' channel=0 rank=0 nibble_mask=0 bank_group=0 bank=0 row=0
column=640 cor_mask=00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 validity_flags='CHANNEL|RANK|BANK GROUP|BANK|ROW|COLUMN' hpa=600000f040
region=region0 region_uuid=00000000-0000-0000-0000-000000000000
```

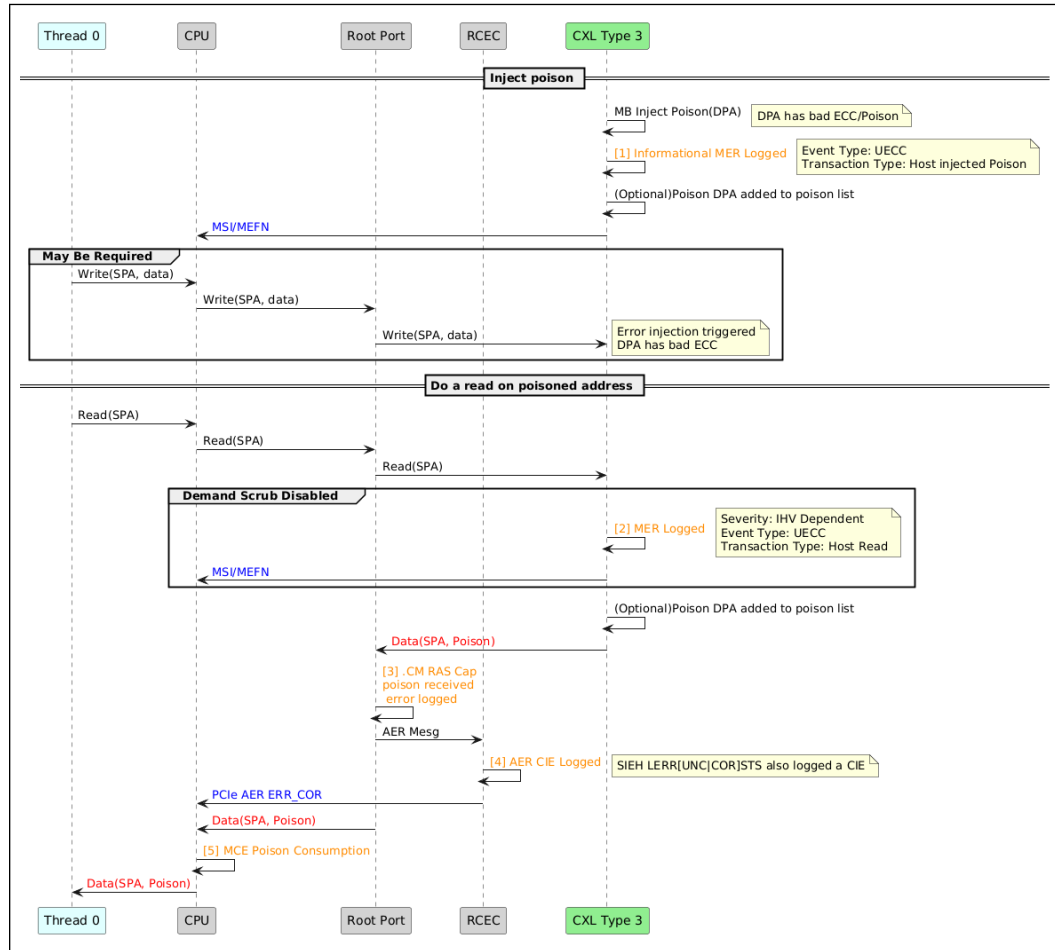
- To Create a "Poison Consumption", refer to [Poison Consumption](#) chapter for the next steps.

11.3.2 Poison Injection (OS)

Test Objective

Inject an uncorrectable ECC error into a CXL Type 3 device using Linux OS.

Figure 21. Poison Injection Flow



Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B0
BIOS	BHSDCRB1.IPC.0034.D82.2408060239
OS Version	CentOS Stream 9
Kernel	6.10.6
HW Configuration	1 Socket
CScripts Version	BHS.2422.6001
Mode of Operation	OS First Mode

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- ["CScripts on page 18"](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability on page 27](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration. [CXL Firmware First vs OS First](#) on how to set the knobs:

- Test is valid for either OS First Mode or Firmware First Mode

Recipe

- Issue an "Inject Poison" CCI Command via Linux debugfs.

```
# Example below injects an error at DPA 0x5000 on "mem0"
echo 0x5000 > /sys/kernel/debug/cxl/mem0/inject_poison

# Use "clear_poison" to clear the poison later on.
# e.g. echo 0x5000 > /sys/kernel/debug/cxl/mem0/clear_poison
```

Observe the FTrace output in the OS.

NOTE

See [CXL Firmware First vs OS First](#) for any known issues with OS First Mode or Firmware First Mode.

Pass / Fail	Criteria
PASS	Check for Injection event. Look for 1 CXL_poison ftrace event. Check: • trace_type=Inject
PASS	Check for a Memory Event Record. Look for 1 CXL_DRAM event • log = 0 : Informational • Descriptor= 1 : 'UNCORRECTABLE_EVENT' , type= 0 : 'ECC Error' • Transaction type =4 : 'Host Inject Poison'
PASS	<i>Note:</i> Additional event(s) are seen with this CXL device. Check with your CXL card vendor for implementation specific behavior.

```
# You should see 2 ftrace events.
# - Error injection event with HPA and DPA address
# - Memory Event Record on the injected event (DRAM event)

cat /sys/kernel/debug/tracing/trace
# tracer: nop
#
# entries-in-buffer/entries-written: 2/2   #P:72
#
#          -----=> irqs-off/BH-disabled
#          /-----=> need-resched
#          | /-----=> hardirq/softirq
#          || /-----=> preempt-depth
#          ||| /-----=> migrate-disable
#          |||| /-----=> delay
#
# TASK-PID   CPU#  | TIMESTAMP | FUNCTION
# | | | | | | | | | | | | | | | | | |
bash-4037   [079]  . . . . .  111.450606: cxl_poison: memdev=mem0 host=0000:64:00.0
```

- To Create a "Poison Consumption", refer to [Poison Consumption](#) chapter for the next steps.

Test Objective

Configuration

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B0
BIOS	BHSDCRB1.IPC.0034.D82.2408060239
OS Version	CentOS Stream 9
Kernel	6.10.6
HW Configuration	1 Socket
CScripts Version	BHS.2422.6001
Mode of Operation	OS First Mode

- ["CScripts on page 18"](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability on page 27](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration. [CXl Firmware First vs OS First](#) on how to set the knobs:

- Test is valid for either in OS First Mode or Firmware First Mode
- In Firmware First mode, the Temperature event is signaled to the host with an MEFN, and in OS first mode it is signaled with a MSI.

Recipe

1. Get the Alert Configuration for your targeted CXl device on the system.

```
>>> cxl.cxl_device_command(0, "2a")
Halting for function execution...
Reading available Opcodes/commands for Socket:0, Port:2a device ...

[GPC core group] Halt on 144 devices -- 12:29:46.869512 2024-09-03
Commands list:
MemCommands:
1) identifyMemory
2) getHealthInfo
3) getAlertConfiguration
4) setAlertConfiguration
5) getPoisonList
6) injectPoison
7) clearPoison
DevCommands:
8) getEventRecords
9) clearEventRecords
10) getEventInterruptPolicy
11) setEventInterruptPolicy
12) getSupportedLogs
13) getLog

q) Exit

Select one option (type a number): 3
Command selected: getAlertConfiguration

RAW table
=====
|Command|Combined Opcode|Input Payload Size[Bytes]|Output Payload Size[Bytes]|
|-----|-----|-----|-----|
|getAlertConfiguration|0x4201|0x0|0x10|
|0x0|0x64ffff1ffabfffbffa100000e00|
=====

Detailed: Output Payload table
=====
=====
|getAlertConfiguration: Output Payload Fields|Output
Value|units|
|-----|-----|
|Valid_Alerts|
Disabled|
|-----|-----|
|Programmable_Alerts|
Device Under-Temperature, Corrected Volatile Memory Error|[[Device Over-Temperature,
Programmable Warning Threshold Prog
by host|
```

2. Get the current temperature of the CXL card

August 2025
Doc. No.: 742874, Rev.: 1.5

```

Detailed: Output Payload table
=====
=====
=====
|getHealthInfo: Output Payload Fields|Output
Value                                |units      |
-----
|Health_Status                      |Maintenance Needed: NO, Performance
Degraded: NO, Hardware Replacement Needed:
NO                                  |-          |
|Media_Status                       |
Normal                             |
|Additional_Status                  |Life Used = Normal, Device Temperature
= Normal, Corrected Volatile Error Count = Normal, Corrected Persistent Error
Count = Normal, |-          |
|Life_Used                          |Not
Implemented                        |
|Device_Temperature                 |%          |
36                                 |°C         |
|Dirty_Shutdown_Count              |
0                                  |count      |
|Corrected_Volatile_Error_Count     |
0                                  |# CorError|
|Corrected_Persistent_Error_Count   |
0                                  |# CorError|
=====
=====
=====

```

3. Enable FTrace

```

# echo 1 > /sys/kernel/debug/tracing/events/cxl/enabled
# cxl monitor

```

4. Set the "Device_Over-Temperature_Programmable_Warning_Threshold" value to be lower than the current temperature of the card.

NOTE

Input payloads are in hex.

```

>>> cxl.cxl_device_command(0, "2a")
Halting for function execution...
Commands list:
MemCommands:
1) identifyMemory
2) getHealthInfo
3) getAlertConfiguration
4) setAlertConfiguration
5) getPoisonList
6) injectPoison
7) clearPoison
DevCommands:
8) getEventRecords
9) clearEventRecords

```

```

10) getEventInterruptPolicy
11) setEventInterruptPolicy
12) getSupportedLogs
13) getLog

q) Exit

Select one option (type a number):
    [GPC core group] Halt on 144 devices -- 12:37:47.730627 2024-09-03
4
Command selected: setAlertConfiguration

Input payload is required!!

Options for Valid_Alert_Actions:
1) Disabled
2) Life Used Programmable Warning Threshold Enable Alert Action
3) Device Over-Temperature Programmable Warning Threshold Enable Alert Action
4) [Life Used, Device Over-Temperature]Programmable Warning Threshold Enable
Alert Action
5) Device Under-Temperature Programmable Warning Threshold Enable Alert Action
6) [Life Used, Device Under-Temperature]Programmable Warning Threshold Enable
Alert Action
7) [Device Over-Temperature, Device Under-Temperature] Programmable Warning
Threshold Enable Alert Action
8) [Life Used, Device Over-Temperature, Device Under-Temperature]
Programmable Warning Threshold Enable Alert Action
9) Corrected Volatile Memory Error Programmable Warning Threshold Enable
Alert Action
10) [Life Used, Corrected Volatile Memory Error]Programmable Warning
Threshold Enable Alert Action
11) [Device Over-Temperature, Corrected Volatile Memory Error] Programmable
Warning Threshold Enable Alert Action
12) [Life Used, Device Over-Temperature, Corrected Volatile Memory
Error]Programmable Warning Threshold Enable Alert Action
13) [Device Under-Temperature, Corrected Volatile Memory Error] Programmable
Warning Threshold Enable Alert Action
14) [Life Used, Device Under-Temperature, Corrected Volatile Memory Error]
Programmable Warning Threshold Enable Alert Action
15) [Device Over-Temperature, Device Under-Temperature, Corrected Volatile
Memory Error] Programmable Warning Threshold Enable Alert Action
16) [Life Used, Device Over-Temperature, Device Under-Temperature, Corrected
Volatile Memory Error]Programmable Warning Threshold Enable Alert Action
Select one option (type a number): 3

Options for Enable_Alert_Actions:
1) Disabled All
2) Enabled ->Life Used Programmable Warning Threshold
3) Enabled ->Device Over-Temperature Programmable Warning Threshold
4) Enabled ->[Life Used, Device Over-Temperature]Programmable Warning
Threshold
5) Enabled ->Device Under-Temperature Programmable Warning Threshold
6) Enabled ->[Life Used, Device Under-Temperature]Programmable Warning
Threshold
7) Enabled ->[Device Over-Temperature, Device Under-Temperature] Programmable
Warning Threshold
8) Enabled ->[Life Used, Device Over-Temperature, Device Under-Temperature]
Programmable Warning Threshold
9) Enabled ->Corrected Volatile Memory Error Programmable Warning Threshold
10) Enabled ->[Life Used, Corrected Volatile Memory Error]Programmable
Warning Threshold
11) Enabled ->[Device Over-Temperature, Corrected Volatile Memory Error]
Programmable Warning Threshold
12) Enabled ->[Life Used, Device Over-Temperature, Corrected Volatile Memory
Error]Programmable Warning Threshold
13) Enabled ->[Device Under-Temperature, Corrected Volatile Memory Error]
Programmable Warning Threshold
14) Enabled ->[Life Used, Device Under-Temperature, Corrected Volatile Memory
Error] Programmable Warning Threshold
15) Enabled ->[Device Over-Temperature, Device Under-Temperature, Corrected
Volatile Memory Error] Programmable Warning Threshold

```



```

16) Enabled ->[Life Used, Device Over-Temperature, Device Under-Temperature,
Corrected Volatile Memory Error]Programmable Warning Threshold
Select one option (type a number): 3

Options for Life_Used_Programmable_Warning_Threshold:
Type numeric value for input Field Payload(0x0-0xFF): 0x5a

Options for Device_Over-Temperature_Programmable_Warning_Threshold:
Type numeric value for input Field Payload(0x0-0xFFFF): 0x20

Options for Device_Under-Temperature_Programmable_Warning_Threshold:
Type numeric value for input Field Payload(0x0-0xFFFF): 0

Options for Corrected_Volatile_Memory_Error_Programmable_Warning_Threshold:
Type numeric value for input Field Payload(0x0-0xFFFF): 0x57

Options for Corrected_Persistent_Memory_Error_Programmable_Warning_Threshold:
Type numeric value for input Field Payload(0x0-0xFFFF): 0x5
Input Payload: 0x5005700000020005a0202

RAW table
=====
|Command          |Combined Opcode|Input Payload Size[Bytes]|Output
Payload Size[Bytes]|Input Payload Value      |Output Payload Value|
=====
|setAlertConfiguration|0x4202          |0xc                      |
0x0                |0x5005700000020005a0202|0x0                      |
=====

Detailed: Input Payload table
=====
|setAlertConfiguration: Input Payload Fields      |Input
Value                                             |units |
=====
|Valid Alert Actions                               |Device Over-
Temperature Programmable Warning Threshold Enable Alert Action|-
|Enable Alert Actions                             |Enabled -
>Device Over-Temperature Programmable Warning Threshold      |
|Life_Used_Programmable_Warning_Threshold          |
90                                                    |
% |                                                    |
|Reserved                                           |
0                                                    |
N/A |                                                    |
|Device_Over-Temperature_Programmable_Warning_Threshold      |
32                                                    |
C |                                                    |
|Device_Under-Temperature_Programmable_Warning_Threshold      |
0                                                    |
C |                                                    |
|Corrected_Volatile_Memory_Error_Programmable_Warning_Threshold |
87                                                    |
Errors|
|Corrected_Persistent_Memory_Error_Programmable_Warning_Threshold|
5                                                    |
Errors|
=====

```

5. Observe the FTrace output in the OS.

NOTE

See [CXL Firmware First vs OS First](#) for any known issues with OS First Mode or Firmware First Mode.

Pass /Fail	Criteria
PASS	Check for alert event. Look for 1 cxl_memory_module ftrace event: Check: "event_type" & 0xf == 0x3 : Temperature Change
PASS	No unexpected events seen. Check with your CXL card vendor for implementation specific behavior.

```
[root@gnr-bkc ~]# cxl monitor
# tracer: nop
#
# entries-in-buffer/entries-written: 1/1      #P:144
#
#          -----> irqs-off/BH-disabled
#          /-----> need-resched
#          | /-----> hardirq/softirq
#          || /-----> preempt-depth
#          ||| /-----> migrate-disable
#          |||| /-----> delay
#          TASK-PID    CPU#    | TIMESTAMP    FUNCTION
#          | |         | |         | |
irq/237-0000:64-2323    [050]    ..... 699.294713: cxl_memory_module:
memdev=mem1 host=0000:64:00.0 serial=2458271785234918685
log=Warning : time=1725388695344198388 uid=fe927475-dd59-4339-
a586-79bab113b774 len=128 flags='0x1' handle=10 related handle=0
maint_op_class=0 : event_type='Temperature Change' health_status=''
media_status='Normal' as_life_used=Normal as_dev_temp=Warning
as_cor_vol_err_cnt=Normal as_cor_per_err_cnt=Normal life_used=255
device_temp=36 dirty_shutdown_cnt=0 cor_vol_err_cnt=0 cor_per_err_cnt=0
```

11.3.4 Temperature Alert (OS)

Test Objective

Issue a temperature alert event to trigger a CXL Event Log interrupt.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B0
BIOS	BHSDCRB1.IPC.0034.D82.2408060239
OS Version	CentOS Stream 9
Kernel	6.10.6
<i>continued...</i>	

HW Configuration	1 Socket
CScripts Version	BHS.2422.6001
Mode of Operation	Firmware First Mode

Precondition and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- ["CScripts on page 18"](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability on page 27](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration. [CXl Firmware First vs OS First](#) on how to set the knobs:

- Test is valid for either in OS First Mode or Firmware First Mode
- In Firmware First mode, the Temperature event is signaled to the host with an MEFN, and in OS first mode it is signaled with a MSI.

Recipe

1. a. Get the current temperature for your targeted CXl device.
b. Set the over-temperature alert to a value less than the current temperature

Instructions

```
[root@gnr-bkc ~]# cxl list -H -m mem0
[
  {
    "memdev":"mem0",
    "ram_size":137438953472,
    "health":{
      "maintenance_needed":false,
      "performance_degraded":false,
      "hw_replacement_needed":false,
      "media_normal":true,
      "media_not_ready":false,
      "media_persistence_lost":false,
      "media_data_lost":false,
      "media_powerloss_persistence_loss":false,
      "media_shutdown_persistence_loss":false,
      "media_persistence_loss_imminent":false,
      "media_powerloss_data_loss":false,
      "media_shutdown_data_loss":false,
      "media_data_loss_imminent":false,
      "ext_life_used":"normal",
      "ext_temperature":"normal",
      "ext_corrected_volatile":"normal",
      "ext_corrected_persistent":"normal",
      "life_used_percent":-95,
      "temperature":36,
      "dirty_shutdowns":0,
      "volatile_errors":0,
      "pmem_errors":0
    },
    "serial":2458271785234918685,
    "numa_node":0,
    "host":"0000:64:00.0"
  }
]
```

```

]
[root@gnr-bkc ~]# cxl list -A -m mem0

[
  {
    "memdev":"mem0",
    "ram_size":137438953472,
    "alert_config":{
      "life_used_prog_warn_threshold_valid":false,
      "dev_over_temperature_prog_warn_threshold_valid":true,
      "dev_under_temperature_prog_warn_threshold_valid":false,
      "corrected_volatile_mem_err_prog_warn_threshold_valid":false,
      "corrected_pmem_err_prog_warn_threshold_valid":false,
      "life_used_prog_warn_threshold_writable":false,
      "dev_over_temperature_prog_warn_threshold_writable":true,
      "dev_under_temperature_prog_warn_threshold_writable":true,
      "corrected_volatile_mem_err_prog_warn_threshold_writable":true,
      "corrected_pmem_err_prog_warn_threshold_writable":false,
      "life_used_crit_alert_threshold":0,
      "life_used_prog_warn_threshold":0,
      "dev_over_temperature_crit_alert_threshold":65441,
      "dev_under_temperature_crit_alert_threshold":65531,
      "dev_over_temperature_prog_warn_threshold":65486,
      "dev_under_temperature_prog_warn_threshold":65521,
      "corrected_volatile_mem_err_prog_warn_threshold":100,
      "corrected_pmem_err_prog_warn_threshold":0
    },
    "serial":2458271785234918685,
    "numa_node":0,
    "host":"0000:64:00.0"
  }
]
[root@gnr-bkc ~]# cxl set-alert-config mem0 --over-temperature-threshold
20 --over-temperature-alert on

{
  "memdev":"mem0",
  "ram_size":"128.00 GiB (137.44 GB)",
  "alert_config":{
    "life_used_prog_warn_threshold_valid":false,
    "dev_over_temperature_prog_warn_threshold_valid":true,
    "dev_under_temperature_prog_warn_threshold_valid":false,
    "corrected_volatile_mem_err_prog_warn_threshold_valid":false,
    "corrected_pmem_err_prog_warn_threshold_valid":false,
    "life_used_prog_warn_threshold_writable":false,
    "dev_over_temperature_prog_warn_threshold_writable":true,
    "dev_under_temperature_prog_warn_threshold_writable":true,
    "corrected_volatile_mem_err_prog_warn_threshold_writable":true,
    "corrected_pmem_err_prog_warn_threshold_writable":false,
    "life_used_crit_alert_threshold":0,
    "life_used_prog_warn_threshold":0,
    "dev_over_temperature_crit_alert_threshold":65441,
    "dev_under_temperature_crit_alert_threshold":65531,
    "dev_over_temperature_prog_warn_threshold":65516,
    "dev_under_temperature_prog_warn_threshold":65521,
    "corrected_volatile_mem_err_prog_warn_threshold":100,
    "corrected_pmem_err_prog_warn_threshold":0
  },
  "serial":"0x221d892a084aed1d",
  "numa_node":0,
  "host":"0000:64:00.0"
}
cxl memdev: cmd_set_alert_config: set alert configuration for 1 mem

```

2. Observe the FTrace output in the OS.

NOTE

See [CXL Firmware First vs OS First](#) for any known issues with OS First Mode or Firmware First Mode.

Pass / Fail	Criteria
PASS	Check for alert event. Look for 1 cxl_memory_module ftrace event: Check: "event_type" & 0xf == 0x3 : "Temperature Change"
PASS	No unexpected events seen. Check with your CXL card vendor for implementation specific behavior.

```
# tracer: nop
#
# entries-in-buffer/entries-written: 1/1 #P:144
#
#          -----> irqs-off/BH-disabled
#          /-----> need-resched
#          | /-----> hardirq/softirq
#          || /-----> preempt-depth
#          ||| /-----> migrate-disable
#          |||| /-----> delay
#
#          TASK-PID    CPU#  | ||||  TIMESTAMP  FUNCTION
#          | |         |   |   | ||||  |         |
#      kworker/0:1-6636    [000]  ....  8026.964797: cxl_memory_module:
memdev=mem0 host=0000:64:00.0 serial=2458271785234918685 log=Warning :
time=1724798017557370956 uuid=fe927475-dd59-4339-a586-79bab113b774 len=128
flags='0x1' handle=10 related_handle=0 maint_op_class=0 :
event_type='Temperature Change' health_status='I' media_status='Normal'
as_life_used=Normal as_dev_temp=Warning as_cor_vol_err_cnt=Normal
as_cor_per_err_cnt=Normal life_used=255 device_temp=36 dirty_shutdown_cnt=0
cor_vol_err_cnt=0 cor_per_err_cnt=0
```

3. Linux dmesg output

NOTE

dmesg output depends on several factors like OS distro and kernel. Information is used for debugging purposes only.

```
[ 8011.541212] cxl_query_cmd: cxl_mem mem0: Query IOCTL
[ 8011.541219] cxl_query_cmd: cxl_mem mem0: Query IOCTL
[ 8011.541222] cxl_send_cmd: cxl_mem mem0: Send IOCTL
[ 8011.541224] handle_mailbox_cmd_from_user: cxl_pci 0000:64:00.0: Submitting
Get Health Info command for user
opcode: 4200
size: 0
[ 8011.541229] __cxl_pci_mbox_send_cmd: cxl_pci 0000:64:00.0: Sending
command: 0x4200
[ 8011.541262] cxl_pci_mbox_wait_for_doorbell: cxl_pci 0000:64:00.0: Doorbell
wait took 0ms
[ 8016.578982] cxl_query_cmd: cxl_mem mem0: Query IOCTL
[ 8016.578989] cxl_query_cmd: cxl_mem mem0: Query IOCTL
[ 8016.578992] cxl_send_cmd: cxl_mem mem0: Send IOCTL
[ 8016.578994] handle_mailbox_cmd_from_user: cxl_pci 0000:64:00.0: Submitting
Get Alert Configuration command for user
opcode: 4201
size: 0
[ 8016.578999] __cxl_pci_mbox_send_cmd: cxl_pci 0000:64:00.0: Sending
command: 0x4201
[ 8016.579027] cxl_pci_mbox_wait_for_doorbell: cxl_pci 0000:64:00.0: Doorbell
wait took 0ms
```

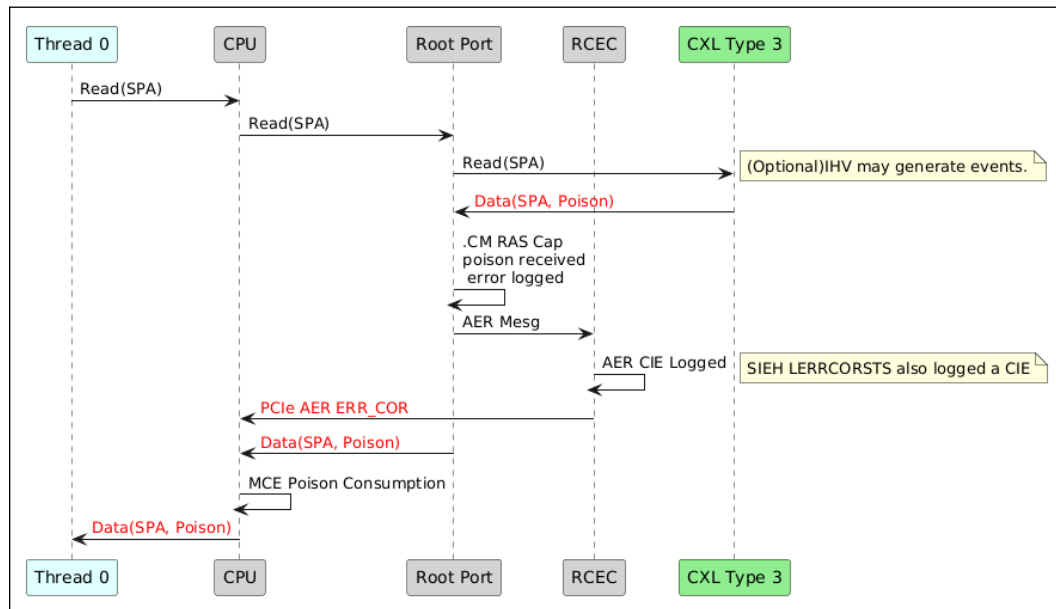
```
[ 8024.874862] cxl_query_cmd: cxl_mem mem0: Query IOCTL
[ 8024.874869] cxl_query_cmd: cxl_mem mem0: Query IOCTL
[ 8024.874872] cxl_send_cmd: cxl_mem mem0: Send IOCTL
[ 8024.874874] handle_mailbox_cmd_from_user: cxl_pci 0000:64:00.0: Submitting
Set Alert Configuration command for user
opcode: 4202
size: c
[ 8024.874880] __cxl_pci_mbox_send_cmd: cxl_pci 0000:64:00.0: Sending
command: 0x4202
[ 8024.874951] cxl_pci_mbox_wait_for_doorbell: cxl_pci 0000:64:00.0: Doorbell
wait took 0ms
[ 8024.874965] cxl_query_cmd: cxl_mem mem0: Query IOCTL
[ 8024.874967] cxl_query_cmd: cxl_mem mem0: Query IOCTL
[ 8024.874970] cxl_send_cmd: cxl_mem mem0: Send IOCTL
[ 8024.874971] handle_mailbox_cmd_from_user: cxl_pci 0000:64:00.0: Submitting
Get Alert Configuration command for user
opcode: 4201
size: 0
[ 8024.874974] __cxl_pci_mbox_send_cmd: cxl_pci 0000:64:00.0: Sending
command: 0x4201
[ 8024.875007] cxl_pci_mbox_wait_for_doorbell: cxl_pci 0000:64:00.0: Doorbell
wait took 0ms
[ 8025.841184] {27}[Hardware Error]: Hardware error from APEI Generic
Hardware Error Source: 0
[ 8025.851329] {27}[Hardware Error]: It has been corrected by h/w and
requires no further action
[ 8025.861248] {27}[Hardware Error]: event severity: corrected
[ 8025.867701] {27}[Hardware Error]: Error 0, type: corrected
[ 8025.874150] {27}[Hardware Error]: section_type: CXL Memory Module Event
[ 8025.885555] cxl_handle_cper_event: CPER event 3 for device 0:100:0.0
```

11.4 Poison Consumption

Test Objective

Do a poison consumption from a CXL address.

Figure 22. Poison Consumption



Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B0
BIOS	BHSDCRB1.IPC.0034.D82.2408060239
OS Version	CentOS Stream 9
Kernel	6.10.6
HW Configuration	1 Socket
CScripts Version	BHS.2422.6001
Mode of operation	Firmware First Mode

Precondition and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- ["CScripts on page 18"](#)
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability on page 27](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration. [CXL Firmware First vs OS First](#) on how to set the knobs:

- Test is valid for either in OS First Mode or Firmware First Mode.

Recipe

1. Cause a poison consumption that is coming from a CXL EP.

See the [CXL Finding Address](#) chapter on how to create a victim/target app that uses CXL memory.

Next inject a poison into CXL memory, see previous chapters, such as [mem data parity](#) and [poison](#) injection recipes to create a poison in CXL device.

Finally, read from the poisoned address using a variety of methods.

- **Using iotools via system-ram mode:** The simplest method is to install [iotools](#). (Recommended)

```
# ./iotools mem-read32 <poisoned address>
Bus error (core dumped)
```

You may need to do a write back and invalidate [WBINVD] command before trying the read in order to avoid any non-poisoned cache. There are 2 ways of doing this.

a. **CScripts**

```
>>> itp.threads.wbinvd()
```

b. **OS Kernel Module**

WBINVD is a privileged instruction and must be done in a kernel context, like a kernel module. Providing such a program is out of scope for this document, but there are plenty of google search examples or use a regenerative AI tool with the following prompt "Write a Linux kernel module that would initiate a wbinvd using sysdebug to trigger."

- **mca-recover as your victim app:** Another way to consume the error is Inject a poison the address specified by the app. Before injecting, convert the HPA returned by the app to DPA using `cxl.addTran()` in Cscripts. Next press <enter> and the app will do a read to the poison address.

NOTE

At some point in the future, this method will unlikely work anymore if you inject a poison using the method in the [Poison Injection](#) chapters. This is because the page the victim app uses will get retired due to the CXL DRAM Event record. So when you finally press <enter> it would have accessed a clean page.

```
# numactl --membind=2 ./mca-recover
vtop(7f663e20a000) = 3d580091000 <== Inject poison to this address.
Use /sys/kernel/debug/apei/einj/... to inject
Then press <ENTER> to access:
```

- **Using Python via devdax mode:** See the Creating a Target App (dev-dax edition) in [CXL Finding Address](#) for instructions.

2. Observe the FTrace output in the OS.

NOTE

See [CXL Firmware First vs OS First](#) for any known issues with OS First Mode or Firmware First Mode.

Pass / Fail	Criteria
PASS	Check for poison consumption MCE Look for 1 mce_record ftrace event: • Check: "bank" == 0x1 : Bank 1 for GNR • Check: "addr" & 0xfffffffffff00 == 0x6000012200 : Poisoned Address • Check: "status" & 0xffff == 0x134 : MCACOD 0x134
PASS	Check for mem failure event Look for 1 memory_failure_event ftrace event:
OS FAIL	Check for received poison CXL Protocol Error on RP Look for 1, but found 0 cxl_aer_correctable_error ftrace event: Due to the above mentioned OS bug, Event type is shown as a Non-standard event in firmware first mode. • Check: "host" == "0000:89:02.0" : RP of poison address location • Check: "status" & 0x20 == 0x20 : Mem Received Poison
PASS	Check for Internal Correctable Look for 1 aer_event ftrace event:
continued...	

Pass / Fail	Criteria
	- Check: "dev_name" == "0000:89:00.4" : RCEC - Check: "severity" & 0xff == 0x2 : Correctable - Check: "status" & 0xffffffff == 0x4000 : Internal Error
PASS	(Optional and depends on IHV implementation, these events are not necessarily expected per CXL Spec) Check for cxl_dram ftrace event: - Check: "host" == "0000:8a:00.0" : EP of poison address location - Check: "hpa" & 0xffffffffffff00 == 0x6000012200 : Injected DPA

```
# tracer: nop
#
# entries-in-buffer/entries-written: 8/8   #P:144
#
#          -----> irqs-off/BH-disabled
#          /-----> need-resched
#          | /-----> hardirq/softirq
#          || /-----> preempt-depth
#          ||| /-----> migrate-disable
#          |||| /-----> delay
#          TASK-PID   CPU#   TIMESTAMP   FUNCTION
#          | |         | |         | |         |
kworker/103:1-1036   [103]   .....   9321.349336: mce_record: CPU: 103,
MCGC/s: f001c20/f, MC1: bd80000000100134, IPID: 0000000000000000, ADDR:
0000006000012200, MISC: 0000000000000086, SYND: 0000000000000000, RIP:
33:0000000000402fff>, TSC: 1832c49700bd, PPIN: 3fe6284b9d076114, vendor: 0,
CPUTID: a06d1, time: 1725404405, socket: 0, APIC: 8f, microcode: 110002f0
iotools-4300   [002]   .....   9321.349928: memory_failure_event: pfn
0x6000012: recovery action for free buddy page: Recovered
kworker/0:1-10   [000]   d..l.   9321.673610: non_standard_event:
severity: 1; sec type:b4efb980-b552-e34d-a777-68784b771048; FRU:
00000000-0000-0000-0000-000000000000 ; data len:260; raw data:67 00 00 00 00
00 00 00 05 00 00 00 00 00 00 00 00 00 02 89 00 00 00 00 00 98 1e b0 0d 7f 1e 00
00 04 06 58 00 00 00 00 00 00 00 00 00 00 00 00 10 a0 42 01 22 80 00 00 27
51 00 00 85 48 7a 00 40 00 85 70 80 25 58 00 c0 03 48 01 1f 00 01 00 00 00 00
00 f7 1f b3 10 49 14 00 00 3e 00 80 01 05 00 1e 00 01 00 00 00 00 00 00 00 38
00 58 00 00 00 00 00 23 00 01 00 98 1e 41 01 07 00 27 00 be 00 26 00 17 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 30 00 00 ff cf 01 00 20 00 00
00 00 00 00 00 00 20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
kworker/0:1-10   [000]   .....   9321.801735: cxl_dram: memdev=mem0
host=0000:8a:00.0 serial=245826362736942365 log=Warning :
time=1725404404723127018 uuid=601dcbb3-9c06-4eab-b8af-4e9bfb5c9624 len=128
flags='0x1' handle=10 related_handle=0 maint_op_class=0 : dpa=6000
dpa_flags='VOLATILE' descriptor='UNCORRECTABLE_EVENT' type='ECC Error'
transaction_type='Host Read' channel=0 rank=0 nibble_mask=0 bank_group=0
bank=0 row=0 column=768 cor_mask=00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 validity_flags='CHANNEL|
RANK|BANK GROUP|BANK|ROW|COLUMN' hpa=6000012200 region=region0
region_uuid=00000000-0000-0000-0000-000000000000
kworker/0:1-10   [000]   .....   9321.801749: cxl_dram: memdev=mem0
host=0000:8a:00.0 serial=245826362736942365 log=Informational :
time=2121750342 uuid=601dcbb3-9c06-4eab-b8af-4e9bfb5c9624 len=0 flags=''
handle=0 related_handle=0 maint_op_class=1 : dpa=0 dpa_flags='VOLATILE|
NOT REPAIRABLE' descriptor='UNCORRECTABLE_EVENT|POISON_LIST_OVERFLOW|0xd0'
type='0x17' transaction_type='0xaa' channel=0 rank=0 nibble_mask=100
bank_group=0 bank=0 row=0 column=256 cor_mask=00 00 00 00 00 00 00 00 00 00
00 00 00 00 46 57 77 7e 00 00 00 00 00 00 00 00 00 00 00 00 00 d5
validity_flags='CHANNEL|RANK|NIBBLE|BANK GROUP|BANK|ROW|COLUMN'
hpa=6000000200 region=region0 region_uuid=00000000-0000-0000-0000-000000000000
kworker/0:1-10   [000]   .....   9321.801767: aer_event: 0000:89:00.4
PCIe Bus Error: severity=Corrected, Corrected Internal Error, TLP Header=Not
available
kworker/0:1-10   [000]   .....   9321.802145: cxl_dram: memdev=mem0
```

```
host=0000:8a:00.0 serial=2458263362736942365 log=Informational :
time=2121750342 uuid=601dcbb3-9c06-4eab-b8af-4e9bfb5c9624 len=0 flags=''
handle=0 related_handle=0 maint_op_class=0 : dpa=0 dpa_flags='VOLATILE|
NOT_REPAIRABLE' descriptor='UNCORRECTABLE_EVENT|POISON_LIST_OVERFLOW|0xd0'
type='0x17' transaction_type='0xaa' channel=0 rank=0 nibble_mask=100
bank_group=0 bank=0 row=0 column=256 cor_mask=00 00 00 00 00 00 00 00 00
00 00 00 00 00 46 57 77 7e 00 00 00 00 00 00 00 00 00 00 00 00 00 d5
validity_flags='CHANNEL|RANK|NIBBLE|BANK GROUP|BANK|ROW|COLUMN'
hpa=60000000200 region=region0 region_uuid=00000000-0000-0000-0000-000000000000
```

3. Linux dmesg output

NOTE

dmesg output depends on several factors like OS distro and kernel. Information is used for debugging purposes only.

```
[ 9320.308802] Disabling lock debugging due to kernel taint
[ 9320.309292] {3}[Hardware Error]: Hardware error from APEI Generic Hardware
Error Source: 0
[ 9320.316053] mce: [Hardware Error]: Machine check events logged
[ 9320.316625] mce: Uncorrected hardware memory error in user-access at
6000012200
[ 9320.316649] Memory failure: 0x6000012: recovery action for free buddy
page: Recovered
[ 9320.325344] {3}[Hardware Error]: event severity: recoverable
[ 9320.325347] {3}[Hardware Error]: Error 0, type: corrected
[ 9320.325347] {3}[Hardware Error]: section_type: CXL DRAM Event
[ 9320.325348] {3}[Hardware Error]: Error 1, type: recoverable
[ 9320.325349] {3}[Hardware Error]: section_type: CXL DRAM Event
[ 9320.325349] {3}[Hardware Error]: Error 2, type: corrected
[ 9320.389745] {3}[Hardware Error]: section_type: CXL Protocol Error
[ 9320.396966] {3}[Hardware Error]: agent_type: 5, CXL Root Port
[ 9320.403905] {3}[Hardware Error]: agent_address: 0000:89:02.0
[ 9320.410736] {3}[Hardware Error]: slot: 11
[ 9320.415699] {3}[Hardware Error]: vendor_id: 0x1e98, device_id: 0x0db0
[ 9320.423392] {3}[Hardware Error]: sub_vendor_id: 0x1e7f, sub_device_id:
0x0000
[ 9320.431976] {3}[Hardware Error]: class_code: 0604
[ 9320.437721] {3}[Hardware Error]: DVSEC length: 0x0038
[ 9320.443849] {3}[Hardware Error]: CXL DVSEC:
[ 9320.448995] {3}[Hardware Error]: 00000000: 00010023 01411e98 00270007
002600be
[ 9320.457562] {3}[Hardware Error]: 00000010: 00000017 00000000 00000000
00000000
[ 9320.466142] {3}[Hardware Error]: 00000020: 00000000 00000000 00000000
00000000
[ 9320.474718] {3}[Hardware Error]: 00000030: 00000000 00000000
Error log length: 0x0058
[ 9320.481447] {3}[Hardware Error]: CXL Error Log:
[ 9320.487967] {3}[Hardware Error]: cxl_ras_uncor_status: 0x00000000
[ 9320.493520] {3}[Hardware Error]: cxl_ras_uncor_mask: 0x00003000
[ 9320.507924] {3}[Hardware Error]: cxl_ras_uncor_severity: 0x0001cfff
[ 9320.515411] {3}[Hardware Error]: cxl_ras_cor_status: 0x00000020
[ 9320.515412] {3}[Hardware Error]: cxl_ras_cor_mask: 0x00000000
[ 9320.529408] {3}[Hardware Error]: cap_control: 0x00002000
[ 9320.535813] {3}[Hardware Error]: Header Log Registers:
[ 9320.542025] {3}[Hardware Error]: 00000000: 00000000 00000000 00000000
00000000
[ 9320.550580] {3}[Hardware Error]: 00000010: 00000000 00000000 00000000
00000000
[ 9320.559148] {3}[Hardware Error]: 00000020: 00000000 00000000 00000000
00000000
[ 9320.567727] {3}[Hardware Error]: 00000030: 00000000 00000000 00000000
00000000
[ 9320.576317] {3}[Hardware Error]: Error 3, type: corrected
[ 9320.582649] {3}[Hardware Error]: section_type: PCIe error
[ 9320.589074] {3}[Hardware Error]: port_type: 10, root complex event
```

```

collector
[ 9320.597473] {3}[Hardware Error]: version: 3.0
[ 9320.602737] {3}[Hardware Error]: command: 0x0504, status: 0x0010
[ 9320.609855] {3}[Hardware Error]: device_id: 0000:89:00.4
[ 9320.616185] {3}[Hardware Error]: slot: 0
[ 9320.620953] {3}[Hardware Error]: secondary_bus: 0x00
[ 9320.626889] {3}[Hardware Error]: vendor_id: 0x8086, device_id: 0x0b23
[ 9320.634478] {3}[Hardware Error]: class_code: 080700
[ 9320.768375] cxl_handle_cper_event: CPER event 2 for device 0:138:0.0
[ 9320.768436] __cxl_dpa_to_region: cxl decoder4.0: dpa:0x6000 mapped in
region:region0
[ 9320.768452] cxl_handle_cper_event: CPER event 2 for device 0:138:0.0
[ 9320.768460] __cxl_dpa_to_region: cxl decoder4.0: dpa:0x0 mapped in
region:region0
[ 9320.768472] pcieport 0000:89:00.4: AER: aer_status: 0x00004000, aer_mask:
0x00000000
[ 9320.768475] pcieport 0000:89:00.4: [14] CorrIntErr
[ 9320.768477] pcieport 0000:89:00.4: AER: aer_layer=Transaction Layer,
aer_agent=Receiver ID
[ 9320.768786] {4}[Hardware Error]: Hardware error from APEI Generic Hardware
Error Source: 0
[ 9320.768788] {4}[Hardware Error]: event severity: recoverable
[ 9320.768789] {4}[Hardware Error]: Error 0, type: recoverable
[ 9320.768791] {4}[Hardware Error]: section_type: CXL DRAM Event
[ 9320.768846] cxl_handle_cper_event: CPER event 2 for device 0:138:0.0
[ 9320.768857] __cxl_dpa_to_region: cxl decoder4.0: dpa:0x0 mapped in
region:region0
[ 9320.836612] Memory failure: 0x6000012: already hardware poisoned

```

11.5 Intel Flat Memory Mode

11.5.1 CXL Flat Memory Mode 101

Intel Flat Memory Mode is a specialized mode where the DDR Near Memory (NM) and CXL Far Memory (FM) serve as an exclusive caching of each other in a 1:1 ratio. Unlike inclusive caching, data only exist in either NM or FM. Hot data is moved to NM for faster performance. Both the DDR and CXL memory are exposed to the system as a combined physical memory.

This section deep dives into the setup of Intel Flat Memory Mode, some exclusive caching fundamentals and some error flows.

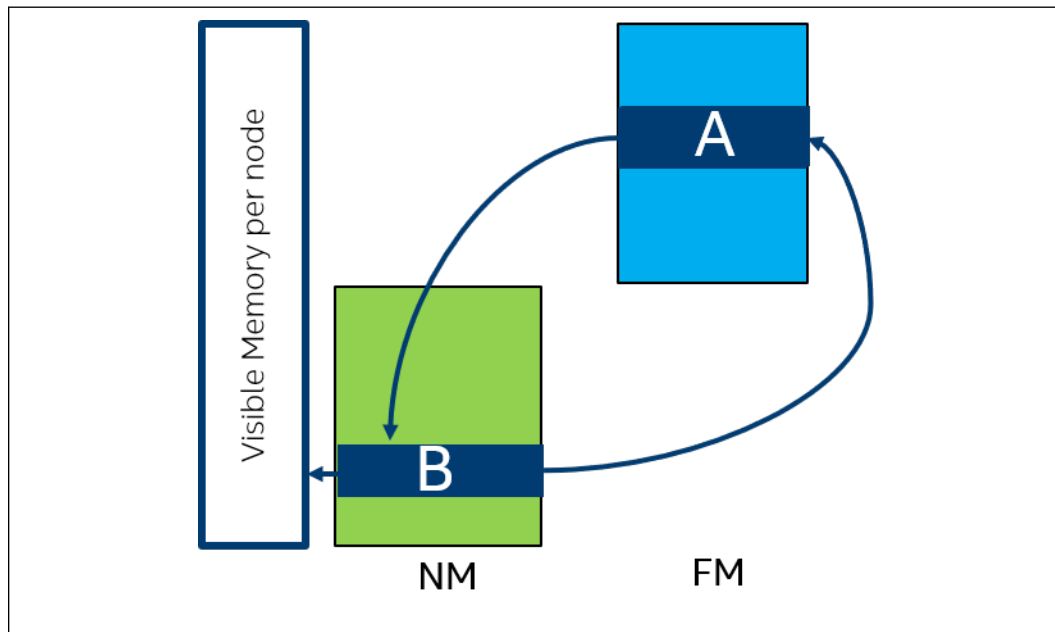
11.5.1.1 Key Terminology

Terminology	Definition
Alias Physical Address	In 2LM mode, a TARGET PA and its ALIAS PA will share the same NM cache location.
Cache	A hardware component that stores data so that future requests for that data can be served faster.
Cache Hit	When a search in a cache for an address is found.
Cache Miss	When a search in a cache for an address is not found.
Cache Line	The unit of memory that is copied to and is individually tracked in a cache. Specifically, 64 bytes of data or instructions aligned on a 64-byte physical address boundary.
<i>continued...</i>	

Terminology	Definition
Cache Line Read	A read to a cache where data size and address aligns with the cache.
Near Memory [NM]	In 2LM mode, near memory refers to the DDR5 cache.
Far Memory [FM]	In 2LM mode, far memory refers to the volatile Intel® Optane™ PMem.
Partial Cache Line Write	A write to a cache where data size is less than a cache line.
Physical Address (PA) or System Physical Address (SPA)	Hardware addresses of physical memory.
System RAM	Main memory.
Target Physical Address	Address that the test is trying to read or write in the 2LM flows that will trigger errors.
Cache Line Write	A write to a cache where data size and address aligns with the cache line.
Dirty/Modified Cache Line	If a cache line has been modified and needs to be flushed back to lower memory, it is called "dirty".

11.5.1.2 CXL Intel Flat Memory Mode Caching Fundamentals

Intel Flat Memory Mode, is a mode that mixes DRAM and CXL memory into an hierarchical fashion. DRAM, or Near Memory (NM), and CXL Memory, or Far Memory (FM), are arranged as exclusive caching setup. Unlike inclusive caching, exclusive caching has no data replication. Data either exists in NM or FM. Hot data is swapped into NM for performance.



Intel Flat Memory Mode utilizes 1b of metadata, protected by ECC, to represent 2 states. In this document, they are referred to as State 0 and State 1. State 0 is considered the "home" state, where Address A naturally maps to DDR memory space,

and Address B naturally maps to a CXL memory space. State B occurs when the addresses have been swapped. Data in hot address B has been copied to DDR, and data in address A has been copied to CXL. In state 1, when Address B is accessed, the memory controller will read the data from DDR memory.

For example, let's assume DDR NM is 24 GB and CXL FM is also 24 GB, maintaining a 1:1 ratio. The total Intel Flat Memory Mode address space is from 0 - 48 GB. In State 0, or home state, address A = 1 GB in DDR, and its corresponding alias, address B = 25 GB in CXL. In State 1, a read request to address of 25 GB is sent to B2CMI. B2CMI then reads the DDR location and determines the data is in State B because of the 1b metadata. For more information see *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644.

11.5.2 Setup

Refer to CXL Supplemental Validation Guide (Doc ID: 814339) and/or CXL Memory Expander (Doc ID: 803836), for information on how to setup the system in 1LM + Intel® Flat Memory Mode. In addition to CScripts system setup as described in Standard System Configuration Chapter, it is also highly suggested to turn directory mode off. If the directory mode is turned on, it is hard to control cache states.

- EDKII -> Socket Configuration -> Uncore Configuration -> Uncore General Configuration -> Directory Mode = Disable

Check your configuration often to ensure that your platform did indeed enter Intel Flat Memory Mode, as many situations can bring the system out of POR configuration.

NOTE

Not all SKUs support Intel Flat Memory Mode. In cscripts check the following:

```
>>> sv.sockets.compute0.uncore.punit.ptpcioregs.ptpcioregs.capid1_cfg.disable_2lm
socket0.compute0.uncore.punit.ptpcioregs.ptpcioregs.capid1_cfg.disable_2lm -
0x00000000
socket1.compute0.uncore.punit.ptpcioregs.ptpcioregs.capid1_cfg.disable_2lm -
0x00000000
```

A quick way to check if the system has entered Intel Flat Memory Mode via cscripts is to read the b2cmi.mode.flat2lm bit.

```
sv.socket0.soc.memss.b2cmi0.mode.show
0x00000001 : flat2lm (15:15) -- \n          Flat 2LM Mode: \n When Near Memory
Caching (NMCaching) is enabled, settin...
0x00000001 : mirrorfusesshadow (14:14) -- \n          This captures the
MirrorFuse value. that the pcode distributes...
0x00000000 : nmcmode (13:13) -- \n          Near Memory Controller mode:\n
When set, this indicates that the M2...
0x00000000 : fmcmode (12:12) -- \n          Far Memory Controller mode:\n
When set, this indicates that the M2M...
0x00000000 : hbm (11:11) -- \n          HBM:\n          When set, this indicates that
the primary memory is HBM. If in a...
0x00000001 : addrmirrorstrap (10:10) -- \n          This captures the AddrMirro
Strap value. Its only readable by B...
0x00000000 : mirror (09:09) -- \n          Mirroring:\n          Enable
mirroring.\n          MmCapabilities[CapMirrorEn] ...
0x00000000 : nmcaching (08:08) -- \n          Near Memory Caching
(NMCaching):\n          Enable Near Memory caching, i....
```

```
0x00000000 : pmem (06:06) -- \n          Persistent Memory (PMem):\n          Enable
persistent memory flows. Furthermor...
0x00000000 : blockregion (05:05) -- \n          Block Region
(BlockRegion):\n          Enable block region memory flows....
0x00000000 : isoch (04:04) -- \n          (deprecated for GNR)          Isochronous Flows
(Isoch):\n          Enable Isochronous f...
```

In Linux use `setpci` to dump out the `b2cmi.mode` register.

```
# setpci -d 0x8086:0x324a 0x318.L
0xC400
```

An example Linux output below, with 1LM + Intel Flat Memory Mode successfully enabled, with SNC set to "0".

- The NUMA nodes 3,4 and 5 in the SRAT table represent the address regions in Flat Memory Mode, each of size "0x400000000".
 - The First half of the address range maps to DDR memory, and second half maps to CXL memory space.
 - Size of DDR range = Size of CXL range = 0x2000000000, aka they are in a 1:1 ratio. This value is used as the "Offset" to calculate the Alias Address in [Finding Address](#).
- "`cxl list -Ru`" returns the CXL regions, region 0 from "0x4080000000", region13 from "0xa080000000", and region14 from "0x10080000000". This is helpful in identifying the start of CXL address in Flat Memory Mode.
- "`cxl list -DRu -r regionx`" returns further decode of the above regions.
- In NUMA node 3: "0x2080000000 - 407FFFFFFF" belongs to DDR memory, and "0x4080000000 to 0x607FFFFFFF " is CXL memory.
- In NUMA node 4: "0x8080000000 - a07FFFFFFF" belongs to DDR memory, and "0xa080000000 to 0xc07FFFFFFF " is CXL memory.
- In NUMA node 3: "0xe080000000 - 1007FFFFFFF" belongs to DDR memory, and "0x10080000000 to 0x1207FFFFFFF " is CXL memory.

NOTE

The other Nodes in the SRAT table correspond to 1LM DDR regions in the current 1LM + Flat Memory Mode.

```
# dmesg | grep ACPI | grep SRAT
[ 0.008767] ACPI: SRAT 0x0000000077781000 00A208 (v03 INTEL INTEL ID 00000002
INTL 20230628)
[ 0.008801] ACPI: Reserving SRAT table memory at [mem 0x77781000-0x7778b207]
[ 0.009016] ACPI: SRAT: Node 0 PXM 0 [mem 0x00000000-0x7fffffff]
[ 0.009018] ACPI: SRAT: Node 0 PXM 0 [mem 0x100000000-0x207fffffff]
[ 0.009019] ACPI: SRAT: Node 1 PXM 1 [mem 0x6080000000-0x807fffffff]
[ 0.009020] ACPI: SRAT: Node 2 PXM 2 [mem 0xc080000000-0xe07fffffff]
[ 0.009021] ACPI: SRAT: Node 3 PXM 3 [mem 0x2080000000-0x607fffffff]
[ 0.009021] ACPI: SRAT: Node 4 PXM 4 [mem 0x8080000000-0xc07fffffff]
[ 0.009022] ACPI: SRAT: Node 5 PXM 5 [mem 0xe080000000-0x1207fffffff]

# cxl list -Ru
[
{
  "region": "region0",
```

```

    "resource": "0x4080000000",
    "size": "128.00 GiB (137.44 GB)",
    "type": "ram",
    "interleave_ways": 1,
    "interleave_granularity": 256,
    "decode_state": "commit",
    "qos_class_mismatch": true
  },
  {
    "region": "region13",
    "resource": "0xa080000000",
    "size": "128.00 GiB (137.44 GB)",
    "type": "ram",
    "interleave_ways": 1,
    "interleave_granularity": 256,
    "decode_state": "commit",
    "qos_class_mismatch": true
  },
  {
    "region": "region14",
    "resource": "0x10080000000",
    "size": "128.00 GiB (137.44 GB)",
    "type": "ram",
    "interleave_ways": 1,
    "interleave_granularity": 256,
    "decode_state": "commit",
    "qos_class_mismatch": true
  }
]

# cxi list -DRu -r region0

[
  {
    "root decoders": [
      {
        "decoder": "decoder0.0",
        "resource": "0x2080000000",
        "size": "256.00 GiB (274.88 GB)",
        "interleave_ways": 1,
        "max_available_extent": "128.00 GiB (137.44 GB)",
        "volatile_capable": true,
        "qos_class": 0,
        "nr_targets": 1,
        "regions": {
          "decoder0.0": [
            {
              "region": "region0",
              "resource": "0x4080000000",
              "size": "128.00 GiB (137.44 GB)",
              "type": "ram",
              "interleave_ways": 1,
              "interleave_granularity": 256,
              "decode_state": "commit",
              "qos_class_mismatch": true
            }
          ]
        }
      }
    ]
  },
  {
    "port decoders": [
      {
        "decoder": "decoder2.0",
        "resource": "0x4080000000",
        "size": "128.00 GiB (137.44 GB)",
        "interleave_ways": 1,
        "region": "region0",
        "nr_targets": 1
      }
    ]
  }
]

```

```

    },
    {
      "endpoint decoders": [
        {
          "decoder": "decoder9.0",
          "resource": "0x4080000000",
          "size": "128.00 GiB (137.44 GB)",
          "interleave_ways": 1,
          "region": "region0",
          "dpa_resource": "0",
          "dpa_size": "128.00 GiB (137.44 GB)",
          "mode": "ram"
        }
      ]
    }
  ]
}

# cxl list -DRu -r region13

[
  {
    "root decoders": [
      {
        "decoder": "decoder0.13",
        "resource": "0x8080000000",
        "size": "256.00 GiB (274.88 GB)",
        "interleave_ways": 1,
        "max_available_extent": "128.00 GiB (137.44 GB)",
        "volatile_capable": true,
        "qos_class": 0,
        "nr_targets": 1,
        "regions:decoder0.13": [
          {
            "region": "region13",
            "resource": "0xa080000000",
            "size": "128.00 GiB (137.44 GB)",
            "type": "ram",
            "interleave_ways": 1,
            "interleave_granularity": 256,
            "decode_state": "commit",
            "qos_class_mismatch": true
          }
        ]
      }
    ]
  },
  {
    "port decoders": [
      {
        "decoder": "decoder3.0",
        "resource": "0xa080000000",
        "size": "128.00 GiB (137.44 GB)",
        "interleave_ways": 1,
        "region": "region13",
        "nr_targets": 1
      }
    ]
  },
  {
    "endpoint decoders": [
      {
        "decoder": "decoder7.0",
        "resource": "0xa080000000",
        "size": "128.00 GiB (137.44 GB)",
        "interleave_ways": 1,
        "region": "region13",
        "dpa_resource": "0",
        "dpa_size": "128.00 GiB (137.44 GB)",
        "mode": "ram"
      }
    ]
  }
]

```



```

    ]
  }
]

# cxl list -DRu -r region14

[
{
  "root decoders":[
    {
      "decoder":"decoder0.14",
      "resource":"0xe08000000",
      "size":"256.00 GiB (274.88 GB)",
      "interleave_ways":1,
      "max_available_extent":"128.00 GiB (137.44 GB)",
      "volatile_capable":true,
      "qos_class":0,
      "nr_targets":1,
      "regions:decoder0.14":[
        {
          "region":"region14",
          "resource":"0x10080000000",
          "size":"128.00 GiB (137.44 GB)",
          "type":"ram",
          "interleave_ways":1,
          "interleave_granularity":256,
          "decode_state":"commit",
          "qos_class_mismatch":true
        }
      ]
    }
  ]
},
{
  "port decoders":[
    {
      "decoder":"decoder4.0",
      "resource":"0x10080000000",
      "size":"128.00 GiB (137.44 GB)",
      "interleave_ways":1,
      "region":"region14",
      "nr_targets":1
    }
  ]
},
{
  "endpoint decoders":[
    {
      "decoder":"decoder8.0",
      "resource":"0x10080000000",
      "size":"128.00 GiB (137.44 GB)",
      "interleave_ways":1,
      "region":"region14",
      "dpa_resource":"0",
      "dpa_size":"128.00 GiB (137.44 GB)",
      "mode":"ram"
    }
  ]
}
]

```

11.5.3 Prelude

This section dives into the necessary steps which are required before injecting the errors. Below topics walks through on finding a good address, and how to setup the cache states for a hit/miss scenarios in NM and FM.

11.5.3.1 Finding Address

It is recommended to Refer [Flat Memory Mode Caching Fundamentals](#) to understand how Target and Alias Addresses are chosen.

The first step is to choose a system RAM address for testing. Keeping consistency with the previous chapters, an address from an OS point of view will be chosen. A few recommended addresses are:

- Anything but a kernel address
- Memory allocated to a test application (Recommended)

It is recommended to use the below mentioned method of choosing a specific application to return an address that is not in use by another application or BIOS operations. In 1lm + Flat Memory Mode, use numactl to bind the application to memory regions in Flat Memory mode. Below mca-recover application from ras-tools is used to return a Physical address from the Flat Memory Mode address regions. See [Setup](#) to find more information on DDR and CXL address mapping in Flat Memory Mode.

In the below example,

- mca-recover allocates an address from NUMA node 3 (Flat Memory Mode region)
 - This would be an address between 0x2080000000-0x607FFFFFFF
 - mca-recover returns the address, 0X607fe07000, which is within the targeted numa node region. Furthermore this belongs to the CXL memory space.

```
# numactl --membind=3 ./mca-recover
vtop(7ffff7ffa000) = 607fe07000
Use /sys/kernel/debug/apei/einj/... to inject
Then press <ENTER> to access:
```

Target Address (TA) = 0x607fe07000

For every recipe, after you choose a target address (TA), aka the Address A , you will also need to find the Alias addresses (AA), aka address B. Depending on the Target Address returned (if mca-recover returns an address in DDR memory or CXL memory), The alias address is calculated differently.

Address	Method
Target Address (TA)	DDR memory
Alias Address (AA)	TA + Offset

OR

Address	Method
Target Address (TA)	CXL memory
Alias Address (AA)	TA - Offset

NOTE

Offset = Size of DDR Memory range = Size of CXL memory range within that NUMA node.

Given that DDR and CXL memory is in 1:1 ratio, the Alias address is going to be an alias of Target address, which can be obtained by adding/subtracting the offset to the TA.

Example:

- Based on the address returned above, the TA is in CXL memory. So the Alias Address chosen will be in the DDR memory space.
- Offset = 0x2000000000. (This corresponds to the size of DDR and CXL address ranges in NUMA node 3)

NOTE

Make sure the Alias Address calculated falls within the CXL address range.

Address	Method
Target Address (TA)	0x607fe07000
Alias Address (AA)	0x607fe07000 - 0x2000000000

Alias Address (AA) = 0x407fe07000

11.5.3.2 Staging a Cache for a Hit/Miss

Refer to [Flat Memory Mode Caching Fundamentals](#) and [Finding Address](#) for more information on how the data is swapped between NM and FM during a hit/miss and on how to choose an address.

Address Being Accessed	Scenario	Expected Cache States	Description
N/A	Initial	NM = TA FM = AA	These are the initial natural states of TA and AA at boot-up. The data at these addresses are at "home".
TA	Hit	NM = TA FM = AA	For a Hit to occur, It is expected for the TA to be located in NM and AA to be in FM. When an access request for TA is sent to NM, since it is present in NM, it is returned and it causes a hit. There is no swap.
TA	Miss	NM = AA FM = TA	For a Miss to occur, It is expected for the AA to be located in NM and TA to be in FM. When an access request for TA is sent to NM, it causes a miss, and the data is swapped with that in FM.
AA	Hit	NM = AA FM = TA	For a Hit to occur, It is expected for the AA to be located in NM and TA to be in FM. When an access request for AA is sent to NM, since it is present in NM, it is returned and it causes a hit. There is no swap.
AA	Miss	NM = TA FM = AA	For a Miss to occur, It is expected for the TA to be located in NM and AA to be in FM. When an access request for AA is sent to NM, it causes a miss because it should be fetched from FM, and the data is swapped with that in FM.

Example Hit/Miss Cache states staging:

The Target_Address (TA) native to NM is being accessed. Alias_Address (AA) is located in CXL memory.

Hit scenario

```
Target_Address = 0x407fe07000 # retrieved from mca-recover app
Alias_Address = 0x607fe07000

# Below commands would setup the system to cause a "hit" for when an error is
# injected into the TA at NM.

itp.threads[0].mem(Alias_Address, 4) # This would bring Alias address (AA) to NM.
itp.threads[0].mem(Target_Address, 4) # This would bring target address (TA) to
NM, and swap the AA to FM.
itp.threads[0].wbinvd() # Flush and save the modified cachelines.

# Current cache states is NM = TA and FM = AA. So cache states are primed to
# cause a hit.
```

Miss scenario

```
Target_Address = 0x407fe07000 # retrieved from mca-recover app
Alias_Address = 0x607fe07000

# Below commands would setup the system to cause a "Miss" for when when an error
# is injected into the TA at NM.

itp.threads[0].mem(Target_Address, 4) # This would bring target address (TA) to
NM.
itp.threads[0].mem(Alias_Address, 4) # This would bring Alias address (AA) to
NM, and swap the TA to FM.
itp.threads[0].wbinvd() # Flush and save the modified cachelines.

# Current cache states is NM = AA and FM = TA. So cache states are primed to
# cause a miss.
```

11.5.4 Cache Line Reads Recipes

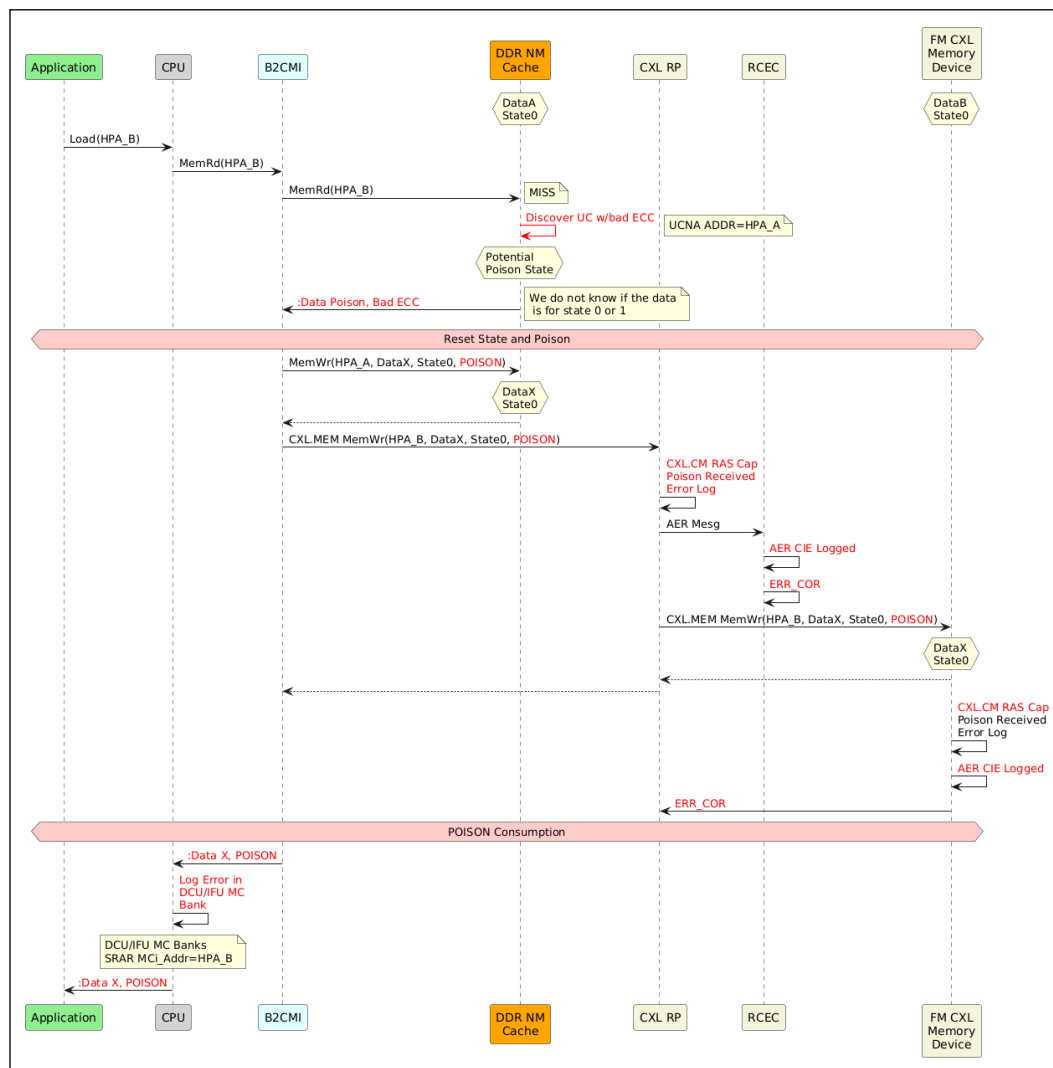
11.5.4.1 Read NM Miss NM UCE

Test Objective

In this Flat Memory Mode configuration, A read to the Near Memory Cache should cause a miss, along with a NM UECC error. During the eviction process, the controller forces poison in the FM write (eviction) as well as in the NM fill write, and both NM and FM are poisoned. This is done to prevent silent data corruption. See the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for more information about CXL error reporting and flows.

The below figure summarizes the error flow of the recipe in this chapter:

Figure 23. Read NM Miss NM UCE



Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B0
BIOS	BHSDCRB1.IPC.3544.P36.2502102219
OS Version	CentOS Stream 9
Kernel	6.10.6
<i>continued...</i>	

HW Configuration	1 Socket
CScripts Version	BHS.2515.6001
Mode of Operation	Firmware First Mode

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- ["CScripts on page 18"](#)
- [FTrace and DMSG for Machine Check Exception \(MCE\) Observability on page 27](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration in [Setup](#). Refer to [CXL Firmware First vs OS First](#) on how to set the knobs:

- Test is valid for either OS First Mode or Firmware First Mode.

Recipe

1. Retrieve Target Address and Alias Address using the steps listed in [Finding Address](#).

```
#numactl --membind=4 ./mca-recover
vtop(7ffff7ffa000) = c07ff91000
Use /sys/kernel/debug/apei/einj/... to inject
Then press <ENTER> to access:
```

2. The below pseudo script uses the Target and Alias Addresses from step 1, with Target Address at FM and Alias Address at NM to cause a miss when the Alias address (also the Error Address) is accessed. Alias Address is staged to cause a miss, as described in [Staging a Cache for hit/miss](#) section.

- **Pseudo Script**

```
halt()

target_address = 0xc07ff91000
alias_address = target_address - 0x2000000000
error_address= alias_address

# Write recognizable data
itp.threads[0].mem("%sp" % target_address, 4, 0xcafecafe)
itp.threads[0].mem("%sp" % alias_address, 4, 0xbabebabe)
itp.threads[0].wbinvd()

# Read addresses just to see what the data is stored
# Setup the system to cause a "miss" when mca-recover <enter>
itp.threads[0].mem("%sp" % target_address, 4) # Will bring the target address into the NM.
itp.threads[0].mem("%sp" % alias_address, 4) # Will bring the Alias address into the NM.
itp.threads[0].wbinvd()

# Arm a UCE error to DDR memory with immediate inject and immediate consume set to false, and
confirm an injector is ARMED
ei.mem.injectMemError(error_address, immInject=False, immConsume=False, errType="uce")

# INJECT error to DDR memory by writing into the error_address.
# Confirm the previously injector is no longer ARMED to confirm that error is injected.
itp.threads[0].mem("%sp" % error_address, 4, 0xdeadbeef)
itp.threads[0].wbinvd()
ei.mem.showMemInjectors()
```

```

go()

# Now the caches are staged such that Data A and Data B are at states 0 (aka home), a UC ECC
error is injected at the NM. A read access request which is sent with the mca-recover
application will be for the FM address data (DataB). The B2CMI controller will first access NM
and since DataA is located at NM, this will cause a miss, and the error at NM will be
consumed.
# click enter in mca-recover to consume the error.

```

• CScripts Output

```

>>> halt()

>>> target_address = 0xc07ff91000
>>> alias_address = target_address - 0x2000000000
>>> error_address= alias_address

# Write recognizable data
>>> itp.threads[0].mem("%sp" % target_address, 4, 0xcafecafe)
>>> itp.threads[0].mem("%sp" % alias_address, 4, 0xbabebabe)
>>> itp.threads[0].wbinvd()

# Read addresses just to see what the data is stored
# Setup the system to cause a "miss" when mca-recover <enter>
>>> itp.threads[0].mem("%sp" % target_address, 4) # Will bring the target address into the NM.
>>> itp.threads[0].mem("%sp" % alias_address, 4) # Will bring the Alias address into the NM.
>>> itp.threads[0].wbinvd()

# ARM error to DDR memory and confirm an injector is ARMED
>>> ei.mem.injectMemError(error_address, immInject=False, immConsume=False, errType="uce")

*****
Core Addr_____ = 0xc0_7ff9_1000 (770.00 G)
Cluster_____ = 0x1
Set Index_____ = 0x440
Tag_____ = 0x50_3ffc
Cha_____ = 53
Perspective_____ = FM
*****
FM ddr5 2LM:
Core Addr_____ = 0xc0_7ff9_1000 (770.00 G)
Socket_____ = 0
Hbm_____ = -1
CXL Mc_____ = 32
Ch_____ = 0
Sub-Ch_____ = -1
*****
NM ddr5 2LM:
Core Addr_____ = 0xa0_7ff9_1000 (642.00 G)
Ch Addr_____ = 0x7_ffff_4400 (32.00 G)
Sub_Ch Addr_____ = 0x3_ffff_2200 (16.00 G)
pseudo_Ch Addr_____ = 0x1_ffff_9100 (8.00 G)
Rank Addr_____ = 0xffff_c880 (4.00 G)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 7
Ch_____ = 0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 5
Bank in Bank Grp = 1
Bank Addr_____ = 21
Row_____ = 0x7d7f
Column_____ = 0x7e0

```

```

*****
*****
Core Addr_____ =      0xc0_7ff9_1000 (770.00 G)
Cluster_____ = 0x1
Set Index_____ = 0x440
Tag_____ = 0x50_3ffc
Cha_____ = 53
Perspective_____ = FM
*****
FM ddr5 2LM:
Core Addr_____ =      0xc0_7ff9_1000 (770.00 G)
Socket_____ = 0
Hbm_____ = -1
CXL Mc_____ = 32
Ch_____ = 0
Sub-Ch_____ = -1
*****
NM ddr5 2LM:
Core Addr_____ =      0xa0_7ff9_1000 (642.00 G)
Ch Addr_____ =      0x7_ffff_4400 (32.00 G)
Sub_Ch Addr_____ =      0x3_ffff_2200 (16.00 G)
pseudo_Ch Addr =      0x1_ffff_9100 (8.00 G)
Rank Addr_____ =      0xffff_c880 (4.00 G)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 7
Ch_____ = 0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 5
Bank in Bank Grp = 1
Bank Addr_____ = 21
Row_____ = 0x7d7f
Column_____ = 0x7e0
*****
System in Non-Mirror mode, and error_type: uce
ECC Injection in Non-Mirror Mode, MC-7, Channel-0, Mirror Scrub Enabled is True
socket0 Mc0 Ch0 Patrol Scrub Disabled
.
socket0 Mc11 Ch0 Patrol Scrub Disabled
Current : dev0 = 1, dev0msk = 0x4, dev1 = 5, dev1msk = 0x1.
adddc_flag=None
adddc_errinj_reg=0x0x0
Error Injection Dfx ARMED
Socket0 MC 7 Channel 0
Injector 0
=====
|               |      Device0      |      Device1      |
|-----|-----|
| Device Status | Currently ARMED   | Currently ARMED   |
|-----|-----|
| Mask (injector) | 00000000 00000000 | 00000000 00000000 |
|-----|-----|
| Match (injector) | 000a0000 ffffc880 | 000a0000 ffffc880 |
|-----|-----|
| memDev        |      0x1         |      0x5         |
|-----|-----|
| XOR Mask      |      0x4         |      0x1         |
|-----|-----|
=====

Restoring Original Values!!!
socket0 Mc0 Ch0 Patrol Scrub Enabled
.
socket0 Mc11 Ch0 Patrol Scrub Enabled

```



```
# INJECT error to DDR memory and confirm injector is no longer ARMED
>>> itp.threads[0].mem("%sp" % error_address, 4, 0xdeadbeef)
>>> itp.threads[0].wbinvd()
>>> ei.mem.showMemInjectors()

Socket0 MC mc0 Channel 0
Injector 0
=====
|           | Device0 | Device1 |
|-----|-----|-----|
| Device Status | Currently unarmed | Currently unarmed |
|-----|-----|-----|
| Mask (injector) | 0001ffff ffffffff | 0001ffff ffffffff |
|-----|-----|-----|
| Match (injector) | 00080000 00000000 | 00080000 00000000 |
|-----|-----|-----|
| memDev | 0x0 | 0x0 |
|-----|-----|-----|
| XOR Mask | 0x0 | 0x0 |
|-----|-----|-----|
.
.
.
Socket0 MC mc7 Channel 0
Injector 0
=====
|           | Device0 | Device1 |
|-----|-----|-----|
| Device Status | Currently unarmed | Currently unarmed |
|-----|-----|-----|
| Mask (injector) | 00000000 00000000 | 00000000 00000000 |
|-----|-----|-----|
| Match (injector) | 00080000 ffffc880 | 00080000 ffffc880 |
|-----|-----|-----|
| memDev | 0x1 | 0x5 |
|-----|-----|-----|
| XOR Mask | 0x4 | 0x1 |
|-----|-----|-----|
.
.
.
>>> go()
[GPC core group] Resuming on 144 devices

# Now the caches are staged such that Data A and Data B are at states 0 (aka home), a UC ECC
error is injected at the NM. A read access request which is sent with the mca-recover
application will be for the FM address data (DataB). The B2CMI controller will first access NM
and since DataA is located at NM, this will cause a miss, and the error at NM will be consumed.
# click enter in mca-recover to consume the error.
```

3. Consume the error by hitting "enter" in the mca-recover app.

```
[root@gnr-bkc ras-tools]# numactl --membind=4 ./mca-recover
vtop(7ffff7ffa000) = c07ff96000
Use /sys/kernel/debug/apci/einj/... to inject
Then press <ENTER> to access:
recover: sig=7 si=0x7ffffffffffd4b0 v=0x7ffffffffffd380
Platform memory error at 0x7ffff7ffa000
addr = 0x7ffff7ffa000 lsb=12
Recovery allocated new page at physical 0xc07ff96000
Successful recovery
```

4. Observe the FTrace output in the OS.

NOTE

See [CXL Firmware First vs OS First](#) for any known issues with OS First Mode or Firmware First Mode.

```
# cat /sys/kernel/debug/tracing/trace
# tracer: nop
#
# entries-in-buffer/entries-written: 4/4   #P:72
#
#          -----> irqs-off/BH-disabled
#          /-----> need-resched
#          | /-----> hardirq/softirq
#          || /-----> preempt-depth
#          ||| /-----> migrate-disable
#          |||| /-----> delay
#
#          TASK-PID      CPU#  | TIMESTAMP  FUNCTION
#          | |          | |      | |          | |
# kworker/75:1-1045    [075]  ....   154.162760: mce_record: CPU: 75, MCGc/s: f001c20/f,
MC1: bd80000000100134, IPID: 0000000000000000, ADDR: 000000c07ff91000, MISC: 0000000000000086,
SYND: 0000000000000000, RIP: 33:<0000000000401348>, TSC: cd2c855825, PPIN: 3fe626ceac513143,
vendor: 0, CPUID: a06d1, time: 1752691547, socket: 0, APIC: 7, microcode: 10003a2
mca-recover-2947    [001]  ....   154.166509: memory_failure_event: pfn 0xc07ff91:
recovery action for dirty LRU page: Recovered
# kworker/0:0-8      [000]  d..l.   154.275406: non_standard_event: severity: 1; sec
type:b4efb980-b552-e34d-a777-68784b771048; FRU: 00000000-0000-0000-0000-000000000000 ; data
len:260; raw data:67 00 00 00 00 00 00 00 05 00 00 00 00 00 00 06 63 00 00 00 00 00 98
1e b2 0d 6f 1e 00 00 04 06 48 00 00 00 00 00 00 00 00 00 00 00 00 00 10 a0 42 01 22 80 00 00
27 51 00 00 85 48 7a 02 40 00 85 70 80 25 48 00 c0 03 48 01 1f 00 01 00 00 00 00 00 f7 1f b3
10 49 14 00 00 3e 00 80 01 05 00 1e 00 01 00 00 00 00 00 00 00 00 00 38 00 58 00 00 00 23 00
01 00 98 1e 41 01 07 00 27 00 be 00 26 00 17 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 30 00 00
ff cf 01 00 20 00 00 00 00 00 00 00 00 20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
# kworker/0:0-8      [000]  d..l.   154.275409: non_standard_event: severity: 1; sec
type:b4efb980-b552-e34d-a777-68784b771048; FRU: 00000000-0000-0000-0000-000000000000 ; data
len:260; raw data:67 00 00 00 00 00 00 00 02 00 00 00 00 00 00 00 00 64 00 00 00 00 00 98
1e e2 01 00 00 00 00 02 05 00 00 00 00 00 00 1d ed 4a 04 cc 95 1d 22 10 b0 02 00 c1 8f 2c 11
37 50 09 00 05 61 43 00 40 00 85 10 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 10 00 b1
00 00 00 00 00 3e 00 80 01 05 00 1e 01 00 00 00 00 00 00 00 00 38 00 58 00 00 00 00 23 00
c1 40 98 1e 81 03 00 00 9e e8 06 40 00 00 00 00 00 01 00 00 00 20 00 00 00 0b 81 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 48 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
ff cf 01 00 20 00 00 00 00 00 00 00 00 20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
# kworker/0:0-8      [000]  .N...   154.287129: aer_event: 0000:63:00.4 PCIe Bus Error:
severity=Corrected, Corrected Internal Error, TLP Header=Not available
# kworker/0:0-8      [000]  .N...   154.300562: aer_event: 0000:64:00.0 PCIe Bus Error:
severity=Corrected, Advisory Non-Fatal Error|Corrected Internal Error, TLP Header=Not available
# kworker/75:1-1045    [075]  .N...   154.836641: mce_record: CPU: 0, MCGc/s: f001c20/0,
MC20: ac00000200a00090, IPID: 0000000000000000, ADDR: 000000a07ff91000, MISC:
080002a3ebfbf086, SYND: 0000000000000000, RIP: 00:<0000000000000000>, TSC: cd2c85af63, PPIN:
3fe626ceac513143, vendor: 0, CPUID: a06d1, time: 1752691548, socket: 0, APIC: 0, microcode:
10003a2
# kworker/75:1-1045    [075]  .N...   154.843335: memory_failure_event: pfn 0xa07ff91:
recovery action for free buddy page: Recovered
```

Pass/Fail	Criteria
PASS	Check for an SRAR error on DCU bank • Check : "bank" == 0x1 : Bank 1 for GNR • Check: "ADDR" == Address being consumed/read (in this case target_address)

continued...

Pass / Fail	Criteria
	Check: MC1_STATUS == bd80000000100134
PASS	Check for a memory failure event at the target_Address
PASS	Check for an uncorrectable error for demand read on IMC banks - Check : "bank" == 20 : any IMC bank - Check: "ADDR" == NM address (in this case alias_address) - Check: MCi_STATUS == ac00000200a00090
PASS	Check for a memory failure event at alias_Address
OS FAIL	Check for CXL Protocol Error Mem Poison Received on the RP. Look for 1, but found 0 cxl_aer_correctable_error ftrace event: Due to the above mentioned OS bug, Event type is shown as a Non-standard event. Can be verified in Dmesg. - Check: "host" == " 0000:63:06.0" (RP) (can vary) - Severity= 1 (Optional) Check: "status" & 0x20 : Mem Received Poison
OS FAIL	Check for CXL Protocol Error Mem Poison Received on the EP. Look for 1, but found 0 cxl_aer_correctable_error ftrace event: Due to the above mentioned OS bug, Event type is shown as a Non-standard event. Can be verified in Dmesg - Check: "host" == "0000:64:00.0" (EP) (can vary) - Severity= 1 (Optional) Check: "status" & 0x20 : Mem Received Poison
PASS	Check for PCIe Correctable Internal Error on the associated RCEC: - Event type: aer_event - BDF info (can vary): 0000:63:00.4 (RCEC) - Error type : PCIe Bus Error - Severity=Corrected, Corrected Internal Error
PASS	Check for CXL.io Correctable Internal Error on EP: - Event type: aer_event - BDF info (can vary): 0000:64:00.0 (EP) - Error type : PCIe Bus Error - Severity= Advisory Non-Fatal Error Corrected Internal Error
PASS	<i>Note:</i> Additional event(s) can be seen with a CXL device. Check with your CXL card vendor for implementation specific behavior.

5. Linux dmesg output

NOTE

The dmesg output depends on several factors like the OS distro and kernel. Information is used for debugging purposes only.

```
[ 153.034301] {1}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 0
[ 153.040398] mce: [Hardware Error]: Machine check events logged
[ 153.043809] mce: Uncorrected hardware memory error in user-access at c07ff91000
[ 153.044135] Memory failure: 0xc07ff91: Sending SIGBUS to mca-recover:2947 due to hardware
memory corruption
[ 153.044148] Memory failure: 0xc07ff91: recovery action for dirty LRU page: Recovered
[ 153.049715] {1}[Hardware Error]: It has been corrected by h/w and requires no further action
[ 153.049716] {1}[Hardware Error]: event severity: corrected
[ 153.049717] {1}[Hardware Error]: Error 0, type: corrected
[ 153.049718] {1}[Hardware Error]: section_type: CXL Protocol Error
[ 153.049718] {1}[Hardware Error]: agent_type: 5, CXL Root Port
[ 153.049719] {1}[Hardware Error]: agent_address: 0000:63:06.0
[ 153.049721] {1}[Hardware Error]: slot: 9
[ 153.049721] {1}[Hardware Error]: vendor_id: 0x1e98, device_id: 0x0db2
[ 153.049722] {1}[Hardware Error]: sub_vendor_id: 0x1e6f, sub_device_id: 0x0000
```

```
[ 153.056343] {1}Hardware error detected on CPU75
[ 153.064624] {1}[Hardware Error]:   class_code: 0604
[ 153.064626] {1}[Hardware Error]:   DVSEC_length: 0x0038
[ 153.064627] {1}[Hardware Error]:   CXL DVSEC:
[ 153.064629] {1}[Hardware Error]:   00000000: 00010023 01411e98 00270007 002600be
[ 153.064630] {1}[Hardware Error]:   00000010: 00000017 00000000 00000000 00000000
[ 153.064631] {1}[Hardware Error]:   00000020: 00000000 00000000 00000000 00000000
[ 153.064631] {1}[Hardware Error]:   00000030: 00000000 00000000
[ 153.064632] {1}[Hardware Error]:   Error log length: 0x0058
[ 153.075703] {1}event severity: recoverable
[ 153.084411] {1}[Hardware Error]:   CXL Error Log:
[ 153.084412] {1}[Hardware Error]:   cxl_ras_uncor_status: 0x00000000
[ 153.084413] {1}[Hardware Error]:   cxl_ras_uncor_mask: 0x00003000
[ 153.084414] {1}[Hardware Error]:   cxl_ras_uncor_severity: 0x0001cfff
[ 153.084414] {1}[Hardware Error]:   cxl_ras_cor_status: 0x00000020
[ 153.084415] {1}[Hardware Error]:   cxl_ras_cor_mask: 0x00000000
[ 153.084415] {1}[Hardware Error]:   cap_control: 0x00002000
[ 153.084415] {1}[Hardware Error]:   Header Log Registers:
[ 153.084416] {1}[Hardware Error]:   00000000: 00000000 00000000 00000000 00000000
[ 153.084417] {1}[Hardware Error]:   00000010: 00000000 00000000 00000000 00000000
[ 153.084417] {1}[Hardware Error]:   00000020: 00000000 00000000 00000000 00000000
[ 153.084418] {1}[Hardware Error]:   00000030: 00000000 00000000 00000000 00000000
[ 153.084418] {1}[Hardware Error]:   Error 1, type: corrected
[ 153.084419] {1}[Hardware Error]:   section_type: PCIe error
[ 153.084419] {1}[Hardware Error]:   port_type: 10, root complex event collector
[ 153.093977] {1}Error 0, type: recoverable
[ 153.100209] {1}[Hardware Error]:   version: 3.0
[ 153.100211] {1}[Hardware Error]:   command: 0x0504, status: 0x0010
[ 153.100212] {1}[Hardware Error]:   device_id: 0000:63:00.4
[ 153.100213] {1}[Hardware Error]:   slot: 0
[ 153.100213] {1}[Hardware Error]:   secondary_bus: 0x00
[ 153.100213] {1}[Hardware Error]:   vendor_id: 0x8086, device_id: 0x0b23
[ 153.100214] {1}[Hardware Error]:   class_code: 080700
[ 153.100214] {1}[Hardware Error]:   Error 2, type: corrected
[ 153.100215] {1}[Hardware Error]:   section_type: CXL Protocol Error
[ 153.100215] {1}[Hardware Error]:   agent_type: 2, CXL Device
[ 153.100216] {1}[Hardware Error]:   agent_address: 0000:64:00.0
[ 153.100216] {1}[Hardware Error]:   slot: 0
[ 153.100217] {1}[Hardware Error]:   vendor_id: 0x1e98, device_id: 0x01e2
[ 153.100217] {1}[Hardware Error]:   sub_vendor_id: 0x0000, sub_device_id: 0x0000
[ 153.100218] {1}[Hardware Error]:   class_code: 0502
[ 153.100218] {1}[Hardware Error]:   DVSEC_length: 0x0038
[ 153.100219] {1}[Hardware Error]:   CXL DVSEC:
[ 153.106458] {1}section_type: general processor error
[ 153.113568] {1}[Hardware Error]:   00000000: 40c10023 03811e98 e89e0000 00004006
[ 153.113570] {1}[Hardware Error]:   00000010: 00000000 00000001 00000020 0000810b
[ 153.113571] {1}[Hardware Error]:   00000020: 00000000 00000000 00000000 00000048
[ 153.113572] {1}[Hardware Error]:   00000030: 00000000 00000000
[ 153.113572] {1}[Hardware Error]:   Error log length: 0x0058
[ 153.113573] {1}[Hardware Error]:   CXL Error Log:
[ 153.113573] {1}[Hardware Error]:   cxl_ras_uncor_status: 0x00000000
[ 153.120403] {1}processor_type: 0, IA32/X64
[ 153.127113] {1}[Hardware Error]:   cxl_ras_uncor_mask: 0x00000000
[ 153.127115] {1}[Hardware Error]:   cxl_ras_uncor_severity: 0x0001cfff
[ 153.127116] {1}[Hardware Error]:   cxl_ras_cor_status: 0x00000020
[ 153.127117] {1}[Hardware Error]:   cxl_ras_cor_mask: 0x00000000
[ 153.127117] {1}[Hardware Error]:   cap_control: 0x00002000
[ 153.127117] {1}[Hardware Error]:   Header Log Registers:
[ 153.127118] {1}[Hardware Error]:   00000000: 00000000 00000000 00000000 00000000
[ 153.127119] {1}[Hardware Error]:   00000010: 00000000 00000000 00000000 00000000
[ 153.127120] {1}[Hardware Error]:   00000020: 00000000 00000000 00000000 00000000
[ 153.131893] {1}processor_isa: 2, X64
[ 153.139491] {1}[Hardware Error]:   00000030: 00000000 00000000 00000000 00000000
[ 153.139493] {1}[Hardware Error]:   Error 3, type: corrected
[ 153.139494] {1}[Hardware Error]:   section_type: PCIe error
[ 153.139495] {1}[Hardware Error]:   port_type: 0, PCIe end point
[ 153.139495] {1}[Hardware Error]:   version: 3.0
[ 153.139496] {1}[Hardware Error]:   command: 0x0546, status: 0x0010
[ 153.147880] {1}error_type: 0x08
[ 153.153037] {1}[Hardware Error]:   device_id: 0000:64:00.0
[ 153.153038] {1}[Hardware Error]:   slot: 0
```

```
[ 153.153039] {1}[Hardware Error]: secondary_bus: 0x00
[ 153.153039] {1}[Hardware Error]: vendor_id: 0x1d1fa, device_id: 0x01e2
[ 153.153040] {1}[Hardware Error]: class_code: 050210
[ 153.153167] pcieport 0000:63:00.4: AER: aer_status: 0x00004000, aer_mask: 0x00000000
[ 153.158717] {1} micro-architectural error
[ 153.164751] pcieport 0000:63:00.4: [14] CorrIntErr
[ 153.164754] pcieport 0000:63:00.4: AER: aer_layer=Transaction Layer, aer_agent=Receiver ID
[ 153.164774] cxl_pci 0000:64:00.0: AER: aer_status: 0x00006000, aer_mask: 0x00000000
[ 153.169814] {1} operation: 0, unknown or generic
[ 153.178193] cxl_pci 0000:64:00.0: [13] NonFatalErr
[ 153.178195] cxl_pci 0000:64:00.0: [14] CorrIntErr
[ 153.178196] cxl_pci 0000:64:00.0: AER: aer_layer=Transaction Layer, aer_agent=Receiver ID
[ 153.703548] {1} version_info: 0x000000000000a06d1
[ 153.708907] {1} processor_id: 0x0000000000000007
[ 153.714279] mce: [Hardware Error]: Machine check events logged
[ 153.720973] Memory failure: 0xa07ff91: recovery action for free buddy page: Recovered
[ 153.730932] {2}Hardware error detected on CPU0
[ 153.736589] {2}event severity: recoverable
[ 153.741263] {2} Error 0, type: recoverable
[ 153.745940] {2} fru_text: Card01, ChnH, DIMM0
[ 153.750949] {2} section_type: memory error
[ 153.755703] {2} error_status: Storage error in DRAM memory (0x00000000000000400)
[ 153.764185] {2} physical_address: 0x000000a07ff91000
[ 153.769929] {2} node:0 card:7 module:0 rank:0 bank:21 device:0 row:32127 column:2016
[ 153.778896] {2} error_type: 4, single-symbol chipkill ECC
[ 153.785139] {2} DIMM location: BANK 7 CPU0_DIMM_H1
```

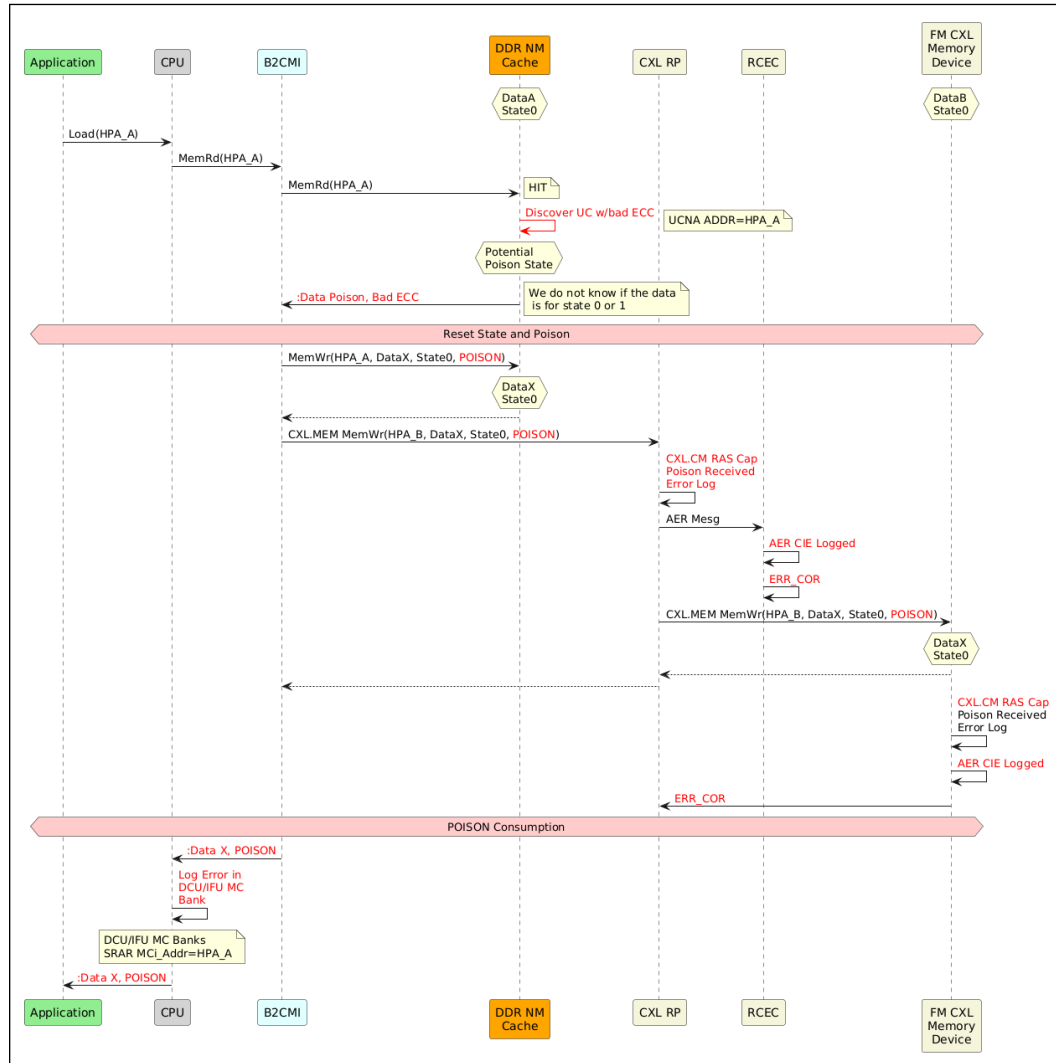
11.5.4.2 Read NM Hit NM UCE

Test Objective

In this Flat Memory Mode configuration, A read to the Near Memory Cache should cause a hit, along with a DDR UECC error. During the eviction process, the controller forces poison in the FM write (eviction) as well as in the NM fill write, and both NM and FM are poisoned. This is done to prevent silent data corruption. See the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for more information about CXL error reporting and flows.

The following figure summarizes the error flow of the recipe in this chapter:

Figure 24. Read NM Hit NM UCE



Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores B0
BIOS	BHSDCRB1.IPC.3544.P36.2502102219
OS Version	CentOS Stream 9
Kernel	6.10.6
<i>continued...</i>	

HW Configuration	1 Socket
CScripts Version	BHS.2515.6001
Mode of Operation	Firmware First Mode

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- "CScripts on page 18"
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration in [Setup](#). Refer to [CXl Firmware First vs OS First](#) on how to set the knobs:

- Test is valid for either OS First Mode or Firmware First Mode

Recipe

1. Retrieve Target Address and Alias Address using the steps listed in [Finding Address](#).

```
# numactl --membind=5 ./mca-recover
vtop(7ffff7ffa000) = 0xff7e88d000
Use /sys/kernel/debug/apei/einj/... to inject
Then press <ENTER> to access:
```

2. The below pseudo script uses the Target and Alias Addresses from step 1, with Target Address at FM and Alias Address at NM to cause a hit when the Alias address (also the Error Address) is accessed. Alias Address is staged to cause a Hit, as described in [Staging a Cache for hit/miss](#) section.

Pseudo Script

```
halt()

alias_address = 0xff7e88b000
target_address = alias_address + 0x2000000000
error_address= alias_address

# Write recognizable data
itp.threads[0].mem("%sp" % target_address, 4, 0xcafecafe)
itp.threads[0].mem("%sp" % alias_address, 4, 0xbabebabe)
itp.threads[0].wbinvd()

# Read addresses just to see what the data is stored
# Setup the system to cause a "miss" when mca-recover <enter>
itp.threads[0].mem("%sp" % target_address, 4) # Will bring the target address into the NM.
itp.threads[0].mem("%sp" % alias_address, 4) # Will bring the Alias address into the NM.
itp.threads[0].wbinvd()

# Arm a UCE error to DDR memory with immediate inject and immediate consume set to false, and
confirm an injector is ARMED
ei.mem.injectMemError(error_address, immInject=False, immConsume=False, errType="uce")

# INJECT error to DDR memory by writing into the error address.
# Confirm the previously injector is no longer ARMED to confirm that error is injected.
itp.threads[0].mem("%sp" % error_address, 4, 0xdeadbeef)
itp.threads[0].wbinvd()
ei.mem.showMemInjectors()
```

```
go()

# Now the caches are staged such that Data A and Data B are at states 0(aka home), a UC ECC
error is injected at the NM. A read access request which is sent with the mca-recover
application will be for the NM address data (DataA). The B2CMI controller will first access NM
and since DataA is located at NM, this will cause a hit, and the error at NM will be
discovered/consumed.
# click enter in mca-recover to consume the error.
```

• CScrips Output

```
>>> halt()

[GPC core group] Halt on 144 devices

>>> alias_address = 0xff7e88d000
>>> target_address = alias_address + 0x2000000000
>>> error_address= alias_address

# Write recognizable data
>>> itp.threads[0].mem("%sp" % target_address, 4, 0xcafecafe)
>>> itp.threads[0].mem("%sp" % alias_address, 4, 0xbabebabe)
>>> itp.threads[0].wbinvd()

# Read addresses just to see what the data is stored
# Setup the system to cause a "hit" when mca-recover <enter>
>>> itp.threads[0].mem("%sp" % target_address, 4) # Will bring the Alias address into the NM.
>>> itp.threads[0].mem("%sp" % alias_address, 4) # Will bring the Alias address into the NM.
>>> itp.threads[0].wbinvd()

# Arm a UCE error to DDR memory with immediate inject and immediate consume set to false, and
confirm an injector is ARMED
>>> ei.mem.injectMemError(error_address, immInject=False, immConsume=False, errType="uce")

*****
Core Addr_____ = 0x11f_7e88_d000 (1.12 T)
Cluster_____ = 0x2
Set Index_____ = 0x340
Tag_____ = 0x7f_bf44
Cha_____ = 98
Perspective_____ = FM
*****
FM ddr5 2LM:
Core Addr_____ = 0x11f_7e88_d000 (1.12 T)
Socket_____ = 0
Hbm_____ = -1
CXL Mc_____ = 32
Ch_____ = 0
Sub-Ch_____ = -1
*****
NM ddr5 2LM:
Core Addr_____ = 0xff_7e88_d000 (1021.98 G)
Ch Addr_____ = 0x7_bfa2_3400 (30.99 G)
Sub_Ch Addr_____ = 0x3_dfd1_1a00 (15.50 G)
pseudo_Ch Addr_____ = 0x1_efe8_8d00 (7.75 G)
Rank Addr_____ = 0xf7f4_4680 (3.87 G)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 5
Ch_____ = 0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 5
Bank in Bank Grp = 1
```



```

Bank Addr_____ = 21
Row_____ = 0x70fa
Column_____ = 0x3e0
*****
*****
Core Addr_____ = 0x11f_7e88_d000 (1.12 T)
Cluster_____ = 0x2
Set Index_____ = 0x340
Tag_____ = 0x7f_bf44
Cha_____ = 98
Perspective_____ = FM
*****
FM ddr5 2LM:
Core Addr_____ = 0x11f_7e88_d000 (1.12 T)
Socket_____ = 0
Hbm_____ = -1
CXL Mc_____ = 32
Ch_____ = 0
Sub-Ch_____ = -1
*****
NM ddr5 2LM:
Core Addr_____ = 0xff_7e88_d000 (1021.98 G)
Ch Addr_____ = 0x7_bfa2_3400 (30.99 G)
Sub_Ch Addr_____ = 0x3_dfd1_1a00 (15.50 G)
pseudo_Ch Addr_____ = 0x1_efe8_8d00 (7.75 G)
Rank Addr_____ = 0xf7f4_4680 (3.87 G)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 5
Ch_____ = 0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 5
Bank in Bank Grp = 1
Bank Addr_____ = 21
Row_____ = 0x70fa
Column_____ = 0x3e0
*****
System in Non-Mirror mode, and error_type: uce
ECC Injection in Non-Mirror Mode, MC=5, Channel=0, Mirror Scrub Enabled is True
socket0 Mc0 Ch0 Patrol Scrub Disabled
.
.
socket0 Mc11 Ch0 Patrol Scrub Disabled

Current : dev0 = 1, dev0msk = 0x4, dev1 = 5, dev1msk = 0x1.
adddc_flag=None
adddc_errinj_reg=0x0x0
Error Injection DFx ARMED
Socket0 MC 5 Channel 0
Injector 0
=====
| | Device0 | Device1 |
|-----|-----|
| Device Status | Currently ARMED | Currently ARMED |
|-----|-----|
| Mask (injector) | 00000000 00000000 | 00000000 00000000 |
|-----|-----|
| Match (injector) | 000a0000 f7f44680 | 000a0000 f7f44680 |
|-----|-----|
| memDev | 0x1 | 0x5 |
|-----|-----|
| XOR Mask | 0x4 | 0x1 |
|-----|-----|
=====
Restoring Original Values!!!

```

```
socket0 Mc0 Ch0 Patrol Scrub Enabled
.
.
socket0 Mc11 Ch0 Patrol Scrub Enabled

# INJECT error to DDR memory and confirm injector is no longer ARMED
>>> itp.threads[0].mem("%sp" % error_address, 4, 0xdeadbeef)
>>> itp.threads[0].wbinvd()
>>> ei.mem.showMemInjectors()

Socket0 MC mc0 Channel 0
Injector 0
=====
|           |           Device0           |           Device1           |
|-----|-----|-----|
| Device Status | Currently unarmed | Currently unarmed |
|-----|-----|-----|
| Mask (injector) | 0001ffff ffffffff | 0001ffff ffffffff |
|-----|-----|-----|
| Match (injector) | 00080000 00000000 | 00080000 00000000 |
|-----|-----|-----|
| memDev         |           0x0         |           0x0         |
|-----|-----|-----|
| XOR Mask       |           0x0         |           0x0         |
|-----|-----|-----|
.
.
Socket0 MC mc5 Channel 0
Injector 0
=====
|           |           Device0           |           Device1           |
|-----|-----|-----|
| Device Status | Currently unarmed | Currently unarmed |
|-----|-----|-----|
| Mask (injector) | 00000000 00000000 | 00000000 00000000 |
|-----|-----|-----|
| Match (injector) | 00080000 f7f44680 | 00080000 f7f44680 |
|-----|-----|-----|
| memDev         |           0x1         |           0x5         |
|-----|-----|-----|
| XOR Mask       |           0x4         |           0x1         |
|-----|-----|-----|
.
.

>>> go()
[GPC core group] Resuming on 144 devices

# Now the caches are staged such that Data A and Data B are at states 0(aka home), a UC ECC
error is injected at the NM. A read access request which is sent with the mca-recover
application will be for the NM address data (DataA). The B2CMI controller will first access NM
and since DataA is located at NM, this will cause a hit, and the error at NM will be
discovered/consumed.
# click enter in mca-recover to consume the error.
```

3. Consume the error by reading the Alias address at NM.

```
# numactl --membind=5 ./mca-recover
vtop(7ffff7ffa000) = 0xff7e88d000
Use /sys/kernel/debug/apei/einj/... to inject
Then press <ENTER> to access:
recover: sig=7 si=0x7fffffd4b0 v=0x7fffffd380
Platform memory error at 0x7ffff7ffa000
addr = 0x7ffff7ffa000 lsb=12
Recovery allocated new page at physical 0x11f7e823000
Successful recovery
```

4. Observe the FTrace output in the OS.

NOTE

See [CXL Firmware First vs OS First](#) for any known issues with OS First Mode or Firmware First Mode.

```
# cat /sys/kernel/debug/tracing/trace
# tracer: nop
#
# entries-in-buffer/entries-written: 4/4   #P:72
#
#
#          -----=> irqs-off/BH-disabled
#          /-----=> need-resched
#          | /-----=> hardirq/softirq
#          || /-----=> preempt-depth
#          ||| /-----=> migrate-disable
#          |||| /-----=> delay
#
#          TASK-PID      CPU#      TIMESTAMP      FUNCTION
#          | |          |          |          |
# kworker/0:1-10      [000] ..... 3445.894497: mce_record: CPU: 0, MCGc/s: f001c20/0,
MC18: ac00000200a00090, IPID: 0000000000000000, ADDR: 000000ff7e88d000, MISC:
080002a387d1f086, SYND: 0000000000000000, RIP: 00:<0000000000000000>, TSC: 9a0e19ffe41, PPIN:
3fe626ceac513143, vendor: 0, CPUID: a06d1, time: 1752701311, socket: 0, APIC: 0, microcode:
10003a2
# mca-recover-3270      [139] .N... 3445.910362: memory_failure_event: pfn 0xff7e88d:
recovery action for dirty LRU page: Recovered
# kworker/0:3-3225      [000] d..l. 3446.457161: non_standard_event: severity: 1; sec
type:b4efb980-b552-e34d-a777-68784b771048; FRU: 00000000-0000-0000-0000-000000000000 ; data
len:260; raw data:67 00 00 00 00 00 00 00 05 00 00 00 00 00 00 00 02 89 00 00 00 00 00 98
1e b0 0d 7f 1e 00 00 04 06 58 00 00 00 00 00 00 00 00 00 00 00 00 00 00 10 a0 42 01 22 80 00 00
27 51 00 00 85 48 7a 00 40 00 85 70 80 25 58 00 c0 03 48 01 1f 00 01 00 00 00 00 00 00 00 00 f7 1f b3
10 49 14 00 00 3e 00 80 01 05 00 1e 00 01 00 00 00 00 00 00 00 00 00 00 38 00 58 00 00 00 00 23 00
01 00 98 1e 41 01 07 00 27 00 be 00 26 00 17 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 30 00 00
ff cf 01 00 20 00 00 00 00 00 00 00 00 00 20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
# kworker/0:3-3225      [000] d..l. 3446.457163: non_standard_event: severity: 1; sec
type:b4efb980-b552-e34d-a777-68784b771048; FRU: 00000000-0000-0000-0000-000000000000 ; data
len:260; raw data:67 00 00 00 00 00 00 00 02 00 00 00 00 00 00 00 00 00 8a 00 00 00 00 00 98
1e e2 01 00 00 00 00 02 05 00 00 00 00 00 00 1d ed 4a 08 38 93 1a 24 10 b0 02 00 c1 8f 2c 11
37 50 01 00 05 61 43 00 40 00 85 10 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 10 00 b1
00 00 00 00 00 3e 00 80 01 05 00 1e 01 00 00 00 00 00 00 00 00 00 38 00 58 00 00 00 00 00 23 00
c1 40 98 1e 81 03 00 00 9e e8 06 40 00 00 00 00 00 01 00 00 00 20 00 00 00 4b 81 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 48 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
ff cf 01 00 20 00 00 00 00 00 00 00 20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
# kworker/0:0-3271      [000] .N... 3446.490714: aer_event: 0000:89:00.4 PCIe Bus Error:
severity=Corrected, Corrected Internal Error, TLP Header=Not available
# kworker/0:0-3271      [000] .N... 3446.515467: aer_event: 0000:8a:00.0 PCIe Bus Error:
severity=Corrected, Corrected Internal Error, TLP Header=Not available
# kworker/0:1-10      [000] .N... 3446.519226: mce_record: CPU: 139, MCGc/s: f001c20/f,
MC1: bd80000000100134, IPID: 0000000000000000, ADDR: 000000ff7e88d000, MISC: 00000000000000086,
SYND: 0000000000000000, RIP: 33:<0000000000401348>, TSC: 9a0e1a098d3, PPIN: 3fe626ceac513143,
vendor: 0, CPUID: a06d1, time: 1752701312, socket: 0, APIC: 127, microcode: 10003a2
```

Pass /Fail	Criteria
PASS	Check for an uncorrectable error for demand read on IMC banks Check : "bank" == 18 : any IMC bank
continued...	

Pass /Fail	Criteria
	- Check: "ADDR" == NM address (in this case alias_address) - Check: MC19_STATUS == ac00000200a00090
PASS	Check for a memory_failure_event at alias_Address
OS FAIL	Check for CXL Protocol Error Mem Poison Received on the RP. Look for 1, but found 0 cxl_aer_correctable_error ftrace event: Due to the above mentioned OS bug, Event type is shown as a Non-standard event - Check: "host" == "0000:89:02.0" (RP) (can vary) - Severity= 1 - Check: "status" & 0x20 : Mem Received Poison
OS FAIL	Check for CXL Protocol Error Mem Poison Received on the EP. Look for 1, but found 0 cxl_aer_correctable_error ftrace event: Due to the above mentioned OS bug, Event type is shown as a Non-standard event - Check: "host" == "0000:8a:00.0" (EP) (can vary) - Severity= 1 - Check: "status" & 0x20 : Mem Received Poison
PASS	Check for an SRAR error on DCU bank - Check : "bank" == 0x1 : Bank 1 for GNR - Check: "ADDR" == Address being consumed/read (in this case alias_address) - Check: MC1_STATUS == bd80000000100134
PASS	Check for PCIe Correctable Internal Error on the associated RCEC: - Event type: aer_event - BDF info (can vary): 0000:89:00.4 (RCEC) - Error type : PCIe Bus Error - Severity=Corrected, Corrected Internal Error
PASS	Check for CXL.io Correctable Internal Error on EP: - Event type: aer_event - BDF info (can vary): 0000:8a:00.0 (EP) - Error type : PCIe Bus Error - Severity= Advisory Non-Fatal Error Corrected Internal Error
PASS	<i>Note:</i> Additional event(s) are seen with this CXL device. Check with your CXL card vendor for implementation specific behavior.

5. Linux dmesg output

NOTE

The dmesg output depends on several factors like OS distro and kernel. Information is used for debugging purposes only.

```
[ 3442.689538] mce: Uncorrected hardware memory error in user-access at ff7e88d000
[ 3442.689544] mce: [Hardware Error]: Machine check events logged
[ 3442.689799] Memory failure: 0xff7e88d: Sending SIGBUS to mca-recover:3270 due to hardware
memory corruption
[ 3442.700488] {9}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 0
[ 3442.705411] Memory failure: 0xff7e88d: recovery action for dirty LRU page: Recovered
[ 3442.716390] {9}[Hardware Error]: It has been corrected by h/w and requires no further action
[ 3442.716393] {9}[Hardware Error]: event severity: corrected
[ 3442.716394] {9}[Hardware Error]: Error 0, type: corrected
[ 3442.716395] {9}[Hardware Error]: section_type: CXL Protocol Error
[ 3442.763531] {9}[Hardware Error]: agent_type: 5, CXL Root Port
[ 3442.770339] {9}[Hardware Error]: agent_address: 0000:89:02.0
[ 3442.777051] {9}[Hardware Error]: slot: 11
[ 3442.781903] {9}[Hardware Error]: vendor_id: 0x1e98, device_id: 0x0db0
[ 3442.789490] {9}[Hardware Error]: sub_vendor_id: 0x1e7f, sub_device_id: 0x0000
```

```
[ 3442.797859] {9}[Hardware Error]: class_code: 0604
[ 3442.803493] {9}[Hardware Error]: DVSEC length: 0x0038
[ 3442.809517] {9}[Hardware Error]: CXL DVSEC:
[ 3442.814571] {9}[Hardware Error]: 00000000: 00010023 01411e98 00270007 002600be
[ 3442.822941] {9}[Hardware Error]: 00000010: 00000017 00000000 00000000 00000000
[ 3442.831310] {9}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000000
[ 3442.839679] {9}[Hardware Error]: 00000030: 00000000 00000000
[ 3442.846283] {9}[Hardware Error]: Error log length: 0x0058
[ 3442.852698] {9}[Hardware Error]: CXL Error Log:
[ 3442.858129] {9}[Hardware Error]: cxl_ras_uncor_status: 0x00000000
[ 3442.858129] {9}[Hardware Error]: cxl_ras_uncor_mask: 0x00003000
[ 3442.872313] {9}[Hardware Error]: cxl_ras_uncor_severity: 0x0001cfff
[ 3442.879698] {9}[Hardware Error]: cxl_ras_cor_status: 0x00000020
[ 3442.879698] {9}[Hardware Error]: cxl_ras_cor_mask: 0x00000000
[ 3442.893497] {9}[Hardware Error]: cap_control: 0x00002000
[ 3442.899818] {9}[Hardware Error]: Header Log Registers:
[ 3442.905936] {9}[Hardware Error]: 00000000: 00000000 00000000 00000000 00000000
[ 3442.914305] {9}[Hardware Error]: 00000010: 00000000 00000000 00000000 00000000
[ 3442.922674] {9}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000000
[ 3442.931030] {9}[Hardware Error]: 00000030: 00000000 00000000 00000000 00000000
[ 3442.939399] {9}[Hardware Error]: Error 1, type: corrected
[ 3442.945612] {9}[Hardware Error]: section_type: PCIe error
[ 3442.951932] {9}[Hardware Error]: port_type: 10, root complex event collector
[ 3442.960100] {9}[Hardware Error]: version: 3.0
[ 3442.965248] {9}[Hardware Error]: command: 0x0504, status: 0x0010
[ 3442.972243] {9}[Hardware Error]: device_id: 0000:89:00.4
[ 3442.978456] {9}[Hardware Error]: slot: 0
[ 3442.983119] {9}[Hardware Error]: secondary_bus: 0x00
[ 3442.988941] {9}[Hardware Error]: vendor_id: 0x8086, device_id: 0x0b23
[ 3442.996421] {9}[Hardware Error]: class_code: 080700
[ 3443.002149] {9}[Hardware Error]: Error 2, type: corrected
[ 3443.008362] {9}[Hardware Error]: section_type: CXL Protocol Error
[ 3443.015451] {9}[Hardware Error]: agent_type: 2, CXL Device
[ 3443.021960] {9}[Hardware Error]: agent_address: 0000:8a:00.0
[ 3443.028671] {9}[Hardware Error]: slot: 0
[ 3443.033427] {9}[Hardware Error]: vendor_id: 0x1e98, device_id: 0x01e2
[ 3443.041014] {9}[Hardware Error]: sub_vendor_id: 0x0000, sub_device_id: 0x0000
[ 3443.049383] {9}[Hardware Error]: class_code: 0502
[ 3443.055016] {9}[Hardware Error]: DVSEC length: 0x0038
[ 3443.061026] {9}[Hardware Error]: CXL DVSEC:
[ 3443.066080] {9}[Hardware Error]: 00000000: 40c10023 03811e98 e89e0000 00004006
[ 3443.074448] {9}[Hardware Error]: 00000010: 00000000 00000001 00000020 0000814b
[ 3443.082817] {9}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000048
[ 3443.091186] {9}[Hardware Error]: 00000030: 00000000 00000000
[ 3443.097789] {9}[Hardware Error]: Error log length: 0x0058
[ 3443.104197] {9}[Hardware Error]: CXL Error Log:
[ 3443.109628] {9}[Hardware Error]: cxl_ras_uncor_status: 0x00000000
[ 3443.109629] {9}[Hardware Error]: cxl_ras_uncor_mask: 0x00000000
[ 3443.123819] {9}[Hardware Error]: cxl_ras_uncor_severity: 0x0001cfff
[ 3443.131203] {9}[Hardware Error]: cxl_ras_cor_status: 0x00000020
[ 3443.131204] {9}[Hardware Error]: cxl_ras_cor_mask: 0x00000000
[ 3443.145002] {9}[Hardware Error]: cap_control: 0x00002000
[ 3443.151322] {9}[Hardware Error]: Header Log Registers:
[ 3443.157440] {9}[Hardware Error]: 00000000: 00000000 00000000 00000000 00000000
[ 3443.165809] {9}[Hardware Error]: 00000010: 00000000 00000000 00000000 00000000
[ 3443.174178] {9}[Hardware Error]: 00000020: 00000000 00000000 00000000 00000000
[ 3443.182546] {9}[Hardware Error]: 00000030: 00000000 00000000 00000000 00000000
[ 3443.190915] {9}[Hardware Error]: Error 3, type: corrected
[ 3443.197128] {9}[Hardware Error]: section_type: PCIe error
[ 3443.203449] {9}[Hardware Error]: port_type: 0, PCIe end point
[ 3443.210159] {9}[Hardware Error]: version: 3.0
[ 3443.215307] {9}[Hardware Error]: command: 0x0546, status: 0x0010
[ 3443.222301] {9}[Hardware Error]: device_id: 0000:8a:00.0
[ 3443.228513] {9}[Hardware Error]: slot: 0
[ 3443.233176] {9}[Hardware Error]: secondary_bus: 0x00
[ 3443.238997] {9}[Hardware Error]: vendor_id: 0x1dfa, device_id: 0x01e2
[ 3443.246477] {9}[Hardware Error]: class_code: 050210
[ 3443.252238] Memory failure: 0xff7e88d: already hardware poisoned
[ 3443.259172] pcieport 0000:89:00.4: AER: aer_status: 0x00004000, aer_mask: 0x00000000
[ 3443.267942] pcieport 0000:89:00.4: [14] CorrIntErr
[ 3443.276397] pcieport 0000:89:00.4: AER: aer_layer=Transaction Layer, aer_agent=Receiver ID
```

```
[ 3443.285768] cxl_pci 0000:8a:00.0: AER: aer_status: 0x00004000, aer_mask: 0x00000000
[ 3443.294433] cxl_pci 0000:8a:00.0: [14] CorrIntErr
[ 3443.301253] cxl_pci 0000:8a:00.0: AER: aer_layer=Transaction Layer, aer_agent=Receiver ID
[ 3443.310567] EDAC skx: Bad imc -1
[ 3443.314272] mce: [Hardware Error]: Machine check events logged
```

11.5.5 Cache Line Write Recipes

In order to do a NM/FM cacheline write, the B2CMI will have to first do a read. So the UCE at NM/FM will be discovered during the read, and the error will be handled as seen in [Cache line Read recipes](#).

12.0 Cross-RAS

This section is for recipes that have multiple RAS features enabled at the same time. Only some RAS features are cross-compatible. See the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for more details on what is supported.

12.1 Runtime Soft PPR then VLS Action

Test Objective

Inject correctable errors, observe BIOS qualify lines for PPR, and repair the affected rows at runtime utilizing runtime soft PPR.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644, for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW44 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	2 DIMMs per socket
CScripts Version	GNR2344.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see the:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [Enhanced MCA Logging \(ELOG\)](#) on page 28

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Runtime PPR/Row Sparing = Enable

- Correctable Error Threshold = 1
- Trigger SW Error Threshold = Enable
- SW Per Row Threshold = 2
- ADDDC Sparing = Enable
- EDKII > Platform Configuration > System Event Log >
 - RAS Log Level = MID (BASIC_FLOW, FUNC_FLOW)
- EDKII > Platform Configuration > Miscellaneous Configuration >
 - Serial Debug Message Level = Normal

NOTE

sPPR is experiencing issues such as sightings report issue #120. BIOS/BMC developers can continue with development by temporarily bypassing the issue by turning off data scrambling.

Recipe

1. Halt the system and reset the injectors.

```
>>> halt()
>>> ei.utils.resetInjectorLockCheck()
```

2. Inject a correctable error then resume the system so BIOS can log the DIMM error record.

```
>>> ei.mem.injectMemError(0x2000_0000, errType="ce")
>>> go()
```

3. Observe in serial log that the SW Row Threshold counter is at 1/2.

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
[Updated] Policy bank level threshold: 3, row threshold: 2, time window: 24.
[Updated] UpdateEnhancedDimmErrRecord started.
[GetRRLErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0
Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x1 Dev:0x1 Col:0x0 Row:0x200 SingleBit:0x0
Transfer:0x0 PersistentError:0x0
Check Error is in VLS record structure.
AddNewVlsFailureSample Started.
[Updated] UpdateVlsFailureSample Entry.
[Updated] CurrSWBank_Counter/Bank_Threshold:1/3.
Check error is in PPR record structure.
AddNewPprFailureSample Started.
[Updated] UpdatePprFailureSample Entry.
[Updated] CurrSWRow_Counter/Row_Threshold:1/2.
[Updated] ErrorThReached=0
[Updated] UpdateEnhancedDimmErrRecord ended.
UpdateMcChanSsrState ch:1:PatrolSPAMode
McSsrState Node:1 :PatrolSPAMode
UpdateMcChanSsrState ch:2:PatrolSPAMode
McSsrState Node:2 :PatrolSPAMode
UpdateMcChanSsrState ch:3:PatrolSPAMode
McSsrState Node:3 :PatrolSPAMode
[imc] memory error handler end!
[ECS] CheckAndHandleECSEvent Enter
[ECS] Timestamp - skt:1 mc:0 ch:0 duration: 9 minutes
[ECS] Timestamp - skt:1 mc:1 ch:0 duration: 9 minutes
[ECS] Timestamp - skt:1 mc:2 ch:0 duration: 9 minutes
[ECS] Timestamp - skt:1 mc:3 ch:0 duration: 9 minutes
```



```
[ECS] CheckAndHandleECSEvent Exit
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]ProcessMcBankError: Scope Uncore
S0 C0 T0 [Mca]EmcaReadMcBankBitField: Scope Uncore, BitField 0x2000
S0 C0 T0 [Mca]McBankBitField: 0x2000
S0 C0 T0 [Mca]EmcaClearMcBankBitField: Scope Uncore, BitField 0x2000
S0 C0 T0 [Mca]McBankErrorHandler: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [Mca]McBankErrorHandler: IoMcaErrorHandler Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]MapErrorSeverity: MCE class is 0x5
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [Mca]McBankErrorHandler: Section Type = EMCA_SECTYPE_PLATMEM
FillPlatformMemoryErrorData Start!
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
FillPlatformMemoryErrorData: BankType:0x200
Error Source is M2M or IMC (MCCHAN)BankType:0x200
```

4. Observe an ELog record was generated for the error via FTrace:

```
kworker/0:1-10 [000] ..... 763.678279: extlog_mem_event: {1} corrected error: single-
bit ECC physical addr: 0000000020000000 (mask lsb: ff) node:0 card:0 module:0 rank:0 bank:4
device:1 row:512 column:0 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0
```

5. Repeat the last step so you reach the SW row threshold counter.

```
>>> halt()
>>> ei.mem.injectMemError(0x2000_0000, errType="ce")
>>> go()
```

6. Confirm in serial log that the SW row threshold was reached and runtime soft PPR RAS action was taken on the failing row.

```
[ECS] CheckAndHandleECSEvent Enter
[ECS] Timestamp - skt:0 mc:0 ch:0 duration: 12 minutes
[ECS] Timestamp - skt:0 mc:1 ch:0 duration: 12 minutes
[ECS] Timestamp - skt:0 mc:2 ch:0 duration: 12 minutes
[ECS] Timestamp - skt:0 mc:3 ch:0 duration: 12 minutes
[ECS] CheckAndHandleECSEvent Exit
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
[Updated] Policy bank level threshold: 3, row threshold: 2, time window: 24.
[Updated] UpdateEnhancedDimmErrRecord started.
[GetRRLErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0
Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x1 Dev:0x1 Col:0x0 Row:0x200 SingleBit:0x0
Transfer:0x0 PersistentError:0x0
Check Error is in VLS record structure.
[Updated] Error is recorded in vls structure!
[Updated] UpdateVlsFailureSample Entry.
[Updated] CurrSWBank Counter/Bank Threshold:2/3.
Check error is in PPR record structure.
[Updated] Error is recorded in sPPR structure!
[Updated] UpdatePprFailureSample Entry.
[Updated] CurrSWRow Counter/Row Threshold:2/2.
[Updated]Current Row Error Time=10, failure time=D
[Updated] Row threshold reached and in time window.
ReportsPPRErrorLog Entry
CreateMemoryErrorRecord Entry
ERROR: C00000002:V0307100A I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AF0B410
LogErrorToPprVls BIOS ID: BHSDCRBL.86B.2748.D01.2310240433
LogErrorToPprVls++, CodeType: 2, Value:307100A
ErrorRecordHeader->SectionCount: 1
ErrorRecordDescription->SectionType: A5BC1114-6F64-4EDE-B863-3E83ED7C83B1
```

```

ErrorRecordHeader->RecordLength: 118
sizeof(MEM_ERROR_RECORD): 118, sizeof(EMCA_MEM_ERROR_MULTI_RECORD): 500
MemoryErrorData->ValidFields: 3C3F9
MemoryErrorData->Node: 0
MemoryErrorData->Card: 0
MemoryErrorData->ModuleRank: 0
MemoryErrorData->Bank: 4
MemoryErrorData->Device: 1
MemoryErrorData->Row: 200
MemoryErrorData->Row[17:16]: 0
MemoryErrorData->Column: 0
MemoryErrorData->RankNum: 0
MemoryErrorData->CardHandle: 15
MemoryErrorData->ModuleHandle: 0
LogErrorToPprVls--, Success
ReportsPPRErrorLog Exit, Success
[Updated] ErrorThReached=1
[Updated] UpdateEnhancedDimmErrRecord ended.
RasEventHndlrEntry index: 0x0
CheckAndHandleRtRowSparing: Node: 0
Socket = 0, Ch = 0, SubCh = 0, Rank = 0
IsSpprResourceAvailable: For MR54/55/56/57: 54 data FFFFFFFF
Adding PPR entry at Index= 0
[SSR] SSR state Save! Flag:0
[SSR] stop and save patrol scrub info
UpdatePPRSpareRegion: NodeId[0].McCh[0].R[0].SubR[0].BankBG[4].Row[512].FailedDevice[0x1]
CopyClSize[32].Loops[4]
UpdatePPRSpareRegion Status=0
RuntimeSPRCopy Status=0
PrintPprEvent: Address-
Socket[0].MC[0].CH[0].D[0].PseudoCh[0].SubCh[0].R[0].CID[0].Bank[4].Row[512].FailDevMask = 0x2
[SSR] SSR state Save! Flag:1
[SSR] restore and restart patrol scrub info
  
```

7. Observe an ELog record was generated for the error:

```

kworker/0:0-22784 [000] .N... 895.207687: extlog_mem_event: {2} corrected error: single-
bit ECC physical addr: 0000000020000000 (mask lsb: ff) node:0 card:0 module:0 rank:0 bank:4
device:1 row:512 column:0 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0
  
```

8. Perform a third injection to the same bank but a different row to reach the SW bank threshold to trigger VLS action.

```

>>> halt()
>>> ei.mem.injectMemError(0x3000_0000, errType="ce")
>>> go()
  
```

9. Observe in serial log that bank VLS action was taken on the failing region.

```

[ECS] CheckAndHandleECSEvent Enter
[ECS] Timestamp - skt:0 mc:0 ch:0 duration: 15 minutes
[ECS] Timestamp - skt:0 mc:1 ch:0 duration: 15 minutes
[ECS] Timestamp - skt:0 mc:2 ch:0 duration: 15 minutes
[ECS] Timestamp - skt:0 mc:3 ch:0 duration: 15 minutes
[ECS] CheckAndHandleECSEvent Exit
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
[Updated] Policy bank level threshold: 3, row threshold: 2, time window: 24.
[Updated] UpdateEnhancedDimmErrRecord started.
[GetRRLErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Node is 0x0 NodeCh is 0x0
Rank:0x0 SubRank:0x0 Bank:0x0 BankGroup:0x1 Dev:0x1 Col:0x0 Row:0x300 SingleBit:0x0
Transfer:0x0 PersistentError:0x0
Check Error is in VLS record structure.
[Updated] Error is recorded in vls structure!
[Updated] UpdateVlsFailureSample Entry.
[Updated] CurrSWBank Counter/Bank Threshold:3/3.
[Updated]Current bank Time=13, failure time=D
[Updated] Bank threshold reached and in time window.
  
```

```

Check error is in PPR record structure.
AddNewPprFailureSample Started.
[Updated] UpdatePprFailureSample Entry.
[Updated] CurrSWRow_Counter/Row_Threshold:1/2.
[Updated] ErrorThReached=1
[Updated] UpdateEnhancedDimmErrRecord ended.
RasEventHndlrEntry index: 0x0
CheckAndHandleRtRowSparing: Node: 0
CheckAndHandleRtRowSparing Exit:
RasEventHndlrEntry index: 0x1
[ADDDC]: Main handler start...
[ADDDC]: HandleNewAdddcEvent() Start...
[ADDDC]: Check Node 0, CH 0...
Bank SwThreshold reached
*****
[GetFailedDeviceInfoFromSwErrorThreshold]: Node: 0, Ch 0, Rank 0, SubRank 0, Bank 0,
BankGrp 1, Device 1.
*****
===== Dump Adddc Status: Node 0, Ch 0 =====
- Rank 0, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
- Rank 1, RMode N, DevSts 0x00000000 -
- SubRank 00:
- Bank : 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31
- Bank Mode: N N N N N N N N N N N N N N N N N N N N N N N N N N
N N N N N N N
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x0, RankOnDimm 0x3 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x0 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x1 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x2 is not enabled!
S 0x0, CH 0x0, Dimm 0x1, RankOnDimm 0x3 is not enabled!

[ADDDC]NonRankVlsMode: Call HandleNonRankVlsStrike()...
[ADDDC]: HandleNonRankVlsStrike() start...
[ADDDC]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0, Bank 4, Device 1.
[ADDDC]: BankMode is 0.
[ADDDC]: No previous error in device 1.
[ADDDC]: Call HandleBankVlsEvent()...
[ADDDC]: HandleBankVlsEvent()... Start
[GetAvailableRegion]: NodeId 0, ChOnNode 0, Region 0, Status Success.
[GetNewNonFailedRankBank]: NodeId 0, ChOnNode 0, Rank 0, SubRank 0 Bank 4.
:No available pair rank for the failed rank 0 in channel 0, so select itself
as buddy rank for the bank vls event.
[GetNewNonFailedRankBank]: Rank 0 - Buddy Rank 0, Bank 4 - Buddy Bank 5.
AdddcEvent0:
Type 3, Region 0 (PRank 0 : PBank 4) - (BRank 0 : BBank 5),
FailedDevice 1.
AdddcEvent1:
Type 3, Region 0 (PRank 0 : PBank 4) - (BRank 0 : BBank 5),
FailedDevice 1.
[ADDDC]: HandleBankVlsEvent()... End
[ADDDC]: HandleNonRankVlsStrike() Status Success.
[ADDDC]: The event has been successfully handled!
[ADDDC]: Call SchedulePendingList()
DisableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
[SSR] SSR state Save! Flag:80
[SSR] disable C6 & GV
[SSR] stop and save patrol scrub info
[ADDDC]: ShedulePendingList()... Start
[ActionOnBankVls] Region 0, NodeId 0, Ch 0, FailedRank 0 - NonFailedRank 0, FailedBank
4 - NonFailedBank 5.
VlsSparingCopy: Bank AdddcSparingMR
Quiesce Successful
ReportSpareErrorLog Entry
S1 C5 T1 CreateRasNonStandardErrorRecord Entry, Severity: 2, ErrorType: 0

```

```

S1 C5 T1 CreateRasNonStandardErrorRecord Exit, Status: Success
ERROR: C00000002:V03071008 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AF0B410
  LogErrorToWhea BIOS ID: BHSDCRB1.86B.2748.D01.2310240433
[WheaLog]LogErrorToWhea++, CodeType: 2
[WheaLog] Erasing all status blocks
[WheaLog] Erasing all status blocks
[WheaLog]RestartAllWheaElog: Success
ErrorRecordHeader SignatureStart: 0x52455043
ErrorRecordHeader Revision: 0x300
ErrorRecordHeader SignatureEnd: 0xFFFFFFFF
ErrorRecordHeader SectionCount: 0x1
ErrorRecordHeader ErrorSeverity: 0x2
ErrorRecordHeader ValidationBits: 0x0
ErrorRecordHeader RecordLength: 0x104
ErrorRecordHeader TimeStamp: 0x0
ErrorRecordHeader PlatformID: 0x00000000-0000-0000-0000-000000000000
ErrorRecordHeader CreatorID: 0x00000000-0000-0000-0000-000000000000
ErrorRecordHeader NotificationType: 0x2DCE8BB1-BDD7-450E-B9AD-9CF4EBD4F890
ErrorRecordHeader RecordID: 0x0
ErrorRecordHeader PersistenceInfo: 0x0
WHEA: Detected Ras Non Standard Error
[WheaLog] Not supported for Memory Error or Processor error or any Ras non standard error
LogErrorToElog++, CodeType: 2, Value: 3071008
isEmcaOptInFlagSet (): 0
ElogEn: 1
ElogIgnOptin: 0
  LogErrorToPprVls BIOS ID: BHSDCRB1.86B.2748.D01.2310240433
LogErrorToPprVls++, CodeType: 2, Value:3071008
ErrorRecordHeader->SectionCount: 1
ErrorRecordDescription->SectionType: CAB360F5-5E8E-4BF0-9EB2-D4691DC0DB29
LogErrorToPprVls--, Success
ReportSpareErrorLog Exit, Status: Success
Creating RAS State variable successful
UpdateRasStateVariable: Success
[ADDDC]: Event type 3, Action Status Success.
[ADDDC]: SchedulePendingList()...End
[ADDDC]: HandleNewAdddcEvent() return, Status = Success.
[ADDDC]: Main handler return, Status Success.

```

10. Observe that an ELog record was generated for the error:

```

kworker/0:0-22784 [000] ..... 1213.639714: extlog_mem_event: {3} corrected error: single-
bit ECC physical addr: 0000000030000000 (mask lsb: ff) node:0 card:0 module:0 rank:0 bank:4
device:1 row:768 column:0 DIMM location: not present. DMI handle: 0x0000 FRU:
00000000-0000-0000-0000-000000000000 Card01, ChnA, DIMM0

```

11. Halt the system and confirm with CScripts that bank VLS action was taken.

```

>>> halt()
>>> ras.mem.adddc status_check(socket=0, mc=0)
***** Sparing Control *****
=====
| Socket | iMC-Channel | Field | Value | Description |
|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | tad_rule | 0x0 | |
|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | bank_sparing | 0x0 | |
|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | region_size | 0x0 | Bank |
|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | adddc_sparing | 0x1 | |
|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | spare_reverse | 0x0 | |
|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | spare_enable | 0x1 | Sparing Enabled |
|-----|-----|-----|-----|-----|
***** Sparing Patrol Status *****
=====
| Socket | iMC-Channel | Field | Value | Description |
|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | patrol_complete | 0x0 | |
|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | copy_complete | 0x0 | |
|-----|-----|-----|-----|-----|
| socket0 | mc0-ch0 | copy_in_progress | 0x1 | |
|-----|-----|-----|-----|-----|

```

***** Sparing Interval Status *****

Socket	iMC-Channel	Field	Value	Description
socket0	mc0-ch0	numspare	0x320	Sparing Operation Period
socket0	mc0-ch0	normopdur	0xc80	Normal Operation Duration
socket0	mc0-ch0	intrpt_sel_smi	0x1	

***** Sparing Source Status *****

Socket	iMC-Channel	Field	Value	Description
socket0	mc0-ch0	source_cs	0x0	Source Chip Select
socket0	mc0-ch0	source_rir	None	RIR index used for ADDDC sparing
socket0	mc0-ch0	source_rir_en	None	Enable bit for source_rir to overwrite the HW value

Register Dump		Region #	Socket	iMC-Channel	CS	SubRank	BankGp	Bank	Region
SVLS	failed	0	socket0	mc0-ch0	0x0	0x0	0x1	0x0	Bank
SVLS	non-failed	0	socket0	mc0-ch0	0x0	0x0	0x1	0x1	Bank
SVLS	---	0	socket0	mc0-ch0	Failed	Device	=	0x1	Rank0
SVLS	---	1	socket0	mc0-ch0	-	-	-	-	disabled
SVLS	---	2	socket0	mc0-ch0	-	-	-	-	disabled
SVLS	---	3	socket0	mc0-ch0	-	-	-	-	disabled

12.2 Full Memory Mirroring and Runtime Soft PPR on the Primary Channel with Correctable Errors

Test Objective

The objective of this test is to show runtime soft PPR RAS action being taken on correctable errors while in full memory mirror mode on the primary channel.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW44'23 BKC
OS Version	RHEL 9.2
Kernel	6.7.0-rc4-2023-12-04-intel-next+
HW Configuration	4 DIMMs per socket, 10x4 single rank, 32 GB DIMMs on slots A1, B1, C1, and D1 (Recommended to use 1 or 2 DPC because they are the only POR prime configurations which support mirroring)
CScripts Version	GNR2334.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#)

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Mirror Mode = Full Mirror Mode
 - Runtime PPR/Row Sparing = Enable
 - Trigger SW Error Threshold = Enable
 - SW Per Row Threshold = 2
- Socket Configuration > Platform Configuration > System Event Log >
 - RAS Log Level = MID (BASIC_FLOW, FUNC_FLOW)

Recipe

1. Halt and confirm there are no preexisting memory related errors on the system. Clear any found error logs or cold boot the system.

```
>>> halt()
>>> error.check_mem_errors()
```

2. Verify that mirroring is enabled by reading the following bit-field.

```
>>> sv.socket0.soc.memss.b2cmi0.mode.mirror
0x1
```

3. Check that the memory configuration aligns with what is expected. Full memory mirroring, therefore, active memory capacity is equal to half the installed capacity; all channels are operating in mirror mode and the mirror size is equal to the active memory size. With the current CScripts version there were issues seen with the reported mirror size values but everything else aligns with expectations.

```
>>> mc.dimminfoNoSPD()
```

```
...
```

	Socket 0	Socket 1	System
Active Memory	64.0GB	64.0GB	128.0GB
Expected Memory	NotAvail	NotAvail	NotAvail
Clustering Mode	Hex	Hex	Hex
1LM Memory Size	64.00GB	64.00GB	128.00GB
2LM Memory Size	0.00 B	0.00 B	0.00 B
AppDirect Memory Size	0.00 B	0.00 B	0.00 B
DDR Frequency	4800MHz	4800MHz	4800MHz
Max DDR Frequency	6400MHz	6400MHz	
MC0 CL-RCD-RP-CMD	40-40-41-1n	40-40-41-1n	
MC1 CL-RCD-RP-CMD	40-40-41-1n	40-40-41-1n	
MC2 CL-RCD-RP-CMD	40-40-41-1n	40-40-41-1n	
MC3 CL-RCD-RP-CMD	40-40-41-1n	40-40-41-1n	
MC0 CL-RCD-RP-CMD	40-40-41-1n	40-40-41-1n	
MC1 CL-RCD-RP-CMD	40-40-41-1n	40-40-41-1n	
MC2 CL-RCD-RP-CMD	40-40-41-1n	40-40-41-1n	
MC3 CL-RCD-RP-CMD	40-40-41-1n	40-40-41-1n	
ECC Checking / PFD	On / On	On / On	On / On
ECC Mode	128b	128b	128b
CAP/WrCRC	On / Disable	On / Disable	On / Disable
Patrol/DemandScrub	On / On	On / On	On/On

Memory Mode	1LM	1LM	1LM
Channel Mode	Mirror	Mirror	Mirror
Refresh Mode	AllBankNormal	AllBankNormal	AllBankNormal
RFM	Off	Off	Off
ZQCal/DQSOSC/RxRt	On / On / Off	On / On / Off	On / On / Off
Mirror Size	64.0GB	64.0GB	0GB
ADDDC	Disabled	Disabled	Disabled
Encryption	Disabled	Disabled	Disabled
Paging Policy	AdaptOpen	AdaptOpen	AdaptOpen
Directory Mode	Off	Off	Off
DDR Scrambling	Enabled	Enabled	Enabled
CKE Throttling	PPD	PPD	PPD
Thermal Throttling	CLTT	CLTT	CLTT
NUMA			On

4. Print the valid TAD arrays to check that they show they are mirrored.

```
>>> mc.addTran()
...
TAD @ Socket0 Mc0

TAD[ S0 ][ MC0 ][ 0 ]: Mirrored
Range      : 0.00 B up to 2.00 GB (0x0 - 0x7fff_ffff)
Span       : 2.00 GB
Ch Offset  : 0x0 (0.00 )
target_ways : 0 (1-way) PA[12] - 4KB granularity
chan_ways   : 2 (4-way) PA[8] - 256B granularity
ddr5_Ch0 Range : 0x0 (0.00 ) - 0x3fff_ffff (1024.00 M)
ddr5 Sub-Ch Range : 0x0 (0.00 ) - 0x1fff_ffff (512.00 M)

TAD[ S0 ][ MC0 ][ 1 ]: Mirrored
Range      : 2.00 GB up to 66.00 GB (0x8000_0000 - 0x10_7fff_ffff)
Span       : 64.00 GB
Ch Offset  : 0x8000_0000 (2.00 G)
target_ways : 0 (1-way) PA[12] - 4KB granularity
chan_ways   : 2 (4-way) PA[8] - 256B granularity
ddr5_Ch0 Range : 0x0 (0.00 ) - 0x7_ffff_ffff (32.00 G)
ddr5 Sub-Ch Range : 0x0 (0.00 ) - 0x3_ffff_ffff (16.00 G)
```

5. Use the following command to translate how a few addresses, within the specified system address range, map to its physical DIMM address. Take note of the memory controller ("mc") column. From this table, system addresses can be selected to test injecting errors into mirrored regions on different memory controllers.

```
>>> ras.mem.sa2da_table(0x100000000, 0x100000400, 64)
```

System Address	Socket	mc	Channel	Sub channel	DIMM	Rank	Sub rank	Bank group	Bank	Row	Column	Mem type	Rank Address	Sub-Channel address
0x100000000	0	0	0	0	0	0	0	0	0	0	8192	0	ddr5	268435456 536870912
0x100000040	0	0	0	0	0	0	0	0	0	0	8192	16	ddr5	268435520 536870976
0x100000080	0	0	0	1	0	0	0	0	0	0	8192	0	ddr5	268435456 536870912
0x1000000c0	0	0	0	1	0	0	0	0	0	0	8192	16	ddr5	268435520 536870976
0x100000100	0	1	0	0	0	0	0	0	0	0	8192	0	ddr5	268435456 536870912
0x100000140	0	1	0	0	0	0	0	0	0	0	8192	16	ddr5	268435520 536870976
0x100000180	0	1	0	1	0	0	0	0	0	0	8192	0	ddr5	268435456 536870912
0x1000001c0	0	1	0	1	0	0	0	0	0	0	8192	16	ddr5	268435520 536870976
0x100000200	0	2	0	0	0	0	0	0	0	0	8192	0	ddr5	268435456 536870912
0x100000240	0	2	0	0	0	0	0	0	0	0	8192	16	ddr5	268435520 536870976
0x100000280	0	2	0	1	0	0	0	0	0	0	8192	0	ddr5	268435456 536870912
0x1000002c0	0	2	0	1	0	0	0	0	0	0	8192	16	ddr5	268435520 536870976
0x100000300	0	3	0	0	0	0	0	0	0	0	8192	0	ddr5	268435456 536870912
0x100000340	0	3	0	0	0	0	0	0	0	0	8192	16	ddr5	268435520 536870976
0x100000380	0	3	0	1	0	0	0	0	0	0	8192	0	ddr5	268435456 536870912
0x1000003c0	0	3	0	1	0	0	0	0	0	0	8192	16	ddr5	268435520 536870976

6. Check if the injectors are locked for all sockets and unlock if locked.

```
>>> ei.utils.checkInjectorLock(and_unlock=True)
```

- Choose the target device for error injection and modify masks for a single device correctable ECC error injection.

```
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=2, dev1msk=0x0)
dev0 = 0x1, dev0msk = 0x3, dev1 = 2, dev1msk = 0x0.
```

- Inject and consume the correctable ECC error. Resume the system so BIOS can log the error.

```
>>> ei.mem.injectMemError(addr=0x000100000000, errType="ce", verbose=True)
>>> go()
```

- Observe in serial log that the SW row threshold is currently 1/2.

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
[Updated] Policy bank level threshold: 3, row threshold: 2, time window: 24.
[Updated] UpdateEnhancedDimmErrRecord started.
S1 C18 T1 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Rank:0x0
SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x0 Row:0x2000 SingleBit:0x0 Transfer:0x0
PersistentError:0x0 InterLocSyn:0x0
Check Error is in VLS record structure.
AddNewVlsFailureSample Started.
[Updated] UpdateVlsFailureSample Entry.
[Updated] CurrSWBank_Counter/Bank_Threshold:1/3.
Check error is in PPR record structure.
AddNewPprFailureSample Started.
[Updated] UpdatePprFailureSample Entry.
[Updated] CurrSWRow_Counter/Row_Threshold:1/2.
[Updated] ErrorThReached=0
[Updated] UpdateEnhancedDimmErrRecord ended.
UpdateMcChanSsrState ch:1:PatrolSPAMode
McSsrState Node:1 :PatrolSPAMode
UpdateMcChanSsrState ch:2:PatrolSPAMode
McSsrState Node:2 :PatrolSPAMode
UpdateMcChanSsrState ch:3:PatrolSPAMode
McSsrState Node:3 :PatrolSPAMode
[imc] memory error handler end!
[ECS] CheckAndHandleECSEvent Enter
[ECS] Timestamp - skt:1 mc:0 ch:0 duration: 40 minutes
[ECS] Timestamp - skt:1 mc:1 ch:0 duration: 40 minutes
[ECS] Timestamp - skt:1 mc:2 ch:0 duration: 40 minutes
[ECS] Timestamp - skt:1 mc:3 ch:0 duration: 40 minutes
[ECS] CheckAndHandleECSEvent Exit
[Mca]CheckMcBankErrorStatus returns TRUE
[Mca]McaDetectAndHandle start
[Mca]McaDetectAndHandle, state is 0x0
[Mca]McaDetectAndHandle, state is 0x1
[Mca]ProcessSocketMcBankError: Inside the function
S0 C0 T0 [Mca]ProcessMcBankError: Scope Uncore
S0 C0 T0 [Mca]EmcaReadMcBankBitField: Scope Uncore, BitField 0x2000
S0 C0 T0 [Mca]McBankBitField: 0x2000
S0 C0 T0 [Mca]EmcaClearMcBankBitField: Scope Uncore, BitField 0x2000
S0 C0 T0 [Mca]McBankErrorHandler: Inside the function
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x1
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [Mca]McBankErrorHandler: IoMcaErrorHandler Skt = 0x0, McBank = 0xD, State = 0x2
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
S0 C0 T0 [Mca]MapErrorSeverity: MCE class is 0x5
S0 C0 T0 [Mca]McBankErrorHandler: Skt = 0x0, McBank = 0xD, State = 0x4
S0 C0 T0 [Mca]McBankErrorHandler: Section Type = EMCA_SECTYPE_PLATMEM
FillPlatformMemoryErrorData Start!
S0 C0 T0 [CpuRas]MC status 0x8C00004200800090, class CORRECTED
FillPlatformMemoryErrorData: BankType:0x200
Error Source is M2M or IMC(MCCHAN)BankType:0x200
```


10. Halt the system and perform the same injection. Resume the system after the injection so BIOS can log the error.

```
>>> halt()
>>> ei.mem.injectMemError(addr=0x000100000000, errType="ce", verbose=True)
>>> go()
```

11. Observe that the SW row threshold was met and runtime soft PPR action was taken on the address in the primary channel.

```
[imc] memory error handler start!
UpdateMcChanSsrState ch:0:PatrolSPAMode
McSsrState Node:0 :PatrolSPAMode
[Updated] Policy bank level threshold: 3, row threshold: 2, time window: 24.
[Updated] UpdateEnhancedDimmErrRecord started.
S1 C139 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Rank:0x0
SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x0 Row:0x2000 SingleBit:0x0 Transfer:0x0
PersistentError:0x0 InterLocSyn:0x0
Check Error is in VLS record structure.
[Updated] Error is recorded in vls structure!
[Updated] UpdateVlsFailureSample Entry.
[Updated] CurrSWBank Counter/Bank_Threshold:2/3.
Check error is in PPR record structure.
[Updated] Error is recorded in sPPR structure!
[Updated] UpdatePprFailureSample Entry.
[Updated] CurrSWRow Counter/Row Threshold:2/2.
[Updated]Current Row Error Time=BF510A, failure time=BF5108
[Updated] Row threshold reached and in time window.
[Updated] ErrorThReached=1
[Updated] UpdateEnhancedDimmErrRecord ended.
RasEventHndlrEntry index: 0x0
CheckAndHandleRtRowSparing: Node: 0
Socket = 0, Ch = 0, SubCh = 0, Rank = 0
IsSpprResourceAvailable: For MR54/55/56/57: 54 data FFFFFFFF
Adding PPR entry at Index= 0
[SSR] SSR state Save! Flag:0
[SSR] stop and save patrol scrub info
UpdatePPRSpareRegion: NodeId[0].McCh[0].R[0].SubR[0].BankBG[0].Row[8192].FailedDevice[0x1]
CopyClSize[32].Loops[4]
UpdatePPRSpareRegion Status=0
RuntimeSPPRCopy Status=0
PrintPprEvent: Address-
Socket[0].MC[0].CH[0].D[0].PseudoCh[0].SubCh[0].R[0].CID[0].Bank[0].Row[8192].FailDevMask = 0x2
ReportsPPRErrorLog Entry
CreateMemoryErrorRecord Entry
ERROR: C00000002:V03071009 I0 C698D0D1-904A-42E9-8406-152A6FF6549B 7AF89210
LogErrorToPprVls BIOS ID: BHSDCRB1.SYS.2526.D01.2307311547
LogErrorToPprVls++, CodeType: 2, Value:3071009
ErrorRecordHeader->SectionCount: 1
ErrorRecordDescription->SectionType: A5BC1114-6F64-4EDE-B863-3E83ED7C83B1
ErrorRecordHeader->RecordLength: 118
sizeof(MEM_ERROR_RECORD): 118, sizeof(EMCA_MEM_ERROR_MULTI_RECORD): 500
MemoryErrorData->ValidFields: 3C3F9
MemoryErrorData->Node: 0
MemoryErrorData->Card: 0
MemoryErrorData->ModuleRank: 0
MemoryErrorData->Bank: 0
MemoryErrorData->Device: 1
MemoryErrorData->Row: 2000
MemoryErrorData->Row[17:16]: 0
MemoryErrorData->Column: 0
MemoryErrorData->RankNum: 0
MemoryErrorData->CardHandle: 15
MemoryErrorData->ModuleHandle: 0
LogErrorToPprVls--, Success
ReportsPPRErrorLog Exit,Success
[SSR] SSR state Save! Flag:1
[SSR] restore and restart patrol scrub info
EnableCorrectableSmiOnCh: Socket 0 ChOnSkt 0
CheckAndHandleRtRowSparing Exit:
```

```
S1 C139 T0 [GetChannelErrorInfo]:raw data EccMode is 0x1. Uncorrect is 0x0 Rank:0x0
SubRank:0x0 Bank:0x0 BankGroup:0x0 Dev:0x1 Col:0x0 Row:0x2000 SingleBit:0x0 Transfer:0x0
PersistentError:0x0 InterLocSyn:0x0
```

12. Verify Runtime sPPR was successful by checking the following bit 1 in the following SPPR_CTRL0 CSR:

```
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.sppr_ctrl0.show()
0x00000001 : failed_device (21:16) -- Programmed by SMM BIOS to indicate failed
0x00000000 : failed_ba (15:14) -- Programmed by SMM BIOS to indicate failed bank
0x00000000 : failed_bg (13:10) -- Programmed by SMM BIOS to indicate failed bank
0x00000000 : failed_c (09:05) -- Programmed by SMM BIOS to indicate failed logical
0x00000000 : failed_cs (04:02) -- Programmed by SMM BIOS to indicate failed DIMM
0x00000001 : success_failed (01:01) -- 1: sPPR is successful, 0: sPPR failed
0x00000000 : start_inprogress (00:00) -- Pcode sets this bit to start sPPR sequence
```

12.3 [TDX][LMCE=ON][OS-First] Inject a UC Non-Fatal Error into a Trust Domain Guest

Test Objective

Inject an Uncorrectable non-fatal error into a Trust Domain (TD) guest. Expected result is for the TD guest with error to be terminated. Any other TD guests which are open and running will continue to run and will not be effected. The system and Intel® Trust Domain Extensions (Intel® TDX) module should recover and continue running. The test is executed in OS-first mode of operation and with LMCE enabled. The Injection can be broken down into four steps:

1. First step is to make sure Intel TDX module is installed, setup and configured.
2. Second step is to launch the TD guest and start the victim application which would return a Guest Physical Address from the TD Guest.
3. Start a Qemu session to dump the memory within the TD guest and perform the address translation from the Guest Physical Address to the Host Virtual Address.
4. Translate the Host Virtual Address to Host Physical Address, and inject the error using APEI EINJ into that address and trace the OS serial log.

Configuration

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW 19'24 BKC
OS Version	Ubuntu 23.10 / Ubuntu 24.04
Kernel	Linux 6.5.0-1003-intel-opt x86_64
HW Configuration	Intel SGX/Intel TDX Hardware and Software Configuration. See <i>Confidential Computing Integration and Validation Guide for the Birch Stream Platform Integration and Validation Guide</i> , document number 815188 for more information on the required configuration.
CScripts Version	GNR2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used.

- [RAS-Tools Scripts](#) on page 20
- [APEI EINJ](#)

Apply the following BIOS knobs to enable Intel® Software Guard Extensions (Intel® SGX)/Intel® TDX, Error injection and reporting, and OS First mode:

- Error Reporting Enabling
 - EDKII > Platform Configuration > System Event Log > WHEA Settings >
 - WHEA Support = Enable
 - WHEA Log Memory Error = Enable
 - WHEA Log Processor Error = Enable
 - WHEA Log PCI Error = Enable
- Error Injection Enabling
 - EDKII > Platform Configuration > System Event Log >
 - System Errors = Enable
 - System Memory Poison = Enable
 - EDKII > Platform Configuration > System Event Log > Error Injection Settings >
 - MCA Bank Error Injection Support = Enable
 - WHEA Error Injection Support = Enable
 - Intel SGX Memory Error Injection Support = Enable
 - EDKII > Platform Configuration > System Event Log > EMCA Settings > LMCE Support = Enable
 - EDKII > Platform Configuration > System Event Log > Memory Error Enabling > Memory Corrected Error = Enable
 - EDKII > Socket Configuration > Uncore Configuration > Uncore General Configuration > Directory Mode Enable = Enable
- Intel SGX/Intel TDX Enabling
 - EDKII > Socket Configuration > Security Configuration >
 - Intel® Memory Encryption (Intel® TME) = Enable
 - Intel® Total Memory Encryption Multi-Tenant (Intel® TME-MT) = Enable
 - Intel TDX = Enable
 - Intel TDX Secure Arbitration Mode Loader (SEAM Loader) = Enable
 - Intel SGX = Enable
 - Intel SGX Auto MP Registration Agent = Enable
- OS First Mode Enabling
 - EDKII > Platform Configuration > System Event Log >
 - eMCA Settings > EMCA Logging Support = Disable
 - IIO Error Enabling > OS Native AER Support = Enable
- Make sure the system is setup for Intel TDX by following this link: <https://github.com/canonical/tdx>. More details on the setup and installation of Intel TDX for end-to-end testing is included in the See *Confidential Computing Integration*

and Validation Guide for the Birch Stream Platform Integration and Validation Guide, document number [815188](#). If the Intel TDX module is not setup as directed, it is not possible to create any TD guest VMs.

Recipe

1. In the host Kernel, check if Intel TDX module is initialized. The output returned must return the CMR ranges and that the Intel TDX module is initialized.

```
# dmesg | grep -i tdx
[ 0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-6.5.0-1003-intel-opt
root=UUID=b16d0f96-1052-4dbf-9bdb-40a27cd7b45c ro kvm_intel.tdx=1 nomodeset quiet splash
ras=cec_disable vt.handoff=7
[ 3.504665] Kernel command line: BOOT_IMAGE=/boot/vmlinuz-6.5.0-1003-intel-opt
root=UUID=b16d0f96-1052-4dbf-9bdb-40a27cd7b45c ro kvm_intel.tdx=1 nomodeset quiet splash
ras=cec_disable vt.handoff=7
[ 6.630011] tdx: BIOS enabled: private KeyID range [64, 128)
[ 24.790445] tdx: TDX module: attributes 0x0, vendor_id 0x8086, major_version 2,
minor_version 0, build_date 20240328, build_num 737
[ 24.790456] tdx: CMR: [0x100000, 0x77800000)
[ 24.790460] tdx: CMR: [0x100000000, 0x1ffe000000)
[ 24.790461] tdx: CMR: [0x2080000000, 0x4000000000)
[ 24.790462] tdx: CMR: [0x4080000000, 0x6000000000)
[ 24.790463] tdx: CMR: [0x6080000000, 0x8000000000)
[ 24.790464] tdx: CMR: [0x8080000000, 0xa000000000)
[ 24.790465] tdx: CMR: [0xa080000000, 0xc000000000)
[ 28.482224] tdx: 3102652 KBs allocated for PAMT.
[ 28.482238] tdx: module initialized.
[ 800.633552] UBSAN: array-index-out-of-bounds in /build/linux-intel-opt-Gcjo4J/linux-intel-
opt-6.5.0/arch/x86/kvm/vmx/tdx.c:2609:59
[ 800.633736] index 1 is out of range for type 'tdx_cpuid_value [*]'
[ 800.633916] tdx_td_init+0x23e/0x330 [kvm_intel]
[ 800.633941] tdx_vm_ioctl+0x9d/0x200 [kvm_intel]
[ 800.634134] ? tdx_vm_enable_cap+0x63/0x90 [kvm_intel]
```

2. Before launching the TD guest, it is required to add additional parameters into the TD launch command file. Add the following arguments to the end of the command in the run_td.sh file.

```
-monitor telnet:127.0.0.1:5555,server,nowait
```

3. Launch the TD guest. Boot into the TD guest with the TD Kernel image. This will return the port number and host ID to SSH into.

NOTE

Make note of the PID returned in the second command, for example, "PID: 5642". This is needed for address translation in step 10.

```
# cd tdx/guest-tools
# TD_IMG=image/tdx-guest-ubuntu-23.10.qcow2 ./run_td.sh
TD VM, PID: 5642, SSH : ssh -p 10022 root@localhost

# ssh -p 10022 root@localhost
(root@localhost) Password:
Welcome to Ubuntu 23.10 (GNU/Linux 6.5.0-1003-intel-opt x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Wed Jun 19 00:04:50 UTC 2024

System load:                1.09
```

```
Usage of /:          5.2% of 50.78GB
Memory usage:       30%
Swap usage:         0%
Processes:          636
Users logged in:    0
IPv4 address for enp0s1: 10.0.2.15
IPv6 address for enp0s1: fec0::5054:ff:fe12:3456
```

7 updates can be applied immediately.
To see these additional updates run: `apt list --upgradable`

The list of available updates is more than a week old.
To check for new updates run: `sudo apt update`

4 updates could not be installed automatically. For more details,
see `/var/log/unattended-upgrades/unattended-upgrades.log`

```
=====
Welcome to Ubuntu mantic (23.10) TDX guest.
Created by Kobuk team
=====
Last login: Tue Jun 18 20:09:15 2024 from 10.0.2.2
root@tdx-guest:~#
```

4. Verify that Intel TDX is enabled in the TD guest by running the following command. It should return "tdx: Guest detected".

```
# dmesg | grep -i tdx

[0.000000] tdx: Guest detected
[0.000000] DMI: QEMU Standard PC (Q35 + ICH9, 2009), BIOS 2023.05-2+tdx1.0 11/05/2023
[0.737228] process: using TDX aware idle routine
[0.737228] Memory Encryption Features active: Intel TDX
[7.105115] systemd[1]: Hostname set to <tdx-guest>.
```

5. As a workload, use the Victim application; Victim is a tool used in MCE testing that can give a Guest Physical Address (GPA) address of VM - TD, which is used to inject into the TD guest. This application is found in ras-tools tool. This application needs to be accessed from the TD guest.
6. The Victim application is compiled and copied into the TD guest using the `virt-copy-in` command. However, this must be done before the TD guest is launched. So if TD guest is launched before copying the application, shutdown and exit from the TD guest before executing the next steps.

```
# virt-copy-in -a <your-guest-image-name> <the-file-you-want-to-copy>
<destination-directory-in-guest-vm>
```

Before running the command:

1. The VM needs to be shutdown
2. You may need to install "libguestfs" via "sudo apt-get install libguestfs"

After the command is done, please restart the VM and you can see files in destination directory of VM.

7. Run the Victim application in the TD guest using the command `./victim -d`. This would give the GPA of the application within the TD guest.

NOTE

There is no need to hit any key to trigger the error, the Victim application is only used to retrieve an address within the TD guest.

```
# cd /home/tdx/
# ./victim -d

physical address of (0x55e22c47ccfa) = 0x29bafcfa
Hit any key to trigger error:
```

8. In a second terminal, connect to the TD using "telnet 127.0.0.1 5555". This parameter is the addition which is made to the TD boot command file.

```
# telnet 127.0.0.1 5555
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
QEMU 8.0.4 monitor - type 'help' for more information
(qemu)
```

9. The Qemu session established is connected to the TD guest which is used to convert the Guest Physical Address (GPA) which is returned by the Victim application into an Host Virtual Address (HVA). Dump the address returned by the Victim application, this will make the host aware of the existence of that corresponding guest page. Follow this with the GPA to HVA translation, and lastly follow with the HVA to Host Physical Address (HPA) translation in a separate step due to the security restrictions of Intel TDX, which restricts it from returning the HPA without root privileges, which is absent in the Qemu session.

```
(qemu) xp/80hx 0x29bafcfa
0000000029bafcfa: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd0a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd1a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd2a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd3a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd4a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd5a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd6a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd7a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd8a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000

(qemu) gpa2hpa 0x29bafcfa
Host physical address for 0x29bafcfa (ram1) is 0xcfa

(qemu) gpa2hva 0x29bafcfa
Host virtual address for 0x29bafcfa (ram1) is 0x7f8a519afcfa
```

NOTE

The function which can translate GPA to HPA is "gpa2hpa <GPA>" returns an address which is not correct. It is important to use this command even if it is not used and is incorrect, as its absence will label the page as unhandleable to the OS.

10. Run the next steps in a third terminal. Now that HVA is obtained, it needs to be translated to HPA, which is done using "vtop" application in the ras-tools. For the address translation, vtop needs the HVA (0x7f8a519afcfa) which is returned in the previous step and the PID (5642) returned in step 3. Make sure to use the PID of the TD guest.

```
# cd ras-tools
# sudo ./vtop 0x7f8a519afcfa 5642
vtop(7f8a519afcfa,5642) = 1a87f1cfa
```

11. The Address returned previously is the HPA = 0x1a87f1cfa, which is the address of error injection. A Memory Uncorrectable non-fatal error is injected.

```
# modprobe einj
# cd /sys/kernel/debug/apei/einj
# echo <HPA> > param1
```

```
# echo 0xffffffffffff000 > param2
# echo 0x10 > error_type
# echo 1 > error_inject
```

- After injecting the error, it is expected for the TD guest and the Qemu session connected to the TD guest to be terminated. It is expected to see a "Connection closed by foreign host" message in terminals running the TD guest and the Qemu sessions. The following is the serial log output which can be accessed with dmesg. The page is recovered and the SIGBUS is sent to the Trust Domain Guest, which can be identified by the PID and the HPA address. A UC patrol scrubbing error is logged at the address. The system should recover and keep running.

Output from TD guest terminal:

```
Connection to localhost closed by remote host.
Connection to localhost closed
```

Output from Qemu terminal:

```
(qemu) Connection closed by foreign host.
```

```
# dmesg
[ 1389.051225] mce: [Hardware Error]: Machine check events logged
[ 1389.114720] Memory failure: 0x1a87f1: Sending SIGBUS to td:5642 due to hardware memory corruption
[ 1389.114881] Memory failure: 0x1a87f1: recovery action for dirty LRU page: Recovered
[ 1389.115783] show_signal_msg: 87 callbacks suppressed
[ 1389.115790] CPU 0/KVM[5647]: segfault at 88 ip 000055c3cad7bb80 sp 00007f8aafb7a530 error 4 in qemu-system-x86_64[55c3ca740000+694000] likely on CPU 81 (core 65, socket 1)
[ 1389.115810] Code: 2e 0f 1f 84 00 00 00 00 90 f3 0f 1e fa 41 56 41 55 41 54 55 53 48 83 ec 20 64 48 8b 04 25 28 00 00 00 48 89 44 24 18 31 c0 <80> 7f 28 00 0f 84 89 01 00 00 44 8b 05 37 03 8e 00 48 89 fb 48 89
[ 1389.117374] EDAC skx MC3: HANDLING MCE MEMORY ERROR
[ 1389.117379] EDAC skx MC3: CPU 0: Machine Check Event: 0x0 Bank 16: 0xac000002001000c0
[ 1389.117381] EDAC skx MC3: TSC 0x1bc6842ca5d
[ 1389.117383] EDAC skx MC3: ADDR 0x1a87f1cc0
[ 1389.117385] EDAC skx MC3: MISC 0x2080032114391886
[ 1389.117386] EDAC skx MC3: PROCESSOR 0:0xa06d0 TIME 1718756044 SOCKET 0 APIC 0x0
[ 1389.117401] EDAC MC3: 1 UE memory scrubbing error on CPU_SrcID#0_MC#3_Ch#0_DIMM#0 (channel:0 slot:0 page:0x1a87f1 offset:0xcc0 grain:32 - err_code:0x0010:0x00c0 SystemAddress:0x1a87f1cc0 ProcessorSocketId:0x0 MemoryControllerId:0x3 ChannelAddress:0x4a1fc7c0 ChannelId:0x0 RankAddress:0x1287f1c0 PhysicalRankId:0x1 DimmSlotId:0x0 DimmRankId:0x1 Row:0x2287 Column:0x230 Bank:0x1 BankGroup:0x6 ChipSelect:0x1)
```

- Decode the MC Status returned from the dmesg for MSCOD and MCACOD values. The MSCOD should indicate an "Uncorr Patrol Scrub Error".

```
>>> error.manualMcaDecode()

Manual Machine Check Bank Decoder
Enter MCi_STATUS, MCi_ADDR, MCi_MISC values
Enter 't' for a short demo
Enter 'Q' or 'q' to exit

Enter MCA Bank Number [0,1,2,3,4,5,6,PCU,MSE,MCHN,B2CMI,CBO,LLC,UPI] or q for quit: MCHN
Enter MCi_STATUS [0x0]: 0xac000002001000c0

Bank name: MCHN, Error = UNCORR UnCorr Patrol Scrub Error

+ MCMCHN_STATUS      0xAC000002001000C0

Range                Field
```

Description	Value	Value	Description
63:63 this register is valid	VALID 0x1	Valid	Indicates that the information within
62:62 error occurred while still in the error- was already set)	OVERFLOW 0x0	Not Overflow	A '1' indicates that a machine-check the results of a previous error were reporting register bank (the VALID bit
61:61 error condition	UC 0x1	UC Detected	A '1' indicates that an uncorrectable the was detected.
60:60 that error flag bit(s) in the	EN 0x0	Not Enabled	Error reporting enabled. A '1' indicates reporting is enabled by the associated IA32_MCi_CTL register.
59:59 register content is	MISCV 0x1	IA32_MCi_MISC Valid	If this flag is '1', the IA32_MCi_MISC valid and can be read.
58:58 register content is	ADDRV 0x1	IA32_MCi_ADDR Valid	If this flag is '1', the IA32_MCi_ADDR valid and can be read.
57:57 indicates that reliable possible and a reset	PCC 0x0	Processor Context Corrupt: A '1'	Processor Context Not Corrupted restarting of the processor may not be is recommended.
56:56 error.	S 0x0	---	Signaling an uncorrectable recoverable
55:55 error.	AR 0x0	---	Recovery action required for UCR
54:53 status	THRESHOLD_BASED_STATUS	Threshold-based error 0x0	---
52:38 including sticky	CORRECTED_ERROR_COUNT 0x0	---	Count of correctable errors in the DCU overflow indicator (bit 52)
37:37 Indicator	FW	FW Update Error Status 0x0	---
36:32 Architecture	OTHER_INFO	Not part of Machine-Check 0x2	---
31:16 Code	MSCOD	Model Specific Error 0x10	UnCorr Patrol Scrub Error
15:0 Code SCRUBBING ERROR CHANNEL_0	MCACOD 0xc0	Machine Check Architecture Error MCCHAN_ERROR(1LM MODE) MEMORY	

12.4 [TDX][LMCE=ON][OS-First] Inject a Correctable Error into a TD Guest

Test Objective

Inject a Correctable error into a Trust Domain (TD) guest. The expected result is for the TD guest with corrected error to be reported. The TD Guest, and any applications running in the guest are to be not effected and will recover. The system and the Intel TDX module should recover and continue running. The test is executed in OS-first mode of operation and with LMCE enabled. The Injection can be broken down into four steps:

1. First step is to make sure Intel TDX module is installed, setup and configured.
2. Second step is to launch the TD guest and start the victim application which would return a Guest Physical Address from the TD Guest.
3. Start a Qemu session to dump the memory within the TD guest and perform the address translation from Guest Physical Address to Host Virtual Address.
4. Translate the Host Virtual Address to Host Physical Address, and inject the error using APEI EINJ into that address and trace the OS serial log.

Configuration

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW 19'24 BKC
OS Version	Ubuntu 23.10 / Ubuntu 24.04
Kernel	Linux 6.5.0-1003-intel-opt x86_64
HW Configuration	Intel SGX/Intel TDX Hardware and Software Configuration. See <i>Confidential Computing Integration and Validation Guide for the Birch Stream Platform Integration and Validation Guide</i> , document number 815188 for more information on the required configuration.
CScripts Version	GNR2406.6000

Preconditions and Setup

Validation tools used in this recipe. See the following for more information about each tool used:

- [RAS-Tools Scripts](#) on page 20
- [APEI EINJ](#)

Apply the following BIOS knobs to enable Intel SGX/Intel TDX, error injection and reporting, and OS-first mode:

- Error Reporting Enabling
 - EDKII > Platform Configuration > System Event Log > WHEA Settings >
 - WHEA Support = Enable
 - WHEA Log Memory Error = Enable
 - WHEA Log Processor Error = Enable
 - WHEA Log PCI Error = Enable

- Error Injection Enabling
 - EDKII > Platform Configuration > System Event Log >
 - System Errors = Enable
 - System Memory Poison = Enable
 - EDKII > Platform Configuration > System Event Log > Error Injection Settings >
 - MCA Bank Error Injection Support = Enable
 - WHEA Error Injection Support = Enable
 - Intel SGX Memory Error Injection Support = Enable
 - EDKII > Platform Configuration > System Event Log > EMCA Settings > LMCE Support = Enable
 - EDKII > Platform Configuration > System Event Log > Memory Error Enabling > Memory Corrected Error = Enable
 - EDKII > Socket Configuration > Uncore Configuration > Uncore General Configuration > Directory Mode Enable = Enable
- Intel SGX/Intel TDX Enabling
 - EDKII > Socket Configuration > Security Configuration >
 - Intel TME = Enable
 - Intel TME-MT = Enable
 - Intel TDX = Enable
 - Intel TDX Secure Arbitration Mode Loader (SEAM Loader) = Enable
 - Intel SGX = Enable
 - Intel SGX Auto MP Registration Agent = Enable
- OS First Mode Enabling
 - EDKII > Platform Configuration > System Event Log >
 - eMCA Settings > EMCA Logging Support = Disable
 - IIO Error Enabling > OS Native AER Support = Enable
- Make sure the system is setup for Intel TDX by following this link: <https://github.com/canonical/tdx>. More details on the setup and installation of Intel TDX for end-to-end testing is included in the *Confidential Computing Integration and Validation Guide for the Birch Stream Platform Integration and Validation Guide*, document number 815188. If the Intel TDX module is not setup as directed, it is not possible to create any TD guest VMs.

Recipe

1. In the host Kernel, check if the Intel TDX module is initialized. The output returned must return the CMR ranges and that the Intel TDX module is initialized.

```
# dmesg | grep -i tdx
[ 0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-6.5.0-1003-intel-opt
root=UUID=b16d0f96-1052-4dbf-9bdb-40a27cd7b45c ro kvm_intel.tdx=1 nomodeset quiet splash
ras=cec_disable vt.handoff=7
[ 3.504665] Kernel command line: BOOT_IMAGE=/boot/vmlinuz-6.5.0-1003-intel-opt
root=UUID=b16d0f96-1052-4dbf-9bdb-40a27cd7b45c ro kvm_intel.tdx=1 nomodeset quiet splash
ras=cec_disable vt.handoff=7
[ 6.630011] tdx: BIOS enabled: private KeyID range [64, 128)
```

```
[ 24.790445] tdx: TDX module: attributes 0x0, vendor_id 0x8086, major_version 2,
minor version 0, build date 20240328, build num 737
[ 24.790456] tdx: CMR: [0x100000, 0x77800000)
[ 24.790460] tdx: CMR: [0x100000000, 0x1ffe000000)
[ 24.790461] tdx: CMR: [0x2080000000, 0x4000000000)
[ 24.790462] tdx: CMR: [0x4080000000, 0x6000000000)
[ 24.790463] tdx: CMR: [0x6080000000, 0x8000000000)
[ 24.790464] tdx: CMR: [0x8080000000, 0xa000000000)
[ 24.790465] tdx: CMR: [0xa080000000, 0xc000000000)
[ 28.482224] tdx: 3102652 KBs allocated for PAMT.
[ 28.482238] tdx: module initialized.
[ 800.633552] UBSAN: array-index-out-of-bounds in /build/linux-intel-opt-Gcjo4J/linux-intel-
opt-6.5.0/arch/x86/kvm/vmx/tdx.c:2609:59
[ 800.633736] index 1 is out of range for type 'tdx_cpuid_value [*]'
[ 800.633916] tdx_td_init+0x23e/0x330 [kvm_intel]
[ 800.633941] tdx_vm_ioctl+0x9d/0x200 [kvm_intel]
[ 800.634134] ? tdx_vm_enable_cap+0x63/0x90 [kvm_intel]
```

2. Before launching the TD guest, it is required to add additional parameters into the TD launch command file. Add the following arguments to the end of the command in the `run_td.sh` file.

```
-monitor telnet:127.0.0.1:5555,server,nowait
```

3. Launch the TD guest. Boot into the TD guest with the TD Kernel image. This will return the port number and host ID to SSH into.

NOTE

Make note of the PID returned in the second command, for example, "PID: 5642". This is needed for address translation in step 10.

```
# cd tdx/guest-tools
# TD_IMG=image/tdx-guest-ubuntu-23.10.qcow2 ./run_td.sh
TD VM, PID: 8558, SSH : ssh -p 10022 root@localhost
intel@ubuntu-labdcgae-16:~/tdx/guest-tools$ ssh -p 10022 root@localhost
(root@localhost) Password:
Welcome to Ubuntu 23.10 (GNU/Linux 6.5.0-1003-intel-opt x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Jun 20 23:21:38 UTC 2024

System load:                0.46
Usage of /:                  5.2% of 50.78GB
Memory usage:               31%
Swap usage:                 0%
Processes:                  641
Users logged in:            0
IPv4 address for enp0s1:    10.0.2.15
IPv6 address for enp0s1:    fec0::5054:ff:fe12:3456

7 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

4 updates could not be installed automatically. For more details,
see /var/log/unattended-upgrades/unattended-upgrades.log

=====
Welcome to Ubuntu mantic (23.10) TDX guest.
Created by Kobuk team
```

```
=====
Last login: Wed Jun 19 00:04:52 2024 from 10.0.2.2
root@tdx-guest:~#
```

4. Verify Intel TDX is enabled in the TD guest by running the following command. It should return "tdx: Guest detected".

```
# dmesg | grep -i tdx

[0.000000] tdx: Guest detected
[0.000000] DMI: QEMU Standard PC (Q35 + ICH9, 2009), BIOS 2023.05-2+tdx1.0 11/05/2023
[0.737228] process: using TDX aware idle routine
[0.737228] Memory Encryption Features active: Intel TDX
[7.105115] systemd[1]: Hostname set to <tdx-guest>.
```

5. As a workload, use the Victim application; Victim is a tool used in MCE testing that can give a Guest Physical Address (GPA) address of VM - TD, which is used to inject into the TD guest. The Victim application is found in ras-tools tool. This application needs to be accessed from the TD guest.
6. The Victim application is compiled and copied into the TD guest using the `virt-copy-in` command. However this must be done before the TD guest is launched. So if TD guest is launched before copying the Victim application, shutdown and exit from the TD guest before executing the next steps.

```
# virt-copy-in -a <your-guest-image-name> <the-file-you-want-to-copy>
<destination-directory-in-guest-vm>

Before running the command:
1. The VM needs to be shutdown
2. You may need to install "libguestfs" via "sudo apt-get install libguestfs"
After the command is done, please restart the VM and you can see files in
destination directory of VM.
```

7. Run the Victim application in the TD guest using the command `./victim -d`. This would give the GPA of the victim application within the TD guest.

NOTE

There is no need to hit any key to trigger the error, the Victim application is only used to retrieve an address within the TD guest.

```
# cd /home/tdx/
# ./victim -d

physical address of (0x7f5efede2000) = 0x2b7fb000
Hit any key to trigger error:
```

8. In a second terminal, connect to the TD using "telnet 127.0.0.1 5555". This parameter is the addition which is made to the TD boot command file.

```
# telnet 127.0.0.1 5555
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^'.
QEMU 8.0.4 monitor - type 'help' for more information
(qemu)
```

9. The Qemu session established is connected to the TD guest which is used to convert the Guest Physical Address (GPA) which is returned by the Victim application into an Host Virtual Address (HVA). Dump the address returned by the Victim application, this will make the host aware of the existence of that corresponding guest page. Follow this with the GPA to HVA translation, and lastly

follow with the HVA to Host Physical Address (HPA) translation in a separate step due to the security restrictions of Intel TDX which restricts it from returning the HPA without root privileges, which is absent in the Qemu session.

```
(qemu) xp/80hx 0x2b7fb000
0000000029bafcfa: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd0a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd1a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd2a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd3a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd4a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd5a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd6a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd7a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0000000029bafd8a: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000

(qemu) gpa2hpa 0x2b7fb000
Host physical address for 0x29bafcfa (ram1) is 0x0

(qemu) gpa2hva 0x2b7fb000
Host virtual address for 0x29bafcfa (ram1) is 0x7f72e35fb000
```

NOTE

The function which can translate GPA to HPA is "gpa2hpa <GPA>" returns an address which is not correct. It is important to use this command even if it is not used and is incorrect, as its absence will label the page as unhandleable to the OS.

- Run the next steps in a third terminal. Now that HVA is obtained, it needs to be translated to HPA, which is done using "vtop" application in the ras-tools. For the address translation, vtop needs the HVA (0x7f72e35fb000) which is returned in the previous step and the PID (5642) returned in step 3. Make sure to use the PID of the TD guest.

```
# cd ras-tools
# sudo ./vtop 0x7f72e35fb000 8558
vtop(7f72e35fb000,8558) = 40ad431000
```

- The address returned previously is the HPA = 0x40ad431000, which is the address of error injection. A memory correctable error is injected using APEI EINJ into the HPA.

```
# modprobe einj
# cd /sys/kernel/debug/apei/einj
# echo <HPA> > param1
# echo 0xfffffffffffff000 > param2
# echo 0x8 > error_type
# echo 1 > error_inject
```

- After injecting the error, it is expected for the TD guest and the Qemu session connected to the TD guest to recover and stay up. The following is the serial log output which can be accessed with dmesg. A CE memory read error is logged at the address. The system and the Trust Domain guest will recover and keep running.

```
# dmesg

[ 3209.111261] mce: [Hardware Error]: Machine check events logged
[ 3209.111654] EDAC skx MC1: HANDLING MCE MEMORY ERROR
[ 3209.111658] EDAC skx MC1: CPU 0: Machine Check Event: 0x0 Bank 14: 0x8c00004200800090
[ 3209.111661] EDAC skx MC1: TSC 0x3bf6f8598a7
[ 3209.111663] EDAC skx MC1: ADDR 0x40ad431000
[ 3209.111664] EDAC skx MC1: MISC 0x200008a016a11086
[ 3209.111665] EDAC skx MC1: PROCESSOR 0:0xa06d0 TIME 1718926217 SOCKET 0 APIC 0x0
[ 3209.111678] EDAC MC1: 1 CE memory read error on CPU_SrcID#0_MC#1_Ch#0_DIMM#0 (channel:0)
```

```
slot:0 page:0x40ad431 offset:0x0 grain:32 syndrome:0x0 - err_code:0x0080:0x0090
SystemAddress:0x40ad431000 ProcessorSocketId:0x0 MemoryControllerId:0x1
ChannelAddress:0xb50c400 ChannelId:0x0 RankAddress:0x2d43100 PhysicalRankId:0x0
DimmSlotId:0x0 DimmRankId:0x0 Row:0x2d4 Column:0x220 Bank:0x1 BankGroup:0x1 ChipSelect:0x0
```

- Decode the MC Status returned from the dmesg for MSCOD and MCACOD values. The MSCOD should indicate an "Uncorr Patrol Scrub Error".

```
>>> error.manualMcaDecode()

Manual Machine Check Bank Decoder
Enter MCI_STATUS, MCI_ADDR, MCI_MISC values
Enter 't' for a short demo
Enter 'Q' or 'q' to exit

Enter MCA Bank Number [0,1,2,3,4,5,6,PCU,MSE,MCHN,B2CMI,CBO,LLC,UPI] or q for
quit: MCHN
Enter MCI_STATUS [0x0]: 0x8c00004200800090

Bank name: MCHN, Error = CORR Transient or correctable Error for Demand or
Underfill Reads

+ MCMCHN_STATUS      0x8C00004200800090
```

Range Description	Field	Value	Value Description
63:63 this register is valid	VALID 0x1	Valid	Indicates that the information within
62:62 error occurred while still in the error- was already set)	OVERFLOW 0x0	Not Overflow	A '1' indicates that a machine-check the results of a previous error were reporting register bank (the VALID bit
61:61 error condition	UC 0x0	No UC	A '1' indicates that an uncorrectable the was detected.
60:60 that error flag bit(s) in the	EN 0x0	Not Enabled	Error reporting enabled. A '1' indicates reporting is enabled by the associated IA32_MCI_CTL register.
59:59 register content is	MISCV 0x1	IA32_MCI_MISC Valid	If this flag is '1', the IA32_MCI_MISC valid and can be read.
58:58 register content is	ADDRV 0x1	IA32_MCI_ADDR Valid	If this flag is '1', the IA32_MCI_ADDR valid and can be read.
57:57 indicates that reliable possible and a reset	PCC 0x0	Processor Context Corrupt: A '1' Processor Context Not Corrupted	restarting of the processor may not be is recommended.
56:56 error.	S 0x0	---	Signaling an uncorrectable recoverable
55:55	AR		Recovery action required for UCR

error.	0x0	---
54:53 THRESHOLD_BASED_STATUS	Threshold-based error	---
status	0x0	---
52:38 CORRECTED_ERROR_COUNT	Count of correctable errors in the DCU	---
including sticky	0x1	---
	overflow indicator (bit 52)	
37:37 FW	FW Update Error Status	---
Indicator	0x0	---
36:32 OTHER_INFO	Not part of Machine-Check	---
Architecture	0x2	---
31:16 MSCOD	Model Specific Error	
Code	0x80	Transient or correctable
Error for Demand or Underfill Reads		
15:0 MCACOD	Machine Check Architecture Error	
Code	0x90	MCCHAN_ERROR(1LM MODE) MEMORY READ
ERROR CHANNEL_0		

13.0 RAS Offload

RAS Offload is a technique that offloads certain error handling to BMC to minimize SMI usage. When RAS Offload is enabled, the processor and platform components assert ERR0 pin when Correctable Errors occur. The BMC monitors the state of the ERR0 pin or the polling timer and executes RAS handlers on signal assertion.

13.1 PCIe: Receiver Error

Test Objective

Verify that PCIe correctable errors are offloaded to BMC. BIOS should disable SMI and enable ERR0 for RAS Offload.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW25'23 BKC
BMC FW	WW25'23 BMC FW
OS Version	Cent OS Stream 8
Kernel	5.19.0-gnr.po.bkc.7.2.21.x86_64
HW Configuration	Generic PCIe Card
CScripts Version	GNR2322.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log >
 - OOB RAS Support = Enable

NOTE

Increment the verbosity "ipmitool raw 0x32 0x23 0x4 0x1 0x2 0x0 0x3F"

PCIe and Memory initialization "ipmitool raw 0x32 0x24 0x05 0x00 0x00 0x00"

Recipe

1. Confirm that no error is seen in the root port and corresponding IEH hierarchy.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.errcorsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.devsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts
0x0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtsts
0x0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtctl.show()
0x00000000 : reserved (31:03) -- Reserved
0x00000001 : fa_sysev_m (02:02) -- Fatal Error Mask(FA_SYSEV_M) Fatal Error System Event...
0x00000001 : nf_sysev_m (01:01) -- Nonfatal Error Mask(NF_SYSEV_M) NonFatal Error...
0x00000001 : co_sysev_m (00:00) -- Correctable Error Mask(CO_SYSEV_M) Correctable...
```

2. Execute error injection. Inject a PCIe CE error to PCIe root port by CScripts command.

```
>>> ei.utils.resetInjectorLockCheck()
>>> ei.iio.injectPcieError(socket=0, port="pxp0.rp2", errType="ce")
```

3. ERR0 will trigger and BMC RAS Offload will be activated. Ensure logging registers are cleared.

ROOT PORT

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.errcorsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.devsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.errsrcid.show()
0x00000000 : efsid (31:16) -- Loaded with the Requester ID indicated in the received
ERR_FATAL...
0x00000000 : ecsid (15:00) -- Loaded with the Requester ID indicated in the received ERR_COR...
```

SIEH

```
>>> sv.sockets.io0.uncore.satieh.satiehs.gerrcorsts
socket0.io0.uncore.satieh.satieh0.gerrcorsts - 0x00000000
socket0.io0.uncore.satieh.satieh1.gerrcorsts - 0x00000000
socket0.io0.uncore.satieh.satieh2.gerrcorsts - 0x00000000
socket0.io0.uncore.satieh.satieh3.gerrcorsts - 0x00000000
socket0.io0.uncore.satieh.satieh4.gerrcorsts - 0x00000000
socket0.io0.uncore.satieh.satieh5.gerrcorsts - 0x00000000
socket0.io0.uncore.satieh.satieh6.gerrcorsts - 0x00000000
socket0.io0.uncore.satieh.satieh7.gerrcorsts - 0x00000000
socket1.io0.uncore.satieh.satieh0.gerrcorsts - 0x00000000
socket1.io0.uncore.satieh.satieh1.gerrcorsts - 0x00000000
socket1.io0.uncore.satieh.satieh2.gerrcorsts - 0x00000000
socket1.io0.uncore.satieh.satieh3.gerrcorsts - 0x00000000
socket1.io0.uncore.satieh.satieh4.gerrcorsts - 0x00000000
socket1.io0.uncore.satieh.satieh5.gerrcorsts - 0x00000000
socket1.io0.uncore.satieh.satieh6.gerrcorsts - 0x00000000
socket1.io0.uncore.satieh.satieh7.gerrcorsts - 0x00000000
>>> sv.sockets.io0.uncore.satieh.satiehs.gsysevtsts
socket0.io0.uncore.satieh.satieh0.gsysevtsts - 0x00000000
socket0.io0.uncore.satieh.satieh1.gsysevtsts - 0x00000000
```

```
socket0.io0.uncore.satieh.satieh2.gsysevtsts - 0x00000000
socket0.io0.uncore.satieh.satieh3.gsysevtsts - 0x00000000
socket0.io0.uncore.satieh.satieh4.gsysevtsts - 0x00000000
socket0.io0.uncore.satieh.satieh5.gsysevtsts - 0x00000000
socket0.io0.uncore.satieh.satieh6.gsysevtsts - 0x00000000
socket0.io0.uncore.satieh.satieh7.gsysevtsts - 0x00000000
socket1.io0.uncore.satieh.satieh0.gsysevtsts - 0x00000000
socket1.io0.uncore.satieh.satieh1.gsysevtsts - 0x00000000
socket1.io0.uncore.satieh.satieh2.gsysevtsts - 0x00000000
socket1.io0.uncore.satieh.satieh3.gsysevtsts - 0x00000000
socket1.io0.uncore.satieh.satieh4.gsysevtsts - 0x00000000
socket1.io0.uncore.satieh.satieh5.gsysevtsts - 0x00000000
socket1.io0.uncore.satieh.satieh6.gsysevtsts - 0x00000000
socket1.io0.uncore.satieh.satieh7.gsysevtsts - 0x00000000
```

GIEH

```
>>> sv.sockets.io0.uncore.globalieh.iehregs.gerrcorsts
socket0.io0.uncore.globalieh.iehregs.gerrcorsts - 0x00000000
socket1.io0.uncore.globalieh.iehregs.gerrcorsts - 0x00000000
>>> sv.sockets.io0.uncore.globalieh.iehregs.gsysevtsts
socket0.io0.uncore.globalieh.iehregs.gsysevtsts - 0x00000000
socket1.io0.uncore.globalieh.iehregs.gsysevtsts - 0x00000000
```

4. OS logs.

```
[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 1
[Hardware Error]: It has been corrected by h/w and requires no further action
[Hardware Error]: event severity: corrected
[Hardware Error]: Error 0, type: corrected
[Hardware Error]: section_type: PCIe error
[Hardware Error]: port_type: 4, root port
[Hardware Error]: version: 3.0
[Hardware Error]: command: 0x0547, status: 0x0010
[Hardware Error]: device_id: 0000:14:06.0
[Hardware Error]: slot: 2
[Hardware Error]: secondary_bus: 0x15
[Hardware Error]: vendor_id: 0x8086, device_id: 0x0db2
[Hardware Error]: class_code: 060400
[Hardware Error]: bridge: secondary_status: 0x2000, control: 0x0003
pcieport 0000:14:06.0: AER: aer_status: 0x00000001, aer_mask: 0x00000000
pcieport 0000:14:06.0: [ 0] RxErr (First)
pcieport 0000:14:06.0: AER: aer_layer=Physical Layer, aer_agent=Receiver ID
```

5. Verify RAS manager logs in BMC.

```
> journalctl -u com.intel.RAS -b --no-pager
RAS_INFO::[ras_manager] ERR0 Asserted
RAS_INFO::[event_listener] PCIe Error Event: Severity: CORRECTED, Slot 2 Seg 00h, PrimBus 14h,
SecBus 15h, Dev 06h, Fun 00h, Vid 8086h, Did 0db2h
RAS_INFO::[whea_listener] PCIe Error CPER added to WHEA buffer.
```

13.2 PCIe: CTO Triggers DPC

Test Objective

Verify that PCIe correctable errors are offloaded to BMC. BIOS should disable SMI and enable ERR0 for RAS Offload.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon®*

6900/6700/6500-Series Processors with P-Cores, IIntel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW25'23 BKC
BMC FW	WW25'23 BMC FW
OS Version	Cent OS Stream 8
Kernel	5.19.0-gnr.po.bkc.7.2.21.x86_64
HW Configuration	Generic PCIe Card
CScripts Version	GNR2322.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log >
 - OOB RAS Support = Enable
- EDKII > Platform Configuration > System Event Log > IIO Error Enabling >
 - IIO EDPC Support = On Fatal and Non-Fatal Errors

NOTE

Increment the verbosity "ipmitool raw 0x32 0x23 0x4 0x1 0x2 0x0 0x3F".

PCIe and Memory initialization "ipmitool raw 0x32 0x24 0x05 0x00 0x00 0x00".

Recipe

1. Confirm that no error is seen in the root port and corresponding IEH hierarchy. Confirm that CTO is unmasked, the severity is configured as expected, and DPC is enabled.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.devsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts
0x0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtsts
0x0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtctl.show()
0x00000000 : reserved (31:03) -- Reserved
0x00000001 : fa_sysev_m (02:02) -- Fatal Error Mask(FA_SYSEV_M) Fatal Error...
0x00000001 : nf_sysev_m (01:01) -- Nonfatal Error Mask(NF_SYSEV_M) NonFatal...
0x00000001 : co_sysev_m (00:00) -- Correctable Error Mask(CO_SYSEV_M) Correctable...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncmsk.show()
0x00000000 : ptlpebm (26:26) -- Poisoned TLP Egress Blocked Mask
```

```

0x00000000 : tpbem (25:25) -- TLP Prefix Blocked Error Mask
0x00000000 : aebem (24:24) -- AtomicOp Egress Blocked Mask
0x00000000 : mcem (23:23) -- MC Blocked TLP Mask
0x00000000 : uiem (22:22) -- Uncorrectable Internal Error Mask
0x00000000 : acsem (21:21) -- ACS Violation Mask
0x00000001 : urem (20:20) -- Unsupported Request Error Mask
0x00000000 : ecrsem (19:19) -- ECRC Error Mask
0x00000000 : mtlpem (18:18) -- Malformed TLP Mask
0x00000000 : roem (17:17) -- Receiver Overflow Mask
0x00000000 : ucem (16:16) -- Unexpected Completion Mask
0x00000000 : caem (15:15) -- Completer Abort Mask
0x00000000 : ctem (14:14) -- Completion Timeout Mask
0x00000000 : fcem (13:13) -- Flow Control Protocol Error Mask
0x00000000 : ptlpem (12:12) -- Poisoned TLP Received Mask
0x00000001 : sldem (05:05) -- This bit does not apply to the...
0x00000000 : dlpem (04:04) -- Data Link Protocol Error Mask
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsev.show()
0x00000000 : ptlpebs (26:26) -- Poisoned TLP Egress Blocked Severity
0x00000000 : tpbem (25:25) -- TLP Prefix Blocked Error Severity
0x00000000 : aebem (24:24) -- AtomicOp Egress Blocked Severity
0x00000000 : mces (23:23) -- MC Blocked TLP Severity
0x00000001 : uies (22:22) -- Uncorrectable Internal Error Severity
0x00000000 : acses (21:21) -- ACS Violation Severity
0x00000000 : ures (20:20) -- Unsupported Request Error Severity
0x00000000 : ecrses (19:19) -- ECRC Error Severity
0x00000001 : mtlpes (18:18) -- Malformed TLP Severity
0x00000001 : roes (17:17) -- Receiver Overflow Error Severity
0x00000000 : uces (16:16) -- Unexpected Completion Error Severity
0x00000000 : caes (15:15) -- Completer Abort Error Severity
0x00000000 : ctes (14:14) -- Completion Timeout Error Severity
0x00000001 : fces (13:13) -- Flow Control Protocol Error Severity
0x00000001 : ptlpes (12:12) -- Poisoned TLP Received Severity
0x00000000 : sldes (05:05) -- This bit does not apply to the...
0x00000001 : dlpes (04:04) -- Data Link Protocol Error Severity
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x00000000 : labs (15:15) -- This bit is set by hardware to indicate...
0x00000001 : lbms (14:14) -- This bit is set by hardware to indicate...
0x00000001 : dllla (13:13) -- This bit indicates the status of the...
0x00000001 : scc (12:12) -- When to lb, this bit indicates that the...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that...
0x00000000 : undefined (10:10) -- Not applicable
0x00000002 : nlw (09:04) -- This field indicates the negotiated...
0x00000003 : cls (03:00) -- This field indicates the negotiated...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcctl.show()
0x00000000 : dlaece (07:07) -- This bit must be RW if the DL_Active ERR_COR...
0x00000000 : dpcst (06:06) -- This bit must be RW if the DPC Software...
0x00000001 : ptlpebe (05:05) -- This bit must be RW if the Poisoned TLP...
0x00000001 : dpcece (04:04) -- When Set, this bit enables the sending of...
0x00000001 : dpcie (03:03) -- When Set, this bit enables the generation of...
0x00000000 : dpccc (02:02) -- This bit controls the Completion Status for...
0x00000002 : dpcte (01:00) -- This field enables DPC and controls the...
    
```

2. Execute CTO error injection at the root port.

```

>>> halt()
>>> ei.utils.resetInjectorLockCheck()
>>> ei.iio.injectPcieError(socket=0, port="pxp0.rp2", errType="uce")
    
```

3. Collect error logs while system is halted.

```

>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.devsts.show()
0x00000000 : tp (05:05) -- When set, this bit indicates that the function has issued...
0x00000000 : apd (04:04) -- Auxiliary Power is not supported.
0x00000000 : urd (03:03) -- This bit indicates that this function received an unsupported...
0x00000000 : fed (02:02) -- This bit indicates that this function has detected a Fatal...
0x00000001 : nfed (01:01) -- This bit indicates that this function has detected a...
0x00000000 : ced (00:00) -- This bit indicates that this function has detected a...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsts.show
0x00000000 : ptlpeb (26:26) -- Poisoned TLP Egress Blocked Status
0x00000000 : tpbe (25:25) -- TLP Prefix Blocked Error Status
    
```

```

0x00000000 : aebe (24:24) -- This bit is set whenever an Atomic...
0x00000000 : mce (23:23) -- This bit is set whenever a Multicast...
0x00000000 : uie (22:22) -- This bit is set whenever an...
0x00000000 : acse (21:21) -- This bit is set whenever an ACS...
0x00000000 : ure (20:20) -- This bit is set whenever an...
0x00000000 : ecrce (19:19) -- ECRC checking is optional for project...
0x00000000 : mtlpe (18:18) -- This bit is set when it receives a...
0x00000000 : roe (17:17) -- This bit is set when the PCI Express...
0x00000000 : uce (16:16) -- This bit is set whenever a completion...
0x00000000 : cae (15:15) -- The bridge sets this bit and logs the...
0x00000001 : cte (14:14) -- For Switch Ports, this bit is set if...
0x00000000 : fce (13:13) -- This bit is set when a flow control...
0x00000000 : ptlpe (12:12) -- This bit is set and the bridge logs...
0x00000000 : slde (05:05) -- This bit is set when a surprise down...
0x00000000 : dlpe (04:04) -- This bit is set when a data link...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcsts.show
0x0000001f : rppiofep (12:08) -- The value of this field identifies...
0x00000000 : dpctre (06:05) -- This field serves as an extension to...
0x00000000 : dpcrpb (04:04) -- When the DPC Trigger Status bit is Set...
0x00000001 : dpcis (03:03) -- This bit is Set if DPC is triggered...
0x00000000 : dpctr (02:01) -- This field indicates why DPC has been...
0x00000001 : dpcts (00:00) -- When Set, this bit indicates that DPC...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x00000000 : labs (15:15) -- This bit is set by hardware to indicate...
0x00000001 : lbms (14:14) -- This bit is set by hardware to indicate...
0x00000000 : dllla (13:13) -- This bit indicates the status of the...
0x00000001 : scc (12:12) -- When to lb, this bit indicates that the...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that...
0x00000000 : undefined (10:10) -- Not applicable
0x00000002 : nlw (09:04) -- This field indicates the negotiated...
0x00000001 : cls (03:00) -- This field indicates the negotiated...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts
0x00

```

4. Release the platform. ERR0 will be signaled and BMC RAS Offload will be activated.

NOTE

DEVSTS is not clearing due to errata.

```

>>> go()
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.devsts.show()
0x00000000 : tp (05:05) -- When set, this bit indicates that the function has issued...
0x00000000 : apd (04:04) -- Auxiliary Power is not supported.
0x00000000 : urd (03:03) -- This bit indicates that this function received an unsupported...
0x00000000 : fed (02:02) -- This bit indicates that this function has detected a Fatal...
0x00000001 : nfed (01:01) -- This bit indicates that this function has detected a...
0x00000000 : ced (00:00) -- This bit indicates that this function has detected a...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.erruncsts
0x00
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.dpcsts.show
0x0000001f : rppiofep (12:08) -- The value of this field identifies...
0x00000000 : dpctre (06:05) -- This field serves as an extension to...
0x00000000 : dpcrpb (04:04) -- When the DPC Trigger Status bit is Set...
0x00000001 : dpcis (03:03) -- This bit is Set if DPC is triggered...
0x00000000 : dpctr (02:01) -- This field indicates why DPC has been...
0x00000000 : dpcts (00:00) -- When Set, this bit indicates that DPC...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.linksts.show
0x00000001 : labs (15:15) -- This bit is set by hardware to indicate...
0x00000001 : lbms (14:14) -- This bit is set by hardware to indicate...
0x00000001 : dllla (13:13) -- This bit indicates the status of the...
0x00000001 : scc (12:12) -- When to lb, this bit indicates that the...
0x00000000 : lt (11:11) -- For PCIe device, this bit indicates that...
0x00000000 : undefined (10:10) -- Not applicable
0x00000002 : nlw (09:04) -- This field indicates the negotiated...
0x00000003 : cls (03:00) -- This field indicates the negotiated...
>>> sv.socket0.io0.uncore.pi5.pxp0.rp2.cfg.rooterrsts
0x00

```

5. OS logs.

```
kernel: pcieport 0000:14:06.0: EDR: EDR event received
kernel: pcieport 0000:14:06.0: DPC: containment event, status:0x1f09 source:0x0000
kernel: pcieport 0000:14:06.0: DPC: unmasked uncorrectable error detected
kernel: pcieport 0000:14:06.0: PCIe Bus Error: severity=Uncorrected (Non-Fatal),
type=Transaction Layer, (Requester ID)
kernel: pcieport 0000:14:06.0: device [8086:0db2] error status/mask=00004000/00180020
kernel: pcieport 0000:14:06.0: [14] CmplttO (First)
kernel: nvme nvme0: frozen state error detected, reset controller
kernel: nvme nvme0: restart after slot reset
kernel: nvme nvme0: 8/0/0 default/read/poll queues
kernel: pcieport 0000:14:06.0: AER: device recovery successful
```

6. Verify the RAS manager logs in BMC.

```
> journalctl -u com.intel.RAS -b --no-pager
RAS_INFO::[ras_manager] ERR0 Asserted
```

13.3 Memory: ADDDC

Test Objective

The objective of this test is to test one bank VLS region on a channel. Multiple correctable ECC errors are injected until the HW threshold is met to trigger ADC/ADDDC flow and various CSRs are read to confirm that RAS Offload is behaving as expected.

Configuration

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW15'24 BKC
BMC FW	WW15'24 BMC FW
OS Version	CentOS* Stream 8
Kernel	5.19.0-gnr.po.bkc.3.13.14.x86_64
HW Configuration	1 DIMM per socket, 10x4 single rank
CScripts Version	GNR 2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMSG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [Kernel* Tools and Scripts](#) on page 20

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 1

- Trigger SW Error Threshold= Disabled
- ADDDC Sparing = Enabled
- EDKII > Platform Configuration > System Event Log >
 - OOB RAS Support = Enable

Recipe

1. Verify RAS handlers and pollers are running.

NOTE

Ensure no warnings/error messages are shown in output - RAS_ERROR , RAS_WARN.

```
> journalctl -u com.intel.RAS -b --no-pager
RAS_INFO::[ras_manager] Dumping RAS Feature Init Summary...
RAS_INFO::[ras_manager] *IEH Correctable Handling: Enabled
RAS_INFO::[ras_manager] *IMC Correctable Handling: Enabled
RAS_INFO::[ras_manager] *ECS Poller: Running
RAS_INFO::[ras_manager] *Mirror Failover Poller: Running
RAS_INFO::[ras_manager] *IMC UCNA Poller: Running
RAS_INFO::[ras_manager] *UPI Correctable Handling: Enabled
RAS_INFO::[ras_manager] *UPI Poller: Running
RAS_INFO::[ras_manager] The current Host Boot State set to the EndOfPost
RAS_INFO::[ras_manager] TPMI Discovery completed
```

2. Halt and confirm there are no pre-existing memory related errors on the system. Confirm ADDDC is enabled.

```
>>> halt()
>>> error.check mem_errors()
>>> mc.dimminfoNoSPD()
INFO: BIOS knob 'Disable BIOS Done' is enabled!
INFO: BIOS knob 'Enable Dfx Features' is enabled!
=====
|          Socket 0          |          Socket 1          |          System          |
=====
Active Memory | 256.0GB | 256.0GB | 512.0GB |
Expected Memory | NotAvail | NotAvail | NotAvail |
Clustering Mode | Hex | Hex | Hex |
1LM Memory Size | 256.00GB | 256.00GB | 512.00GB |
2LM Memory Size | 0.00 B | 0.00 B | 0.00 B |
AppDirect Memory Size | 0.00 B | 0.00 B | 0.00 B |
DDR Frequency | 5600MHz | 5600MHz | 5600MHz |
Max DDR Frequency | 6400MHz | 6400MHz | 6400MHz |
MC0 CL-RCD-RP-CMD | 46-46-45-1n | 46-46-45-1n | 46-46-45-1n |
MC1 CL-RCD-RP-CMD | 46-46-45-1n | 46-46-45-1n | 46-46-45-1n |
MC6 CL-RCD-RP-CMD | 46-46-45-1n | 46-46-45-1n | 46-46-45-1n |
MC7 CL-RCD-RP-CMD | 46-46-45-1n | 46-46-45-1n | 46-46-45-1n |
MC0 CL-RCD-RP-CMD | 46-46-45-1n | 46-46-45-1n | 46-46-45-1n |
MC1 CL-RCD-RP-CMD | 46-46-45-1n | 46-46-45-1n | 46-46-45-1n |
MC6 CL-RCD-RP-CMD | 46-46-45-1n | 46-46-45-1n | 46-46-45-1n |
MC7 CL-RCD-RP-CMD | 46-46-45-1n | 46-46-45-1n | 46-46-45-1n |
ECC Checking / PFD | On / On | On / On | On / On |
ECC Mode | 125b | 125b | 125b |
CAP/WrCRC | On / Disable | On / Disable | On / Disable |
Patrol/DemandScrub | On / On | On / On | On/On |
Memory Mode | 1LM | 1LM | 1LM |
Channel Mode | Independent | Independent | Independent |
Refresh Mode | SameBankFine | SameBankFine | SameBankFine |
RFM | Off | Off | Off |
ZQCal/DQSOSC/RxRt | On / On / Off | On / On / Off | On / On / Off |
ADDDC | Enabled | Enabled | Enabled |
Encryption | Disabled | Disabled | Disabled |
```

Paging Policy	AdaptOpen	AdaptOpen	AdaptOpen
Directory Mode	On	On	On
DDR Scrambling	Enabled	Enabled	Enabled
CKE Throttling	PPD	PPD	PPD
Thermal Throttling	CLTT	CLTT	CLTT
NUMA			On

3. Before injecting errors using the EINJ interface, translate the system address which will be the injected error through Cscript.

```
>>> mc.addTran()

*****
***** Graniterapids Address Translator *****
*****

1) Translate Address
2) Show naming convention
3) Show core address map
4) Print SAD arrays
5) Print RT arrays
6) Print TAD arrays
7) Print RIR arrays
8) Print NM SAD arrays
9) Tests
10) Decode memicals error log
11) Turn ON debug logging
q) Quit
Selection: 1
What do you have?
1) System address
2) Channel address
3) (MCR) Pseudo channel address
4) (DDR5) Sub channel address
5) Dram address
b) Back
q) Quit
Selection: 5
Enter the Socket (0-1): 0
Enter the Imc #
Non-contiguous MCs - Only mc[0, 1, 6, 7] are enabled. (0-7): 1
Enter the Physical Sub-Channel # (0-1): 0
Enter the Rank (0-1): 0
Enter the Bank Group BG[2:0] (0-7): 0
Enter the Bank Address in a Bank Group BA[1:0] (0-3): 0
Bank Address (BG*4+BA) = 0
Enter the Row (0x0-0xffff): 0x80
Enter the Column (0x0-0x7ff): 0x80
*****
*****
Core Addr_____ = 0x1000_4000 (256.02 M)
Cluster_____ = 0x2
Set Index_____ = 0x100
Tag_____ = 0x800
Cha_____ = 60
Perspective_____ = FM
*****
*****
FM ddr5 1LM:
Core Addr_____ = 0x1000_4000 (256.02 M)
Ch Addr_____ = 0x400_1000 (64.00 M)
Sub_Ch Addr_____ = 0x200_0800 (32.00 M)
pseudo Ch Addr_____ = 0x100_0400 (16.00 M)
Rank Addr_____ = 0x80_0400 (8.00 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 1
Ch_____ = 0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
```



```

Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 0
Bank in Bank Grp = 0
Bank Addr_____ = 0
Row_____ = 0x80
Column_____ = 0x80
Mirror_____ = 0
Primary_____ = -1
*****
Hit Enter key to continue

1) Translate Address
2) Show naming convention
3) Show core address map
4) Print SAD arrays
5) Print RT arrays
6) Print TAD arrays
7) Print RIR arrays
8) Print NM SAD arrays
9) Tests
10) Decode memicals error log
11) Turn ON debug logging
q) Quit
Selection: q
>>> go()

```

4. Inject memory error through the EINJ interface under the OS using the converted address.

```

[root@gnr-bkc ~]# modprobe einj
[root@gnr-bkc ~]# cd /sys/kernel/debug/apei/einj
[root@gnr-bkc einj]# echo 0x10004000 > param1
[root@gnr-bkc einj]# echo 0xfffffffffff000 > param2
[root@gnr-bkc einj]# echo 0x8 > error_type
[root@gnr-bkc einj]# echo 1 > error_inject

```

5. Wait copy complete(suggest waiting for at least 60 seconds), then verify the BMC RAS Monitor logs to ensure ADDDC trigger and Memory CE is handled by BMC. The keyword strings "RasSpareEventAdddc", "RasSpareCopyInProgress" and "RasSpareCopyDone" will be displayed.

```

RAS_INFO::[ras_manager] ERR0 Asserted
RAS_INFO::[event_listener] Sparing Event: Severity: CORRECTED, RasSpareEventAdddc,
RasSparingForward, RasSpareCopyInProgress, Skt 0, Ch 1, EventIndex 0, NumEvents 2, FailedRank
0, FailedBank 0, NonFailedRank 2, NonFailedBank 1
RAS_INFO::[event_listener] Platform Mem Error Event: Severity: CORRECTED, ErrorType 02h
(EFI_PLATFORM_MEMORY_ERROR_SINGLEBIT_ECC), Node 0, Card 1, ModuleRank 0, RankNum 0, Bank 0,
Device 1, Row 128, Column 128
RAS_INFO::[whea_listener] Platform Mem Error CPER added to WHEA buffer.
RAS_INFO::[ras_manager] ERR0 Asserted
RAS_INFO::[event_listener] Sparing Event: Severity: CORRECTED, RasSpareEventAdddc,
RasSparingForward, RasSpareCopyDone, Skt 0, Ch 1, EventIndex 0, NumEvents 2, FailedRank 0,
FailedBank 0, NonFailedRank 2, NonFailedBank 1

```

6. Check the sparing registers.

```

>>> sv.socket0.soc.memss.mcl.ch0.mcchan.sparing_control.show
0x00000000 : tad_rule (14:12)
0x00000000 : bank_sparing (10:10)
0x00000000 : region_size (08:07)
0x00000001 : adddc_sparing (03:03)
0x00000000 : spare_reverse (01:01)
0x00000000 : spare_enable (00:00)
>>> sv.socket0.soc.memss.mcl.ch0.mcchan.sparing_patrol_status.show()

```

```
0x00000000 : patrol_complete (02:02)
0x00000000 : copy_complete (01:01)
0x00000000 : copy_in_progress (00:00)
```

7. Verify that the ADDDC_REGION and ADDDC_REGION_CONTROL registers were modified by BMC RAS handle and indicate the failed and non-failed (buddy) ranks.

```
>>> ras.mem.adddc_status_check(socket=0,mc=1,ch=0)
...
=====
| Register Dump |           | Region # | Socket | iMC-Channel | CS |
| SubRank | BankGp | Bank | Region |
=====
| SVLS | failed | 0 | socket0 | mcl-ch0 | 0x0 |
| 0x0 | 0x0 | 0x0 | Bank |
=====
| SVLS | non-failed | 0 | socket0 | mcl-ch0 | 0x2 |
| 0x0 | 0x0 | 0x1 | Bank |
=====
| SVLS | --- | 0 | socket0 | mcl-ch0 | Failed |
| Device | = | 0x1 | Rank0 |
=====
| SVLS | --- | 1 | socket0 | mcl-ch0 | - |
| - | - | - | disabled |
=====
| SVLS | --- | 2 | socket0 | mcl-ch0 | - |
| - | - | - | disabled |
=====
| SVLS | --- | 3 | socket0 | mcl-ch0 | - |
| - | - | - | disabled |
=====
***** Memory Correctable ECC Error Count *****
=====
| Socket | iMC-Channel | Rank | Count | Count Overflow | Threshold | Thrshld
Overflow |
=====
| socket0 | mcl-ch0 | 0 | 0x0 | | 0x1
|
=====
| socket0 | mcl-ch0 | 1 | 0x0 | | 0x1
|
=====
| socket0 | mcl-ch0 | 2 | 0x0 | | 0x1
|
=====
| socket0 | mcl-ch0 | 3 | 0x0 | | 0x1
|
=====
| socket0 | mcl-ch0 | 4 | 0x0 | | 0x1
|
=====
| socket0 | mcl-ch0 | 5 | 0x0 | | 0x1
|
```

```

-----
| socket0 |  mcl-ch0 |  6  | 0x0 |           |  0x1
|         |         |    |    |           |
-----
| socket0 |  mcl-ch0 |  7  | 0x0 |           |  0x1
|         |         |    |    |           |
=====
=====

```

8. Verify logs at the OS.

```

{1}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 1
{1}[Hardware Error]: It has been corrected by h/w and requires no further
action
{1}[Hardware Error]: event severity: corrected
{1}[Hardware Error]: Error 0, type: corrected
{1}[Hardware Error]: fru_text: Card01, ChnB, DIMM0
{1}[Hardware Error]: section_type: memory error
{1}[Hardware Error]: error_status: Storage error in DRAM memory
(0x0000000000000400)
{1}[Hardware Error]: physical_address: 0x0000000010004000
{1}[Hardware Error]: node:0 card:1 module:0 rank:0 bank:0 device:1 row:128
column:128
{1}[Hardware Error]: error_type: 2, single-bit ECC
{1}[Hardware Error]: DIMM location: not present. DMI handle: 0x0000
EDAC skx MC1: HANDLING MCE MEMORY ERROR
EDAC skx MC1: CPU 0: Machine Check Event: 0x0 Bank 255: 0x9c0000000000009f
EDAC skx MC1: TSC 0x0
EDAC skx MC1: ADDR 0x10004000
EDAC skx MC1: MISC 0x8c
EDAC skx MC1: PROCESSOR 0:0xa06d0 TIME 1713197497 SOCKET 0 APIC 0x0
EDAC DEBUG: skx_mce_output_error: err_code:0x0000:0x009f
SystemAddress:0x10004000 ProcessorSocketId:0x0 MemoryControllerId:0x1
ChannelAddress:0x4001000 ChannelId:0x0 RankAddress:0x800400
PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x80 Column:0x80
Bank:0x0 BankGroup:0x0 ChipSelect:0x0
EDAC MC1: 0 CE memory read error on CPU_SrcID#0_MC#1_Chan#0_DIMM#0 (channel:0
slot:0 page:0x10004 offset:0x0 grain:32 syndrome:0x0-
err_code:0x0000:0x009f SystemAddress:0x10004000 ProcessorSocketId:0x0
MemoryControllerId:0x1 ChannelAddress:0x4001000 ChannelId:0x0
RankAddress:0x800400 PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0
Row:0x80 Column:0x80 Bank:0x0 BankGroup:0x0 ChipSelect:0x0)

```

13.4 Memory: Correctable Error Reporting

Test Objective

The intent of this test case is to make sure that BMC is handling the error handling for Memory CE error when RAS Offload is enabled.

Configuration

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW15'24 BKC
BMC FW	WW15'24 BMC FW
OS Version	CentOS* Stream 8
<i>continued...</i>	

Kernel	5.19.0-gnr.po.bkc.3.13.14.x86_64
HW Configuration	1 DIMM per socket, 10x4 single rank
CScripts Version	GNR 2406.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27
- [Kernel* Tools and Scripts](#) on page 20

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Correctable Error Threshold = 1
- EDKII > Platform Configuration > System Event Log >
 - OOB RAS Support = Enable

Recipe

1. Verify RAS handlers and pollers are running.

NOTE

Ensure no warnings/error messages are shown in output - RAS_ERROR , RAS_WARN.

```
> journalctl -u com.intel.RAS -b --no-pager
RAS_INFO::[ras_manager] Dumping RAS Feature Init Summary...
RAS_INFO::[ras_manager] *IEH Correctable Handling: Enabled
RAS_INFO::[ras_manager] *IMC Correctable Handling: Enabled
RAS_INFO::[ras_manager]   *ECS Poller: Running
RAS_INFO::[ras_manager]   *Mirror Failover Poller: Running
RAS_INFO::[ras_manager]   *IMC UCNA Poller: Running
RAS_INFO::[ras_manager] *UPI Correctable Handling: Enabled
RAS_INFO::[ras_manager]   *UPI Poller: Running
RAS_INFO::[ras_manager] The current Host Boot State set to the EndOfPost
RAS_INFO::[ras_manager] TPMI Discovery completed
```

2. Before injecting errors using the EINJ interface, translate the system address which will be the address the error is injected.

```
>>> mc.addTran()
*****
***** Graniterapids Address Translator *****
*****
1) Translate Address
2) Show naming convention
3) Show core address map
4) Print SAD arrays
5) Print RT arrays
6) Print TAD arrays
7) Print RIR arrays
```

```

    8) Print NM SAD arrays
    9) Tests
   10) Decode memicals error log
   11) Turn ON debug logging
    q) Quit
Selection: 1
What do you have?
    1) System address
    2) Channel address
    3) (MCR) Pseudo channel address
    4) (DDR5) Sub channel address
    5) Dram address
    b) Back
    q) Quit
Selection: 5
Enter the Socket (0-1): 0
Enter the Imc # (0-1): 0
Enter the Physical Sub-Channel # (0-1): 0
Enter the Dimm (0-1): 0
Enter the Rank (0-1): 0
Enter the Bank Group BG[2:0] (0-7): 1
Enter the Bank Address in a Bank Group BA[1:0] (0-3): 0
Bank Address (BG*4+BA) = 4
Enter the Row (0x0-0xffff): 0x380
Enter the Column (0x0-0x7ff): 0
*****
Core Addr_____ =          0x7000_0000 (1.75 G)
Cluster_____ = 0x0
Set Index_____ = 0x0
Cha_____ = 12
Perspective_____ = FM
*****
FM ddr5 1LM:
Core Addr_____ =          0x7000_0000 (1.75 G)
Ch Addr_____ =          0x3800_0000 (896.00 M)
Sub_Ch Addr_____ =          0x1c00_0000 (448.00 M)
pseudo_Ch Addr_____ =          0xe00_0000 (224.00 M)
Rank Addr_____ =          0x380_0000 (56.00 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 0
Ch_____ = 0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 1
Bank in Bank Grp = 0
Bank Addr_____ = 4
Row_____ = 0x380
Column_____ = 0x0
Mirror_____ = 0
Primary_____ = -1
*****

```

3. Inject memory error through the EINJ interface under the OS using the converted address.

```

# modprobe einj
# cd /sys/kernel/debug/apei/einj
# echo 0x70000000 > param1
# echo 0xffffffffffff000 > param2
# echo 0x8 > error_type
# echo 1 > error_inject

```

4. Verify the BMC RAS Monitor logs to ensure Memory CE is offloaded to the BMC. address should align with EINJ injection.

```
RAS_INFO::[ras_manager] ERROR Asserted
RAS_INFO::[event_listener] Platform Mem Error Event: Severity: CORRECTED, ErrorType 02h
(EFI_PLATFORM_MEMORY_ERROR_SINGLEBIT_ECC), Node 0, Card 0, ModuleRank 0, RankNum 0, Bank 0,
Device 1, Row 896, Column 0
RAS_INFO::[ras_manager] The Memory Failure Prediction support is DISABLED.
RAS_INFO::[whea_listener] Platform Mem Error CPER added to WHEA buffer.
```

5. Verify logs at the OS.

```
kernel: {4}[Hardware Error]: Hardware error from APEI Generic Hardware Error
Source: 1
kernel: {4}[Hardware Error]: It has been corrected by h/w and requires no
further action
kernel: {4}[Hardware Error]: event severity: corrected
kernel: {4}[Hardware Error]: Error 0, type: corrected
kernel: {4}[Hardware Error]: fru_text: Card01, ChnA, DIMM0
kernel: {4}[Hardware Error]: section_type: memory error
kernel: {4}[Hardware Error]: error_status: Storage error in DRAM memory
(0x00000000000000400)
kernel: {4}[Hardware Error]: physical_address: 0x0000000070000400
kernel: {4}[Hardware Error]: node:0 card:0 module:0 rank:0 bank:0 device:1
row:896 column:0
kernel: {4}[Hardware Error]: error_type: 2, single-bit ECC
kernel: {4}[Hardware Error]: DIMM location: BANK 0 CPU0 DIMM_A1
kernel: mce: [Hardware Error]: Machine check events logged
kernel: EDAC skx MC0: HANDLING MCE MEMORY ERROR
kernel: EDAC skx MC0: CPU 0: Machine Check Event: 0x0 Bank 255:
0x9c0000000000009f
kernel: EDAC skx MC0: TSC 0x0
kernel: EDAC skx MC0: ADDR 0x70000400
kernel: EDAC skx MC0: MISC 0x8c
kernel: EDAC skx MC0: PROCESSOR 0:0xa06f3 TIME 1715218635 SOCKET 0 APIC 0x0
kernel: EDAC DEBUG: skx_mce_output_error: err_code:0x0000:0x009f
SystemAddress:0x70000400 ProcessorSocketId:0x0 MemoryControllerId:0x0
ChannelAddress:0x38000200 ChannelId:0x0 RankAddress:0x3800080 PhysicalRankI>
kernel: EDAC MC0: 0 CE memory read error on CPU_SrcID#0 MC#0 Chan#0_DIMM#0
(channel:0 slot:0 page:0x70000 offset:0x400 grain:32 syndrome:0x0 -
err_code:0x0000:0x009f SystemAddress:0x70000400 ProcessorSocketId:0>
```

13.5 Memory: Soft Post Package Repair (sPPR)

Test Objective

Inject correctable errors and repair the affected rows at runtime utilizing runtime Soft Post Package Repair (sPPR). Ensure that the memory CE is offloaded to BMC and WHEA happens through MMBI.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW32'23 BKC
BMC FW	WW32'23 BMC FW
<i>continued...</i>	

OS Version	CentOS* Stream 8
Kernel	5.19.0-gnr.po.bkc.3.13.14.x86_64
HW Configuration	1 DIMM per socket, 10x4 single rank
CScripts Version	GNR2334.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - Runtime PPR/Row Sparing = Enable
 - Correctable Error Threshold = 1
- EDKII > Socket Configuration > Memory Configuration > Memory DFX Configuration >
 - Data Scrambling for DDR5 = Disable
- EDKII > Platform Configuration > System Event Log >
 - OOB RAS Support = Enable

NOTE

sPPR is experiencing issues such as sightings report issue #120. BIOS/BMC developers can continue with development by temporarily bypassing the issue by turning off data scrambling.

Recipe

1. Verify that the RAS handlers and pollers are running.

NOTE

Ensure that no warnings or error messages are shown in the output - **RAS_ERROR** , **RAS_WARN**.

```
journalctl -u com.intel.RAS -b --no-pager
RAS_INFO::[ras_manager] Dumping RAS Feature Init Summary...
RAS_INFO::[ras_manager] *IEH Correctable Handling: Enabled
RAS_INFO::[ras_manager] *IMC Correctable Handling: Enabled
RAS_INFO::[ras_manager] *ECS Poller: Running
RAS_INFO::[ras_manager] *Mirror Failover Poller: Running
RAS_INFO::[ras_manager] *IMC UCNA Poller: Running
RAS_INFO::[ras_manager] *UPI Correctable Handling: Enabled
RAS_INFO::[ras_manager] *UPI Poller: Running
RAS_INFO::[ras_manager] *SRAR Errors Handling: Disabled
RAS_INFO::[ras_manager] TPMI Discovery completed
```

2. Check sPPR registers are cleared and there are no existing failures.

```
>>> sv.sockets.soc.memss.mcs.chs.mcchan.sppr_ctrl0
socket0.soc.memss.mc0.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket0.soc.memss.mc1.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket0.soc.memss.mc2.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket0.soc.memss.mc3.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket1.soc.memss.mc0.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket1.soc.memss.mc1.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket1.soc.memss.mc2.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket1.soc.memss.mc3.ch0.mcchan.sppr_ctrl0 - 0x00000000
>>> sv.sockets.soc.memss.mcs.chs.mcchan.sppr_ctrl1
socket0.soc.memss.mc0.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket0.soc.memss.mc1.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket0.soc.memss.mc2.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket0.soc.memss.mc3.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket1.soc.memss.mc0.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket1.soc.memss.mc1.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket1.soc.memss.mc2.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket1.soc.memss.mc3.ch0.mcchan.sppr_ctrl1 - 0x00000000
```

3. Inject a memory correctable error.

```
>>>
ei.mem.injectMemError(socket=0,channel=0,dimm=0,rank=0,sub_channel=0,bank_group=0,bank=0,errType="ce",verbose=True,sub_rank=0,ps_ch=0)
```

4. Verify sPPR registers. Verify the failed information; that is, that the device, bank, and bank group are matching with injected address location. Also verify that the success_failed status is set to 0x1 using the next command:

```
>>> sv.sockets.soc.memss.mcs.chs.mcchan.sppr_ctrl0
socket0.soc.memss.mc0.ch0.mcchan.sppr_ctrl0 - 0x00010002
socket0.soc.memss.mc1.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket0.soc.memss.mc2.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket0.soc.memss.mc3.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket1.soc.memss.mc0.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket1.soc.memss.mc1.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket1.soc.memss.mc2.ch0.mcchan.sppr_ctrl0 - 0x00000000
socket1.soc.memss.mc3.ch0.mcchan.sppr_ctrl0 - 0x00000000
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.sppr_ctrl0.show()
0x00000001 : failed_device (21:16) --
0x00000000 : failed_ba (15:14) --
0x00000000 : failed_bg (13:10) --
0x00000000 : failed_c (09:05) --
0x00000000 : failed_cs (04:02) --
0x00000001 : success_failed (01:01) --
0x00000000 : start_inprogress (00:00) --
>>> sv.sockets.soc.memss.mcs.chs.mcchan.sppr_ctrl1
socket0.soc.memss.mc0.ch0.mcchan.sppr_ctrl1 - 0x00000100
socket0.soc.memss.mc1.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket0.soc.memss.mc2.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket0.soc.memss.mc3.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket1.soc.memss.mc0.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket1.soc.memss.mc1.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket1.soc.memss.mc2.ch0.mcchan.sppr_ctrl1 - 0x00000000
socket1.soc.memss.mc3.ch0.mcchan.sppr_ctrl1 - 0x00000000
>>> sv.socket0.soc.memss.mc0.ch0.mcchan.sppr_ctrl1.show()
0x00000100 : failed_row (17:00) --
```

5. Verify the BMC RAS Monitor logs to ensure that sPPR triggered and that Memory CE is handled by the BMC.

```
journalctl -u com.intel.RAS -b --no-pager
RAS_INFO::[ras_manager] ERROR Asserted
RAS_INFO::UpdatePPRSpareRegion:
```



```

NodeId[0].McCh[0].R[0].SubR[0].BankBG[0].Row[256].FailedDevice[0x1]
CopyClSize[32].Loops[4]
RAS_INFO:: RuntimeSPPRCopy Status=0
RAS_INFO::PrintPprEvent: Address-
Socket[0].MC[0].CH[0].D[0].PseudoCh[0].SubCh[0].R[0].CID[0].Bank[0].Row[256].FailDevMask = 0x2
RAS_INFO::[event_listener] sPPR Event: Severity: CORRECTED, ErrorType 02h, Node 0, Card 0,
ModuleRank 0, RankNum 0, Bank 0, Device 1, Row 256, Column 256
RAS_INFO::[ras_manager] The Memory Failure Prediction support is DISABLED.
RAS_INFO::[event_listener] Platform Mem Error Event: Severity: CORRECTED, ErrorType 02h, Node
0, Card 0, ModuleRank 0, RankNum 0, Bank 0, Device 1, Row 256, Column 256
RAS_INFO::[ras_manager] The Memory Failure Prediction support is DISABLED.
RAS_INFO::[whea_listener] Platform Mem Error CPER added to WHEA buffer.

```

6. Verify logs at the OS.

```

[ 3051.785933] mce: [Hardware Error]: Machine check events logged
[ 3051.786715] EDAC skx MC0: HANDLING MCE MEMORY ERROR
[ 3051.786720] EDAC skx MC0: CPU 0: Machine Check Event: 0x0 Bank 13: 0x8c00004200800090
[ 3051.786722] EDAC skx MC0: TSC 0x473dc741acf
[ 3051.786723] EDAC skx MC0: ADDR 0x10008200
[ 3051.786725] EDAC skx MC0: MISC 0x400080008008086
[ 3051.786727] EDAC skx MC0: PROCESSOR 0:0xa06d0 TIME 1696551497 SOCKET 0 APIC 0x0
[ 3051.786731] EDAC DEBUG: skx_mce output_error: err_code:0x0080:0x0090
SystemAddress:0x10008200 ProcessorSocketId:0x0 MemoryControllerId:0x0 ChannelAddress:0x4002000
ChannelId:0x0 RankAddress:0x1000800 PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x100
Column:0x100 Bank:0x0 BankGroup:0x0 ChipSelect:0x0
[ 3051.786742] EDAC MC0: 1 CE memory read error on CPU_SrcID#0_MC#0_Chan#0_DIMM#0 (channel:0
slot:0 page:0x10008 offset:0x200 grain:32 syndrome:0x0 - err_code:0x0080:0x0090
SystemAddress:0x10008200 ProcessorSocketId:0x0 MemoryControllerId:0x0 ChannelAddress:0x4002000
ChannelId:0x0 RankAddress:0x1000800 PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x100
Column:0x100 Bank:0x0 BankGroup:0x0 ChipSelect:0x0)
[ 3052.223781] {1}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 1
[ 3052.223789] {1}[Hardware Error]: It has been corrected by h/w and requires no further action
[ 3052.223791] {1}[Hardware Error]: event severity: corrected
[ 3052.223794] {1}[Hardware Error]: Error 0, type: corrected
[ 3052.223795] {1}[Hardware Error]: fru_text: Card01, ChnA, DIMM0
[ 3052.223797] {1}[Hardware Error]: section_type: memory error
[ 3052.223800] {1}[Hardware Error]: error_status: Storage error in DRAM memory
(0x00000000000000400)
[ 3052.223802] {1}[Hardware Error]: physical_address: 0x0000000010008200
[ 3052.223805] {1}[Hardware Error]: node:0 card:0 module:0 rank:0 bank:0 device:1 row:256
column:256
[ 3052.223806] {1}[Hardware Error]: error type: 2, single-bit ECC
[ 3052.223809] {1}[Hardware Error]: DIMM location: not present. DMI handle: 0x0000

```

13.6 Memory: Mirror Mode - Correctable Error on Primary Channel

Test Objective

The objective of this test is to ensure BMC offloads correctable ECC error flow on the primary channel for a mirrored region.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW32'23 BKC
BMC FW	WW32'23 BMC FW
OS Version	CentOS* Stream 8
Kernel	5.19.0-gnr.po.bkc.3.13.14.x86_64
HW Configuration	4 DIMMs per socket, 10x4 single rank, 32 GB DIMMs on slots A1, B1, C1, and D1 (Recommended to use 1 or 2 DPC because they are the only POR prime configurations which support mirroring)
CScripts Version	GNR 2334.6000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Socket Configuration > Memory Configuration > Memory RAS Configuration >
 - MirrorMode = Full Mirror Mode
- EDKII > Platform Configuration > System Event Log >
 - OOB RAS Support = Enable

Recipe

1. Verify that the RAS handlers and pollers are running.

NOTE

Ensure that no warnings or error messages are shown in the output - RAS_ERROR , RAS_WARN.

```
journalctl -u com.intel.RAS -b --no-pager
RAS_INFO::[ras_manager] Dumping RAS Feature Init Summary...
RAS_INFO::[ras_manager] *IEH Correctable Handling: Enabled
RAS_INFO::[ras_manager] *IMC Correctable Handling: Enabled
RAS_INFO::[ras_manager] *ECS Poller: Running
RAS_INFO::[ras_manager] *Mirror Failover Poller: Running
RAS_INFO::[ras_manager] *IMC UCNA Poller: Running
RAS_INFO::[ras_manager] *UPI Correctable Handling: Enabled
RAS_INFO::[ras_manager] *UPI Poller: Running
RAS_INFO::[ras_manager] *SRAR Errors Handling: Disabled
RAS_INFO::[ras_manager] TPMI Discovery completed
```

2. Check sPPR registers are cleared and there are no existing failures. Verify that mirroring is enabled by reading the following bit-field.

```
>>> halt()
>>> error.check_mem_errors()
>>> sv.socket0.soc.memss.b2cmi0.mode.mirror
0x1
```

- Check that the memory configuration aligns with what is expected. Full memory mirroring, therefore, active memory capacity is equal to half the installed capacity; all channels are operating in mirror mode and the mirror size is equal to the active memory size. With the current CScripts version there were issues seen with the reported mirror size values but everything else aligns with expectations.

```
>>> mc.dimminfoNoSPD()
INFO: BIOS knob 'Disable BIOS Done' is enabled!
INFO: BIOS knob 'Enable Dfx Features' is enabled!
=====
| Socket 0 | Socket 1 | System |
=====
Active Memory | 64.0GB | 64.0GB | 128.0GB |
Expected Memory | NotAvail | NotAvail | NotAvail |
Clustering Mode | Hex | Hex | Hex |
1LM Memory Size | 64.00GB | 64.00GB | 128.00GB |
2LM Memory Size | 0.00 B | 0.00 B | 0.00 B |
AppDirect Memory Size | 0.00 B | 0.00 B | 0.00 B |
MC2 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
MC1 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
MC2 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
MC3 CL-RCD-RP-CMD | 40-40-41-1n | 40-40-41-1n | |
ECC Checking / PFD | On / On | On / On | On / On |
ECC Mode | 128b | 128b | 128b |
CAP/WrCRC | On / Disable | On / Disable | On / Disable |
Patrol/DemandScrub | On / On | On / On | On/On |
Memory Mode | 1LM | 1LM | 1LM |
Channel Mode | Mirror | Mirror | Mirror |
Refresh Mode | AllBankNormal | AllBankNormal | AllBankNormal |
RFM | Off | Off | Off |
ZQCal/DQSOSC/RxRt | On / On / Off | On / On / Off | On / On / Off |
Mirror Size | 64.0GB | 64.0GB | 0GB |
ADDDC | Disabled | Disabled | Disabled |
Encryption | Disabled | Disabled | Disabled |
Paging Policy | AdaptOpen | AdaptOpen | AdaptOpen |
Directory Mode | Off | Off | Off |
DDR Scrambling | Enabled | Enabled | Enabled |
CKE Throttling | PPD | PPD | PPD |
Thermal Throttling | CLTT | CLTT | CLTT |
NUMA | | | On |
=====
```

- Use the following command to translate how a few addresses, within the specified system address range, map to its physical DIMM address. Take note of the memory controller ("mc") column. From this table, system addresses can be selected to test injecting errors into mirrored regions on different memory controllers.

```
>>> ras.mem.sa2da_table(0x100000000, 0x100000400, 64)
=====
| System Address | Socket | mc | Channel | Sub channel | DIMM | Rank | Sub rank | Bank group | Bank | Row | Column | Mem type | Rank Address | Sub-Channel address |
=====
| 0x100000000 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x100000040 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
=====
| 0x1000000c0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000100 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x100000140 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000180 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x1000001c0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000200 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x100000240 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000280 | 0 | 2 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x1000002c0 | 0 | 2 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000300 | 0 | 3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x100000340 | 0 | 3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
| 0x100000380 | 0 | 3 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 0 | ddr5 | 268435456 | 536870912 |
| 0x1000003c0 | 0 | 3 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 8192 | 16 | ddr5 | 268435520 | 536870976 |
=====
```

5. Check if the injectors are locked for all sockets and unlock if locked. Choose the target device for error injection and modify masks for a single device correctable ECC error injection. Inject and consume the correctable ECC error.

```
>>> ei.utils.checkInjectorLock(and unlock=True)
>>> ei.mem.memDevs(dev0=1, dev0msk=0x3, dev1=2, dev1msk=0x0)
dev0 = 0x1, dev0msk = 0x3, dev1 = 2, dev1msk = 0x0.
>>> ei.mem.injectMemError(addr=0x000100000000,errType="ce",verbose=True)
```

6. Verify that the target's IMC machine check bank has a correctable read error logged. Ensure that MSCOD == 0x80.

```
>>> error.check_mem_errors(socket_id=0)
Uncore MCA from Socket: 0
=====
| Valid | MC Bank Name | Status | Addr | Misc | Ctl1 | Ctl2 |
=====
| * | BANK #13 - MC CH0 merged | 0x8c00004200800090 | 0x000000001800000000 | 0x0400080100000086 | 0x00003cc0ffffe5ff | 0x0000000040000001 |
=====

Machine Check Decoder Summary
=====
Uncore MCA from Socket: 0
=====
| MC Bank Name | Status | Addr | Misc | Ctl1 | Ctl2 |
=====
| BANK #13 - MC CH0 merged | 0x8c00004200800090 | 0x000000001800000000 | 0x0400080100000086 | 0x00003cc0ffffe5ff | 0x0000000040000001 |
=====
.....

>>> error.manualMcaDecode()

Manual Machine Check Bank Decoder
Enter MCI_STATUS, MCI_ADDR, MCI_MISC values
Enter 't' for a short demo
Enter 'Q' or 'q' to exit

Enter MCA Bank Number [0,1,2,3,4,5,6,MSE,MCHN,B2CMI,CBO,LLC,UPI] or q for quit: MCHN
Enter MCI_STATUS [0x0]: 0x8c00004200800090
Enter MCI_ADDR [0x0]:
Enter MCI_MISC [0x0]:

Bank name: MCHN, Error = CORR Transient or correctable Error for Demand or Underfill Reads

+ MCMCHN_STATUS 0x8c00004200800090

Range Field Description Value Value Description
-----
63:63 VALID Indicates that the information within this register is valid 0x1 Valid
62:62 OVERFLOW A '1' indicates that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (the VALID bit was already set) 0x0 Not Overflow
61:61 UC A '1' indicates that an uncorrectable the error condition was detected. 0x0 No UC
60:60 EN Error reporting enabled. A '1' indicates that error reporting is enabled by the associated flag bit(s) in the IA32_MCI_CTL register. 0x0 Not Enabled
59:59 MISCV If this flag is '1', the IA32_MCI_MISC register content is valid and can be read. 0x1 IA32_MCI_MISC Valid
58:58 ADDRv If this flag is '1', the IA32_MCI_ADDR register content is valid and can be read. 0x1 IA32_MCI_ADDR Valid
57:57 PCC Processor Context Corrupt: A '1' indicates that reliable restarting of the processor may not be possible and a reset is recommended. 0x0 Processor Context Not Corrupted
56:56 S Signaling an uncorrectable recoverable error. 0x0 ---
55:55 AR Recovery action required for UCR error. 0x0 ---
54:53 THRESHOLD_BASED_STATUS Threshold-based error status 0x0 ---
52:38 CORRECTED_ERROR_COUNT Count of correctable errors in the DCU including sticky overflow indicator (bit 52) 0x1 ---
37:37 FW FW Update Error Status Indicator 0x0 ---
36:32 OTHER_INFO Not part of Machine-Check Architecture 0x2 ---
31:16 MSCOD Model Specific Error Code 0x80 Transient or correctable Error for Demand or Underfill Reads
15:0 MCACOD Machine Check Architecture Error Code 0x90 MCCHAN_ERROR(1LM MODE)|MEMORY READ ERROR|CHANNEL_0
```

7. Verify the BMC RAS Monitor logs to ensure sPPR triggered and Memory CE is handled by BMC.

```
journalctl -u com.intel.RAS -b --no-pager
ras-manager[209]: RAS_INFO::[event_listener] Platform Mem Error Event: Severity: CORRECTED,
ErrorType 02h, Node 0, Card 0, ModuleRank 0, RankNum 0, Bank 0, Device 1, Row 8192, Column 0
ras-manager[209]: RAS_INFO::[ras_manager] The Memory Failure Prediction support is DISABLED.
ras-manager[209]: RAS_INFO::[whea_listener] Platform Mem Error CPER added to WHEA buffer.
```

8. Verify logs at the OS.

```
[ 3626.744877] mce: [Hardware Error]: Machine check events logged
[ 3626.745736] EDAC skx MC0: HANDLING MCE MEMORY ERROR
[ 3626.745740] EDAC skx MC0: CPU 0: Machine Check Event: 0x0 Bank 13: 0x8c00004200800090
[ 3626.745744] EDAC skx MC0: TSC 0x557e80638e3
[ 3626.745745] EDAC skx MC0: ADDR 0x1800000000
[ 3626.745746] EDAC skx MC0: MISC 0x400080100000086
[ 3626.745747] EDAC skx MC0: PROCESSOR 0:0xa06d0 TIME 1696567683 SOCKET 0 APIC 0x0
[ 3626.745752] EDAC DEBUG: skx_mce_output_error: err_code:0x0080:0x0090 SystemAddress:0x1800000000
ProcessorSocketId:0x0 MemoryControllerId:0x0 ChannelAddress:0x80000000 ChannelId:0x0
RankAddress:0x20000000 PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x1000 Column:0x0 Bank:0x0
BankGroup:0x4 ChipSelect:0x0
[ 3626.745765] EDAC MC0: 1 CE memory read error on CPU_SrcID#0_MC#0_Chan#0_DIMM#0 (channel:0 slot:0
page:0x180000 offset:0x0 grain:32 syndrome:0x0 - err_code:0x0080:0x0090 SystemAddress:0x1800000000
ProcessorSocketId:0x0 MemoryControllerId:0x0 ChannelAddress:0x80000000 ChannelId:0x0
RankAddress:0x20000000 PhysicalRankId:0x0 DimmSlotId:0x0 DimmRankId:0x0 Row:0x1000 Column:0x0 Bank:0x0
BankGroup:0x4 ChipSelect:0x0)
[ 3626.853556] {1}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 1
[ 3626.853561] {1}[Hardware Error]: It has been corrected by h/w and requires no further action
[ 3626.853565] {1}[Hardware Error]: event severity: corrected
[ 3626.853568] {1}[Hardware Error]: Error 0, type: corrected
[ 3626.853569] {1}[Hardware Error]: fru_text: Card01, ChnA, DIMM0
[ 3626.853571] {1}[Hardware Error]: section_type: memory error
[ 3626.853573] {1}[Hardware Error]: error_status: Storage error in DRAM memory (0x0000000000000400)
[ 3626.853575] {1}[Hardware Error]: physical_address: 0x00000000180000000
[ 3626.853578] {1}[Hardware Error]: node:0 card:0 module:0 rank:0 bank:0 device:1 row:8192 column:0
[ 3626.853580] {1}[Hardware Error]: error_type: 2, single-bit ECC
[ 3626.853583] {1}[Hardware Error]: DIMM location: not present. DMI handle: 0x0000

[root@gnr-bkc tracing]# cat trace_pipe
kworker/0:0-6 [000] ..... 3627.950894: mce_record: CPU: 0, MCGc/s: f000c20/0, MC13:
8c00004200800090, IPID: 0000000000000000, ADDR/MISC/SYND:
00000000 0180000000/0400080100000086/0000000000000000,
RIP: 00:<0000000000000000>, TSC: 557e80638e3, PROCESSOR: 0:a06d0, TIME: 1696567683, SOCKET: 0, APIC: 0
kworker/0:0-6 [000] ..... 3628.059669: mce_record: CPU: 0, MCGc/s: f000c20/0, MC255:
9c0000000000009f, IPID: 0000000000000000, ADDR/MISC/SYND:
00000000 00180000000/000000000000008c/0000000000000000,
RIP: 00:<0000000000000000>, TSC:0, PROCESSOR: 0:a06d0, TIME: 1696567683, SOCKET: 0, APIC: 0
```

13.7 Intel UPI CE (Polling)

Test Objective

Verify that Intel UPI correctable errors are offloaded to BMC. SMI duties are offloaded to BMC and OS is notified via SCI. The triggering happens through polling of Intel UPI shadow registers at a default 60 seconds interval.

Configuration

The following table details the system configuration used to develop this recipe. It may or may not reflect a BKC or POR configuration. See the corresponding Platform Design Guide, the latest BKC release, and the *Birch Stream Platform: Intel® Xeon® 6900/6700/6500-Series Processors with P-Cores, Intel® Xeon® 6900/6700-Series Processors with E-Cores, and Clearwater Forest Error Architecture and RAS*, document number 728644 for POR configurations and RAS feature requirements/compatibility.

Stepping	Intel® Xeon® 6900-Series Processors with P-Cores A step
BIOS	WW15 BKC
BMC FW	WW15'23 BMC FW
OS Version	CentOS* Stream 8
<i>continued...</i>	

Kernel	5.19.0-gnr.po.bkc.3.13.14.x86_64
HW Configuration	Generic PCIe Card
CScripts Version	GNR2314.4000

Preconditions and Setup

Validation tools used in this recipe. For more information about each tool, see:

- [CScripts](#) on page 18
- [FTrace and DMESG for Machine Check Exception \(MCE\) Observability](#) on page 27

Apply the [BIOS Standard System Configuration](#) preconditions in addition to the recipe specific BIOS configuration.

- EDKII > Platform Configuration > System Event Log >
— OOB RAS Support = Enable

Recipe

1. Confirm no Intel UPI errors are logged.

```
>>> halt()
>>> upi.topology

2 sockets populated in the platform
*****
* Socket 0 (A1) UPI Config
*
*****
```

	Port 0	Port 1	Port 2	Port 3	Port 4	Port 5
Connected to	socket 1, port 1	socket 1, port 0	u/c	u/c	socket 1, port 5	socket 1, port 4
UPI LS Init Status	0x3 - Init Done	0x3 - Init Done			0x3 - Init Done	0x3 - Init Done
Current CRC Mode	0x0 - CRC16	0x0 - CRC16			0x0 - CRC16	0x0 - CRC16
Lane Reversal	No	No			No	No
Retry State	0x0 - Retry Local Normal	0x0 - Retry Local Normal			0x0 - Retry Local Normal	0x0 - Retry Local Normal
Tx state	0x0d - L0P	0x0d - L0P			0x0d - L0P	0x0d - L0P
UPI Tx Lane Status*	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000			0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000
Rx state	0x0d - L0P	0x0d - L0P			0x0d - L0P	0x0d - L0P
UPI Rx Lane Status**	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000			0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000
L0P	En (T:0xb342df6d) (R:0xb4e7acac) En (T:0xe0c2b6cc) (R:0xe27e30c1)				En (T:0x1515842f) (R:0x1736b84d) En (T:0x42a05580) (R:0x44ff4f14)	
L1	Disabled	Disabled			Disabled	Disabled
PFS	0x00000000	0x00000000			0x00000000	0x00000000
Link Speed	20.0 GT/s	20.0 GT/s	I/A	I/A	20.0 GT/s	20.0 GT/s

```
*****
* Socket 1 (A1) UPI Config
*
*****
```

	Port 0	Port 1	Port 2	Port 3	Port 4	Port 5
Connected to	socket 0, port 1	socket 0, port 0	u/c	u/c	socket 0, port 5	socket 0, port 4
UPI LS Init Status	0x3 - Init Done	0x3 - Init Done			0x3 - Init Done	0x3 - Init Done
Current CRC Mode	0x0 - CRC16	0x0 - CRC16			0x0 - CRC16	0x0 - CRC16
Lane Reversal	No	No			No	No
Retry State	0x0 - Retry Local Normal	0x0 - Retry Local Normal			0x0 - Retry Local Normal	0x0 - Retry Local Normal
Tx state	0x0d - L0P	0x0d - L0P			0x0d - L0P	0x0d - L0P
UPI Tx Lane Status*	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000			0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000
Rx state	0x0d - L0P	0x0d - L0P			0x0d - L0P	0x0d - L0P
UPI Rx Lane Status**	0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000			0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000
L0P	En (T:0x75c3ee2) (R:0x77e00ce9) En (T:0x9a529fa3) (R:0x9b889113)				En (T:0xc6fbadfb) (R:0xc8bb7541) En (T:0xed712b5a) (R:0xef4428cd)	
L1	Disabled	Disabled			Disabled	Disabled
PFS	0x00000000	0x00000000			0x00000000	0x00000000

	Link Speed	20.0 GT/s		20.0 GT/s		I/A	I/A		20.0 GT/s		20.0 GT/s	
Notes:												
* txlandetectstat - Tx Data Lane Detection Status. A bit value of 1 indicates the TX lane is dropped.												
** rxdataareadystat - Rx Data Lane Ready Status. A bit value of 1 indicates Rx lane is dropped.												
>>> upi.printErrors												
Errors	S0_P0	S0_P1	S0_P2	S0_P3	S0_P4	S0_P5	S1_P0	S1_P1	S1_P2	S1_P3	S1_P4	S1_P5
ktiph_rx10c_err_s...	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0
ktiph_rx10c_err_l...	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0
ktiph_rx10c_err_l...	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0
ktiph_l0pexit_err...	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0
ktierRdi0 (Generic)	0x0	0x0	0x3fff*	0x3fff*	0x0	0x0	0x0	0x0	0x3fff*	0x3fff*	0x0	0x0
ktierRdi1 (Generic)	0x0	0x0	0x1fff*	0x1fff*	0x0	0x0	0x0	0x0	0x1fff*	0x1fff*	0x0	0x0
ktireut_ph_pis (G...	0x0	0x0	0x41fe0016*	0x0	0x0	0x0	0x0	0x0	0x41fe0016*	0x0	0x0	0x0
ktireut_ph_pis (G...	0xccc00000*	0xccc00000*	0x0	0x0	0xccc00000*	0xccc00000*	0xccc02000*	0xccc02000*	0x0	0x0	0xccc02000*	0xccc02000*
ktireut_ph_pss (G...	0x0	0x0	0x0	0x70908*	0x0	0x0	0x0	0x0	0x606*	0x0	0x0	0x0
ktiererrcnt_cntr (...)	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0
ktiererrcnt1_cntr (...)	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0
ktiererrcnt2_cntr (...)	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0
ktiererrcnt0_mask (...)	0x7fffff*	0x7fffff*	0x0	0x0	0x7fffff*	0x7fffff*	0x7fffff*	0x7fffff*	0x0	0x0	0x7fffff*	0x7fffff*
ktiererrcnt1_mask (...)	0x7fffff*	0x7fffff*	0x0	0x0	0x7fffff*	0x7fffff*	0x7fffff*	0x7fffff*	0x0	0x0	0x7fffff*	0x7fffff*
ktiererrcnt2_mask (...)	0x7fffff*	0x7fffff*	0x0	0x0	0x7fffff*	0x7fffff*	0x7fffff*	0x7fffff*	0x0	0x0	0x7fffff*	0x7fffff*
ktiererrcntcnt (Gen...	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0
ktiles (Generic)	0x1*	0x1*	0x1*	0x0	0x1*	0x1*	0x1*	0x1*	0x0	0x0	0x1*	0x1*
ktiviral (Generic)	0x10	0x10	0x10	0x10	0x10	0x10	0x10	0x10	0x10	0x10	0x10	0x10
bios_kti_err_st (...)	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0

2. Confirm the Intel UPI topology and execute error injection.

```
>>> halt()
>>> ei.utils.resetInjectorLockCheck()
>>> ei.iio.injectUPIError(socket=0, port=1, num_crcs=1)
Halting to do check on injector lock
INFO: BIOS knob 'Disable BIOS Done' is enabled!

Success to clear errinj status bit!
After injection, check DFX injection control register
Register 'socket0.io0.uncore.upi.upil.upi_regs.ktirxeinjctl0' = 0xabca0020
[CPC core group] Halt on 48 devices -- 00:32:38.704692 2023-05-03
Error is injected!
Clear DFX injection control register
Register 'socket0.io0.uncore.upi.upil.upi_regs.ktirxeinjctl0' = 0xcaca0020
Success to stop error injection!
340223936 new CRC error(s) detected in UPI CRC Error Counter
Checking UPI Errors...

Checking errors in the following IPs:
  upi_1
  machine_check

Please be patient...this operation might take several minutes.

Uncore MCA from Socket: 0
=====
| Valid | MC Bank Name | Status | Addr | Misc | Ctl1 | Ctl2 |
=====
| * | BANK #5 - UPI Merged | 0x8800004400300e0f | 0x0000000000000000 | 0x400000e0000ffbe1 | 0x0000000081ff0007 | 0x0000000040000001 |
=====

Uncore MCA from Socket: 1
Socket 1 : No errors logged in any Uncore-specific Machine Check Banks

Decoding Machine Checks...

Machine Check Decoder Summary

=====
Uncore MCA from Socket: 0
=====
| MC Bank Name | Status | Addr | Misc | Ctl1 | Ctl2 |
=====
| BANK #5 - UPI Merged | 0x8800004400300e0f | 0x0000000000000000 | 0x400000e0000ffbe1 | 0x0000000081ff0007 | 0x0000000040000001 |
=====

Bank name: UPI, Error = COR LL Rx detected CRC error: successful LLR without Phy Reinit
+ MC5_STATUS 0x8800004400300E0F

=====
Range Field Description Value Value Description
=====
63:63 VALID Indicates that the information within this register is valid 0x1 Valid
62:62 OVERFLOW A '1' indicates that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (the VALID bit was already set) 0x0 Not Overflow
61:61 UC A '1' indicates that an uncorrectable the error condition was detected. 0x0 No UC
60:60 EN Error reporting enabled. A '1' indicates that error reporting is enabled by the associated flag bit(s) in the IA32_MCI_CTL register. 0x0 Not Enabled
59:59 MISCV If this flag is '1', the IA32_MCI_MISC register content is valid and can be read. 0x1 IA32_Mci_MISC Valid
58:58 ADDRv If this flag is '1', the IA32_Mci_ADDR register content is valid and can be read. 0x0 IA32_Mci_ADDR Not Valid
57:57 PCC Processor Context Corrupt: A '1' indicates that reliable restarting of the processor may not be possible and a reset is recommended. 0x0 Processor Context Not Corrupted
56:56 S Signaling an uncorrectable recoverable error. 0x0 ---
55:55 AR Recovery action required for UCR error. 0x0 ---
54:53 THRESHOLD_BASED_STATUS Threshold-based error status 0x0 ---
52:38 CORRECTED_ERROR_COUNT Count of filtered correctable MCA errors including sticky overflow indicator (bit 52) 0x1 ---
37:37 FW Firmware update bit 0x0 ---
=====
```

36:32	OTHER_INFO	Other Info	0x4	---
31:16	MSCOD	Model Specific Error Code	0x30	COR LL Rx detected CRC error: successful LLR without Phy
15:0	MCACOD	Machine Check Architecture Error Code	0xe0f	---
+ MC5_MISC 0x400000E0000FFBE1				
Range	Field	Description	Value	Value Description
63:56	COR_ERR	Captures Correctable Errors, indicating all correctable error cases that have occurred since last cleared.	0x40	No implemented
55:54	RESERVED_54_55	RESERVED	0x0	---
53:41	NERR	Captures the Next Error, which is any later error which occurred after the primary error was logged (in mscod_codes)	0x0	UC Phy Initialization Failure (NumInit)
40:40	RX_LANE_REVERSAL	Indicates if physical layer Rx is reversed	0x0	---
39:37	PHY_STATE	Link State from Physical Layer on error used for isolating lane in CRC error cases	0x7	Lower 8 logical lane active in L0p
36:36	ILLEGAL_FLIT_FIELD	For Unexpected Flit Errors this bit is set if unsupported error was caused by bits not captured in Flit_HDR fields	0x0	---
35:20	SYNDROME_OR_L0CBADLANES	Captures Syndrome for CRC check, or in L0c error captures 8-bit vector of bad lanes	0x0	---
19:0	FLIT_HDR	HDR bits captured	0xffbe1	HIB Slot 0 Opcode Slot 0 MC Slot 1 Opcode Slot 1 MC Slot 2 Opcode
=====				
UPI_1: REGISTERS AND FIELDS DESCRIPTIONS				
=====				
Register: socket0.io0.uncore.upi.upi1.upi_regs.ktierrcnt				
counter = 0x1				
Counter increments every time the Link Layer detects a CRC error.				
=====				
Fields with non default value				
=====				
IP block # fields with non-default value				
MACHINE_CHECK				
UPI_1 0x1				
=====				
=====				

3. Confirm that an error is logged and the corresponding error information is captured.

```
>>> upi.printErrors
Errors
|S0_P0| |S0_P1| |S0_P2| |S0_P3| |S0_P4| |S0_P5| |S1_P0| |S1_P1| |S1_P2| |S1_P3| |S1_P4| |S1_P5| |
|-----|
ktiph_rx10c_err_s... | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | |
ktiph_rx10c_err_1... | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | |
ktiph_rx10c_err_1... | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | |
ktiph_l0pexit_err... | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | |
ktierdis0 (Generic) | 0x0 | | 0x0 | | 0x1fff* | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x3fff* | 0x3fff* | 0x0 | 0x0 | #
ktierdis1 (Generic) | 0x0 | | 0x0 | | 0x1fff* | | 0x1fff* | | 0x0 | | 0x0 | | 0x0 | | 0x1fff* | 0x1fff* | 0x0 | 0x0 | #
ktireut_ph_pis (G...) | 0x0 | | 0x0 | | 0x0 | | 0x41fe0016* | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x41fe0016* | 0x0 | 0x0 | 0x0 | #
ktireut_ph_pis (G...) | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | #
ktireut_ph_pss (G...) | 0x0 | | 0x0 | | 0x0 | | 0x70908* | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x606* | 0x0 | 0x0 | 0x0 | 0x0 | #
ktiercnt0_mask (...) | 0x0 | | 0x1* | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | #
ktiercnt1_mask (...) | 0x0 | | 0x1* | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | #
ktiercnt2_mask (...) | 0x0 | | 0x1* | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | #
ktiercnt0_mask (...) | 0x7fffff* | | 0x7fffff* | | 0x0 | | 0x0 | | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x0 | 0x0 | 0x7fffff* | 0x7fffff* | #
ktiercnt1_mask (...) | 0x7fffff* | | 0x7fffff* | | 0x0 | | 0x0 | | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x0 | 0x0 | 0x7fffff* | 0x7fffff* | #
ktiercnt2_mask (...) | 0x7fffff* | | 0x7fffff* | | 0x0 | | 0x0 | | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x7fffff* | 0x0 | 0x0 | 0x7fffff* | 0x7fffff* | #
kticrcerrCnt (Gen...) | 0x0 | | 0x1* | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0xb* | | 0x0 | | 0x0 | | 0x0 | | 0x10* | #
ktiles (Generic) | 0x1* | | 0x1* | | 0x1* | | 0x0 | | 0x1* | | 0x1* | | 0x1* | | 0x1* | | 0x1* | | 0x1* | | 0x1* | #
ktiviral (Generic) | 0x10 | | 0x10 | | 0x10 | | 0x10 | | 0x10 | | 0x10 | | 0x10 | | 0x10 | | 0x10 | | 0x10 | | 0x10 | #
bios_kti_err_st (...) | 0x0 | | 0x8800004000300e0f* | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | | 0x0 | #

>>> sv.socket0.io0.uncore.upi.upi1.upi_regs.ktierrcnt.show()
0x00000000 : overflow (31:31) -- Overflow bit - set when the number...
0x00000001 : counter (30:00) -- Counter increments every time the...
>>> sv.socket0.io0.uncore.upi.upi1.upi_regs.kti_mc_st.show()
0x00000001 : valid (63:63) -- Set by hardware when an uncorrectable...
0x00000000 : over (62:62) -- MCA Overflow Flag Set on uncorrected...
0x00000000 : uc (61:61) -- Error Uncorrected Set for error which...
0x00000000 : en (60:60) -- Error Enable Flag Set by hardware when...
0x00000001 : genadv (59:59) -- RTI MC_MISC valid flag Indicates...
0x00000000 : addrv (58:58) -- Address Valid in RTI MC AD. Bit set...
0x00000000 : pcc (57:57) -- Processor Context Corrupt Flag Bit set on all uncorrectable errors
0x00000000 : s (56:56) -- Field never set by hardware
0x00000000 : ar (55:55) -- Field never set by hardware
0x00000000 : cor_err_sta_ind (54:53) -- 00 - No tracking - No...
0x00000001 : cor_err_cnt (52:38) -- Corrected Error Count 15-bit...
0x00000000 : fw_upd (37:37) -- Firmware update bit. will be set by...
0x00000000 : other_info (36:32) -- Other Info Field used for design...
0x00000000 : mscod_spare (31:22) -- MSCOD SPARE Field used for design...
0x00000030 : mscod_code (21:16) -- MSCOD Error Code Error Code from...
0x00000000 : mcacod_rsvd (15:12) -- Field never set by hardware
0x00000001 : mcacod_int (11:11) -- MCACOD - Bus Interconnect Error...
0x00000003 : mcacod_pp (10:09) -- MCACOD - Bus Int Error PP Set...
0x00000000 : mcacod_t (08:08) -- Field never set by hardware
0x00000000 : mcacod_rrrr (07:04) -- Set to 4 for SGX/PTX/ELE errors, else 0...
0x00000003 : mcacod_ii (03:02) -- SCOD Bus Interconnect Error II Set to 1...
0x00000003 : mcacod_ll (01:00) -- MSCOD Bus Interconnect Error LL Set...
>>> sv.socket0.io0.uncore.upi.upi1.upi_regs.kti_mc_misc.show()
0x00000040 : cor_err (63:56) -- Captures Correctable Errors, indicating...
0x00000000 : rsvd (55:54) -- rsvd.
0x00000000 : nerr (53:41) -- Captures the Next Error, which is any...
0x00000000 : rx_lane_reversal (40:40) -- Indicates if physical layer...
0x00000007 : phy_state (39:37) -- Link State from Physical Layer on...
0x00000000 : illegal_flit_field (36:36) -- For Unexpected Flit Errors...
0x00000000 : syndrome_or_l0cbadlanes (35:20) -- Captures Syndrome for...
0x00ffbe1 : flit_hdr (19:00) -- HDR bits captured: [0] - HIB...
```


4. OS logs:

```
{13}[Hardware Error]: processor_id: 0x0000000000000000
{14}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 1
{14}[Hardware Error]: It has been corrected by h/w and requires no further action
{14}[Hardware Error]: event severity: corrected
{14}[Hardware Error]: Error 0, type: corrected
{14}[Hardware Error]: section_type: general processor error
{14}[Hardware Error]: processor_type: 0, IA32/X64
{14}[Hardware Error]: processor_isa: 2, X64
{14}[Hardware Error]: error_type: 0x00
{14}[Hardware Error]: operation: 0, unknown or generic
{14}[Hardware Error]: version_info: 0x000000000000a06d0
{14}[Hardware Error]: processor_id: 0x0000000000000000
{15}[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 1
{15}[Hardware Error]: It has been corrected by h/w and requires no further action
{15}[Hardware Error]: event severity: corrected
{15}[Hardware Error]: Error 0, type: corrected
{15}[Hardware Error]: section_type: general processor error
{15}[Hardware Error]: processor_type: 0, IA32/X64
{15}[Hardware Error]: processor_isa: 2, X64
{15}[Hardware Error]: error_type: 0x00
{15}[Hardware Error]: operation: 0, unknown or generic
{15}[Hardware Error]: version_info: 0x000000000000a06d0
{15}[Hardware Error]: processor_id: 0x0000000000000000
```

5. Ensure Intel UPI CE is offloaded to BMC.

```
journalctl -u com.intel.RAS -b --no-pager
ras-manager[203]: RAS_INFO::[event_listener] Generic Processor Error Event: Severity:
CORRECTED, Type 0, Isa 2, ErrorType 00h, Operation 0, Version a06d0h, Socket 0
ras-manager[203]: RAS_INFO::[whea_listener] Generic Processor Error CPER added to WHEA buffer.
ras-manager[203]: RAS_INFO::[event_listener] UPI Event: Severity: CORRECTED, Skt 0, LaneIndex
1, Mscod 0xe0f
```